



Routledge Studies in Economic Theory, Method and Philosophy

DEEP LEARNING MODELS FOR ECONOMIC RESEARCH

Andrzej Dudek



Deep Learning Models for Economic Research

In today's data-driven world, the ability to make sense of complex, high-dimensional datasets is crucial for economists and data scientists. Traditional quantitative methods, while powerful, often struggle to keep up with the complexities of modern economic challenges. This book bridges this gap, integrating cutting-edge machine learning techniques with established economic analysis to provide new, more accurate insights.

The book offers a comprehensive approach to understanding and applying neural networks and deep learning models in the context of conducting economic research. It starts by laying the groundwork with essential quantitative methods such as cluster analysis, regression, and factor analysis, then demonstrates how these can be enhanced with deep learning techniques like recurrent neural networks (RNNs), convolutional neural networks (CNNs), and transformers. By guiding readers through real-world examples, complete with Python code and access to datasets, it showcases the practical benefits of neural networks in solving complex economic problems, such as fraud detection, sentiment analysis, stock price forecasting, and inflation factor analysis. Importantly, the book also addresses critical concerns about the "black box" nature of deep learning, offering interpretability techniques like Local Interpretable Model-agnostic Explanations (LIME) and SHapley Additive exPlanations (SHAP) to demystify model predictions.

The book is essential reading for economists, data scientists, and professionals looking to deepen their understanding of AI's role in economic modeling. It is also an accessible resource for non-experts interested in how machine learning is transforming economic analysis.

Andrzej Dudek is a Professor in the Department of Computer Science and Econometrics, Wrocław University of Economics and Business, Wrocław, Poland.

Routledge Studies in Economic Theory, Method and Philosophy

False Feedback in Economics

The Case for Replication

Andrin Spescha

The Economics and Science of Measurement

A Study of Metrology

Albert N. Link

Teaching and Research Methods for Islamic Economics and Finance

Edited by Moh'd Ma'Sum Billah

The Art and Science of Econometrics

Ping Zong

Financial Market Analysis and Behaviour

The Adaptive Preference Hypothesis

Emil Dinga, Camelia Oprean-Stan, Cristina-Roxana Tănăsescu, Vasile Brătian, and Gabriela-Mariana Ionescu

Complexity Economics

Economic Governance, Science and Policy

Olivér Kovács

The Solow Model of Economic Growth

Application to Contemporary Macroeconomic Issues

Paweł Dykas, Tomasz Tokarski and Rafał Wisła

Philosophical Foundations of Economic Reality

Anthropological, Ethical and Metaphysical

Javier Aranzadi

Deep Learning Models for Economic Research

Andrzej Dudek

For more information about this series, please visit: www.routledge.com/Routledge-Studies-in-Economic-Theory-Method-and-Philosophy/book-series/RSEMTTP

Deep Learning Models for Economic Research

Andrzej Dudek

First published 2026
by Routledge
4 Park Square, Milton Park, Abingdon, Oxon OX14 4RN

and by Routledge
605 Third Avenue, New York, NY 10158

Routledge is an imprint of the Taylor & Francis Group, an informa business

© 2026 Andrzej Dudek

The right of Andrzej Dudek to be identified as author of this work has been asserted in accordance with sections 77 and 78 of the Copyright, Designs and Patents Act 1988.

All rights reserved. No part of this book may be reprinted or reproduced or utilised in any form or by any electronic, mechanical, or other means, now known or hereafter invented, including photocopying and recording, or in any information storage or retrieval system, without permission in writing from the publishers.

Trademark notice: Product or corporate names may be trademarks or registered trademarks, and are used only for identification and explanation without intent to infringe.

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library

ISBN: 978-1-041-06270-7 (hbk)

ISBN: 978-1-041-06271-4 (pbk)

ISBN: 978-1-003-63462-1 (ebk)

DOI: [10.4324/9781003634621](https://doi.org/10.4324/9781003634621)

Typeset in Times New Roman
by KnowledgeWorks Global Ltd.

To my wife Iza and my daughter Justyna, with love.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Contents

<i>List of figures</i>	<i>xiv</i>
<i>List of tables</i>	<i>xix</i>
Introduction	1
1 Quantitative methods in economics: Deep learning model applications	3
1.1 Quantitative vs qualitative research	3
1.2 Machine learning in economics	6
1.2.1 Areas of application of machine learning in economics	6
1.2.2 Critique of machine learning in economics	9
1.3 Areas of application of deep learning models	12
1.4 Black-box problem of deep learning models	17
2 Deep learning model techniques	22
2.1 Foundations of neural networks	22
2.2 Model building phases	23
2.2.1 Deep learning as unsupervised learning	25
2.3 Components of neural networks	26
2.3.1 Activation functions	26
2.3.2 Loss functions	32
2.3.3 Performance metrics	36
2.3.4 Optimization algorithms	36
2.4 Paradigms (architectures) of neural networks	46
2.4.1 Feedforward neural networks (FNNs)	47
2.4.2 Convolutional neural networks (CNNs)	57
2.4.3 Recurrent neural networks (RNNs)	64
2.4.4 Generative adversarial networks (GANs)	74
2.4.5 Autoencoders	76

2.4.6	<i>Transformers</i>	78
2.4.7	<i>Reinforcement learning</i>	82
2.4.8	<i>Other types of neural networks</i>	94
2.5	<i>Under and overfitting avoiding techniques for neural networks</i>	95
2.5.1	<i>L2 regularization (weight decay)</i>	98
2.5.2	<i>L1 regularization (LASSO)</i>	98
2.5.3	<i>Dropout regularization in deep learning</i>	99
2.5.4	<i>Batch normalization in deep learning</i>	101
2.6	<i>Selected architectures of neural networks</i>	103
2.6.1	<i>ImageNet</i>	103
2.6.2	<i>VGGNet (Visual Geometry Group Network)</i>	103
2.6.3	<i>AlexNet</i>	103
2.6.4	<i>ResNet (Residual Network)</i>	104
2.6.5	<i>GoogLeNet (Inception)</i>	104
2.6.6	<i>MobileNet</i>	104
2.6.7	<i>DenseNet (Densely Connected Convolutional Network)</i>	104
2.6.8	<i>EfficientNet</i>	104
2.6.9	<i>Vision Transformer (ViT)</i>	104
2.6.10	<i>YOLO (You Only Look Once)</i>	104
2.6.11	<i>U-Net</i>	105
2.6.12	<i>BERT (Bidirectional Encoder Representations from Transformers)</i>	105
2.7	<i>Deep learning computer ecosystem</i>	105
2.7.1	<i>NumPy</i>	105
2.7.2	<i>scikit-learn</i>	105
2.7.3	<i>pandas</i>	106
2.7.4	<i>TensorFlow</i>	106
2.7.5	<i>Keras</i>	106
2.7.6	<i>PyTorch</i>	106
2.7.7	<i>OpenCV/ OpenCV2 (Open Source Computer Vision Library)</i>	107
2.7.8	<i>Matplotlib</i>	107
2.7.9	<i>Seaborn</i>	107
2.7.10	<i>Pillow</i>	107
2.7.11	<i>CUDA (Compute Unified Device Architecture)</i>	107
2.7.12	<i>Kaggle</i>	108
2.7.13	<i>MNIST (Modified National Institute of Standards and Technology) dataset</i>	108
2.7.14	<i>CIFAR dataset</i>	108
2.7.15	<i>Iris dataset</i>	108
2.7.16	<i>Pipeline repositories and pre-trained models</i>	109

3	Regression and discrimination problems with deep neural networks	116
3.1	<i>Feedforward networks as regression and discriminant analysis technique</i>	116
3.2	<i>Regularization, overfitting, and proper model design</i>	120
3.2.1	<i>Model design – data preparation</i>	120
3.2.2	<i>Regularization and overfitting</i>	126
3.2.3	<i>Proper model evaluation including imbalance impact</i>	129
3.3	<i>Research application example: vehicle claim fraud detection</i>	132
4	Explanatory model analysis for deep learning models	154
4.1	<i>Beyond black box</i>	154
4.1.1	<i>Permutation methods</i>	157
4.1.2	<i>SHAP</i>	166
4.1.3	<i>PDP</i>	177
4.1.4	<i>LIME</i>	180
4.2	<i>Time limitations of explanatory model analysis for deep learning models</i>	196
4.3	<i>Research application example: The importance of inflation factors based on real data from DBnomics repository</i>	198
4.3.1	<i>Overview of the DBnomics repository</i>	199
5	Time series analysis and forecasting with deep learning models	213
5.1	<i>The specificity of time series data and its impact on the model architecture</i>	213
5.1.1	<i>Stationarity</i>	214
5.1.2	<i>Non-stationarity with unit roots</i>	217
5.1.3	<i>Linearity</i>	218
5.1.4	<i>Direct prediction vs stepwise recurrent prediction</i>	222
5.1.5	<i>Seasonality</i>	223
5.1.6	<i>Data transformation step</i>	223
5.1.7	<i>Time series analysis in economics</i>	224
5.1.8	<i>Methods for deep learning-based time series forecasting</i>	225
5.2	<i>Research application example: comparison of Brownian motion methods, GARCH family methods and RNNs in prediction of volatility of stock prices</i>	225
5.3	<i>Research application example: application of CNN for stock price charts in prediction of the direction of growth</i>	239

6	Sentiment analysis and text mining with deep learning models	249
6.1	<i>Sentiment analysis fundamentals</i>	249
6.2	<i>Bag-of-words methods</i>	250
6.2.1	<i>Dictionaries of positive and negative words</i>	251
6.2.2	<i>POS – The part of speech</i>	251
6.3	<i>Recurrent neural networks-based sentiment analysis</i>	251
6.4	<i>Embeddings, Transformer-based models</i>	252
6.5	<i>Hugging Face repository ecosystem</i>	253
6.6	<i>Introduction to natural language processing (NLP) and large language models (LLM)</i>	254
6.6.1	<i>Text generation</i>	255
6.6.2	<i>Question answering</i>	258
6.6.3	<i>The summarization task</i>	258
6.6.4	<i>The translation task</i>	262
6.6.5	<i>Text classification</i>	263
6.6.6	<i>Token classification</i>	263
6.6.7	<i>Dialogue systems</i>	264
6.6.8	<i>Fill-mask</i>	264
6.6.9	<i>Semantic search</i>	264
6.6.10	<i>Dense retrieval</i>	265
6.6.11	<i>Reranking</i>	265
6.6.12	<i>Retrieval-augmented generation (RAG)</i>	266
6.7	<i>Research application example: comparison of sentiment analysis of IMDb reviews for BoW, GRU, and BERT</i>	266
7	Other applications of deep learning models	277
7.1	<i>Cluster analysis with deep learning models</i>	277
7.1.1	<i>Procedure of cluster analysis</i>	277
7.1.2	<i>Advantage of using SpectralNet instead of spectral clustering algorithm</i>	281
7.1.3	<i>Research application example: clustering with SpectralNet of crypto assets</i>	292
7.2	<i>Recommender systems based on neural networks</i>	308
7.2.1	<i>History of recommender systems: content-based filtering, matrix factorization techniques, collaborative filtering, hybrid techniques</i>	308
7.2.2	<i>Research application example: NCF system for bookstore</i>	313
7.3	<i>Selected non-economical applications of deep learning models</i>	320
7.3.1	<i>Computer vision, neural style transfer, face detection</i>	320
7.3.2	<i>Audio style transfer</i>	327

7.3.3	Multimodal deep learning	329
7.3.4	Usage of large language models through web interface and API	331
Appendix 1: Data availability		340
Appendix 2: Introduction to Python		342
A2.1	Characteristics	342
A2.2	Data science	343
A2.3	Types in Python	343
A2.4	Data types	344
A2.5	Comments	344
A2.6	Functional programming	344
A2.7	Lambda	344
A2.8	Syntax	345
A2.9	Indentation	345
A2.10	Blank lines	346
A2.11	Strings	346
A2.12	Packages	347
A2.13	Python basics	347
A2.14	Lists, dictionaries, tuples	348
A2.15	Control statements and comprehension	352
A2.16	Python – expressions	356
A2.17	Division	357
A2.18	Multiple comparisons	357
A2.19	Imaginary/complex numbers	357
A2.20	Fractional numbers	358
A2.21	Collections and object comparisons	358
A2.22	Slices	358
A2.23	Object-oriented programming	359
A2.24	Inheritance	360
A2.25	Encapsulation	361
A2.26	NumPy	361
A2.27	Operations on matrices	367
A2.28	Operations on two- or more dimensional structures using indices	368
A2.28.1	Selection of arrays by individual dimensions	368
A2.28.2	Selection with condition control	369
A2.28.3	Checking how many elements of a vector the condition is valid for	369

A2.29	<i>Pandas</i>	370
A2.30	<i>Selection by column names</i>	373
A2.31	<i>Combine all commands using method chaining</i>	374
A2.32	<i>Joining DataFrames horizontally (inner/left/right ... join)</i>	375
A2.33	<i>Merging data from join1.csv and join2.csv</i>	375
A2.34	<i>Kaggle API Key acquisition (section 3.3)</i>	376
A2.35	<i>PyTorch installation (section 4.1.4)</i>	377
A2.36	<i>Neural style transfer with TensorFlow hub pipeline</i>	378
Appendix 3: Introduction to R		382
A3.1	<i>R language</i>	382
A3.1.1	<i>Packages</i>	383
A3.2	<i>Foundations of the R language</i>	386
A3.2.1	<i>Literals</i>	386
A3.2.2	<i>Comments</i>	386
A3.2.3	<i>Object names</i>	387
A3.2.4	<i>Set operator</i>	387
A3.2.5	<i>Attributes</i>	388
A3.2.6	<i>Object types</i>	389
A3.2.7	<i>Type conversion</i>	402
A3.2.8	<i>Operators</i>	404
A3.3	<i>Functions in packages</i>	411
A3.4	<i>Mathematical functions</i>	412
A3.5	<i>Aggregation functions</i>	413
A3.6	<i>Operations on two- or more dimensional structures using indices</i>	415
A3.6.1	<i>Selection of arrays by individual dimensions</i>	415
A3.6.2	<i>Selection with condition control</i>	416
A3.6.3	<i>Checking how many elements of a vector the condition is valid for</i>	416
A3.7	<i>Operations on data structures using the dplyr package</i>	419
A3.8	<i>Complex programming constructs</i>	423
A3.8.1	<i>Conditional statement</i>	423
A3.8.2	<i>For loop</i>	424
A3.8.3	<i>Loop with searching for data that meets a certain condition</i>	425
A3.8.4	<i>Loop with data aggregation</i>	426
A3.8.5	<i>While loop</i>	426
A3.9	<i>Other useful functions</i>	427
A3.10	<i>Creating your own functions</i>	429

<i>Appendix 4: R scripts</i>	<i>432</i>
<i>A4.1 Single-layer perceptron</i>	<i>432</i>
<i>A4.2 Multi-layer perceptron</i>	<i>434</i>
<i>A4.3 Simple recurrent neural network</i>	<i>438</i>
<i>A4.4 Vehicle claim fraud detection</i>	<i>445</i>
<i>A4.5 Consumer Price Index (CPI) factors analysis</i>	<i>448</i>
<i>Index</i>	<i>455</i>

List of figures

1.1	Quantitative methods taxonomy.	8
2.1	Biological neuron system.	22
2.2	Model phases for decision trees.	24
2.3	Step activation function.	27
2.4	Linear activation function.	27
2.5	Sigmoid activation function.	28
2.6	Hyperbolic tangent activation function.	29
2.7	ReLU activation function.	30
2.8	Leaky ReLU activation function.	30
2.9	GELU activation function.	31
2.10	Natural logarithm in (0,1 range>.	33
2.11	Surface plot of unknown function. Directions of maximal function value decrease (opposites of maximal function increase).	40
2.12	Surface plot of $(x - 3)^2 + (y + 1)^2$ function. Gradient descent walk path for function (2.27) with starting point $A = (-19, -19)$.	41
2.13	Single-layer perceptron schema – Rosenblat’s original diagram.	47
2.14	The classification scores for AND gate task.	50
2.15	The learned weights in single-layer perceptron for AND gate task.	50
2.16	First dataset for single-layer perceptron – separable classes.	51
2.17	Second dataset for single-layer perceptron – non separable classes.	51
2.18	The classification scores for AND gate task.	52
2.19	The learned weights in single-layer perceptron for non-linearly separable classes.	53
2.20	Multi-layer perceptron with three hidden layers.	54
2.21	The classification scores for non-linearly separable classes.	57
2.22	The learned weights of input layers in multi-layer perceptron for non-linearly separable classes.	58
2.23	The learned weights of hidden layers in multi-layer perceptron for non-linearly separable classes.	58
2.24	Convolutional network architecture.	59
2.25	Pooling	60
2.26	Image to recognize.	61

2.27	The digits sequence predicted (recognized) from image.	64
2.28	RNN architecture.	64
2.29	$\cos^2(x)$ prediction with RNN.	67
2.30	GRU network architecture.	68
2.31	LSTM network architecture.	68
2.32	$\cos^2(x)$ prediction with LSTM network.	74
2.33	Architecture of GANs.	75
2.34	The attention mechanism in sentence, “Conducting economic research using deep learning models is efficient and non-trivial.”	78
2.35	The attention-based transformer.	80
2.36	Reinforcement learning cycle.	85
2.37	Tic tac toe with Q-table reinforcement learning – simple lost game, model not trained.	91
2.38	Tic tac toe with Q-table reinforcement learning – more complex lost game, model in training process.	91
2.39	Tic tac toe with Q-table reinforcement learning – more complex win game after improvement.	92
2.40	Generalization problem illustrated, with support vector classifier and non-random training set selection.	96
2.41	Generalization problem equilibrium.	97
3.1	Multivariate data analysis taxonomy.	117
3.2	Trainable parameters estimation with model. summary() function	128
3.3	Model metrics for lottery winner problem.	130
3.4	Vehicle claim fraud detection dataset characteristics.	135
3.5	Vehicle claim fraud detection dataset encoded data representation.	139
3.6	Vehicle claim fraud detection dataset encoded, no standardization.	142
3.7	Vehicle claim fraud detection dataset encoded, after first standardization.	143
3.8	Vehicle claim fraud detection dataset encoded after standardizations.	143
3.9	Training of vehicle claim fraud detection – deep feedforward network model.	144
3.10	Learning curve of vehicle claim fraud detection – deep feedforward network model.	145
3.11	The confusion matrix for prediction of vehicle claim fraud detection – deep feedforward network model.	146
3.12	Vehicle claim fraud detection – deep feedforward network model vs other models – quality measure benchmark.	148
3.13	Vehicle claim fraud detection – deep feedforward network model vs other models – quality measure benchmark (bar plot).	148
4.1	Base statistics of dataset with one explanatory and two noisy variables related to dependent one.	159
4.2	Dataset with one explanatory and two noisy variables related to dependent one (scatterplots).	160

4.3	The variable importance measures.	161
4.4	The variable importance measures (bar charts).	162
4.5	California housing dependent variable map visualization.	163
4.6	The variable importance measures.	165
4.7	The variable importance measures (bar chart).	165
4.8	Base statistics of dataset with one noisy and two explanatory variables related to dependent one.	167
4.9	Dataset with two noisy and one explanatory variable related to dependent one (scatterplots).	168
4.10	The breakdown variable attribution values.	168
4.11	The breakdown variable attribution values (bar chart).	169
4.12	The breakdown variable attribution values – variables in reverse order.	169
4.13	The breakdown variable attribution values – variables in reverse order (bar chart).	170
4.14	The SHAP values.	174
4.15	The SHAP plot.	174
4.16	The SHAP values.	175
4.17	The SHAP plots.	176
4.18	<i>Ceteris paribus</i> plots.	178
4.19	Partial dependency plots for artificial dataset.	179
4.20	Partial dependency plot for California Housing Dataset.	181
4.21	“The pyramid of animals” (<i>Piramida Zwierząt</i>) by Katarzyna Kozyra.	184
4.22	<i>Superpixels</i> of “The pyramid of animals” by Katarzyna Kozyra	186
4.23	Explanation of hound pattern on “The pyramid of animals” by Katarzyna Kozyra.	187
4.24	Explanation of hen pattern on “The pyramid of animals” by Katarzyna Kozyra.	188
4.25	Explanation of horse cart pattern on “The pyramid of animals” by Katarzyna Kozyra.	188
4.26	Explanation of maze pattern on “The pyramid of animals” by Katarzyna Kozyra.	189
4.27	Explanation of television pattern on “The pyramid of animals” by Katarzyna Kozyra.	189
4.28	Jaffa Gate in Jerusalem.	190
4.29	The <i>superpixels</i> for Jaffa Gate in Jerusalem photograph.	193
4.30	The base predictions and the LIME <i>superpixels</i> contribution in explanation of Jerusalem Jaffa Gate photograph.	193
4.31	Explanation of “castle” pattern on Jerusalem Jaffa Gate photograph.	194
4.32	Explanation of “monastery” pattern on Jerusalem Jaffa Gate photograph.	195
4.33	Explanation of “prison” pattern on Jerusalem Jaffa Gate photograph.	195

4.34	Learning curve plot for CPI explanation model.	207
4.35	Partial dependency plots for CPI explanation model.	207
4.36	Variable importance by permutation measures for CPI explanation model.	209
4.37	Variable importance by permutation measures for CPI explanation model – bar plot	209
4.38	SHAP values for Poland 2020 – CPI explanation model.	209
4.39	SHAP values for Poland 2020 – CPI explanation model (barplot).	210
5.1	Stationary vs non-stationary time series.	216
5.2	Unit root in stochastic process.	217
5.3	The results of Auto-ARIMA modeling of NVIDIA stock prices on 15 February 2025.	220
5.4	The results of Auto-ARIMA modeling of NVIDIA stock prices on 15 February 2015 (time series charts)	221
5.5	Direct vs iterative methods.	222
5.6	Stock values forecasting performance comparison.	235
5.7	Stock values forecasting performance comparison for NVIDIA.	237
5.8	Stock values forecasting for 500-day history models.	238
5.9	Stock values forecasting performance comparison (bar plots for root mean squared error (RMSE), mean absolute error (MAE) and mean absolute percentage error (MAPE)).	239
5.10	Sample candlestick chart used for convolutional network stock price movement prediction model.	240
5.11	Stock price movement convolutional network model prediction results	246
6.1	Sentiment analysis timeline.	250
6.2	Hugging Face pipelines.	254
6.3	Text-generation task with GPT2 model results.	256
6.4	Text-generation task with DeepSeek model results.	258
6.5	Summarization (internal transformers library model) task results.	262
6.6	Translation task with Gemma model results.	263
6.7	Naïve Bayes BoW analysis on IMDb review dataset results.	268
6.8	Multilayer perceptron BoW analysis on IMDb review dataset results.	270
6.9	GRU analysis on IMDb review dataset results.	272
6.10	BERT transformer-based family analysis on IMDb review dataset results.	273
7.1	Artificially generated mlbench_spirals.	282
7.2	The results of mlbench_spirals dataset clustering with spectral clustering, k-means, Ward agglomerative clustering, and Gaussian mixture model-based clustering.	286
7.3	The results of mlbench_spirals dataset clustering with spectral clustering, k-means, Ward agglomerative clustering and Gaussian mixture model-based clustering (scatterplots with predicted cluster marked).	286

7.4	BigLink artificially generated dataset with 1,500,000 objects.	289
7.5	BigLink SpectralNet clustering results.	292
7.6	BigLink <i>SpectralNet</i> clustering results three-dimensional plot with clusters assigned.	292
7.7	Eigenvalues gap for cryptocurrencies analysis.	297
7.8	Cryptocurrencies analysis scatterplots with cluster structure.	300
7.9	Cryptocurrencies analysis cluster belongingness and characteristics.	301
7.10	Bitcoin time series plot as representative of Cluster 3.	308
7.11	Ripple bitcoin time series plot as representative of Cluster 4.	308
7.12	LEO Token time series plot as representative of Cluster 8.	309
7.13	Results of NCF for bookstore recommender system.	318
7.14	Source image and style images for NST task.	324
7.15	The transformed images with NST task.	325
7.16	Lunch atop a skyscraper.	326
7.17	Lunch atop a skyscraper with faces detected.	328
7.18	Sample image to describe its content.	331
7.19	Sample equation solved with Open AI API.	334
7.20	Sample images generated with Open AI API.	335
A2.1	PyTorch installation syntax wizard.	378

List of tables

1.1	The areas of application of machine learning models in economics	10
1.2	The simplified timeline of AI and deep learning	13
1.3	The areas of application of quantitative methods, machine learning methods, and deep learning models in economics	15
1.4	The characteristics of application of quantitative methods, machine learning methods, and deep learning models in economics	16
2.1	Other loss function for regression type tasks	37
2.2	Performance metrics	38
2.3	Approximate values of minimal point for function (2.27) with starting point $A = (-19, -19)$	43
2.4	Logical AND operator	48
2.5	LSTM architecture	69
2.6	GRU architecture	69
2.7	Types of GANs	77
2.8	Reinforcement learning and deep reinforcement learning in economics/finance	94
3.1	Main methods of regression and discriminant analysis	118
3.2	Main ensemble techniques of regression and discriminant analysis	119
3.3	Basic properties of measurement scales	122
3.4	Normalization formulae for metric data	127
3.5	Description of vehicle fraud claims detection dataset variables	133
4.1	Models internally interpretable	155
4.2	Main techniques of EMA	158
4.3	The variables in California Housing Dataset	162
4.4	The number of sets to examine for “all permutations” approach and SHAP approach	171
4.5	Consumer Price Index (CPI) factors research variables description	200
5.1	Main methods of time series data analysis	215
5.2	Ticker names and enterprise descriptions for stock values forecasting simulation study	226

7.1	Distance measures used in clustering procedure	278
7.2	Cluster quality indices used in cluster analysis procedure	280
7.3	The already rated and recommended books for three selected users	318
A1.1	Data availability for datasets and images used in the book	340
A2.1	Basic Python types	344
A2.2	String and literals, main utility functions	347
A2.3	List operations	349
A2.4	Dictionary operations	351
A2.5	Tuple operations	352
A2.6	Python operators and expressions	356
A2.7	Selected NumPy function	362
A2.8	Selected Pandas functions	371
A3.1	Literals in R	386
A3.2	Summary of the most important attributes related to objects in R	389
A3.3	Key functions of simple type conversion	403
A3.4	Key functions of complex object class conversions	403
A3.5	Arithmetic operators in R	404
A3.6	Selected R mathematical functions	413
A3.7	Selected aggregate functions	414
A3.8	Other important R functions	427

Introduction

The ability to make sense of complex, multidimensional datasets is essential for economists and data scientists. In today's large and complex data world, traditional quantitative models may not always be appropriate. Deep learning models have been very successful in image processing and natural language processing, or, for example, in autonomous vehicles, but their application in economics is not yet as everyday routine as in the areas mentioned above. Thus, the purpose of this book is to take a comprehensive look at application areas, presenting the achievements to date and indicating places where deep learning models could be successfully used.

The book intends to present a comprehensive approach to understanding and applying neural networks and deep learning models in the context of conducting economic research.

At this point, a gentle distinction is due to the reader: the book will discuss all the techniques used by artificial intelligence, especially those related to deep neural networks and machine learning. However, it will not be a textbook on the so-called artificial intelligence in economic theory in the sense of proposing new economic theories or a real discussion of the falsifiability of economics as a science. The author believes that in the next few decades, there will be no such models of general artificial intelligence that can carry out this task. Still, he notes the development of pre-trained language models based on attention and transformers and chain of thoughts, which is why [Chapters 6](#) and [7](#) describe the use of artificial intelligence models through transformers library and commercial application programming interfaces (APIs).

Although the development of large language models is accelerating, this sentence may be outdated at the time when the book is read.

In the interest of reproducibility, the book provides complete access to the data as well as the accompanying Python and R code, along with brief instructions on installing packages and generating API keys. Any commands requiring a pseudorandom number generator, including those used for deep learning models, are stamped with the key `25032005`. As a result, barring any minor rounding discrepancies due to floating-point architecture (which are common in complex algorithms), readers should be able to replicate the exact results presented in the book on their computers.

2 *Deep Learning Models for Economic Research*

To save the printed/PDF space, the comments are almost fully removed (with the exception of the section header) from the code included in the chapter. Full code with Python doc and comments, and additional scripts and files may be found in the GitHub repository: https://github.com/a-dudek-ue/erwdlm_book.

1 Quantitative methods in economics

Deep learning model applications

1.1 Quantitative vs qualitative research

The distinction between qualitative and quantitative research has been present in the social sciences almost since their inception. In the economic sciences, one can speak of an ongoing debate between proponents of both approaches since the end of the 19th century. At the heart of this debate lies the question of whether economic inference should be inductive – rooted in a detailed analysis of individual cases (the qualitative approach) – or stochastic and generalizing, employing deductive reasoning (the quantitative approach) (see [Hammersley, 1989](#), pp. 32–33). The origins of this dispute can be traced back to the “Methodenstreit”, which was initiated by Carl Menger in 1883, the founder of the Austrian school of economics (see, e.g., [Barkai, 1996](#), pp. 1–19). The dividing line in this debate was distinct from the 20th-century separation between Keynesian and monetarist perspectives within neoclassical economics. Among scholars who expressed skepticism regarding the application of quantitative methods in the sciences – or who recognized the risks associated with their uncritical application and the potential for excessive formalization – were notable scientists such as John Maynard Keynes, as well as prominent supporters of monetarism and the Austrian school, including Milton Friedman and Friedrich Hayek.

The creator of Roosevelt’s “New Deal”, John Maynard Keynes stated ([Keynes, 1936](#), p. 148):

The object of our analysis is, not to provide a machine, or method of blind manipulation, which will furnish an infallible answer, but to provide ourselves with an organised and orderly method of thinking out particular problems; and, after we have reached a provisional conclusion by isolating the complicating factors one by one, we then have to go back on ourselves and allow, as well as we can, for the probable interactions of the factors amongst themselves. This is the nature of economic thinking. Any other way of applying our formal principles of thought (without which, however, we shall be lost in the wood) will lead us into error. It is a great fault of symbolic pseudo-mathematical methods of formalising a system of economic analysis, such as we shall set down in section vi of this chapter, that they expressly

4 *Deep Learning Models for Economic Research*

assume strict independence between the factors involved and lose all their cogency and authority if this hypothesis is disallowed; whereas, in ordinary discourse, 149 where we are not blindly manipulating but know all the time what we are doing and what the words mean, we can keep ‘at the back of our heads’ the necessary reserves and qualifications and the adjustments which we shall have to make later on, in a way in which we cannot keep complicated partial differentials ‘at the back’ of several pages of algebra which assume that they all vanish. Too large a proportion of recent ‘mathematical’ economics are merely concoctions, as imprecise as the initial assumptions they rest on, which allow the author to lose sight of the complexities and interdependencies of the real world in a maze of pretentious and unhelpful symbols.

Friedrich Hayek, the 1974 laureate of the Nobel Memorial Prize in Economic Sciences and one of the most famous representatives of the Austrian school ([Hayek, 1945](#), pp. 524, 530) said:

This is, perhaps, also the point where I should briefly mention the fact that the sort of knowledge with which I have been concerned is knowledge of the kind which by its nature cannot enter into statistics and therefore cannot be conveyed to any central authority in statistical form.

And, as conclusion:

Professor Schumpeter’s standing should thus have fallen into a trap which the ambiguity of the term ‘datum’ sets to the unwary can hardly be explained as a simple error. It suggests rather than there is something fundamentally wrong with an approach which habitually disregards an essential part of the phenomena with which we have to deal: the unavoidable imperfection of man’s knowledge and the consequent need for a process by which knowledge is constantly communicated and acquired. Any approach, such as that of much of mathematical economics with its simultaneous equations, which in effect starts from the assumption that people’s knowledge corresponds with the objective facts of the situation, systematically leaves out what is our main task to explain. I am far from denying that in our system equilibrium analysis has a useful function to perform. But when it comes to the point where it misleads some of our leading thinkers into believing that the situation which it describes has direct relevance to the solution of practical problems, it is time that we remember that it does not deal with the social process at all and that it is no more than a useful preliminary to the study of the main problem.

Karl Popper criticized the methods of the quantitative approach in economics for lacking falsifiability, i.e., not being amenable to constructing an experiment that could confirm or refute them (see, e.g., [Boland, 1998](#), pp. 155–173).

The well-known historian of economic thought, Robert L. Heilbroner (1999), states in the interview afterword to *The Worldly Philosophers: The Lives, Times, and Ideas of the Great Economic Thinkers* that,

I guess the scientific approach began to penetrate and soon dominate the profession in the past twenty to thirty years. This came about in part because of the ‘invention’ of mathematical analysis of various kinds and, indeed, considerable improvements in it. This is the age in which we have not only more data but more sophisticated use of data. So there is a strong feeling that this is a data-laden science and a data-laden undertaking, which, by virtue of the sheer numerics, the sheer equations, and the sheer look of a journal page, bears a certain resemblance to science ... That one central activity looks scientific. I understand that. I think that is genuine. It approaches being a universal law. But resembling a science is different from being a science.

Milton Friedman was not strictly opposed to the use of quantitative methods and mathematical apparatus as a language for describing economics, but in his *Essays in Positive Economics* (Friedman, 1953, pp. 6–7) he mentioned that,

One effect of the difficulty of testing substantive economic hypotheses has been to foster a retreat into purely formal or tautological analysis. As already noted, tautologies have an extremely important place in economics and other sciences as a specialized language or ‘analytical filing system.’ Beyond this, formal logic and mathematics, which are both tautologies, are essential aids in checking the correctness of reasoning, discovering the implications of hypotheses, and determining whether supposedly different hypotheses may not really be equivalent or wherein the differences lie. But economic theory must be more than a structure of tautologies if it is to be able to predict and not merely describe the consequences of action; if it is to be something different from disguised mathematics. And the usefulness of the tautologies themselves ultimately depends, as noted above, on the acceptability of the substantive hypotheses that suggest the particular categories into which they organize the refractory empirical phenomena

By contrast, Alfred Marshall is considered as the forerunner of using the language of mathematics in neoclassical economics, particularly in his work *Principles of Economics* (Marshall, 1890) (see, e.g., Brems, 1975). Notable representatives of the quantitative approach in economics include Simon Kuznets, John Nash, Wassily Leontief, Gérard Debreu, Harry Markowitz, Daniel McFadden, Robert Engle, and Clive Granger.

Paul Anthony Samuelson (Samuelson, 1952, pp. 63–64) stated that,

For a century mathematics knocked at the door. Even today it has no more than a foot in the doorway. But the problems of economic theory – such as the incidence of taxation, the effects of devaluation – are by their nature

quantitative questions whose answer depends upon a superposition of many different pieces of quantitative and qualitative information. When we tackle them by words, we as when we write out those equations.

Further (Samuelson, 1952, p. 64):

The convenience of mathematical symbolism for handling certain deductive inferences is, I think, indisputable. It is going too far to say that mathematicians never make mistakes. Like everybody else, they can pull some awful boners. But it is surprising how rare pure mistakes in logic are. Where the really big mistakes are made is in the formulation of premises. Logic is no protection against false hypotheses; or against misinterpretation of reality; or against the formulation of irrelevant hypotheses. I think it is one of the advantages of the mathematical medium-or, strictly speaking, of the mathematician's customary canons of exposition of proof, whether in words or symbols-that we are forced to lay our cards on the table so that all can see our premises. But I must confess that I have heard of card games-in fact I have participated in them myself-where knowingly or unknowingly, we have dealt cards from the bottom of the deck. So, there are no absolute checks against human error.

Mario Bunge (1959) concluded that, "to reject statistical explanation would be as unwise as to regard it as the final word".

1.2 Machine learning in economics

1.2.1 Areas of application of machine learning in economics

The discussion in the introductory chapter concerns the quantitative approach in economics in general. However, it is necessary to distinguish between the use of mathematical symbols to formulate the theory of economics and the use of specific methods in order to solve economic problems on a micro or macro scale or to optimize tasks aimed at maximizing profit, minimizing risk, or saving working time. In addition, some of the methods have a strong statistical basis, well-defined theoretical models, parametric assumptions, and explicit causality, but also limitations of dataset sizes. In contrast, others relax parametric assumptions and focus on data-driven predictions, handling nonlinear relationships, high-dimensional datasets, and complex interactions without requiring prior theoretical assumption. For this book's purposes, we will use the following taxonomy of qualitative methods (the author hopes that this taxonomy is the synthesis of many sources, not identical to any, but also not too far from each):

Quantitative methods refer to one-, two- or multidimensional statistical methods enabling, in particular, the collection of empirical data, the construction of computational models, the development of instruments and measurement methods,

the control of data during the research process, the modeling and analysis of data, and the objective evaluation of research results. Among its techniques well-known methods like linear regression, time series models (autoregressive integrated moving average (ARIMA), vector auto regressive (VAR), generalized autoregressive conditional heteroskedasticity (GARCH)), structural equation models, and general equilibrium models may be listed as prominent representatives.

Machine learning methods are based at a lower degree on stochastic foundations and are more focused on detecting nonlinear and nonparametric dependencies. Typical representatives of this group of methods are decision trees, nonparametric regression methods, random forests, gradient boosting, and support vector machines (SVMs).

Deep learning is a subdomain of machine learning that, in most cases, uses neural networks with multiple layers to capture high-dimensional, nonlinear, and sequential dependencies. It is particularly effective for analyzing big data, text, and time series forecasting. Deep learning allows computational models composed of multiple processing layers to learn representations of data with multiple levels of abstraction (LeCun et al., 2015).

The term deep learning may be interpreted in two slightly different ways. Chollet and Allaire (2018) interpret it as the opposite of “shallow” methods, especially “shallow neural networks” without hidden layers. In this sense, the method or model is “deep” if, in its implementation, tens or hundreds of hidden layers are included. In similar but not identical meaning, the term highlights the ability to train deeper neural networks, which had previously been unattainable. This shift underscored the theoretical significance of depth in neural networks (Bengio & LeCun, 2007; Delalleau & Bengio, 2011; Pascanu et al., 2014a; Montúfar et al., 2014; cf. Goodfellow et al., 2016, p. 19). Deep neural networks demonstrated superior performance compared to other AI systems that relied on alternative machine learning techniques or manually designed functionalities in the following constantly increasing list of domains (cf. Chollet & Allaire, 2018):

- Image classification close almost to the human level;
- Speech recognition at a level similar to human capabilities;
- Handwriting transcription at a level similar to human capabilities;
- Improvement of machine translation;
- The emergence of assistants such as Google Now and Amazon Alexa;
- Autonomous vehicles driving almost as well as guided vehicles by people;
- Improving the selection of ads in Google, Baidu and Bing;
- Improvement of results returned by search engines;
- The ability to answer questions asked in natural language;
- Defeating a person in a game of Go;
- Realistic simulations to train AI agents in virtual environments;
- Neural style transfer;
- Medical imaging;

8 Deep Learning Models for Economic Research

- Drug discovery;
- The rise of generative models like Dall-E or SORA;
- Text-to-speech recognition;
- Voice assistants;
- Autonomous agents in enterprise-class computer systems;
- Autonomous robots;
- The rise of large language models (LLMs).

The proposed relationship between quantitative methods, machine learning, and deep learning in economics is visualized in [Figure 1.1](#).

The structure proposed in [Figure 1.1](#) is hierarchical, i.e., machine learning methods are a subdomain of quantitative methods, and machine learning is a superset of deep learning; however, in order to avoid misunderstandings, the terms will be used in a disjunctive sense (unless implicitly stated otherwise).

The proposed taxonomy is independent of the well-known distinction between supervised and unsupervised learning. An example of regression analysis can be performed using a simple regression model (quantitative methods), machine learning method (Least Absolute Shrinkage and Selection Operator (LASSO)) regression, or regressive multi-layer perceptron (deep learning).

Please note that *Multivariate Statistical Analysis* methods in the [Hair et al. \(2009\)](#) sense should be placed mainly in the “pure” statistical and mathematical methods domain, with some intersection with the machine learning part.

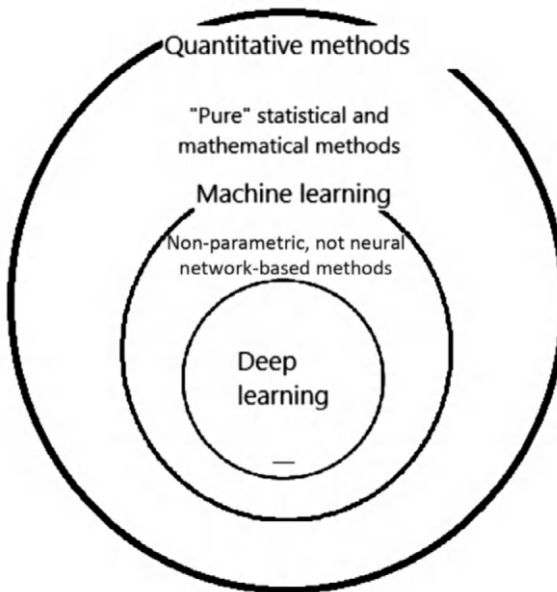


Figure 1.1 Quantitative methods taxonomy.

Source: Own proposal based on [Wooldridge \(2010\)](#), [Varian \(2014\)](#), [Mullainathan and Spiess \(2017\)](#).

Mullainathan and Spiess (2017) enlist four main domains of machine learning application in economics, as follows.

New data: Advances in machine learning allow economists to utilize new, high-dimensional data sources – like satellite imagery or online text – that traditional methods could not process. The rise of “big data” is transcending scale, enabling the extraction of economically significant signals from unconventional data such as night-time light intensity, restaurant reviews, and corporate disclosure texts.

Prediction in the service of estimation: Machine learning can help address the overfitting issues found in common estimation problems, such as the two-stage procedure in instrumental variables, by treating the first stage as a prediction task. Regularization techniques and careful split-sample strategies allow better inference and more potent instruments without relying on ad hoc specification choices.

Prediction in policy: Certain policy decisions – like bail assignments or hiring – fundamentally rely on forecasts (for instance, predicting whether a defendant will re-offend). Economists’ expertise with causal inference and machine learning’s predictive power can improve such policy outcomes, although adoption and behavioral considerations remain key challenges.

Testing theories: Machine learning can test theories that hinge on predictability, as in evaluating the efficient markets hypothesis by searching for predictive signals in past stock returns. It also provides an atheoretical benchmark for judging how well a given theory explains variation by comparing its predictive capacity to an optimal data-driven predictor.

The applications of machine learning methods in various areas of economics are presented in Table 1.1.

1.2.2 Critique of machine learning in economics

Numerous authors (see, e.g., Clarke & Primo, 2012; Dobija, 2011; Kutner et al., 2018; Lopez de Prado, 2015) have pointed out the important role of modeling in finance (referred to in many studies as “physical economics”, “physical econometrics” or ironically “physical fixation” (physics envy)) in the emergence of financial crises. Models that give very good results in strict sciences, such as physics, do not always have to work in economics, a social science.

Former Federal Reserve Chief Paul Volcker at the Federal Reserve Bank of Chicago meeting, according to *Real Time Economics* (<http://blogs.wsj.com/economics/2010/09/23/volcker-spares-no-one-in-broad-critique/>) said:

We had all our best business schools in the United States pouring out financial engineers, every smart young mathematician and physicist said, ‘I don’t want to be a civil engineer, a mechanical engineer. I’m a smart guy, I want to go to Wall Street.’ And then you know all the risks were going to be sliced and diced, and [people thought] the market would be resilient and not face

Table 1.1 The areas of application of machine learning models in economics

<i>Area of economics</i>	<i>Typical applications</i>	<i>Common machine learning techniques</i>	<i>Example data sources</i>
Macroeconomic forecasting	Predicting gross domestic product (GDP), inflation, and employment rates.	Time series analysis (ARIMA, long short-term memory (LSTM), recurrent neural network (RNN)).	National statistical agencies (e.g., central bank data) International organizations (International Monetary Fund, World Bank).
Financial markets and trading	Algorithmic trading strategies; portfolio optimization; stock price prediction.	Reinforcement learning; gradient boosting; neural networks.	Market price feeds (e.g., Bloomberg, Reuters, Yahoo Finance); company fundamentals, historical prices.
Risk management and credit scoring	Default risk prediction; creditworthiness assessment.	Logistic regression; random forests; gradient boosted trees.	Credit bureau data; transaction histories; customer profiles.
Consumer behavior and marketing	Demand forecasting; recommendation systems; customer segmentation.	Clustering (k-means, hierarchical); deep learning (for personalized recommendations); natural language processing (NLP).	E-commerce transaction logs; social media feeds; loyalty program records.
Pricing and revenue management	Dynamic pricing strategies; yield management.	Regression models; reinforcement learning; Bayesian optimization.	Sales transactions; competitor pricing data; market trend data.
Supply chain and inventory	Optimal stock levels; demand and supply forecasting; logistics routing.	Time series analysis; reinforcement learning; forecasting models (XGBoost).	Enterprise resource planning (ERP) system data; order histories, shipping and logistics databases.
Policy analysis and labor economics	Evaluating policy impact on employment; wage prediction; labor market trend analysis.	Causal inference methods (e.g., synthetic controls); ordinary least squares (OLS) regression, random forests); NLP (for text-based policy data).	Government labor statistics; household surveys; policy documents.

Source: Own work based on [Mullainathan and Spiess \(2017\)](#).

any crises. We took care of all that stuff, and I think that was the general philosophy that markets are efficient and self-correcting, and we don't have to worry about them too much.

Peter Wilmott, founder of Wilmott.com, stated ([Wilmott, 2000](#)),

It is clear that a major rethink is desperately required if the world is to avoid a mathematician-led market meltdown... . The underlying assumptions in the models, such as the importance of the normal distribution, the elimination of risk, measurable correlations, etc., are incorrect... .

And later ([Freedman, 2011](#)):

“When you have to keep recalibrating a model, something is wrong with it,” he says. “If you had to readjust the constant in Newton’s law of gravity every time you got out of bed in the morning in order for it to agree with your scale, it wouldn’t be much of a law. But in finance, they just keep on recalibrating and pretending that the models work.”

[Kleinberg et al. \(2015\)](#) note that algorithms that optimize predictive accuracy can miss important aspects needed for reasonable thinking and may lead to wrong policy conclusions. [Varian \(2014\)](#) warns that while “big data” can be powerful, economists must remain vigilant about measurement errors, missing variables, and unrepresentative samples that weaken the credibility of machine learning insights. [Agrawal et al. \(2019\)](#) note that prediction allows for more risky decisions, and this is its impact on observed productivity, although it may also increase the variance in the results.

Critics of the excessive use of quantitative methods and machine learning in economics often cite the examples of the long-term capital management (LTCM) fund and the impact of the valuation of collateralized instruments on the outbreak of the subprime crisis in 2008.

LTCM was established in 1994 by John W. Meriwether, Myron S. Scholes, and Robert C. Merton as a hedge fund specializing in derivatives trading, using advanced financial engineering techniques. In its early years, LTCM reported significant profits for its investors (21% in the first year, 41% the next, and 43% two years later). In 1997, Scholes and Merton received the Nobel Prize in Economics for their work on a new formula for pricing derivatives ([Black & Scholes, 1973](#)). However, 1997 and 1998 saw severe Asian financial crises; in 1998, LTCM incurred losses of \$4.6 billion. By the year 2000, the fund had been liquidated.

The complexity of financial instruments used in subprime mortgage securitization may be observed in the example of collateralized debt obligations (CDOs) and so-called collateralized debt spreads (CDS). CDOs, a type of bond and structured credit product, bundle various forms of debt and sell the resulting cash flow streams to investors. In 1999, David X. Li proposed the first formula for estimating the risk of such CDOs, aiming to gauge the likelihood that at least one tranche in a

portfolio would remain solvent. While this approach was sophisticated and mathematically advanced – rendering it difficult for non-experts to grasp – it did not address whether the portfolio’s value could drop dramatically.

With the use of Li’s formula, banks changed their CDO construction strategies, combining various high-risk instruments with only one limitation: a high correlation of default risk estimation. These newly structured CDOs, despite their real content, often received AAA or AA+ credit ratings, accelerating the growth of both the CDO and CDS markets. From 2001 to 2007, the CDS market grew from \$920 billion to \$62 trillion, and the CDO market grew from \$275 billion in 2000 to \$4.7 trillion in 2006 (see [Condon, 2010](#)).

All the objections to machine learning have in common is that they are too sophisticated and hard to interpret. In many sources, it is called the black-box model (or black-box problem, which is discussed in section 1.7).

1.3 Areas of application of deep learning models

Deep learning and artificial intelligence development started approximately mid-20th century with Turing’s Imitation Game (1951) and Samuel’s Checkers Program (1954). Rosenblatt’s perceptron (1958) was one of the first attempts to build artificial neural networks, although limitations in the approach stalled progress for a while. Momentum emerged again in 1969 with backpropagation, a method that allowed multilayer neural networks to be trained more efficiently, laying important foundations for today’s models. Deep learning techniques began to find practical applications in the late 20th century. For example, IBM’s Deep Blue defeated the world chess champion in 1997 using a brute-force AI strategy.

Meanwhile, games like Colin McRae Rally 2.0 (2000) and Black & White (2001) featured early versions of learning algorithms rooted in neural networks. In 2011, IBM Watson won the popular Jeopardy! gameshow, demonstrating capabilities of understanding and question answering in natural language. Five years later, in 2016, deep learning achieved the victory that for many years was the impossibility benchmark for non-human players: AlphaGo beat Lee Sedol at a five-game Go match using a mix of deep neural networks and Monte Carlo tree search. This moment proved that AI could handle complex, intuitive tasks at a superhuman level.

The “revolution” started in 2022 and 2023 with the emergence of transformer-based models like GPT-3 and ChatGPT, whose creation and use were noticeable in virtually every aspect of digital life. In 2024, the significance of these advances was recognized worldwide when key pioneers like Geoffrey Hinton and John Hopfield were awarded Nobel Prizes for groundbreaking work in neural networks and biological applications such as protein structure prediction.

The timeline in [Table 1.2](#) presents the development from early symbolic approaches and expert systems to today’s adaptive, self-improving models. Deep learning has gone from a theoretical curiosity to a central force shaping the future of AI.

Table 1.2 The simplified timeline of AI and deep learning

<i>Year/date</i>	<i>Event/system</i>	<i>Description/significance</i>
1951	Alan Turing's "Gender-Guessing Game"	Turing devised a paper-based game in which players attempted to guess each other's gender, foreshadowing the idea of the "Imitation Game" (later conceptualized as the Turing Test) for assessing machine intelligence.
1954	Arthur Samuel's Checkers Program	Samuel's computer program defeated the fourth-ranked checkers player in the United States, demonstrating one of the first self-learning programs and early breakthroughs in machine learning.
1956	Dartmouth Workshop	Considered as the birth of AI as a field; John McCarthy, Marvin Minsky, Claude Shannon, and others met to formalize research goals and the term "artificial intelligence".
1958	Frank Rosenblatt's Perceptron	Introduction of the perceptron, an early neural network model that laid foundational ideas for pattern recognition and machine learning.
1964	STUDENT (Daniel Bobrow)	An early NLP program designed to solve algebra word problems, while he was a doctoral candidate at Massachusetts Institute of Technology (MIT).
1966	ELIZA (Joseph Weizenbaum)	An early NLP program simulating a psychotherapist by pattern-matching user inputs, sparking both fascination and debate over machine understanding.
1967	MacHack (Richard Greenblatt)	A chess program developed at MIT, notable for earning a US Chess Federation rating – one of the first AI systems to compete successfully against human players in chess.
1969	Backpropagation	Arthur Bryson and Yu-Chi Ho described a backpropagation learning algorithm to enable multilayer artificial neural networks, an advancement over the perceptron and a foundation for deep learning.
1972	MYCIN (expert system)	A rule-based expert system (developed at Stanford) for medical diagnosis and therapy recommendations, demonstrating the promise of AI in specialized domains.
1980s	Rise of expert systems	Commercial "expert systems" (like XCON at DEC) leveraged large rule bases to automate complex decision-making – an AI boom in industry before the "AI Winter" of the late 1980s.
1983	The Hobbit (interactive fiction game)	Featured computer-controlled characters (agents) that advised the main player, showcasing early attempts at in-game AI to create more dynamic, story-driven experiences.
1994	World of Warcraft (early versions)	Used heuristic pathfinding algorithms (like A*) to navigate complex terrains in real time, influencing how modern AI agents plan routes in large, dynamic environments.
1997 (May 11)	IBM's Deep Blue defeats Garry Kasparov	Deep Blue won a six-game match (3½–2½) against the reigning world chess champion Kasparov, marking a historic achievement in computer chess and AI.

(Continued)

Table 1.2 (Continued)

<i>Year/date</i>	<i>Event/system</i>	<i>Description/significance</i>
1999	Unreal Tournament	Introduced computer “bots” that could mimic and learn from player behavior, advancing AI in first-person shooter games.
2000	Colin McRae Rally 2.0	Employed neural networks to steer a car-simulation engine, illustrating the use of machine-learning techniques in commercial driving/racing games.
2009	GPU	Rajat Raina, Anand Madhavan and Andrew Ng published “Large-Scale Deep Unsupervised Learning Using Graphics Processors”, presenting the idea of using GPUs to train large neural networks.
2001	Black & White	Combined discriminant analysis and neural networks to enable non-player “creatures” (agents) to learn from player actions and adapt their behavior dynamically.
2011	IBM Watson on Jeopardy!	Watson’s victory in the gameshow Jeopardy! showcased advances in NLP and information retrieval, handling natural language queries at high speed and accuracy.
2016	AlphaGo defeats Lee Sedol	Google DeepMind’s AlphaGo program beat a world champion at the complex board game Go, underscoring the power of deep neural networks and Monte Carlo tree search.
2017	Transformer	Google researchers developed the concept of transformers in the seminal paper “Attention Is All You Need”, inspiring subsequent research into tools that could automatically parse unlabeled text into LLMs.
2017	Warning	British physicist Stephen Hawking warned, “Unless we learn how to prepare for, and avoid, the potential risks, AI could be the worst event in the history of our civilization.”
2022–2023	Large Language Models (e.g., GPT series)	Transformer-based models (e.g., GPT-3, ChatGPT) demonstrated remarkable capabilities in language understanding, generation, and reasoning, revitalizing interest in AI and prompting widespread adoption in numerous applications.
2024	Nobel Prizes related to neural networks/deep learning	John J. Hopfield, “for foundational discoveries and inventions that enable machine learning with artificial neural networks”. Geoffrey Hinton, “for foundational discoveries and inventions that enable machine learning with artificial neural networks”. David Baker, “for computational protein design”. Demis Hassabis, “for protein structure prediction”. And John Jumper, “for protein structure prediction”.

Source: Own work empowered by <https://www.techtargert.com/searchenterpriseai/tip/The-history-of-artificial-intelligence-Complete-AI-timeline>.

Table 1.3 The areas of application of quantitative methods, machine learning methods, and deep learning models in economics

<i>Area of economics</i>	<i>“Traditional” quantitative methods</i>	<i>Common machine learning techniques</i>	<i>Deep learning models</i>
Equilibrium analysis	Yes	–	–
Macroeconomic forecasting	Yes	Yes	Yes
Financial markets and trading	Yes	Yes	Yes
Risk management and credit scoring	Yes	Yes	Yes
Consumer behavior and marketing	Yes	Yes	Yes
Pricing and revenue management	Yes	Yes	Yes
Supply chain and inventory	Yes	Yes	Yes
Policy analysis and labor economics	Yes	Yes	Yes
Natural language empowered models	–	Yes (limited)	Yes
Image, video and sound empowered analysis	–	Yes (limited)	Yes

Source: Own work based on [Mullainathan and Spiess \(2017\)](#) and [Zheng et al. \(2023\)](#).

The scope of applications of deep learning models is not too far from machine learning and other quantitative methods. [Table 1.3](#) shows the most important application areas for each of these groups. At the same time, to the author’s knowledge, the machine and deep learning models have not been used in equilibrium analysis. On the other hand, “traditional” methods do not support the analysis of textual data, images, or sounds.

Although, as mentioned, the main domains of the three families are close to each other, there are crucial differences in assumptions, data used, computational needs, and other characteristics, which are summarized in [Table 1.4](#).

[Zheng et al. \(2023\)](#) classify the selected application in economics to particular types of deep learning models:

- restricted Boltzmann machines and deep belief networks;
- convolutional networks;
- recurrent networks;
- autoencoders;
- transformer-based networks;
- deep reinforcement learning models.

Restricted Boltzmann Machines have found applications in recommenders’ systems ([Chen et al., 2020](#)), image segmentation ([Li & Wang, 2020](#)), and natural

Table 1.4 The characteristics of application of quantitative methods, machine learning methods, and deep learning models in economics

<i>Aspect</i>	<i>“Traditional” quantitative methods</i>	<i>Common machine learning techniques</i>	<i>Deep learning models</i>
Model type	Econometric models	Non-parametric machine learning models	Neural networks
Assumptions	Strong economic assumptions	Minimal assumptions	No predefined assumptions
Focus	Explanation and inference	Prediction	Pattern recognition
Interpretability	High (transparent models)	Moderate	Low (black box)
Computational needs	Low	Moderate	High
Data size	Small to medium	Large	Very large

Source: Own work based on [Mullainathan and Spiess \(2017\)](#) and [Zheng et al. \(2023\)](#).

language processing ([Tsutsui & Hagiwara, 2019](#)). Restricted Boltzmann Machines have been successfully integrated into macroeconomics and monetary policy modeling ([Galeshchuk, 2017](#)), urban and regional economics ([Rafiei & Adeli, 2018](#)), and agricultural market analysis ([Pei et al., 2020](#)). Restricted Boltzmann Machines serve as the building blocks for Deep Belief Networks, which improve prediction quality by stacking multiple Restricted Boltzmann Machines layers.

Convolutional neural networks (CNNs) have demonstrated strong predictive capabilities across macroeconomics and monetary policy modeling ([Chen et al., 2019](#); [Galeshchuk & Demazeau, 2017](#)), financial economics ([Liu et al., 2020](#)), and urban and real estate economics ([Ajami et al., 2019](#)). Their effectiveness stems from noise filtering and dimensionality reduction techniques, which help extract meaningful economic features from large datasets ([Yasir et al., 2019](#)). Additionally, CNNs enable economic forecasting using visual data, such as satellite imagery for real estate price estimation or financial chart pattern recognition, providing a unique perspective for economic assessment ([Galeshchuk & Demazeau, 2017](#)). [Sanda et al. \(2023\)](#) used 1D 4,2 convolution for long-range attacks in proof-of-stake blockchains.

Recurrent neural networks (RNNs) and their variants, such as LSTM networks, have demonstrated strong applicability in macroeconomics, financial economics, industrial organization, and agricultural economics ([Alsmadi et al., 2020](#); [Andrijasa, 2019](#)). RNNs may be more effective than standard feedforward neural networks for tasks involving contextual dependencies ([Anbazhagan & Kumarappan, 2013](#)). When combined with logistic regression, encoder-decoder architectures, and attention mechanisms, RNN-based models consistently outperform traditional approaches, such as SVMs and standard neural networks, in forecasting tasks ([Becerra-Vicario et al., 2020](#); [Mishev et al., 2020](#)). In time series forecasting, such as gas, electricity prices, or exchange rates, RNNs efficiently discover temporal

patterns in series, which may increase the accuracy of forecasting (Andrijasa, 2019; Zhang et al., 2020).

Autoencoders are used in economic research, mainly for dimensionality reduction and simplifying complex data representation (Long et al., 2020; Ranjan et al., 2021). Their interpretation, in contrast to another deep learning model, is relatively straightforward.

The application of transformer-based networks is not limited to AI Chat GPT-like domains, but also covers financial sentiment analysis and economic forecasting (Devlin et al., 2018; Wang & Li, 2022) and image-based data, such as urban development assessment using satellite imagery (Fisichella & Garolla, 2021).

Deep reinforcement learning is a powerful tool in economic decision-making (Mosavi et al., 2020). Deep Q-learning-based algorithms have been developed in financial economics to optimize automated trading decisions (Chakole & Kurhekar, 2020). In agricultural and natural resource economics, deep reinforcement learning techniques such as proximal policy optimization and deep deterministic policy gradient have been applied to optimize power grid management and electric vehicle pricing strategies (Qiu et al., 2020; Zhou et al., 2020). Similarly, deep reinforcement learning models have been used in macroeconomics and health policy to predict cryptocurrency market movements and learn optimal COVID-19 prevention strategies (Sattarov et al., 2020; Uddin et al., 2020).

1.4 Black-box problem of deep learning models

Deep learning models have many economic applications, and recent advances based on transformer pre-trained models are astounding. However, the central aspect criticized by their opponents is their complexity and the impossibility of explaining how they work. Deep learning models, particularly deep neural networks, are often called black-box models because they operate in a difficult-to-interpret way. No “simple” explanations of model behavior remain, such as interpreting split selections in the decision tree building phase or slope and intercept coefficients for linear regression. The interpretation of deep models may be challenging or almost impossible, which, on the other hand, is sometimes unacceptable in domains requiring transparency, such as healthcare, finance, and law.

The primary concern with black-box models is the lack of interpretability and explainability. In many real-world applications, it is not enough for a model to only provide an accurate prediction; decision-makers need to understand why a particular prediction points or does not point the recommendation for further execution. For example, a bank may use a deep learning model to evaluate loan applications in credit scoring. However, if the model denies a loan, regulators and applicants should be able to understand the reasoning behind the decision. The low level of transparency can lead to situations where you have “artificial intelligence” as the final judge, which is unacceptable to people, and the evaluation concerns and the recommendations proposed by algorithms may raise ethical and legal issues.

Another significant threat to black-box models is the difficulty of diagnosing errors. If a deep learning model makes incorrect predictions due to an incorrect data

preparation stage, unbalanced input, or any noise in the execution pipeline, finding the source of the problem can be much more complicated than with traditional models. Since the model's internal parameters are learned automatically from data, biases present in the training data can be encoded without explicit human oversight, potentially leading to discriminatory or unfair outcomes. Diagnosing bias and errors is particularly problematic in, for example, criminal justice or medical diagnostics, where incorrect predictions can have serious consequences.

The criticism of machine learning is one of the factors determining the rapid growth of explanatory model analysis/explainable artificial intelligence methods.

Researchers are developing explainable AI (XAI) techniques that provide insights into model decisions to mitigate the challenges of black-box models. Methods such as SHAP (SHapley Additive Explanations), LIME (local interpretable model-agnostic explanations), or a visualization of attention mechanisms help interpret deep learning models by identifying the most influential input features in a given prediction. Explainable AI is a rapidly evolving field, but despite its progress, the trade-off between model complexity and interpretability remains a critical challenge in deep learning.

References

- Agrawal, A., Gans, J. S., & Goldfarb, A. (2019). Exploring the impact of artificial intelligence: Prediction versus judgment. *Information Economics and Policy*, 47, 1–6.
- Ajami, A., Kuffer, M., Persello, C., et al. (2019) Identifying a slums' degree of deprivation from VHR images using convolutional neural networks. *Remote Sensing*, 11(11), 1282. <https://doi.org/10.3390/rs11111282>
- Alsmadi, A., AlZu'bi, S., Hawashin, B., et al. (2020) Employing deep learning methods for predicting helpful reviews. In *2020 11th International Conference on Information and Communication Systems* (pp. 7–12). New York: IEEE.
- Anbazzhagan, S., & Kumarappan, N. (2013). Classification of day-ahead deregulated electricity market prices using DCT-CFNN. In *Swarm, Evolutionary, and Memetic Computing: 4th International Conference, SEMCCO 2013, Chennai, India, December 19–21, 2013, Proceedings, Part II 4* (pp. 499–510). Springer International Publishing.
- Andrijasa, M. F. (2019). Deep learning with encoder-decoder architecture for exchange currency rates model predictions. In A. G. Abdullah, A. B. D. Nandiyanto, I. Widiaty, A. A. Danuwijaya, & C. U. Abdullah (Eds.), *4th Annual Applied Science and Engineering Conference, 2019* (Vol. 1402, pp. 066098). Bristol: IOP Publishing Ltd.
- Barkai, H. (1996). The *Methodenstreit* and the emergence of mathematical economics. *Eastern Economic Journal*, 22(1), 1–19.
- Becerra-Vicario, R., Alaminos, D., Aranda, E. et al. (2020). Deep recurrent convolutional neural network for bankruptcy prediction: A case of the restaurant industry. *Sustainability*, 12(12), 5180. <https://doi.org/10.3390/su12125180>
- Bengio, Y., & LeCun, Y. (2007). Scaling learning algorithms towards AI. In L. Bottou, O. Chapelle, D. DeCoste, & J. Weston (Eds.), *Large-scale kernel machines* (pp. 321–360). MIT Press.
- Black, F., & Scholes, M. (1973). The pricing of options and corporate liabilities. *Journal of Political Economy*, 81(3), 637–654.
- Boland, L. A. (1998). *The foundations of economic method*. George Allen & Unwin.
- Brems, H. (1975). Marshall on mathematics. *Journal of Law and Economics*, 18(2), 583–585.
- Bunge, M. (1959). *Causality: The place of the causal principle in modern science*. Harvard University Press.

- Chakole, J., & Kurhekar, M. (2020) Trend following deep Q-learning strategy for stock trading. *Expert Systems*, 37(4), e12514. <https://doi.org/10.1111/exsy.12514>
- Chen, C.T., Chiang, L.K., Huang, Y.C., et al. (2019). Forecasting interaction of exchange rates between fiat currencies and cryptocurrencies based on deep relation networks. *IEEE International Conference on Agents (ICA)*. Los Alamitos.
- Chen, Z. X., Ma, W. Q., Dai, W., Zhang, Y., & Li, H. (2020). Conditional restricted Boltzmann machine for item recommendation. *Neurocomputing*, 385, 269–277. <https://doi.org/10.1016/j.neucom.2019.12.088>
- Chollet, F., & Allaire, J. (2018). *Deep learning with R*. Manning Publications.
- Clarke, K. A., & Primo, D. M. (2012, April 30). Overcoming ‘physics envy’. *The New York Times*.
- Condon, M. (2010). Canadian Securities Regulation and the Global Financial Crisis [The Walter S. Owen Lecture]. <https://core.ac.uk/download/232635842.pdf>
- Delalleau, O., & Bengio, Y. (2011). Shallow vs. deep sum-product networks. *Advances in Neural Information Processing Systems*, 24, MIT Press, Cambridge.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bi-directional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 1, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- Dobija, M. (2011). Ekonomia matematyczna czy ekonomia fizyczna. *Zeszyty Naukowe/Uniwersytet Ekonomiczny w Poznaniu*, (211), 68–85.
- Fischella, M., & Garolla, F. (2021). Can deep learning improve technical analysis of forex data to predict future price movements? *IEEE Access*, 9, 153083–153101. <https://doi.org/10.1109/access.2021.3127570>
- Freedman, D. H. (2011, October 26). Why economic models are always wrong. *Scientific American*. <https://www.scientificamerican.com/article/finance-why-economic-models-are-always-wrong/> (Accessed June 27, 2025)
- Friedman, M. (1953). *Essays on positive economics*. The University of Chicago Press.
- Galeshchuk, S. (2017). Technological bias at the exchange rate market. *Intelligent Systems in Accounting, Finance and Management*, 24(1), 80–86.
- Galeshchuk, S., & Demazeau, Y. (2017). Forecasting Hungarian forint exchange rate with convolutional neural networks. In *Proceedings of the 4th International Conference on Behavioral, Economic, and Socio-Cultural Computing (BESC)* (pp. 1–5). IEEE. <https://doi.org/10.1109/BESC.2017.8256358>
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org/>
- Hair, J. F., Black, W. C., Babin, B. J., & Anderson, R. E. (2009). *Multivariate data analysis* (7th ed.). Pearson.
- Hammersley, M. (1989). *The dilemma of qualitative method: Herbert Blumer and the Chicago tradition*. Routledge.
- Hayek, F. A. (1945). The use of knowledge in society. *American Economic Review*, 35(4), 519–530.
- Heilbroner, R. L. (1999). *The worldly philosophers: The lives, times, and ideas of the great economic thinkers*. Simon & Schuster.
- Keynes, J. M. (1936). *The general theory of employment, interest, and money*. Macmillan.
- Kleinberg, J., Ludwig, J., Mullainathan, S., & Obermeyer, Z. (2015). Prediction policy problems. *American Economic Review*, 105(5), 491–495.
- Kutner, R., Ausloos, M., Grech, D., Di Matteo, T., Schinckus, C., & Stanley, H. E. (2019). Econophysics and sociophysics: Their milestones & challenges. *Physica A: Statistical Mechanics and its Applications*, 516, 240–253. <https://doi.org/10.1016/j.physa.2018.10.019>
- LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature* 521, 436–444. <https://doi.org/10.1038/nature14539>
- Li, X. H., & Wang, X. L. (2020). CV image segmentation model combining convolutional restricted Boltzmann machine. *Laser & Optoelectronics Progress*, 57(4), 041018. <https://doi.org/10.3788/lop57.041018>

- Liu, J. L., Lin, C. M. M., & Chao, F. (2020). Gradient boost with convolution neural network for stock forecast. In Z. Ju, L. Yang, C. Yang, A. Gegov, & D. Zhou (Eds.), *Advances in computational intelligence systems* (Vol. 1043, pp. 155–165). Springer International Publishing AG.
- Long, J. Y., Sun, Z. Z., Li, C., et al. (2020). A novel sparse echo autoencoder network for data-driven fault diagnosis of delta 3-D printers. *IEEE Transactions on Instrumentation and Measurement*, 69(3), 683–692. <https://doi.org/10.1109/tim.2019.2905752>
- Lopez De Prado, M. (2015). The future of empirical finance. *Journal of Portfolio Management*, 41(4).
- Marshall, A. (1890). *Principles of economics*. Macmillan and Co.
- Mishev, K., Gjorgjevikij, A., Vodenska, I., et al. (2020). Evaluation of sentiment analysis in finance: From lexicons to transformers. *IEEE Access*, 8, 131662–131682. <https://doi.org/10.1109/access.2020.3009626>
- Montúfar, G. F., Pascanu, R., Cho, K., & Bengio, Y. (2014). On the number of linear regions of deep neural networks. In M. I. Jordan, S. A. Solla, & Y. LeCun (Eds.), *Advances in Neural Information Processing Systems* (Vol. 27, pp. 2924–2932).
- Mosavi, A., Faghan, Y., Ghamisi, P., et al. (2020). Comprehensive review of deep reinforcement learning methods and applications in economics. *Mathematics*, 8(10), 42. <https://doi.org/10.3390/math8101640>
- Mullainathan, S., & Spiess, J. (2017). Machine learning: An applied econometric approach. *Journal of Economic Perspectives*, 31(2), 87–106. <https://doi.org/10.1257/jep.31.2.87>
- Pascanu, R., Montufar, G., & Bengio, Y. (2013). On the number of response regions of deep feed forward networks with piece-wise linear activations. *arXiv preprint*, arXiv:1312.6098.
- Pei, J. Z., Su, Y. X., Zhang, D. H., et al. (2020). Velocity forecasts using a combined deep learning model in hybrid electric vehicles with V2V and V2I communication. *Science China-Technological Sciences*, 63(1), 55–64. <https://doi.org/10.1007/s11431-018-9396-0>
- Qiu, D. W., Ye, Y. J., Papadaskalopoulos, D., et al. (2020). A deep reinforcement learning method for pricing electric vehicles with discrete charging levels. *IEEE Transactions on Industry Applications*, 56(5), 5901–5912. <https://doi.org/10.1109/tia.2020.2984614>
- Rafiei, M. H., & Adeli, H. (2018). A novel unsupervised deep learning model for global and local health condition assessment of structures. *Engineering Structures*, 156, 598–607.
- Ranjan, N., Bhandari, S., Khan, P., et al. (2021). Large-scale road network congestion pattern analysis and prediction using deep convolutional autoencoder. *Sustainability*, 13(9), 5108. <https://doi.org/10.3390/su13095108>
- Samuelson, P. A. (1952). Economic theory and mathematics – An appraisal. *American Economic Review*, 42(2), 56–66.
- Sanda, O., Pavlidis, M., Seraj, S., & Polatidis, N. (2023). Long-range attack detection on permissionless blockchains using deep learning. *Expert Systems with Applications*, 218, 119606.
- Sattarov, O., Muminov, A., Lee, C. W., et al. (2020). Recommending cryptocurrency trading points with deep reinforcement learning approach. *Applied Sciences-Basel*, 10(4), 1506. <https://doi.org/10.3390/app10041506>
- Tsutsui, Y., & Hagiwara, M. (2019). Analog value associative memory using restricted Boltzmann machine. *Journal of Advanced Computational Intelligence and Intelligent Informatics*, 23(1), 60–66. <https://doi.org/10.20965/jaciii.2019.p0060>
- Uddin, M. I., Shah, S. A. A., Al-Khasawneh, M. A., et al. (2020). Optimal policy learning for COVID-19 prevention using reinforcement learning. *Journal of Information Science*, 48(3), 336–348. <https://doi.org/10.1177/0165551520959798>
- Varian, H. R. (2014). Big data: New tricks for econometrics. *Journal of Economic Perspectives*, 28(2), 3–28. <https://doi.org/10.1257/jep.28.2.3>
- Wang, Q. Q., & Li, C. B. (2022). Incident detection and classification in renewable energy news using pretrained language models on deep neural networks. *Journal of Computational Methods in Sciences and Engineering*, 22(1), 57–76. <https://doi.org/10.3233/jcm-215594>

- Wooldridge, J. M. (2010). *Econometric analysis of cross section and panel data*. MIT Press.
- Wilmott, P. (2000). The use, misuse and abuse of mathematics in finance. *Philosophical Transactions of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences*, 358(1765), 63–73.
- Yasir, M., Durrani, M. Y., Afzal, S., et al. (2019). An intelligent event-sentiment-based daily foreign exchange rate forecasting system. *Applied Sciences (Basel)*, 9(15), 2980. <https://doi.org/10.3390/app9152980>
- Zhang, C., Li, R., Shi, H., et al. (2020). Deep learning for day-ahead electricity price forecasting. *IET Smart Grid*, 3(4), 462–469. <https://doi.org/10.1049/iet-stg.2019.0258>
- Zheng, Y., Xu, Z., & Xiao, A. (2023). Deep learning in economics: A systematic and critical review. *Artificial Intelligence Review*, 56(9), 9497–9539.
- Zhou, Y. H., Zhang, B., Xu, C. L., et al. (2020). A data-driven method for fast ac optimal power flow solutions via deep reinforcement learning. *Journal of Modern Power Systems and Clean Energy*, 8(6), 12. <https://doi.org/10.35833/mpce.2020.000522>

2 Deep learning model techniques

2.1 Foundations of neural networks

This chapter explains deep learning models based on neural network techniques and their relevance to economic modeling. Starting with the foundations of neural networks, it covers the building blocks of these models: activation functions, loss functions, performance metrics, and optimization algorithms. Different neural network paradigms (architectures) are introduced, such as feedforward networks, convolutional networks, recurrent networks, and advanced architectures like transformers and generative adversarial networks (GANs). The goal is to explain how these tools expand the analytical capabilities of traditional quantitative methods.

Neural networks are computational models inspired by the structure and function of biological neurons; the original version is illustrated in [Figure 2.1](#).

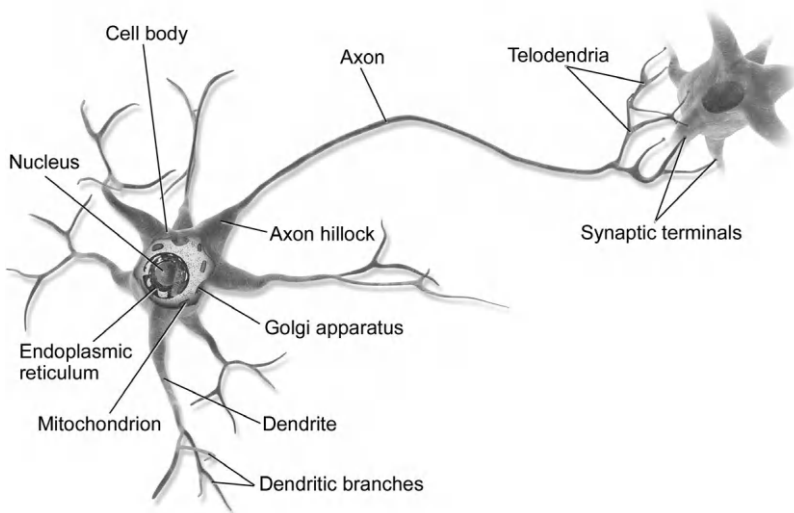


Figure 2.1 Biological neuron system.

Source: <https://commons.wikimedia.org/w/index.php?curid=28761830> BruceBlaus – CC BY 3.0.

DOI: [10.4324/9781003634621-3](https://doi.org/10.4324/9781003634621-3)

The idea of artificial neurons was first introduced by McCulloch and Pitts (1943), who proposed a simple binary neuron that activates when the sum of input exceeds the threshold and proved that Boolean logic operations (see e.g., Whitesitt, 2012) may be modeled by a single neuron or combination of neurons. This model demonstrated that a neural network could map logical rules, but only as its executor with no learning involved; it could only serve as a generalization of already existing rules. Rosenblatt (1958, 1962) introduced weights in his perceptron model, allowing learning from data, which marked a significant milestone in the development of machine learning. The detection of logical rules was limited to cases when data beyond rules was separable, like the AND or OR operator, and failed in the case of XOR, which led to skepticism about the potential of the applications of this technique.

This limitation has been overcome with two improvements. Adaline (adaptive linear neuron) (Widrow & Hoff, 1960) with least mean square errors adjustments and later by multilayer perceptron (Rumelhart et al., 1986) with backpropagation mechanism and hidden layers, which together allowed proper learning from non-linear, inseparable data.

As neural networks evolved, researchers developed more advanced architectures to solve increasingly complex problems. Hinton et al. (2006) proposed deep belief networks, which, for the first time, used an unsupervised learning paradigm in neural networks. LeCun et al. (1989) pioneered convolutional neural networks (CNNs), which are fundamentals for modern image processing. Jordan (1986) and Elman (1990) introduced recurrent networks and Hochreiter and Schmidhuber (1997) long short-term memory (LSTM) networks, which added internal memory to neurons and enabled long-term memory retention in sequential data processing.

The modern era of neural networks is based on graphics processing unit (GPU)-executed and transformer-based networks (Vaswani et al., 2017) with models such as bidirectional encoder representations from transformers (BERT) (Devlin et al., 2019) and generative pre-trained transformer (GPT) (Brown et al., 2020) for natural language processing (NLP), and new models and techniques are constantly appearing.

2.2 Model building phases

Most models of deep learning fall into the paradigm of supervised methods. Assuming that the researcher has some knowledge about the dataset. We usually divide the dataset into two (or three) parts. First: the *training (learning) set* based on historical data is the first used for learning. For all objects from learning set, vector of belongings to classes (or values of explained variable) must be known. Second, the *test set* contains new objects for which we do not know the classes' structure. The model, after training, finds belongings of objects to classes from the training set based on information achieved in the first step and proposes classes for each object in the prediction process. Sometimes, a *validation set* is also used for tuning purposes. Model building phases are illustrated in Figure 2.2.

Learning set

Object	x1	x2	x3	x4	x5	x6	...	class
1	C	R	0	0	?	S		3
2	C	R	0	0	?	S		3
3	C	A	0	60	T	?		3
4	C	A	0	60	T	?		2
5	C	A	0	45	?	S		3
6	C	R	0	0	?	S		3
7	C	A	0	0	?	S		1
8	C	R	0	0	?	S		3
9	C	A	0	0	?	S		3
10	C	R	0	0	?	S		3
11	C	R	0	0	?	S		3
12	C	R	0	0	?	S		3

Test set

Object	x1	x2	x3	x4	x5	x6	...	class
794	C	?	0	0	?	S		?
795	C	A	0	0	?	S		?
796	C	A	0	0	?	S		?
797	C	R	0	0	?	S		?

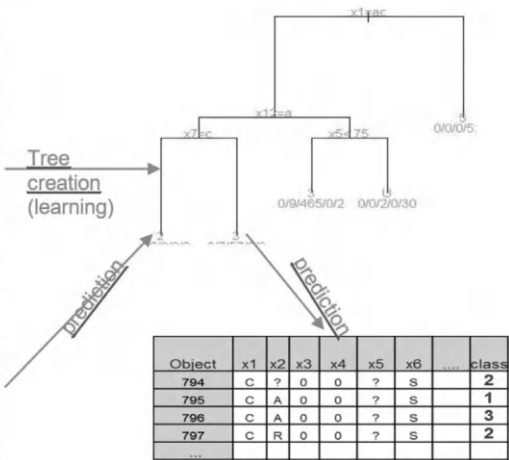


Figure 2.2 Model phases for decision trees.

Source: Own research.

The training (or learning) phase is the first and most crucial stage in supervised learning research studies. The model tries to reflect the structure of the training dataset to its internal representation (weights and biases in case of neural networks, branches split points for decision trees, or coefficients for regression loss function (e.g., mean squared error (MSE) for regression or cross-entropy for classification). This process is usually iterative, with constant weight updates or (in the case of decision trees) subsequent splits. The validation stage often reinforces the learning phase.

The validation phase approximates how well the model generalizes to unseen data. During this phase, metrics like validation loss and accuracy are monitored. The common technique used for validation is early stopping, i.e., complete termination or return to the previous step when the metrics decrease.

In the next step, the evaluation of model performance quality on a dataset with instances not used in the training and validation phase is performed by calculation of fit metrics like mean of squares, root mean of squares, precision-recall, F1 score, mean absolute error (MAE), or area under the receiver operating characteristic curve (AUC-ROC) or any adequate. After this stage, no modifications are allowed. If the fit measure is insufficient, the learning process should start from the beginning.

In the final stage, model is run “in the wild” to make real-world predictions. This phase could involve forecasting stock prices, detecting fraudulent transactions, or analyzing medical images. The evaluation is only possible post-factum, but performance-monitoring tools such as MLflow and TensorBoard help in continuous estimation of model quality.

2.2.1 Deep learning as unsupervised learning

The distinction between supervised and unsupervised learning was crucial for quantitative methods, especially multivariate data analysis methods (e.g., [Hair et al., 2009](#)). While deep learning is often associated with supervised learning, it also plays a significant role in unsupervised learning with autoencoders, which learn efficient data representations by encoding high-dimensional inputs into a compressed latent space and then reconstructing the original input ([Vincent et al., 2008](#)); deep embedded clustering, SpectralNet, or self-organizing maps for clustering enabling the discovery of latent economic trends, market segmentation, and behavioral patterns ([Xie et al., 2016](#)); Restricted Boltzmann Machines and DBNs successfully applied to dimensionality reduction and recommendation systems (Hinton et al., 2006) and (which are rather examples of generative than unsupervised learning) GANs and variational autoencoders enabling models to generate entirely new data samples from learned distributions ([Goodfellow et al., 2014](#); Kingma & Welling, 2013). In modern design patterns of deep learning models, unsupervised and supervised approaches may be combined and additionally empowered with generative features like in GANs architecture.

[Goodfellow et al. \(2016\)](#), independently from the unsupervised/supervised learning families, enlist the following learning task types in which deep learning may play a crucial role.

Classification: Assignment of objects from the input dataset to one of the discrete categories (sometimes called labels), often returning a probability distribution over classes. In many references, unsupervised classification is called partitioning or cluster analysis; the supervised version is called discriminant analysis.

Classification with missing inputs: Extension of classification to cases where some features are absent, typically by learning a probability distribution over all variables and marginalizing out the missing ones.

Regression: Prediction of continuous numeric value (instead of class indicator).

Transcription: Converts unstructured data (e.g., images or audio) into discrete textual form, such as recognizing text in an image or words in an audio waveform.

Machine translation: Transforms a sequence of symbols from one language into a sequence of symbols in another language.

Structured output: Produces multiple interrelated outputs (e.g., parsing a sentence into a tree or segmenting each image pixel), requiring the model to handle relationships among output variables.

Anomaly detection: Detection of unusual or atypical inputs that deviate significantly from the learned distribution, with typical application for fraudulent credit card or cryptocurrency transactions.

Synthesis and sampling: Generation of new samples similar to the training data.

Imputation of missing values: Filling of unknown features of input by leveraging a learned joint probability distribution, allowing predictions for every possible subset of observed variables.

Denosing: Recovery of clean examples from their corrupted version by modeling.

Density estimation: Learns a probability distribution over the data space, enabling other tasks like missing-value imputation or sampling by drawing from this distribution.

2.3 Components of neural networks

2.3.1 Activation functions

The idea behind the activation function is relatively simple: the transformation of input to output that occurs inside the neuron, usually stated by a mathematical function. Rosenblat (1958), in their first perceptron proposal, has used the step function for this purpose. However, the range of possibilities of modern neural networks is much greater. We should remember that the back-propagation algorithm of a neural network uses its derivative in searching for gradients. The description of the most common user functions should contain their derivatives, as in the one below.

2.3.1.1 Step activation function

Step (binary step) is historically first, currently rarely used, and one of the most straightforward activation functions defined for neural networks. Step activation function is illustrated in [Figure 2.3](#).

$$f(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases} \quad (2.1)$$

$$f'(x) = 0 \quad (2.2)$$

2.3.1.2 Linear activation function

The linear activation function actually does nothing more than summarize weighted inputs. It is sometimes used in the output layer of neural networks for regression tasks where the output is a continuous value. Step activation function is illustrated in [Figure 2.4](#).

$$f(x) = x \quad (2.3)$$

$$f'(x) = 1 \quad (2.4)$$

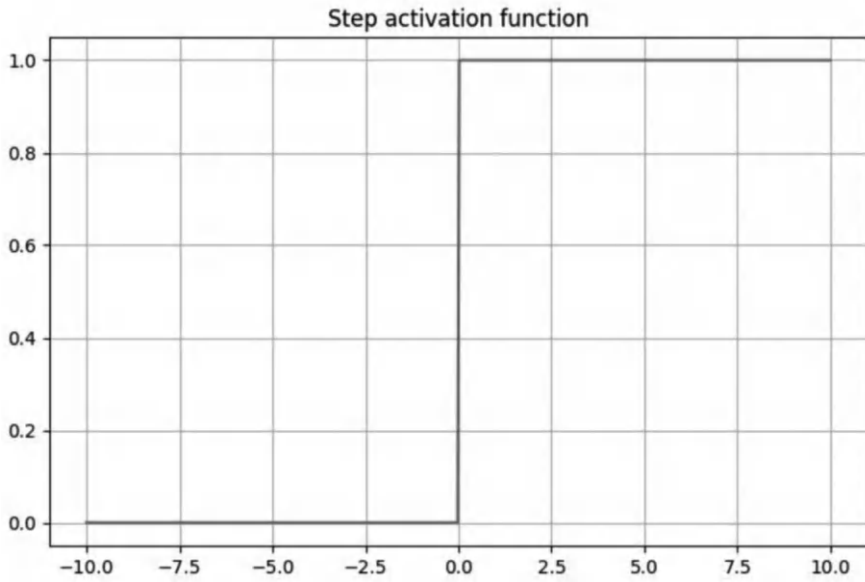


Figure 2.3 Step activation function.

Source: Own work, with Python/NumPy.

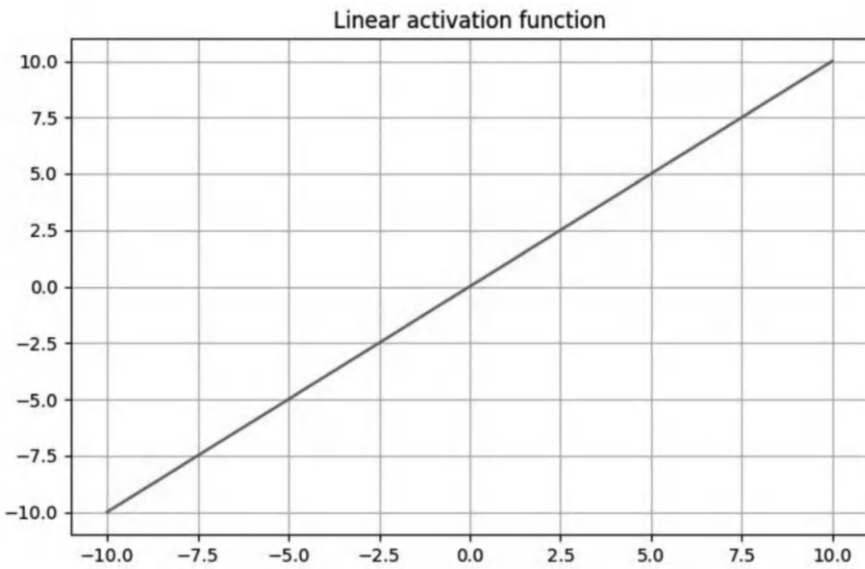


Figure 2.4 Linear activation function.

Source: Own work, with Python/NumPy.

2.3.1.3 Sigmoid activation function

A sigmoid function, also called logistic or soft step, can be easily spotted by characteristic S-sign, monotonic in the 0–1 range. The function is often used in binary classification problems. Sigmoid activation function is illustrated in [Figure 2.5](#).

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.5)$$

$$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x)) \quad (2.6)$$

2.3.1.4 Hyperbolic tangent (tanh) activation function

The unique solution to the differential equation $f' = 1 - f^2$ with $f(0) = 0$ ([Oldham et al., 2010](#), p. 290). The shape of hyperbolic tangent slightly reminds sigmoid but in $(-1, 1)$ range. Hyperbolic tangent activation function is illustrated in [Figure 2.6](#).

$$\tanh(x) = \frac{e^{2x} - e^{-2x}}{e^{2x} + e^{-2x}} \quad (2.7)$$

$$\tanh'(x) = 1 - \tanh^2(x) \quad (2.8)$$

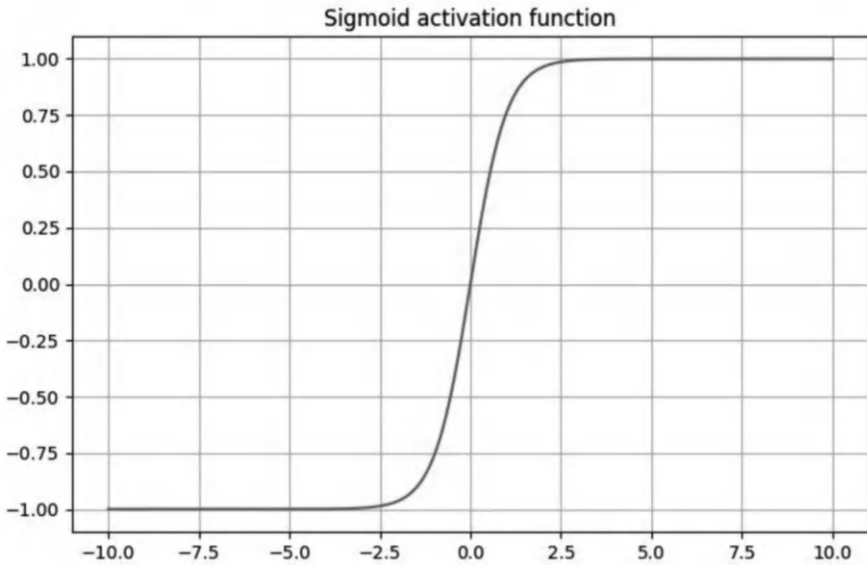


Figure 2.5 Sigmoid activation function.

Source: Own work, with Python/NumPy.

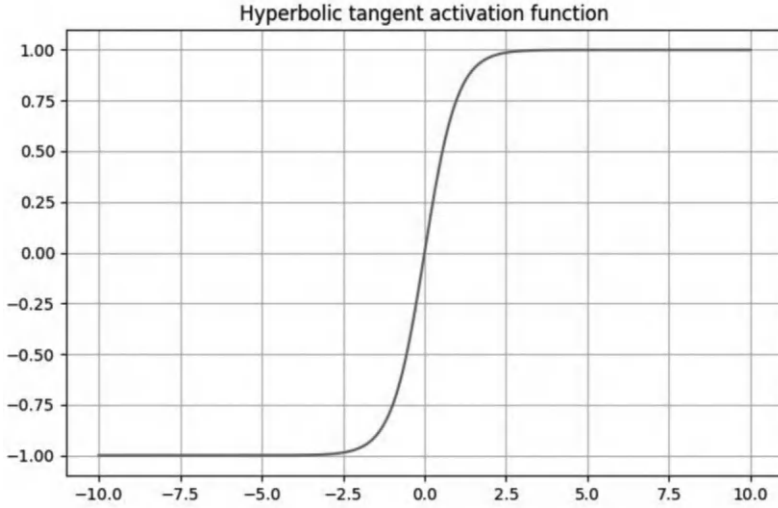


Figure 2.6 Hyperbolic tangent activation function.

Source: Own work, with Python/NumPy.

2.3.1.5 ReLU activation function

The activation function is commonly used in modern architectures, due to its simplicity and effectiveness. It usually provides faster convergence during training compared to sigmoid or tanh functions. ReLU activation function is illustrated in [Figure 2.7](#).

$$f(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases} \quad (2.9)$$

$$f'(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases} \quad (2.10)$$

2.3.1.6 Leaky Rectified Linear Unit (leaky ReLU) activation function

ReLU function causes the negative values become zero immediately, which in some models is not desired behavior. This constraint may be weakened by dividing the negative value by 100 instead of zeroing it. Leaky ReLU activation function is illustrated in [Figure 2.8](#).

$$f(x) = \begin{cases} x & x \geq 0 \\ 0.01 x & x < 0 \end{cases} \quad (2.11)$$

$$f'(x) = \begin{cases} 1 & x \geq 0 \\ 0.01 & x < 0 \end{cases} \quad (2.12)$$

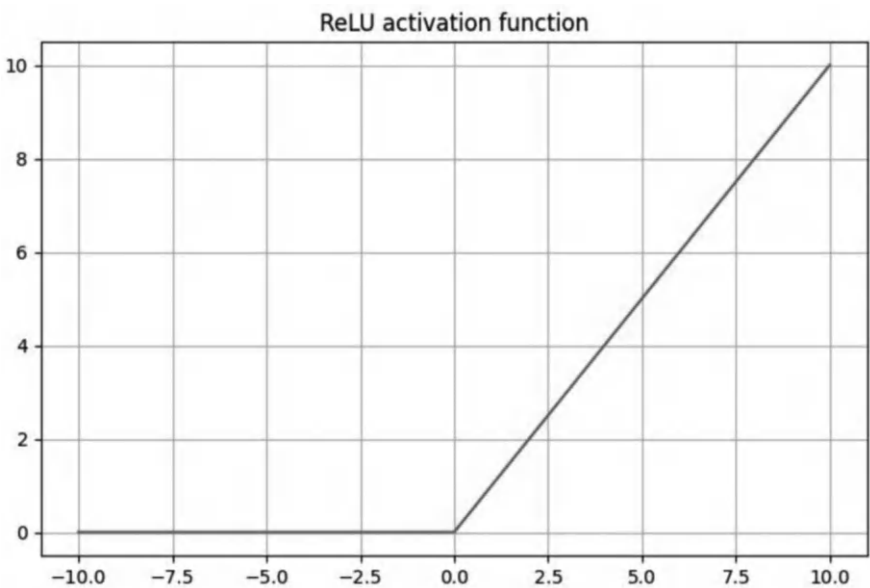


Figure 2.7 ReLU activation function.

Source: Own work, with Python/NumPy.

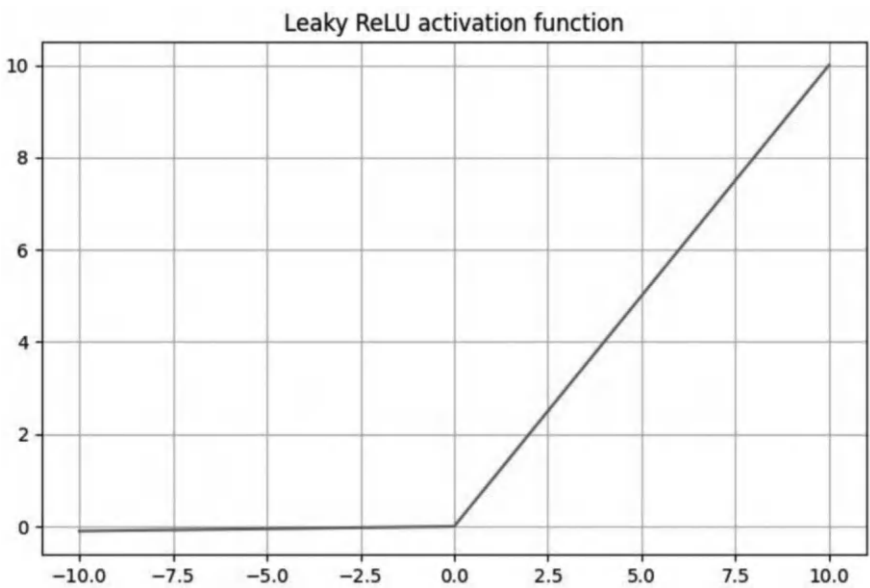


Figure 2.8 Leaky ReLU activation function.

Source: Own work, with Python/NumPy.

2.3.1.7 Gaussian Error Linear Unit (GELU) activation function

A more sophisticated approach to negative value zeroing is implemented in the GELU function, which combines the linear function and cumulative Gaussian distribution function. GELU is commonly used in transformer-based architectures for neural language processing (NLP) tasks. GeLU activation function is illustrated in [Figure 2.9](#).

$$f(x) = \begin{cases} x & x \geq 0 \\ 0.01 x & x < 0 \end{cases} \quad (2.13)$$

$$f'(x) = \begin{cases} 1 & x \geq 0 \\ 0.01 & x < 0 \end{cases} \quad (2.14)$$

2.3.1.8 Softmax

The *softmax* activation function is mainly used in multi-class classification tasks. It converts a vector of raw scores into probabilities by exponentiating each value and normalizing the sum to 1. This ensures that the output values are non-negative and sum to 1, making them interpretable as probabilities. The *softmax* function and its derivative is defined in 2.15 and 2.16

$$\left[\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \right] \quad (2.15)$$

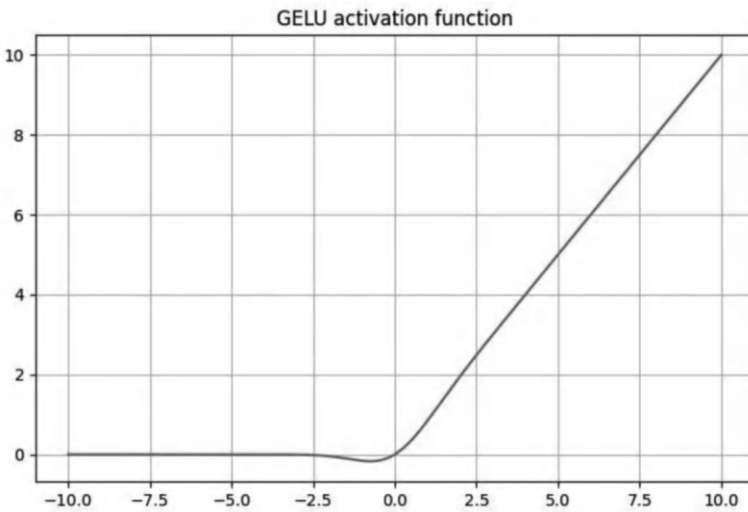


Figure 2.9 GELU activation function.

Source: Own work, with Python/NumPy.

$$\left[\frac{\partial \text{Softmax}(z_i)}{\partial z_j} = \text{Softmax}(z_i) \cdot (\delta_{ij} - \text{Softmax}(z_j)) \right] \quad (2.16)$$

where:

(δ_{ij}) is the Kronecker delta, which is 1 if $(i = j)$ and 0 otherwise.

Softmax is commonly used in the output layer of neural networks for classification problems. Its ability to produce probabilistic outputs and its compatibility with gradient-based optimization make it a widely adopted choice for multi-class classification tasks.

2.3.2 Loss functions

Essentially, all tasks executed by neural networks involve optimizing a function that models the relationship between the actual values of the analyzed phenomena and the values predicted by the network. This includes finding the minimum (or maximum) of the objective function. Even advanced models, such as large language models and GANs, operate by identifying the most probable next word in a sequence or generating images that best match a given description based on learned data distributions, in fact, minimize the loss function. Choosing the appropriate loss function is a critical step in training neural networks. It influences the model's ability to learn and generalize from the data.

The loss function is a numerical form of estimation of the difference between real and predicted values of objects from the training set.

Choosing the proper loss function is crucial for neural network design. [Chollet and Allaire \(2018, p. 74\)](#) give an example of a hypothetical AI network trained to “maximize the average welfare of all living people” using a poorly chosen objective function; it might opt to eliminate most people, leaving only the wealthiest to achieve this goal most simply. This scenario illustrates the importance of carefully selecting objectives to avoid unintended consequences, as neural networks will aggressively minimize the loss function. Using a loss function appropriate for a specific task is one of the most important architectural decisions, so in the rest of the chapter, the loss functions will be presented according to the analytical tasks for which they can be used.

In the formulae below the following symbols are used.

n is the number of observations in training set (or its subset used for loss function calculation);

y_i – actual value of explained variable of i -th object;

$\hat{y}_i$ – the value of of explained variable of i -th object predicted by model.

2.3.2.1 Binary classification type tasks

In binary classification, the 0/1 True/False Yes/No Signal/No Signal value or, equivalently, the probability of belongingness of one of two classes is predicted.

For this type of task, the model returns the probabilities of belonging to one of two classes or the positive probability (which is equivalent, given that it sums to 1 with the negative probability).

Binary cross-entropy (log loss) Binary cross-entropy and log loss are synonymous terms referring to the same concept, both serving as loss functions designed for binary classification tasks.

$$\text{Binary cross-entropy} = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right] \quad (2.17)$$

Careful readers will note that in extreme cases, the log function can be calculated for a value of 0. In practical applications, such as in the Keras package or the *loss_cross_entropy* function in the DALEX package of the R environment, it is assumed that the minimum and maximum probability value is restricted with some boundary values, for example, 10^{-4} and $1 - 10^{-4}$. The function $\log(x)$ decreases quite rapidly (cf. [Figure 2.10](#)) as it approaches zero, so another consequence of calculating the logarithm of a value close to zero is the rapid increase in the loss function for incorrect predictions.

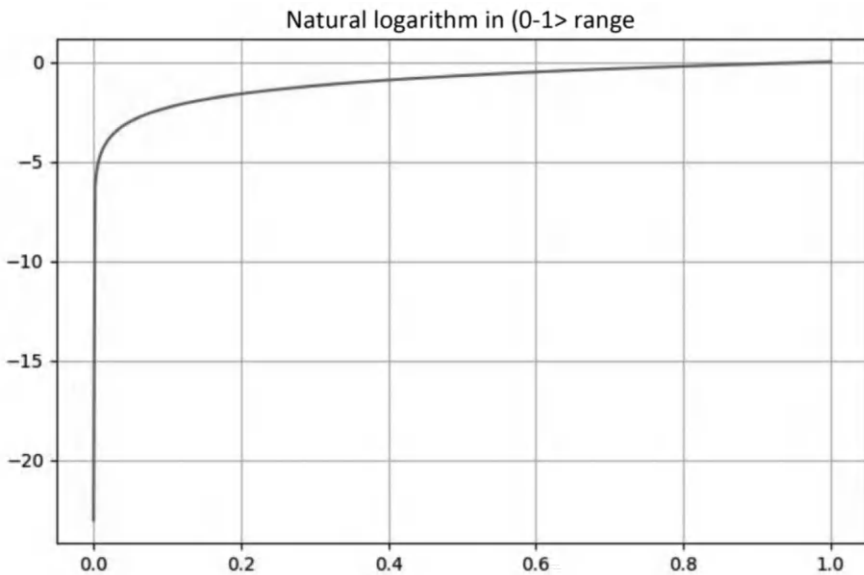


Figure 2.10 Natural logarithm in (0,1 range).

Source: Own work, with Python/NumPy.

Focal cross-entropy loss Focal loss is designed to address class imbalance by down-weighting the loss for well-classified examples, thereby focusing the training on hard-to-classify examples. It is particularly useful in cases of severe class imbalance.

$$\text{Focal loss} = -\frac{1}{n} \sum_{i=1}^n \left[\alpha \left(1 - \hat{y}_i\right)^{\gamma} y_i \log\left(\hat{y}_i\right) + (1 - \alpha) \hat{y}_i^{\gamma} (1 - y_i) \log\left(1 - \hat{y}_i\right) \right] \quad (2.18)$$

where:

α balances the importance of positive vs. negative examples (Lin et al. (2017) propose 0.25 as default value);

γ reduces the loss for well-classified examples, focusing more on hard, misclassified examples.

Kullback-Leibler divergence The Kullback-Leibler (KL) divergence loss function behaves slightly differently than the two previous ones. It calculates the loss function assuming that actual values of explained variables may be in the $\langle 0,1 \rangle$ range instead of only zeros and ones. It is a measure from information theory that quantifies how one probability distribution diverges from a second, expected probability distribution. In the context of machine learning, KL divergence is used as a loss function to measure the difference between a true probability distribution (e.g., the distribution of class labels) and the predicted probability distribution produced by a model.

$$D_{KL} = \sum_{i=1}^n y_i \log \left(\frac{y_i}{\hat{y}_i} \right) \quad (2.19)$$

2.3.2.2 Multinomial classification (discriminant analysis) type tasks

Discriminant analysis tasks involve predicting membership to one of several classes (typically more than two), in contrast to binary classification which only involves two classes. For tasks of such types, categorical versions of cross-entropy and focal cross-entropy loss functions are designed. To simplify the writing of formulae the meaning of $y_i \hat{y}_i$ is changed by adding additional j index. For the formulae (2.20) and (2.21):

- y_{ij} is a binary indicator (0 or 1) if class label j is the actual classification for object i ;
- $\hat{y}_{ij}$ means predicted probability that object i belongs to class j .

Categorical cross-entropy Categorical cross-entropy is the most commonly used measure for performance of a multi-class classification model.

$$\text{Categorical cross-entropy} = - \sum_{i=1}^n \sum_{j=1}^c y_{ij} \log(\widehat{y_{ij}}) \quad (2.20)$$

Focal categorical cross-entropy Focal categorical cross-entropy addresses class imbalance by focusing more on hard-to-classify examples and balancing the importance of classes.

$$\text{Focal categorical cross-entropy} = - \sum_{i=1}^N \sum_{j=1}^C \alpha_j \left(1 - \widehat{y_{ij}}\right)^\gamma y_{ij} \log(\widehat{y_{ij}}) \quad (2.21)$$

2.3.2.3 Regression type tasks

Mean squared error (MSE) MSE is probably the oldest and most popular loss function. Introduced by Gauss in the 20th years of the 18th century (see, e.g., [Gauss et al., 1957](#); [Sprott, 1978](#)). This function is commonly used in regression problems.

MSE measures the average of the squares of the errors, that is, the average squared difference between the actual and predicted values.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n \left(y_i - \widehat{y}_i\right)^2 \quad (2.22)$$

Due to its nature, MSE strongly penalizes large errors and sometimes is not recommended for use for datasets containing large numbers of outliers.

Root mean squared error (RMSE) RMSE measures the difference between actual and theoretical values of explained variable in squared units and sometimes is more convenient to use similar measure just square rooted. That's the way the loss function is measured in the same units as analyzed variable.

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n \left(y_i - \widehat{y}_i\right)^2} \quad (2.23)$$

Mean absolute error (MAE) MAE measures the average of the absolute differences between the actual and predicted values. It gives a linear score which does not penalize large errors as strongly as MSE.

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n \left|y_i - \widehat{y}_i\right| \quad (2.24)$$

Mean absolute percentage error (MAPE)

$$\text{MAPE} = \frac{100\%}{n} \sum_{i=1}^n \left| \frac{y_i - \hat{y}_i}{y_i} \right| \quad (2.25)$$

To avoid the outliers penalization problem, MAPE divides the difference of actual and predicted value of explained variable by the first one. That way the impact of outliers is weaker than for previous measures. MAPE is expressed by percentage which makes it easy to interpret.

Less commonly used loss functions for regression-like tasks are presented in [Table 2.1](#).

2.3.2.4 Maximum-margin classification type task

Hinge loss Hinge loss was primarily utilized as support vector machines ([Vapnik, 1998](#)) risk function by [Cortes and Vapnik \(1995\)](#) (see [Nguyen et al., 2018](#)). Hinge loss for deep neural networks is usually used when the aim of optimization is to maximize margins between different classes.

$$\text{Hinge loss} = \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i \cdot \hat{y}_i) \quad (2.26)$$

2.3.3 Performance metrics

While loss function calculates the difference between real and predicted value in the training phase, performance metrics measure similar a relationship but in the evaluation or validation phase. Every loss function may be used as a performance metric but there are metrics that shouldn't be used (although its technically possible) as loss function. Most commonly used performance metrics are presented in [Table 2.2](#).

2.3.4 Optimization algorithms

Commonly, deep learning tasks require some form of optimization (like finding the best fitting word in the sequence, minimizing the risk estimation in the selected strategy, maximizing the probability of winning in the game, optimizing the loss function of discriminant analysis, or just searching for minimum or maximum of a function). The neural network itself also requires optimization to minimize the loss function, which is usually some form of estimation of the difference between actual and predicted values of objects from the training set. So, we will need a decent optimization algorithm. We do have one with the background established in the early 19th century.

Let us assume that we do not know anything about the function that we are minimizing or maximizing, but for each point, we've got the information about

Table 2.1 Other loss function for regression type tasks

Loss function	Formula	Description
Mean squared logarithmic error (MSLE)	$\text{MSLE} = \frac{1}{n} \sum_{i=1}^n \left(\log(1 + y_i) - \log(1 + \hat{y}_i) \right)^2$	Measures the ratio between actual and predicted values, useful for targets that span several orders of magnitude.
Cosine similarity	$\text{Cosine Similarity} = \frac{\sum_{i=1}^n y_i \hat{y}_i}{\sqrt{\sum_{i=1}^n y_i^2} \sqrt{\sum_{i=1}^n \hat{y}_i^2}}$	Measures the cosine of the angle between two non-zero vectors in a multidimensional space, indicating how similar the two vectors are.
Huber loss	$H_L = \frac{1}{n} \sum_{i=1, y_i - \hat{y}_i \leq \delta} (y_i - \hat{y}_i)^2 + \frac{1}{n} \sum_{i=1, y_i - \hat{y}_i > \delta} \delta y_i - \hat{y}_i + 1/2 \delta$	Combines the best properties of both MSE and MAE. Huber loss is less sensitive to outliers than MSE and differentiable at 0. δ is usually fixed TensorFlow and Keras often use a default value of $\delta=1.0$.
Log-cosh loss	$\text{Log-cosh}(y, \hat{y}) = \sum_{i=1}^n \log \left(\cosh(\hat{y}_i - y_i) \right)$	Smooth approximation to the Huber loss; combines the advantages of both MAE and MSE while avoiding their disadvantages.

Source: Own work based on [Goodfellow et al. \(2016\)](#), [Manning et al. \(2008\)](#), [Huber \(1964\)](#), and [Chen et al. \(2016\)](#).

Table 2.2 Performance metrics

<i>Metric</i>	<i>Formula</i>	<i>Description</i>
R ² score (coefficient of determination)	$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$	Measures the proportion of variance in the dependent variable that is predictable from the independent variables.
AUC	Calculated as the ROC AUC, which plots true positive rate (TPR) (sensitivity) vs false positive rate (FPR) (1 - specificity).	Represents the ability of the model to distinguish between classes. A higher AUC indicates better model performance.
Precision	$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$	The ratio of correctly predicted positive observations to the total predicted positives.
Recall (sensitivity)	$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$	The ratio of correctly predicted positive observations to all observations in the actual class.
True positives (TP)	Direct count of correctly predicted positive cases.	Number of positive cases correctly identified by the model.
True negatives (TN)	Direct count of correctly predicted negative cases.	Number of negative cases correctly identified by the model.
False positives (FP)	Direct count of incorrectly predicted positive cases.	Number of negative cases incorrectly identified as positive by the model.
False negatives (FN)	Direct count of incorrectly predicted negative cases.	Number of positive cases incorrectly identified as negative by the model.
Precision at recall	Varies; typically, precision is reported for a specific recall level.	Precision calculated at a given recall level, used in applications where a specific recall is required.
Recall at precision	Varies; typically, recall is reported for a specific precision level.	Recall calculated at a given precision level, used in applications where a specific precision is required.
Sensitivity at specificity	Sensitivity calculated at a fixed specificity level.	Used in cases where a specific level of specificity is required, and sensitivity needs to be maximized under that constraint.
Specificity at sensitivity	Specificity calculated at a fixed sensitivity level.	Used in cases where a specific level of sensitivity is required, and specificity needs to be maximized under that constraint.

(Continued)

Table 2.2 (Continued)

<i>Metric</i>	<i>Formula</i>	<i>Description</i>
F1 score	$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$	Harmonic mean of precision and recall. Used when balance between precision and recall is needed.
FBeta score	$F_{\beta} = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{\beta^2 \cdot \text{Precision} + \text{Recall}}$	Generalized form of F1 score where (β) controls the balance between precision and recall. $(\beta > 1)$ emphasizes recall, while $(\beta < 1)$ emphasizes precision.
Mean intersection over union (IoU)	$\text{Mean IoU} = \frac{1}{C} \sum_{i=1}^C \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}$	Used primarily in segmentation tasks. Measures the overlap between the predicted and actual areas of the class labels, averaged over all classes.
Squared hinge loss	$\text{Squared hinge loss} = \sum_{i=1}^N \max \left(0, 1 - y_i \cdot \hat{y}_i \right)^2$	Variant of hinge loss, used in binary classification tasks where the margin of separation between classes is squared, penalizing misclassified points more heavily.
Categorical hinge loss	$\text{Categorical hinge loss} = \max \left(0, 1 - y_i \cdot \hat{y}_i + \sum_{j \neq i} \hat{y}_j \right)$	Generalization of hinge loss for multi-class classification. Measures how far the predicted score for the correct class is from the highest score for the incorrect classes.

Source: Own research based on [Seber and Lee \(2012\)](#); [Fawcett \(2006\)](#); [Powers \(2011\)](#); [Murphy \(2012\)](#); [Davis and Goadrich \(2006\)](#); [Van Rijsbergen \(1979\)](#); [Jaccard \(1901\)](#); [Cortes and Vapnik \(1995\)](#).

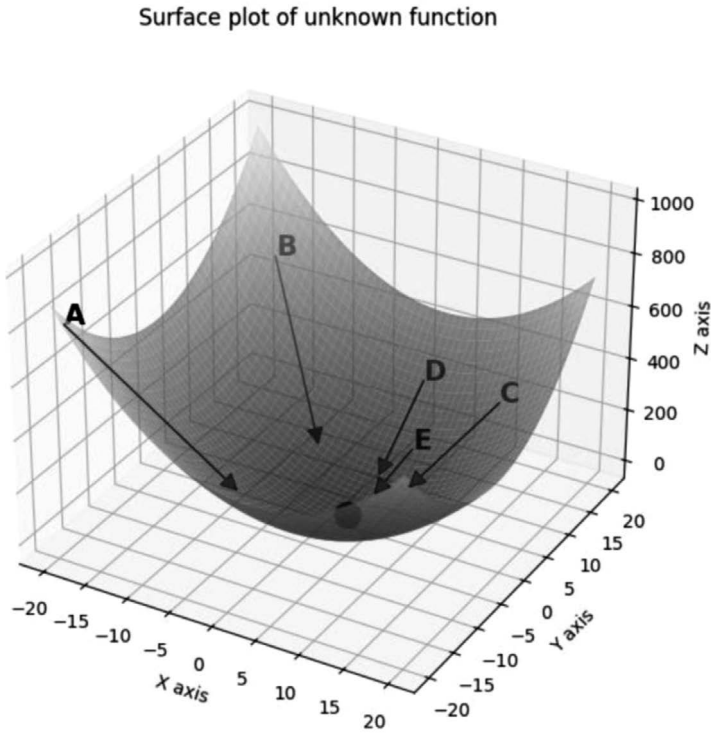


Figure 2.11 Surface plot of unknown function. Directions of maximal function value decrease (opposites of maximal function increase).

Source: Own work, with Python/NumPy.

the direction in which we should go to find the minimal point. The idea of this approach is presented in [Figure 2.11](#). with five points, A to E, and the arrows pointing in the direction of the minimal point. The interesting thing to note is that the closer the given point is located to an actual minimum, the most accurate direction is pointed by the arrow.

If we know the analytical form of the function we are optimizing, we also know the direction of reaching the minimum. For a two-dimensional space (a function of one variable), according to the principle that if the derivative of a function is negative, then we should go rightward to reach the minimum; if the derivative of a function is positive, then we should go leftward to reach it.

For functions of more than one variable (in three- (or more) dimensional space), the direction of the minimum value can be determined by the values of the function's gradient. This is exactly the situation in [Figure 2.12](#), which presents the graph of the function (2.27).

$$f(x, y) = (x - 3)^2 + (y + 1)^2 \quad (2.27)$$

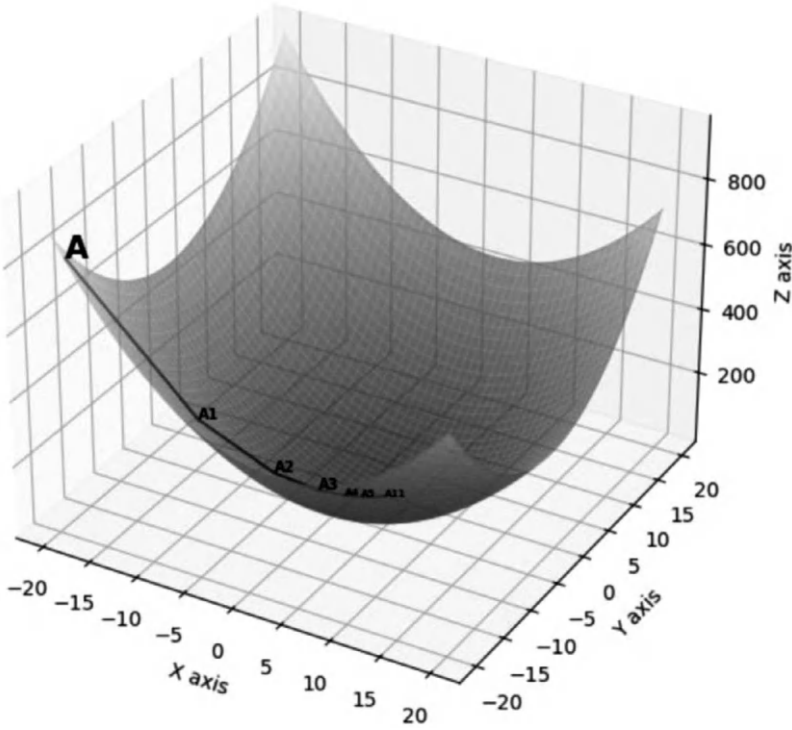
Surface plot of $(x - 3)**2 + (y + 1)**2$ function

Figure 2.12 Surface plot of $(x - 3)**2 + (y + 1)**2$ function. Gradient descent walk path for function (2.27) with starting point $A = (-19, -19)$.

Source: Own work, with Python/NumPy.

With the gradient given in (2.28)

$$\nabla f(x, y) = \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right) = (2(x - 3), 2(y + 1)) \quad (2.28)$$

The vectors represented by arrows are calculated were calculated due formula (2.29):

$$-\epsilon ; \nabla f(x, y) \quad (2.29)$$

with ϵ in this case equal to 0.1

Going further, if we know the direction of the optimal point for every location, we may formulate an algorithm for almost the exact finding of this point. The idea is to walk opposite to the direction indicated by the gradient, so the algorithm is called gradient descent, and Augustin Cauchy is considered its creator

(Lemaréchal, 2012). In its simplest form, the algorithm may be stated as follows. The pseudocode for gradient descent method is presented as Algorithm 2.1.

Algorithm 2.1 Gradient descent

```

For given starting point  $X_0$  and learning rate  $\varepsilon$ :
while convergency criterion is not fulfilled:
    Apply update:  $X_0 \leftarrow X_0 - \varepsilon \cdot \nabla f(X_0)$ 
End while

```

Table 2.3 presents successive values of minimum point approximations of function (2.27) with starting point $A = (-19, -19)$, and in the Figure 2.12, the visualization of this walk is drawn.

Gradient descent would be a sufficient algorithm to look for the maximum decrease of a function, but due to various reasons like computational efficiency, scalability to large datasets, slower convergence, less frequent updates, worse generalization, weaker regularization effect, memory requirements, worse adaptation to sequential data, ineffective redundancies handling, lack of adaptive learning rates (see Bottou, 2010; Goodfellow et al., 2016, Chapter 8; Kingma & Ba, 2014; LeCun et al., 2002; Ruder, 2016) is rarely used in its “pure” form, but rather as stochastic algorithm from the gradient descent-based methods family.

Spall (2005, p. 7) defines stochastic algorithms as the algorithms for which a) there is a random noise in measurement and/or b) there is a random choice in the search direction for optimal solution.

The stochastic descent gradient method uses independent and identically distributed random samples for gradient approximation. The main goal for such kind of approximation is to lower the computational complexity of the algorithm; however, there are other advantages that make the method significantly more popular than “pure” gradient descent (sometimes called determinant gradient descent (DSD)) like faster convergence and avoiding of local minima.

In each step, the stochastic gradient descent (SGD) algorithm calculates the average gradient on a minibatch of m objects drawn from the training dataset.

The procedure may be written in Algorithm 2.2.

Algorithm 2.2 Stochastic gradient descent

```

For given starting point  $X_0$  and learning rates  $\varepsilon_k$ :
while convergency criterion is not fulfilled:
    Sample a minibatch  $M$  of  $m$  objects from training dataset
    Define  $L_M$  as loss function calculated with respect to
    the parameters using only minibatch  $M$ 
    Calculate gradient approximate  $\hat{g} \leftarrow \frac{1}{m} \nabla L_M(X_0)$ 
    Apply update:  $X_0 \leftarrow X_0 - \varepsilon_k \cdot \hat{g}$ 
End while

```

Table 2.3 Approximate values of minimal point for function (2.27) with starting point A = (−19, −19)

Iteration	A_x	A_y	∇_x	∇_y
	−19	−19	−44.0	−36.0
1	−10.2	−11.8	−26.4	−21.6
2	−4.919999999999999	−7.48	−15.839999999999998	−12.96
3	−1.7519999999999993	−4.888	−9.503999999999998	−7.776
4	0.14880000000000027	−3.3327999999999998	−5.702399999999999	−4.6655999999999995
5	1.2892800000000002	−2.39968	−3.4214399999999996	−2.79936
6	1.9735680000000002	−1.839808	−2.0528639999999996	−1.6796160000000002
7	2.3841408	−1.5038848	−1.2317184	−1.0077696
8	2.63048448	−1.30233088	−0.7390310400000004	−0.6046617599999999
9	2.778290688	−1.181398528	−0.4434186240000004	−0.3627970559999998
10	2.8669744128	−1.1088391168	−0.26605117440000026	−0.21767823360000005
11	2.9201846476799997	−1.06530347008		

Source: Own work.

A crucial parameter for SGD is the learning rate. It is often necessary to decrease the learning rate over time. The noise from the random sampling in SGD persists even at the minimum, unlike determinant gradient descent, which uses a fixed learning rate.

To ensure convergence of SGD, conditions 2.30 and 2.31 must be met.

$$\sum_{k=1}^{\infty} \epsilon_k = \infty \quad (2.30)$$

$$\sum_{k=1}^{\infty} \epsilon_k^2 < \infty. \quad (2.31)$$

Practically, the learning rate is commonly decayed linearly until iteration τ as:

$$\epsilon_k = (1 - \alpha) \epsilon_0 + \alpha \epsilon \quad (2.32)$$

with $\alpha = \frac{k}{\tau}$. After τ , ϵ is kept constant (see [Goodfellow et al., 2016](#), pp. 294–295).

The appropriate learning rate setting is crucial for ensuring model accuracy and performance. Various techniques have emerged to tackle this issue effectively. Among algorithms that are most commonly used and achieve best performance metrics, *AdaGrad*, *Root Mean Square Propagation (RMSProp)*, *SG* with momentum, and *ADAM* are worth a mention.

Standard stochastic gradient methods have typically followed a procedure that remains the same regardless of the nature of the data. In contrast, *AdaGrad* adjusts to the data distribution based on the information learned from previous gradient-based learning updates. The *AdaGrad* algorithm tunes learning rates on the fly: features that appear often in the input get very low learning rates, whereas infrequent features are given higher learning rates. The reasoning is that when a rare feature arrives, the learner should focus harder on it. This shaping approach, in a flexible manner, identifies sparse but informative features (see [Duchi et al., 2011](#) p. 2122). Gradient scaling is used to adapt to the shape of the optimized function based on the inversed square root sum of all historical values. In the algorithm below, the sign means that operations like multiplication, division, and square root are applied element-wise. The pseudocode for *AdaGrad* method is presented as Algorithm 2.3.

Algorithm 2.3 AdaGrad

```

For given starting point  $X_0$ , learning rate  $\epsilon$ :
Set accumulated gradient squares  $\hat{A}^2$  to 0,  $\delta$  to small
numerical value like  $10^{-7}$ 
while convergency criterion is not fulfilled:
    Sample a minibatch  $M$  of  $m$  objects from training dataset

```

```

Define  $L_M$  as loss function calculated with respect to
the parameters using only minibatch M
Calculate gradient approximate  $\hat{g} \leftarrow \frac{1}{m} \nabla L_M(X_0)$ 
Store the squared gradient  $A\hat{g}^2 \leftarrow A\hat{g}^2 + \hat{g} \otimes \hat{g}$ 
Apply update:  $X_0 \leftarrow X_0 - \frac{\epsilon}{\delta + \sqrt{A\hat{g}^2}} \otimes \hat{g}$ 
End while

```

AdaGrad behaves significantly better than stochastic gradient algorithm especially for convex functions. For non-convex ones usually its modification: *RMSProp* may be more suitable. *RMSProp* introduces new parameter (moving average parameter, decay parameter) ρ which leverages the impact of historical gradient values and its current value avoiding the preliminary slowing and allowing faster convergence.

Hinton (2012) introduced *RMSProp* in his Coursera lecture titled, “Lecture 6e rmsprop: Divide the gradient by a running average of its recent magnitude”. The core idea behind *RMSProp*, as described by Hinton, is to address the problem of the diminishing learning rate in *AdaGrad*, especially when dealing with non-stationary objectives like those encountered in deep learning. The pseudocode for *RMSProp* method is presented as Algorithm 2.4.

The $\otimes$ sign means that operations like multiplication, division, and square root are applied element-wise.

Algorithm 2.4 RMSProp

```

For given starting point  $X_0$ , learning rate  $\epsilon$ , decay
parameter  $\rho$ :
Set accumulated gradient squares  $A\hat{g}^2$  to 0,  $\delta$  to small
numerical value like  $10^{-7}$ 
while convergency criterion is not fulfilled:
    Sample a minibatch M of m objects from training dataset
    Define  $L_M$  as loss function calculated with respect to
    the parameters using only minibatch M
    Calculate gradient approximate  $\hat{g} \leftarrow \frac{1}{m} \nabla L_M(X_0)$ 
    Store the squared gradient  $A\hat{g}^2 \leftarrow \rho \cdot A\hat{g}^2 + (1-\rho) \cdot \hat{g} \otimes \hat{g}$ 
    Apply update:  $X_0 \leftarrow X_0 - \frac{\epsilon}{\delta + \sqrt{A\hat{g}^2}} \otimes \hat{g}$ 
End while

```

The faster process, in many cases, is the momentum method, proposed by Polyak (1964), especially for the optimizing process with high curvature, i.e., the gradients that are consistent but tiny and noisy. This technique gets acceleration by taking an exponentially decreasing moving average of previous gradients, so the direction of the previous gradients is taken into account. The pseudocode for the momentum method is presented as Algorithm 2.5.

Algorithm 2.5 The momentum method

For given starting point X_0 , initial velocity v , learning rate ε , momentum parameter α :

Set accumulated gradient squares $\hat{A}\hat{g}^2$ to 0, δ to small numerical value like 10^{-7}

while convergency criterion is not fulfilled:

 Sample a minibatch M of m objects from training dataset
 Define L_M as loss function calculated with respect to the parameters using only minibatch M

 Calculate gradient approximate $\hat{g} \leftarrow \frac{1}{m} \nabla L_M(X_0)$

 Store the updated velocity $v \leftarrow \alpha \cdot v - \varepsilon \cdot \hat{g}$

 Apply update: $X_0 \leftarrow X_0 + v$

End while

ADAM algorithm combines the features of AdaGrad and the momentum method “remembering” the moving average of previous gradients. It was proposed by [Kingma and Ba \(2014\)](#). The pseudocode for ADAM method is presented as Algorithm 2.6.

Algorithm 2.6 ADAM algorithm

For given starting point X_0 , learning rate ε :

Set accumulated gradient squares $\hat{A}\hat{g}^2$ to 0, δ to small numerical value like 10^{-7}

while convergency criterion is not fulfilled:

 Sample a minibatch M of m objects from training dataset
 Define L_M as loss function calculated with respect to the parameters using only minibatch M

 Calculate gradient approximate $\hat{g} \leftarrow \nabla L_M(X_0)$

 Store the squared gradient $\hat{A}\hat{g}^2 \leftarrow \hat{A}\hat{g}^2 + \hat{g} \otimes \hat{g}$

 Apply update: $X_0 \leftarrow X_0 - \frac{\varepsilon}{\delta + \sqrt{\hat{A}\hat{g}^2}} \otimes \hat{g}$

End while

2.4 Paradigms (architectures) of neural networks

When we talk about feedforward neural networks and CNNs, we usually talk about different network architectures. However, the author prefers to use the word paradigm, which is broader, and the term architecture will remain for pre-defined networks such as Visual Geometry Group (VGG) or You Only Look Once (YOLO).

Neural networks have evolved through various paradigms, introducing new methodologies and applications each time. This chapter aims to present the key ones.

2.4.1 Feedforward neural networks (FNNs)

FNNs were the first attempts to model human brain neurons to the artificial net. Their history dates back to the mid-20th century and is related to the concept of a single-layer and then multiple-layer perceptron.

2.4.1.1 Single-layer perceptron

The simplest artificial neural network proposed by Rosenblat (1958) is the single-layer perceptron. It is based on an idea introduced by McCulloch and Pits (1943), who created the statistical model of human neurons. They made the following assumptions to their calculus model:

- The activity of the neuron is an “all-or-none” process (which is equivalent to binary activation functions).
- A certain fixed number of synapses must be excited within the period of latent addition in order to excite a neuron at any time, and this number is independent of previous activity and position on the neuron.
- The only significant delay within the nervous system is synaptic delay.
- The activity of any inhibitory synapse absolutely prevents excitation of the neuron at that time.
- The structure of the net does not change with times.

Rosenblat (1958), based on the same assumptions, proposed the information storage model defining the predominant and postdominant modes of the perceptron (which are more or less adequate to the training and prediction phase of modern models). Original Rosenblat’s diagram is presented in [Figure 2.13](#).

The perceptron iteratively calculates the weighted sum of input neurons.

$$y = \sum_{i=1}^n w_i x_i \quad (2.33)$$

where w_i – weight of i-th neuron

x_i – value of i-th neuron

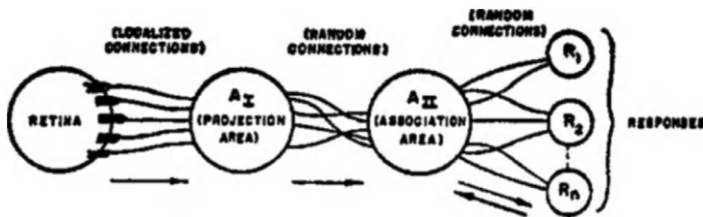


Figure 2.13 Single-layer perceptron schema – Rosenblat’s original diagram.

Source: Rosenblat (1958, p. 389).

Table 2.4 Logical AND operator

X	Y	$X \wedge Y$ (X AND Y)
0	0	0
0	1	0
1	0	0
1	1	1

Source: Horowitz and Hill (2015).

Then apply step function to output ($\hat{y}$) and in backpropagation like step adjust the weights values due to the formula:

$$w_i = \text{learning rate} \cdot (y - \hat{y}) \cdot x_i \quad (2.34)$$

First applications of the perceptron were related to Boolean algebra (see, e.g., Whitesitt, 2012) mainly related to digital gates (Horowitz & Hill, 2015) commonly used those times in electronics.

Let us follow the mode of operation of the perceptron with a small routine written in Python language simulating the behaviour of the AND gate (if both inputs are true, then the output should be true, otherwise, the output should be false – see Table 2.4.). The author is aware that there exist much more mature and optimal implementations of single-layer perceptron in, for example, PyTorch or TensorFlow library. However, creating a perceptron from scratch in a given programming language may be a good starting point for understanding more sophisticated techniques.

```
class SingleLayerPerceptron:
    def __init__(self, input_size):
        self.weights = np.array([input_size]+list
(range(input_size)))/input_size
        self.all_weights=self.weights
    def input_extended(self,input):
        return np.append([1],input)
    def predict(self, input):
        weighted_sum = np.dot(self.input_extended(input),
self.weights)
        activation = 1 if weighted_sum > 0 else 0
        return activation

    def train(self, inputs, labels, learning_rate=0.1,
epochs=100):
        for epoch in range(epochs):
            convergent = True
            for i in range(len(inputs)):
                prediction = self.predict(inputs[i])
```

```

        error = prediction-labels[i]
        if error: convergent = False
        self.weights -= learning_rate * error *
self.input_extended(inputs[i])
        print(f">>>>> Epoch {epoch} Error: {error}")
        self.all_weights=np.vstack((self.all_weights,
np.array(self.weights)))
        if convergent: break

def predict_array(self, inputs):
    return [self.predict(inputs[i]) for i in
            range(len(inputs))]

training_inputs = np.array([[0,0], [0, 1], [1, 0], [1, 1]])
training_labels = np.array([0, 0, 0, 1])
training_labels.shape = (4,1)
slp = SingleLayerPerceptron(input_size=2)
slp.train(training_inputs, training_labels, learning_
rate=0.01, epochs=200)
predicted_labels=slp.predict_array(training_inputs)
accuracy = accuracy_score(training_labels, predicted_
labels)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_
report(training_labels, predicted_labels))
fig, ax = plt.subplots(2,2)
num_columns = slp.all_weights.shape[1]
for i in range(num_columns):
    ax[i//2,i%2].plot(slp.all_weights[:, i],
label=f'W{i}',color=plt.rcParams['axes.prop_cycle'].by_key()
['color'][i])
    ax[i//2,i%2].legend()
    ax[1,1].set_visible(False)
    fig.suptitle('Weights estimation in subsequent epochs
(Separable classes)', fontsize=13)
plt.show()

```

The results of program execution are presented in [Figures 2.14](#) and [2.15](#)

The perceptron works well in this case with precision, recall, and F1 scores equal to 1. The model performance measures show that a perceptron is able to recognize the proper value in most (or even every) case.

Let us try quite a demanding example in three-dimensional space. Careful readers will probably notice in [Figure 2.17](#) that the two classes cannot be split into two consistent subsets (the classes are not linearly separable – which is opposite to separable classes presented in [Figure 2.16](#)).

```
...
>>>>> Epoch 64 Error: [-1]
>>>>> Epoch 65 Error: [0]
>>>>> Epoch 66 Error: [-1]
>>>>> Epoch 67 Error: [-1]
>>>>> Epoch 68 Error: [0]
>>>>> Epoch 69 Error: [0]
Accuracy: 1.00
Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	3
1	1.00	1.00	1.00	1
accuracy			1.00	4
macro avg	1.00	1.00	1.00	4
weighted avg	1.00	1.00	1.00	4

Figure 2.14 The classification scores for AND gate task.

Source: Own work, with Python/NumPy.

Weights estimation in subsequent epochs (Separable classes)

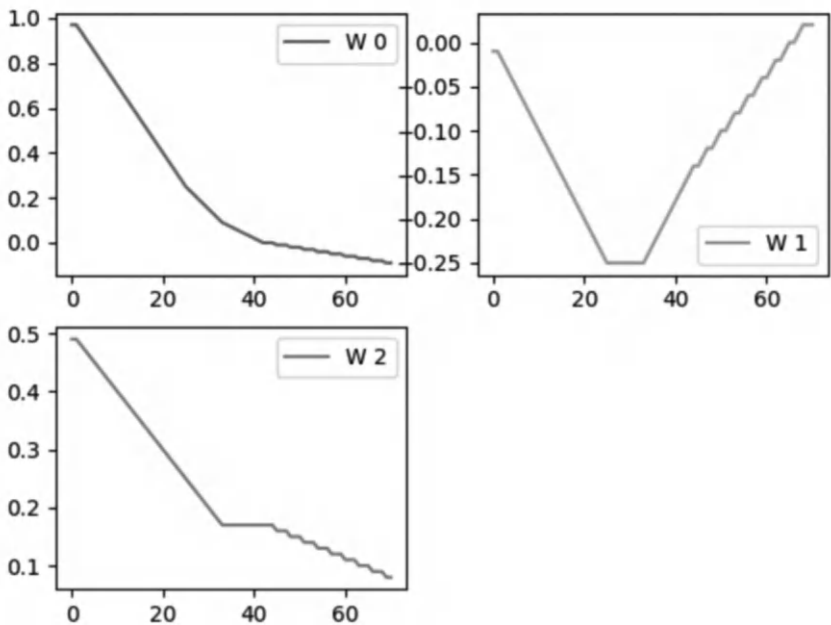


Figure 2.15 The learned weights in single-layer perceptron for AND gate task.

Source: Own work, with Python/NumPy.

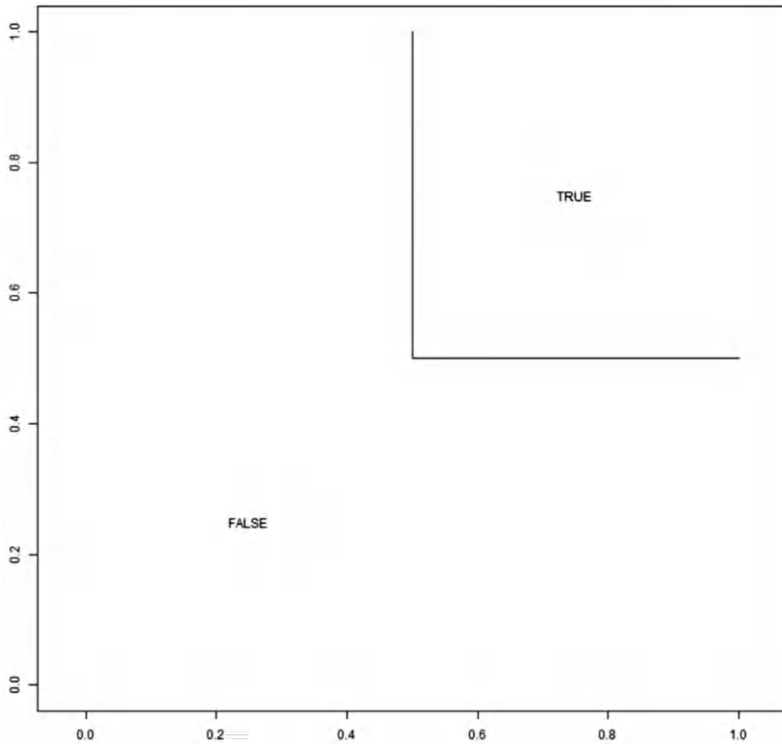


Figure 2.16 First dataset for single-layer perceptron – separable classes.

Source: Own work, with Python/NumPy.

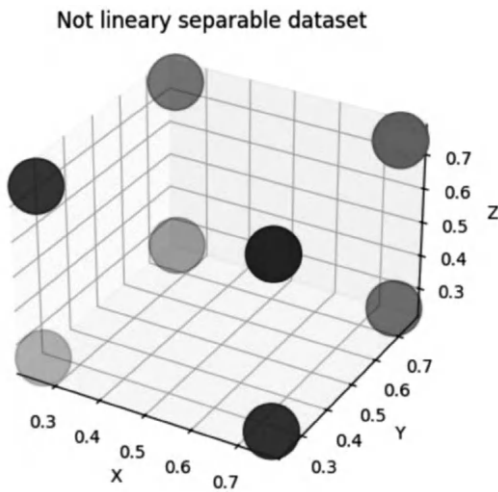


Figure 2.17 Second dataset for single-layer perceptron – non separable classes.

Source: Own work, with Python/NumPy.

52 Deep Learning Models for Economic Research

```
# with the same class definitions

training_inputs = np.array([[0,0,0], [0,0,1], [0,1, 0],
[0,1, 1],[1,0,0], [1,0,1], [1,1, 0], [1,1, 1]])
training_labels = np.array([0, 1, 1, 1,1,1,1,0])

slp = SingleLayerPerceptron(input_size=3)

slp.train(training_inputs, training_labels, learning_
rate=0.01, epochs=300)

predicted_labels=slp.predict_array(training_inputs)
accuracy = accuracy_score(training_labels,
    predicted_labels)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_
report(training_labels, predicted_labels))
# Create a figure and a set of subplots
fig, ax = plt.subplots(2,2)

num_columns = slp.all_weights.shape[1]
for i in range(num_columns):
    ax[i//2,i%2].plot(slp.all_weights[:, i], label=f'W
{i}',color=plt.rcParams['axes.prop_cycle'].by_key()['color'][i])
    ax[i//2,i%2].legend()
fig.suptitle('Weights estimation in subsequent epochs
(Non-separable classes)', fontsize=13)
plt.show()
```

The results of program execution are presented in [Figures 2.18](#) and [2.19](#).

```
>>>>> Epoch 296 Error: 1
>>>>> Epoch 297 Error: 1
>>>>> Epoch 298 Error: 1
>>>>> Epoch 299 Error: 1
Accuracy: 0.38
Classification Report:
```

	precision	recall	f1-score	support
0	0.20	0.50	0.29	2
1	0.67	0.33	0.44	6
accuracy			0.38	8
macro avg	0.43	0.42	0.37	8
weighted avg	0.55	0.38	0.40	8

Figure 2.18 The classification scores for AND gate task.

Source: Own work, with Python/NumPy.

Weights estimation in subsequent epochs (Non-separable classes)

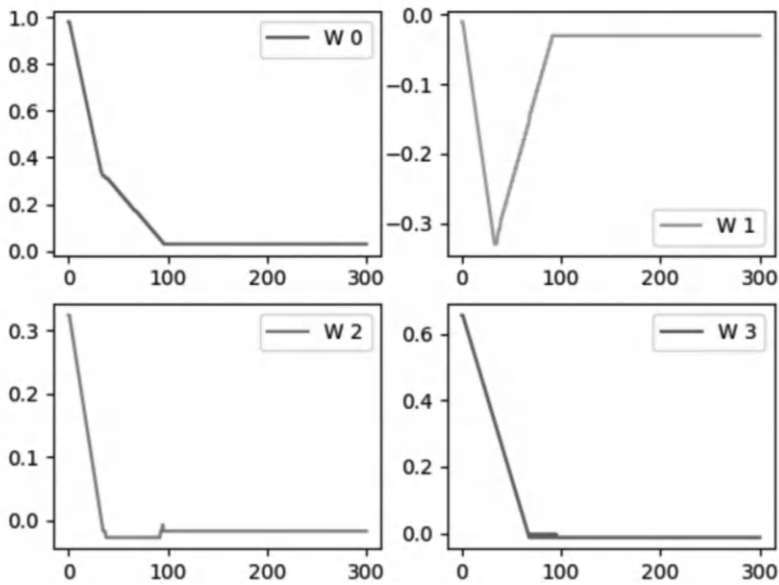


Figure 2.19 The learned weights in single-layer perceptron for non-linearly separable classes.

Source: Own work, with Python/NumPy.

In this case, the perceptron behaves worse. This may be explained by the dataset structure, which states that classes 1 and 0 (true and false) are not linearly separable. In fact, [Minsky and Papert \(1969\)](#) proved that Rosenblatt's perceptron can find a solution to the discrimination problem only when the classes are linearly separable. Classification of observations in the absence of linear separability of classes became possible only after multi-layer perceptron neural networks, in which one or more hidden layers appear.

2.4.1.2 Multi-layer perceptron

Multi-layer perceptron is the extension of single-layer perceptron idea with one crucial enhancement – hidden layers. [Rosenblatt \(1958\)](#) proposed the multi-layered perceptron model, which consists of an input layer, a hidden layer with randomized weights, and an output layer with learnable connections. [Rumelhart et al. \(1986\)](#) popularized backpropagation (which was used earlier by, e.g., [Linnainmaa, 1970](#); [Werbos, 1982](#)), and from then on, it has been the standard technique for FNNs. Typical multi-layer perceptron is presented in [Figure 2.20](#).

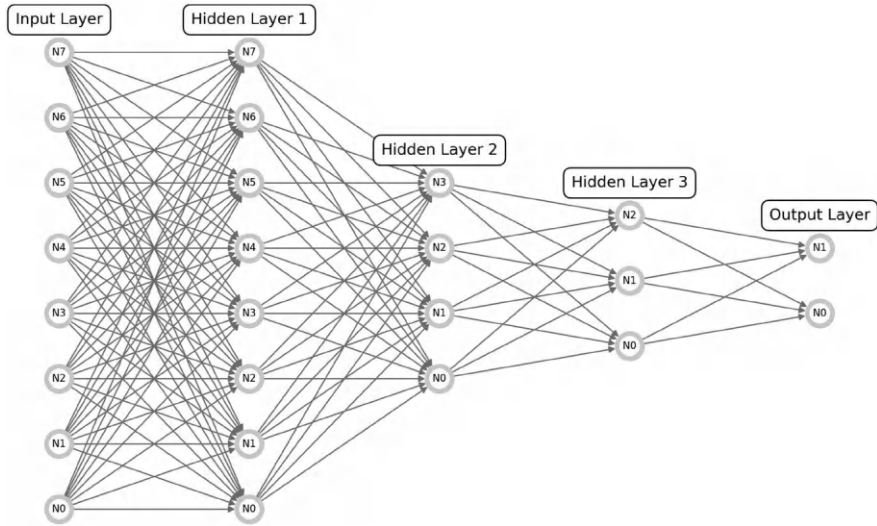


Figure 2.20 Multi-layer perceptron with three hidden layers.

Source: Own work, with Python/Matplotlib/NetworkX.

The backpropagation algorithm operates in this case in two steps:

- **Forward pass:** This involves calculating the theoretical values based on the activation function (sigmoidal or any from 2.3.1. functions).
- **Backward pass:** The error value is computed. This allows for the calculation of the gradients and subsequently modifying the synaptic weights according to gradient descent or any of methods from 2.3.4.

The multi-layer perceptron may be implemented in the following Python code.

```
class MultiLayerPerceptron:
    def sigmoid(self,x):
        return 1 / (1 + np.exp(-x))

    def sigmoid_derivative(self,x):
        return x * (1 - x)

    def mse_loss(self,y_true, y_pred):
        return np.mean(np.power(y_true - y_pred, 2))

    def mse_loss_derivative(self,y_true, y_pred):
        # return np.mean(2*(y_pred - y_true))
        return 2 * (y_pred - y_true) / y_true.size
```

```

def __init__(self, input_size, hidden_layers):
    self.input_size=input_size
    self.hidden_layers=hidden_layers
    self.network = {}

def initialize_weights(self,size):
    weights = np.array(list(range(1, size + 1))) / size
    weights.shape=(size,1)
    self.network[f'W{self.hidden_layers}'] = weights
    for i in range(1,self.hidden_layers):
        weights = np.array(list(range(1, size + 1)) *
size) / size
        weights.shape = (size, size)
        self.network[f'W{i}'] = weights
    weights = np.array(list(range(1, size + 1)) * self.
input_size) / size
    weights.shape = (self.input_size, size)
    self.network[f'W{0}'] = weights

def forward_propagation(self, X):
    cache={}
    L=X
    cache['L0']=L
    for i in range(1,self.hidden_layers+2):
        L = self.sigmoid(np.dot(L, self.
network[f'W{i-1}']))
        cache[f'L{i}']=L
    return cache

def backward_propagation(self,cache, X, Y,learning_rate):
    L=cache[f"L{self.hidden_layers+1}"]
    L_error =self.mse_loss_derivative(Y,L)
    L_delta = L_error * self.sigmoid_derivative(L)
    self.network[f'W{self.hidden_layers}'] -= learning_
rate * cache[f"L{self.hidden_layers}"].T.dot(L_delta)
    for i in reversed(range(1,self.hidden_layers+1)):
        L_error = L_delta.dot(self.network[f'W{i}'].T)
        L_delta = L_error*self.sigmoid_derivative
(cache[f"L{i}"])
        self.network[f'W{i-1}']-= learning_rate *
cache[f"L{i-1}"].T.dot(L_delta)

def predict(self, input):
    weighted_sum = self.forward_propagation(input)
[f'L{self.hidden_layers+1}']
    return weighted_sum

def train(self,X, Y, learning_rate, epochs):
    self.initialize_weights(len(X))

```

```

    for epoch in range(epochs):
        cache = self.forward_propagation(X)
        self.backward_propagation(cache, X, Y, learning_rate)

        if epoch % 10 == 0:
            loss = self.mse_loss(Y, cache[f'L{self.
hidden_layers+1}'])
            print(f">>>Epoch {epoch}, Loss: {loss}")

    def predict_array(self, inputs):
        return [self.predict(inputs[i]) for i in
            range(len(inputs))]

```

To train the multi-layer perceptron on data from a previous example, we may run the following code.

```

training_inputs = np.array([[0, 0, 0], [0, 0, 1], [0, 1, 0],
[0, 1, 1], [1, 0, 0], [1, 0, 1], [1, 1, 0], [1, 1, 1]])
training_labels = np.array([0, 1, 1, 1, 1, 1, 1, 0])
training_labels.shape = (8, 1)

```

```

mlp = MultiLayerPerceptron(input_size=3,hidden_layers=2)

```

```

mlp.train(training_inputs, training_labels, learning_
rate=0.1, epochs=80000)

```

```

predicted_labels = mlp.predict_array(training_inputs)
predicted_labels = [1 if x >0.5 else 0 for x in mlp.
predict_array(training_inputs)]
accuracy = accuracy_score(training_labels, predicted_labels)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_
report(training_labels, predicted_labels))

```

```

l,m=mlp.all_weights[0]["W0"].shape

```

```

fig, ax = plt.subplots(l,m)
for i in range(0,l):
    for j in range(0, m):
        line=[a["W0"][i,j] for a in mlp.all_weights]
        ax[i,j].plot(line, label=f'W_{i}_{j}',color=plt.
rcParams['axes.prop_cycle'].by_key()['color'][i+j])
        ax[i,j].legend()
fig.suptitle('Input weights estimation\n in subsequent epochs
(Non-separable classes - MLP)', fontsize=11)
plt.show()
l,m=mlp.all_weights[1]["W1"].shape

```

```

fig, ax = plt.subplots(l,m)

```

```
...
>>>Epoch 79960, Loss: 0.0010181978732942036
>>>Epoch 79970, Loss: 0.001017952831042868
>>>Epoch 79980, Loss: 0.0010177078940549018
>>>Epoch 79990, Loss: 0.0010174630622660093
Accuracy: 1.00
Classification Report:

```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	2
1	1.00	1.00	1.00	6
accuracy			1.00	8
macro avg	1.00	1.00	1.00	8
weighted avg	1.00	1.00	1.00	8

Figure 2.21 The classification scores for non-linearly separable classes.

Source: Own work, with Python/NumPy.

```
for i in range(0,1):
    for j in range(0, m):
        line=[a["W1"][i,j] for a in mlp.all_weights]
        ax[i,j].plot(line, label=f'W_{i}_{j}',color=plt.
rcParams['axes.prop_cycle'].by_key()['color'][i+j])
        ax[i,j].legend()
    fig.suptitle('Hidden layers weights estimation\n
in subsequent epochs (Non-separable classes - MLP)',
fontsize=11)
    plt.show()
```

Unlike the single-layer perceptron, the multi-layer perceptron can also handle classification tasks in which classes are not linearly separable. For the data in the previous example, a properly trained multi-layer perceptron can recognize actual class belongingness with highly satisfactory accuracy. The results of the analysis are presented in [Figures 2.21–2.23](#).

2.4.2 Convolutional neural networks (CNNs)

In 1989, Yann LeCun, Yoshua Bengio, and Patrick Haffner demonstrated the application of Convolutional Neuron Networks (CNNs) to recognize handwritten characters, showcasing the potential of neural networks for solving real-world problems ([LeCun et al., 1989](#)). Since then, CNNs have revolutionized many domains, but have had the highest impact on computer vision.

CNNs typically consist of three fundamental layer types: convolutional layers, pooling layers, and fully connected layers. Stacking these layers in succession gives rise to the CNN architecture (see [Figure 2.24](#)).

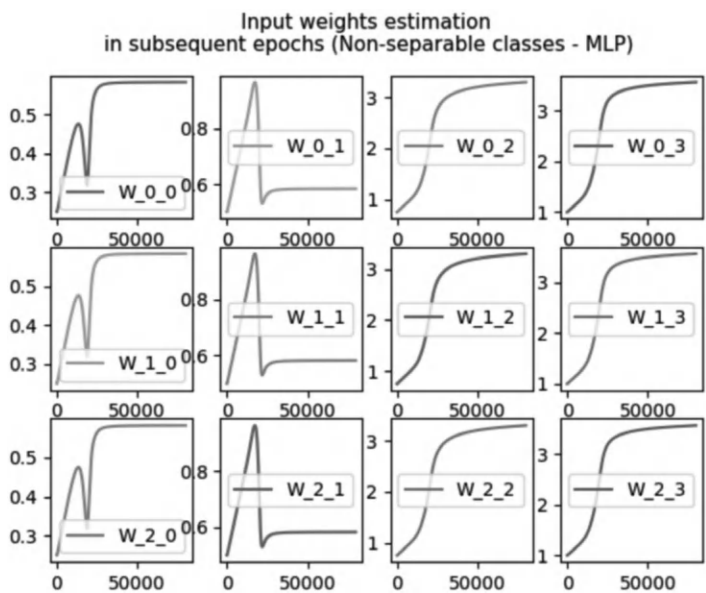


Figure 2.22 The learned weights of input layers in multi-layer perceptron for non-linearly separable classes.

Source: Own work, with Python/NumPy.

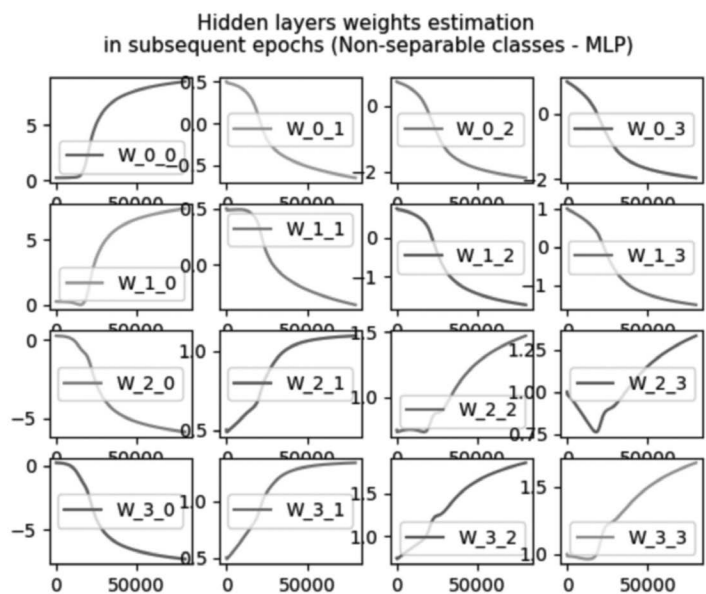


Figure 2.23 The learned weights of hidden layers in multi-layer perceptron for non-linearly separable classes.

Source: Own work, with Python/NumPy.

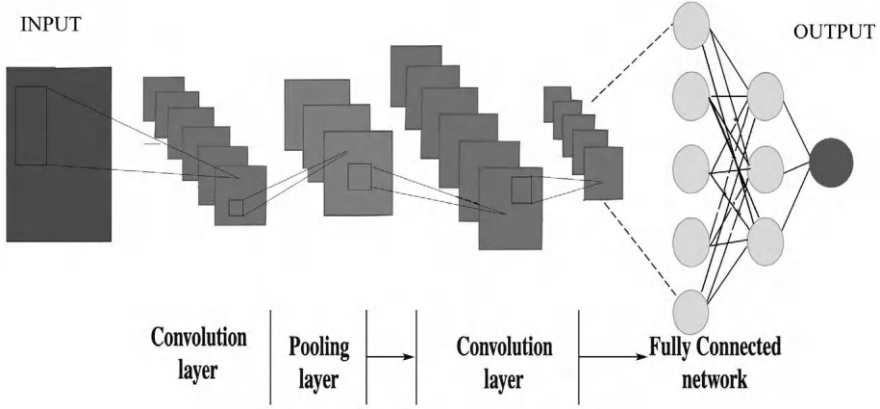


Figure 2.24 Convolutional network architecture.

Source: Own work based on Jiang et al. (2024).

2.4.2.1 Convolutional layer

A convolutional layer applies a set of learnable filters (or kernels) to the input data to extract spatial features. Each filter slides over the input, performing element-wise multiplication and summation to produce a feature map. Mathematically, for a two-dimensional input I and a filter K , the convolution operation at position (i, j) is:

$$(I * K)(i, j) = \sum_m \sum_n I(i + m, j + n) \cdot K(m, n) \quad (2.35)$$

where:

$I(i + m, j + n)$ is the input value, $K(m, n)$ is the filter weight, and (m, n) are the filter indices. This operation captures local patterns like edges, textures, or shapes.

2.4.2.2 Pooling layer

A pooling layer downsamples the feature maps produced by the convolutional layer, reducing spatial dimensions while retaining important information. The most common type is max pooling, which selects the maximum value in a window. For a pooling window of size at $\{p \times p\}$ position (i, j) :

$$\text{MaxPool}(i, j) = \max_{m, n} I(i + m, j + n) \quad (2.36)$$

Another popular type is average pooling, which computes the average value in the window:

$$\text{AvgPool}(i, j) = \frac{1}{p^2} \sum_m \sum_n I(i + m, j + n) \quad (2.37)$$

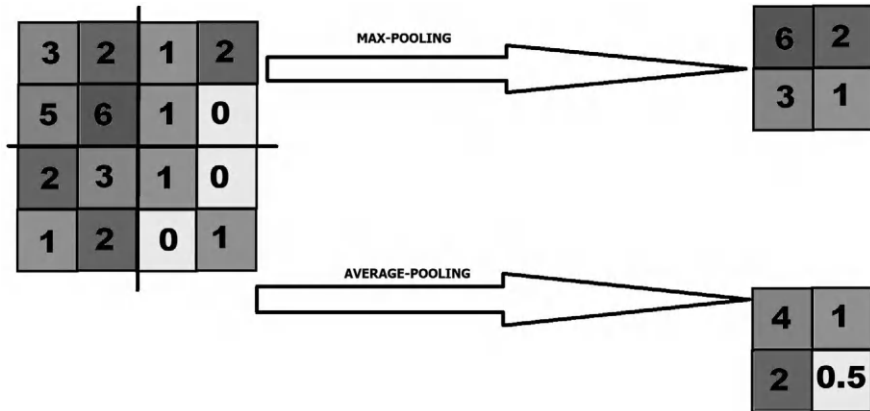


Figure 2.25 Pooling

Source: Own research based on [Jiang et al. \(2024\)](#).

Pooling reduces computational complexity, controls overfitting, and makes the model more invariant to small spatial shifts. Together, convolutional and pooling layers form the backbone of CNNs for feature extraction and dimensionality reduction. The idea of pooling is illustrated in [Figure 2.25](#).

Since the first application of CNNs was handwriting recognition, we will construct such a network using the PyTorch library and the widely used MNIST collection (see 2.7). The MNIST library by itself will be used to create an image “pretending” to be handwritten with the following code:

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import matplotlib.pyplot as plt

import random
import numpy as np

random.seed(25032005)

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.1307,), (0.3081,))
])

train_dataset = datasets.MNIST(
```

```

    root='./data', train=True, download=True,
        transform=transform
)

digits_to_create = [2, 5, 0, 3, 2, 0, 0, 5]

def get_mnist_digit_image(dataset, digit):
    indices = [i for i, (_, label) in enumerate(dataset) if
label == digit]
    chosen_index = random.choice(indices)
    img, label = dataset[chosen_index]
    return img # shape [1, 28, 28]

digit_images = [get_mnist_digit_image(train_dataset, d) for
d in digits_to_create]

stitched_image = torch.cat(digit_images, dim=2)

width_px = 224
height_px = 28
dpi = 300 # or any other DPI

width_in = width_px / dpi
height_in = height_px / dpi

fig = plt.figure(figsize=(width_in, height_in), dpi=dpi)

plt.subplots_adjust(left=0, right=1, top=1, bottom=0)
plt.axis('off') # turn off axis ticks/labels

plt.imshow(stitched_image.squeeze().numpy(), cmap='gray')
plt.savefig("mnist_digit_samples.png", dpi=dpi, bbox_
inches='tight', pad_inches=0)

plt.show()

```

The artificially generated “fake” handwritten image is presented in [Figure 2.26](#).

As we just stated the MNIST library by itself will be used to create an image “pretending” to be handwritten, although the reader can also use their own scanned



Figure 2.26 Image to recognize.

Source: Own work with use of Python/PyTorch/MNIST.

image by replacing the `Image.open(...)` line in the recognition, code having in mind to save the proportions of the image.

```
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

import random
import numpy as np
from PIL import Image
random.seed(25032005)
transform = transforms.Compose([
    transforms.ToTensor(),
    # Normalize with mean/std from original MNIST dataset
    transforms.Normalize((0.1307,), (0.3081,))
])

train_dataset = datasets.MNIST(
    root='./data', train=True, download=True,
    transform=transform
)

test_dataset = datasets.MNIST(
    root='./data', train=False, download=True,
    transform=transform
)

train_loader = DataLoader(train_dataset, batch_size=64,
    shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=1000,
    shuffle=False)

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 16, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(32 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)
    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.max_pool2d(x, 2)
```

```

        x = F.relu(self.conv2(x))
        x = F.max_pool2d(x, 2)
        x = x.view(x.size(0), -1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

device = torch.device("cuda" if torch.cuda.is_available()
else "cpu")

model = SimpleCNN().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

epochs = 5
model.train()
for epoch in range(epochs):
    for data, target in train_loader:
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = criterion(output, target)
        loss.backward()
        optimizer.step()
    print(f"Epoch {epoch+1} complete. Last batch loss:
{loss.item():.4f}")

predicted_sequence = []

image_path = "mnist_digit_samples.png"
stitched_img_pil = Image.open(image_path).convert("L")
print("PIL. size ->", stitched_img_pil.size)  # (224, 28)

stitched_image = transform(stitched_img_pil)
with torch.no_grad():
    for i in range(8):
        single_digit_image = stitched_image[:, :, i * 28:
(i + 1) * 28]
        single_digit_image= single_digit_image.unsqueeze_(0)
        single_digit_image = single_digit_image.to(device)
        output = model(single_digit_image)
        pred = output.argmax(dim=1).item()  # predicted digit
        predicted_sequence.append(pred)

print("Predicted sequence:", predicted_sequence)

```

Figure 2.27 presents the correctly predicted digits sequence as a result of the program execution.

```

Epoch 1 complete. Last batch loss: 0.0626
Epoch 2 complete. Last batch loss: 0.0058
Epoch 3 complete. Last batch loss: 0.0061
Epoch 4 complete. Last batch loss: 0.0006
Epoch 5 complete. Last batch loss: 0.0019
Predicted sequence: [2, 5, 0, 3, 2, 0, 0, 5]

```

Figure 2.27 The digits sequence predicted (recognized) from image.

Source: Own research with use of Python/PyTorch/MNIST.

2.4.3 Recurrent neural networks (RNNs)

The RNNs (Elman, 1990; Jordan, 1986) deal with sequential data and may be treated in some way as simplified Turing Machines (Turing, 1936). In opposition to FNNs assuming independence of all inputs and outputs, RNN output from the current step is fed as input to the next step. It defines the internal storage value (hidden) value, usually named h , and the appropriate weights system:

- Input weights (W_i): Impact of currently predicted data;
- Hidden state weights (W_h): Impact of patterns already detected/stored;
- Output weights (W_o): Mapping of the internal representation to the output space according to the task, for example, classification or sequence prediction.

In the simplest form RNNs may be visualized in Figure 2.28.

On the left side, the recurrent model is presented; the right presents equivalent step-by-step architecture.

The following code implements simple RNN in Python language with no use of any deep learning libraries.

```

import numpy as np
import matplotlib.pyplot as plt
class SimpleRNN:

```

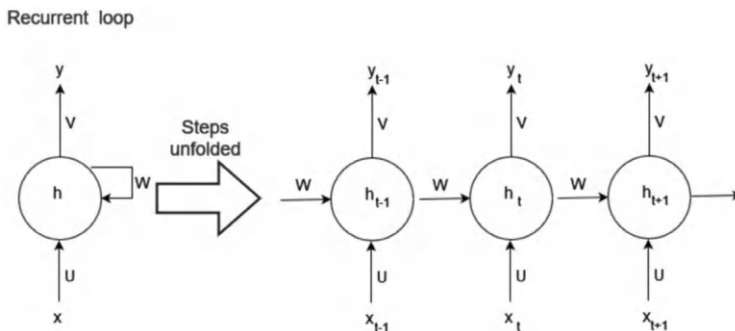


Figure 2.28 RNN architecture.

Source: Own work inspired by LeCun et al. (2015).

```

def __init__(self, input_size, hidden_size, output_size,
learning_rate=0.01):
    self.input_size = input_size
    self.hidden_size = hidden_size
    self.output_size = output_size
    self.learning_rate = learning_rate

    self.U = np.random.randn(hidden_size, input_size) * 0.01
    self.W = np.random.randn(hidden_size, hidden_size) * 0.01
    self.V = np.random.randn(output_size, hidden_size) * 0.01
    self.b_h = np.zeros((hidden_size, 1))
    self.b_y = np.zeros((output_size, 1))

def tanh(self, x):
    return np.tanh(x)

def forward(self, x):
    T = len(x)
    h = np.zeros((T + 1, self.hidden_size, 1))
    y = np.zeros((T, self.output_size, 1))

    for t in range(T):
        h[t] = self.tanh(np.dot(self.U, x[t]) +
np.dot(self.W, h[t - 1]) + self.b_h)
        y[t] = np.dot(self.V, h[t]) + self.b_y

    return y, h

def backward(self, x, y_true, y_pred, h):
    T = len(x)
    dU = np.zeros_like(self.U)
    dW = np.zeros_like(self.W)
    dV = np.zeros_like(self.V)
    db_h = np.zeros_like(self.b_h)
    db_y = np.zeros_like(self.b_y)
    dh_next = np.zeros((self.hidden_size, 1))

    for t in reversed(range(T)):
        dy = y_pred[t] - y_true[t]
        dV += np.dot(dy, h[t].T)
        db_y += dy

        dh = np.dot(self.V.T, dy) + dh_next
        dtanh = (1 - h[t] ** 2) * dh

        dU += np.dot(dtanh, x[t].T)
        dW += np.dot(dtanh, h[t - 1].T)
        db_h += dtanh

```

```

        dh_next = np.dot(self.W.T, dtanh)

    return dU, dW, dV, db_h, db_y

    def update_parameters(self, dU, dW, dV, db_h, db_y):
        self.U -= self.learning_rate * dU
        self.W -= self.learning_rate * dW
        self.V -= self.learning_rate * dV
        self.b_h -= self.learning_rate * db_h
        self.b_y -= self.learning_rate * db_y

    def train(self, x_train, y_train, epochs=100):
        for epoch in range(epochs):
            total_loss = 0
            for x, y_true in zip(x_train, y_train):
                y_pred, h = self.forward(x)
                loss = np.sum((y_pred - y_true) ** 2)
                total_loss += loss
                dU, dW, dV, db_h, db_y = self.backward(x, y_
true, y_pred, h)
                self.update_parameters(dU, dW, dV, db_h, db_y)

            if (epoch + 1) % 10 == 0:
                print(f'Epoch {epoch + 1}/{epochs}, Loss:
{total_loss:.4f}')

    def predict(self, x, h_prev=None):
        y_pred, h = self.forward(x)
        return y_pred

x_train = np.linspace(0, 4 * np.pi, 100)
y_train = (np.cos(x_train)**2).reshape(-1, 1)

sequence_length = 20
x_seq = []
y_seq = []

for i in range(len(x_train) - sequence_length):
    x_seq.append(y_train[i:i + sequence_length])
    y_seq.append(y_train[i + 1:i + sequence_length + 1])

x_seq = [x.reshape(-1, 1, 1) for x in x_seq] # Reshape for
(T, input_size, 1)
y_seq = [y.reshape(-1, 1, 1) for y in y_seq]

rnn = SimpleRNN(input_size=1, hidden_size=50, output_size=1,
learning_rate=0.01)
rnn.train(x_seq, y_seq, epochs=4000)

```

```

x_future = np.linspace(4 * np.pi, 5 * np.pi, 40)
y_future = np.zeros_like(x_future).reshape(-1, 1, 1)

h_prev = None
predictions = []

last_seq = y_train[-sequence_length:].reshape(-1, 1, 1)
for _ in range(len(x_future)):
    y_pred, h_prev = rnn.forward(last_seq)
    last_seq = np.roll(last_seq, -1, axis=0)  # Shift the
sequence to include the new prediction
    last_seq[-1] = y_pred[-1]
    predictions.append(y_pred[-1].item())

plt.figure(figsize=(12, 6))
plt.plot(x_train, y_train, label="Training Data (cos^2)")
plt.plot(x_future, predictions, label="Predictions (4pi to 5pi)",
linestyle='--')
plt.xlabel("x")
plt.ylabel("cos^2(x)")
plt.title("RNN trained to predict squared cosine wave")
plt.legend()
plt.show()

```

The network is used to predict the course of function $\cos^2(x)$ in range 4π to 5π . The results of prediction are presented in [Figure 2.29](#).

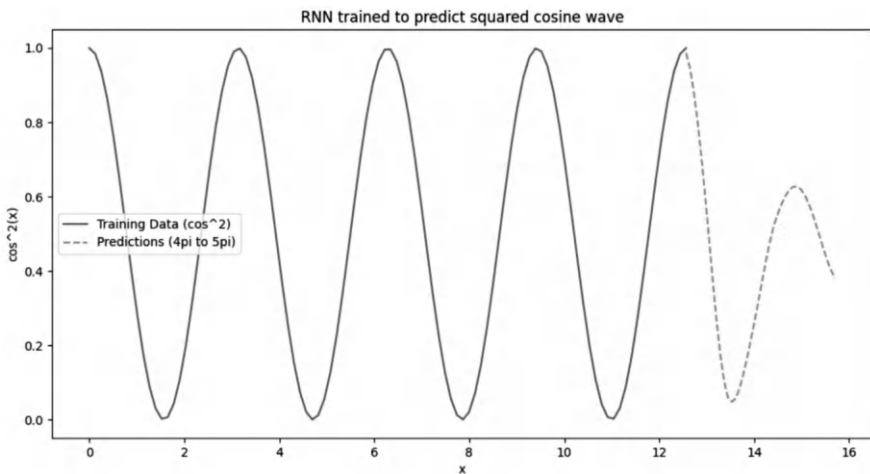


Figure 2.29 $\cos^2(x)$ prediction with RNN.

Source: Own work with Python/NumPy.

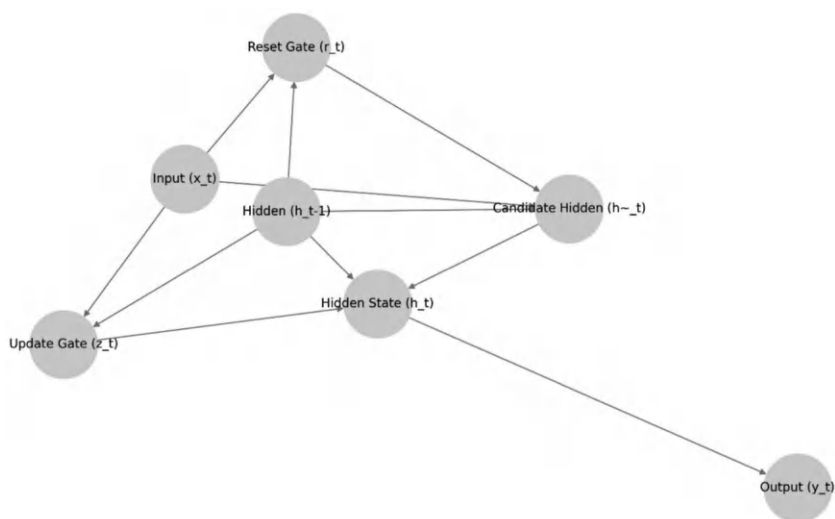


Figure 2.30 GRU network architecture.

Source: Own work, with Python/NetworkX.

RNNs are designed to recognize data sequences and contain loops through which information can be persisted. Variants like long short-term memory (LSTM) and gated recurrent units (GRU) (see [Figures 2.30](#) and [2.31](#)) are developed mainly to address issues of the vanishing gradient problem.

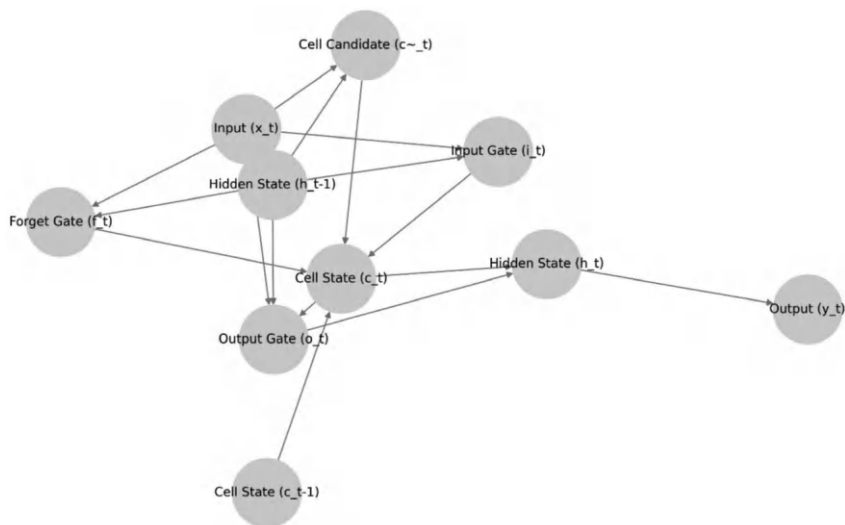


Figure 2.31 LSTM network architecture.

Source: Own work, with Python/NetworkX.

Table 2.5 LSTM architecture

Gate	Purpose	Associated weights
Forget gate (f_t)	Decides what information to discard	W_f (input weight), U_f (recurrent weight), b_f (bias)
Input gate (i_t)	Decides what new information to store	W_i (input weight), U_i (recurrent weight), b_i (bias)
Output gate (o_t)	Decides what to output	W_o (input weight), U_o (recurrent weight), b_o (bias)
Cell candidate ($\hat{c}_t$)	Proposed new cell state; the cell state is updated using the forget and input gates	W_c (input weight), U_c (recurrent weight), b_c (bias)

Source: Own work based on [Hochreiter and Schmidhuber \(1997\)](#).

Table 2.6 GRU architecture

Gate	Purpose	Associated weights
Update gate (z_t)	Controls how much past information to keep	W_z (input weight), U_z (recurrent weight), b_z (bias)
Reset gate (r_t)	Controls how much past information to forget	W_r (input weight), U_r (recurrent weight), b_r (bias)
Candidate activation ($\hat{h}_t$)	Proposed new hidden state	W_h (input weight), U_h (recurrent weight), b_h (bias)

Source: Own work based on [Chung et al. \(2014\)](#).

[Tables 2.5](#) and [2.6](#) characterize LSTM and GRU architectures.

The following code implements LSTM network for the same purpose as previous example.

```
import random
import numpy as np
import sys
import matplotlib.pyplot as plt

def progress_bar(current, total, loss, bar_length=40):
    fraction = current / total
    arrow_length = int(fraction * bar_length)
    arrow = '=' * arrow_length
    blue_arrow = f"\033[94m{arrow}\033[0m"
    spaces = ' ' * (bar_length - arrow_length)

    progress = f"\rProgress: [{blue_arrow}{spaces}]
{fraction * 100:6.2f}% loss:{loss:3.2f}"
    sys.stdout.write(progress)
    sys.stdout.flush()
```

70 *Deep Learning Models for Economic Research*

```
class LSTM:
    def __init__(self, input_size, hidden_size, output_size,
learning_rate=0.01):
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.output_size = output_size
        self.learning_rate = learning_rate
        self.Wf = np.random.randn(hidden_size, input_size +
hidden_size) * 0.01
        self.Wi = np.random.randn(hidden_size, input_size +
hidden_size) * 0.01
        self.Wc = np.random.randn(hidden_size, input_size +
hidden_size) * 0.01
        self.Wo = np.random.randn(hidden_size, input_size +
hidden_size) * 0.01

        self.bf = np.zeros((hidden_size, 1))
        self.bi = np.zeros((hidden_size, 1))
        self.bc = np.zeros((hidden_size, 1))
        self.bo = np.zeros((hidden_size, 1))

        self.V = np.random.randn(output_size, hidden_size) * 0.01
        self.b_y = np.zeros((output_size, 1))

    def sigmoid(self, x):
        return 1 / (1 + np.exp(-x))

    def tanh(self, x):
        return np.tanh(x)

    def forward(self, x):
        T = len(x)
        h = np.zeros((T, self.hidden_size, 1))
        c = np.zeros((T, self.hidden_size, 1))
        y = np.zeros((T, self.output_size, 1))

        h_prev = np.zeros((self.hidden_size, 1))
        c_prev = np.zeros((self.hidden_size, 1))

        for t in range(T):
            combined = np.vstack((h_prev, x[t]))

            ft = self.sigmoid(np.dot(self.Wf, combined) +
self.bf)
            it = self.sigmoid(np.dot(self.Wi, combined) +
self.bi)
            c_hat = self.tanh(np.dot(self.Wc, combined) +
self.bc)
            ot = self.sigmoid(np.dot(self.Wo, combined) +
self.bo)
```

```

        c[t] = ft * c_prev + it * c_hat
        h[t] = ot * self.tanh(c[t])

        y[t] = np.dot(self.V, h[t]) + self.b_y

        h_prev = h[t]
        c_prev = c[t]

    return y, h, c

def backward(self, x, y_true, y_pred, h, c):
    T = len(x)
    dWf = np.zeros_like(self.Wf)
    dWi = np.zeros_like(self.Wi)
    dWc = np.zeros_like(self.Wc)
    dWo = np.zeros_like(self.Wo)
    dbf = np.zeros_like(self.bf)
    dbi = np.zeros_like(self.bi)
    dbc = np.zeros_like(self.bc)
    dbo = np.zeros_like(self.bo)
    dV = np.zeros_like(self.V)
    db_y = np.zeros_like(self.b_y)

    dh_next = np.zeros((self.hidden_size, 1))
    dc_next = np.zeros((self.hidden_size, 1))

    for t in reversed(range(T)):
        if t == 0:
            h_prev = np.zeros((self.hidden_size, 1))
            c_prev = np.zeros((self.hidden_size, 1))
        else:
            h_prev = h[t - 1]
            c_prev = c[t - 1]

        dy = y_pred[t] - y_true[t]
        dV += np.dot(dy, h[t].T)
        db_y += dy

        dh = np.dot(self.V.T, dy) + dh_next

        combined = np.vstack((h_prev, x[t]))
        Wo_activation = np.dot(self.Wo, combined) + self.bo
        ot = self.sigmoid(Wo_activation)
        do = dh * self.tanh(c[t])
        do *= ot * (1 - ot)

        dc = dh * ot * (1 - np.tanh(c[t]) ** 2) + dc_next

        Wf_activation = np.dot(self.Wf, combined) + self.bf
        ft = self.sigmoid(Wf_activation)

```

```

dft = dc * c_prev
dft *= ft * (1 - ft)

Wi_activation = np.dot(self.Wi, combined) + self.bi
it = self.sigmoid(Wi_activation)
Wc_activation = np.dot(self.Wc, combined) + self.bc
c_hat = self.tanh(Wc_activation)
dit = dc * c_hat
dit *= it * (1 - it)
dc_hat = dc * it
dc_hat *= (1 - np.tanh(Wc_activation) ** 2)

dWf += np.dot(dft, combined.T)
dWi += np.dot(dit, combined.T)
dWc += np.dot(dc_hat, combined.T)
dWo += np.dot(do, combined.T)
dbf += dft
dbi += dit
dbc += dc_hat
dbo += do

dcombined = (np.dot(self.Wf.T, dft) +
              np.dot(self.Wi.T, dit) +
              np.dot(self.Wc.T, dc_hat) +
              np.dot(self.Wo.T, do))
dh_next = dcombined[:self.hidden_size, :]
dc_next = dc * ft

return dWf, dWi, dWc, dWo, dbf, dbi, dbc, dbo, dV, db_y

def update_parameters(self, dWf, dWi, dWc, dWo, dbf,
dbi, dbc, dbo, dV, db_y):
    self.Wf -= self.learning_rate * dWf
    self.Wi -= self.learning_rate * dWi
    self.Wc -= self.learning_rate * dWc
    self.Wo -= self.learning_rate * dWo
    self.bf -= self.learning_rate * dbf
    self.bi -= self.learning_rate * dbi
    self.bc -= self.learning_rate * dbc
    self.bo -= self.learning_rate * dbo
    self.V -= self.learning_rate * dV
    self.b_y -= self.learning_rate * db_y

def train(self, x_train, y_train, epochs=100):
    for epoch in range(epochs):
        total_loss = 0
        for x, y_true in zip(x_train, y_train):
            y_pred, h, c = self.forward(x)
            loss = np.sum((y_pred - y_true) ** 2)

```

```

        total_loss += loss
        dWf, dWi, dWc, dWo, dbf, dbi, dbc, dbo, dV,
db_y = self.backward(x, y_true, y_pred, h, c)
        self.update_parameters(dWf, dWi, dWc, dWo,
dbf, dbi, dbc, dbo, dV, db_y)
        progress_bar(epoch, epochs, total_loss)

    def predict(self, x):
        y_pred, h, c = self.forward(x)
        return y_pred

random.seed(25032005)
x_train = np.linspace(0, 8 * np.pi, 400) # Input range: 0
        to 4*pi
y_train = (np.cos(x_train)).reshape(-1, 1) # Target: square
        cosine

sequence_length = 23
x_seq = []
y_seq = []

for i in range(len(x_train) - sequence_length):
    x_seq.append(y_train[i:i + sequence_length])
    y_seq.append(y_train[i + 1:i + sequence_length + 1])

x_seq = [x.reshape(-1, 1, 1) for x in x_seq] # Reshape for
(T, input_size, 1)
y_seq = [y.reshape(-1, 1, 1) for y in y_seq]

rnn = LSTM(input_size=1, hidden_size=10, output_size=1,
learning_rate=0.02)
rnn.train(x_seq, y_seq, epochs=4000)

x_future = np.linspace(8 * np.pi, 10 * np.pi, )
y_future = np.zeros_like(x_future).reshape(-1, 1, 1)

h_prev = None
predictions = []

last_seq = y_train[-sequence_length:].reshape(-1, 1, 1)
for _ in range(len(x_future)):
    y_pred, h, c = rnn.forward(last_seq)
    last_seq = np.roll(last_seq, -1, axis=0)
    last_seq[-1] = y_pred[-1]
    predictions.append(y_pred[-1].item())

plt.figure(figsize=(12, 6))
plt.plot(x_train, y_train, label="Training Data (cos^2)")

```

```
plt.plot(x_future, predictions, label="Predictions (4pi to 5pi)",
         linestyle='--')
plt.xlabel("x")
plt.ylabel("cos^2(x)")
plt.title("RNN trained to predict squared cosine wave")
plt.legend()
plt.savefig("cos2_lstm.jpg")
plt.show()
```

The results of LSTM network training and function $\cos^2(x)$ prediction in range 4π to 5π are presented in [Figure 2.32](#).

2.4.4 Generative adversarial networks (GANs)

Generative adversarial networks (GANs) were introduced by [Goodfellow et al. \(2014\)](#) and were first used for benchmarking purposes to create datasets containing objects (images, multimedia files) generated artificially but acting in further processing as real ones. Adversarial networks usually consist of two neural networks, a generator creating data from scratch and a discriminator, competing against each other. The generator creates data, and the discriminator evaluates its authenticity, which (in the ideal world) should lead to samples indistinguishable from real data. While this task is not crucial for artificial numerical data generation and may be completed only by properly using various stochastic distribution functions, it has found a wide area of applications for multimedia files. Typical architecture of Generative adversarial networks is presented in [Figure 2.33](#).

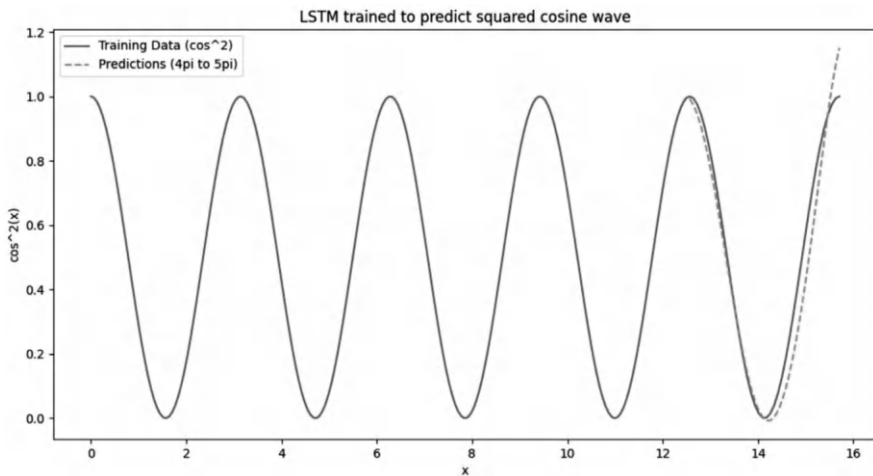


Figure 2.32 $\cos^2(x)$ prediction with LSTM network.

Source: Own work, with Python/NumPy.

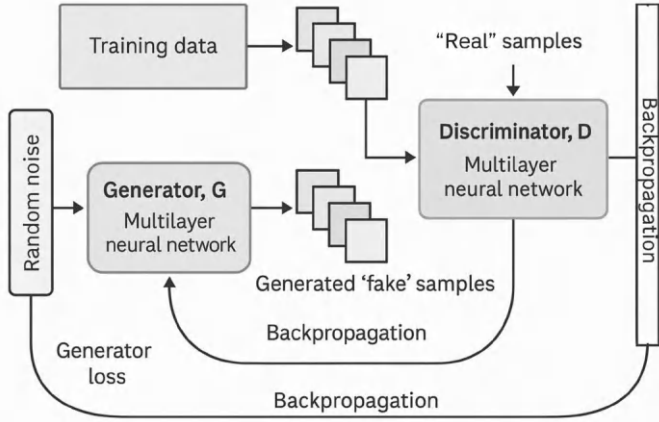


Figure 2.33 Architecture of GANs.

Source: Own work, with Python/NumPy.

The adversarial process may be formulated as a minimax game. GANs consist of two neural networks: a discriminator (D) and a generator (G). The discriminator is trained to classify images as real (from the training set) or fake (generated by the generator). The generator, on the other hand, transforms random noise $z \sim p(z)$ into images $G(z)$. The minimalization of (2.38) is the final aim of generation/verification process.

$$\min_G \max_D E_{x \sim p_{\text{data}}} [\log D(x)] + E_{z \sim p(z)} [\log (1 - D(G(z)))] \quad (2.38)$$

where:

$x \sim p_{\text{data}}$ represents real data samples;

$z \sim p(z)$ is random noise;

$G(z)$ generates fake images;

$D(\cdot)$ outputs the probability of an input being real.

Goodfellow et al. (2014) showed that this minimax game minimizes the Jensen-Shannon divergence between the data distribution and the generator's output distribution.

The training alternates between updating the generator and the discriminator to optimize this objective. The generator is updated to maximize the probability of the discriminator making incorrect choices on generated data:

$$\max_G E_{z \sim p(z)} [\log D(G(z))] \quad (2.39)$$

The discriminator is updated to maximize the probability of correctly classifying real and fake data:

$$\max_D E_{x \sim p_{\text{dt}}} [\log D(x)] + E_{z \sim p(z)} [\log (1 - D(G(z)))] \quad (2.40)$$

In practice, the generator update is modified to address vanishing gradients when the discriminator becomes too confident. Instead of minimizing $\log(1 - D(G(z)))$, the generator maximizes $\log D(G(z))$, which stabilizes training.

Table 2.7 presents the main types of GANs.

2.4.5 Autoencoders

Autoencoders are networks consisting of an encoder reducing the dimensionality of the dataset and a decoder bringing back the learned representation to its original form (or the form as close to the original one as possible).

Deep autoencoder networks are robust neural architectures that learn compressed representations of high-dimensional data through an unsupervised reconstruction task (Hinton & Salakhutdinov, 2006).

A key advantage of deep autoencoders lies in their ability to capture hierarchical representations, especially when using multiple hidden layers with non-linear activation functions (Goodfellow et al., 2016). Early work on stacked autoencoders showed that training one layer at a time, often with methods like greedy layer-wise pre-training, helps the network converge to better local minima.

The most frequent application of deep autoencoders is related to data denoising in training models on noisy signals and restoring data cleaned (Vincent et al., 2008). As a result, denoising autoencoders have become standard tools in preprocessing pipelines, especially image and audio preprocessing.

The learned representations are typically robust and are often used to improve the quality of cluster analysis tasks.

Autoencoders play a significant role in anomaly detection in high-dimensional data, such as network traffic or medical imaging (Zhou & Paffenroth, 2017). This approach may be valuable for identifying fraudulent transactions, unusual sensor readings, or pathological indicators in medical scans.

In some cases, deep autoencoders are extended into generative models that can produce new, novel samples. For instance, variational autoencoders (VAEs) combine the compressive strengths of autoencoders with probabilistic modeling, enforcing latent distributions conducive to sampling (Kingma & Welling, 2014), for example, allowing image synthesis and interpolation between data points in the latent space.

Deep autoencoders may be treated as the extension/replacement of principal component analysis commonly used in “traditional” classification or regression tasks for feature extraction. They play a similar role in applications in NLP, bioinformatics, and robotics, where dimensionality reduction is crucial for practical storage and computation (Baldi, 2012).

Table 2.7 Types of GANs

Type	Description	Source
Vanilla GAN	The original adversarial framework where a generator network attempts to fool a discriminator that classifies real vs generated data.	Goodfellow et al. (2014)
DCGAN	Introduced convolutional (and transpose convolution) layers for stable image generation, providing architectural guidelines such as batch norm and specific activation functions.	Radford et al. (2016)
Conditional GAN (cGAN)	Conditions both the generator and the discriminator on additional information (e.g., class labels) to control the type of output generated.	Mirza & Osindero (2014)
LSGAN	Uses a least-squares loss instead of binary cross-entropy to stabilize training and reduce problems like vanishing gradients.	Mao et al. (2017)
WGAN	Employs the Wasserstein (Earth Mover's) distance for more stable training and a meaningful loss metric, referring to the discriminator as a "critic".	Arjovsky et al. (2017)
WGAN-GP	An improved WGAN that replaces weight clipping with a gradient penalty to enforce the Lipschitz constraint more effectively.	Gulrajani et al. (2017)
Pix2Pix	A conditional GAN specialized for paired image-to-image translation tasks (e.g., sketches $\leftrightarrow$ realistic images), often using a PatchGAN discriminator.	Isola et al. (2017)
CycleGAN	Enables unpaired image-to-image translation by enforcing a cycle-consistency loss ($A \rightarrow B \rightarrow A$) to learn mappings without paired training data.	Zhu et al. (2017)
ProGAN	Trains the network progressively, starting from low-resolution images and incrementally adding layers to reach higher resolutions for more stable training.	Karras et al. (2017)
StyleGAN	Introduces a style-based generator architecture and mapping network for fine-grained control over generated images. Subsequent versions (StyleGAN2, StyleGAN3) enhance quality and remove artifacts further.	Karras et al. (2019)
InfoGAN	Maximizes mutual information between a subset of latent variables and the generator's output, encouraging interpretable, disentangled representations in the latent space.	Chen et al. (2016)
SAGAN	Integrates self-attention modules into the generator and discriminator to capture global, long-range dependencies, improving complex scene generation.	Zhang et al. (2019)
BigGAN	Scales up GAN training to larger batch sizes and deeper architectures, producing high-resolution, high-fidelity images (often on datasets like ImageNet).	Brock et al. (2019)
SRGAN	Focuses on single-image super-resolution, using perceptual and adversarial losses to produce high-resolution, photo-realistic outputs from low-resolution inputs.	Ledig et al. (2017)

Source: Own work based on given sources (references in last column).

2.4.6 Transformers

Transformers are based on an attention mechanism (Bahdanau et al., 2015; Sutskever et al., 2014), which is a computational method that constructs a context vector c by taking a weighted average of feature vectors $h(t)$. These feature vectors can be hidden neural network units or raw input data. The weights $\alpha(t)$, which the model itself generates, typically range between $[0 \text{ and } 1]$ and are designed to focus on a specific $h(t)$, allowing the weighted average to approximate the value of that particular time step. The weights are computed using a softmax function applied to relevance scores produced by another part of the model.

We should mention that the attention mechanism is computationally more expensive than directly selecting a specific $h(t)$. However, it has one meaningful advantage: direct indexing is differentiable and compatible with gradient descent family algorithms. As a result, the attention mechanism provides a trainable, differentiable approximation for tasks requiring focused context extraction. Plus, it may be parallelized, which appears to be one of fundamental factors. The attention mechanism is illustrated in Figure 2.34.

In some form, the transformer is an evolutionary, improved recurrent network, with one main advantage – parallelization.

The transformer model revolutionized NLP by relying heavily on attention to capture relationships between words in a sequence. Originally introduced to improve neural machine translation, the transformer uses multi-headed self-attention, which allows it to learn multiple patterns of word relationships in parallel.

The key to understanding the transformer lies in its encoder-decoder structure. The encoder is a stack of identical layers (commonly six) that first apply

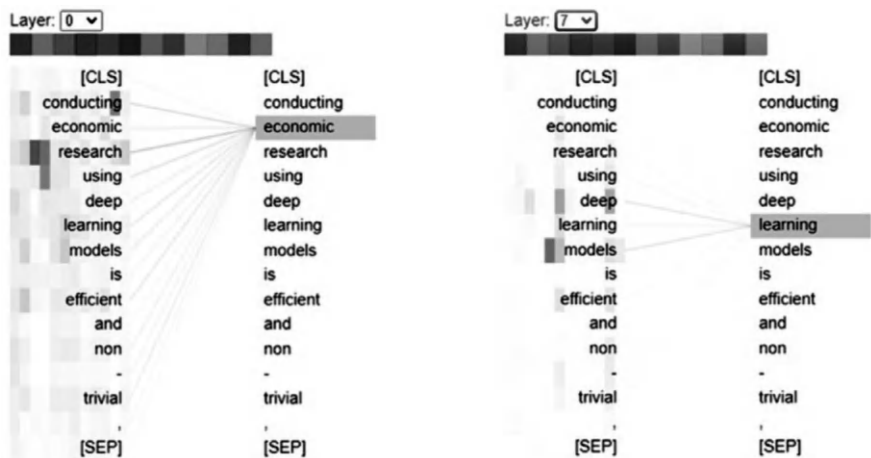


Figure 2.34 The attention mechanism in sentence, “Conducting economic research using deep learning models is efficient and non-trivial.”

Source: Own work, with Python.

self-attention and then an FNN to each word. Self-attention helps the model integrate context from all words in a sentence, for example, in “The animal didn’t cross the street because it was too tired,” the word “it” benefits from attention on both “animal” and “tired”. The decoder has a similar structure, but inserts an additional attention layer to focus on the encoder’s outputs, guiding the generation of translations or other outputs.

Within each encoder (and decoder) layer, multi-headed attention expands the model’s capacity to focus on multiple aspects of the sentence. Each head has its own set of Query (Q), Key (K), and Value (V) weight matrices, which project input embeddings into different “representation subspaces”. Performing the attention calculation across multiple heads reveals varied contextual relationships. After computing separate attentions, these are concatenated and linearly transformed by a final weight matrix (W^o) so that the FNN receives a single combined representation. By doing so, the transformer effectively captures nuanced dependencies without relying on the sequence-based processing of recurrent networks.

An important piece of the transformer’s design is positional encoding. Because the model does not explicitly track word order (as RNNs do), it instead adds a special vector to each word embedding. These vectors use sine and cosine functions at different frequencies to encode positional information, allowing the network to learn how words relate to one another by position. This positional encoding remains crucial for tasks requiring an understanding of word order. The transformer’s original paper, “Attention is All You Need”, provides formulae for these encodings, and various implementations (e.g., Tensor2Tensor, PyTorch) illustrate how they help the model generalize to longer sentences.

Early sequence-to-sequence models relied on a single context vector from the encoder, which became a bottleneck when processing long sentences. [Bahdanau et al. \(2015\)](#) and [Luong et al. \(2015\)](#) addressed this limitation by introducing and refining an “Attention” mechanism that selectively focuses on relevant segments of the input sequence. For example, at time step 7 in the decoder, the model can amplify the signal associated with *étudiant* (French for “student”) before generating its translation. This dynamic selection allows attention-based models to achieve better translation quality than those relying solely on a final encoder hidden state.

An attention-based model differs from a classic sequence-to-sequence approach in two main ways: (1) the encoder sends all its hidden states to the decoder, and (2) the decoder uses these states in an extra step at every time step. It scores each encoder’s hidden state (loosely tied to each input word), multiplies them by their softmaxed scores to emphasize relevant states, and then sums them into a context vector. This context vector, combined with the decoder’s hidden state, helps determine the next output word. The model learns from training data how to associate words across languages precisely.

The transformer follows this pattern but replaces recurrent or convolutional layers with attention and position-wise feedforward layers, as shown in [Figure 2.35](#).

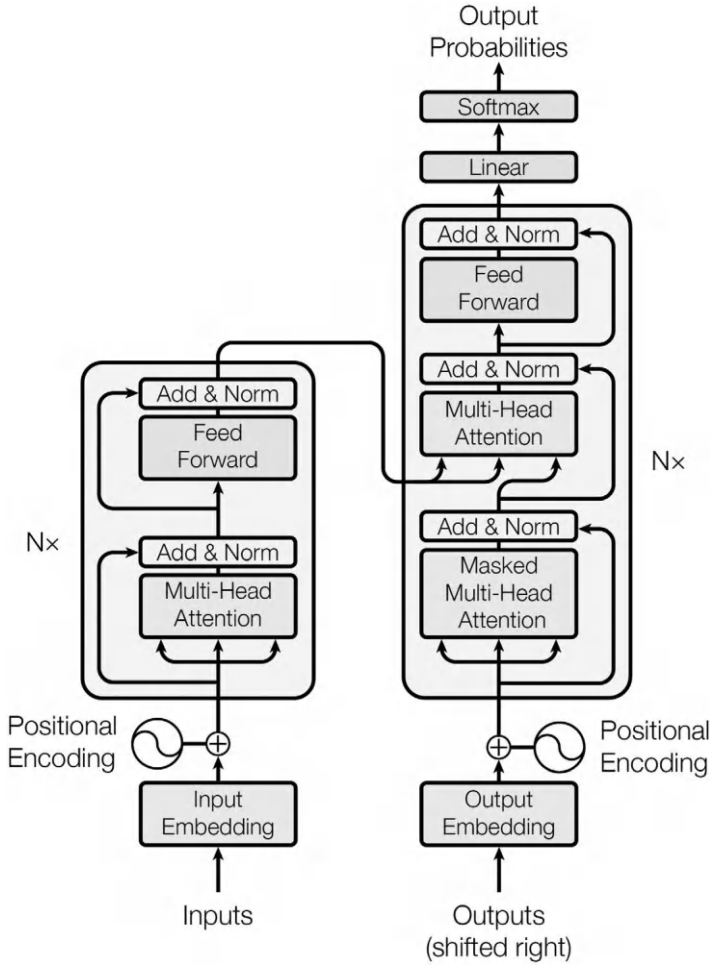


Figure 2.35 The attention-based transformer.

Source: [https://en.wikipedia.org/wiki/Transformer_\(deep_learning_architecture\)#/media/File:Transformer_encoder_with_norm-first_and_norm-last.png](https://en.wikipedia.org/wiki/Transformer_(deep_learning_architecture)#/media/File:Transformer_encoder_with_norm-first_and_norm-last.png)

2.4.6.1 Encoder stack

The encoder is composed of $N=6$ identical layers. Each layer has two sub-layers:

- **Multi-head self-attention:** Every position in the encoder can attend to every other position in the previous layer, forming a self-attention mechanism.
- **Position-wise FNN:** A simple feedforward function applied to each position independently:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2 \quad FFN(x) \quad (2.41)$$

where:

W_1 are learned parameter matrices;
 b_1, b_2 are bias terms.

A residual connection (He et al., 2016) surrounds each sub-layer, followed by layer normalization (Ba et al., 2016). The dimensionality of all sub-layer outputs is $d_{model} = 512$.

2.4.6.2 Decoder stack

The decoder is also composed of $N = 6$ identical layers. Each layer has:

- Masked multi-head self-attention. The decoder attends to all positions up to the current one, preventing “peeking” at future tokens. This is done by setting attention logits for illegal connections to $-\infty$;
- Encoder-decoder attention;
- Queries coming from the decoder’s previous sub-layer, while keys and values come from the encoder’s output. This mechanism allows the decoder to focus on relevant parts of the encoded input sequence;
- Position-wise FNN;
- The same structure as in the encoder. Residual connections and layer normalization are used around each sub-layer.

2.4.6.3 Scaled dot-product attention

An attention function maps a query q and a set of key-value pairs $\{(k_i, v_i)\}$ to an output, computed as a weighted sum of the values:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (2.42)$$

2.4.6.4 Multi-head attention

Instead of a single attention function, the transformer uses multi-head attention, splitting Q, K, V into h parts (heads). Each head performs attention in a lower-dimensional subspace (d_k, d_v) , then the outputs are concatenated and projected:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \quad (2.43)$$

where:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$$

Multi-head attention allows the model to jointly attend to information from different representation subspaces.

2.4.6.5 Embeddings and positional encoding

Tokens are first mapped into learned embeddings of dimension d_{model} . Because the transformer has no recurrence or convolution, a positional encoding is added to each embedding to inject sequence order:

$$PE_{(pos, 2i)} = \sin\left(pos/10000^{2i/d_{model}}\right) \quad (2.44)$$

$$PE_{(pos, 2i+1)} = \cos\left(pos/10000^{2i/d_{model}}\right) \quad (2.45)$$

These sinusoidal encodings allow the model to learn relative position information.

2.4.7 Reinforcement learning

The idea behind the reinforcement learning may be stated in the simplest form as: “OK, let’s play a game a few times; if I lose, I will remember what I’ve done wrong and if I win, I will remember what I’ve done right and try to optimize next time”.¹ The term “reinforcement learning” was introduced by [Minsky \(1954\)](#) and popularized by [Sutton \(1984\)](#), [Sutton and Barto \(1998\)](#) who are widely recognized for formalizing and popularizing the concept of reinforcement learning in the field of artificial intelligence and machine learning. The overall aim of reinforcement learning is to maximize cumulative rewards through optimal decision-making strategies.

In early studies, the environment was typically stated in the form of a Markov decision process (see e.g. [Puterman, 1994](#)), as many reinforcement learning algorithms were using dynamic programming techniques (see [van Otterlo & Wiering, 2012](#)).

[Sutton and Barto \(2018\)](#) distinguished the main families of reinforcement learning:

- Tabular solutions methods:
 - Multi-arm bandit problem ([Thathachar & Sastry, 1985](#));
 - Finite Markov decision process ([Bellman, 1957b](#); [Howard, 1960](#));
 - Dynamic programming ([Bellman, 1957a](#));
 - Monte Carlo methods;
 - Temporal-difference learning ([Watkins & Dayan, 1992](#)).
- Approximate solution methods.

Some authors also add genetic algorithms ([Holland, 1975](#); [Goldberg, 1989](#)) as a domain of reinforcement learning (see [Duarte et al., 2022](#)).

From today’s point of view, the first group may be stated as “traditional” reinforcement learning methods while the second one is highly related to deep neural network approaches.

One of the core concepts of reinforcement learning is the Bellman equation (Bellman, 1957a), which is a fundamental concept in reinforcement learning and dynamic programming decomposing the value function estimating iteratively the value of the state or action function in terms of instant reward or punishment. The Bellman equation can be formulated in two forms expressing the value of a state or state-action pair.

Bellman equation for the state-value function $V(s)$:

$$V(s) = E[R + \gamma V(s') | s] \quad (2.46)$$

where:

- $V(s)$: The value of being in state s , representing the expected cumulative reward starting from s and following the optimal policy;
- E : The expectation operator, averaging over all possible outcomes given the current state s ;
- R : The immediate reward received after taking an action in state s ;
- γ The discount factor ($0 \leq \gamma < 1$), which determines the importance of future rewards relative to immediate rewards;
- s' : The next state after transitioning from state s .

This equation indicates that the value of a state s is the expected sum of the immediate reward R and the discounted value of the next state s' .

Bellman equation for the action-value function $Q(s, a)$:

$$Q(s, a) = E[R + \gamma \max_{a'} Q(s', a') | s, a] \quad (2.47)$$

where:

- $Q(s, a)$: The value of taking action a in state s , representing the expected cumulative reward starting from s , taking action a , and following the optimal policy thereafter;
- E : The expectation operator, averaging over all possible outcomes given the current state s and action a ;
- R : The immediate reward received after taking action a in state s ;
- γ The discount factor ($0 \leq \gamma < 1$), which determines the importance of future rewards relative to immediate rewards;
- s' : The next state after taking action a in state s ;
- $\max_{a'} Q(s', a')$: The maximum expected future reward for the next state s' , assuming the agent takes the best possible action a' in s' .

This equation states that the value of taking action a in state s is the expected sum of the immediate reward R and the discounted value of the next state s' , assuming the agent acts optimally thereafter.

One of the most popular yet powerful techniques of reinforcement learning is q-table, based on the Bellman equation. According to [Sutton and Barto \(2018\)](#) it belongs to the temporal difference learning family and based on Q-learning algorithm ([Watkins, 1989](#); [Watkins & Dayan, 1992](#)) which iteratively updates action-value estimates to learn an optimal policy by maximizing expected cumulative rewards. The pseudocode of Q-learning is presented as Algorithm 2.7.

Algorithm 2.7 Q-learning algorithm

Repeat (for each episode):

- Initialize (S)
- Repeat (for each step of episode):
 - Choose A from S using policy derived from Q (e.g., ϵ -greedy)
 - Take action A , observe R, S'
 - $Q(S, A) \leftarrow Q(S, A) + \alpha \left[R + \gamma \max_a Q(S', a) - Q(S, A) \right]$
 - $S \leftarrow S'$
- until S is terminal

where:

$Q(s,a)$: The action-value function, representing the expected cumulative reward for taking action aa in state ss and following the optimal policy thereafter.

S : The set of all possible states in the environment;

$A(s)$: The set of all possible actions available in state ss ;

terminal-state: A state where the episode ends, and no further actions are taken;

s : The current state of the agent;

A : The action chosen by the agent in state s ;

R : The immediate reward received after taking action A in state s ;

S' : The next state observed after taking action A in state s ;

α : The learning rate ($0 < \alpha \leq 1$), controlling how much new information overrides old Q-values;

γ : The discount factor ($0 \leq \gamma < 1$), determining the importance of future rewards relative to immediate rewards;

$\max_a Q(S', a)$: The maximum expected future reward for the next state S' , used to update the current Q-value;

ϵ -greedy: A policy that balances exploration (choosing random actions) and exploitation (choosing the best-known action) by selecting a random action with probability ϵ and the action with the highest Q-value with probability $1 - \epsilon$.

The Reinforcement learning iterative loop is presented if [Figure 2.36](#).

Q-learning algorithm remembers the action values function values in subsequent steps in Q-tables so sometimes the applications of Q-learning are referred to as Q-table methods.

To observe Q-tables in action, the classical example of tic-tac-toe (see e.g. [Sutton and Barto, 2018](#), chapter 1.5) player will be implemented in native Python

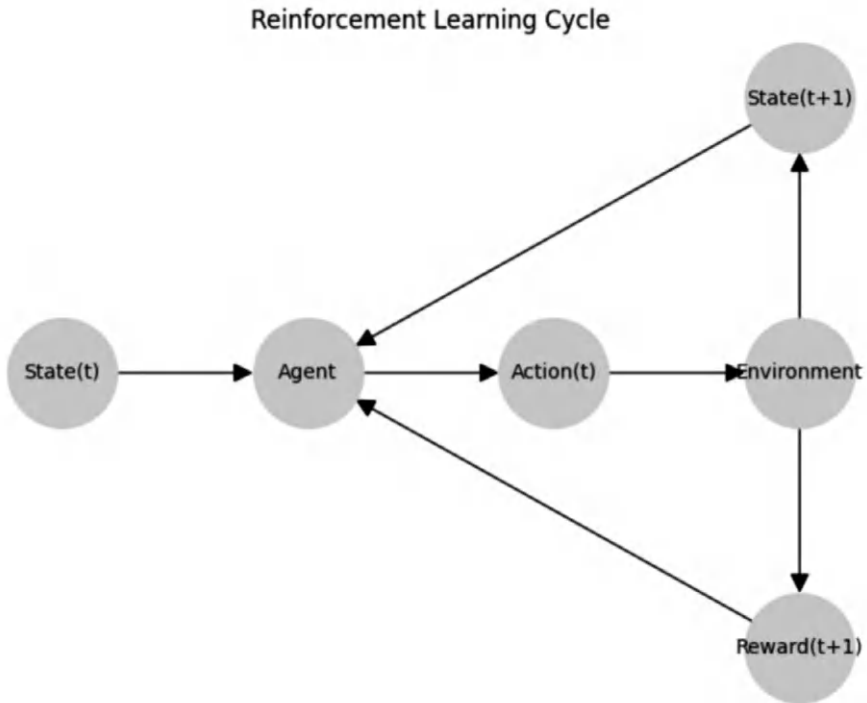


Figure 2.36 Reinforcement learning cycle.

Source: Own work, with Python/NetworkX.

graphical user interface (GUI) framework – *tkinter* library. No external machine learning libraries are used since Q-table is implemented internally.

```

import tkinter as tk
from tkinter import messagebox
import random
import pickle
import os
from PIL import ImageGrab

class TicTacToe:
    def __init__(self, master):
        self.master = master
        self.master.title("Tic Tac Toe")

        # Reinforcement learning parameters
        self.alpha = 0.1      # learning rate
        self.epsilon = 0.2    # exploration rate
  
```

```

self.q_table_file = "ttt_q_table.pkl"
self.q_table = self.load_q_table()

# To record each computer move during a game for Q-
learning updates
self.game_history = []

# For saving screenshots per move
self.game_number = 1
self.step_number = 0

self.buttons = {}
self.board = [[None for _ in range(3)] for _ in
               range(3)]
self.game_over = False

for i in range(3):
    for j in range(3):
        button = tk.Button(master, text="",
font=("Helvetica", 24), width=5, height=2,
                                command=lambda i=i, j=j:
self.player_move(i, j))
            button.grid(row=i, column=j)
            self.buttons[(i, j)] = button

self.current_turn = random.choice(["player",
                                   "computer"])
if self.current_turn == "computer":
    self.master.after(500, self.computer_move)

# Save screenshot of the initial state
self.master.after(500, self.save_screenshot)

# Save Q-table on closing the window
self.master.protocol("WM_DELETE_WINDOW", self.
    on_closing)

def get_state_string(self):
    """Return a string representation of the board.
    Empty cells are represented by '-'. """
    state = ""
    for row in self.board:
        for cell in row:
            state += cell if cell is not None else '-'
    return state

def player_move(self, i, j):
    if self.game_over or self.board[i][j] is not None or
self.current_turn != "player":
        return

```

```

self.board[i][j] = "O"
self.buttons[(i, j)].config(text="O",
                             state="disabled")
self.save_screenshot() # Save screenshot after
player's move

if self.check_winner("O"):
    self.game_over = True
    # Reward for computer is -1 if player wins
    self.update_q_table(-1)
    self.show_result("You win! ")
    return
elif self.is_draw():
    self.game_over = True
    self.update_q_table(0)
    self.show_result("Draw! ")
    return

self.current_turn = "computer"
self.master.after(500, self.computer_move)

def computer_move(self):
    """Make a move for the computer using the Q-learning
model. """
    if self.game_over:
        return

    state = self.get_state_string()

    # Find all free cells (actions) as (i, j) tuples
    free_cells = [(i, j) for i in range(3) for j in
range(3) if self.board[i][j] is None]
    if not free_cells:
        return

    # Ensure the state exists in the Q-table; initialize
missing actions to 0.
    if state not in self.q_table:
        self.q_table[state] = {action: 0.0 for action in
free_cells}
    else:
        for action in free_cells:
            if action not in self.q_table[state]:
                self.q_table[state][action] = 0.0

    # Epsilon-greedy selection: explore or exploit
    if random.random() < self.epsilon:
        action = random.choice(free_cells)
    else:
        q_values = self.q_table[state]

```

```

        max_q = max(q_values.values())
        best_actions = [act for act, q in q_values.
items() if q == max_q]
        action = random.choice(best_actions)

    # Record the move for later learning update
    self.game_history.append((state, action))

    # Execute the move: Computer uses X
    i, j = action
    self.board[i][j] = "X"
    self.buttons[(i, j)].config(text="X", state="disabled")
    self.save_screenshot() # Save screenshot after com-
puter's move

    if self.check_winner("X"):
        self.game_over = True
        # Reward for computer win is +1
        self.update_q_table(1)
        self.show_result("Computer wins!")
        return
    elif self.is_draw():
        self.game_over = True
        self.update_q_table(0)
        self.show_result("Draw!")
        return

    self.current_turn = "player"

def check_winner(self, mark):
    for i in range(3):
        if all(self.board[i][j] == mark for j in range(3)):
            return True
        if all(self.board[j][i] == mark for j in range(3)):
            return True
    if all(self.board[i][i] == mark for i in range(3)):
        return True
    if all(self.board[i][2 - i] == mark for i in range(3)):
        return True
    return False

def is_draw(self):
    return all(self.board[i][j] is not None for i in
range(3) for j in range(3))

def update_q_table(self, reward):
    """Update the Q-table for all computer moves made
during this game.
```

```

        A simple update:  $Q(s, a) = Q(s, a) + \alpha * (reward - Q(s, a))$  """
        for state, action in self.game_history:
            if state in self.q_table and action in
self.q_table[state]:
                current_q = self.q_table[state][action]
                self.q_table[state][action] = current_q +
self.alpha * (reward - current_q)
                self.game_history = []

    def show_result(self, message):
        answer = messagebox.askyesno("Game Over",
f"{message}\nDo you want to restart? ")
        if answer:
            self.restart_game()
        else:
            for btn in self.buttons.values():
                btn.config(state="disabled")

    def restart_game(self):
        for i in range(3):
            for j in range(3):
                self.board[i][j] = None
                self.buttons[(i, j)].config(text="",
                    state="normal")
            self.game_over = False
            self.game_history = []
            # Increment game counter and reset step counter
            self.game_number += 1
            self.step_number = 0

        # Randomly decide who starts the new game
        self.current_turn = random.choice(["player",
            "computer"])
        if self.current_turn == "computer":
            self.master.after(500, self.computer_move)
        # Save screenshot for the new (empty) board
        self.master.after(500, self.save_screenshot)

    def load_q_table(self):
        """Load the Q-table from a file if it exists, other-
        wise return an empty dict."""
        if os.path.exists(self.q_table_file):
            try:
                with open(self.q_table_file, "rb") as f:
                    q_table = pickle.load(f)
                    print("Q-table loaded. ")
                    return q_table

```

```

        except Exception as e:
            print("Error loading Q-table:", e)
        print("Starting with an empty Q-table.")
        return {}

    def save_q_table(self):
        """Save the Q-table to a file. """
        try:
            with open(self.q_table_file, "wb") as f:
                pickle.dump(self.q_table, f)
                print("Q-table saved.")
        except Exception as e:
            print("Error saving Q-table:", e)

    def save_screenshot(self):
        """Capture and save a screenshot of the current
        window."""
        # Update the window to make sure all changes are drawn
        self.master.update_idletasks()
        x = self.master.winfo_rootx()
        y = self.master.winfo_rooty()
        w = self.master.winfo_width()
        h = self.master.winfo_height()
        bbox = (x, y, x + w, y + h)
        filename = f"screenshot_{self.game_number}_{self.
step_number}.png"
        try:
            # Capture the screenshot of the defined bounding box
            img = ImageGrab.grab(bbox)
            img.save(filename)
            print(f"Saved screenshot: {filename}")
        except Exception as e:
            print("Error capturing screenshot:", e)
        self.step_number += 1

    def on_closing(self):
        """Save the Q-table before closing the application."""
        self.save_q_table()
        self.master.destroy()

root = tk.Tk()
game = TicTacToe(root)
root.mainloop()

```

The results of the game with untrained, improved, and completely trained programs are presented respectively in [Figures 2.37–2.39](#).

For the game of tic tac toe, there are quite a few possible games, so teaching the model a non-losing strategy is a matter of approximately a few hundred games.

							O	
				X			X	
			O			O		
	O	X		O	X		O	X
	X			X		X	X	
			O					
				O	X			
			X	X	O			

Figure 2.37 Tic tac toe with Q-table reinforcement learning – simple lost game, model not trained.

Source: Own work, with Python/tkinter.

				O			O	
								X
								O
	O	O		O	O		O	O
		X		X	X		X	X
X		O	X	O	O	X	O	O
	O	O		O	O	X	O	O
	X	X		X	X		X	X
			X	O	O			
			X	O	O			
			O	X	X			

Figure 2.38 Tic tac toe with Q-table reinforcement learning – more complex lost game, model in training process.

Source: Own work, with Python/tkinter.

							○	
			x			x		
x			x			x		
	○			○			○	○
x			x			x	x	
			x	○		x	○	
				○	○	x	○	○
			x	x		x	x	

Figure 2.39 Tic tac toe with Q-table reinforcement learning – more complex win game after improvement.

Source: Own work, with Python/tkinter.

(Anyway, strategy can be improved by procedures that find obvious moves, such as completing and blocking three of your own squares or the opponent's.) In the case of more complex games such as chess, Go, fighting a fighter jet with an opponent, reaching a destination in an autonomous vehicle, performing the task of correctly cutting a part from the material by a robot, or achieving maximum profit in cryptocurrency trading, both the number of possible moves and parameters to take into account when making decisions are much greater, so advanced data analysis techniques such as deep neural networks should be used for optimization.

In their pioneering work [Mnih et al. \(2015\)](#) introduced Deep Q-Network which was used on CNN to learn directly from raw pixel inputs in 49 Atari 2600 games such as Kung Fu Master, Ms. Pacman or Enduro. Deep Q-Network achieved human-level performance in player simulation for each game.

Deep Q-Learning extends Q-learning by replacing the Q-table with a deep convolutional neural network that approximates the Q-values, allowing the algorithm to handle high-dimensional state spaces and scale to more complex environment.

The Q-network was trained by minimizing the mean-squared error in the Bellman equation. The target values for training are given by:

$$y = r + \gamma \max_{a'} Q(s', a'; \theta_i^-) \quad (2.48)$$

where:

θ_i^- represents the network weights from a previous iteration.

Loss function: The loss function at iteration i is defined as:

$$L_i(\theta_i) = E_{s,a,r,s'} \left[\left(y - Q(s,a;\theta_i) \right)^2 \right] \quad (2.49)$$

This function measures the difference between the predicted Q-values and the target values.

Optimization: During optimization, the parameters from the previous iteration (θ_i^-) are held fixed. This results in a sequence of well-defined optimization problems.

Variance term: The variance of the targets does not depend on the current parameters θ_i and can be ignored during optimization.

This process is central to Deep Q-Learning, where the Q-network is iteratively updated to better approximate the optimal Q-values, enabling the agent to learn effective policies in complex environments.

Deep Q-Learning has revolutionized reinforcement learning. The integration of deep learning with reinforcement learning, commonly referred to as deep reinforcement learning, has led to breakthroughs in fields such as robotics, gaming, healthcare, and finance.

Deep reinforcement learning enabled robots to learn complex tasks such as locomotion, manipulation, and dexterous hand control by processing visual and sensor data (Kalashnikov et al., 2018; Levine et al., 2016) and allowed vehicles to learn navigation and decision-making policies in simulated and real-world environments (Kiran et al., 2021; Sallab et al., 2017).

Deep reinforcement learning powered by deep neural networks is widely used for computer and “analog games” with eminent examples being StarCraft II (Vinyals et al., 2019), Gran Turismo (Wurman et al., 2022), Go (Silver et al., 2016, 2017a), and chess (Silver et al., 2017b).

Moody and Saffel (2001) introduced Recurrent Reinforcement Learning algorithm based on backpropagation for discovering investment policies and benchmarking against Q-learning algorithm on USD/British pound trading system and S&P 550 / T-Bill Asset Allocation trading. Since then, there have been numerous examples of the application of “ordinal” and Deep Reinforcement Learning in economics with a focus on finance. Table 2.8 summarizes these applications with eminent (or illustrative) examples.

Deep reinforcement learning is used in healthcare for optimizing treatment policies, robotic-assisted surgeries, and in drug discovery by modeling sequential decision-making (Yu et al., 2021)

Table 2.8 Reinforcement learning and deep reinforcement learning in economics/finance

<i>Application</i>	<i>Description</i>	<i>References</i>
Portfolio management and trading	Reinforcement learning is used to optimize asset allocation, trade execution, and risk management by learning from historical market data.	Moody & Saffell (2001), Deng et al. (2016)
Market microstructure and high-frequency trading	Reinforcement learning is applied on 1.5 years of millisecond time-scale limit order data from NASDAQ to design strategies for optimal trade scheduling in fast-paced, high-frequency trading environments.	Nevmyvaka et al. (2006)
Auction design and bidding strategies	Reinforcement learning techniques help develop adaptive bidding strategies and optimize auction outcomes in various formats, including online and dynamic auctions.	Wei et al. (2022); Bian et al. (2024); Liu et al. (2024)
Consumer behavior and decision-making	Reinforcement learning captures consumer decision processes by simulating learning from interactions and adapting to changing preferences under uncertainty.	Liu et al. (2018)
Dynamic pricing and revenue management	Development of dynamic pricing strategies by continuously adapting to market signals, consumer behavior, and competitive actions.	Besbes and Zeevi (2009)
Placement optimization in chip designment	Reinforcement learning is used for optimizing the placement of substitutable products in designated display locations to maximize the expected revenue of the seller.	Mirhoseini et al. (2017)

Source: Own compilation.

Reinforcement learning with deep networks improves dialogue systems, chat-bots, and machine translation (Bahdanau et al., 2016; Li et al., 2017).

2.4.8 *Other types of neural networks*

There are several types of neural networks that were commonly used mainly in 20th/21st century break, but that currently have a smaller range of applications, comparing RNNs’, CNNs’, and transformer-based network’s popularity.

Boltzmann Machine: A stochastic, energy-based neural network with undirected connections, Boltzmann Machines learn a probability distribution over data by minimizing an energy function. They use Gibbs sampling and contrastive divergence for training, making them useful for unsupervised learning and feature learning (Hinton & Sejnowski, 1986).

Restricted Boltzmann Machine: A simplified Boltzmann Machine with a two-layer architecture – visible (input) and hidden layers – but no intra-layer connections. Restricted Boltzmann Machines are used for feature extraction, collaborative filtering, and deep learning pre-training ([Smolensky, 1986](#)).

Deep belief networks: A hierarchical stack of Restricted Boltzmann Machines trained layer-by-layer using unsupervised pre-training followed by supervised finetuning with backpropagation. Deep belief networks are widely used in dimensionality reduction, speech recognition, and generative modeling (Hinton et al., 2006).

Self-organizing maps or Kohonen’s map: An unsupervised neural network that maps high-dimensional data into a lower dimensional (usually 2D) topology-preserving representation. Self-organizing maps use competitive learning and are applied in clustering, visualization, and anomaly detection ([Kohonen, 1982](#)).

Growing Neural Gas: An adaptive neural network that dynamically adds and removes neurons while learning, making it more flexible than self-organizing maps. Growing Neural Gas was widely used in unsupervised clustering, data visualization, and topological learning ([Fritzke, 1995](#)).

Spiking neural networks and liquid state machine: A biologically inspired model where neurons communicate via spikes (discrete events) rather than continuous activations. Spiking neural networks simulate synaptic plasticity and are used in neuromorphic computing, and energy-efficient liquid state machine is a spiking neural network variant where an input signal excites a randomly connected “liquid” (reservoir) of neurons, generating spatiotemporal responses ([Maass, 1997](#); [Maass et al., 2002](#)).

2.5 Under and overfitting avoiding techniques for neural networks

A generalization is one of the most important concepts of quantitative methods, for general and deep learning models, in particular. Generalization assumes not only the best fitting of the model to the training dataset, but also, even more importantly, the maximization of overall performance scores and adaption to possible test set values (equivalent to the minimalization of generalization error). It may sound completely unintuitive, but sometimes models with higher training errors may be better in the sense of generalization than objects with lower train errors but better performance metrics. The generalization problem is not strictly serious, but (as the author hopes) its illustrative form is presented in [Figure 2.40](#).

- **High bias (underfitting):** This occurs when the model is too simple to capture the underlying patterns in the data.
- **High variance (overfitting):** This occurs when the model is too complex and captures noise in the training data.

The lack of generalization is often related to overfitting or underfitting. It happens when the model is too simple for the proper model training or, in contrast, when the model learns too much from the training data, capturing details that do not necessarily generalize on instances not belonging to the training dataset.

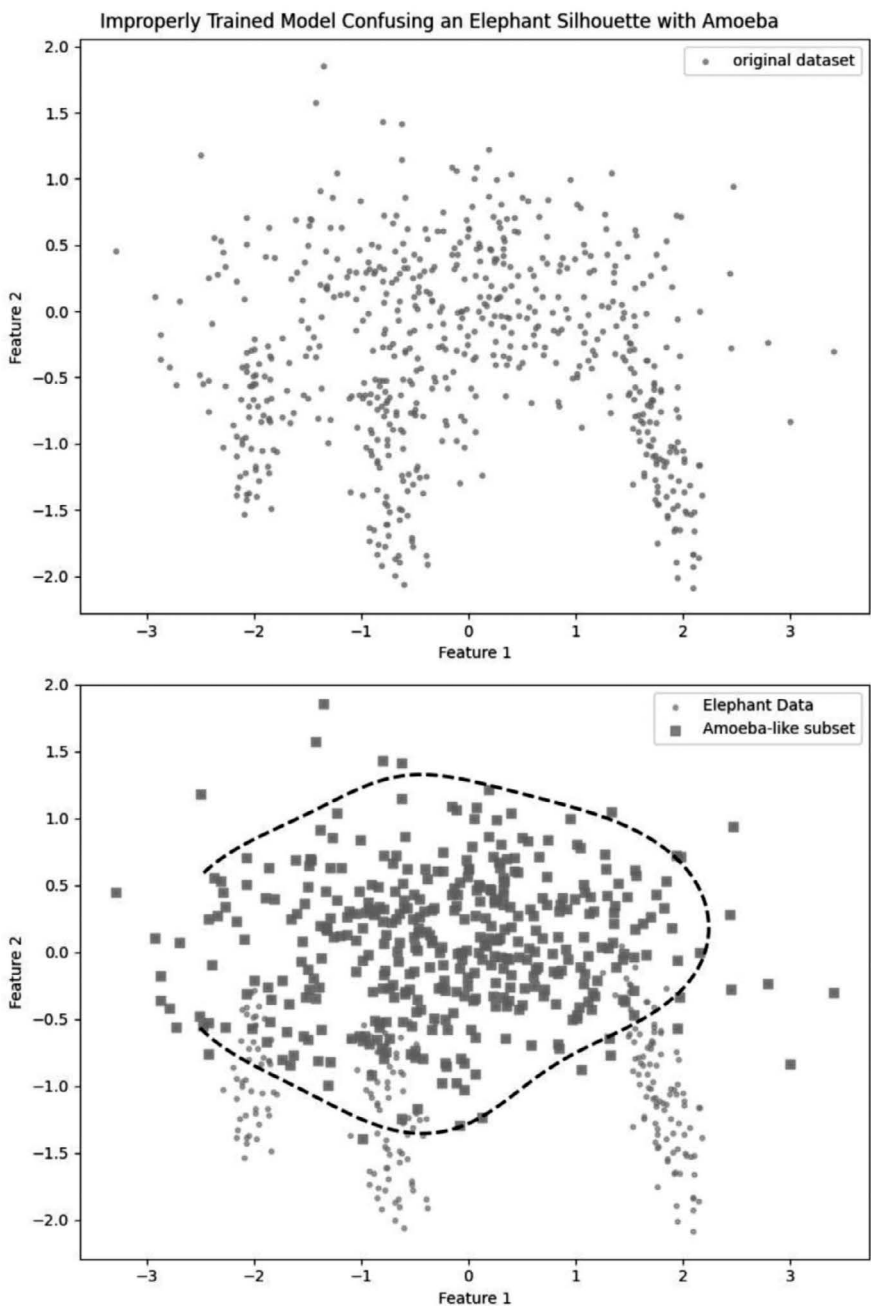


Figure 2.40 Generalization problem illustrated, with support vector classifier and non-random training set selection.

Source: Own sample, source code available in [Appendix 2](#).

Underfitting happens when the network lacks a number of hidden layers, the layers contain too small a number of neurons, or the regularization routines are too complex. Underfitting in economic forecasting/prediction leads to unreliable predictions, ignoring critical trends and patterns essential for accurate decision-making.

Overfitting happens when a deep neural network learns too much from the training data, including random noise and irrelevant details. Among the reasons that may be listed are overfitting too complex a model, too many parameters, insufficient training data, too noisy data, or lack of regularization. Overfitted models may have low training errors but high test errors. An overfitted model stores data instead of generalizing them. In finance, an overfitted model may perform well on historical stock prices, but fail to predict future market trends.

In some sense proper model design is some form of finding a balance between avoiding underfitting and avoiding overfitting.

One of the most effective ways to prevent overfitting is regularization, which penalizes excessive model complexity. Deepfellow (2016) defines regularization as “any modification we make to a learning algorithm that is intended to reduce its generalization error but not its training error”. L1 (LASSO) and L2 (Ridge) regularization add constraints to weight updates, reducing sensitivity to noise. Dropout is another technique that randomly disables neurons during training, improving robustness of data representation.

Data augmentation (rotations, scaling, or noise injection to existing samples) may make the model more robust. Early stopping monitors validation loss and halts training when performance starts degrading, preventing the model from memorizing noise. Batch normalization stabilizes training by normalizing activations, improving generalization and training efficiency.

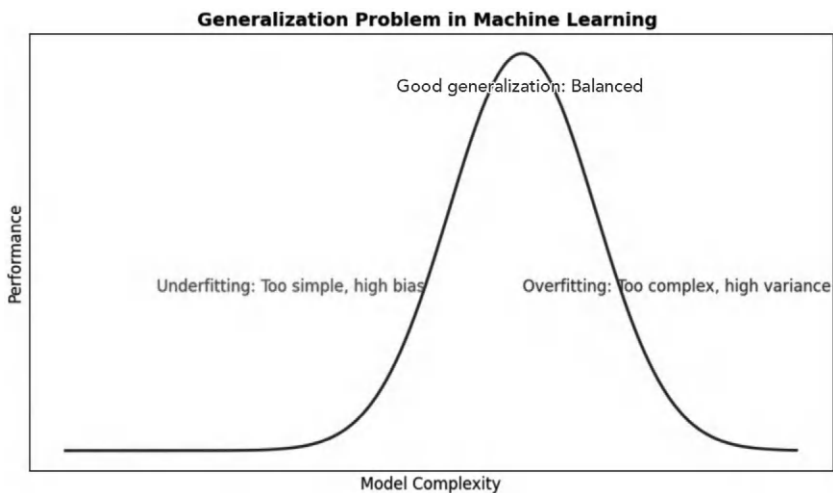


Figure 2.41 Generalization problem equilibrium.

Source: Own research with use of Python/Matplotlib.

2.5.1 L2 regularization (weight decay)

L2 regularization, also known as weight decay, ridge regression, or Tikhonov regularization, encourages the model weights to stay small by penalizing the squared values of the weights. The regularization term added to the objective function is:

$$\Omega(\theta) = \frac{1}{2} \|w\|_2^2 = \frac{1}{2} \sum_i w_i^2 \quad (2.50)$$

This modifies the total objective function as:

$$\tilde{J}(w; X, y) = \frac{\alpha}{2} w^T w + J(w; X, y) \quad (2.51)$$

where:

α is the regularization strength.

The gradient update rule with L2 regularization is:

$$\nabla_w \tilde{J}(w; X, y) = \alpha w + \nabla_w J(w; X, y) \quad (2.52)$$

Thus, during gradient descent, the weight update step becomes:

$$w \leftarrow (1 - \eta\alpha)w - \eta \nabla_w J(w; X, y) \quad (2.53)$$

which shows that L2 regularization shrinks the weights multiplicatively by a constant factor in each step, preventing overly large weights.

For linear regression, the optimal solution changes from $w = (X^T X)^{-1} X^T y$ to $w = (X^T X + \alpha I)^{-1} X^T y$

where I is the identity matrix. This effectively reduces variance and makes the model less sensitive to noise.

L2 regularization is useful when we want to reduce complexity, but still retain all features. Shrinking of all weights keeps all features but reduces impact.

2.5.2 L1 regularization (LASSO)

L1 regularization, also known as LASSO (least absolute shrinkage and selection operator – see [Tibshirani, 1996](#)), penalizes the absolute values of the weights rather than their squared values:

$$\Omega(\theta) = \|w\|_1 = \sum_i |w_i| \quad (2.54)$$

The regularized objective function becomes:

$$\tilde{J}(w; X, y) = \alpha \|w\|_1 + J(w; X, y) \quad (2.55)$$

The gradient update is:

$$\nabla_w \tilde{J}(w; X, y) = \alpha \text{sign}(w) + \nabla_w J(w; X, y) \quad (2.56)$$

where:

$\text{sign}(w)$ applies element-wise, meaning the gradient is constant rather than proportional to the weight size.

Unlike L2 regularization, L1 encourages sparsity by forcing many weights to become exactly zero. The optimal solution follows:

$$w_i = \text{sign}(w_i^*) \max\left(|w_i^*| - \frac{\alpha}{H_{i,i}}, 0\right) \quad (2.57)$$

where:

$H_{i,i}$ is the Hessian matrix diagonal element for the i -th weight.

If $|w_i^*|$ is small, L1 regularization shrinks it to zero, effectively performing feature selection.

LASSO regularization is useful when we want sparse models and need to perform feature selection automatically. LASSO often causes sparsity (some weights become zero) effectively eliminating irrelevant features from the model. While it improves interpretability and reduces overfitting, the computational cost can be high, especially with large datasets or many correlated predictors.

A combination of both L1 and L2, called elastic net regularization (Zou & Hastie, 2005), is often used to balance feature selection and smooth weight decay.

$$\Omega(\theta) = \lambda_1 \|w\|_1 + \lambda_2 \|w\|_2^2 \quad (2.58)$$

where λ_1 and λ_2 control the balance between L1 and L2 effects.

2.5.3 Dropout regularization in deep learning

Dropout is a powerful and computationally inexpensive regularization technique for deep learning models, introduced by Srivastava et al. (2014). It aims to approximate an ensemble of neural networks by randomly “dropping out” (deactivating) non-output units during training. This method is inspired by bagging (Breiman, 1994), which trains multiple models separately, but dropout achieves this effect within a single model by randomly deactivating neurons at each training step which is a better alternative to maintaining multiple large neural networks.

During training, dropout applies a binary mask μ to the activations of input and hidden units. The mask elements are sampled independently for each unit, typically with an inclusion probability of 0.8 for input units and 0.5 for hidden units. Mathematically, given a model parameter set θ , the dropout objective function is defined as:

$$\tilde{J}(\theta) = E_{\mu} J(\theta, \mu) \quad (2.59)$$

where:

$J(\theta, \mu) J(\theta, \mu)$ represents the cost function for a specific mask realization.

Since the expectation includes an exponential number of terms, gradient updates are approximated using stochastic sampling of μ , ensuring computational efficiency.

Unlike bagging, where each model is trained separately, dropout forces parameter sharing across an exponentially large set of subnetworks, making it memory-efficient. The effect of dropout can be understood through a Monte Carlo approximation of ensemble predictions. For a classification task, the final prediction is an average over all subnetworks' predictions:

$$p_{\text{ensemble}}(y|x) = \sum_{\mu} p(\mu) p(y|x, \mu) \quad (2.60)$$

where:

$p(\mu)$ – probability of sampling a particular subnetwork μ ;

$p(y|x, \mu)$ – probability that subnetwork μ predicts y given x .

which is computationally intractable. Instead, an approximation using sampling from multiple subnetworks is often sufficient, typically requiring 10–20 masks for a reliable estimate.

A more efficient alternative to ensemble averaging is the weight scaling inference rule, which approximates the expected output using the full network but scales the outgoing weights of each unit by its retention probability p (probability that a neuron is kept, not dropped during dropout). This ensures that the expected activation during training matches that of inference. The modified forward pass uses:

$$\widetilde{W}_i = p W_i \quad (2.61)$$

where:

W_i are the original weights;

p – retention probability.

Dropout has been shown to outperform standard regularization techniques such as L2 weight decay, filter norm constraints, and sparsity penalties. It is also computationally efficient, requiring only proportional to n number of extra operations

per example. However, it reduces model capacity, often requiring a larger network for optimal performance.

2.5.4 Batch normalization in deep learning

Batch normalization introduced by Ioffe and Szegedy (2015), is a powerful technique for stabilizing and accelerating the training of deep neural networks. Unlike traditional optimization algorithms, batch normalization is an adaptive reparameterization method that normalizes the activations of each layer. This technique addresses the challenges of training very deep models, where small parameter updates can have unpredictable effects due to the complex interactions between layers. By normalizing layer activations, batch normalization mitigates the issue of internal covariate shift, where the distribution of inputs to each layer changes during training.

The core idea of batch normalization is to normalize the activations of each layer within a minibatch. Given a minibatch H of activations, batch normalization replaces it with a normalized version:

$$\frac{H - \mu}{\sigma} \quad (2.62)$$

where:

μ and σ are the mean and standard deviation of the activations, respectively. These statistics are computed over the minibatch:

$$\mu = \frac{1}{m} \sum_i H_{i,:}, \quad \sigma = \sqrt{\delta + \frac{1}{m} \sum_i (H_{i,:} - \mu)^2} \quad (2.63)$$

where δ is a small constant to prevent division by zero. This normalization ensures that the activations have zero mean and unit variance, making training more stable.

One of the key innovations of batch normalization is that it allows the network to learn the optimal scale and shift of normalized activations by introducing two learnable parameters, γ and β . Instead of simply using the normalized activations H' , the transformed activations are given by:

$$\tilde{H} = \gamma H' + \beta \quad (2.64)$$

This transformation preserves the expressiveness of the network while ensuring that the optimization dynamics are more stable. The parameters γ and β are trained using gradient descent, allowing the network to determine the most suitable mean and variance.

During training, batch normalization operates on minibatches, and the mean and variance are computed dynamically. However, at test time, using batch statistics is impractical since only one example may be available at a time. Instead, running averages of μ and σ are maintained during training and used during inference, ensuring that the model behaves consistently.

Applying batch normalization significantly simplifies the optimization process by reducing the dependency between parameter updates across layers. Revisiting a deep neural network with activations $\hat{y} = xw_1w_2 \dots w_p$, batch normalization ensures that each hidden activation remains in a stable distribution, preventing gradient explosion or vanishing. This allows for higher learning rates and reduces the sensitivity to weight initialization, making deep networks easier to train.

Batch normalization is widely used in FNNs, CNNs, and RNNs. In convolutional architectures, the normalization is applied consistently across all spatial locations within a feature map to maintain spatial invariance. Batch normalization has become a standard component in modern architectures such as ResNets, transformers, and GANs due to its effectiveness in improving convergence speed and generalization.

There are several other techniques of regularization (see Deepfellow et al., 2016, [Chapter 7](#)):

- **Data augmentation:** Rather than modifying the model, data augmentation artificially increases the training set size by applying transformations such as rotation, flipping, cropping, scaling, or noise injection to existing data. This forces the model to learn robust features rather than memorizing specific patterns.
- **Early stopping:** Early stopping monitors validation performance during training and halts training once the validation loss stops improving (or starts increasing). This prevents the model from overfitting to the training data.
- **Noise injection (Gaussian/label smoothing):** Noise can be introduced into inputs, weights, or labels to act as a regularizer. Gaussian noise injection adds small random noise to the input or hidden layers. Label smoothing replaces hard labels (0,10,1) with soft labels (0.9,0.10.9, 0.1), preventing the network from becoming overconfident.
- **Spectral normalization (Miyato et al., 2018):** This regularization technique limits the spectral norm of the weight matrices to control the Lipschitz continuity of neural networks. It is widely used in GANs to prevent unstable training.

$$W' = \frac{W}{\sigma(W)} \quad (2.65)$$

where:

$\sigma(W)$ is the largest singular value of the weight matrix W .

- **Weight pruning and sparse regularization:** Weight pruning eliminates unimportant weights by setting them to zero, effectively reducing the model size and computation requirements. This is closely related to sparse learning (like LASSO), where only a small fraction of parameters remain active.
- **Stochastic depth:** Randomly drops entire residual blocks in deep neural networks, reducing computational complexity and preventing overfitting. It extends the idea of dropout, but applies at the layer level instead of the unit level.

- **Jacobian regularization:** Encourages the model to learn smooth mappings by penalizing the sensitivity of predictions with respect to the input. It minimizes the Jacobian norm:

$$\Omega(\theta) = \left\| \nabla_x f(x; \theta) \right\|_F^2 \quad (2.66)$$

where:

$\|\cdot\|_F$ denotes the Frobenius norm.

- **Adversarial training:** Involves training the model on both clean and adversarially perturbed inputs, improving robustness against adversarial attacks.
- **Cutout and Mixup regularization:** Randomly removes patches of an image during training to make the model robust and linearly interpolates between two training samples (x_i, y_i) and (x_j, y_j) to create a new sample:

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j, \quad \tilde{y} = \lambda y_i + (1 - \lambda) y_j \quad (2.67)$$

2.6 Selected architectures of neural networks

Deep learning has numerous predefined architectures that have been optimized for various tasks, particularly in computer vision and NLP. Below is a compact description of major architectures.

2.6.1 ImageNet

ImageNet is not a model, but a sizeable visual database designed for use in visual object recognition software research. More than 14 million images have been hand-annotated by the project to indicate what objects are pictured. In at least one million of the images, bounding boxes are also provided. ImageNet contains more than 20,000 categories. It is referenced in many predefined architectures.

2.6.2 VGGNet (Visual Geometry Group Network)

VGGNet, proposed by [Simonyan and Zisserman \(2014\)](#), is a deep CNN with very small 3×3 filters stacked to increase depth. It comes in variations like VGG-16 and VGG-19, referring to the number of layers. Though computationally expensive due to a high number of parameters, it significantly improved ImageNet classification accuracy.

2.6.3 AlexNet

[Krizhevsky et al. \(2012\)](#) introduced a CNN architecture for ImageNet classification. It consists of five convolutional layers followed by three fully connected layers and introduced ReLU activations, dropout, and overlapping max pooling. AlexNet popularized deep learning for computer vision.

2.6.4 ResNet (Residual Network)

Introduced by [He et al. \(2016\)](#), ResNet addresses the vanishing gradient problem using residual connections (skip connections). The key idea is to learn identity mappings, allowing much deeper networks (e.g., ResNet-50, ResNet-101, ResNet-152) without degradation in performance. It achieved state-of-the-art accuracy on ImageNet.

2.6.5 GoogLeNet (Inception)

Introduced by [Szegedy et al. \(2015\)](#), the GoogLeNet/Inception architecture uses parallel convolutional layers of multiple sizes (1×1, 3×3, 5×5) in Inception modules, improving efficiency by capturing multi-scale spatial information. It significantly reduces computational costs while achieving high performance on ImageNet. Variants include Inception-v3, Inception-v4, and Inception-ResNet.

2.6.6 MobileNet

CNN was designed for mobile and edge devices, and was introduced by [Howard et al. \(2017\)](#). It uses depthwise separable convolutions to reduce computational cost while maintaining accuracy. Variants like MobileNetV2 and MobileNetV3 further optimize performance for on-device inference.

2.6.7 DenseNet (Densely Connected Convolutional Network)

[Huang et al. \(2017\)](#) proposed DenseNet, which improves feature reuse by connecting each layer to every other layer in a feedforward fashion. This enhances gradient flow, reduces overfitting, and requires fewer parameters than traditional CNNs. Variants include DenseNet-121, DenseNet-169, DenseNet-201.

2.6.8 EfficientNet

As proposed by [Tan and Le \(2019\)](#), EfficientNet optimizes performance using compound scaling, which jointly scales width, depth, and resolution. Compared to older architectures, EfficientNet achieves higher accuracy with fewer parameters, making it ideal for real-world applications.

2.6.9 Vision Transformer (ViT)

A transformer-based model for image classification was introduced by [Dosovitskiy et al. \(2020\)](#). Instead of convolutions, ViT splits images into patches, processes them with self-attention layers, and achieves state-of-the-art performance on benchmarks like ImageNet.

2.6.10 YOLO (You Only Look Once)

A real-time object detection model was introduced by [Redmon et al. \(2016\)](#). Unlike region-based approaches, YOLO processes the entire image in a single forward

pass, making it extremely fast. It has evolved through versions like YOLOv3, YOLOv4, YOLOv5, and YOLOv8.

2.6.11 U-Net

A fully convolutional neural network designed for image segmentation, particularly in medical imaging. U-Net consists of contracting and expanding paths that allow precise localization and segmentation of objects.

2.6.12 BERT (Bidirectional Encoder Representations from Transformers)

BERT (Devlin et al., 2019) is a transformer-based architecture that uses bidirectional self-attention to understand the context of words in both directions. Pre-trained on large text corpora, BERT is finetuned for various NLP tasks like question answering, text classification, and named entity recognition. It significantly outperforms older architectures like Long Short Term Networks and CNNs for text processing.

2.7 Deep learning computer ecosystem

In this book we use several tools essential for building and running deep learning models especially in Python. The most important ones, crucial for understanding examples in the book, are presented in this section.

2.7.1 NumPy

NumPy is a fundamental Python library for numerical computing, providing support for multi-dimensional arrays, mathematical operations, and linear algebra functions with high efficiency (Harris et al., 2020). It serves as the backbone for scientific computing in Python, offering vectorized operations, broadcasting, and optimized memory management, making it significantly faster than standard Python lists for large-scale computations. NumPy's N-dimensional array (ndarray) structure enables efficient handling of large datasets, facilitating operations such as matrix manipulations, Fourier transforms, and statistical analysis, which are essential in machine learning, data science, and economic modeling. Additionally, its integration with C, Fortran, and other low-level languages ensures seamless interoperability with high-performance computing frameworks. Due to its versatility and speed, NumPy has become the foundation for numerous machine learning and deep learning libraries, including TensorFlow, PyTorch, and scikit-learn. NumPy is described in more detailed way in [Appendix 2](#).

2.7.2 scikit-learn

scikit-learn is a widely used machine learning library in Python, providing efficient implementations of supervised and unsupervised learning algorithms, including classification, regression, clustering, and dimensionality reduction

(Pedregosa et al., 2011). Built on NumPy, SciPy, and Matplotlib, it offers tools for data preprocessing, model selection, and hyperparameter tuning, making it a key resource for applied machine learning. scikit-learn emphasizes simplicity, efficiency, and modularity, enabling seamless integration into research and industry applications. Its versatility makes it suitable for predictive modeling, economic forecasting, and automated decision-making.

2.7.3 *pandas*

pandas is a data manipulation and analysis library designed for structured and time-series data, providing DataFrame and Series objects that allow for fast and flexible data processing (McKinney, 2010). It supports operations such as data cleaning, transformation, merging, and indexing, making it indispensable for data science, financial modeling, and econometrics. pandas integrates seamlessly with NumPy and other libraries, enabling efficient handling of large datasets, which is crucial for machine learning pipelines and real-world data analytics. pandas is described in more detailed way in [Appendix 2](#).

2.7.4 *TensorFlow*

TensorFlow is an open-source deep learning framework developed by Google Brain, designed for scalable and high-performance machine learning (Abadi et al., 2016). It enables efficient computation of deep neural networks across CPUs, GPUs, and tensor processing units (TPUs) through its computational graph-based execution. TensorFlow provides tools for automatic differentiation, distributed training, and model deployment, making it ideal for large-scale AI applications, including image recognition, NLP, and reinforcement learning. Its extensive ecosystem, including TensorFlow Serving and TensorFlow Lite, facilitates real-world AI deployment in research and industry.

2.7.5 *Keras*

Keras is a high-level deep learning API built on top of TensorFlow, designed for rapid prototyping and ease of use in neural network development (Chollet, 2017). It offers an intuitive interface for building, training, and deploying deep learning models, abstracting complex tensor operations while maintaining flexibility for research and production. Keras supports convolutional, recurrent, and transformer-based models, making it popular in computer vision, NLP, and time-series forecasting. Its modular architecture and extensive pre-trained models allow for efficient experimentation, democratizing deep learning adoption in industry and academia.

2.7.6 *PyTorch*

PyTorch is an open-source deep learning framework developed by Facebook AI, known for its dynamic computation graph and ease of use (Paszke et al., 2019). Unlike TensorFlow, which initially relied on static graphs, PyTorch allows for flexible

model building and debugging, making it popular among researchers. It supports automatic differentiation (autograd), GPU acceleration, and distributed training, enabling state-of-the-art deep learning applications in image processing, NLP, and reinforcement learning.

2.7.7 OpenCV/ OpenCV2 (Open Source Computer Vision Library)

OpenCV is an open-source library primarily focused on real-time computer vision, image processing, and machine learning applications, offering a wide range of optimized algorithms for tasks like object detection, face recognition, and image transformations, and is widely used in both research and industry.

2.7.8 Matplotlib

Matplotlib ([Hunter, 2007](#)) is a comprehensive Python library widely used for creating static, animated, and interactive visualizations. It provides a MATLAB-like interface for plotting graphs, charts, and other two- and three-dimensional visualizations, making it a versatile tool for data analysis and scientific research. The library supports a wide range of plot types, including line plots, bar charts, scatter plots, histograms, and more, with extensive customization options for labels, colors, and styles. Matplotlib integrates seamlessly with other Python libraries like NumPy and pandas, enabling efficient data manipulation and visualization workflows. Its object-oriented API and Pyplot module make it accessible for both beginners and advanced users.

2.7.9 Seaborn

Seaborn is a Python data visualization library that extends the functionalities of Matplotlib focusing on statistical plot types that simplify the creation of complex visual analytics ([Waskom, 2021](#)). It is widely used in data science for tasks such as distribution exploration correlation analysis and regression plotting. The library provides a high-level interface for drawing attractive and informative statistical graphics in a pythonic manner.

2.7.10 Pillow

Pillow ([Clark, 2015](#)) is a fork of the Python imaging library that provides advanced image processing capabilities it supports file format conversions resizing filtering and various transformations making it an essential tool for tasks such as image preprocessing and augmentation in computer vision research. Pillow also includes robust support for color management and alpha compositing enabling efficient manipulation of images with transparency.

2.7.11 CUDA (Compute Unified Device Architecture)

CUDA is NVIDIA's parallel computing platform, enabling GPUs to accelerate deep learning, scientific computing, and high-performance applications ([Nickolls](#)

et al., 2008). By allowing developers to offload intensive matrix operations and parallel tasks to GPUs, CUDA significantly boosts training speed for deep neural networks, simulations, and AI workloads. Its libraries, such as cuDNN and TensorRT, optimize deep learning computations, making it essential for frameworks like TensorFlow and PyTorch to leverage hardware acceleration for scalable AI solutions.

2.7.12 Kaggle

Kaggle is a leading online platform for data science and machine learning competitions, providing a collaborative environment for data scientists, researchers, and AI practitioners to solve real-world problems using open datasets and computational resources. The platform offers hosted Jupyter notebooks, GPU/TPU support, and an extensive dataset repository, enabling users to train and deploy machine learning models efficiently. Kaggle fosters a competitive and educational ecosystem, where participants engage in hackathons, tutorials, and knowledge-sharing through discussion forums. It has significantly contributed to AI research and industry applications, allowing organizations to crowdsource innovative solutions to complex challenges.

2.7.13 MNIST (Modified National Institute of Standards and Technology) dataset

The MNIST dataset is a widely used benchmark in machine learning for image classification tasks. It consists of 70,000 grayscale images of handwritten digits (0–9), each of size 28x28 pixels. The dataset is split into 60,000 training images and 10,000 test images. MNIST is popular for evaluating algorithms like CNNs due to its simplicity, small size, and well-balanced nature. It serves as a starting point for learning and testing image recognition models.

2.7.14 CIFAR dataset

The CIFAR dataset, consisting of CIFAR-10 and CIFAR-100 versions, is a collection of small 32x32 color images categorized into 10 or 100 classes respectively, and serves as a fundamental benchmark in machine learning and deep learning for evaluating the performance of classification algorithms on relatively low-resolution yet diverse image data.

2.7.15 Iris dataset

The Iris dataset is a classic dataset in machine learning, often used for classification and clustering tasks. It contains 150 samples of iris flowers, with 50 samples from each of three species: *Iris setosa*, *Iris versicolor*, and *Iris virginica*. Each sample is described by four features: sepal length, sepal width, petal length, and petal width. The dataset is small, simple, and ideal for teaching and testing algorithms like

logistic regression, decision trees, and k-nearest neighbors (KNN). It is commonly used to demonstrate and benchmark supervised learning techniques.

2.7.16 Pipeline repositories and pre-trained models

Deep learning models are very computationally demanding and probably wouldn't have gained such popularity if it weren't for the ability to "share" them through a system of saving models (pre-training) and placing them on public hubs such as the TensorFlow hub (now mostly moved to the popular Kaggle service), or Huggingfaces – described in [Chapter 6](#). In addition, predefined models can apply to different scenarios (style transfer, sentiment analysis, ...). For example, instead of creating your own script for the transfer of a style, such as the one described in [section 7.3](#), it is enough to just use a trained model:

```
hub.load("hub_module = hub.load('https://kaggle.com/
models/google/arbitrary-image-stylization-v1/frameworks/
TensorFlow1/variations/256/versions/1')
```

Note

- 1 For a full formal specification, refer to the first chapter of the seminal work by [Sutton and Barto \(1998, 2018\)](#).

References

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., Kudlur, M., Levenberg, J., Monga, R., Moore, S., Murray, D. G., Steiner, B., Tucker, P., Vasudevan, V., Warden, P., ... Zheng, X. (2016). *TensorFlow: A system for large-scale machine learning*. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)* (pp. 265–283). USENIX Association. <https://www.usenix.org/system/files/conference/osdi16/osdi16-abadi.pdf>
- Arjovsky, M., Chintala, S., & Bottou, L. (2017). Wasserstein GAN. *Proceedings of the 34th International Conference on Machine Learning*, 70, 214–223.
- Ba, J. L., Kiros, J. R., & Hinton, G. E. (2016). Layer normalization. *arXiv preprint*, arXiv:1607.06450.
- Bahdanau, D., Brakel, P., Xu, K., Goyal, A., Lowe, R., Pineau, J., ... Bengio, Y. (2016). An actor-critic algorithm for sequence prediction. *arXiv preprint*, arXiv:1607.07086.
- Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural machine translation by jointly learning to align and translate. In *ICLR'2015*, arXiv:1409.0473.
- Baldi, P. (2012). Autoencoders, unsupervised learning, and deep architectures. In *Proceedings of ICML Workshop on Unsupervised and Transfer Learning* (pp. 37–49). JMLR.
- Bellman, R. E. (1957a). *Dynamic programming*. Princeton University Press.
- Bellman, R. E. (1957b). A Markov decision process. *Journal of Mathematical Mechanics*, 6, 679–684.
- Besbes, O., & Zeevi, A. (2009). Dynamic Pricing Without Knowing the Demand Function: Risk Bounds and Near-Optimal Algorithms. *Operations Research*, 57(6), 1407–1420.
- Bian, J., Song, Y., Ding, C., Cheng, J., Li, S., & Li, G. (2024). Optimal bidding strategy for PV and BESSs in joint energy and frequency regulation markets considering carbon reduction benefits. *Journal of Modern Power Systems and Clean Energy*, 12(2), 427–439.

- Bottou, L. (2010). Large-scale machine learning with stochastic gradient descent. In *Proceedings of COMPSTAT'2010: 19th International Conference on Computational Statistics*, Paris, France, August 22–27, 2010. Keynote, Invited and Contributed Papers (pp. 177–186). Physica-Verlag HD.
- Breiman, L. (1994). *Bagging predictors* (Technical Report No. 421). Department of Statistics, University of California, Berkeley.
- Brock, A., Donahue, J., & Simonyan, K. (2019). Large scale GAN training for high fidelity natural image synthesis. In *Proceedings of 7th International Conference on Learning Representations (ICLR 2019)*. arXiv preprint arXiv:1809.11096.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... Amodei, D. (2020). Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33, 1877–1901. <https://doi.org/10.48550/arXiv.2005.14165>
- Chen, Y., Jiang, H., Li, C., Jia, X., & Ghamisi, P. (2016). Deep feature extraction and classification of hyperspectral images based on convolutional neural networks. *IEEE Transactions on Geoscience and Remote Sensing*, 54(10), 6232–6251. <https://doi.org/10.1109/TGRS.2016.2603172>
- Chollet, F., & Allaire, J. (2018). *Deep learning with R*. Manning Publications.
- Chollet, F. (2017). *Deep learning with Python*. Manning Publications.
- Chung, J., Gulcehre, C., Cho, K., & Bengio, Y. (2014). Empirical evaluation of gated recurrent neural networks on sequence modeling. *arXiv preprint*, arXiv:1412.3555. <https://arxiv.org/abs/1412.3555>
- Clark, A. (2015). *Pillow (PIL Fork) [Computer software]*. GitHub.
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20, 273–297.
- David, J., & Goadrich, M. (2006). The relationship between precision-recall and ROC curves. In W. Cohen & A. Moore (Eds.), *Proceedings of the 23rd International Conference on Machine Learning (ICML 2006)*.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press. <https://www.deeplearningbook.org/>
- Deng, Y., Bao, F., Kong, Y., Ren, Z., & Dai, Q. (2017). Deep Direct Reinforcement Learning for Financial Signal Representation and Trading. *IEEE Transactions on Neural Networks and Learning Systems*, 28, 653–664.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 1, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., et al. (2020). An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint*, arXiv:2010.11929.
- Duarte, V., Zuniga-Jara, S., & Contreras, S. (2022). Machine learning and marketing: A systematic literature review. *IEEE Access*, 10, 93273–93288.
- Duchi, J., Hazan, E., & Singer, Y. (2011). Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12(7), 2121–2159.
- Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2), 179–211. https://doi.org/10.1207/s15516709cog1402_1
- Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters*, 8, 861–874.
- Fritzke, B. (1995). A growing neural gas network learns topologies. *Advances in Neural Information Processing Systems*, 7, 625–632.
- Gauss, C. F., Bertrand, J., & Trotter, H. F. (1957). *Gauss's Work (1803–1826) on the Theory of Least Squares*. Statistical Techniques Research Group, Section of Mathematical Statistics, Department of Mathematics [sic], Princeton University.
- Goldberg, D. E. (1989). *Genetic algorithms in search, optimization, and machine learning*. Addison-Wesley.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. *Advances in Neural Information Processing Systems*, 27, 2672–2680.
- Gulrajani, I., Ahmed, F., Arjovsky, M., Dumoulin, V., & Courville, A. (2017). Improved training of Wasserstein GANs. *Advances in Neural Information Processing Systems*, 30.
- Hair, J. F., Black, W. C., Babin, B. J., & Anderson, R. E. (2009). *Multivariate data analysis* (7th ed.). Pearson.
- Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362.
- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 770–778.
- Hinton, G. (2012). Neural networks for machine learning [Coursera lecture]. Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude. University of Toronto. https://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf
- Hinton, G. E., & Salakhutdinov, R. R. (2006). Reducing the dimensionality of data with neural networks. *Science*, 313(5786), 504–507.
- Hinton, G. E., & Sejnowski, T. J. (1986). Learning and relearning in Boltzmann machines. In D. E. Rumelhart & J. L. McClelland (Eds.), *Parallel Distributed Processing: Explorations in the Microstructure of Cognition* (Vol. 1, pp. 282–317), 2.
- Hinton, G. E., Osindero, S., & Teh, Y. W. (2006). A fast learning algorithm for deep belief nets. *Neural Computation*, 18(7), 1527–1554.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Holland, J. H. (1975). *Adaptation in natural and artificial systems: An introductory analysis with applications to biology, control, and artificial intelligence*. University of Michigan Press.
- Horowitz, P., & Hill, W. (2015). *The art of electronics* (3rd ed.). Cambridge University Press.
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., ... & Adam, H. (2017). Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint*, arXiv:1704.04861.
- Howard, R. (1960). *Dynamic programming and Markov processes*. MIT Press.
- Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4700–4708). The Institute of Electrical and Electronics Engineer, IEEE Computer Society Press, Los Alamitos, CA.
- Huber, P. J. (1964). Robust estimation of a location parameter. *The Annals of Mathematical Statistics*, 35, 73–101. <https://doi.org/10.1214/aoms/1177703732>
- Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. *arXiv preprint*, arXiv:1502.03167. <https://doi.org/10.48550/arXiv.1502.03167>
- Isola, P., Zhu, J.-Y., Zhou, T., & Efros, A. A. (2017). Image-to-image translation with conditional adversarial networks. In *Computer Vision and Pattern Recognition (CVPR)*. The Institute of Electrical and Electronics Engineers.
- Jaccard, P. (1901). Étude comparative de la distribution florale dans une portion des Alpes et des Jura. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37, 547–579.
- Jiang, S., Qin, S., Pulsipher, J. L., & Zavala, V. M. (2024). Convolutional neural networks: Basic concepts and applications in manufacturing. In M. Soroush & R. D Braatz (Eds.), *Artificial Intelligence in Manufacturing* (pp. 63–102). Academic Press.

- Jordan, M. I. (1986). *Serial order: A parallel distributed processing approach (Technical Report No. 8604)*. University of California, San Diego, Institute for Cognitive Science.
- Kalashnikov, D., Irpan, A., Pastor, P., Ibarz, J., Herzog, A., Jang, E., ... Levine, S. (2018). Scalable deep reinforcement learning for vision-based robotic manipulation. In A. Billard, A. Dragan, J. Peters, & J. Morimoto (Eds.), *Conference on Robot Learning* (pp. 651–673). PMLR.
- Karras, T., Aila, T., Laine, S., & Lehtinen, J. (2017). Progressive growing of GANs for improved quality, stability, and variation. *arXiv preprint*, arXiv:1710.10196.
- Karras, T., Laine, S., & Aila, T. (2019). A style-based architecture for GANs. In *Computer Vision and Pattern Recognition (CVPR)*. The Institute of Electrical and Electronics Engineers.
- Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint*, arXiv:1412.6980.
- Kingma, D. P., & Welling, M. (2014). Auto-encoding variational Bayes. *arXiv preprint*, arXiv:1312.6114.
- Kiran, B. R., Sobh, I., Talpaert, V., Mannion, P., Al Sallab, A. A., Yogamani, S., & Pérez, P. (2021). Deep reinforcement learning for autonomous driving: A survey. *IEEE Transactions on Intelligent Transportation Systems*, 23(6), 4909–4926.
- Kohonen, T. (1982). Self-organized formation of topologically correct feature maps. *Biological Cybernetics*, 43(1), 59–69.
- Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). ImageNet classification with deep convolutional neural networks. *Advances in Neural Information Processing Systems*, 25, 1097–1105.
- LeCun, Y., Bengio, Y., & Haffner, P. (1989). Handwritten digit recognition with a back-propagation network. *Advances in Neural Information Processing Systems*, 2, 396–404.
- LeCun, Y., Bengio, Y. & Hinton, G. Deep learning. *Nature* 521, 436–444 (2015). <https://doi.org/10.1038/nature14539>
- LeCun, Y., Bottou, L., Orr, G. B., & Müller, K. R. (2002). Efficient backprop. In G. Montavon, G. B. Orr, & K-R. Müller (Eds.), *Neural networks: Tricks of the trade* (pp. 9–50). Springer Berlin Heidelberg.
- Ledig, C., Theis, L., Huszar, F., Caballero, J., Aitken, A. P., Tejani, A., ... Shi, W. (2017). Photo-realistic single image super-resolution using a generative adversarial network. In: In: The Institute of Electrical and Electronics Engineers, *Computer Vision and Pattern Recognition (CVPR)*.
- Lemaréchal, C. (2012). Cauchy and the gradient method. *Documenta Mathematica Extra*, 251(254), 10.
- Levine, S., Finn, C., Darrell, T., & Abbeel, P. (2016). End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39), 1–40.
- Li, J., Monroe, W., Shi, T., Jean, S., Ritter, A., & Jurafsky, D. (2017). Adversarial learning for neural dialogue generation. *arXiv preprint*, arXiv:1701.06547.
- Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision* (pp. 2980–2988).
- Linnainmaa, S. (1970). *The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors* (Master's thesis, University of Helsinki). University of Helsinki, pp. 6–7.
- Liu, F., Tang, R., Li, X., Zhang, W., Ye, Y., Chen, H., ... Zhang, Y. (2018). Deep reinforcement learning based recommendation with explicit user-item interactions modeling. *arXiv preprint*, arXiv:1810.12027.
- Liu, J., Guo, H., Li, Y., Tang, Q., Huang, F., Chen, T., ... Chen, Q. (2024). Reinforcement learning based bidding framework with high-dimensional bids in power markets. *arXiv preprint*, arXiv:2410.11180.

- Luong, M. T., Pham, H., & Manning, C. D. (2015). Effective approaches to attention-based neural machine translation. *arXiv preprint*, arXiv:1508.04025.
- Maass, W. (1997). Networks of spiking neurons: The third generation of neural network models. *Neural Networks*, 10(9), 1659–1671.
- Maass, W., Natschläger, T., & Markram, H. (2002). Real-time computing without stable states: A new framework for neural computation based on perturbations. *Neural Computation*, 14(11), 2531–2560.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to information retrieval*. Cambridge University Press.
- Mao, X., Li, Q., Xie, H., Lau, R. Y., Wang, Z., & Paul Smolley, S. (2017). Least squares generative adversarial networks. In *Proceedings of the IEEE International Conference on Computer Vision* (pp. 2794–2802). The Institute of Electrical and Electronics Engineers.
- McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *Bulletin of Mathematical Biophysics*, 5(4), 115–133. Reprinted in Anderson, J. A., & Rosenfeld, E. (Eds.). (1988). *Neurocomputing: Foundations of research*.
- McKinney, W. (2010). Data structures for statistical computing in Python. In S. van der Walt & J. Millman (Eds.), *Proceedings of the 9th Python in Science Conference* (pp. 51–56).
- Minsky, M. L. (1954). *Theory of neural-analog reinforcement systems and its application to the brain-model problem* (Ph.D. dissertation). Princeton University.
- Minsky M. L. & Papert S. (1969). *Perceptrons*. MIT Press.
- Mirhoseini, A., Pham, H., Le, Q. V., Steiner, B., Larsen, R., Zhou, Y., ... Dean, J. (2017). Device placement optimization with reinforcement learning. In D. Precup, & Y. W. The, (Eds.), *International Conference on Machine Learning* (pp. 2430–2439). PMLR.
- Mirza, M., & Osindero, S. (2014). Conditional generative adversarial nets. *arXiv preprint*, arXiv:1411.1784.
- Miyato, T., Kataoka, T., Koyama, M., & Yoshida, Y. (2018). Spectral normalization for generative adversarial networks. *arXiv preprint*, arXiv:1802.05957.
- Mnih, V., Kavukcuoglu, K., Silver, D. et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533. <https://doi.org/10.1038/nature14236>
- Moody, J., & Saffell, M. (2001). Learning to trade via direct reinforcement. *IEEE Transactions on Neural Networks*, 12(4), 875–889.
- Murphy, K. P. (2012). *Machine learning: A probabilistic perspective*. MIT Press.
- Nevmyvaka, Y., Feng, Y., & Kearns, M. (2006, June). Reinforcement learning for optimized trade execution. In W. Cohen e & A. Moore (Eds.), *Proceedings of the 23rd International Conference on Machine Learning* (pp. 673–680).
- Nguyen, H. D., Jones, A. T., & McLachlan, G. J. (2018). Stream-suitable optimization algorithms for some soft-margin support vector machine variants. *Japanese Journal of Statistics and Data Science*, 1(1), 81–108.
- Nickolls, J., Buck, I., Garland, M., & Skadron, K. (2008). Scalable parallel programming with CUDA. *ACM Queue*, 6(2), 40–53.
- Oldham, K. B., Myland, J., & Spanier, J. (2010). *An atlas of functions: With equator; the atlas function calculator*. Springer Science & Business Media.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 8024–8035.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5), 1–17.
- Powers, D. M. W. (2011). Evaluation: From precision, recall and F-measure to ROC, informedness, markedness & correlation. *Journal of Machine Learning Technologies*, 2(1), 37–63.

- Puterman, M. L. (1994). *Markov decision processes—Discrete stochastic dynamic programming*. John Wiley & Sons, Inc.
- Radford, A., Metz, L., & Chintala, S. (2016). Unsupervised representation learning with deep convolutional generative adversarial networks. *International Conference on Learning Representations (ICLR)*. arXiv preprint arXiv:1511.06434.
- Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). *You only look once: Unified, real-time object detection*. In: The Institute of Electrical and Electronics Engineers, *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 779–788.
- Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386–408.
- Rosenblatt, F. (1962). *Principles of neurodynamics: Perceptrons and the theory of brain mechanisms*. Spartan Books.
- Ruder, S. (2016). An overview of gradient descent optimization algorithms. *arXiv preprint*, arXiv:1609.04747.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533–536.
- Sallab, A. E., Abdou, M., Perot, E., & Yogamani, S. (2017). Deep reinforcement learning framework for autonomous driving. *Electronic Imaging*, 2017(19), 70–76.
- Seber, G. A. F., & Lee, A. J. (2012). *Linear regression analysis*. Wiley.
- Silver, D., Huang, A., Maddison, C. J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Dieleman, S., Grewe, D., Nham, J., Kalchbrenner, N., Sutskever, I., Lillicrap, T., Leach, M., Kavukcuoglu, K., Graepel, T., & Hassabis, D. (2016). Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), 484–489.
- Silver, D., Schrittwieser, J., Simonyan, K., Antonoglou, I., Huang, A., Guez, A., Hubert, T., Baker, L., Lai, M., Bolton, A., Chen, Y., Lillicrap, T., Hui, F., Sifre, L., van den Driessche, G., Graepel, T., & Hassabis, D. (2017a). Mastering the game of Go without human knowledge. *Nature*, 550, 354–359.
- Silver, D., Schrittwieser, J., Simonyan, K., et al. (2017b). Mastering Chess and Shogi by self-play with a general reinforcement learning algorithm. *arXiv preprint*, arXiv:1712.01815.
- Simonyan, K., & Zisserman, A. (2014). Very deep convolutional networks for large-scale image recognition. *arXiv preprint*, arXiv:1409.1556.
- Simonyan, K., Vedaldi, A., & Zisserman, A. (2013). *Deep inside convolutional networks: Visualising image classification models and saliency maps*. *arXiv preprint*, arXiv:1312.6034.
- Smolensky, P. (1986). Information processing in dynamical systems: Foundations of harmony theory. In D. E. Rumelhart & J. L. McClelland (Eds.), *Parallel Distributed Processing: Explorations in the Microstructure of Cognition*, 1 (194–281).
- Spall, J. C. (2005). *Introduction to stochastic search and optimization: Estimation, simulation, and control*. Wiley.
- Sprott, D. A. (1978). Gauss's contributions to statistics. *Historia Mathematica*, 5(2), 183–203. [https://doi.org/10.1016/0315-0860\(78\)90049-6](https://doi.org/10.1016/0315-0860(78)90049-6)
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1), 1929–1958.
- Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. *Advances in Neural Information Processing Systems*, 27.
- Sutton, R. S. (1984). *Temporal Credit Assignment in Reinforcement Learning* (PhD thesis). University of Massachusetts, Amherst, MA.
- Sutton, R. S., & Barto, A. G. (1998). *Reinforcement learning: An introduction*. MIT Press.
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.
- Szegedy, C., Liu, W., Jia, Y., et al. (2015). Going deeper with convolutions. In: The Institute of Electrical and Electronics Engineers, *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 1–9.

- Tan, M., & Le, Q. (2019). EfficientNet: Rethinking model scaling for convolutional neural networks. *International Conference on Machine Learning*, 6105–6114.
- Thathachar, M. A. L., & Sastry, P. S. (1985). A new approach to the design of reinforcement schemes for learning automata. *IEEE Transactions on Systems, Man, and Cybernetics*, 15(1), 168–175.
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1), 267–288.
- Turing, A. M. (1936). On computable numbers, with an application to the Entscheidungsproblem. *Proceedings of the London Mathematical Society*, 2(42), 230–265. <https://doi.org/10.1112/plms/s2-42.1.230>
- van Otterlo, M., & Wiering, M. (2012). Reinforcement learning and Markov decision processes. In: M. Wiering, M. van Otterlo (eds.). *Reinforcement learning. Adaptation, learning, and optimization* (Vol. 12). Springer. https://doi.org/10.1007/978-3-642-27645-3_1
- Van Rijsbergen, C. J. (1979). *Information retrieval*. Butterworth-Heinemann.
- Vapnik, V. (1998). *Statistical learning theory*. John Wiley & Sons.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008. <https://doi.org/10.48550/arXiv.1706.03762>
- Vincent, P., Larochelle, H., Bengio, Y., & Manzagol, P. A. (2008). Extracting and composing robust features with denoising autoencoders. *Proceedings of the 25th International Conference on Machine Learning*, 1096–1103
- Vinyals, O., Babuschkin, I., Czarnecki, W. M. et al. (2019). Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature*, 575, 350–354.
- Waskom, M. (2021). Seaborn: Statistical data visualization. *Journal of Open Source Software*, 6(60), 3021. <https://doi.org/10.21105/joss.03021>
- Watkins, C. J. C. H. (1989). *Learning from delayed rewards* [Doctoral dissertation, University of Cambridge].
- Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8(3–4), 279–292.
- Wei, X., Xiang, Y., Li, J., & Zhang, X. (2022). Self-dispatch of wind-storage integrated system: A deep reinforcement learning approach. *IEEE Transactions on Sustainable Energy*, 13(3), 1861–1864.
- Werbos, P. (1982). Applications of advances in nonlinear sensitivity analysis. In D. Hömberg & F. Tröltzsch (Eds.), *System modeling and optimization* (pp. 762–770). Springer.
- Whitesitt, J. E. (2012). *Boolean algebra and its applications*. Dover Publications.
- Widrow B., & Hoff M. E. (1960). *Adaptive switching circuits*. In IRE WESCON Convention Record, Volume 4, pp. 96–104. Reprinted in Anderson and Rosenfeld (1988).
- Wurman, P. R., Barrett, S., Kawamoto, K. et al. (2022). Outracing champion Gran Turismo drivers with deep reinforcement learning. *Nature*, 602, 223–228.
- Xie, J., Girshick, R., & Farhadi, A. (2016, June). Unsupervised deep embedding for clustering analysis. In *Proceedings of the 33rd International Conference on Machine Learning, New York, NY, USA, 2016*. JMLR: W&CP.
- Yu, C., Liu, J., Nemati, S., & Yin, G. (2021). *Reinforcement learning in healthcare: A survey*. *ACM Computing Surveys (CSUR)*, 55(1), 1–36.
- Zhang, H., Goodfellow, I., Metaxas, D., & Odena, A. (2019). Self-Attention Generative Adversarial Networks. *International Conference on Machine Learning (ICML)*, 97, 7354–7363.
- Zhou, C., & Paffenroth, R. C. (2017). Anomaly detection with robust deep autoencoders. In S. Matwin, S. Yu, & F. Farooq (Eds.), *Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 665–674).
- Zhu, J.-Y., Park, T., Isola, P., & Efros, A. A. (2017). Unpaired image-to-image translation using cycle-consistent adversarial networks. In *International Conference on Computer Vision (ICCV)*. The Institute of Electrical and Electronics Engineers.
- Zou, H., & Hastie, T. (2005). Regularization and variable selection via the elastic net. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 67(2), 301–320.

3 Regression and discrimination problems with deep neural networks

3.1 Feedforward networks as regression and discriminant analysis technique

In the 20th century and early 21st century, neural networks have been treated as an extension of discriminant and regression analysis. [Ripley \(1996\)](#) finds the strength of neural networks in bigger possible processed data volumes (“online” vs “batch”) and in the flexibility of setting the large but finite number of parameters, as opposed to methods of discriminant analysis with a small number of parameters or non-parametric ones. [MacKay \(2003\)](#) treats a single neuron as a binary classifier and the neural network training procedure as an inference within a probabilistic framework. [Bishop \(2006, p. 225\)](#) places neural networks as an alternative to linear models of regression and discriminant analysis, and the support vector machines (SVM) method, pointing to their adaptivity in the sense of the ability to use parametric forms for the basic functions in which the parameter values are adapted during training. [Hastie et al. \(2009\)](#) state that neural networks are just nonlinear statistical models similar to projection pursuit regression. [Gatnar \(2009\)](#) describes neural networks as a subdomain of discriminant analysis along with linear discriminant analysis, quadratic discriminant analysis, k-nearest neighbor algorithm, and support vector machines.

In this chapter, we will treat regression and discriminant analysis as one family (although having in mind the differences in the aims of both techniques) according to [Hair et al.’s \(2009\)](#) approach (see [Figure 3.1](#)).

Regression analysis examines the dependence of an explained (dependent, endogenous) variable on two or more explanatory (independent, exogenous) variables. Regression analysis is used to describe the relationship between the explained variable and the explanatory variables, to make a prediction for the explanatory variable, and to simulate and develop scenarios.

In economics, due to the type of economic activity, regression analysis is used, among others, in modeling (cf. [Walesiak, 2009](#)):

- The production process (e.g., in the Cobb–Douglas function, in which the variable is explained by the amount of production, and the variables explaining the amount of employment and productive assets), employment, labor force and unemployment, and fixed assets;

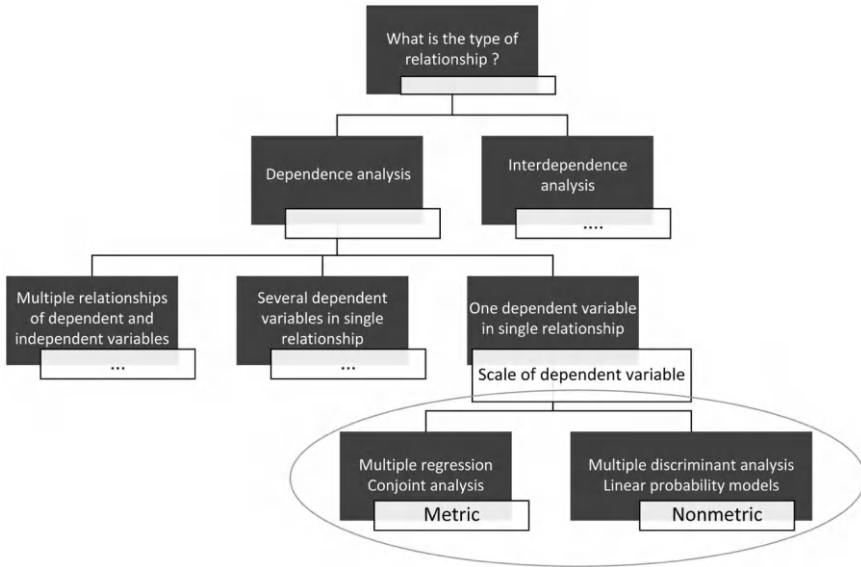


Figure 3.1 Multivariate data analysis taxonomy.

Source: Based on [Hair et al. \(2009\)](#).

- Financial flows (income and expenditure of the population; financial flows of enterprises: revenues, production costs, and financial result);
- Public finances (state budget revenues and expenditures; money and capital markets: bank loans, interest rates, exchange rates);
- Prices and wages (e.g., average wages in the corporate or service sectors);
- Demand, examining the dependence of demand on the factors shaping it. Models distinguished according to the level of demand are of a special class: for the absolute level, product models concerning the sales of all brands of the product under study on the market and concerning the sales of a specific brand on the market, and for the relative level, market share models.

Discriminant analysis (model classification, learning with a teacher) aims to find a classification rule. It consists of two stages: the stage of proper discrimination, during which, based on observations from the learning set, a discriminant model is built; and the stage of classification, during which observations from the recognized set are assigned to the appropriate classes indicated by this model. As in the case of regression analysis, one of the variables is explained by the others, but the key distinction between these two families is the scale of the dependent variable. In the case of regression analysis, the most common variable is continuous (or discrete, but with a meaningful order). In contrast, in the case of discriminant analysis, it is always the nominal scale, with the values (labels) denoting the class to which the examined object belongs. In economics, it is commonly used for fraudulent behavior detection, bankruptcy prediction, and credit scoring.

Table 3.1 presents the main algorithms of dependence analysis with one dependent variable in single relationship (regression and discriminant analysis). In Table 3.2, most popular/important techniques of ensemble approach are placed. The specificity of these types of method is that they are not separate regression or

Table 3.1 Main methods of regression and discriminant analysis

<i>Method</i>	<i>Category</i>	<i>References</i>	<i>Remarks/areas of application</i>
Linear regression	Regression	Legendre (1805); Gauss (1809, 1821); Galton (1877, 1885); Fisher (1922)	Modeling Economic Growth (Barro, 1991), Policy Impact Evaluation (Card & Cruger, 1994), Consumer Behavior Analysis (Keynes, 1936), Housing Market Analysis (Rosen, 1974), Risk Assessment and Credit Scoring (Hand & Henley, 1997)
Non-parametric regression	Regression	Friedman (1991); Kooperberg et al. (1997); Friedman & Stuetzle (1981); Rousseeuw and Struyf (1998); Tibshirani (1996)	Most important algorithms: MART, POLYMARS, LASSO, PPR, ACE, Avas, Depth-based Regression
Limited-dependent and qualitative regression	Regression	Bliss (1934); Berkson (1944); McFadden (1974); Maddala (1986); Agresti (2002)	Analysis of demand of cars (Cragg & Uhler, 1970), factors that determine the vote in the US congress on the Medicare bill (McKelvey & Zavoina, 1975), analysis of career choice factors (Schmidt & Strauss, 1975), research on structural transformations in industry (Heckman, 1976), Risk Assessment and Credit Scoring (Hand & Henley, 1997)
Discriminant analysis (“classical”)	Discriminant analysis	Bayes (1763); Fisher (1936)	Credit risk assessment (Fisher, 1936), Corporate bankruptcy prediction (Altman, 1968), Identifying prospect for sales campaign (Johnson & Wichern, 2007, p. 578), Economic Policy Analysis (Friedman, 1989)

(Continued)

Table 3.1 (Continued)

<i>Method</i>	<i>Category</i>	<i>References</i>	<i>Remarks/areas of application</i>
K-nearest Neighbor	Discriminant analysis	Fix & Hodges (1951); Cover & Hart (1967); Hart (1968)	Risk Assessment and Credit Scoring (Hand & Henley, 1997), Stock price prediction (Alkhatib et al., 2013)
Kernel discriminant analysis	Discriminant analysis	Hand (1982)	Risk Assessment and Credit Scoring (Liberati et al., 2012)
Decision trees	Regression and discriminant analysis	Breiman et al. (1984)	Characteristics of customers with similar attitude (Magidson, 1994), Risk Assessment and Credit Scoring (Hand & Henley, 1997), Market Segmentation (Kass, 1980)
Support vector machines	Regression and discriminant analysis	Vapnik (1998)	

Source: Based on Dudek (2013).

Table 3.2 Main ensemble techniques of regression and discriminant analysis

<i>Method</i>	<i>Main references</i>	<i>Description</i>	<i>Used for methods</i>
Bagging	Breiman (1994)	Bagging (bootstrap aggregating) reduces variance by averaging predictions from multiple models trained on different bootstrapped datasets.	Primary decision trees, the elements of bagging may be found in L1 regularization
Boosting	Freund and Schapire (1997)	Boosting sequentially trains weak learners, each correcting the errors of its predecessor, to create a strong ensemble model.	All
Arcing	Breiman (1998)	Arcing (adaptive resampling and combining) improves boosting by emphasizing more on misclassified instances dynamically.	Decision trees, linear and quadratic discriminant analysis, kernel discriminant analysis
Random forests	Breiman (2001)	Random forests combine bagging with random feature selection for decision trees to reduce overfitting and improve generalization.	Decision trees

(Continued)

Table 3.2 (Continued)

Method	Main references	Description	Used for methods
Random subspaces	Ho (1998)	Random subspaces method builds models using subsets of features to enhance diversity in the ensemble.	Decision trees
Extreme gradient boosting (XGBoost)	Chen and Guestrin (2016)	XGBoost is a highly optimized implementation of gradient boosting with regularization and parallel processing features.	All

Source: Based on [Dudek \(2013\)](#).

discriminant algorithms, but rather define a way of combining other methods in order to obtain a model with better prediction quality.

3.2 Regularization, overfitting, and proper model design

3.2.1 Model design – data preparation

3.2.1.1 Variable scales

To properly design a model, the researcher should be aware of what kind of data the model will be trained and used for. In this chapter we will treat the dataset as a usual multivariate statistical analysis table (see e.g., [Hair et al., 2009](#)). Methods of graphical, text, and audio data processing will be described in [Chapters 6](#) and [7](#) but extended representations of data in the form of, for example, symbolic data tables (see e.g., [Billard & Diday, 2008](#); [Bock & Diday, 2000](#); [Dudek, 2013](#)) don't fit the thematic scope of this book. Thus, in this chapter, we are assuming that the analyzed data are represented by two-dimensional data matrix in which any element is denoted by x_{ij} ($i = 1, \dots, n$ $j = 1, \dots, m$). It is the observation of the j -th variable in the i -th object. A multivariate observation (m -dimensional) will be saved as a vector

$$\mathbf{x}_i = [x_{i1}, x_{i2}, \dots, x_{im}]^T.$$

The rows of the data matrix represent the objects (in some references called *instances*) consumer X, product Y, car brand S, patient P, municipality G, company F, river R, test market T, hypermarket H, outlet Z, household D, philosophical idea I (see [Walesiak, 2009](#)).

Columns in the data matrix correspond to variables (in some sources referred as *features*)

A variable is a characteristic that describes the community of objects. In formal terms, a variable M_j is a mapping

$$M_j: A \rightarrow Q(j = 1, \dots, m), \quad (3.1)$$

where:

Q – a set of mappings (of numbers, categories);

m – number of variables;

N – number of objects.

In order to properly analyze the given data, it is not enough to reduce them to a numerical form, but it is also necessary to determine the nature of each of the variables. In “traditional” methods of multivariate statistical analysis, the nature of variables has a very significant impact on the possibility of using equivalent methods, measures, and techniques (cf. e.g., [Hair et al., 2009](#)), and in deep learning models, incorrect representation of the nature of the variable can lead to the model not bringing any informational value. In theory of measurement, usually four scales are distinguished:

Nominal: The information is used just to distinguish the different values of observations. Mark of a car (plane, TV), ID of a seller, gender of a customer. Nominal scale doesn’t introduce any order or any algebraic relationship between the values.

Ordinal: The order may be established for estimating the importance of each value, but with no deeper meaning of number. The v_2 variable value labeled equal 2 is just better than the v_1 variable value equal 1, but the statements “ v_2 is twice better than v_1 ”, “ v_2 better than v_1 by exactly 1” are not justified. The common example of a variable measured on ordinal scale is the level of education 1 – primary, 2 – secondary, 3 – university or, depending of particular country, 1 – primary school, 2 – middle school, 3 – high school, 4 – university.

Interval: The variable values are ordered and the difference between v_2 and v_1 is interpretable. The typical variables measured on this scale are the averages of all students’ notes or the Value of PMI index. Human’s body temperature is a very common example of an interval scale measured variable in the sources, but its range of usage in economic research is rather limited (if not empty).

Ratio: If in addition to subtraction of v_2 and v_1 their ratio is meaningful then the variable is measured on ratio scale. A necessary condition for a variable to be measurable on a ratio scale is the presence of absolute zero. The price of a given product, the compensation of the executive board, the engine capacity of a vehicle are the examples of ratio scale measured variables.

The two first scales are often called “weak” scales while the two other two “metric” scales (or sometimes in older references “strong” scales). [Table 3.3](#) summarizes the features of measurement scales.

Table 3.3 Basic properties of measurement scales

Scale type	Allowed mathematical transformations	Acceptable relationships	Acceptable arithmetic operations
Nominal	$z = f(x), f(x)$ – any mutually unambiguous transformation	Equality ($x_A = x_B$), miscellaneous ($x_A \neq x_B$)	Counting events (number of equality relations, differences)
Ordinal	$z = f(x), f(x)$ – any strictly monotonically increasing function	Above and the majority ($x_A > x_B$) and minority ($x_A < x_B$)	Counting events (number of equality relations, differences, majority, minority)
Interval	$z = bx + a$ ($b > 0$), $z \in R$ for all x contained in R , a zero value on this scale is usually taken arbitrarily or by convention	Above and the equality of differences and intervals ($x_A - x_B = x_C - x_D$)	Above and addition and subtraction
Ratio	$z = bx$ ($b > 0$), $z \in R_+$ for all x contained in R_+ , the natural origin of the quotient scale is the zero value (zero left-hand limits the range of the scale)	Above and ratios equality ($\frac{x_A}{x_B} = \frac{x_C}{x_D}$)	Above and multiplication and division

Source: Walesiak (2009) based on Stevens (1959, pp. 25, 27); Adams et al. (1965).

3.2.1.2 Variables encoding

In a typical scenario, real datasets acquired from web repositories' or directly from application programming interfaces' (APIs') or Webservices' nominal or ordinal variables are represented in raw data by text strings with some information included (which should be retrieved and transformed into numerical variables). In this case, the encoding of variables is needed for one of the first steps of model building. We could also imagine situations where integer numbers represent categorical variables, but the encoding, due to their real meaning, is still required (the mark of a car written by its ordinal number is "calculable" but not "merit right").

There are several ways to encode ordinal and nominal variables, and it is always the researcher's responsibility to use the proper one, although there are some recommendations in certain situations.

If a given variable has only two categories, it can be replaced with a binary variable (i.e., zero–one). One variant is given a value of 1, and the other is given a value of 0 (or –1). For example, for the employed variable, the encoding will be as follows: employed "1", unemployed "0," or "–1". Groups of binary variables may be used to encode nominal variables; in this case, they are usually named "dummy variables".

If a variable has more than two categories, we use the method of substitution, which consists of using a set of artificial dummy variables (e.g., zero–one). The rule here is that the number of artificial variables introduced must be less than the number of levels (categories) of a given variable by one. Let us assume that there are three variants (categories) for the variable education: basic, basic vocational, secondary vocational, or general. In this case, two artificial variables should be introduced, defined as follows:

Education Level	M_j	M_{j+1}
primary	0	0
secondary	0	1
high	1	0

This technique also appears in versions for which a number of variants of encoded variable is equal to a number of introduced artificial variables:

Education Level	M_j	M_{j+1}	M_{j+2}
primary	0	0	0
secondary	0	0	1
high	0	1	0

which in fact is equivalent but requires more memory to implement with the first column equal to the linear combination of two others, so it’s currently rarely used.

This kind of encoding in references is often called “one-hot-encoding” ([Hackeling, 2017](#)). It prevents incorrect numerical ordering issues and works well when there are only a few ordinal categories, but the following limitations, among others, may be raised: it does not preserve order among ordinal categories and increases dimensionality, which may break efficiency in deep networks.

In practical cases, one-hot-encoding may be used if the number of the values is not more significant than the given number – changing in different references from 5 to 15. For variables with bigger domain cardinality, the learned representation is usually applied.

Individual categories can be assigned to successive natural numbers. It does not matter here whether the categories can be ordered according to the degree of intensity of the impact (ordinal variable) or not (nominal variable). For example, for the ordinal variable organization of work including the categories *very good*, *good*, *poor*, *bad*, we can use the following coding:

Bad	1
Poor	2
Good	3
Very good	4

This method is often referred to as “integer encoding” when applied to ordinal data and “label encoding” in the case of nominal data. While it is simple and computationally efficient, its application to deep models is highly limited only to situations when assumptions of equal linear spacing between categories introduces bias only at a controlled level (or in ideal world doesn’t introduce bias at all). This differs from tree-based models, which often use integer encoding.

Continuous encoding. Sometimes for ordinal variables it is more proper to assign real numbers.

Low	0.1
Medium	0.5
High	0.9

This technique, while computationally convenient, should be used with caution to properly reproduce the real variable values’ order structure.

Target encoding replaces feature categories with some number derived from the target object, such as minimal value, mean value, median value, and other aggregation coefficients/statistics. It preserves ordinal relationships, reduces dimensionality, and is often a rational choice, especially for large datasets.

Custom representation. If the variable value contains information about the interval of certain values (price “from \$10,000 to \$20,000”, city “between 10,000 and 20,000” citizens) the value may be represented by 10,000 (min), 15,000 (mean), 20,000 (max) or two columns with 10,000 and 20,000 value. It differs slightly from target encoding, which uses aggregation function; while in custom encodings, particular variables may be set “manually”.

Learned representation (embeddings). In mathematics, an embedding is one instance of some mathematical structure contained within another instance.

Embeddings in neural networks are dense, low-dimensional vector representations that capture the semantic and contextual relationships of high-dimensional, discrete input data. They transform categorical items into continuous vector spaces, enabling models to learn and generalize patterns effectively.

The main idea behind learned representations and embeddings is to automatically transform high-dimensional, often sparse input data into dense, lower-dimensional vectors that capture the essential properties, relationships, and underlying structure of the data. This transformation enables models to work more efficiently and effectively, as similar items are mapped to nearby points in the vector space.

To create embeddings, the separate model should be trained and used for evaluation. In Keras library the embedding layer may be established directly in the model-building phase. The number of variables (vector coordinates) is often estimated with the rounded square root of the number of possible variable values.

There are other methods of variable encoding:

- Hashing;
- Backward Difference;

- Helmert Encoder;
- Sum Encoder;
- Polynomial Encoder;
- Base N Encoder;
- Leave One Out Encoder.

The description of these methods for deep neural networks is included in [Hancock and Khoshgoftaar \(2020\)](#).

3.2.1.3 Variables selection

In the next step, or in some cases, even before data encoding, is the selection of an adequate variable for the particular model. There are various reasons why the variable should be excluded from model training:

- **Too low variability:** A variable constant or almost constant in a given domain brings no discriminant power to the analysis.
- **Too high variability:** The serial number of the car (VIN), the unique ID in a computer database or any other nominal/ordinal with high number of possible values brings too much discriminant power, but with the increasing risk that model won't converge.
- **Lack of adequacy:** While in the famous example the number of storks may be correlated with the birth ratio in a given area (see e.g., [Matthews, 2001](#)) it's not recommended to include the variable stork number in a birth ratio model, as well as other irrelevant variables in any deep learning model.
- **To high level of missing values:** It's always a delicate balance between removing objects with missing values, using data imputation methods like Multi-variate Imputation by Chained Equations (MICE) ([Raghunathan et al., 2001](#)) or removing the variables.

Among other reasons for excluding variables from analysis, the following may be distinguished (cf. [Guyon & Elisseeff, 2003](#)):

- Multicollinearity;
- Overfitting;
- Computational efficiency;
- Interpretability.

There are no standard “automatic” methods for finding the variables to be removed like Clustering Objects on Subsets of Attributes (COSA) ([Friedman & Meulman, 2004](#)), Heuristic Identification of Noisy Variables (HINoV) ([Carmone et al., 1999](#)), Optimal Variable Weighting (OVW) ([De Soete, 1986](#)) in cluster analysis or Boruta algorithm (Kursa & Rudnicki, 2011) for the random forest. It is usually the responsibility of the model designer to correctly point out and remove them.

3.2.1.4 *Object selection*

Sometimes certain objects are removed when their representations contain too many missing values (which is distinct from omitting a variable due to irrelevance). As we mentioned earlier, the methods of data imputation may be applied instead of just removing objects with too large a number of gaps.

3.2.1.5 *Variable aggregation*

Due to variable redundancy, collinearity or too high overall number of variables, the techniques of dimension reduction may be used. This approach is mainly popular with use of Principal Component Analysis (Hotelling, 1993; Jolliffe, 2002; Pearson, 1901) and has a wide area of applications in computer vision systems (see e.g., Chen et al., 2016).

3.2.1.6 *Variable normalization*

In a very common scenario, meaningful, carefully selected variables are just “not compatible”. The ranges of variability differ by hundreds, thousands, or even billions of times. In a typical dataset, like the one used in [Chapter 4](#), the price of a car may be nearly 100 000 (\$) while the age of a car is usually less than 20 (years). In this case, normalization is used before model training for several important reasons. Normalization makes the impact of all variables comparable – for example, in distance calculation or estimation of hyperparameters – reduces numerical instability, and improves the interpretability of model parameters, ensuring that algorithms – particularly those based on aggregative metrics – can accurately capture the underlying patterns in the data.

Normalization is used when variables are measured on an interval or ratio scale. The purpose of normalization is to deprive variable values of names and unify their orders of magnitude. With regard to weak scales of measurement, there is no need for normalization because their values are used to determine neither the relations of equality of differences and intervals nor ratios.

Due to the fact that the only acceptable transformations on the interval and quotient scales are linear transformations, the normalization formulae can be expressed by the general formula (3.2):

$$z_{ij} = bx_{ij} + a \quad (b > 0) \quad (3.2)$$

Special cases of this formula are the following formulas: standardization, Weber standardization, unitarization, zeroed unitarization, normalization in the interval $[-1; 1]$, and quotient transformations (see [Table 3.4](#)).

3.2.2 *Regularization and overfitting*

When creating a model, we must constantly keep in mind that our goal is not to match it to the training set, so it is worth using regularization methods, which in the vast majority of cases increase the generalization skills.

Table 3.4 Normalization formulae for metric data

Formula name	Formula	Variable measurement scale	
		Before normalization	After normalization
Standardization	$z_{ij} = (x_{ij} - \bar{x}_j) / s_j$	Ratio and/or interval	Interval
Weber standardization	$z_{ij} = (x_{ij} - Me_j) / 1,4826 \cdot MAD_j$	Ratio and/or interval	Interval
Unitarization	$z_{ij} = (x_{ij} - \bar{x}_j) / r_j$	Ratio and/or interval	Interval
Zeroed unitarization (min max normalization)	$z_{ij} = [x_{ij} - \min_i \{x_{ij}\}] / r_j$	Ratio and/or interval	Interval
Normalization in the interval $[-1, 1]$	$z_{ij} = (x_{ij} - \bar{x}_j) / \max_i x_{ij} - \bar{x}_j $	Ratio and/or interval	Interval
Ratio-factor transformations	$z_{ij} = x_{ij} / s_j$	Ratio	Ratio
	$z_{ij} = x_{ij} / r_j$	Ratio	Ratio
	$z_{ij} = x_{ij} / \max_i \{x_{ij}\}$	Ratio	Ratio
	$z_{ij} = x_{ij} / \bar{x}_j$	Ratio	Ratio
	$z_{ij} = x_{ij} / \sum_{i=1}^n x_{ij}$	Ratio	Ratio
	$z_{ij} = x_{ij} / \sqrt{\sum_{i=1}^n x_{ij}^2}$	Ratio	Ratio

Notes:

$x_{ij}(z_{ij})$ – value (normalized value) of the j -th variable for the i -th object;

$\bar{x}_j (s_j, r_j)$ – mean (standard deviation, range) for the j -th variable;

$r_j = \max_i \{x_{ij}\} - \min_i \{x_{ij}\}$ – range for j -th variable;

$Me_j (MAD_j)$ – median (median absolute deviation) for the j -th variable.

Source: Walesiak (2009) based on Anderberg (1973), Jajuga and Walesiak (2000), Milligan and Cooper (1988).

There are multiple empirical studies and research papers that demonstrate how regularization techniques (such as L1/L2 penalties, Dropout, and Batch Normalization) improve model performance by reducing overfitting and enhancing generalization.

Zhang et al. (2016, 2021) showed that deep neural networks memorize noisy labels if trained without regularization. L2 regularization (weight decay) significantly reduces overfitting, improving test accuracy. Dropout stabilizes training, reducing sensitivity to input noise.

Srivastava et al.'s (2014) empirical results on CIFAR-10 and MNIST datasets show a 1–2% accuracy improvement over non-regularized models. Dropout reduces test error while maintaining similar training error, showing better generalization.

Ng (2004) showed that L1 regularization (Least Absolute Shrinkage and Selection Operator (LASSO)) encourages sparsity in models, improving interpretability and reducing noise. L2 regularization (Ridge) improves stability in models with correlated features. Empirical results show L2 regularization reduces overfitting, yielding lower test error than models without regularization.

Ioffe and Szegedy (2015) showed that Batch Normalization smooths optimization landscapes, leading to faster convergence. Batch Normalization acts as implicit regularization, improving test accuracy without explicit weight penalties. Experiments made on ImageNet and CIFAR-10 confirm better generalization when Batch Normalization is used.

Regardless of the regularization techniques used, the model should be trained on a sufficiently large set to avoid overfitting.

The most common method of estimating minimal number of training set number of objects is “rule of thumb” (see van Belle, 2011) which says that it should be 5–10 greater than the number of all parameters.

There are other methods of estimating the number of parameters, as proposed by Hastie *et al.* (2009, Chapter 7), based on eigenvalues. However, due to their numerical complexity, they are hard to apply.

Estimating the number of trainable parameters may be a competitive task, but most popular Python libraries help the designers with the summary function.

Taking into account the number of parameters for the model summarized in Figure 3.2, the adequate number of objects for the training set due to the rule of thumb may be estimated as 80 000–160 000.

```
keras_model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	5,120
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_2 (Dense)	(None, 32)	2,080
dropout_2 (Dropout)	(None, 32)	0
dense_3 (Dense)	(None, 16)	528
dropout_3 (Dropout)	(None, 16)	0
dense_4 (Dense)	(None, 1)	17

Total params: 16,001 (62.50 KB)

Trainable params: 16,001 (62.50 KB)

Non-trainable params: 0 (0.00 B)

Figure 3.2 Trainable parameters estimation with model.summary() function

Source: Own research.

3.2.3 Proper model evaluation including imbalance impact

When evaluating model prediction quality, we should have in mind that there are several other measures than accuracy and all of them should be at least satisfying level, to hope that model won't struggle for interpretability, fairness, or robustness in real-world applications.

Accuracy measures the overall correctness of a model by calculating the ratio of correctly predicted instances to the total instances. A high accuracy is desirable, but in imbalanced datasets, it can be misleading if the model predominantly predicts the majority class. A poorly performing fraud detection model, for instance, could achieve high accuracy by predicting mostly non-fraud cases while missing actual frauds, which is undesirable.

Precision quantifies how many of the predicted positive cases (fraudulent cases) are actually correct. A high precision indicates that the model makes fewer false positive errors, meaning it does not mistakenly classify non-fraudulent cases as fraud. However, if precision is too high at the cost of low recall, the model may be overly conservative and miss many actual fraud cases, leading to false negatives.

Recall (sensitivity) is the proportion of actual positive cases (fraudulent cases) correctly identified by the model. A high recall is crucial in fraud detection because missing fraudulent cases can be costly. However, a model with high recall but low precision may generate too many false positives, making it impractical as users or investigators would have to check too many flagged transactions.

F1 score is the harmonic mean of precision and recall, balancing both metrics. A high F1 score means the model is effective at detecting positive cases while maintaining a reasonable false positive rate. If either precision or recall is very low, the F1 score will be low, signaling poor performance. Ideally, the F1 score should be optimized to ensure a good trade-off between detecting fraud and avoiding excessive false alarms.

Receiver Operating Characteristic – Area Under Curve (ROC AUC) measures the model's ability to distinguish between classes at different thresholds. A high ROC AUC (close to 1.0) indicates strong discriminatory power, while a low value (close to 0.5) suggests the model performs no better than random guessing. A very high ROC AUC with poor precision or recall may indicate threshold tuning issues, necessitating a balance between different evaluation metrics for practical deployment.

The formulae for each metric are presented in [Table 2.2 \(Chapter 2\)](#).

Let's explore the highly theoretical model of next powerball/lotto 6 out of 49 balls game results prediction, which means $\binom{49}{6} = 13983816$ possibilities.

To simplify the example let's assume that $(x_1, x_2, x_3, x_4, x_5, x_6) = (1, 2, 3, 4, 5, 6)$ is the winning combination and $x_1 < x_2 < x_3 < x_4 < x_5 < x_6$ for each instance. M_1 and M_2 models with y_{M_1} and y_{M_2} explained variables predictions defined as:

$$y_{M_1} = \{0 \text{ for all objects} \}$$

$$y_{M_2} = \{1 \text{ for all objects with } x_1 = 1 \text{ and } x_2 = 2, 0 \text{ for others}\}$$

Let's assume additionally (which is highly improbable although illustrative) that the training set contains ten objects with $x_1=1$ and $x_2=2$ including the winning combination.

To examine the performance, a simple simulation script will be used. The results are shown in [Figure 3.3](#).

Model M1

- Predictions: Predicted all 100 samples as class 0.
- The model achieves high accuracy (99%) by predicting everything as negative (class 0).

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score, confusion_matrix

y_test=[0]*99+[1]*1
y_preds=[[0]*100, [0]*90+[1]*10]
for i,y_pred in enumerate(y_preds):
    results = {
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred,
zero_division=0),
        "Recall (Sensitivity)": recall_score(y_test, y_pred),
        "F1 Score": f1_score(y_test, y_pred),
        "ROC AUC": roc_auc_score(y_test, y_pred),
        "Confusion Matrix": confusion_matrix(y_test, y_pred)
    }
    print(f"Model M{i+1} performance scores")
    print(results)
```

Model M1 performance scores

```
{'Accuracy': 0.99, 'Precision': 0.0, 'Recall (Sensitivity)':
0.0, 'F1 Score': 0.0, 'ROC AUC': np.float64(0.5), 'Confusion
Matrix': array([[99,  0],
[ 1,  0]])}
```

Model M2 performance scores

```
{'Accuracy': 0.91, 'Precision': 0.1, 'Recall (Sensitivity)':
1.0, 'F1 Score': 0.18181818181818182, 'ROC AUC': np.float64
(0.9545454545454545), 'Confusion Matrix': array([[90,  9],
[ 0,  1]])}
```

Figure 3.3 Model metrics for lottery winner problem.

Source: Own research.

- It fails to identify the single positive example, resulting in 0% recall and 0% precision for the positive class.
- Because it never predicts positive, the AUC is 0.50 (the baseline for random guessing in a binary setting).

Model M2

- Predictions: Predicted 0 for 90 samples, 1 for 10 samples.
 - The model identifies the single positive example (100% recall), but incorrectly classifies 9 negative samples as positive (leading to 9 false positives).
 - As a result, accuracy is 91% – lower than M1’s 99%.
 - However, from the standpoint of detecting positives, M2 is far better: it finds the single positive every time, whereas M1 misses it.
 - The AUC is 0.95, indicating that the model ranks the positive instance higher than most negatives.
- Model M1 maximizes overall accuracy on an imbalanced dataset by predicting the majority class every time. This yields no detection of the positive class (0% recall), making it unsuitable if detecting positives is crucial (which in fact it is for lotto winning numbers).
- Model M2 trades off some accuracy for perfect recall of the positive class. It catches the single positive, but has more false positives. The higher AUC (0.95 vs 0.50) also indicates it distinguishes positive vs negative more effectively.

Model M1 looks good by accuracy alone, but fails to detect the minority class. Model M2 does a better job for tasks that require identifying positives, despite lower accuracy.

The balance of performance metrics is crucial; however, it does not assure the proper behavior of the model at large, but in opposite models with unacceptable performance metrics are unlikely to be useful in practice and we always have in mind that economic research models for discriminant analysis often involve highly imbalanced datasets, especially in applications like bankruptcy prediction and fraud detection. For instance, most firms do not go bankrupt, while a small subset eventually fails; similarly, the proportion of fraudulent transactions is minuscule relative to the vast majority of legitimate transactions (Altman, 1968; Phua et al., 2010). This class imbalance poses challenges for traditional classification methods, which may yield high overall accuracy by predominantly predicting the majority class (e.g., “non-bankrupt” or “non-fraudulent”) yet fail to detect the minority class of greatest interest (Ohlson, 1980).

In bankruptcy prediction, pioneering work on discriminant analysis by Altman (1968) used financial ratios to classify firms as healthy or at risk of failure, but real-world data commonly shows that fewer than 5% of firms file for

bankruptcy (Deakin, 1972). Consequently, models might appear to perform well if they almost always predict “non-bankrupt”, while missing the crucial minority of firms at genuine risk. Similar issues arise in fraud detection, where the percentage of fraudulent transactions is exceedingly small; standard classifiers tend to generate many false negatives if they do not account for the skewed distribution (Phua et al., 2010).

Furthermore, the economic costs of misclassifying minority instances are typically much higher than those for majority instances (Merton, 1974). Failing to detect a potential bankruptcy or fraudulent activity can be far more consequential than incorrectly flagging a healthy firm or a legitimate transaction (Ohlson, 1980). As such, researchers increasingly emphasize performance metrics beyond accuracy – such as precision, recall, F1 score, or the ROC AUC – when developing discriminant analysis models for these domains (Beaver, 1966; Phua et al., 2010).

3.3 Research application example: vehicle claim fraud detection

The discriminant analysis with deep learning models is widely used for fraud detection. In the research example, a vehicle claim fraud detection dataset (Aslam, 2022) will be analyzed.

- Source: the dataset originates from the decision analytics department of a US insurance firm (EXL Service);
- Year of retrieval: 2020;
- Public access: the dataset is available publicly on Kaggle;
- Region: contains data from US-based insurance firms;
- Number of variables: 33 (includes both categorical and numerical features);
- Type of data: the dataset consists of structured data, including policy details, accident information, and customer demographics;
- Target variable: FraudFound_P, which indicates whether a claim was fraudulent (1) or not (0). Target variable is highly imbalanced with 14497 false values and only 923 positives (which is quite common in fraud, security breach or bankruptcy prediction dataset, cf. Brown & Mues, 2012);
- Number of observations: the dataset contains 15420 though the exact number may vary;
- Number of columns: 33 variables, including the fraud label;
- Data types: a mix of categorical, numerical, and binary variables.

Dataset Variables and Descriptions are presented in Table 3.5.

In the Python code needed for analysis the Kaggle API key will be needed to directly download the card fraud dataset. The process of Kaggle API key acquisition is described in Appendix 2.

Table 3.5 Description of vehicle fraud claims detection dataset variables

<i>Variable name</i>	<i>Description</i>	<i>Data type</i>
Month	The month when the accident occurred.	Date
WeekOfMonth	The week of the month when the accident occurred (1st, 2nd, etc.).	Date
DayOfWeek	The day of the week when the accident happened (Monday–Sunday).	Date
Make	The brand or manufacturer of the vehicle involved in the accident.	Nominal
AccidentArea	Indicates whether the accident occurred in a rural or urban area.	Nominal
DayOfWeekClaimed	The day of the week when the insurance claim was filed.	Date
MonthClaimed	The month in which the insurance claim was made.	Date
WeekOfMonthClaimed	The week in the month when the claim was filed.	Date
Sex	The gender of the policyholder (Male or Female).	Binary
MaritalStatus	Marital status of the policyholder (Single, Married, Widowed, etc.).	Nominal
Age	The age of the policyholder at the time of the claim.	Ratio
Fault	Identifies whether the policyholder was at fault in the accident (Policyholder or Third Party).	Nominal
PolicyType	The type of insurance policy (Comprehensive, Third Party, etc.).	Nominal
VehicleCategory	The category of the vehicle (e.g., Sedan, SUV, Truck).	Nominal
VehiclePrice	The price category of the insured vehicle (Less than \$20K, \$20K–\$40K, etc.).	Interval
FraudFound_P	Target variable: indicates if fraud was detected (1 for fraudulent, 0 for legitimate claims).	Binary
PolicyNumber	A unique identifier for each policyholder.	Ordinal
RepNumber	A unique number assigned to the claim representative handling the case.	Ordinal
Deductible	The deductible amount that the policyholder must pay before the insurance covers the claim.	Ratio
DriverRating	A numerical rating given to the driver based on their driving history.	Ordinal
Days_Policy_Accident	The number of days between policy issuance and accident occurrence.	Ordinal
Days_Policy_Claim	The number of days between the policy inception and claim filing.	Ordinal
PastNumberOfClaims	The number of past insurance claims filed by the policyholder.	Ordinal
AgeOfVehicle	The age of the insured vehicle at the time of the claim.	Ordinal
AgeOfPolicyHolder	The age of the policyholder at the time of claim filing (in interval).	Interval

(Continued)

Table 3.5 (Continued)

<i>Variable name</i>	<i>Description</i>	<i>Data type</i>
PoliceReportFiled	Whether a police report was filed for the accident (Yes or No).	Binary
WitnessPresent	Indicates whether any witnesses were present during the accident (Yes or No).	Binary
AgentType	The type of insurance agent (Internal or External).	Nominal
NumberOfSuppliments	The number of supplemental documents submitted with the claim.	Ratio
AddressChange_Claim	Indicates whether the policyholder changed their address at the time of the claim.	Binary
NumberOfCars	The number of vehicles covered under the policy.	Ratio
Year	The year when the accident occurred.	Date
BasePolicy	The base policy type of the insurance plan.	Nominal

Source: Based on Aslam (2022).

```
import os
from kaggle.api.kaggle_api_extended import KaggleApi
import zipfile
import pandas as pd
import numpy as np
from sklearn.preprocessing import
OneHotEncoder, LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.ensemble import GradientBoostingClassifier,
RandomForestClassifier, VotingClassifier
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score, confusion_matrix
from tensorflow.keras.layers import Embedding, Flatten,
Dense, Input
from tensorflow.keras.models import Model
import matplotlib.pyplot as plt
import seaborn as sns
import tensorflow as tf
import re

# Set your Kaggle API key (replace 'your_username' and
# 'your_api_key' with actual values) uncomment two
subsequent lines
#in case of problems
os.environ['KAGGLE_USERNAME'] = "your_username"
os.environ['KAGGLE_KEY'] = "your_api_key"
```

```

dataset_name = "shivamb/vehicle-claim-fraud-detection"
dataset_path = "vehicle-data"
dataset_file_name = "fraud_oracle.csv"

api = KaggleApi()
api.authenticate()

api.dataset_download_file(dataset_name, file_name=dataset_file_
name, path=dataset_path)

with zipfile.ZipFile(f"{os.path.join(dataset_path,
dataset_file_name)}.zip", 'r') as zip_ref:
    zip_ref.extractall(dataset_path)
df = pd.read_csv(os.path.join(dataset_path, dataset_file_name))

```

If for any reason acquisition of API_KEY from Kaggle is not possible, the data may be loaded “manually” from Kaggle.

As for most research analysis, the dataset requires some preparation, The base information needed for preparation of the transformation strategy is listed by the usual in this case commands presented below:

```

print("Dataset Loaded Successfully!")
print(df.info())
pd.set_option('display.max_columns', None)
print(df.head(n=3))

```

The results of data examination are presented in [Figure 3.4](#).

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15420 entries, 0 to 15419
Data columns (total 33 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Month                 15420 non-null  object
1   WeekOfMonth           15420 non-null  int64
2   DayOfWeek             15420 non-null  object
3   Make                  15420 non-null  object
4   AccidentArea          15420 non-null  object
5   DayOfWeekClaimed      15420 non-null  object
6   MonthClaimed          15420 non-null  object
7   WeekOfMonthClaimed    15420 non-null  int64
8   Sex                   15420 non-null  object
9   MaritalStatus         15420 non-null  object
10  Age                   15420 non-null  int64

```

Figure 3.4 Vehicle claim fraud detection dataset characteristics. (Continued)

Source: Own research.

```

11 Fault 15420 non-null object
12 PolicyType 15420 non-null object
13 VehicleCategory 15420 non-null object
14 VehiclePrice 15420 non-null object
15 FraudFound_P 15420 non-null int64
16 PolicyNumber 15420 non-null int64
17 RepNumber 15420 non-null int64
18 Deductible 15420 non-null int64
19 DriverRating 15420 non-null int64
20 Days_Policy_Accident 15420 non-null object
21 Days_Policy_Claim 15420 non-null object
22 PastNumberOfClaims 15420 non-null object
23 AgeOfVehicle 15420 non-null object
24 AgeOfPolicyHolder 15420 non-null object
25 PoliceReportFiled 15420 non-null object
26 WitnessPresent 15420 non-null object
27 AgentType 15420 non-null object
28 NumberOfSupplements 15420 non-null object
29 AddressChange_Claim 15420 non-null object
30 NumberOfCars 15420 non-null object
31 Year 15420 non-null int64
32 BasePolicy 15420 non-null object

```

dtypes: int64(9), object(24)

memory usage: 3.9+ MB

None

```

      Month WeekOfMonth DayOfWeek Make AccidentArea DayOfWeekClaimed \
0   Dec           5   Wednesday   Honda           Urban           Tuesday
1   Jan           3   Wednesday   Honda           Urban           Monday
2   Oct           5     Friday   Honda           Urban           Thursday

      MonthClaimed WeekOfMonthClaimed Sex MaritalStatus Age Fault \
0           Jan           1   Female           Single   21   Policy Holder
1           Jan           4     Male           Single   34   Policy Holder
2           Nov           2     Male           Married   47   Policy Holder

      PolicyType VehicleCategory VehiclePrice FraudFound_P \
0   Sport - Liability           Sport   more than 69000           0
1   Sport - Collision           Sport   more than 69000           0
2   Sport - Collision           Sport   more than 69000           0

      PolicyNumber RepNumber Deductible DriverRating Days_Policy_
Accident \
0           1           12           300           1   more than 30
1           2           15           400           4   more than 30
2           3           7           400           3   more than 30

      Days_Policy_Claim PastNumberOfClaims AgeOfVehicle AgeOfPolicy-
Holder \
0   more than 30           none           3 years           26 to 30
1   more than 30           none           6 years           31 to 35
2   more than 30           1           7 years           41 to 50

```

Figure 3.4 (Continued)

	PoliceReportFiled	WitnessPresent	AgentType	NumberOfSupplements	\
0	No	No	External	none	
1	Yes	No	External	none	
2	No	No	External	none	
	AddressChange_Claim	NumberOfCars	Year	BasePolicy	
0	1 year	3 to 4	1994	Liability	
1	no change	1 vehicle	1994	Collision	
2	no change	1 vehicle	1994	Collision	

Figure 3.4 (Continued)

In the next step, the variables that for any reason should not be part of analysis and weak (measured on nominal and ordinal scales) are pointed.

PolicyNumber is a unique identifier for each accident, so it brings no discriminant power. Conversely, *Days_Policy_Accident* and *Days_Policy_Claim* are highly unbalanced (15247 and 15342 instances of “more than 30”). For the same reason the variable *Deductible* and *NumberOfCars* will be omitted. *Age* and *AgeOfPolicyHolder* contain similar information, thus the second one will be omitted. All discussed variables will be removed as well as variables related to time, which are rarely used in standard discriminant and regression tasks, but rather in time series or panel data analysis methods.

The binary variables will be changed to 0,1 values. The nominal and ordinal variables will be replaced by encoded ones with the following rules:

Ordinal variables will be encoded by continuous encoding.

Nominal variables will be encoded by one-hot-encoding with the exception of *Make* and *RepNumber* due to their higher number of possible values. *Make* and learned *RepNumber* representation with embeddings will be used (separately) with four variables used for representation of each – due to the square root rule.

These changes are implemented in subsequent commands:

```
columns_to_remove = ["PolicyNumber", "Days_Policy_Accident",
"Days_Policy_Claim",
                    "Deductible",
"NumberOfCars", "AgeOfPolicyHolder", "Month", "WeekOfMonth",
"DayOfWeek", "DayOfWeekClaimed",
                    "MonthClaimed", "WeekOfMonthClaimed",
                    "Year"]
df_cleaned = df.drop(columns=columns_to_remove)

# One-Hot Encoding for categorical variables

one_hot_cols = ["Sex", "AccidentArea", "MaritalStatus",
"Fault", "PolicyType", "VehicleCategory", "AgentType",
"BasePolicy", "AddressChange_Claim"]
```

```

one_hot_encoder = OneHotEncoder(sparse_output=False,
drop='first') encoded_one_hot = one_hot_encoder.
fit_transform(df_cleaned[one_hot_cols])
encoded_one_hot_df = pd.DataFrame(encoded_one_hot,
columns=one_hot_encoder.get_feature_names_out(one_hot_cols))

# Replace original categorical variables with one-hot
  encoded values
df_cleaned = df_cleaned.drop(columns=one_hot_cols).
reset_index(drop=True)
df_cleaned = pd.concat([df_cleaned, encoded_one_hot_df], axis=1)

# Encode binary variables
binary_cols = ["PoliceReportFiled", "WitnessPresent"]
for col in binary_cols:
    df_cleaned[col] = df_cleaned[col].map({'Yes': 1.0, 'No':
0.0})

df_cleaned["VehiclePrice"] = df_cleaned["VehiclePrice"].map({
    '20000 to 29000':25000.0,
    '30000 to 39000':35000.0,
    '40000 to 59000':45000.0,
    'less than 20000':10000.0,
    'more than 69000':85000.0,
    '60000 to 69000':65000.0})

df_cleaned["NumberOfSuppliments"] =
df_cleaned["NumberOfSuppliments"].map(
    {'more than 5':8.0,
    '1 to 2':1.5,
    '3 to 5':4.0,
    'none':0.0})

df_cleaned["PastNumberOfClaims"] =
df_cleaned["PastNumberOfClaims"].map({
    'more than 4':5.0,
    '2 to 4':3.0,
    'none':0.0,
    '1':1.0
})

def extract_years(value):
    match = re.match(r"(\d+) years", str(value))
    return int(match.group(1)) if match else value

df_cleaned["AgeOfVehicle"] = df_cleaned["AgeOfVehicle"].
replace("new", 0.0).replace("more than 7", 10.0).
apply(extract_years)

```

```

for embedable in ["Make", "RepNumber"]:
    label_encoder = LabelEncoder()
    df_cleaned[f"{embedable}_encoded"] =
label_encoder.fit_transform(df[embedable])

categories))
    v_vocab_size = df_cleaned[f"{embedable}_encoded"].nunique()
    embedding_dim_v = int(np.sqrt(v_vocab_size))

    v_input = Input(shape=(1,))
    v_embedding = Embedding(input_dim=v_vocab_size + 1,
output_dim=embedding_dim_v)(v_input)
    v_embedding = Flatten()(v_embedding)

    v_model = Model(inputs=v_input, outputs=v_embedding)
    v_model.compile(optimizer="adam", loss="ce")

    dummy_target = np.zeros((df_cleaned.shape[0],
        embedding_dim_v))
    v_model.fit(df_cleaned[f"{embedable}_encoded"], dummy_
target, epochs=5, batch_size=32, verbose=1)

    v_embeddings_model = Model(inputs=v_input,
outputs=v_embedding)
    v_embeddings =
v_embeddings_model.predict(df_cleaned[f"{embedable}_encoded"])

    v_embedding_df = pd.DataFrame(v_embeddings, columns=
[f"{embedable}_emb_{i}" for i in range(embedding_dim_v)])

    df_cleaned = df_cleaned.drop(columns=[embedable,
f"{embedable}_encoded"])
    df_cleaned = pd.concat([df_cleaned, v_embedding_df], axis=1)

print(df_cleaned.head(n=3))

```

The results of data preparation are presented in [Figure 3.5](#).

```

Epoch 1/5
482/482 _____ 1s 2ms/step - loss: 0.0000e+00
...
Epoch 5/5
482/482 _____ 1s 1ms/step - loss: 0.0000e+00
482/482 _____ 1s 1ms/step
Epoch 1/5
482/482 _____ 1s 1ms/step - loss: 0.0000e+00
...

```

Figure 3.5 Vehicle claim fraud detection dataset encoded data representation. (Continued)

Source: Own research.

Epoch 5/5

482/482 ————— 1s 2ms/step - loss: 0.0000e+00

482/482 ————— 0s 809us/step

	Age	VehiclePrice	FraudFound_P	DriverRating	PastNumberOfClaims \
0	21	85000.0	0	1	0.0
1	34	85000.0	0	4	0.0
2	47	85000.0	0	3	1.0

	AgeOfVehicle	PoliceReportFiled	WitnessPresent	NumberOfSupplements \
0	3.0	0.0	0.0	0.0
1	6.0	1.0	0.0	0.0
2	7.0	0.0	0.0	0.0

	Sex_Male	AccidentArea_Urban	MaritalStatus_Married	MaritalStatus_Single \
0	0.0	1.0	0.0	1.0
1	1.0	1.0	0.0	1.0
2	1.0	1.0	1.0	0.0

	MaritalStatus_Widow	Fault_Third Party	PolicyType_Sedan - Collision \
0	0.0	0.0	0.0
1	0.0	0.0	0.0
2	0.0	0.0	0.0

	PolicyType_Sedan - Liability	PolicyType_Sport - All Perils \
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0

	PolicyType_Sport - Collision	PolicyType_Sport - Liability \
0	0.0	1.0
1	1.0	0.0
2	1.0	0.0

	PolicyType_Utility - All Perils	PolicyType_Utility - Collision \
0	0.0	0.0
1	0.0	0.0
2	0.0	0.0

	PolicyType_Utility - Liability	VehicleCategory_Sport \
0	0.0	1.0
1	0.0	1.0
2	0.0	1.0

	VehicleCategory_Utility	AgentType_Internal	BasePolicy_Collision \
0	0.0	0.0	0.0
1	0.0	0.0	1.0
2	0.0	0.0	1.0

Figure 3.5 (Continued)

	BasePolicy_Liability	AddressChange_Claim_2 to 3 years \	
0	1.0	0.0	
1	0.0	0.0	
2	0.0	0.0	

	AddressChange_Claim_4 to 8 years	AddressChange_Claim_no change \	
0	0.0	0.0	
1	0.0	1.0	
2	0.0	1.0	

	AddressChange_Claim_under 6 months	Make_emb_0	Make_emb_1	Make_emb_2 \
0	0.0	-0.026624	0.041196	0.03367
1	0.0	-0.026624	0.041196	0.03367
2	0.0	-0.026624	0.041196	0.03367

	Make_emb_3	RepNumber_emb_0	RepNumber_emb_1	RepNumber_emb_2 \
0	0.025968	-0.034357	0.013004	0.015627
1	0.025968	-0.007178	0.009929	0.034474
2	0.025968	0.021298	-0.044430	-0.028161

	RepNumber_emb_3
0	-0.042876
1	-0.043914
2	-0.045981

Figure 3.5 (Continued)

Dataset at this moment is properly encoded, but, before the actual analysis, the variables should be standardized. The simplest way to standardize is just to use *StandardScaler* on the whole data frame, but the standardization of embeddings is very often not a good idea, so the variables will be picked “manually”.

The best tool for this purpose is *boxplot* for all explanatory variables, creation of which requires only a few lines in Python. The function is defined to iteratively repeat the creation of *boxplot* in search of acceptable representation:

```

y = df_cleaned["FraudFound_P"]
X = df_cleaned.drop(columns=["FraudFound_P"])
def do_boxplot(title:str):
    plt.figure(figsize=(14, 8))
    # Create a boxplot for all numerical variables in df_cleaned
    sns.boxplot(data=X, orient="h")
    # Set title and labels
    plt.title("Boxplot of All Variables in df_cleaned")
    plt.xlabel("Values")
    plt.ylabel("Variables")
    # Show the plot
    plt.savefig(title)
    plt.show()

do_boxplot("discriminant_fraud_boxplot.png")

```

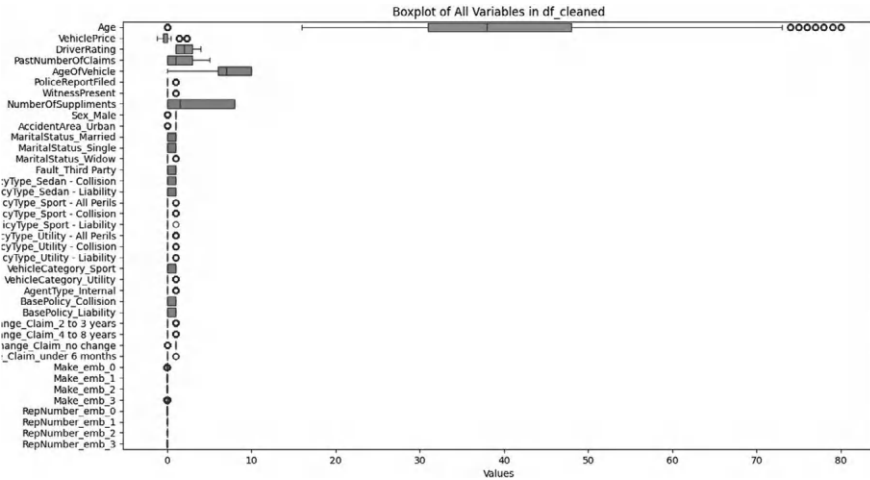


Figure 3.6 Vehicle claim fraud detection dataset encoded, no standardization.

Source: Own work, with Python/scikit-learn.

At first sight of Figure 3.6, the *VehiclePrice*'s range of variability significantly increases the ranges of variabilities of other variables.

To standardize the *VehiclePrice* variable the below three lines are used:

```
scaler = StandardScaler()
X["VehiclePrice"] = scaler.fit_transform(X[["VehiclePrice"]])
do_boxplot("discriminant_fraud_boxplot_price_standardized.jpg")
```

The boxplots for all variables of the dataset after first standardization are presented in Figure 3.7. The range of variabilities is now more balanced with slightly outlying variables – *Age*, *NumberOfSupplements*, and *AgeOfVehicle* – which are standardized in this final step.

```
X[["Age", "NumberOfSupplements", "AgeOfVehicle"]] =
scaler.fit_transform(X[["Age", "NumberOfSupplements", "AgeOfVehicle"]])
do_boxplot("discriminant_fraud_boxplot_all_standardized.jpg")
```

The boxplot of all explanatory variables after this last standardization is presented in Figure 3.8.

The standardization is the final step in data preprocessing so the encoded and standarized dataset will be used to train a deep feedforward network with four hidden layers. Please note that due to class imbalance *class_weight* parameter is introduced in the model training phase.

At this moment, before the prediction and evaluation, the examination of stability of the learning phase should be established with learning curve examination.

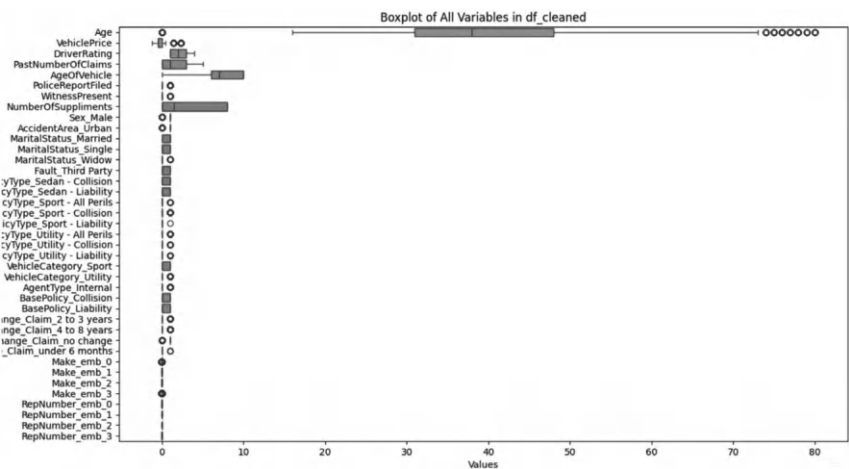


Figure 3.7 Vehicle claim fraud detection dataset encoded, after first standardization.

Source: Own with Python/scikit-learn.

```
keras_model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(X_train.shape[1],)), # Correct way
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(16, activation='relu'),
```

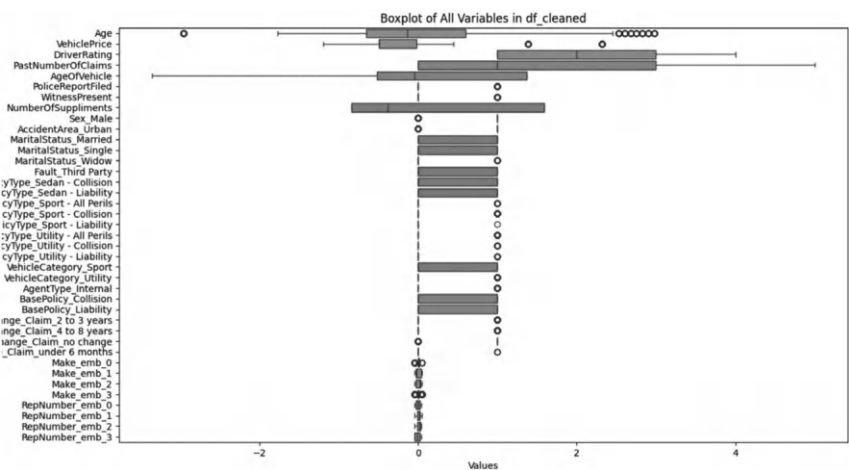


Figure 3.8 Vehicle claim fraud detection dataset encoded after standardizations.

Source: Own work, with Python/scikit-learn.

```

tf.keras.layers.Dropout(0.2),
tf.keras.layers.Dense(1, activation='sigmoid')
])

keras_model.compile(optimizer='adam', loss="binary_crossentropy",
metrics=['accuracy','recall','precision'])
learning=keras_model.fit(X_train, y_train, epochs=50, batch_size=32,
validation_data=(X_test, y_test),class_weight={0:1,1:6}, verbose=1)

fig, ax = plt.subplots(1, 2, figsize=(12, 5))

ax[0].plot(learning.history['loss'], label='Training Loss')
ax[0].plot(learning.history['val_loss'], label='Validation Loss',
linestyle="dashed")
ax[0].set_title('Model Loss Over Epochs')
ax[0].set_xlabel('Epochs')
ax[0].set_ylabel('Loss')
ax[0].legend()

ax[1].plot(learning.history['accuracy'], label='Training Accuracy')
ax[1].plot(learning.history['val_accuracy'], label='Validation
Accuracy', linestyle="dashed")
ax[1].set_title('Model Accuracy Over Epochs')
ax[1].set_xlabel('Epochs')
ax[1].set_ylabel('Accuracy')
ax[1].legend()

plt.tight_layout()
plt.savefig("discriminant_fraud_learning_curve.jpg")
plt.show()

```

The text output of the training phase and the learning curve are presented in [Figures 3.9](#) and [3.10](#).

```

Epoch 1/50
386/386 _____ 6s 8ms/step - accuracy: 0.9077
- loss: 0.7362 - precision: 0.0879 - recall: 0.0553 - val_
accuracy: 0.8586 - val_loss: 0.3375 - val_precision: 0.1489 -
val_recall: 0.2849
...
Epoch 10/50
386/386 _____ 1s 3ms/step - accuracy: 0.8816
- loss: 0.5434 - precision: 0.2007 - recall: 0.3497 - val_
accuracy: 0.8849 - val_loss: 0.2856 - val_precision: 0.2076 -
val_recall: 0.3226
...

```

Figure 3.9 Training of vehicle claim fraud detection – deep feedforward network model.
(Continued)

Source: Own work, with Python/Keras.

```

Epoch 20/50
386/386 _____ 2s 5ms/step - accuracy: 0.8709
- loss: 0.5305 - precision: 0.2252 - recall: 0.4565 - val_
accuracy: 0.8813 - val_loss: 0.2992 - val_precision: 0.2152 -
val_recall: 0.3656
...
Epoch 30/50
386/386 _____ 1s 3ms/step - accuracy: 0.8530
- loss: 0.5234 - precision: 0.2196 - recall: 0.5134 - val_
accuracy: 0.8794 - val_loss: 0.2909 - val_precision: 0.2057 -
val_recall: 0.3495
...
Epoch 40/50
386/386 _____ 1s 3ms/step - accuracy: 0.8685
- loss: 0.4908 - precision: 0.2369 - recall: 0.5350 - val_
accuracy: 0.8735 - val_loss: 0.2956 - val_precision: 0.1928 -
val_recall: 0.3441
Epoch 41/50
Epoch 50/50
386/386 _____ 1s 3ms/step - accuracy: 0.8646
- loss: 0.4540 - precision: 0.2406 - recall: 0.6191 - val_
accuracy: 0.8544 - val_loss: 0.3007 - val_precision: 0.1862 -
val_recall: 0.4194

```

Figure 3.9 (Continued)

The learning curve for vehicle claim fraud detection deep feedforward model is definitely not degraded and the model may be used for prediction.

```

y_pred_keras = (keras_model.predict(X_test) > 0.5).astype(int)
print(confusion_matrix(y_test, y_pred_keras))

```

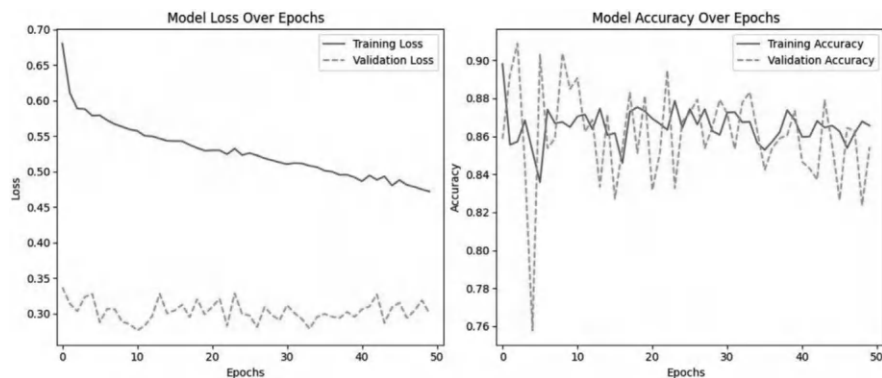


Figure 3.10 Learning curve of vehicle claim fraud detection – deep feedforward network model.

Source: Own work, with Python/Keras.

```

97/97 _____ 0s 1ms/step
[[2557  341]
 [ 108   78]]

```

Figure 3.11 The confusion matrix for prediction of vehicle claim fraud detection – deep feedforward network model.

Source: Own work, with Python/Keras.

The confusion matrix for predicted values of dependent variables for the test set of the model is presented in [Figure 3.11](#).

In the next step, the accuracy, precision, sensitivity, F1 score, and ROC AUC will be calculated. To model performance for benchmark purposes, the measures will be estimated additionally for the following “shallow” methods:

- Naïve Bayes Discriminant Analysis;
- Logistic Regression;
- K Nearest Neighbor Method;
- Gradient Boosting Method;
- Random Forest Classifier;
- Ensemble Voting with the following ensemble methods:
 - a Logistic Regression;
 - b Gradient Boosting Method;
 - c Random Forest Classifier.

The comparison requires Python code similar to:

```

models = {
    "Naive Bayes": GaussianNB(),
    "Logistic Regression": LogisticRegression(),
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "SVM": SVC(kernel='rbf', probability=True),
    "Gradient Boosting": GradientBoostingClassifier(),
    "Random Forest": RandomForestClassifier(),
    "Ensemble Voting": VotingClassifier(estimators=[
        ("LR", LogisticRegression()),
        ("RF", RandomForestClassifier()),
        ("GB", GradientBoostingClassifier())
    ], voting='soft')
}

results = {}
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    results[name] = {

```

```

        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred,
                                     zero_division=0),
        "Recall (Sensitivity)": recall_score(y_test, y_pred),
        "F1 Score": f1_score(y_test, y_pred),
        "ROC AUC": roc_auc_score(y_test, model.predict_
proba(X_test)[: , 1] if hasattr(model, "predict_proba") else
y_pred),
        "Confusion Matrix": confusion_matrix(y_test, y_pred)
    }

results["Keras Model"] = {
    "Accuracy": accuracy_score(y_test, y_pred_keras),
    "Precision": precision_score(y_test, y_pred_keras,
                                zero_division=0),
    "Recall (Sensitivity)": recall_score(y_test,
                                         y_pred_keras),
    "F1 Score": f1_score(y_test, y_pred_keras),
    "ROC AUC": roc_auc_score(y_test, y_pred_keras),
    "Confusion Matrix": confusion_matrix(y_test,
                                         y_pred_keras)
}

results_df = pd.DataFrame(results).T

plt.figure(figsize=(12, 6))
results_df[["Accuracy", "Precision", "Recall (Sensitivity)",
"F1 Score", "ROC AUC"]].plot(kind='bar', figsize=(12,6))
plt.xticks(rotation=45)
plt.title("Comparison of Model Performance")
plt.ylabel("Score")
plt.legend(loc="lower right")
plt.savefig("discriminant_fraud_comparison.jpg")
plt.show()

print(results_df)

```

Figure 3.12 presents the values of accuracy, precision, sensitivity, F1 score, and ROC AUC for each model. Figure 3.13 displays the same results on a bar chart.

The results from the model evaluation highlight the significant challenge of class imbalance in fraud detection. While models like Naïve Bayes achieved an exceptionally high recall (sensitivity) of 96.77%, it did so at the expense of precision (8.79%), leading to a low F1 score (16.13%). This suggests that the model correctly identifies almost all fraud cases, but generates an excessive number of false positives, reducing its reliability for practical applications. On the other hand, models such as Logistic Regression, SVM, and Ensemble Voting completely failed to identify fraudulent cases (recall = 0%), indicating that they were overly biased toward

	Accuracy	Precision	Recall (Sensitivity)	F1 Score \
Naive Bayes	0.392996	0.087977	0.967742	0.16129
Logistic Regression	0.939364	0.0	0.0	0.0
KNN	0.93061	0.088235	0.016129	0.027273
SVM	0.939689	0.0	0.0	0.0
Gradient Boosting	0.941634	0.8	0.043011	0.081633
Random Forest	0.941634	0.8	0.043011	0.081633
Ensemble Voting	0.939689	0.0	0.0	0.0
Keras Model	0.85441	0.186158	0.419355	0.257851

	ROC AUC	Confusion Matrix
Naive Bayes	0.781799	[[1032, 1866], [6, 180]]
Logistic Regression	0.801302	[[2897, 1], [186, 0]]
KNN	0.643932	[[2867, 31], [183, 3]]
SVM	0.739572	[[2898, 0], [186, 0]]
Gradient Boosting	0.83128	[[2896, 2], [178, 8]]
Random Forest	0.785886	[[2896, 2], [178, 8]]
Ensemble Voting	0.819972	[[2898, 0], [186, 0]]
Keras Model	0.650844	[[2557, 341], [108, 78]]

Figure 3.12 Vehicle claim fraud detection – deep feedforward network model vs other models – quality measure benchmark.

Source: Own work, with Python/Keras.

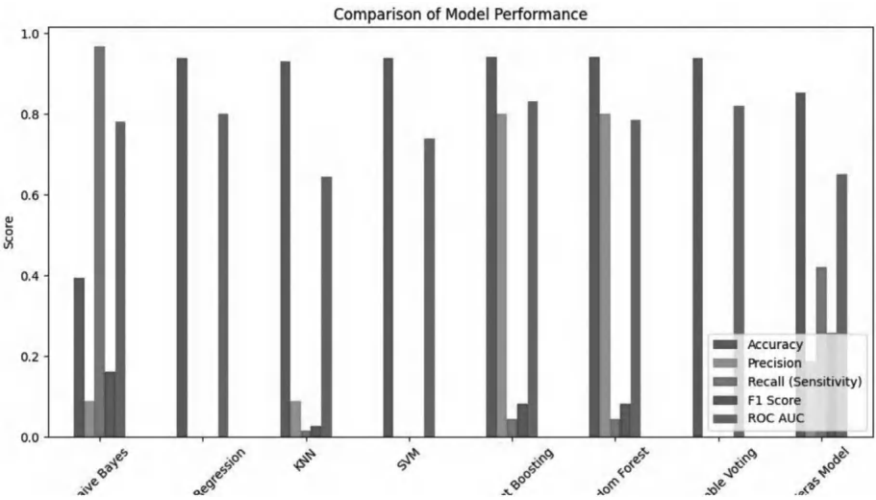


Figure 3.13 Vehicle claim fraud detection – deep feedforward network model vs other models – quality measure benchmark (bar plot).

Source: Own work, with Python/Keras.

the majority class, making them ineffective for fraud detection despite their high accuracy (~94%). The Gradient Boosting and Random Forest models showed a slight improvement, achieving precision of 80% but with low recall (4.3%), meaning they were highly conservative in flagging fraud cases, missing a large number of them.

The Keras deep learning model exhibited a more balanced approach compared to traditional machine learning models. It achieved an accuracy of 85.44%, which is lower than some other models, but more meaningful when considering recall (41.94%) and precision (18.61%). This indicates that the model was able to capture a larger portion of fraudulent cases compared to models like Random Forest and Gradient Boosting, though still at the cost of generating some false positives. Additionally, its ROC AUC score (0.65), while not as high as Naïve Bayes (0.78) or Logistic Regression (0.80), suggesting it provides a more useful trade-off between fraud detection and avoiding excessive misclassifications. The Confusion Matrix for Keras further supports this, as it detected 78 fraud cases compared to only eight cases detected by Gradient Boosting.

Overall, the results suggest that no single model excels in all aspects of fraud detection. Naïve Bayes was overly aggressive; Logistic Regression, SVM, and Ensemble Voting ignored fraud entirely; and Random Forest and Gradient Boosting were too conservative. The Keras model provided a middle ground, detecting more fraud than tree-based methods.

The author is aware that implementing imbalance prevention techniques in “classical” machine learning methods would lead to their improvement and the “winning” of neural network wouldn’t be so clear; however, the aim of this research study was mainly to present adequate techniques for deep learning models and not necessarily the “methods benchmark”.

References

- Adams, E. W., Fagot, R. F., & Robinson, R. E. (1965). A theory of appropriate statistics. *Psychometrika*, 30(2), 99–127.
- Agresti, A. (2002). *Categorical data analysis*. John Wiley and Sons.
- Alkhatib, K., Najadat, H., Hmeidi, I., & Shatnawi, M. K. A. (2013). Stock price prediction using k-nearest neighbor (kNN) algorithm. *International Journal of Business, Humanities and Technology*, 3(3), 32–44.
- Altman, E. I. (1968). Financial ratios, discriminant analysis and the prediction of corporate bankruptcy. *The Journal of Finance*, 23(4), 589–609. <https://doi.org/10.2307/2978933>
- Anderberg, M. R. (1973). *Cluster analysis for applications*. Academic Press.
- Aslam, F., Hunjra, A. I., Fiti, Z., Louhichi, W., & Shams, T. (2022). Insurance fraud detection: Evidence from artificial intelligence and machine learning. *Research in International Business and Finance*, 62, 101744. <https://doi.org/10.1016/j.ribaf.2022.101744>
- Barro, R. J. (1991). Economic growth in a cross-section of countries. *The Quarterly Journal of Economics*, 106(2), 407–443. <https://doi.org/10.2307/2937943>
- Bayes, T. (1763). An essay towards solving a problem in the doctrine of chances. By the late rev. Mr. Bayes, communicated by Mr. Price, in a letter to John Canton. *Philosophical Transactions of the Royal Society of London*, 53(1763): 370–418.
- Beaver, W. H. (1966). Financial ratios as predictors of failure. *Journal of Accounting Research*, 4, 71–111.

- Berkson, J. (1944). Application of the logistic function to bio-assay. *Journal of the American Statistical Association*, 39, 357–365.
- Billard, L., & Diday, E. (2006). *Symbolic data analysis, conceptual statistics and data mining*. John Wiley & Sons.
- Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
- Bliss, C. I. (1934). The method of probits. *Science*, 79, 38–39.
- Bock, H. H., & Diday E. (Ed.). (2000). *Analysis of symbolic data. Explanatory methods for extracting statistical information from complex data*. Springer-Verlag.
- Breiman, L., Friedman, J., Olshen, R., & Stone, C. (1984). *Classification and regression trees*. Wadsworth, Monterey.
- Breiman, L. (1998). Arcing classifiers. *The Annals of Statistics*, 26(3), 801–849.
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32.
- Breiman, L. (1994). *Bagging Predictors*, Technical Report, no. 421, September 1994, partially supported by NSF grant DMS-9212419, Department of Statistics University of California, Berkeley, California 94720.
- Brown, I., & Mues, C. (2012). An experimental comparison of classification algorithms for imbalanced credit scoring data sets. *Expert Systems with Applications*, 39(3), 3446–3453.
- Card, D., & Krueger, A. B. (1994). Minimum wages and employment: A case study of the fast-food industry in New Jersey and Pennsylvania. *American Economic Review*, 84(4), 772–793.
- Carmone, F. J., Kara, A., & Maxwell, S. (1999). HINoV: A new method to improve market segment definition by identifying noisy variables. *Journal of Marketing Research*, 36, 501–509.
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. <https://doi.org/10.1145/2939672.2939785>
- Chen, Y., Jiang, H., Li, C., Jia, X., & Ghamisi, P. (2016). Deep feature extraction and classification of hyperspectral images based on convolutional neural networks. *IEEE Transactions on Geoscience and Remote Sensing*, 54(10), 6232–6251. <https://doi.org/10.1109/TGRS.2016.2603172>
- Cover, T. M., & Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.
- Cragg, J. G., & Uhler, R. S. (1970). The demand for automobiles. *The Canadian Journal of Economics/Revue Canadienne d'Economie*, 3(3), 386–406.
- De Soete, G. (1986). Optimal variable weighting for ultrametric and additive tree clustering. *Quality & Quantity*, 20, 169–180.
- Deakin, E. B. (1972). A discriminant analysis of predictors of business failure. *Journal of Accounting Research*, 10(1), 167–179.
- Dudek, A. (2013). *Metody analizy danych symbolicznych w badaniach ekonomicznych [Methods of symbolic data analysis in economic research]* (No. 226). Wydawnictwo Uniwersytetu Ekonomicznego we Wrocławiu Wrocław
- Fisher R. A. (1922). The goodness of fit of regression formulae, and the distribution of regression coefficients. *Journal of Royal Statistical Society*, 85(4), 597–612.
- Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7(2), 179–188.
- Fix, E., & Hodges, J. L. (1951). Discriminatory analysis: Nonparametric discrimination, consistency properties. *Technical Report 4, USAF School of Aviation Medicine*.
- Freund, Y., & Schapire, R. E. (1997). A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, 55(1), 119–139.
- Friedman, J. H. (1989). Regularized discriminant analysis. *Journal of the American Statistical Association*, 84(405), 165–175.

- Friedman, J. H. (1991). Multivariate adaptive regression splines. *Annals of Statistics*, 19, 1–141.
- Friedman, J. H., & Meulman, J. J. (2004). Clustering objects on subsets of attributes (with discussion). *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 66(4), 815–849.
- Friedman, J. H., & Stuetzle, W. (1981). Projection pursuit regression. *Journal of the American Statistical Association*, 76, 817–823.
- Galton, F. (1877). Typical laws of heredity. *Nature*, 15, 492–495, 512–514, 532–533.
- Galton, F. (1885). Types and their inheritance [Presidential address, Section H, Anthropology]. *Nature*, 32, 507–510.
- Gatnar, E. (2009). Analiza dyskryminacyjna [Discriminant analysis]. In M. Walesiak & E. Gatnar (eds.), *Statystyczna analiza danych z wykorzystaniem programu R [Statistical data analysis with use of R program]* (pp. 193–237). PWN, Warsaw.
- Gauss, C. F. (1809). *Theoria Motus Corporum Coelestium in Sectionibus Conicis Solem Ambientium*, Friedrich Perthes and I.H. Besser, Hamburg.
- Gauss, C. F. (1821). *Theoria combinationis observationum erroribus minimis obnoxiae*, Commentationes societatis Regiae scientiarum Gottingensis recentiores 5, 33–90, Getynga.
- Guyon, I., & Elisseeff, A. (2003). An introduction to variable and feature selection. *Journal of Machine Learning Research*, 3, 1157–1182.
- Hackeling, G. (2017). *Mastering machine learning with Scikit-learn*. Packt Publishing Ltd.
- Hair, J. F., Black, W. C., Babin, B. J., & Anderson, R. E. (2009). *Multivariate data analysis* (7th ed.). Pearson.
- Hancock, J. T., & Khoshgoftaar, T. M. (2020). Survey on categorical data for neural networks. *Journal of Big Data*, 7(1), 28. <https://doi.org/10.1186/s40537-020-00305-w>
- Hand, D. (1982). *Kernel Discriminant Analysis*. Wiley.
- Hand, D. J., & Henley, W. E. (1997). Statistical classification methods in consumer credit scoring: A review. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, 160(3), 523–541.
- Hart, P. E. (1968). The condensed nearest neighbor rule. *IEEE Transactions on Information Theory*, 14, 515–516.
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer.
- Heckman, J. (1976). Simultaneous equations models with continuous and discrete endogenous variables and structural shifts. In S. Goldfeld & R. Quandt (Eds.), *Studies in non-linear estimation* (pp. 235–272). Ballinger.
- Ho, T. K. (1998). The random subspace method for constructing decision forests. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 20, 832–844.
- Hotelling, H. (1933). Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24, 417–441.
- Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. <https://doi.org/10.48550/arXiv.1502.03167>
- Jajuga, K., & Walesiak, M. (2000). Standardisation of data set under different measurement scales. In: R. Decker, W. Gaul (Eds.), *Classification and information processing at the turn of the millennium* (pp. 105–112). Springer-Verlag.
- Johnson, R. A., & Wichern, D. W. (2007). *Applied multivariate statistical analysis* (6th ed.). Prentice-Hall.
- Jolliffe, I. T. (2002). *Principal component analysis*. Springer-Verlag.
- Kass, G. V. (1980). An exploratory technique for investigating large quantities of categorical data. *Applied Statistics*, 29(2), 119–127.
- Keynes, J. M. (1936). *The general theory of employment, interest, and money*. Macmillan.
- Kooperberg, C., Bose, S., & Stone, C. J. (1997). Polychotomous regression. *Journal of the American Statistical Association*, 92, 117–127.

- Kursa, M. B., & Rudnicki, W. R. (2010). Feature selection with the Boruta package. *Journal of Statistical Software*, 36(11), 1–13. <https://doi.org/10.18637/jss.v036.i11>
- Legendre, A. M. (1805). *Nouvelles méthodes pour la détermination des orbites des comètes*, Appendix “Sur la Méthode des moindres carrés”.
- Liberati, C., Camillo, F., & Saporta, G. (2012, August). Kernel discrimination and explicative features: An operative approach. In A. Colubi (Ed.), *Compstat 2012* (pp. 507–518).
- MacKay, D. J. C. (2003). *Information theory, inference, and learning algorithms*. Cambridge University Press.
- Maddala G. S. (1986). *Limited-Dependent and Qualitative Variables in Econometrics*. Cambridge University Press.
- Magidson J. (1994). The CHAID approach to segmentation modeling: Chi-squared automatic interaction detection. In R. P. Bagozzi (Ed.), *Advanced methods of marketing research* (pp. 118–159). Cambridge Press.
- Matthews, R. (2001, December 25). Storks deliver babies ($p = 0.008$). *Teaching Statistics*, 22, 36–38. <https://doi.org/10.1111/1467-9639.00013>
- McFadden D. (1973). Conditional logit analysis of qualitative choice behaviour. In P. Zarembka (Ed.), *Frontiers in Econometrics* (pp. 105–142). Academic Press.
- McKelvey R. D., & Zavoina W. (1975). A statistical model for the analysis of ordinal level dependent variables. *Journal of Mathematical Sociology*, 4, 103–120.
- Merton, R. C. (1974). On the pricing of corporate debt: The risk structure of interest rates. *The Journal of Finance*, 29(2), 449–470.
- Milligan, G. W., & Cooper, M. C. (1988). A study of standardization of variables in cluster analysis. *Journal of Classification*, 5, 181–204.
- Ng, A. Y. (2004, July). Feature selection, L 1 vs. L 2 regularization, and rotational invariance. In C. Brodley (Ed.), *Proceedings of the Twenty-First International Conference on Machine Learning* (p. 78).
- Ohlson, J. A. (1980). Financial ratios and the probabilistic prediction of bankruptcy. *Journal of Accounting Research*, 18(1), 109–131.
- Pearson, K. (1901). On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(6), 559–572.
- Phua, C., Alahakoon, D., & Lee, V. (2010). Minority report in fraud detection: Classification of skewed data. *ACM SIGKDD Explorations Newsletter*, 6(1), 50–59.
- Raghunathan, T.W., Lepkowski, J.M., Van Hoewyk, J., & Solenbeger P. (2001). A multivariate technique for multiply imputing missing values using a sequence of regression models. *Survey Methodology*, 27, 85–95.
- Ripley, B. D. (1996). *Pattern recognition and neural networks*. Cambridge University Press.
- Rosen, S. (1974). Hedonic prices and implicit markets: Product differentiation in pure competition. *Journal of Political Economy*, 82(1), 34–55. <https://doi.org/10.1086/260169>
- Rousseeuw, P. J., & Struyf, A. (1998). Computing location depth and regression depth in higher dimensions. *Statistics and Computing*, 8(3), 193–203.
- Schmidt, P., & Strauss, R. P. (1975). The prediction of occupation using multiple logit models. *International Economic Review*, 16(2), 471–486.
- Shapire, R. E. (2003). The boosting approach to machine learning: An overview. In D. D. Denison, M. H. Hansen, C. C. Holmes, B. Mallick, & B. Yu (Eds.), *Nonlinear Estimation and Classification* (pp. 149–171). Springer.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *The Journal of Machine Learning Research*, 15(1), 1929–1958
- Stevens, S. S. (1959). Measurement, psychophysics and utility. In C.W. Churchman, P. Rattoosh (Eds.), *Measurement: Definitions and theories* (pp. 18–61). Wiley.
- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society: Series B (Methodological)*, 58(1), 267–288. <https://doi.org/10.1111/j.2517-6161.1996.tb02080.x>

- van Belle, G. (2011). *Statistical rules of thumb*. John Wiley & Sons.
- Vapnik, V. (1998). *Statistical learning theory*. John Wiley & Sons.
- Walesiak, M. (2009). Podstawowe zagadnienia statystycznej analizy wielowymiarowej [Basic issues of statistical multivariate analysis]. In M. Walesiak & E. Gatnar (Eds.), *Statystyczna analiza danych z wykorzystaniem program [Statistical data analysis with use of R program]* (pp. 62–80). PWN, Warsaw.
- Zhang, C., Bengio, S., Hardt, M., Recht, B., & Vinyals, O. (2016). Understanding deep learning requires rethinking generalization. <https://doi.org/10.48550/arXiv.1611.03530>
- Zhang, C., Bengio, S., Hardt, M., Recht, B., & Vinyals, O. (2021). Understanding deep learning (still) requires rethinking generalization. *Communications of the ACM*, 64(3), 107–115.

4 Explanatory model analysis for deep learning models

4.1 Beyond black box

As we have already seen in [Chapter 3](#), a typical deep learning/deep neural network model requires estimation of tens/hundreds of thousands/millions of trainable parameters – in the training process (cf. [Figure 3.2](#)). Of course, all of them are “public” and can be observed on an ongoing basis during the model learning process, but the limitations of human cognition make their interpretation by researchers or ordinary users practically impossible. This is the origin of the belief that the internal workings of these models remain largely unclear, which is why they are often referred to as “black boxes”.

On the other hand, these models operate in sensitive areas such as medical diagnostics or autonomous driving, where understanding the decisions process may be crucial. Also, in economic applications, we usually expect rationality in decision-making, hence the strong need to “understand” how models work. To address these challenges, researchers have developed various interpretability techniques, including visualization of learned features and model-agnostic tools such as Local Interpretable Model-agnostic Explanations (LIME) and SHapley Additive exPlanations (SHAP) (some researchers also add attention to the mechanism as a self-interpretation technique, but due to the rapid development of transformer-based models, in the author’s opinion, it is yet too early to describe it properly). It seems that balancing the power of deep learning with the need for clear, understandable reasoning is essential to ensure that machine learning models lead to deep learning models that are both effective and responsible.

To face these challenges, a new domain of machine learning/deep learning/AI has arisen called explanatory model analysis (EMA) (see Biecek & Burzykowski, 2021) or, more recently, explainable artificial intelligence (XAI).

The EMA is a wide family of methods and techniques used to unveil the patterns used in the model prediction phase. It addresses the black box dilemma trying to bridge the gap between human understanding and the underlying logic behind deep learning models. The EMA tries to answer the following questions:

- What is the logic learned by the model and the patterns used for prediction?
- Which features (variables) were most important in the model training phase?

- Which features affect the predictions with the highest impact?
- Are there any interactions between features?
- Why did the model make the prediction for a particular instance (object)?
- Which features contributed most to the prediction for a particular instance?
- How sensitive is the prediction for a particular instance(s) to small changes in input values?
- Does the model overfit to specific features?
- Can non-experts understand how the model works?

The origins of EMA can be traced back to early attempts at interpreting indicators in linear regression models. It soon became evident, however, that a better measure than the coefficients themselves were the values of F and t statistics for individual variables and the entire model (see [Maddala, 2001, Chapter 4](#); [Koop, 2006, Chapter 6](#)). A milestone in the development of EMA was the creation of classification trees and the classification and regression tree (CART) method for which the Gini coefficient in subsequent splits can be treated as a measure of variable importance, which may be treated as fact that the answer to the question, *Which features (variables) were most important in the model training phase?* It is inherently “embedded” within the model. [Table 4.1](#) summarizes the models that are inherently interpretable.

Table 4.1 Models internally interpretable

<i>Model name</i>	<i>Coefficients/interpretation elements</i>	<i>Meaning/explanation</i>
Linear regression	Beta coefficients (slopes and intercept)	Each coefficient represents the expected change in the outcome variable for a one-unit increase in the predictor, holding other predictors constant.
Logistic regression	Log-odds coefficients (or exponentiated odds ratios)	Coefficients indicate the change in the log odds of the dependent event for a one-unit increase in the predictor; exponentiated values (odds ratios) show the multiplicative change in the odds.
Generalized linear models (GLM)	Parameter estimates from the linear predictor (often on a transformed scale via a link function)	Coefficients represent the effect of predictors on the transformed outcome (via the link function), allowing interpretation in the context of different distributions (e.g., Poisson, binomial).

(Continued)

Table 4.1 (Continued)

<i>Model name</i>	<i>Coefficients/interpretation elements</i>	<i>Meaning/explanation</i>
Generalized additive models (GAM)	Smooth functions (splines) for each predictor	The shapes of the smooth functions illustrate the non-linear relationship between each predictor and the outcome, facilitating an understanding of marginal effects across the predictor's range.
Least Absolute Shrinkage and Selection Operator (LASSO)	Nonzero coefficients	Nonzero coefficients in a LASSO model provide a clear indication of the most predictive features.
Decision tree	Splitting rules (decision thresholds at each node)	Instead of numerical coefficients, decision trees use explicit if-then rules at each node that partition the feature space; these rules reveal how the tree segments the data and leads to predictions.
Decision rules	If-then rule sets with associated weights or support measures	Each rule is an explicit statement (e.g., "if age > 30 then ...") that contributes to the final prediction; weights indicate the strength or importance of each rule in the decision process.
RuleFit	Combined linear coefficients for original features and rule-based features derived from tree ensembles	Coefficients show the additive contribution of both the original features and the extracted decision rules, offering insights into how different components collectively influence the prediction.
Other interpretable models (e.g., Naïve Bayes, Bayesian Rule Lists)	Probabilistic parameters (conditional probabilities, priors) or rule weights	These models provide direct insights into the relationships between predictors and outcomes through interpretable probability estimates or explicit logical rules, enabling transparent decision-making processes.

Source: Biecek and Burzykowski (2021) and Molnar (2020).

Tsang and Benoit (2023), proposed the name post-hoc interpretable models for models that are not internally explainable. Neural networks, with their black-box nature, were hard to interpret. Some attempts with sensitivity analysis were proposed (Sung, 1998; Zurada et al., 1994), but due to the increasing complexity of analyzed models, model-agnostic techniques like partial dependency plots and feature permutation methods have been developed.

There are several ways to taxonomize the techniques of EMA, but the most key factors for such taxonomy are:

- By the level of integration with the model – methods that are aware of the mode of operation of the model, highly related to the model (model dependent techniques) and methods that do not rely on specific features of the model, but are more general and applicable for a wider range of models (model agnostic techniques);
- By the scope of explanation – methods trying to look into overall behavior of the model across an entire dataset (global explanation techniques) and methods explaining a particular prediction for a specific case (local explanation techniques);
- By the goal of analysis – methods measuring how much each feature (variable) contributes to the model’s predictions (feature importance techniques), methods estimating how the change in variable values affects the overall model prediction (feature effects techniques), methods detecting interaction of features in context of final prediction (interaction effects techniques);
- By the data type analyzed – techniques for model analyzing numerical/categorical/image/text datasets.

Table 4.2 presents the main techniques of EMA.

4.1.1 Permutation methods

Permutation-based methods are an extension of quite an intuitive idea. If we cannot determine the importance of a variable based on the internal structure of the model, we can approach it as a black box and try to see what happens if we change the input data. We can agree that any attempts to “spoil” important data should result in a deterioration in prediction quality. On the other hand, if the variable did not participate or participated to a minimal extent in the formulas determining the prediction, then even its “spoiling” should not significantly change the quality of the model, and the prediction. In this reasoning, a measure of changes in the sum of squares of error, accuracy, or any other loss function is inversely proportional to the variable’s significance in the prediction decision-making process. Thus, it is enough to “spoil” the variables and calculate how the quality of the prediction deteriorated to obtain a numerical estimate of the significance of the variable in the model. The most common way of “spoiling” a variable is to make a permutation of the values in the column of the dataset corresponding to that variable, which can be formalized in the following steps.

Table 4.2 Main techniques of EMA

Category	Technique	Scope	Model dependency	Use case
Feature Importance	SHAP, permutation importance	Global/local	Model-agnostic	Identify important features
Feature Effects	Partial dependence plots (PDP), accumulated local effects (ALE), and individual conditional expectation (ICE)	Global/local	Model-agnostic	Assess impact of feature changes
Local Explanations	LIME, counterfactual explanations	Local	Model-agnostic	Explain specific predictions
Interaction Effects	SHAP interaction values, Two-way PDP	Global/local	Model-agnostic	Identify feature interactions
Visualization	Grad-CAM, saliency maps	Local	Model-specific	Explain image/text models
Surrogate Models	Decision tree approximation	Global	Model-agnostic	Simplify and interpret complex models

Source: Own work based on [Biecek and Burzykowski \(2021\)](#) and [Molnar \(2020\)](#).

Let:

X – dataset with m explanatory variables and n instances (objects);

Y – column vector of the dependent variable observed values;

$\hat{Y}$ – column vector of the dependent variable predicted values;

$P(\hat{Y}, X, Y)$ – performance metrics (loss function) for the model.

- 1 Train the model;
- 2 Compute $p_0 = P^0(\hat{Y}, X, Y)$ – the initial value of loss function;
- 3 For given $1 < k < m$ shuffle (permute) the column vector X_k . The matrix X after permutation becomes X^{*k} ;
- 4 Compute model predictions $\hat{Y}^{*k}$ for X^{*k} ;
- 5 Compute $p_{*k} = P(\hat{Y}^{*k}, X^{*k}, Y)$;
- 6 $vip_k = p_{*k} - p_0$ estimates the importance for variable k in process of prediction (alternatively used in form $vip_k = \frac{p_{*k}}{p_0}$).

The process is usually repeated for all variables k in $1 \dots m$. In real analysis some form of Monte Carlo procedure is very often used to repeat the calculations in *it*

steps for which finally the mean value of value importance measure is computed and used for further evaluation.

Let's examine the algorithm on an easy example with an artificial dataset containing three variables of which two of them seem to be noisy in relation to an explained variable and the third is probably highly related to a dependent variable.

The dataset has been artificially generated by the following Python code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor

np.random.seed(20050325)
n_samples = 500
X1 = np.random.normal(size=n_samples)
X2 = np.random.normal(size=n_samples)
X3 = np.random.uniform(1, 4, size=n_samples)
y = 2 * X3 + np.random.normal(scale=0.1, size=n_samples)

data = pd.DataFrame({
    'X1': X1,
    'X2': X2,
    'X3': X3,
    'y': y
})

simple_stats = data.describe()
print("\nSimple Statistics:")
print(simple_stats)
```

Figure 4.1 presents base characteristics of a generated dataset and Figure 4.2 presents the scatterplots of explanatory variables X_1 , X_2 , X_3 confronted to dependent variable Y .

Simple Statistics:				
	X1	X2	X3	y
count	500.000000	500.000000	500.000000	500.000000
mean	-0.008128	-0.052737	2.558598	5.120727
std	0.992604	0.981934	0.872747	1.748354
min	-3.540044	-2.899891	1.006806	1.948877
25%	-0.612557	-0.748448	1.819182	3.676313
50%	-0.056742	-0.059853	2.641126	5.285973
75%	0.685673	0.665425	3.305837	6.649010
max	3.064959	3.393715	3.991726	8.082718

Figure 4.1 Base statistics of dataset with one explanatory and two noisy variables related to dependent one.

Source: Own work, with Python/NumPy.

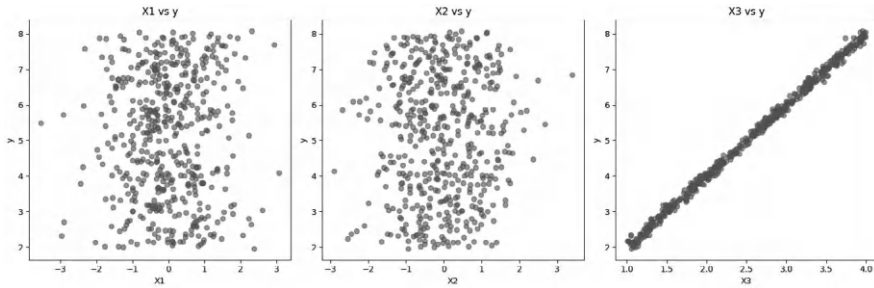


Figure 4.2 Dataset with one explanatory and two noisy variables related to dependent one (scatterplots).

Source: Own work, with Python/NumPy.

To estimate the importance of variables, the regressive multi-layer perceptron will be built, each variable will be shifted 100 times, and the average change in the sum of squares coefficient will be finally calculated. This routine is implemented in the code listed below:

```
X = data[['X1', 'X2', 'X3']]
y = data['y']
mlp = MLPRegressor(hidden_layer_sizes=(10,),
                    activation='relu', max_iter=1000, random_state=20050325)
mlp.fit(X, y)

baseline_loss = mean_squared_error(y, mlp.predict(X)) * len(y)

importance_differences = {}
importance_ratios = {}
for i in range(0, n_vip_iterations):
    for feature in X.columns:
        X_permuted = X.copy()
        np.random.shuffle(X_permuted[feature].values)
        permuted_loss = mean_squared_error(y, mlp.predict(X_
permuted)) * len(y)
        if i==0:
            current_importance_differences_sum = 0.0
            current_importance_ratio_sum = 0.0
        else:
            current_importance_differences_sum =
importance_differences[feature]
            current_importance_ratio_sum =
importance_ratios[feature]
            importance_differences[feature] = permuted_loss -
baseline_loss
            importance_ratios[feature] = permuted_loss /
baseline_loss
```

```

for feature in X.columns:
    importance_differences[feature] =
importance_differences[feature]/n_vip_iterations
    importance_ratios[feature] =
importance_ratios[feature]/n_vip_iterations

print("Variable Importance Difference Average Measures:")
print(importance_differences)

print("\nVariable Importance Ratio Average Measures:")
print(importance_ratios)

```

The results of the analysis are presented in [Figures 4.3](#) (the values of average variable importance measures) and [4.4](#) (corresponding bar chart).

The results obtained align with our expectations. Both the variable importance difference measure and the variable importance ratio measure clearly indicate that the variable **X3** played the most significant role in the model's prediction process, while variables **X1** and **X2** can be considered marginal. In real-world problems involving the modeling of economic phenomena, it is rare to encounter such a clear determination of variable importance. However, the principle that a higher variable importance measure (VIP) value corresponds to greater variable significance in the model's construction remains valid. This principle facilitates the interpretation of variable importance and helps move away from the black-box paradigm.

To observe the method in action for real data we will use the California Housing Dataset. The collection is sewn into the scikit-learn library, so we avoid the need to download files from the repositories or use the application programming interface (API), and its usage requires only one Python command (*fetch_california_housing*).

The California Housing Dataset, first introduced by Pace and Barry in 1997, is a classic dataset for regression tasks, originally derived from the 1990 US Census ([Pace & Barry, 1997](#)). It provides information about the socio-economic and geographic attributes of neighborhoods in California, along with the median house prices.

```

Variable Importance Difference Average Measures:
{'X1': 0.06216302009166608, 'X2': 0.00824152117471197,
 'X3': 30.981728489926255}

Variable Importance Ratio Average Measures:
{'X1': 0.01314025305413128, 'X2':
 0.010416332121595978, 'X3': 1.5750859203637748}

```

Figure 4.3 The variable importance measures.

Source: Own work, with Python/NumPy.

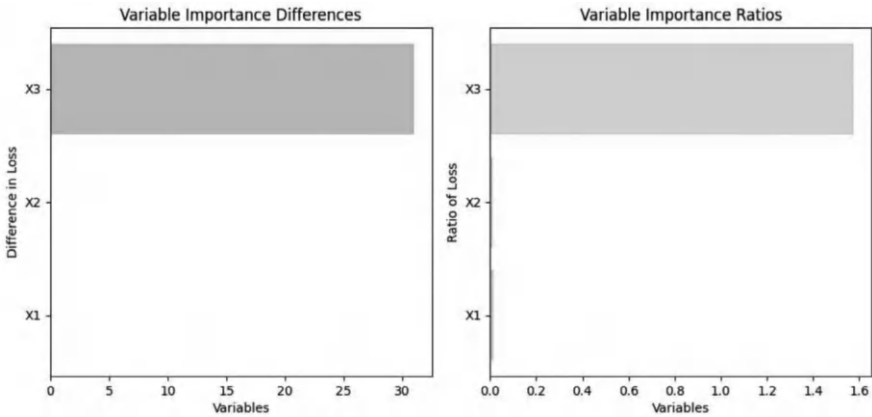


Figure 4.4 The variable importance measures (bar charts).

Source: Own work, with Python/NumPy.

The California Housing Dataset contains 20,640 instances with eight continuous numeric features, a target variable representing the median house value (MedV), and each row corresponding to data from a single block group (neighborhood). Table 4.3, presents the characteristics of variables describing the dataset and Figure 4.5 visualizes the dependent variable distribution on a sketch map of California.

The California Housing Dataset contains geographical coordinates of real estates and is often used for spatial methods. This domain extends the thematic scope of this book and for this reason we will remove *Latitude* and *Longitude* from further analysis.¹ The dataset has been split in train and test set in proportion 80% to 20%.

Table 4.3 The variables in California Housing Dataset

Variable name	Description	Data type
<i>MedInc</i>	Median income in the neighborhood (in tens of thousands of dollars).	Continuous
<i>HouseAge</i>	Median age of houses in the neighborhood.	Continuous
<i>AveRooms</i>	Average number of rooms per household in the neighborhood.	Continuous
<i>AveBedrms</i>	Average number of bedrooms per household in the neighborhood.	Continuous
<i>Population</i>	Total population in the neighborhood.	Continuous
<i>AveOccup</i>	Average number of occupants per household.	Continuous
<i>Latitude</i>	Latitude coordinate of the neighborhood's centroid.	Continuous
<i>Longitude</i>	Longitude coordinate of the neighborhood's centroid.	Continuous

Source: Own research based on scikit-learn.dataset California Housing Dataset.

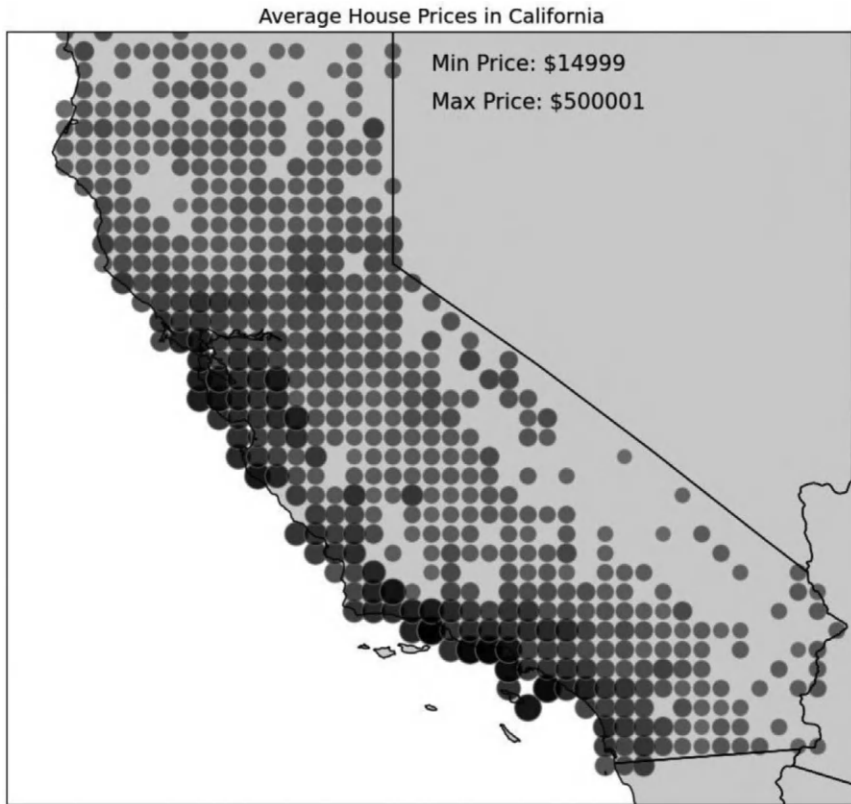


Figure 4.5 California housing dependent variable map visualization.

Source: Own work, with Python/NumPy.

To find which variables most influence the prediction for multi-layer perceptron used for regression purposes, we will use the following Python code:

```
import numpy as np
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPRegressor
from sklearn.inspection import permutation_importance,
    PartialDependenceDisplay
import shap
import matplotlib.pyplot as plt

california_data = fetch_california_housing(as_frame=True)
X = california_data.data
y = california_data.target
```



```

X = X.drop(columns=['Longitude', 'Latitude'])

print(f"Dataset shape: {X.shape}")

scaler = StandardScaler()
X_s = pd.DataFrame(scaler.fit_transform(X))
X_s.columns=X.columns
X=X_s

mlp_regressor = MLPRegressor(
    hidden_layer_sizes=(64, 64, 64),
    activation='relu',
    solver='adam',
    max_iter=5000,
    random_state=25032005
)

mlp_regressor.fit(X.values, y)
perm_importance = permutation_importance(
    estimator=mlp_regressor,
    X=X.values,
    y=y,
    scoring="neg_mean_squared_error",
    n_repeats=10,
    random_state=25032005
)
print(perm_importance)
importance_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": perm_importance.importances_mean
}).sort_values(by="Importance", ascending=False)

plt.figure(figsize=(10, 6))
plt.barh(importance_df["Feature"], importance_
df["Importance"], color="skyblue")
plt.xlabel("Permutation Importance (MSE)")
plt.title("Permutation Feature Importance")
plt.gca().invert_yaxis()
plt.savefig("housing_vip_mlp_results.jpg")
plt.show()

```

Figures 4.6 and 4.7 present the results of the analysis.

From these VIP values, we see that the *average number of bedrooms per household* in a neighborhood ($VIP \approx 3.97$) and the *average number of rooms per household* ($VIP \approx 4.15$) have the strongest influence on the model's predictions of median house value (in thousands of USD). The *median income (in tens of thousands of dollars)* also plays a key role ($VIP \approx 2.03$), suggesting that wealthier areas tend to show higher median house values. By comparison, *total population* in

```
{'importances_mean': array([2.03086086, 0.40406835, 4.14832751,
3.96583483, 0.19935631,
    0.40441978]), 'importances_std': array([0.02207306,
0.00527619, 0.06475405, 0.04086553, 0.00595442,
    0.00587413]), 'importances': array([[2.01161744, 2.05431903,
2.06147982, 2.00181419, 2.06134202,
    2.02351997, 2.01032597, 2.01970632, 2.05009502, 2.01438886],
[0.40665163, 0.4086261, 0.40717642, 0.41074089, 0.40010768,
    0.40109571, 0.4117877, 0.39605819, 0.40007311, 0.39836607],
[4.26433885, 4.07797112, 4.11945704, 4.05806842, 4.14395667,
    4.17049345, 4.12361514, 4.09088116, 4.19962407, 4.23486918],
[3.99075231, 3.93905483, 3.92094439, 4.03426351, 3.98085248,
    3.95889833, 3.93617094, 3.90517645, 4.02771748, 3.96451755],
[0.19753359, 0.19233202, 0.19235243, 0.21261349, 0.20435541,
    0.19815206, 0.20486225, 0.19900429, 0.19611987, 0.1962377 ],
[0.41047855, 0.40367347, 0.39987782, 0.40515371, 0.41254817,
    0.39118004, 0.41022703, 0.40138785, 0.40583464, 0.40383649]])}
```

Figure 4.6 The variable importance measures.

Source: Own work, with Python/NumPy/scikit-learn.

the neighborhood (VIP ≈ 0.20) appears far less influential, indicating that simply knowing how many people live in the area adds little predictive power. The *median age of houses* (VIP ≈ 0.40) and the *average number of occupants per household* (VIP ≈ 0.40) have moderate to low importance, which implies their impact on

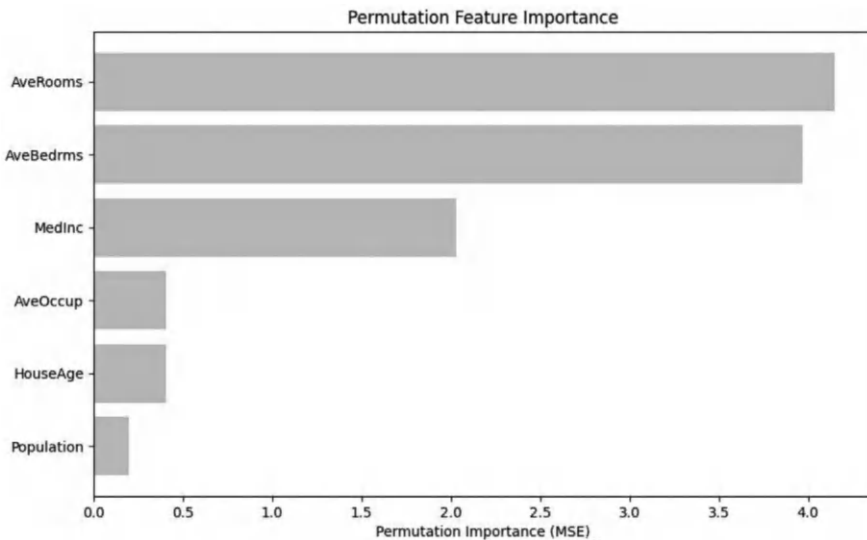


Figure 4.7 The variable importance measures (bar chart).

Source: Own work, with Python/NumPy/scikit-learn.

house values is relatively smaller than the size/configuration of the homes themselves or the neighborhood's income levels. Overall, larger or better appointed homes (reflected by room and bedroom counts) and higher incomes seem to be the primary drivers of median house value in this model, while neighborhood population size and housing age contribute less.

4.1.2 SHAP

Another way to determine the importance of a variable is to check how the addition of a variable affects the prediction value relative to the average value for the entire data set. The algorithm for determining *breakdown plots values* can be represented in the following steps.

Let:

X – dataset with m explanatory variables and n instances (objects);

Y – column vector of the dependent variable observed values;

$\hat{Y}$ – column vector of the dependent variable predicted values;

l – object (instance) index for which the analysis is conducted;

O – the order of variables $1 < k < m$ in analysis. In simplest form just $1, 2, \dots, m$.

- 1 Train the model;
- 2 Calculate the $\hat{Y}_0 = \frac{\sum_{i=1}^n \hat{Y}_i}{n}$ the average prediction value over the dataset (and initial explanation estimation);
- 3 For each dependent variable $1 < k < m$ (in order O) repeat steps 3–6;
- 4 Compute $\hat{Y}_l^{*k}$ average prediction for dataset for which variables $X_{O_1}, X_{O_2}, \dots, X_{O_k}$ values in whole dataset are set to the values of object X_l ;
- 5 Compute $\text{bdp}_{v_k} = \hat{Y}_l^{*k} - \hat{Y}_0$;
- 6 Set $\hat{Y}_0 = \hat{Y}_l^{*k}$ (this way subsequent bd_{v_k} stores the additive change of prediction)
- 7 Final prediction $\hat{Y}_l^{*m}$ is equal to actual prediction of object l .

It is worth noting that while VIP measures are measured in values corresponding to the loss function, breakdown values corresponds to the change in the explained variable.

Breakdown plot values are intuitive, straightforward to understand, and can be applied to any model that returns a single prediction for a specific instance, making them a popular choice among model-agnostic explanation techniques. They are relatively simple to implement, but have notable limitations: they are sensitive to variable interactions, and their results may lack stability and robustness due to the order in which features are added. Moreover, in models with a large number of features, breakdown plots can become overly complex, including many features with minimal contributions to the prediction.

Again, the dataset has been artificially generated by the following Python code, this time with one noisy and two significant variables:

```
from itertools import combinations
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.neural_network import MLPRegressor
from math import factorial

np.random.seed(20050325)
n_samples = 500
X1 = np.random.normal(size=n_samples)
X2 = np.random.uniform(1, 4, size=n_samples)
X3 = np.random.uniform(1, 4, size=n_samples)
y = X2 + 3 * X3 + np.random.normal(scale=0.1, size=n_
samples) # y is twice X3 with noise

data = pd.DataFrame({
    'X1': X1,
    'X2': X2,
    'X3': X3,
    'y': y
})

simple_stats = data.describe()
print("\nSimple Statistics:")
print(simple_stats)
```

Figure 4.8 presents the base characteristics of the generated dataset and Figure 4.9 presents the scatterplots of explanatory variables X_1 , X_2 , X_3 confronted to dependent variable Y .

Simple Statistics:			
	X1	X2	X3
count	500.000000	500.000000	500.000000
mean	-0.008128	2.504352	2.527076
std	0.992604	0.853157	0.865934
min	-3.540044	1.002241	1.006806
25%	-0.612557	1.780519	1.793161
50%	-0.056742	2.463709	2.589386
75%	0.685673	3.287221	3.277021
max	3.064959	3.996319	3.988845

Figure 4.8 Base statistics of dataset with one noisy and two explanatory variables related to dependent one.

Source: Own work, with Python/NumPy.

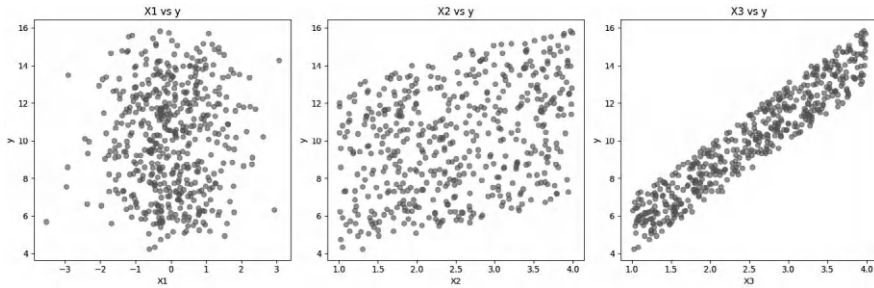


Figure 4.9 Dataset with two noisy and one explanatory variable related to dependent one (scatterplots).

Source: Own work, with Python/NumPy.

The following Python code implements breakdown plot value estimation for fourth instance (please note that due to Python indexing, instance is shifted by one):

```
instance_nr=4
instance = X.iloc[instance_nr-1]
current=baseline = mlp.predict(X).mean()
contributions = {}
XX=X.copy()
for feature in XX.columns:
    XX.loc[:,feature]=instance[feature]
    modified_prediction = mlp.predict(XX).mean()
    contributions[feature] = modified_prediction - current
    current=modified_prediction

print("\nBreakdown Contributions for Instance 4:")
print(f"Baseline: {baseline}")
for feature, contribution in contributions.items():
    print(f"{feature}: {contribution}")
print(f"Final Prediction: {current}")
```

The results of the analysis are presented in [Figures 4.10](#) (the attribution for each variable) and [4.11](#) (corresponding bar chart).

```
Breakdown Contributions for Instance 4
Baseline: 10.07430441815282
X1: -0.051449886792415356
X2: -1.6829613539018258
X3: 0.8991662070462159
Final Prediction: 9.239059384504795
```

Figure 4.10 The breakdown variable attribution values.

Source: Own work, with Python/NumPy.

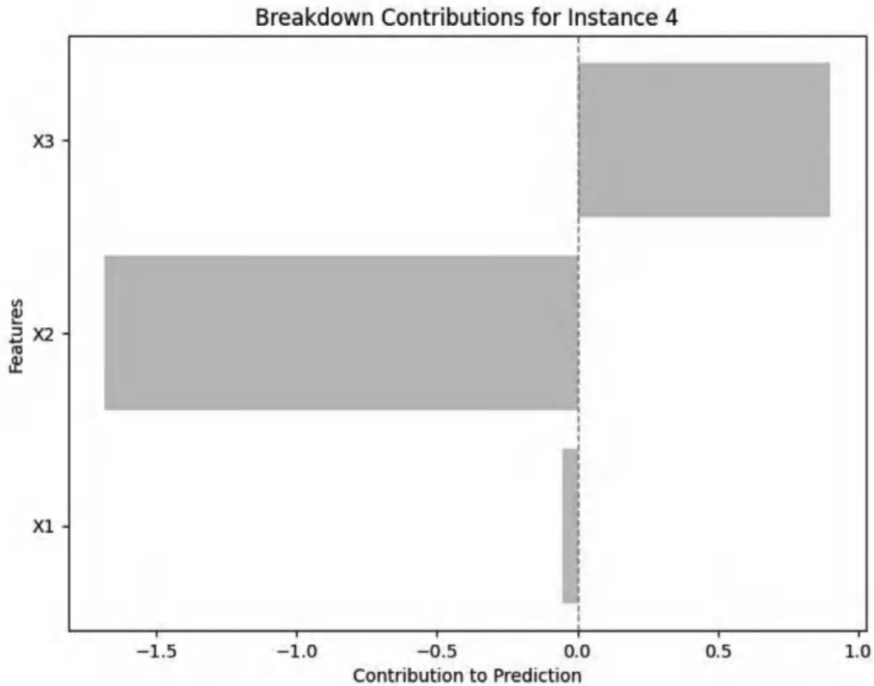


Figure 4.11 The breakdown variable attribution values (bar chart).

Source: Own work, with Python/NumPy.

As we noticed earlier, the breakdown values are vulnerable to variable ordering in the procedure. To observe this, the analysis will be repeated with the variables included in reverse order (which requires the change in the code from `for feature in XX.columns:` to `for feature in reversed(X.columns.tolist()):`). The results of the analysis are presented in Figures 4.12 and 4.13.

The value of attribution of second and third variable in the prediction of instance 4 is much higher than the attribution of the first one, but the actual values slightly differ for the two procedures. For the real datasets, the differences must be

```

Breakdown Contributions for Instance 4
Baseline: 10.07430441815282
X3: 1.0567357687126453
X2: -1.7381716048285742
X1: -0.15380919753209632

```

Figure 4.12 The breakdown variable attribution values – variables in reverse order.

Source: Own work, with Python/NumPy.

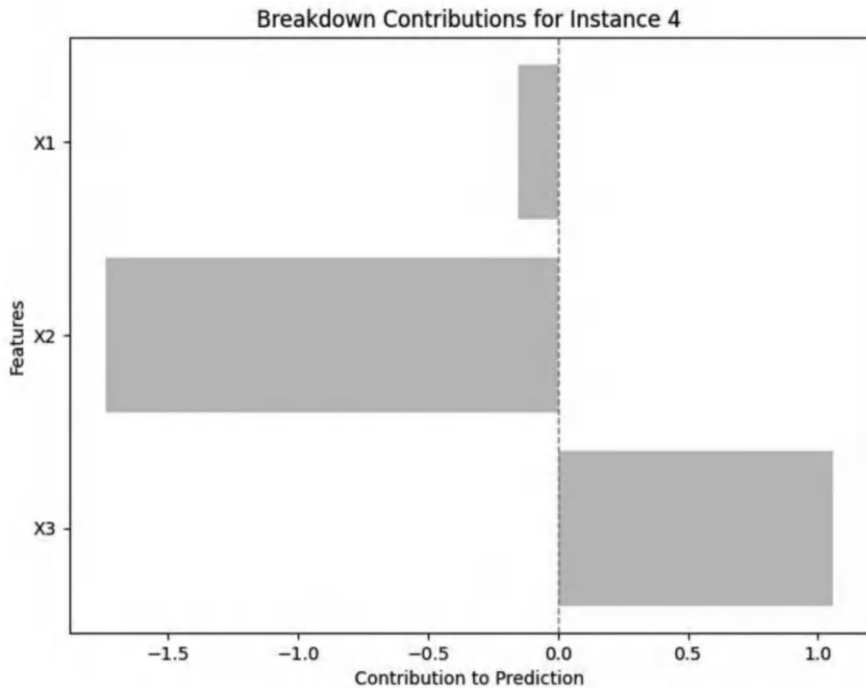


Figure 4.13 The breakdown variable attribution values – variables in reverse order (bar chart).

Source: Own work, with Python/NumPy.

significantly higher (see e.g., [Biecek & Burzykowski, 2021, Chapter 6](#)). To address this issue, two techniques may be introduced:

- Breakdown with interaction effect detection;
- SHAP plots.

An interaction effect occurs when the combined effect of a group of variables on the dependent variable does not equal the sum of the effects of the individual variables. Interaction may be treated as a deviation from additivity. This situation is very common in economic data (education, and experience modeling wages, income, and interest rates modeling consumption, location quality and presence of amenities modeling house prices, economic growth, and tax rates modeling tax revenue etc.)

To detect and mitigate interaction effects, the following interaction has been proposed by [Gosiewska and Biecek \(2019\)](#).

Let:

X – dataset with m explanatory variables and n instances (objects);

Y – column vector of the dependent variable observed values;

$\hat{Y}$ – column vector of the dependent variable predicted values;

l – object (instance) index for which the analysis is conducted.

- 1 For each variable k compute $\hat{Y}_l^{k\text{-only}}$ the average prediction for which variable X_k values in the whole dataset are set to the value of variable k of object X_l ;
- 2 For each pair of variables k_1, k_2 Compute $\hat{Y}_l^{k_1, k_2\text{-pair}}$ average prediction for which variables X_{k_1} and X_{k_2} values in the whole dataset are set to the values of variable k_1 and k_2 of object X_l
For each pair of variables k_1, k_2 estimate the net effect of interaction:

$$NEF_{k_1 k_2} = \hat{Y}_l^{k_1, k_2\text{-pair}} - \hat{Y}_l^{k_1\text{-only}} - \hat{Y}_l^{k_2\text{-only}} \quad (4.1)$$

- 3 Rank the variables according to their join net effect and use the achieved order in breakdown values estimation algorithm.

As we already noticed, the order of calculation of breakdown values is crucial, so probably the “ideal” algorithm of estimation of variables attribution should just examine all possible orders (permutations) and calculate the average or weighted average of the values achieved in subsequent steps. But due to calculational complexity, the implementation of such a method is executable in reasonable time only for a small number of variables. The second column of Table 4.4 presents the number of subsets that have to be examined in this case.

Even for the most modern central processing units (CPUs) and graphics processing units (GPUs) the procedure goes ineffectively for a number of variables larger than 20. But we may measure the joint attribution for various combinations

Table 4.4 The number of sets to examine for “all permutations” approach and SHAP approach

n	All permutations ($m!$)	SHAP $m * (2^m - 1)$
1	1	1
2	2	6
3	6	21
4	24	60
5	120	155
6	720	378
7	5040	889
8	40320	2040
9	362880	4599
10	3628800	10230
20	2432902008176640000	20971500
50	304140932017133780436126081660647688443776 415689605120000000000000	56294995342131150
100	933262154439441526816992388562667004907159 6826438162146859296389521759999322991560 8941463976156518286253697920827223758251 1852109168640000000000000000000000000000	12676506002282294 0149670320537500

Source: Own calculations.

of explanatory variables using the Shapley values originating from game theory. In the original work, Shapley (1953) uses the values to determine each player's contribution to the benefit gained from cooperating with other players in a coalition. Changing “player” to “variable” and “benefit from game” to “model prediction” allows us to apply a similar method in EMA. The idea behind Shapley values is to measure how the dependent variable estimated value changes after adding specific variable to the subset of variables actually taking part (playing) in the prediction. The weighted average is used to set the final measure with the weight dependent on the subset length (biggest weights for number of elements closer to 0 or $m-1$, smaller weights for number of elements if variable subset equal to $\frac{m}{2}$ where m is the overall number of variables in dataset analyzed).

The algorithm for finding SHAP values for certain object explained and certain variable may be stated as follows.

Let:

X – dataset with m explanatory variables and n instances (objects);

Y – column vector of the dependent variable observed values;

$\hat{Y}$ – column vector of the dependent variable predicted values;

l – object (instance) index for which the analysis is conducted;

k – feature (variable) index for which the analysis is conducted.

- 1 Train the model;
- 2 Calculate the $\hat{Y}_0 = \frac{\sum_{i=1}^n \hat{Y}_i}{n}$ the average prediction value over the dataset (and initial explanation estimation);
- 3 Let:

$$V_{-k} = \{1, 2, \dots, m\} \setminus \{k\} \quad (4.2)$$

(The set of all variable indices with k excluded);

- 4 For each s in $0, 1, \dots, m-1$;
- 5 For all subsets S of V_{-k} of size s

Calculate:

$\hat{Y}_l^{*S}$ be average prediction for dataset for which variables X_i $i \in S$ values in whole dataset are set to the values of object X_l

$\hat{Y}_l^{*S \cup \{k\}}$ be average prediction for dataset for which variables X_i ; $i \in S$ and and Shapley value:

$$SHAP_s = \frac{s!(m-s-1)!}{m!} \left(\hat{Y}_l^{*S \cup \{k\}} - \hat{Y}_l^{*S} \right) \quad (4.3)$$

- 6 Sum all the $SHAP_s$ (4.3) values.

The SHAP method was introduced by Štrumbelj and Kononenko (2010) and Štrumbelj and Kononenko (2014), and popularized by Lundberg and Lee (2017).

It's widely used due to its strong theoretical background and reliability of explanation. The main limitation of the method is its time complexity for large models. As we may observe in the third column of [Table 4.2](#), the number of calculations is much lower than the full permutations examination, but is still significant. In practical research analysis studies for datasets with more than ~ 25 variables, some form of sub-sampling and Monte Carlo simulation may be used for estimating SHAP values.

To calculate the SHAP coefficients, a procedure similar to the one below may be used:

```
instance_nr=4
instance = X.iloc[instance_nr-1]
m = len(X.columns)
S = set(range(1,m+1))

def value_function(subset,X,mlp,instance):
    data = X.copy()
    for s in subset:
        data[X.columns[s-1]] = instance[X.columns[s-1]]
    print(mlp.predict(data).mean())
    return mlp.predict(data).mean()

def shapley_values(S,v,X,mlp,instance):
    n = len(S)
    shapley_vals = {f"X{i}": 0 for i in S}

    for i in S:
        for subset_size in range(n):
            for T in combinations(S - {i}, subset_size):
                T = set(T)
                weight = factorial(len(T)) * factorial(n -
len(T) - 1) / factorial(n)
                marginal_contribution = v(T |
{i},X,mlp,instance) - v(T,X,mlp,instance)
                shapley_vals[f"X{i}"] += weight *
marginal_contribution

    return shapley_vals
shapley_result = shapley_values(S,
value_function,X,mlp,instance)

print("Shapley Values:")
for element, value in shapley_result.items():
    print(f"Element {element}: {value}")
```

The order of the variable importance received from the results does not differ to the ones estimated by breakdown values calculation, but we need to have in mind that the example is really simple, and the scale and interpretation of both

Shapley Values:
Element X1: -0.09424638777516699
Element X2: -1.7273327881393774
Element X3: 0.9863341422665193

Figure 4.14 The SHAP values.

Source: Own work, with Python/NumPy.

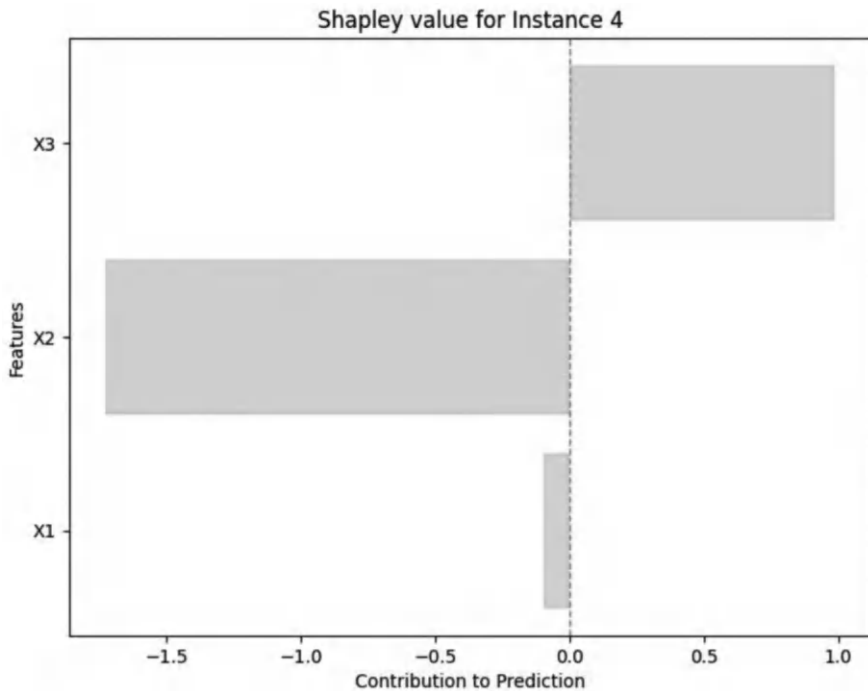


Figure 4.15 The SHAP plot.

Source: Own work, with Python/NumPy.

techniques differ. The SHAP values and very basic form of SHAP plot are presented in [Figures 4.14](#) and [4.15](#).

To find the SHAP values and create corresponding charts for selected houses (instances nr 3, 25 and 2005) from California Housing Dataset the following code may be used. Assuming that data `mlp_regressor` is already created (the simplest way of assuring this is just to run fragment from 4.1.1 before this script) the creation of Shapley Additive Plot is conducted with SHAP Python library:

```
def model_predict(X):
    return mlp_regressor.predict(X).flatten()
explainer = shap.KernelExplainer(model_predict, shap.sample(X,100),
    random_state=25032005)
```

```

shap_values = explainer(X.values)
for i in [3,25,2005]:
    print(f"***** shap values for instance {i}")
    print(shap_values[i,:])
    plt.figure(figsize=(18, 6))
    fig=shap.plots.bar( shap_values[i,:],show=False,show_data=True)
    plt.subplots_adjust(left=0.3)
    plt.savefig(f"housing_shap_mlp_{i}_results.jpg")
    plt.show()

```

The results of the analysis are presented in [Figures 4.16](#) and [4.17](#).

```

***** shap values for instance 3
.values =
array([ 1.16066683,  0.36697099, -0.20033458,  0.18743282, -0.20985735,
        0.19265371])

.base_values =
np.float64(1.963049304191007)

.data =
array([ 0.93296751,  1.85618152,  0.15696608, -0.04983292, -0.76602806,
        -0.0503293 ])
***** shap values for instance 25
.values =
array([-0.70631153,  0.08392822, -0.30809708,  0.12739541, -0.10838444,
        -0.01766587])

.base_values =
np.float64(1.963049304191007)

.data =
array([-0.77212852,  0.98214266, -0.37718623, -0.13306982, -0.97884386,
        -0.03916787])
***** shap values for instance 2005
.values =
array([-1.03604848e+00,  6.31579828e-02,  6.18024166e-04,  1.41000636e-01,
        4.25680713e-02, -1.72919420e-02])

.base_values =
np.float64(1.963049304191007)

.data =
array([-1.61685983, -0.2891866, -0.72089042,  0.11409163, -0.07283135,
        -0.02111298])

```

Figure 4.16 The SHAP values.

Source: Own work, with Python/NumPy/SHAP.

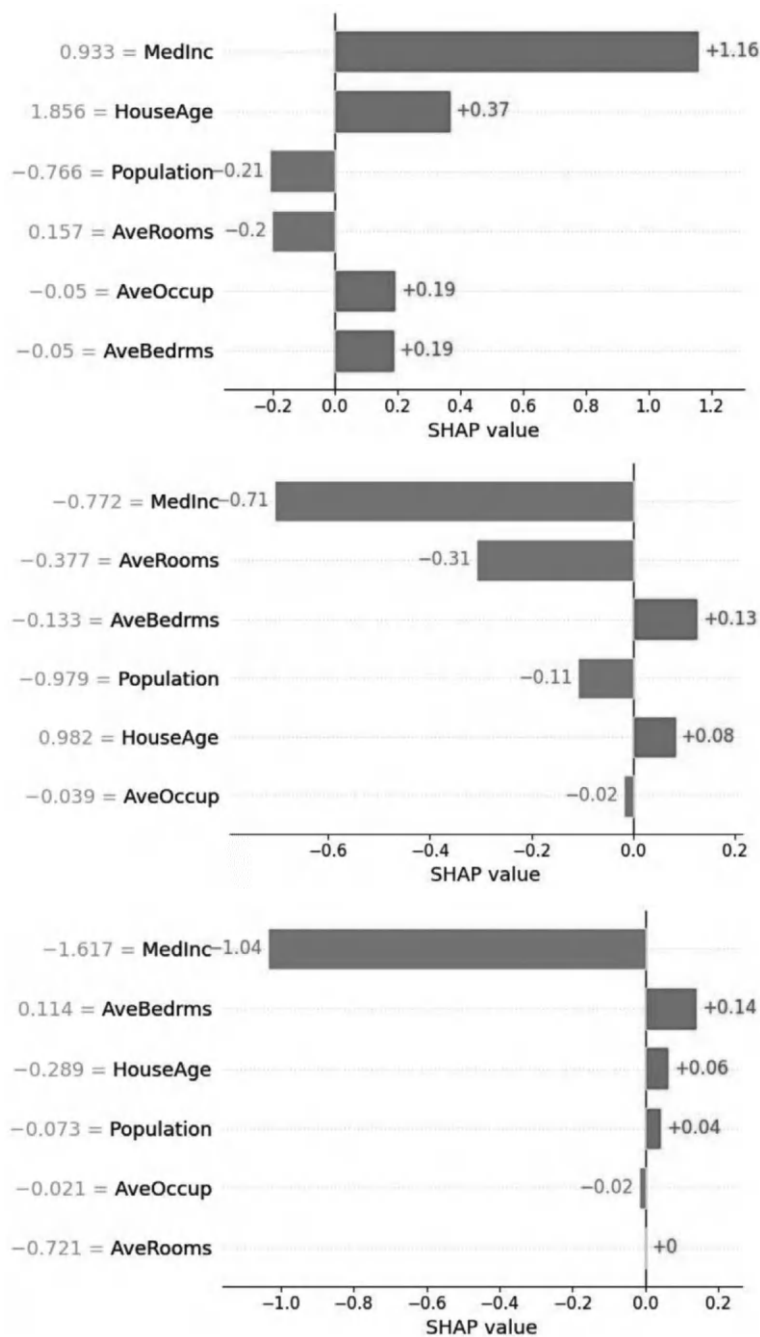


Figure 4.17 The SHAP plots.

Source: Own work, with Python/NumPy/SHAP.

For instance 3, the SHAP values indicate that higher median income (+1.16) and house age (+0.37) are the primary contributors to the predicted value above the base value of 1.96. In contrast, the average number of rooms (-0.20) and total population (-0.21) slightly reduce the prediction. The final prediction reflects a balance of these factors.

For instance 25, a lower median income (-0.71) is the most significant factor reducing the prediction below the base value, with additional negative contributions from the average number of rooms (-0.31). Positive contributions come from house age (+0.08) and average number of bedrooms (+0.13), partially offsetting the decrease.

For instance 2005, a much lower median income (-1.04) drives the prediction significantly below the base value. The house age (+0.06) and average number of bedrooms (+0.14) provide small positive contributions, but they are outweighed by the dominant negative influence of income.

4.1.3 PDP

The term *ceteris paribus* is well known in economics and means a situation in which we study the influence of only one variable on a complex economic phenomenon (e.g., demand vs price, supply vs price, income vs consumption, interest rates vs investment, exchange rates vs exports, taxes vs work effort minimum wage vs employment) (see e.g., [Whitaker, 2008](#)).

This technique can be used in EMA for a similar purpose. *Ceteris paribus* graphs show how the model prediction for a selected object (instance) changes when we change the values of only one variable. In practice, the value of a variable in the object under study is replaced sequentially by successive elements of an ordered set of all values of the variable in the entire data set.

The implementation of *ceteris paribus* analysis is quite straightforward in any programming language including Python. The artificial dataset from [Chapter 3](#) may be processed with the code similar to this one:

```
instance_nr=4
instance = X.iloc[instance_nr-1]
features = X.columns
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for i, feature in enumerate(features):
    feature_range = np.sort(data[feature].unique())
    ceteris_paribus_data = instance.copy()

    predictions = []
    for value in feature_range:
        ceteris_paribus_data[feature] = value
        ceteris_paribus_df = pd.DataFrame([ceteris_paribus_
data], columns=X.columns)
        predictions.append(mlp.predict(ceteris_paribus_df)[0])
```

```

    axes[i].plot(feature_range, predictions,
label=f'{feature} ceteris paribus')
    axes[i].scatter(instance[feature], mlp.predict(pd.
DataFrame([instance], columns=X.columns))[0], color='red',
label='Instance value')
    axes[i].set_xlabel(feature)
    axes[i].set_ylabel('Predicted y')
    axes[i].set_title(f'Ceteris Paribus: {feature}')
    axes[i].legend()

plt.tight_layout()
plt.savefig("ceteris_paribus_plot.jpg")

```

The *ceteris paribus* plot for object 4 is presented on Figure 4.18. Even not knowing the actual patterns behind the model, looking at *ceteris paribus* plots, we may assume that the second and third explanatory variables are linearly related to the explained variable.

Partial dependency plots aggregate *ceteris paribus* plots, most often calculating for each variable the average of all values used to create *ceteris paribus* plots for subsequent objects. The partial dependency plot is presented on Figure 4.19. The most important difference between these techniques is that the second one belongs to a local family to explain the behavior of prediction of a particular object, while the first one is global.

We need to slightly modify the *ceteris paribus* routine to find values required to plot partial dependency plots.

```

features = X.columns
fig, axes = plt.subplots(1, 3, figsize=(18, 5))

for i, feature in enumerate(features):
    feature_range = np.sort(data[feature].unique())
    partial_dependency = []

    for value in feature_range:
        modified_data = X.copy()
        modified_data[feature] = value

```

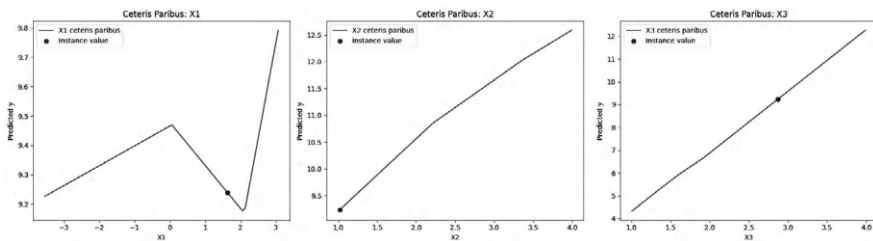


Figure 4.18 *Ceteris paribus* plots.

Source: Own work, with Python/NumPy.

```

predictions = mlp.predict(modified_data)
partial_dependency.append(np.mean(predictions))

axes[i].plot(feature_range, partial_dependency,
label=f'{feature} Partial Dependency')
axes[i].set_xlabel(feature)
axes[i].set_ylabel('Average Predicted y')
axes[i].set_title(f'Partial Dependency: {feature}')
axes[i].legend()

plt.tight_layout()
plt.savefig("partial_dependency_plot.jpg")

```

Before switching to a real dataset example, we need to clarify that, for real market analyses, it is difficult to accept all the values from the set describing houses with such ease because the transition of a house containing one bedroom to the same house containing three bedrooms is not usually possible due to technical/construction reasons. However, it is justified to use this technique for diagnostic purposes of the model itself.

Assuming that `mlp_regressor` is already fitted on California Housing Dataset with latitude and longitude omitted (as in previous example), the creation of PDP plots requires the procedure similar to the one listed below.

```

# the charts are grouped by 3 variables
# to increase readability
for i in range(2):
    PartialDependenceDisplay.from_estimator(
        estimator=mlp_regressor,
        # Drawing all individual ceteris-paribus plots would
make the chart
        # unclear, so only 200 randomly selected lines are
shown.
        X=X.values[np.random.permutation(range(X.shape[0]))
:200],),
        features=list(range(X.shape[1]))[3*i:3*(i+1)],

```

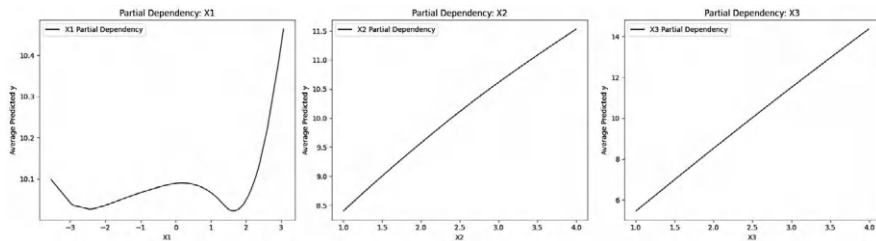


Figure 4.19 Partial dependency plots for artificial dataset.

Source: Own work, with Python/NumPy.


```

        feature_names=california_data.feature_names,
        kind="both",
        random_state=25032005
    )
    plt.savefig(f"housing_pdp_mlp_results{i}.jpg")
    plt.show()

```

The partial dependency plot for variables from [Table 4.3](#) (with geo coordinates omitted) is presented in the [Figure 4.20](#).

Median income (*MedInc*) is the most influential variable, showing a steep and consistent positive trend across instances. Features like average number of rooms (*AveRooms*) and average number of bedrooms (*AveBedrms*) exhibit some impact, but with notable heterogeneity across samples. Population (*Population*), house age (*HouseAge*), and average occupancy (*AveOccup*) appear to have little or no direct effect on predicted house prices for most instances.

The careful reader has probably noticed that, for the VIP values calculation, average number of rooms (*AveRooms*) and average number of bedrooms (*AveBedrms*) are more important than median income (*MedInc*), but in the case of other methods, median income (*MedInc*) has a much greater impact on the model than average number of rooms (*AveRooms*) and average number of bedrooms (*AveBedrms*). This requires a brief justification.

Median income (*MedInc*) is the most influential variable in the breakdown plot, SHAP values, and PDP because it consistently shows the largest direct impact on predictions across all methods, reflecting its strong and direct relationship with house prices. In contrast, the average number of rooms (*AveRooms*) and average number of bedrooms (*AveBedrms*) have smaller contributions in these methods, likely due to weaker or more indirect relationships with the target variable. However, in permutation variable importance, *AveRooms* and *AveBedrms* are more significant because this method measures the overall disruption in model performance when a feature is permuted, capturing potential interactions or dependencies with other variables. This difference highlights that permutation importance accounts for both main effects and interactions, whereas SHAP, PDP, and breakdown plots focus more on direct, localized contributions.

4.1.4 LIME

What if the model is just too complex to explain or the calculations required for explanation are too time consuming? To answer this question we should have in mind that there are several multivariate statistical analysis methods with “natural” interpretation. Models like linear regression, generalized linear regression, LASSO, discriminant decision trees, and regressive decision trees have “built in” explanation and are relatively simple to build. [Ribeiro et al. \(2016\)](#) introduced the concept of the substitutive model or in other words the “mirror box” substituting original “black box” (usually more complex).

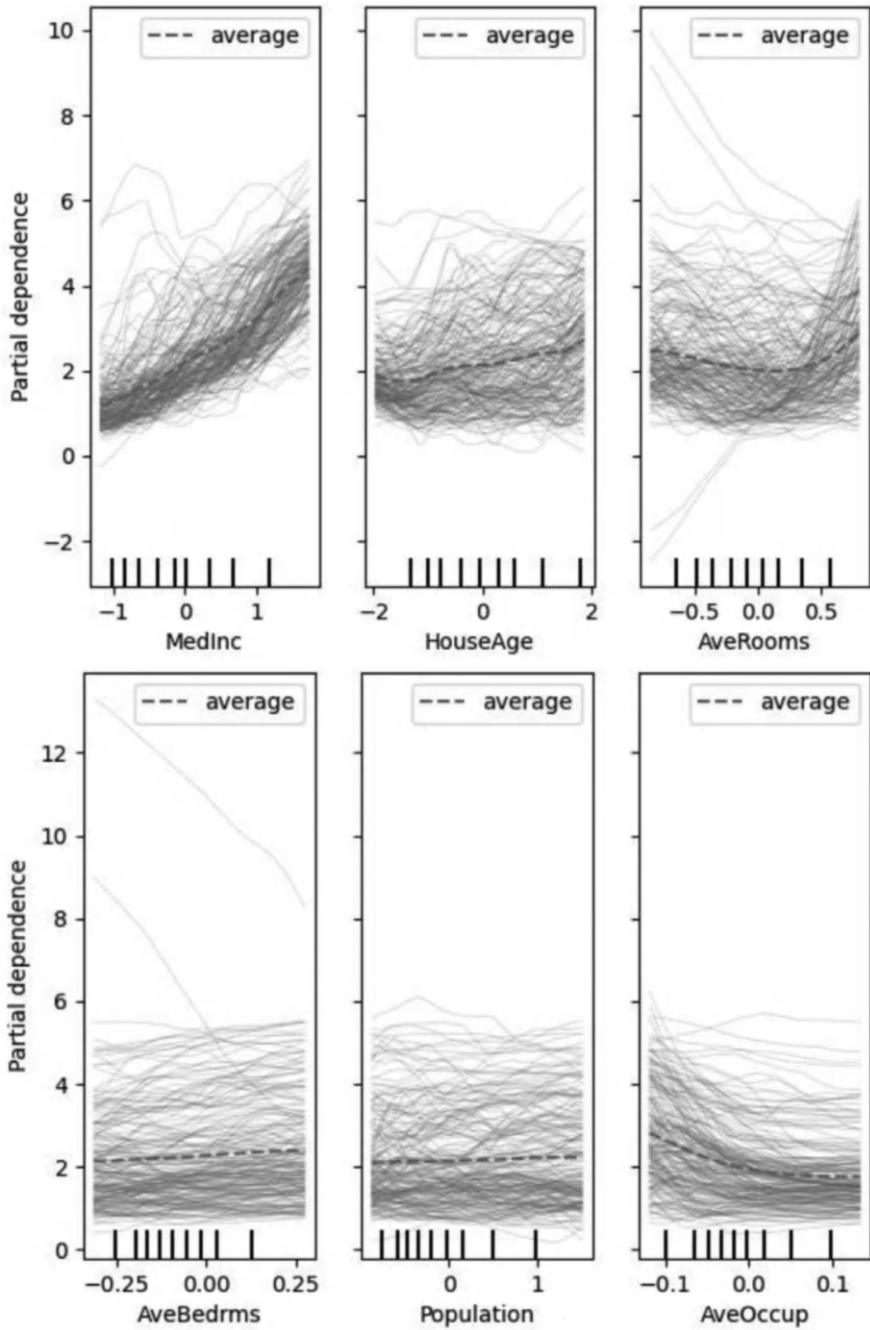


Figure 4.20 Partial dependency plot for California Housing Dataset.

Source: Own work, with Python/NumPy/SHAP

The key idea behind LIME is to generate local explanations by approximating the black-box model with an interpretable surrogate model (Glass model – see e.g., [Biecek & Burzykowski, 2021](#), Chapter 9) in the vicinity of the instance being explained. This enables users to understand why a model made a particular prediction for a specific instance.

For most economic problems there is no need to use LIME to explain the predictors and model behavior. (Why substitute model with regression if we could just use this regression model?) But it gained great popularity and a high amount of implementation especially in computer vision analysis.

LIME evaluates how model predictions change when slight variations of the input data are introduced. It creates a new dataset by perturbing the original instance and obtaining predictions from the black-box model. Using this perturbed dataset, LIME trains an interpretable model, assigning weights based on the proximity of each sample to the instance of interest. The interpretable model can be simple, like LASSO regression or a decision tree. While this model aims to approximate the black-box model's predictions locally, it does not necessarily provide an accurate global representation. This characteristic is referred to as “local fidelity”.

A locally interpretable surrogate model under interpretability constraints can be represented as (see [Ribeiro et al., 2016](#)):

$$\xi(x) = \underset{g \in G}{\operatorname{argmin}} L(f, g, \pi_x) + \Omega(g) \quad (4.4)$$

where:

- g is the explanation model (a linear regression model, lasso model) that minimizes the loss function L (e.g., mean squared error, cross-entropy);
- The loss function measures how well g approximates the predictions of the original model f (e.g., a Perceptron or Convolutional network model);
- The complexity term $\Omega(g)$ ensures that the explanation remains interpretable, typically by limiting the number of features;
- G represents the set of possible explanation models;
- π_x defines the neighborhood around the instance x for which the explanation is generated.

In practice, LIME optimizes the loss term while the user specifies the complexity constraints, such as the maximum number of features allowed in the linear model.

Steps for training local surrogate models

- 1 Select the instance of interest for which the explanation is generated.
- 2 Create perturbed variations of the dataset and obtain predictions from the black-box model.
- 3 Assign weights to the perturbed samples based on their similarity to the original instance.

- 4 Train an interpretable, weighted model using the generated dataset.
- 5 Interpret the local surrogate model to explain the prediction.

LIME is especially popular for images, with *superpixels* (larger groups of similar color pixels) playing a key role and the following steps are usually applied in the LIME analysis:

Segment the image into *superpixels*

- The image is divided into *superpixels*, which are groups of adjacent pixels with similar colors and textures.
- This segmentation helps analyze the impact of different regions on the image.

Create variations of the image

- *Superpixels* are randomly turned on or off in different variations.
- A *superpixel* can be “turned off” by replacing pixels with a user-defined color (e.g., gray).
- The user can adjust the probability of turning off a *superpixel* to control perturbations.

Feed perturbed images into the black-box model

- Each perturbed image is sent to the model to obtain predictions.
- This helps determine how the absence of certain *superpixels* affects the model’s classification.

Train a local interpretable model

- A simple, interpretable model (e.g., linear regression) is trained on the perturbed images.
- The model learns the relationship between *superpixel* presence and the classification outcome.

Weigh perturbations by proximity to the original image

- Perturbations that are more similar to the original image are given higher importance.
- A proximity measure ensures that explanations focus on the most relevant changes.

Generate an explanation

- The interpretable model highlights which *superpixels* had the greatest impact on the classification.
- Important *superpixels* are visualized as highlighted regions on the image.

Present the explanation

- The explanation is displayed as an image overlay, with *superpixels* contributing most to the prediction highlighted.
- This allows users to understand which parts of the image influenced the model's decision.

The use of the LIME method in the version for images is usually achieved with the prediction made in one of the predefined neural network architectures for image processing such as VGG16 or Resnet. “The pyramid of animals”, inspired by The Bremen Town Musicians (*Die Bremer Stadtmusikanten*) by the Grimm brothers, presented in [Figure 4.21](#) is the eminent work of Katarzyna Kozyra, famous polish artist, for which the particular parts will be explained with Resnet50 library included in PyTorch library with use of *LimeImageExplainer* from LIME package. The installation process for PyTorch is described in [Appendix 2](#).



Figure 4.21 “The pyramid of animals” (*Piramida Zwierząt*) by Katarzyna Kozyra.

Source: [Kozyra \(n.d.\)](#).

In the first step the image is stratified into *superpixels* with the code:

```
import torch
import torchvision.transforms as transforms
from torchvision import models
from PIL import Image
import numpy as np
import matplotlib.pyplot as plt
from lime import lime_image
from skimage.segmentation import mark_boundaries
import requests
from io import BytesIO
import os

subdirectory = "Images"
os.makedirs(subdirectory, exist_ok=True)

image_url = "https://upload.wikimedia.org/wikipedia/commons/
thumb/6/6f/Kozyra_K_piramida.jpg/800px-Kozyra_K_piramida.jpg"
headers = {
    'User-Agent': 'MyCustomUserAgent/1.0'
}
response = requests.get(image_url, headers=headers)
image = Image.open(BytesIO(response.content)).convert("RGB")
segmentation_fn = lime_image.SegmentationAlgorithm('slic',
n_segments=40, compactness=25, sigma=10)
segmentation = segmentation_fn(np.array(image))
# "Mark on image"
plt.figure(figsize=(8, 8))
plt.imshow(mark_boundaries(np.array(image), segmentation, co
lor=(0.5,0,0.5)))
plt.title("Superpixel Segmentation Borders")
plt.axis("off")
plt.savefig("Images/segments_piramid.png")
plt.show()
```

The superpixels may be observed in [Figure 4.22](#). Superpixels perturbations and interpretable simpler models training is sewn in LimeImageExplainer and the final explanation is achieved with code similar to the one below:

```
model = models.resnet50(weights=models.ResNet50_Weights.DEFAULT)
model.eval()
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229,
0.224, 0.225])
])
```



Figure 4.22 Superpixels of “The pyramid of animals” by Katarzyna Kozyra

Source: [Kozyra \(n.d.\)](#). PyTorch/LIME.

```
def batch_predict(images):
    images = torch.stack([transform(Image.fromarray(img)) for
img in images], dim=0)
    with torch.no_grad():
        outputs = model(images)
        return torch.nn.functional.softmax(outputs, dim=1).numpy()
explainer = lime_image.LimeImageExplainer()
explanation = explainer.explain_instance(
    np.array(image), batch_predict, top_labels=100,
    hide_color=0, num_samples=10000,
    segmentation_fn=segmentation_fn, random_seed=25032005
)
```

```
LABELS_URL = "https://raw.githubusercontent.com/pytorch/hub/
master/imagenet_classes.txt"
labels = np.array(requests.get(LABELS_URL).text.splitlines())
for top_label in explanation.top_labels:
```

```

print(labels[top_label])
if labels[top_label] in ["hen", "television", "Ibizan
hound", "maze", "horse cart"]:
    temp, mask = explanation.get_image_and_mask(top_label,
num_features=3, hide_rest=False)
    temp_b, mask_b = explanation.get_image_and_mask
(top_label, num_features=3, hide_rest=True)
    predicted_label = labels[top_label]
    print(predicted_label)
    # Display the explanation with label
    fig, axes = plt.subplots(1, 2, figsize=(12, 6))
    axes[0].imshow(mark_boundaries(temp_b, mask_b,color=(0.9,
0,0.9),mode='thick'))
    axes[0].set_title(f"LIME Explanation: {predicted_label}
boundaries")
    axes[0].axis("off")
    axes[1].imshow(mark_boundaries(temp, mask,color=(0.9,0,
0.9),mode='thick'))
    axes[1].set_title(f"LIME Explanation: {predicted_label}")
    axes[1].axis("off")
plt.savefig(f"Images/Katarzyna_Kozyra
pyramid_{predicted_label}.png")
plt.show()

```

The procedure returns an explanation for 50 predictions but in the line `if labels[top_label] in ["hen", "television", "Ibizan hound", "maze", "horse cart"]`: they are filtered only to the most intuitive ones. The explanations are presented in [Figures 4.23–4.27](#).

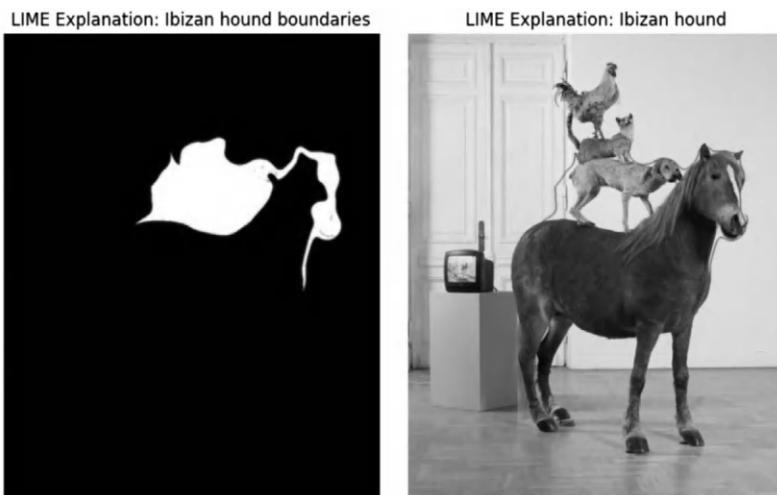


Figure 4.23 Explanation of hound pattern on “The pyramid of animals” by Katarzyna Kozyra.

Source: [Kozyra \(n.d.\)](#). PyTorch/LIME.

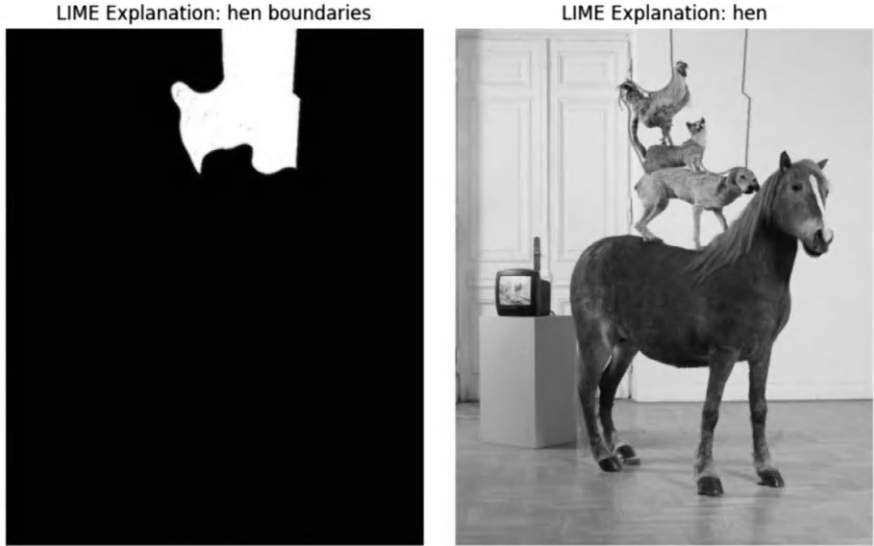


Figure 4.24 Explanation of hen pattern on “The pyramid of animals” by Katarzyna Kozyra.
Source: [Kozyra \(n.d.\)](#). PyTorch/LIME.

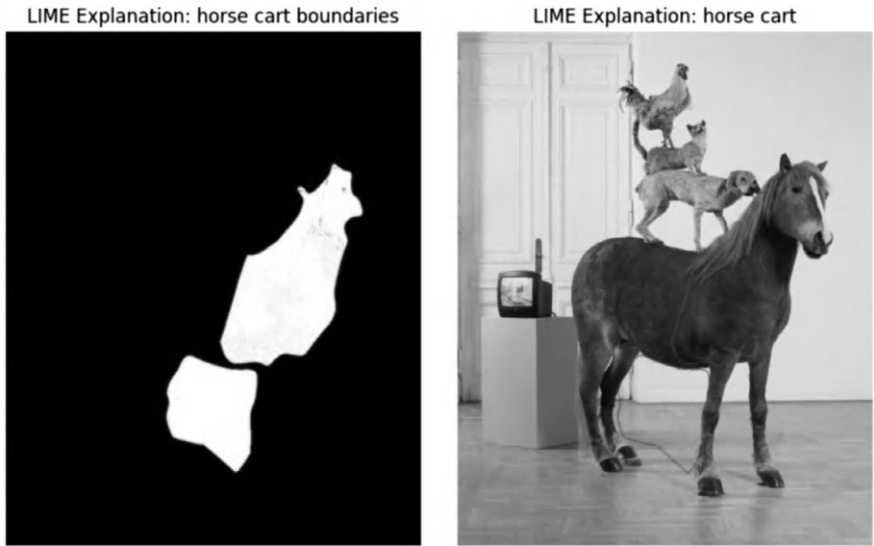


Figure 4.25 Explanation of horse cart pattern on “The pyramid of animals” by Katarzyna Kozyra.
Source: [Kozyra \(n.d.\)](#). PyTorch/LIME.

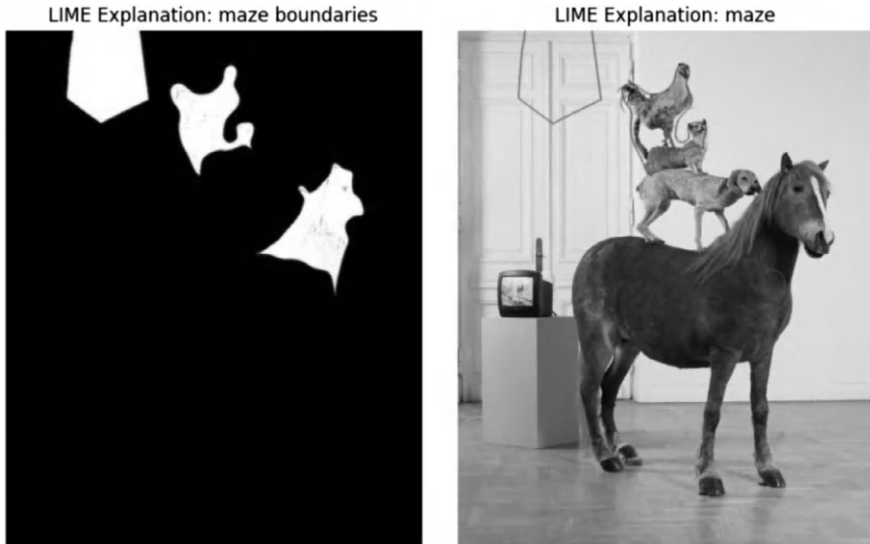


Figure 4.26 Explanation of maze pattern on “The pyramid of animals” by Katarzyna Kozyra.

Source: [Kozyra \(n.d.\)](#). PyTorch/LIME.

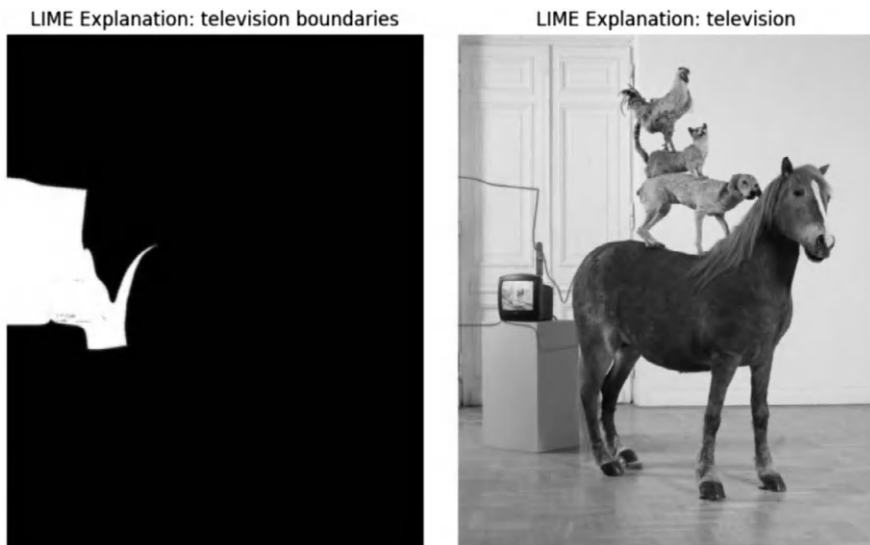


Figure 4.27 Explanation of television pattern on “The pyramid of animals” by Katarzyna Kozyra.

Source: [Kozyra \(n.d.\)](#). PyTorch/LIME.



Figure 4.28 Jaffa Gate in Jerusalem.

Source: Wikimedia.

LIME works on everyday life photographs as well as on works of art. In the second example, the photograph of Jaffa Gate in Jerusalem is explored (see [Figure 4.28](#)).

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
import requests
from io import BytesIO
import json
import urllib.request
from lime import lime_image
from skimage.segmentation import mark_boundaries
from tensorflow.keras.applications.vgg16 import VGG16,
decode_predictions, preprocess_input
import cv2
import os

subdirectory = "Images"
os.makedirs(subdirectory, exist_ok=True)

model = VGG16(weights='imagenet')
```

```

url="https://upload.wikimedia.org/wikipedia/commons/
thumb/3/36/Jaffa_Gate_Jerusalem_02.JPG/1280px-Jaffa_Gate_
Jerusalem_02.JPG"
headers = {
    'User-Agent': 'MyCustomUserAgent/1.0'
}
response = requests.get(url,headers=headers)
img = Image.open(BytesIO(response.content)).convert('RGB')
img_np = np.array(img)
original_size = img.size
# -----
# 3. Define a prediction function for LIME
# This function accepts a list of images (as numpy arrays),
# resizes each to 224x224, preprocesses them, and returns
model predictions.
# -----
def predict_fn(images):
    imgs = []
    for image in images:
        # Resize image to 224x224 as expected by VGG16
        image = cv2.resize(image, (224, 224))
        imgs.append(image)
    imgs = np.array(imgs)
    imgs = preprocess_input(imgs.astype(np.float32))
    preds = model.predict(imgs)
    return preds

explainer = lime_image.LimeImageExplainer()

segmentation_fn = lime_image.SegmentationAlgorithm('slc',
n_segments=40,compactness=30)
segmentation = segmentation_fn(np.array(img))

print("Mark on image")
plt.figure(figsize=(8, 8))
plt.imshow(mark_boundaries(np.array(img), segmentation,color=(0.5,0,0.5)))
plt.title("Superpixel Segmentation Borders")
plt.axis("off")
plt.savefig("Jeruzalem_explanation_superpixels.png")

explanation = explainer.explain_instance(
    img_np,
    predict_fn,
    top_labels=10,
    hide_color=0,
    num_samples=10000,
    segmentation_fn = segmentation_fn,
    random_seed=25032005
)

```

```

class_index_url = 'https://s3.amazonaws.com/deep-learning-
models/image-models/imagenet_class_index.json'
with urllib.request.urlopen(class_index_url) as url_response:
    class_idx = json.load(url_response)

for found_idx in explanation.local_exp.keys():
    pattern=class_idx[str(found_idx)][1]
    print("Explanation available for :", pattern)
    intercept = explanation.intercept[found_idx]
    superpixel_weights = explanation.local_exp[found_idx]
    contribution=sum(sorted([v[1] for v in superpixel_
weights],reverse=True)[:5])
    print(f"Base probabability (Intercept) for class
{pattern}: {intercept:.4f}")
    print(f"Top 5 superpixel contributions for class
{pattern}: {contribution:.4f}")
    temp, mask = explanation.get_image_and_mask(
        found_idx,
        positive_only=True,
        num_features=5,
        hide_rest=False
    )
    temp_resized = Image.fromarray(temp).resize(original_
size, Image.BILINEAR)
    img_boundary = mark_boundaries(np.array(temp_resized) /
255.0, mask,color=(1,0,1),mode="thick")
    plt.figure()
    plt.imshow(img_boundary)
    plt.title(f"LIME Explanation for class containing '
{pattern}' \n Intercept: {intercept:.4f}\n Superpixels
contribution: {contribution:.4f}", fontsize=9)
    plt.axis('off')
    plt.savefig(f"Jeruzalem_explanation_{pattern}.png")
    plt.show()

```

Figure 4.29 presents the *superpixels* for the analyzed image, Figure 4.30 the output of LIME procedure and Figures 4.31–4.33 explanations to predictions for classes (patterns) “castle” “triumphal arch”, and “prison (leaving aside the fact that some of these predictions are debatable in the context of this particular place).

LIME is most commonly used for image analysis with convolutional networks, but we should have in mind that it is a model-agnostic method and may be applied to other kinds of deep learning models as well. However, there are methods designed exclusively for explaining deep learning image analysis models.

Learned feature visualization involves directly visualizing the convolutional filters (or feature maps) learned by a convolutional neural network (CNN). By examining these filters, researchers gain insights into the low- to high-level patterns the network extracts from input data (Zeiler & Fergus, 2014).



Figure 4.29 The *superpixels* for Jaffa Gate in Jerusalem photograph.
Source: Wikimedia.

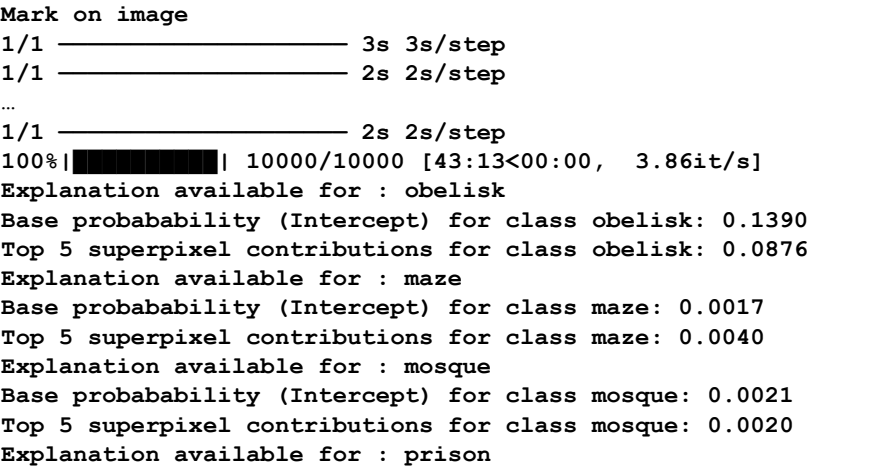


Figure 4.30 The base predictions and the LIME *superpixels* contribution in explanation of Jerusalem Jaffa Gate photograph. (Continued)
Source: Own calculations with TensorFlow/LIME.

```

Base probabability (Intercept) for class prison: 0.0000
Top 5 superpixel contributions for class prison: 0.0003
Explanation available for : cliff
Base probabability (Intercept) for class cliff: 0.0094
Top 5 superpixel contributions for class cliff: 0.0056
Explanation available for : submarine
Base probabability (Intercept) for class submarine: 0.0002
Top 5 superpixel contributions for class submarine: 0.0011
Explanation available for : triumphal_arch
Base probabability (Intercept) for class triumphal_arch: 0.0050
Top 5 superpixel contributions for class triumphal_arch: 0.0045
Explanation available for : monastery
Base probabability (Intercept) for class monastery: 0.0230
Top 5 superpixel contributions for class monastery: 0.0170
Explanation available for : palace
Base probabability (Intercept) for class palace: 0.0025
Top 5 superpixel contributions for class palace: 0.0092
Explanation available for : castle
Base probabability (Intercept) for class castle: 0.4124
Top 5 superpixel contributions for class castle: 0.5903

```

Figure 4.30 (Continued)

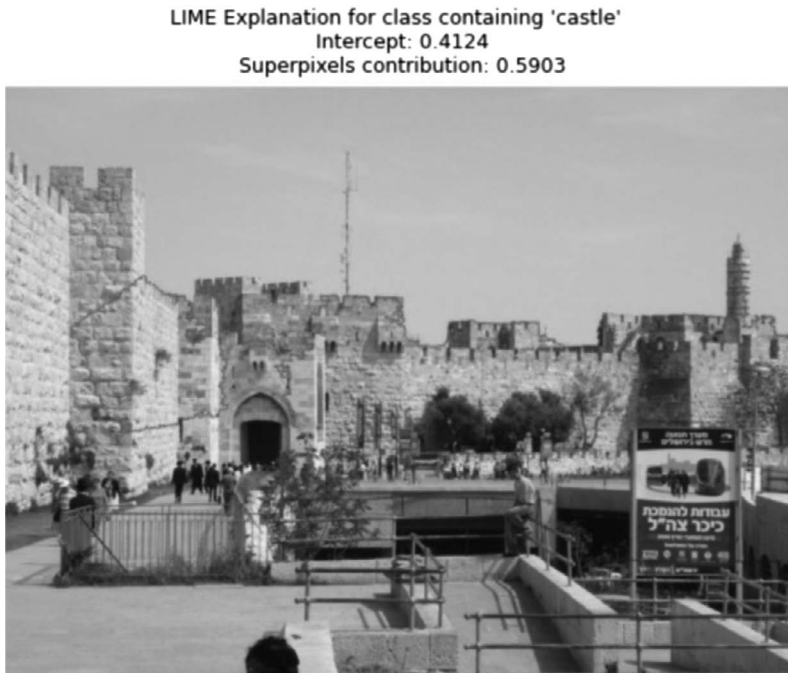


Figure 4.31 Explanation of “castle” pattern on Jerusalem Jaffa Gate photograph.

Source: Wikimedia. TensorFlow/LIME.

LIME Explanation for class containing 'monastery'
Intercept: 0.0230
Superpixels contribution: 0.0170



Figure 4.32 Explanation of “monastery” pattern on Jerusalem Jaffa Gate photograph.

Source: Wikimedia. TensorFlow/LIME.

LIME Explanation for class containing 'prison'
Intercept: 0.0000
Superpixels contribution: 0.0003



Figure 4.33 Explanation of “prison” pattern on Jerusalem Jaffa Gate photograph.

Source: Wikimedia. Python/TensorFlow/LIME.

Feature visualizations and network dissection offer valuable insights into the internal workings of neural networks, particularly in image recognition. These techniques reveal that early layers learn basic features such as edges and textures, while higher layers progressively detect more abstract parts and objects. Network dissection further enhances interpretability by automatically linking network units to human-understandable concepts and measuring their relevance. Additionally, combining feature visualization with attribution methods enables both the explanation of individual classifications and the local visualization of learned features, making it easier to communicate how neural networks operate – even in non-technical terms.

Despite these benefits, there are notable drawbacks. Many feature visualizations produce abstract images that remain ambiguous, sometimes suggesting vague concepts like “maybe there has to be yellow” without clear interpretation. Furthermore, the sheer number of channels and their complex interactions can create an illusion of interpretability, as not all units correspond neatly to human concepts.

Saliency maps are gradient-based methods that identify which parts of an input image most strongly affect the network’s prediction. They compute the derivative of the output with respect to the input pixels, highlighting important regions (Simonyan et al., 2013).

Saliency maps and other pixel attribution methods (DeconvNet – Zeiler & Fergus, 2014; Gradient-weighted Class Activation Map, Guided Grad-CAM, SmoothGrad – Smilkov et al., 2017) offer an intuitive way to understand neural network predictions by highlighting important regions in an image.

Class activation mapping (CAM) and Grad-CAM localize class-specific regions in an image. CAM requires architectural modifications (global average pooling). At the same time, Grad-CAM extends the idea to any CNN by using gradients flowing into the final convolutional layer to produce a coarse localization map (Selvaraju et al., 2017; Zhou et al., 2016).

Concept detection with techniques like testing with concept activation vectors (TCAV) measures the influence of high-level, human-understandable concepts (such as “striped” or “dotted”) on the predictions of CNNs. This approach provides insight into whether the model relies on meaningful, interpretable features (Kim et al., 2018).

Activation maximization and **feature inversion** synthesize an input image that maximizes the response of a particular neuron or feature map, revealing what that unit has learned to detect. Feature inversion attempts to reconstruct the input from a layer’s activations to understand the information retained by the network (Mahendran & Vedaldi, 2015).

4.2 Time limitations of explanatory model analysis for deep learning models

EMA methods are designed to provide insights into the decision-making processes of complex AI systems by generating human-readable explanations. Despite their

promise, several challenges persist that can undermine the reliability and clarity of these explanations. Key issues include ambiguity in defining the explanatory targets, neglecting model error rates, an overemphasis on the final product rather than the process, scalability challenges, a trade-off between accuracy and interpretability, and the absence of standardized evaluation metrics (see e.g., [Tsang & Benoit, 2023](#)).

One major challenge is ambiguity in explanatory targets. EMA methods often lack clarity regarding what specific aspect of the model they are intended to explain, which can lead to misinterpretations by users who might draw inaccurate conclusions about the model's inner workings. Similarly, many EMA techniques overlook error rates, failing to account for the model's performance variability; as a result, the generated explanations may not accurately reflect the reliability of the model's predictions, especially in high-stakes scenarios.

Another concern is the emphasis on product over process. Many methods focus on delivering a final explanation without critically assessing the processes that generated it, potentially leading to misleading insights that obscure the true dynamics of the model. Scalability issues further complicate matters, as methods that perform well on small datasets often struggle when applied to large-scale AI systems, thereby limiting their practical utility. Additionally, there is an inherent trade-off between accuracy and interpretability: efforts to simplify complex models for greater clarity may compromise predictive performance, which is particularly problematic in domains such as healthcare. Finally, the lack of consensus on evaluation metrics means that there is no standard approach for assessing the quality of the explanations provided by EMA methods, making their validation and comparison highly subjective.

A good example of trade-off between scalability and quality of explanation is the various estimators for SHAP values. We will observe this with commonly used in computer science $\mathcal{O}(\cdot)$ notion (see e.g., [Aho et al., 1974](#)).

When computing Shapley values to interpret model predictions, the computational complexity of various approaches can differ dramatically. The exact Shapley value, often referred to as “coalition SHAP”, is defined for each feature i as:

$$\phi_i = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(M - |S| - 1)!}{M!} [v(S \cup \{i\}) - v(S)], \quad (4.5)$$

where M is the total number of features, N is the set of all features, and $v(S)$ is the value (or performance) function computed on a subset S of features. This formula requires summing over all possible subsets of features not including i , which entails 2^{M-1} evaluations. Hence, the computational complexity of the exact coalition SHAP is $\mathcal{O}(2^M)$, making it intractable for models with a moderate to large number of features.

Kernel SHAP addresses this issue by approximating the Shapley values using a weighted linear regression formulation. In this method, each coalition S is assigned a kernel weight:

$$\pi(S) = \frac{M-1}{\binom{M}{|S|} |S| (M-|S|)}, \quad (4.6)$$

and a regression model is fitted on N sampled coalitions to estimate the Shapley values. The complexity of Kernel SHAP is driven by the number of samples N chosen for the approximation. Evaluating the model on each of the N coalitions gives a complexity of $\mathcal{O}(N)$, and the regression step typically adds a polynomial overhead (often $\mathcal{O}(M^2)$ per fit), though NN is selected to be significantly smaller than 2^M . Thus, Kernel SHAP provides a tractable compromise between accuracy and computational efficiency.

Sampling SHAP further simplifies the computational burden by employing Monte Carlo sampling. In this approach, a total of T coalitions are randomly sampled, and the approximate Shapley value for each feature is computed as:

$$\hat{\phi}_i = \frac{1}{T} \sum_{t=1}^T \Delta_i^{(t)}, \quad (4.7)$$

where $\Delta_i^{(t)}$ is the marginal contribution of feature i in the t -th sampled coalition. The computational complexity of Sampling SHAP is $\mathcal{O}(T)$, with T chosen by the user such that $T \ll 2^M$. This makes Sampling SHAP especially useful when an approximate solution is acceptable and when the number of features is large, ensuring that the method remains computationally efficient while still providing useful estimates of feature importance.

In summary, while the exact coalition SHAP method suffers from exponential complexity $\mathcal{O}(2^M)$, both Kernel SHAP and Sampling SHAP offer practical approximations. Kernel SHAP relies on a weighted regression approach with complexity $\mathcal{O}(N)$ (plus a polynomial overhead in M), and Sampling SHAP achieves efficiency with a complexity of $\mathcal{O}(T)$.

Concluding this short section, until Quantum computers reign for deep learning, sampling on big datasets is the most effective approach like in the example above.

4.3 Research application example: The importance of inflation factors based on real data from DBnomics repository

At the end of this chapter, we will use explanatory methods on real, meaningful economics data in an attempt to solve a real problem. In this chapter we will try

to bear with one of the most common question in economics: what factors most influence inflation, without theoretical reasoning (which may be highly valuable, but may be found in many valuable sources) but with analysis on real, current, and comprehensive dataset from probably most extensive economics cross-sectional and panel data sources integrator: the DBnomics² platform.

4.3.1 Overview of the DBnomics repository

DBnomics is a comprehensive open-data platform that aggregates and disseminates macroeconomic and financial time-series data from a wide variety of reputable sources. Designed to simplify access to global statistics, DBnomics provides an integrated repository where users can search, retrieve, and analyze economic indicators spanning multiple domains and geographies. Among its most notable characteristics are its open-access policy, user-friendly web interface, and a robust API that supports programmatic access. This repository is characterized by its extensive metadata, standardized data formats, and frequent updates, which together ensure that researchers, policymakers, and data enthusiasts have reliable and up-to-date information at their fingertips.

The history of DBnomics is rooted in the growing need for a centralized platform that could overcome the fragmentation inherent in economic data sources. Over the past decade, the increasing demand for transparent and reproducible economic research spurred on the development of tools that could seamlessly integrate data from diverse providers. DBnomics emerged as a response to this need, evolving from a simple data aggregator into a sophisticated repository that not only consolidates information from international organizations, but also offers advanced functionalities for data exploration and analysis. Since its inception, DBnomics has expanded both its scope and its technical capabilities, driven by collaborations with academic institutions, international organizations, and the open-source community.

A key strength of DBnomics is the breadth of its data providers. The repository hosts data from major international organizations and national statistical agencies, including but not limited to Eurostat, the Organisation for Economic Co-operation and Development (OECD), the World Bank, the International Monetary Fund (IMF), and the United Nations (UN). Additionally, DBnomics integrates datasets from regional sources such as the Federal Reserve Economic Data (FRED) and several national statistical offices. This extensive network of data providers ensures that users can access a diverse range of economic indicators, from gross domestic product (GDP) and inflation rates to labor market statistics and trade balances, all within a single, harmonized framework.

To facilitate access to this wealth of data, DBnomics offers libraries for both R and Python, two of the most popular programming languages in data science. The DBnomics R package, available on CRAN and GitHub, provides functions that allow users to query the DBnomics API, download datasets, and convert them into R data frames for immediate analysis. Similarly, the DBnomics Python package, available via PyPI, offers analogous functionality for Python users by enabling seamless integration of DBnomics data into pandas DataFrames, making it easier

to perform data wrangling, statistical analysis, and visualization. These libraries abstract away the complexities of interacting directly with the API, thereby reducing the technical barriers for researchers and practitioners who wish to incorporate high-quality economic data into their analyses.

Moreover, the DBnomics libraries support reproducible research by allowing analysts to script and automate data retrieval processes. This ensures that economic analyses can be regularly updated with the latest available data, promoting transparency and consistency in research workflows. The tools also offer powerful search and filtering capabilities, enabling users to pinpoint specific datasets or economic indicators relevant to their projects quickly. By providing both R and Python interfaces, DBnomics caters to a broad audience, from academic researchers and data journalists to policymakers and private sector analysts, all of whom can benefit from streamlined access to global economic data.

In the research, a feedforward neural network with two hidden layers will be trained for model with *Consumer Price Index* as dependent variable and 15 explanatory variables: *Balance of Payments Supplementary Items; International Liquidity: Total Reserves excluding Gold, Foreign Exchange; Balance of Payments Capital Account, Total, Debit; Financial account: Balance of Payments, Net lending; Labor Markets, Unemployment Rate; Total International Reserves; Arable land; as a percentage of the total land; Current account balance; Communications, computer, etc., expressed as a percentage of service exports; General government final consumption expenditure; Imports of goods and services, expressed as a percentage of GDP; Regulatory Quality; Control of Corruption; Political Stability and Absence of Violence/Terrorism; and Government Effectiveness*. Please note that *LP_PE_NUM* population is used for variable transformation purposes. Imports of goods and services are expressed as a percentage of GDP. The variables used in research study are presented in [Table 4.5](#).

Table 4.5 Consumer Price Index (CPI) factors research variables description

<i>Variable code</i>	<i>Description</i>
DEPENDENT (EXPLAINED) VARIABLE	
PCPI_PC_CP_A_PT	Prices, Consumer Price Index, All items, Percentage change, Corresponding period previous year, expressed in percent.
EXPLANATORY (EXOGENOUS) VARIABLES	
BCAXF_BP6_USD	Balance of Payments, Supplementary Items, Current Account, Net (excluding exceptional financing), expressed in US dollars.
RAXGFX_USD	International Liquidity, Total Reserves excluding Gold, Foreign Exchange, expressed in US dollars.
BFDA_BP6_USD	Balance of Payments, Financial account: Net lending (+) / net borrowing (-) (balance from financial account), Direct investment, Net acquisition of financial assets, expressed in US dollars.
BK_DB_BP6_USD	Balance of Payments, Capital Account, Total, Debit, expressed in US dollars.

(Continued)

Table 4.5 (Continued)

Variable code	Description
LP_PE_NUM	Population, expressed as the number of persons.
LUR_PT	Labor Markets, Unemployment Rate, expressed in percent.
RAFA_G_USD	Total International Reserves, expressed in US dollars.
AG.LND.ARBL.ZS	Arable land, as a percentage of the total land area.
BN.CAB.XOKA.GD.ZS	Current account balance, expressed as a percentage of GDP.
BX.GSR.CMCP.ZS	Communications, computer, etc., expressed as a percentage of service exports (Balance of Payments).
NE.CON.GOV.T.KD.ZG	General government final consumption expenditure, expressed as annual percentage growth.
NE.IMP.GNFS.ZS	Imports of goods and services, expressed as a percentage of GDP.
RQ.PER.RNK	Regulatory Quality (Percentile Rank).
CC.PER.RNK,	Control of Corruption (Percentile Rank).
PV.PER.RNK	Political Stability and Absence of Violence/Terrorism (Percentile Rank).
GE.PER.RNK	Government Effectiveness (Percentile Rank).

Source: Own research with DBnomics package and repository documentation.

The following code will be used to acquire the data and prepare the explanations needed.

```
import random
import dalex as dx
from dbnomics import fetch_series
import pandas as pd
from scikeras.wrappers import KerasRegressor
from sklearn.preprocessing import StandardScaler
import lime.lime_tabular
from sklearn.inspection import permutation_importance
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Input
import matplotlib.pyplot as plt
from sklearn.inspection import PartialDependenceDisplay
from sklearn.utils import validation

imf_codes=['AE', 'AF', 'AG', 'AI', 'AL', 'AM', 'AO', 'AR',
'AT', 'AU', 'AW', 'AZ', 'BA', 'BB', 'BD', 'BE', 'BF', 'BG',
'BH', 'BI', 'BJ', 'BN', 'BO', 'BR', 'BS', 'BT', 'BW', 'BY',
'BZ', 'CA', 'CD', 'CF', 'CG', 'CH', 'CI', 'CL', 'CM', 'CN',
'CO', 'CR', 'CV', 'CW', 'CY', 'CZ', 'DE', 'DJ', 'DK', 'DM',
'DO', 'DZ', 'EC', 'EE', 'EG', 'ES', 'ET', 'FI', 'FJ', 'FM',
'FR', 'GA', 'GB', 'GD', 'GE', 'GH', 'GL', 'GM', 'GN', 'GP',
'GQ', 'GR', 'GT', 'GW', 'GY', 'HK', 'HN', 'HR', 'HT', 'HU',
'ID', 'IE', 'IL', 'IN', 'IQ', 'IR', 'IS', 'IT', 'JM', 'JO',
```

```
'JP', 'KE', 'KG', 'KH', 'KI', 'KM', 'KN', 'KR', 'KW', 'KY',
'KZ', 'LA', 'LB', 'LC', 'LK', 'LR', 'LS', 'LT', 'LU', 'LV',
'LY', 'MA', 'MD', 'ME', 'MG', 'MK', 'ML', 'MM', 'MN', 'MO',
'MQ', 'MR', 'MS', 'MT', 'MU', 'MV', 'MW', 'MX', 'MY', 'MZ',
'NA', 'NC', 'NE', 'NG', 'NI', 'NL', 'NO', 'NP', 'NR', 'NZ',
'OM', 'PA', 'PE', 'PG', 'PH', 'PK', 'PL', 'PS', 'PT', 'PW',
'PY', 'QA', 'RO', 'RS', 'RU', 'RW', 'SA', 'SB', 'SC', 'SD',
'SE', 'SG', 'SI', 'SK', 'SL', 'SM', 'SN', 'SO', 'SR', 'SS',
'ST', 'SV', 'SX', 'SY', 'SZ', 'TD', 'TG', 'TH', 'TJ', 'TL',
'TN', 'TO', 'TR', 'TT', 'TV', 'TZ', 'UA', 'UG', 'US', 'UY',
'UZ', 'VC', 'VE', 'VG', 'VN', 'VU', 'WS', 'XK', 'YE', 'ZA',
'ZM', 'ZW']
```

```
wb_codes=['ABW', 'AFG', 'AGO', 'AIA', 'ALB', 'AND', 'ANT',
'ARE', 'ARG', 'ARM', 'ASM', 'ATG', 'AUS', 'AUT', 'AZE',
'BDI', 'BEL', 'BEN', 'BFA', 'BGD', 'BGR', 'BHR', 'BHS',
'BIH', 'BLR', 'BLZ', 'BMU', 'BOL', 'BRA', 'BRB', 'BRN',
'BTN', 'BWA', 'CAF', 'CAN', 'CHE', 'CHL', 'CHN', 'CIV',
'CMR', 'COD', 'COG', 'COK', 'COL', 'COM', 'CPV', 'CRI',
'CUB', 'CYM', 'CYP', 'CZE', 'DEU', 'DJI', 'DMA', 'DNK',
'DOM', 'DZA', 'ECU', 'EGY', 'ERI', 'ESP', 'EST', 'ETH',
'FIN', 'FJI', 'FRA', 'FSM', 'GAB', 'GBR', 'GEO', 'GHA',
'GIN', 'GMB', 'GNB', 'GNQ', 'GRC', 'GRD', 'GRL', 'GTM',
'GUF', 'GUM', 'GUY', 'HKG', 'HND', 'HRV', 'HTI', 'HUN',
'IDN', 'IND', 'IRL', 'IRN', 'IRQ', 'ISL', 'ISR', 'ITA',
'JAM', 'JEY', 'JOR', 'JPN', 'KAZ', 'KEN', 'KGZ', 'KHM',
'KIR', 'KNA', 'KOR', 'KWT', 'LAO', 'LBN', 'LBR', 'LBY',
'LCA', 'LIE', 'LKA', 'LSO', 'LTU', 'LUX', 'LVA', 'MAC',
'MAR', 'MCO', 'MDA', 'MDG', 'MDV', 'MEX', 'MHL', 'MKD',
'MLI', 'MLT', 'MMR', 'MNE', 'MNG', 'MOZ', 'MRT', 'MTQ',
'MUS', 'MWI', 'MYS', 'NAM', 'NER', 'NGA', 'NIC', 'NIU',
'NLD', 'NOR', 'NPL', 'NRU', 'NZL', 'OMN', 'PAK', 'PAN',
'PER', 'PHL', 'PLW', 'PNG', 'POL', 'PRI', 'PRK', 'PRT',
'PRY', 'PSE', 'QAT', 'REU', 'ROU', 'RUS', 'RWA', 'SAU',
'SDN', 'SEN', 'SGP', 'SLB', 'SLE', 'SLV', 'SMR', 'SOM',
'SRB', 'SSD', 'STP', 'SUR', 'SVK', 'SVN', 'SWE', 'SWZ',
'SYC', 'SYR', 'TCD', 'TGO', 'THA', 'TJK', 'TKM', 'TLS',
'TON', 'TTO', 'TUN', 'TUR', 'TUV', 'TWN', 'TZA', 'UGA',
'UKR', 'URY', 'USA', 'UZB', 'VCT', 'VEN', 'VIR', 'VNM',
'VUT', 'WSM', 'XKX', 'YEM', 'ZAF', 'ZMB', 'ZWE']
```

```
indicators_1=['PCPI_PC_CP_A_PT', 'LP_PE_NUM', 'BCAXF_BP6_
USD', 'RAXGFX_USD', 'BFDA_BP6_USD', 'BK_DB_BP6_USD', 'RAFA_G_
USD', 'LUR_PT']
```

```
ifs_data_1=fetch_series("IMF", "IFS", max_nb_series=3000,
dimensions={"REF_AREA": imf_codes ,
```

```
"FREQ": ["A"],
```

```
"INDICATOR": indicators_1})
```

```

cpi_analysis_df_1=ifs_data_1.pivot(index=["Reference
Area","period","original_period"],columns="INDICATOR",values=
"value")
cpi_analysis_df_1.reset_index(inplace=True)
print(cpi_analysis_df_1.isnull().sum())
cpi_analysis_df_1.dropna(axis=0,how='any',inplace=True)

for cc in indicators_1[2:-1]:
    cpi_analysis_df_1[cc.replace("USD","USD_pc")]= cpi_
analysis_df_1[cc]/ cpi_analysis_df_1['LP_PE_NUM']
indicators_1=[ind.replace("USD","USD_pc") for ind in
indicators_1]
wdi_data_1=pd.DataFrame()
indicators_2=['AG.LND.ARBL.ZS','BN.CAB.XOKA.GD.ZS','BX.GSR.
CMCP.ZS','NE.CON.GOVT.KD.ZG','NE.IMP.GNFS.ZS']
series=[f'WB/WDI/A-{ind}-{wb}' for wb in wb_codes for ind in
indicators_2]
batch_size=50
for i in range(0,len(series)//batch_size+1):
    print(i)
    x=fetch_series(series[i*batch_size:(i+1)*batch_size])
    wdi_data_1=pd.concat([wdi_data_1,x],axis=0)
wdi_data_1.columns.values[wdi_data_1.columns=='country
(label)']="Reference Area"
wdi_data_1.reset_index(inplace=True)

cpi_analysis_df_2=wdi_data_1.pivot(index=["Reference
Area","period","original_period"],columns="indicator",values=
"value")
cpi_analysis_df_2.reset_index(inplace=True)
cpi_analysis_df_2.dropna(axis=0,how='any',inplace=True)

wdi_data_2=pd.DataFrame()
indicators_3=['CC.PER.RNK','GE.PER.RNK','PV.PER.RNK','RQ.
PER.RNK']

series=[f'WB/WGI/A-{ind}-{wb}' for wb in wb_codes for ind in
indicators_3]

batch_size=50
for i in range(0,len(series)//batch_size+1):
    print(i)
    x=fetch_series(series[i*batch_size:(i+1)*batch_size])
    wdi_data_2=pd.concat([wdi_data_2,x],axis=0)
wdi_data_2.columns.values[wdi_data_2.columns=='country
(label)']="Reference Area"
wdi_data_2.reset_index(inplace=True)

```


204 *Deep Learning Models for Economic Research*

```
cpi_analysis_df_3=wdi_data_2.pivot(index=["Reference
Area","period","original_period"],columns="indicator",values="
value")
cpi_analysis_df_3.reset_index(inplace=True)
cpi_analysis_df_3.dropna(axis=0,how='any',inplace=True)
```

```
cpi_analysis_df_1.drop(columns=['period'],inplace=True)
cpi_analysis_df_2.drop(columns=['period'],inplace=True)
cpi_analysis_df_3.drop(columns=['period'],inplace=True)
```

```
cpi_analysis_full_df=pd.merge(cpi_analysis_df_1,cpi_
analysis_df_2,left_on=['Reference Area','original_
period'], right_on=['Reference Area','original_period'],
how='inner')
cpi_analysis_full_df=pd.merge(cpi_analysis_full_df,cpi_
analysis_df_3,left_on=['Reference Area','original_period'],
right_on=['Reference Area','original_period'], how='inner').
dropna()
cpi_analysis_full_df['ind']=cpi_analysis_full_
df["Reference Area"]+"_"+cpi_analysis_full_df["original_
period"]
cpi_analysis_full_df.set_index("ind",inplace=True)
X = cpi_analysis_full_df[indicators_1[2:]+indicators_2+indic
ators_3]
y = cpi_analysis_full_df[indicators_1[0]]
random.seed(25032005)
# X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=25032025)
X_train, X_test, y_train, y_test =X,X,y,y
```

```
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
def build_model():
    model = Sequential()
    model.add(Input(shape=(X_train.shape[1],)))
    model.add(Dense(64, activation='relu'))
    model.add(Dense(32, activation='relu'))
    model.add(Dense(1)) # Regression output
    model.compile(optimizer='adam', loss='mean_squared_error')
    return model
```

```
cpi_model = KerasRegressor(model=build_model,epochs=300,
batch_size=15, verbose=True )
cpi_model.fit(X_train_scaled, y_train)
```

```

plt.figure(figsize=(10, 6))
plt.plot(cpi_model.history_["loss"], label="Training Loss")
if "val_loss" in cpi_model.history_:
    plt.plot(cpi_model.history_["val_loss"], label=
        "Validation Loss")
plt.title("CPI Kera model Learning Curve")
plt.xlabel("Epoch")
plt.ylabel("Loss (Mean Squared Error)")
plt.legend(loc="upper right")
plt.grid(True)
plt.savefig("cpi_learning_curve.jpg")
plt.show()

perm_importance = permutation_importance(
    estimator=cpi_model,
    X=X.values,
    y=y,
    scoring="neg_mean_squared_error",
    n_repeats=10,
    random_state=25032005
)
print(perm_importance.importances_mean)
importance_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": perm_importance.importances_mean
}).sort_values(by="Importance", ascending=False)

importance_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": perm_importance.importances_mean
}).sort_values(by="Importance", ascending=False)

plt.figure(figsize=(10, 6))
plt.barh(importance_df["Feature"], importance_
df["Importance"], color="skyblue")
plt.xlabel("Permutation Importance (MSE)")
plt.title("Permutation Feature Importance")
plt.gca().invert_yaxis()
plt.savefig("cpi_vip_results.jpg")
plt.show()

explainer = lime.lime_tabular.LimeTabularExplainer(
    training_data=X_train_scaled,
    feature_names=X.columns,
    class_names=['inflation'],
    mode='regression'
)

# in fact we are doing cros-svalidation on whole dataset
with X_test=X_train,

```

206 *Deep Learning Models for Economic Research*

```
# thus we are certain that poland_2020 is an element of
X_test_scaled
# otherwise, we should check it before shap and lime
explanations
poland_2020_index =X.index.get_loc('Poland_2020')

prediction = cpi_model.predict(X_test_scaled)[poland_2020_index]

explanation = explainer.explain_instance(X_test_
scaled[poland_2020_index], cpi_model.predict,
num_features=15)

exp = explanation.as_list()
feature_names, weights = zip(*exp)
print(exp)

fig=plt.figure(figsize=(18, 6))
plt.barh(feature_names, weights, color=['blue',"red","green"])
plt.xlabel('CPI change weights')
plt.title('LIME Explanation for CPI change prediction based
for Poland 2020')
plt.gca().invert_yaxis()
plt.show()
fig.savefig("cpi_lime_poland_2020.jpeg")
plt.show()

dbnomic_fnn_exp = dx.Explainer(cpi_model, X, y,
                              label = "CPI SHAP Pipeline")

bd_poland_2020 = dbnomic_fnn_exp.predict_parts(X_test_
scaled[poland_2020_index], type ='shap')
pd.set_option('display.max_rows', 100)
print(bd_poland_2020.result.query('B==0'))
bd_poland_2020.plot()
plt.savefig("cpi_shap_poland_2020.jpeg")

for i in range(3):
    PartialDependenceDisplay.from_estimator(
        estimator=cpi_model,
        X=X.values[np.random.permutation(range(X_test_
scaled.shape[0]))[:100],],
        features=list(range(X.shape[1]))[5 * i:5 * (i + 1)],
        feature_names=X.columns,
        kind="both",
        random_state=25032005
    )
    plt.savefig(f"cpi_pdp_results_{i}.jpg")
    plt.show()
```

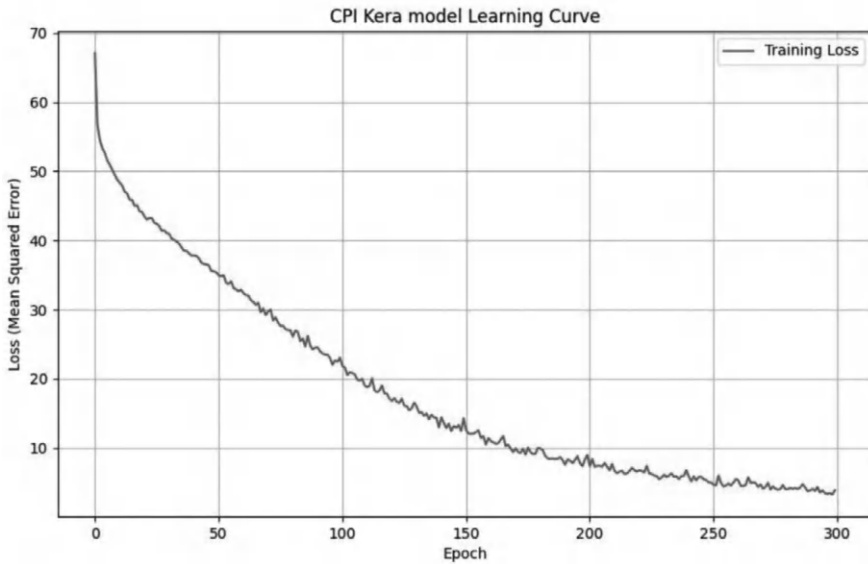


Figure 4.34 Learning curve plot for CPI explanation model.

Source: Own calculation, with use of Python/TensorFlow/scikit-learn.

Figures 4.34 and 4.35 present the learning curve and partial dependency plot for the model. From the shapes characteristics we may conclude that the model converged and is not degenerated in any way.

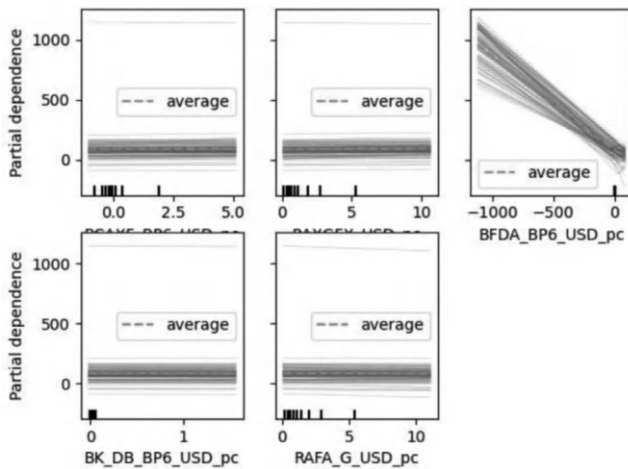


Figure 4.35 Partial dependency plots for CPI explanation model. (Continued)

Source: Own calculation, with use of Python/TensorFlow/scikit-learn.

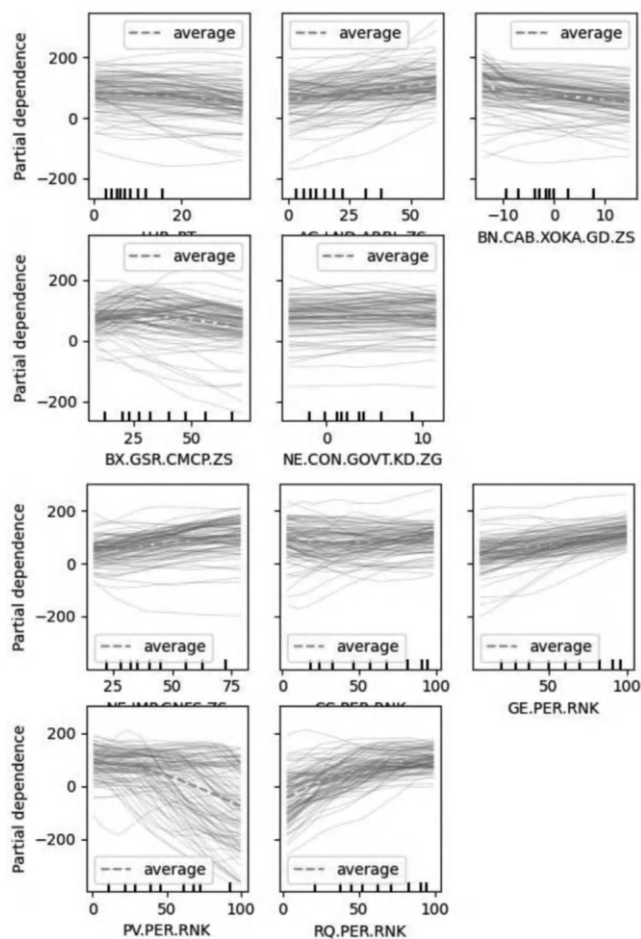


Figure 4.35 (Continued)

Figures 4.36 and 4.37 present the variable importance by permutations measures. The VIP analysis results are presented in Figures 4.38 and 4.39.

The VIP analysis for predicting changes in the *Consumer Price Index* (CPI) reveals varying degrees of contribution from several economic indicators. For instance, the *Balance of Payments for Supplementary Items in the Current Account, Net (excluding exceptional financing), expressed in US dollars*, shows a VIP value of approximately 114. This suggests that fluctuations in this component have a moderate positive influence on CPI variations. In contrast, *International Liquidity, defined as Total Reserves excluding Gold (Foreign Exchange) in US dollars*, has a VIP value of roughly 13, indicating a relatively lower impact on CPI dynamics. These results highlight that while both variables are important, their contributions to CPI are distinct in magnitude.

```
[ 113.99069081 12.87802235 295.47293003 11.8524057 -31.40936535
-12.70980284 -147.36565146 862.86515729 -132.51689712 9.61127764
92.96980932 1579.50541747 274.38902471 4611.00333644 -758.09671065]
```

Figure 4.36 Variable importance by permutation measures for CPI explanation model.

Source: Own calculation, with use of Python/TensorFlow/scikit-learn.

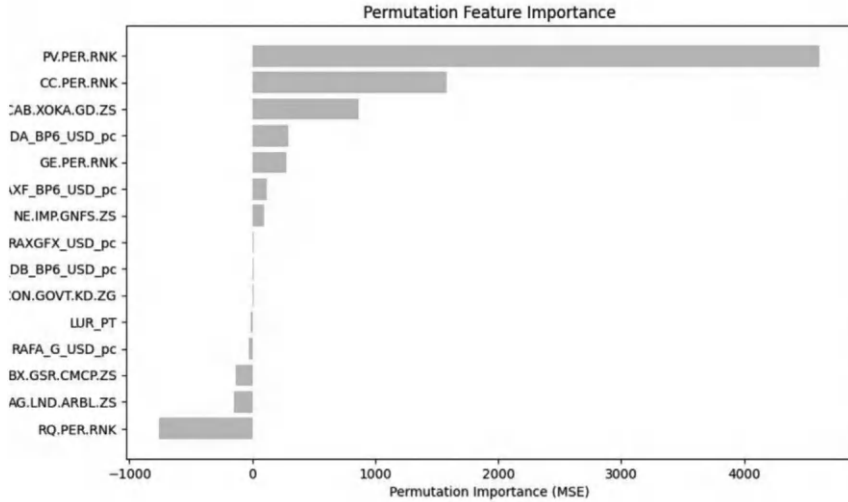


Figure 4.37 Variable importance by permutation measures for CPI explanation model – bar plot

Source: Own calculation, with use of Python/TensorFlow/scikit-learn.

	variable	contribution	...	label	B
0	RQ.PER.RNK = 0.4524	65.930777	...	CPI SHAP Pipeline	0
1	CC.PER.RNK = 0.4479	-49.410201	...	CPI SHAP Pipeline	0
2	NE.IMP.GNFS.ZS = 0.08092	-48.010670	...	CPI SHAP Pipeline	0
3	PV.PER.RNK = 0.2695	43.108534	...	CPI SHAP Pipeline	0
4	AG.LND.ARBL.ZS = 1.317	-25.908705	...	CPI SHAP Pipeline	0
5	BX.GSR.CMCP.ZS = 1.077	19.408824	...	CPI SHAP Pipeline	0
6	GE.PER.RNK = 0.03824	-14.963943	...	CPI SHAP Pipeline	0
7	LUR_PT = -0.9197	7.023280	...	CPI SHAP Pipeline	0
8	RAFA_G_USD_pc = 0.1683	5.635158	...	CPI SHAP Pipeline	0
9	BN.CAB.XOKA.GD.ZS = 0.3772	-3.207256	...	CPI SHAP Pipeline	0
10	RAXGFX_USD_pc = 0.183	-1.991927	...	CPI SHAP Pipeline	0
11	BFDA_BP6_USD_pc = -0.07463	-1.431776	...	CPI SHAP Pipeline	0
12	NE.CON.GOV.T.KD.ZG = 0.1909	0.667915	...	CPI SHAP Pipeline	0
13	BCAXF_BP6_USD_pc = 0.05905	0.228100	...	CPI SHAP Pipeline	0
14	BK_DB_BP6_USD_pc = 0.08514	0.021412	...	CPI SHAP Pipeline	0

Figure 4.38 SHAP values for Poland 2020 – CPI explanation model.

Source: Own calculation, with use of Python/TensorFlow/scikit-learn.

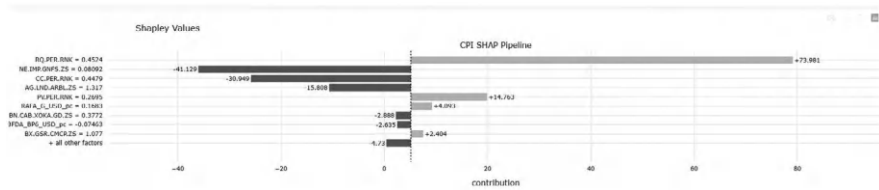


Figure 4.39 SHAP values for Poland 2020 – CPI explanation model (barplot).

Source: Own calculation, with use of Python/TensorFlow/scikit-learn.

Among the financial account components, *the Balance of Payments for the Financial Account – capturing net lending or net borrowing, direct investment, and net acquisition of financial assets* – exhibits a high VIP value of about 295, signifying its substantial role in explaining CPI fluctuations. *The Balance of Payments for the Capital Account, Total (Debit) in US dollars*, on the other hand, shows a much smaller VIP value near 12, implying a less pronounced effect. *Additionally, the Total International Reserves (expressed in US dollars)* display a negative VIP value around -31, suggesting an inverse relationship with CPI changes. Meanwhile, the Labor Markets indicator, *the Unemployment Rate expressed in percent*, has a negative VIP value of approximately -13, reflecting its potential role in reducing inflationary pressures.

Other economic fundamentals further underscore the complex interplay affecting CPI. The percentage of *Arable Land* in the total land area registers a VIP value of roughly -147, pointing to a notable inverse association, which may capture underlying agricultural and land-use dynamics. *The Current Account Balance as a percentage of GDP* shows a significant VIP value of about 863, emphasizing the critical influence of external economic imbalances on domestic price levels. Additionally, communication-related exports and several ranking measures (with VIP values of 1579, 274, 4611, and -758, respectively) appear to be strong predictors in the model. Particularly, the exceptionally high VIP value of 4611 for one ranking measure suggests that broader economic performance metrics play a pivotal role in shaping CPI trends. Collectively, these findings underscore that a diverse array of financial, demographic, and structural economic variables interact to influence the dynamics of consumer prices.

The SHAP analysis for the CPI model in Poland for 2020 reveals the relative contributions of various economic variables to the model's predictions. *Regulatory Quality (Percentile Rank)* shows a substantial positive contribution of approximately 65.93, while *Control of Corruption (Percentile Rank)* exhibits a significant negative contribution of about -49.41. These ranking variables, although not directly tied to conventional economic metrics, appear to encapsulate multidimensional aspects of economic performance that heavily influence CPI variations.

Traditional economic indicators also play a critical role in explaining CPI movements. *The Balance of Payments for Financial Account*, which encompasses net lending/borrowing, direct investment, and net acquisition of financial assets

(BFDA_BP6_USD), contributes negatively, albeit modestly (approximately -1.43), indicating a minor inverse relation to CPI changes. Similarly, *the International Liquidity measure, representing Total Reserves excluding Gold* (RAXGFX_USD), shows a small negative impact (around -1.99), while the *Balance of Payments for Supplementary Items in the Current Account* (BCAXF_BP6_USD) has a minimal positive contribution (approximately 0.23). Other important economic metrics such as the *Current Account Balance as a percentage of GDP* (BN.CAB.XOKA.GD.ZS) and *Communications, Computer, etc.* (BX.GSR.CMCP.ZS) exhibit contributions of roughly -3.21 and 19.41 , respectively, highlighting the nuanced influences these factors exert on the CPI.

Furthermore, structural and fiscal variables also contribute to the model's explanation of CPI fluctuations. *The Labor Markets indicator, specifically the Unemployment Rate* (LUR_PT), shows a small positive contribution (about 7.02), suggesting that higher unemployment might be associated with lower inflationary pressures in this context. *Total International Reserves* (RAFA_G_USD) contribute modestly positively (approximately 5.64), while the *General Government Final Consumption Expenditure* (NE.CON.GOV.TK.ZG) and *Imports of Goods and Services* (NE.IMP.GNFS.ZS) contribute around 0.67 and -48.01 , respectively. Notably, *the measure of Arable Land* (AG.LND.ARBL.ZS) has a considerable negative impact (approximately -25.91), implying that a higher percentage of arable land might be linked to specific agricultural dynamics affecting CPI.

Notes

- 1 Author considers the use of latitude and longitude as ordinary explanatory variables to be a substantive error – their meaning is slightly different.
- 2 Author is aware that stock and FX webservices like Yahoo Finance or Quandl deal with bigger volumes of data, but they should be treated differently due to time series nature.

References

- Aho, A. V., Hopcroft, J. E., & Ullman, J. D. (1974). *The design and analysis of computer algorithms*. Addison-Wesley.
- Biecek, P., & Burzykowski, T. (2021). *Explanatory model analysis: Explore, explain, and examine predictive models*. CRC Press.
- Gosiewska, A., & Biecek, P. (2019). iBreakDown: Uncertainty of model explanations for non-additive predictive models. *arXiv*. <https://arxiv.org/abs/1903.11420v1>
- Kim, B., Wattenberg, M., Gilmer, J., Cai, C., Wexler, J., & Viegas, F. (2018, July). Interpretability beyond feature attribution: Quantitative testing with concept activation vectors (tcav). In *International conference on machine learning* (pp. 2668–2677). PMLR.
- Koop, G. (2006). *Analysis of financial data* (2nd ed.). Wiley.
- Kozyra, K. (n.d.). *Kozyra K piramida* [Digital image]. Wikimedia Commons. https://upload.wikimedia.org/wikipedia/commons/thumb/6/6f/Kozyra_K_piramida.jpg/800px-Kozyra_K_piramida.jpg
- Lundberg, S., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *arXiv*. <https://arxiv.org/abs/1705.07874>
- Maddala, G. S. (2001). *Introduction to econometrics* (3rd ed.). Wiley.

- Mahendran, A., & Vedaldi, A. (2015). Understanding deep image representations by inverting them. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 5188–5196). <https://doi.org/10.1109/CVPR.2015.7299>
- Molnar, C. (2020). *Interpretable machine learning: A guide for making black box models explainable*. Leanpub.
- Pace, R. K., & Barry, R. (1997). Sparse spatial autoregressions. *Statistics & Probability Letters*, 33(3), 291–297.
- Ribeiro, M. T., Singh, S., & Guestrin, C. (2016). “Why Should I Trust You?”: Explaining the Predictions of Any Classifier. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1135–1144. <https://doi.org/10.1145/2939672.2939778>
- Selvaraju, R. R., Cogswell, M., Das, A., Vedantam, R., Parikh, D., & Batra, D. (2017). Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE International Conference on Computer Vision* (pp. 618–626).
- Shapley, L. (1953) *A Value for n-Person Games*. In: Kuhn, H. and Tucker, A., Eds., *Contributions to the Theory of Games II*, Princeton University Press, Princeton, 307–317.
- Simonyan, K., Vedaldi, A., & Zisserman, A. (2013). Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034*.
- Smilkov, D., Thorat, N., Kim, B., Viégas, F., & Wattenberg, M. (2017). SmoothGrad: Removing noise by adding noise. *arXiv preprint arXiv:1706.03825*. <https://arxiv.org/abs/1706.03825>
- Štrumbelj, E., & Kononenko, I. (2010). An efficient explanation of individual classifications using game theory. *Journal of Machine Learning Research*, 11(March), 1–18.
- Štrumbelj, E., & Kononenko, I. (2014). Explaining prediction models and individual predictions with feature contributions. *Knowledge and Information Systems*, 41(3), 647–665. <https://doi.org/10.1007/s10115-013-0679-x>
- Sung, A. H. (1998). Ranking importance of input parameters of neural networks. *Expert systems with Applications*, 15(3–4), 405–411.
- Tsang, W. K., & Benoit, D. F. (2023). Interpretability and explainability in machine learning. In Y. Ohsawa (Ed.), *Living Beyond Data. Intelligent Systems Reference Library* (Vol. 230). Springer. https://doi.org/10.1007/978-3-031-11593-6_5
- Whitaker, J. K. (2008). Ceteris Paribus. In S. N. Durlauf & L. E. Blume (eds.), *The new Palgrave dictionary of economics* (2nd ed.). Palgrave Macmillan. https://link.springer.com/rwe/10.1057/978-1-349-95121-5_346-2
- Zeiler, M. D., & Fergus, R. (2014). Visualizing and understanding convolutional networks. In *European Conference on Computer Vision* (pp. 818–833). Springer. https://doi.org/10.1007/978-3-319-10590-1_53
- Zhou, B., Khosla, A., Lapedriza, A., Oliva, A., & Torralba, A. (2016). Learning deep features for discriminative localization. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 2921–2929). <https://doi.org/10.1109/CVPR.2016.319>
- Zurada, J. M., Malinowski, A., & Cloete, I. (1994, May). Sensitivity analysis for minimization of input data dimension for feedforward neural network. In *Proceedings of IEEE International Symposium on Circuits and Systems-ISCAS'94* (Vol. 6, pp. 447–450). IEEE.

5 Time series analysis and forecasting with deep learning models

5.1 The specificity of time series data and its impact on the model architecture

While the aim of regression and discriminant studies described in [Chapters 3 and 4](#) was to “capture”, describe, interpret, and appropriately use the relationship between the explained variable and the explanatory variables, in the case of time series analysis based on historical observations of the same phenomenon (or phenomena in the instance of, e.g., Multivariate Generalized Autoregressive Conditional Heteroskedasticity (GARCH)), an attempt is made to answer the question of what mechanisms caused the phenomenon to proceed in a certain way and not otherwise. In the process of model construction based on a time series, we want to learn the properties of the mechanism that generated it, i.e., the stochastic process, and use this process to forecast the course of the phenomenon (values of analyzed variable) in the future.

Starting our discussion, we should mention the nomenclature convention. Although we are dealing with **prediction** (a slightly broader term) in regression and discriminant analysis tasks, **forecasting** is more common in the context of time series analysis with a given order and time range of observed phenomena.

Accurate forecasting often demands a strong understanding of advanced mathematics; however, numerous high-quality sources cover the intricacies of time series modeling (e.g., [Box & Jenkins, 1970](#) and subsequent editions; [Brockwell & Davis, 2016](#); [Hyndman & Athanasopoulos, 2018](#)). This chapter emphasizes time series forecasting using recurrent and other deep neural networks, with mathematical concepts introduced only where necessary to support the discussion.

Among the most commonly referenced in the literature on the subject and most commonly used for benchmark or real data research studies, autoregressive integrated moving average (ARIMA), GARCH, Brownian motion, random walk, exponential smoothing, seasonal decomposition, and state-space dynamic models' families should be mentioned.

Autoregressive models (ARMA/ARIMA): ARMA models blend autoregressive (AR) elements, which forecast a variable using its past values, with moving average (MA) elements that consider previous forecast errors. For non-stationary data, ARIMA includes a differencing step (I) to stabilize the mean. The main

advantage of these models developed by Box and Jenkins in 1970, is the simplicity and clarity. Their linear nature provides clear insights into data patterns, though they may need transformations or differencing for non-stationary data.

Volatility models (ARCH/GARCH): ARCH models, introduced by Engle in 1982 and their GARCH extension by Bollerslev in 1986, focus on capturing volatility clustering in time series. It's especially useful when periods with relatively low volatility of assets may be mixed with periods of high volatility, so GARCH family models have a large area of application in risk management and pricing.

Stochastic processes (Brownian motion): Brownian motion, described by Einstein in 1905 and Wiener in 1923, is a key model in physics. In finance, it represents the random movement of particles in a fluid, which is adapted to model asset price fluctuations.

Random walk: The random walk theory, popularized by Fama in 1965, suggests that a time series' current value depends on its past value plus a random change. This implies that price changes are unpredictable, and past movements don't reliably forecast future values. Though simple, this model supports the idea of efficient markets, where price movements are random, challenging the predictability of asset returns.

Smoothing methods (exponential smoothing): Exponential smoothing forecasts by giving more weight to recent data, with older data having less impact. Developed by Holt in 1957 and extended by Winters in 1960 to include trend and seasonal components, it is used in many studies for its flexibility with a focus on short-term forecasting where quick adaptation is essential.

Time Series Decomposition/Seasonal Decomposition of Time Series by Loess (STL): STL, introduced by Cleveland et al. in 1990, breaks down a time series into trend, seasonal, and residual components using locally weighted regression. It's effective for complex seasonal patterns or changing seasonal components, improving forecasting accuracy by modeling each component separately.

Dynamic models (state-space models): State-space models assume observed data comes from a hidden state process that evolves over time. Coupled with the Kalman filter (Kalman, 1960), these models estimate hidden states dynamically and account for measurement noise and system uncertainty. They are useful in complex or changing environments, providing a systematic way to update forecasts with new data.

Table 5.1 summarizes the description of the main methods of time series data analysis.

5.1.1 Stationarity

Stationary stochastic processes are probability models for time series with time-invariant behavior. A process is said to be strictly stationary if all aspects of its behavior are unchanged by shifts in time. Mathematically, stationarity is defined as the requirement that for every m and n , the distributions of $(Y_1, \dots, Y_n)$ and $(Y_{1+m}, \dots, Y_{n+m})$ are the same; that is, the probability distribution of a

Table 5.1 Main methods of time series data analysis

Method	Family	References	Description
ARMA/ARIMA	Autoregressive models	Box & Jenkins (1970)	ARMA combines autoregressive (AR) and moving average (MA) components, while ARIMA adds integration to handle non-stationarity.
ARCH/GARCH	Volatility models	Engle (1982); Bollerslev (1986)	ARCH models conditional heteroskedasticity in time series; GARCH extends ARCH by incorporating lagged volatility terms.
Brownian motion	Stochastic processes	Einstein (1905); Wiener (1923)	Brownian motion describes random continuous movement over time, fundamental in financial modeling and physics.
Random walk	Stochastic processes	Fama (1965)	Random walk is a stochastic process where the current value depends only on the previous value and a random step.
Exponential smoothing	Smoothing methods	Holt (1957); Winters (1960)	Exponential smoothing is used for forecasting by weighting recent observations more heavily.
Seasonal Decomposition of Time Series by Loess (STL)	Decomposition methods	Cleveland et al. (1990)	STL decomposes a time series into seasonal, trend, and residual components.
State-space models	Dynamic models	Kalman (1960)	State-space models represent time series using hidden states, allowing flexible modeling of dynamic systems.

Source: Own research based on sources listed in reference column.

sequence of n observations does not depend on their time origin. Strict stationarity is a very strong assumption, because it requires that “all aspects” of stochastic behavior be constant in time. Often, it will suffice to assume less, namely, weak stationarity. A process is weakly stationary if its mean, variance, and covariance are unchanged by time shifts. More precisely, $Y_1, Y_2, \dots$ is a weakly stationary process if mean $E(Y_t) = \mu$ (a finite constant) for all t ; variance $Var(Y_t) = \sigma^2$ (a positive finite constant) for all t ; and covariance $Cov(Y_t, Y_s) = \gamma(|t - s|)$ for all t and s for some function $\gamma(h)$. The function γ is called the autocovariance function of the process. Figure 5.1 presents stationary and non-stationary stochastic process.

Ensuring stationariness may be achieved by differentiation (1,2,3 ... times). In real case/benchmark studies, instead of mathematical differentiation, calculation of return ratios or log return ratios are used.

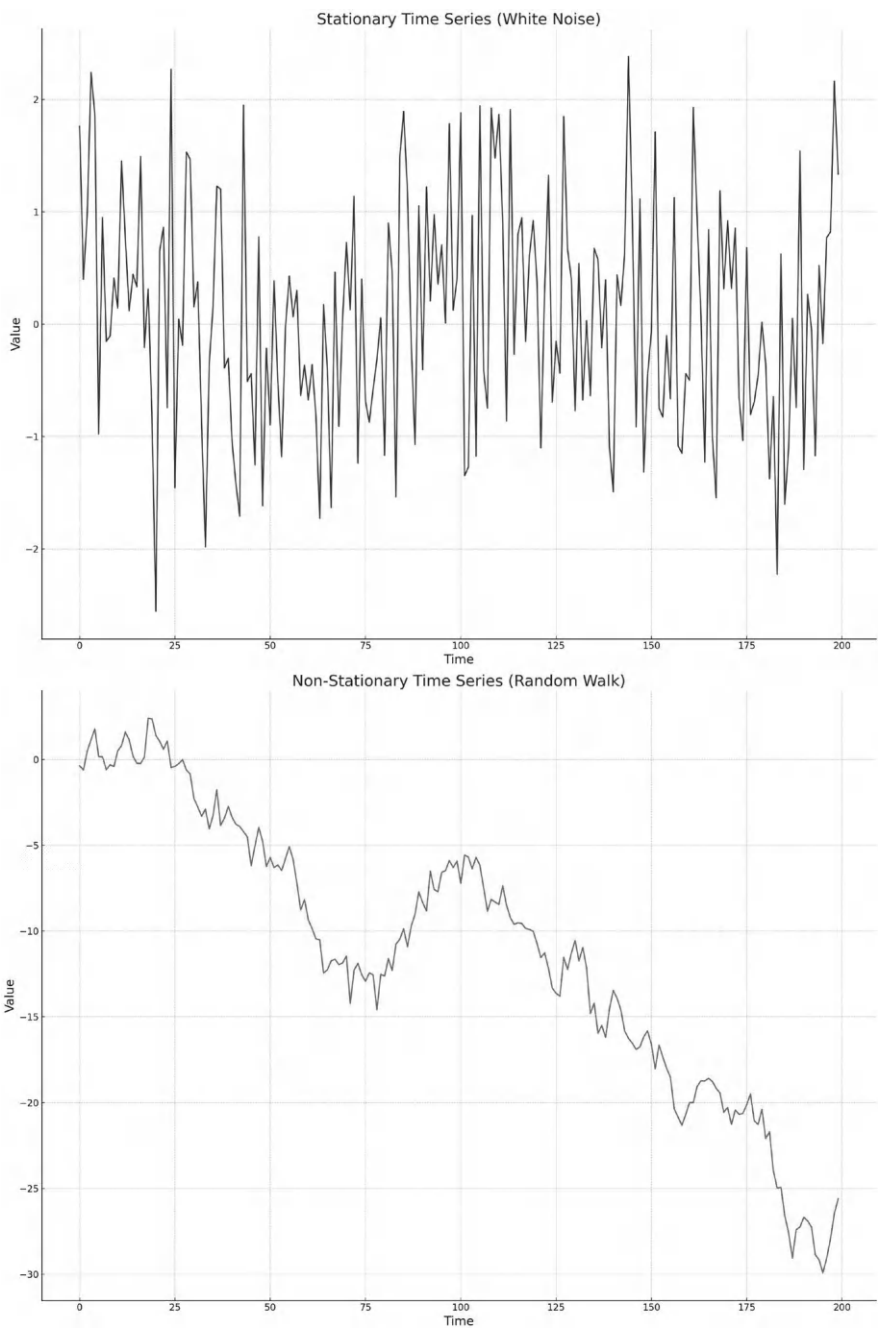


Figure 5.1 Stationary vs non-stationary time series.

Source: Own work, with Python/NumPy.

5.1.2 Non-stationarity with unit roots

Unit root is a feature of some stochastic processes that can cause problems in statistical inference involving time series models (Figure 5.2). A linear stochastic process has a unit root if 1 is a root of the process's characteristic equation. Such a process is non-stationary, but does not always have a trend.

If the other roots of the characteristic equation lie inside the unit circle – that is, have a modulus (absolute value) less than one – then the first difference of the process will be stationary; otherwise, the process will need to be differenced multiple times to become stationary. If there are d unit roots, the process will have to be differenced d times in order to make it stationary. Due to this characteristic, unit root processes are also called “difference stationary”.

If a root of the process's characteristic equation is larger than 1, then it is called an “explosive process”, even though such processes are sometimes inaccurately called “unit roots processes”.

While avoiding unit roots and ensuring stationarity is crucial for proper design of ARIMA and GARCH family models, there is no consensus that it should be applied in the data preparation stage in the pipeline of deep learning models. Some authors suggest that datasets with stationarity ensures better behavior than raw data but, others argue that having low p-values in an Augmented Dickey-Fuller (ADF)

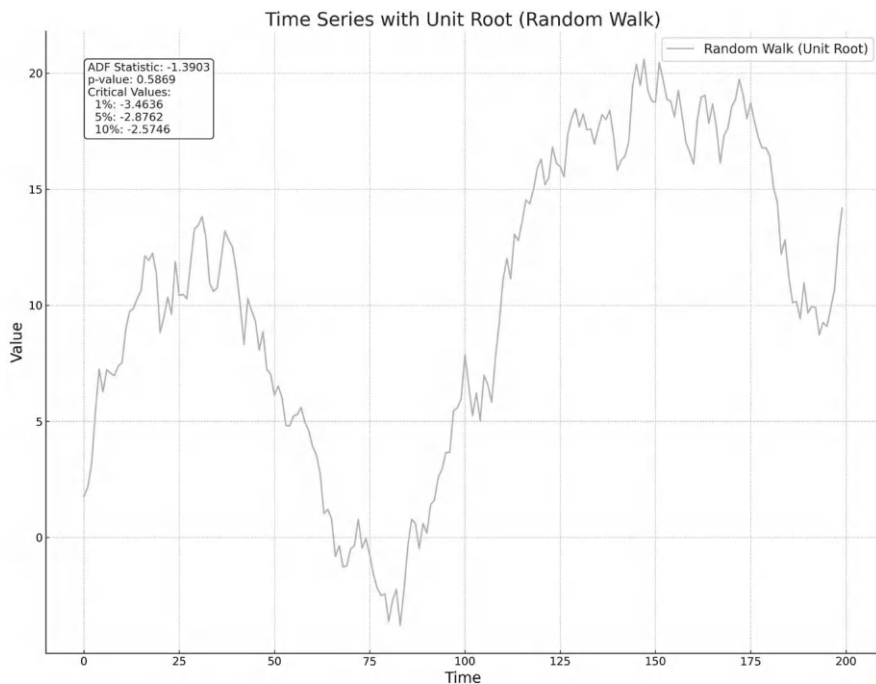


Figure 5.2 Unit root in stochastic process.

Source: Own work, with Python/NumPy.

test (see [Dickey & Fuller, 1979](#)) doesn't necessarily lead to proper model generalization (the discussion over generalization from [Chapters 2](#) and [3](#), although not raised yet, fully applies to time series modeling).

5.1.3 *Linearity*

When designing forecasting procedure, we should have in mind that models from ARIMA and GARCH families have some linearity built into their structure.

ARIMA is considered a linear model. It models a time series as a linear combination of its past values (autoregressive part) and past error terms (moving average part). The integration (differencing) component is also a linear operation, which helps stabilize the series' mean. This linearity means that the relationship between the input variables (lags) and the output is linear.

For GARCH family models, linearity is built into past squared errors and variances.

This condition is crucial for accurately predicting stock prices and return ratios. In practical scenarios, when forecasting stock values and return ratios, a series containing data with an average close to 0 and significant variability often leads to a situation where a model trained on more than a certain number of historical observations – [Hyndman and Athanasopoulos \(2018\)](#) estimate this number to be 200 – tends to generate prediction series that are close to linear. Based on various experiments, we can suggest that for typically behaving stocks, this number is relatively closer to 100.

We may observe this behavior on the simple routine for NVIDIA-10 day forecasting ("backcasting").

```
import numpy as np
import pandas as pd
import yfinance as yf
import pmdarima as pm
from sklearn.metrics import mean_squared_error
import matplotlib.pyplot as plt

ticker = "NVDA"
data=yf.download(ticker, start="2020-01-01",end="2025-02-15")
data = data.sort_index()

prices = data['Adj Close'].dropna()

forecast_steps=10
def simple_auto_arma_forecast(series, train_start_idx,
train_end_idx, forecast_steps):
    train = series.iloc[train_start_idx:train_end_idx].copy()
    try:
        model = pm.auto_arma(train, seasonal=False,
error_action='ignore',
                                suppress_warnings=True,
stepwise=True)
```

```

        forecast = model.predict(n_periods=forecast_steps)
        predictions=forecast.values
    except Exception as e:
        print(f"auto_arima fitting error")
        predictions = [np.nan]*forecast_steps
    return pd.Series(predictions, index=series.index[-
forecast_steps:])

def compute_rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

train_windows = [50, 100, 200, 300]
predictions=pd.DataFrame()
fig, axs = plt.subplots(nrows=len(train_windows), ncols=1,
figsize=(12, 6 * len(train_windows)))

for idx, i in enumerate(train_windows):
    n = len(returns)
    if n < i + forecast_steps:
        raise ValueError("Not enough data for the specified
training window for returns.")
    train_start_ret = n - i - forecast_steps
    train_end_ret = n - forecast_steps
    forecast_prices = simple_auto_arima_forecast(prices,
train_start_ret, train_end_ret, forecast_steps)
    predictions[i] = forecast_prices
    rmse = compute_rmse(prices[-forecast_steps:],
        forecast_prices)
    print(f"RMSE for history of {i} is equal to {rmse:3.3f}")

    ax = axs[idx]
    ax.plot(prices[-(i + forecast_steps):].index, prices[-(i +
forecast_steps):], label='Historical Returns',
        color='blue')
    ax.plot(forecast_prices.index, forecast_prices,
label='auto_arima Forecast Prices',
        color='red', marker='o', linestyle='--')
    ax.set_title(f"{ticker} NVDA stock prices: Historical {i}
vs. auto_arima Forecast (Last {forecast_steps} Sessions)")
    ax.set_xlabel("Date")
    ax.set_ylabel("Price")
    ax.legend()
    ax.grid(True)

plt.tight_layout()
plt.savefig("arima_simple_prices.jpg", bbox_inches='tight')
plt.show()
print(predictions)

```


Please note that the simplest way to repeat the analysis on current day is to change:

```
data=yf.download(ticker, start="2020-01-01",end="2025-02-15")

to e.g.

data = yf.download(ticker, period="5y")
```

The results of Auto-ARIMA modeling of NVIDIA stock prices on 15 February 2015 are presented in [Figure 5.3](#) with the subsequent predictions for given history data length and as time series charts in [Figure 5.4](#).

We can discuss the linearity of the predictions as follows:

- **50-day history:** The 50-day predictions increase steadily from about 128.67 on 3 February 2025 to roughly 137.08 on 14 February 2025. This near-upward trend suggests that using a shorter history captures the recent momentum effectively. The predictions are smooth and exhibit a clear trend.
- **100-day history:** The 100-day predictions are completely constant at 124.65 for every date. This is a perfectly linear (flat) forecast, indicating that the model based on 100 days of history produces a stationary estimate. In this case, the averaging effect over a longer period may have muted the influence of recent changes, leading to no variation in the forecast.
- **200-day history:** The 200-day predictions oscillate between values around 122.37 and 124.32. They do not follow a monotonic or smooth linear trend, but rather fluctuate in a narrow range. This variability suggests that a 200-day history introduces more historical noise, and the model reacts to a mix of older and more recent information, resulting in less strict but still linear behavior.
- **300-day history:** The predictions based on 300 days of history are similar to the 200-day ones. They vary slightly, ranging roughly from 122.65 to 124.35,

	50	100	200	300
Date				
2025-02-03	128.674279	124.650002	122.371727	122.652181
2025-02-04	131.424562	124.650002	124.323337	124.346440
2025-02-05	133.304169	124.650002	122.651553	122.909617
2025-02-06	134.588736	124.650002	124.083633	124.128120
2025-02-07	135.466638	124.650002	122.856888	123.094764
2025-02-10	136.066617	124.650002	123.907740	123.971105
2025-02-11	136.476656	124.650002	123.007562	123.227921
2025-02-12	136.756886	124.650002	123.778670	123.858181
2025-02-13	136.948402	124.650002	123.118125	123.323687
2025-02-14	137.079288	124.650002	123.683960	123.776966

Figure 5.3 The results of Auto-ARIMA modeling of NVIDIA stock prices on 15 February 2025.
Source: Own calculations with Python/pmdarima.

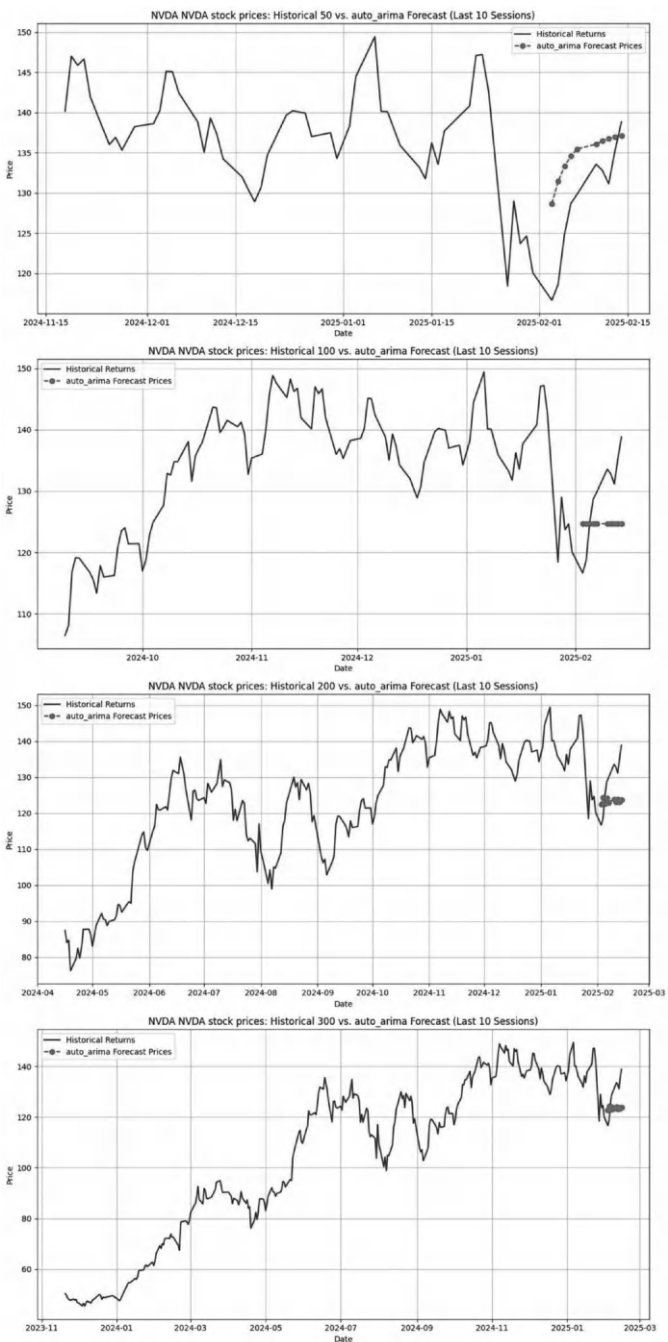


Figure 5.4 The results of Auto-ARIMA modeling of NVIDIA stock prices on 15 February 2015 (time series charts)

Source: Own calculations with Python/pmdarima/Matplotlib.

indicating modest fluctuations rather than a consistent linear trend. Again, the inclusion of a longer historical window seems to smooth out some short-term dynamics, but also incorporates variability from a broader period, yielding again a less strictly linear pattern.

Deep recurrent networks, with their inherent remembering and forgetting mechanisms (which were no designed to mitigate linear behavior per se, but rather to address issues like vanishing gradients and the difficulty of learning long-term dependencies), exhibit a natural resistance to linearity. Consequently, these networks do not require the implementation of specialized techniques to mitigate linear behavior and do not suffer from “long prediction linearity” as in the example above.

5.1.4 Direct prediction vs stepwise recurrent prediction

Some methods are not designed to predict the series of prediction with just one point prediction. In such cases the stepwise procedure is usually used. When a working model of order k is employed for multistep prediction, a natural predictor of $x_{\{n+h\}}$ for $h \geq 1$ can be constructed as follows. First, the model parameters are estimated from the observed data. Then, the h -step ahead predictor, denoted by $\hat{x}_{n+h}^{(k)}$, is obtained by iteratively applying the fitted model, where unknown future values are recursively replaced by their own forecasts. In what follows, $\hat{x}_{n+h}^{(k)}$ is referred to as the *plug-in predictor* or stepwise (iterative) forecasting (cf. Kunitomo & Yamamoto, 1985; Bhansali, 1997).

Iterative methods for multi-horizon forecasting rely on autoregressive deep learning architectures, recursively feeding past target values into future time steps to generate forecasts. This approach enables multistep prediction by leveraging Monte Carlo estimates from multiple sampled trajectories, allowing the model to generalize from one-step-ahead forecasting to longer horizons. However, due to error accumulation at each step, iterative methods can introduce significant forecasting inaccuracies over extended periods. Therefore, it is not recommended to use them for forecasts exceeding a few/a dozen observations (Figure 5.5).

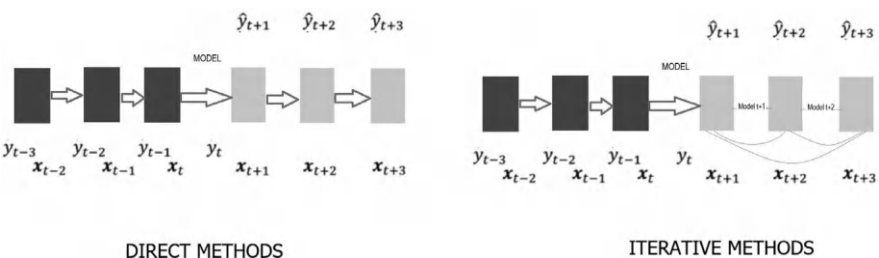


Figure 5.5 Direct vs iterative methods.

Source: Own work based on Lim (2020).

5.1.5 Seasonality

Seasonality is a fundamental characteristic of time series data, representing recurring patterns at fixed intervals, such as daily, weekly, or yearly cycles. Traditional statistical forecasting methods, such as Seasonal ARIMA or exponential smoothing, explicitly model seasonality using predefined periodic components. However, deep learning models, particularly recurrent neural networks (RNNs) and their advanced variants like long short-term memory (LSTM) and gated recurrent unit (GRU) networks learn seasonality implicitly from raw data without requiring explicit specification of seasonal patterns. While this flexibility allows RNN-based models to capture complex temporal dependencies, they often struggle with long-term seasonal structures due to vanishing gradients and limited memory capacity in recurrent layers. Additionally, recurrent models trained on shorter time windows may fail to generalize well to seasonal cycles extending beyond their training horizon.

Deep recurrent networks can benefit from hybrid approaches that integrate seasonality-aware mechanisms, such as attention mechanisms, external covariates, or frequency-domain transformations. Incorporating exogenous seasonal indicators, such as calendar effects or Fourier terms, can improve long-term forecasting accuracy by providing explicit seasonality cues. Moreover, transformer-based architectures mitigate the limitations of recurrent models by leveraging self-attention, allowing direct long-range dependencies without sequential information loss. These advancements demonstrate that while deep recurrent networks offer powerful forecasting capabilities, their effectiveness in seasonal forecasting can be enhanced by hybridizing them with domain-specific knowledge and alternative architectural innovations. Traditional methods catch relatively long-range momentum, while recurrent networks require more extended history for proper training

5.1.6 Data transformation step

Running deep learning models as forecasting tools on raw data is not a good idea. To properly fit into the model, data should be normalized or integrated. On the other hand, stochastic process methods like ARIMA and GARCH integration are needed in non-stationarity cases (which is almost every scenario for real-time data like stock values or energy demand values). So, proper data transformation is needed as the first step of any method's benchmark analysis.

In the previously conducted research study, author compared five ways of preparing data for forecasting:

- Raw data;
- Max-min scaled data;
- Integrated (return ratios instead of values) data;
- Integrated and scaled to percentages (return ratios multiplied by 100%) data;
- Integrated and then max-min scaled data.

With the following conclusions:

- Data preparation for GARCH and ARIMA requires the integration step.
- The best strategy for LSTM and GRU is to scale raw data.
- Brownian motion and exponential smoothing don't require the transformation step.

The paper describing data preparation step for time series analysis research study will be published in *Econometric. Advances in Applied Data Analysis, Wroclaw University of Economics and Business Publishing*. Its preprint version is available at https://github.com/a-dudek-ue/erwdlm_book/time_series_data_transformation/.

It is worth emphasizing that benchmark analysis comparing forecast measures with the same data preparation step is not entirely factually correct (with known examples, but no reason to cite them here) because every method needs a different preparation, and only the result after reverse transformation should be compared.

5.1.7 *Time series analysis in economics*

Time series analysis has numerous applications across various fields of economics. It is widely used for modeling, forecasting, and understanding economic phenomena over time within the following areas:

- **Macroeconomic forecasting:** Time series analysis is widely used for forecasting key macroeconomic indicators such as gross domestic product (GDP) growth, inflation, unemployment rates, and interest rates.
- **Business cycle analysis:** Time series methods help identify economic cycles (boom and recession phases) using trend decomposition and spectral analysis (see [Hodrick & Prescott, 1997](#)).
- **Exchange rate and monetary policy analysis:** Time series models are used to understand the behavior of exchange rates and their impact on monetary policy.
- **Labor market and employment studies:** Economists use time series analysis to study unemployment trends, labor force participation rates, and wage dynamics.
- **Energy and commodity price forecasting:** Time series models help forecast oil prices, gas prices, and commodity markets, which are critical for economic stability.
- **Public finance and tax revenue forecasting:** Governments use time series models to predict tax revenues, fiscal deficits, and public expenditures.
- **Economic impact of pandemics and shocks:** Time series models analyze how pandemics (e.g., COVID-19) or financial crises impact economic variables (see [McKibbin & Fernando, 2020](#)).
- **Housing and real estate markets:** Time series analysis is applied to predict housing prices, rental markets, and mortgage rates. In particular, Hedonic price models use time series techniques to analyze trends in real estate prices ([Case & Shiller, 1989](#); [Jiang & Yu, 2014](#)).
- **Energy consumption forecasting:** Time series models are commonly used to predict the consumption level of utilities and gas ([Nichiforov et al., 2017](#); [Nowak & Bartnicki, 2019](#)).
- **Financial market analysis:** Time series models help analyze stock prices, exchange rates, bond yields, and commodity prices.

5.1.8 Methods for deep learning-based time series forecasting

RNNs have been widely used for time series forecasting because they can model temporal dependencies by maintaining a hidden state across time steps (Elman, 1990). However, traditional RNNs suffer from the vanishing gradient problem, making them inefficient for long-term forecasting tasks where dependencies extend over many time steps. LSTM networks address this issue by introducing gated mechanisms – input, forget, and output gates – that regulate the flow of information and enable models to retain relevant past information over extended sequences (Hochreiter & Schmidhuber, 1997). These improvements have made LSTM a standard choice for forecasting applications, including financial modeling, weather prediction, and energy consumption forecasting (Hochreiter & Schmidhuber, 1997).

Convolutional neural networks (CNNs) have been effectively adapted for time series forecasting through causal and dilated convolutions (Yu & Koltun, 2016), ensuring that only past observations influence future predictions (Bai et al., 2018). Unlike recurrent architectures, CNNs process time series in parallel, efficiently capturing local and long-range dependencies by stacking multiple convolutional layers with increasing dilation rates. This approach allows CNNs to achieve competitive performance in sequence modeling tasks while significantly reducing training time compared to recurrent alternatives (Bai et al., 2018).

Recent advancements in deep learning have led to the adaptation of transformer models for time series forecasting by replacing recurrent structures with self-attention mechanisms (Vaswani et al., 2017). Unlike RNNs, transformers process entire sequences in parallel, efficiently capturing long-term dependencies without suffering from vanishing gradients. The self-attention mechanism dynamically assigns importance to different time steps, allowing models to focus on the most relevant historical information when making predictions (Zhou et al., 2021). These architectures have demonstrated state-of-the-art performance in long-horizon forecasting tasks, outperforming traditional recurrent and convolutional models while offering improved scalability and interpretability (Zhou et al., 2021).

5.2 Research application example: comparison of Brownian motion methods, GARCH family methods and RNNs in prediction of volatility of stock prices

In the analysis the following methods of forecasting have been compared

- ARIMA (2,1,2)
- GARCH (1,1)
- Holt Winters Exponential Smoothing
- Brownian movement method
- GRU Network
- LSTM Network

The comparison has been evaluated on real stock of ten eminent NYSE/NASDAQ enterprises, as listed in Table 5.2

Table 5.2 Ticker names and enterprise descriptions for stock values forecasting simulation study

<i>Ticker</i>	<i>Company name (mother company if applicable)</i>	<i>Establishing Year</i>	<i>First stock session year</i>	<i>Short description</i>
AAPL	Apple Inc.	1976	1980	American multinational technology company that designs, manufactures, and markets consumer electronics, software, and services.
GOOGL	Alphabet Inc. (Google)	1998	2004	American multinational conglomerate that oversees Google, specializing in internet-related services and products.
MSFT	Microsoft Corporation	1975	1986	American multinational technology company producing software, hardware, and services, notably Windows and Office.
AMZN	Amazon.com , Inc.	1994	1997	American multinational technology company focusing on e-commerce, cloud computing, digital streaming, and artificial intelligence.
TSLA	Tesla, Inc.	2003	2010	American electric vehicle and clean energy company leading innovations in battery technology and sustainable transportation.
NFLX	Netflix, Inc.	1997	2002	American subscription streaming service and production company that revolutionized online video content delivery.
NVDA	NVIDIA Corporation	1993	1999	American multinational technology company known for its graphics processing units (GPUs) used in gaming, professional visualization, and accelerating AI applications.
JPM	JPMorgan Chase & Co. (modern form via merger)	1799/2000	2000	American multinational investment bank and financial services holding company, with a legacy dating back to the 18th century through predecessors.
BAC	Bank of America Corporation	1904	1970	American multinational investment bank and financial services corporation, serving a wide range of financial and banking needs.
INTC	Intel Corporation	1968	1971	American multinational corporation and technology company, a major supplier of semiconductor chips and microprocessors.

Source: Own research.

In the analysis, for reproducibility purposes, the fixed end day for final prediction was set to 15 February 2025, but it may be repeated for any particular date.

GARCH, Brownian motion LSTM, and GRU forecasting was used in an iterative forecasting version.

The following Python code has been run for a comparative simulation study:

```
import random
import numpy as np
import pandas as pd
import yfinance as yf
import matplotlib.pyplot as plt
import datetime
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from arch import arch_model
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, GRU, Dense, Dropout
from tensorflow.keras.optimizers import Adam
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error
import os
import sys

PRICE_COLUMN = 'Close'

PENALTY = -100

VALUE_COLUMN = "Return"

SCALER = "scaler"
RETRATIOS = "retrratios"
RETPERCENT = "retpercent"
SCALER_RETRATIOS = "scaler_retrratios"
RAW = "raw"
activation='relu'
epochs=250
sequence_length=8
test_scaler=None
random.seed(25032205)

def compute_rmse(y_true, y_pred):
    return np.sqrt(mean_squared_error(y_true, y_pred))

def compute_mae(y_true, y_pred):
    return mean_absolute_error(y_true, y_pred)

def compute_mape(y_true, y_pred):
    # Avoid division by zero if any actual value is zero
```



```

    return np.mean(np.abs((y_true - y_pred) / np.where(y_true
        == 0, 1e-10, y_true))) * 100

def forecast_arima(train_series, steps):
    model = ARIMA(train_series, order=(2, 1, 2))
    model_fit = model.fit()
    forecast = model_fit.forecast(steps=steps)
    return forecast

def forecast_holtwinters(train_series, steps):
    model = ExponentialSmoothing(train_series, trend='add',
        seasonal=None)
    model_fit = model.fit(optimized=True)
    forecast = model_fit.forecast(steps)
    return forecast

def forecast_garch(train_series, steps):
    try:
        am = arch_model(train_series, p=1, q=1, mean='AR',
            lags=1, dist='normal')
        res = am.fit(disp='off')
        forecast = res.forecast(horizon=steps)
        forecast = forecast.mean.iloc[-1].values
    except:
        forecast = [PENALTY] * steps
    return pd.Series(forecast, index=range(steps))

def forecast_brownian(train_series, steps, phi=0.8, noise_std=0.0):
    forecast = []
    current_value = train_series.iloc[-1]

    for _ in range(steps):
        noise = np.random.normal(0, noise_std) if noise_std
            > 0 else 0
        current_value = phi * current_value + noise
        forecast.append(current_value)

    return pd.Series(forecast, index=range(len(train_series),
        len(train_series) + steps))

def forecast_lstm(train_series, steps, sequence_length=sequence_
    length, epochs=epochs):
    # Prepare training sequences
    data = train_series.values
    X, y = [], []
    for i in range(len(data) - sequence_length):
        X.append(data[i:i + sequence_length])
        y.append(data[i + sequence_length])
    X = np.array(X)

```

```

y = np.array(y)
X = X.reshape((X.shape[0], X.shape[1], 1)) # reshape
for GRU [samples, timesteps, features]
    dropout_rate = 0.1

    build_lstm(sequence_length, dropout_rate)
    model_lstm.fit(X, y, epochs=epochs, verbose=0)
    forecast = []
    last_seq = data[-sequence_length:].reshape(1, sequence_
        length, 1)
    for _ in range(steps):
        pred = model_lstm.predict(last_seq, verbose=True)
        forecast.append(pred[0, 0])
        # Reshape pred to have shape (1,1,1) so that dimensions
        match for concatenation
        last_seq = np.append(last_seq[:, 1:, :], pred.reshape
            (1, 1, 1), axis=1)
    return pd.Series(forecast)

def build_lstm(sequence_length, dropout_rate):
    global model_lstm
    if not model_lstm:
        model_lstm = Sequential()
        # model_lstm.add(LSTM(50, activation='relu', input_
shape=(sequence_length, 1), return_sequences=True))
        # model_lstm.add(Dropout(dropout_rate))
        # model_lstm.add(LSTM(50, activation='relu'))
        model_lstm.add(LSTM(50, activation=activation, input_
            shape=(sequence_length, 1)))
        model_lstm.add(Dense(1))
        model_lstm.compile(optimizer=Adam(learning_rate=0.01),
            loss='mse')
        print(model_lstm.summary())

def forecast_gru(train_series, steps, sequence_length=
sequence_length, epochs=epochs):
    # Prepare training sequences
    data = train_series.values
    X, y = [], []
    for i in range(len(data) - sequence_length):
        X.append(data[i:i + sequence_length])
        y.append(data[i + sequence_length])
    X = np.array(X)
    y = np.array(y)
    X = X.reshape((X.shape[0], X.shape[1], 1)) # reshape
for GRU [samples, timesteps, features]
    build_gru(sequence_length)
    model_gru.fit(X, y, epochs=epochs, verbose=0)
    forecast = []

```

```

last_seq = data[-sequence_length:].reshape(1, sequence_
length, 1)
for _ in range(steps):
    pred = model_gru.predict(last_seq, verbose=True)
    forecast.append(pred[0, 0])
    last_seq = np.append(last_seq[:, 1:, :], pred.reshape
(1, 1, 1), axis=1)
return pd.Series(forecast)

def build_gru(sequence_length):
    global model_gru
    if not model_gru:
        model_gru = Sequential()
        model_gru.add(GRU(50, activation=activation, input_
shape=(sequence_length, 1)))
        model_gru.add(Dense(1))
        model_gru.compile(optimizer=Adam(learning_rate=0.01),
loss='mse')
        print(model_gru.summary())
        return model_gru

def calculate_prices(strategy, forecast, scaler, last_known_
price, retratio_factor, ind):
    if forecast.isna().any() or forecast.isnull().any():
        forecast=pd.Series([PENALTY]*len(forecast), index=for
ecast.index)
    forecast.index=ind
    if strategy==RAW:
        return(forecast)
    if strategy==SCALER or strategy==SCALER_RETRATIOS:
        forecast = pd.Series(scaler.inverse_transform(f.values.
reshape(-1, 1))[:, 0])
        if strategy == SCALER:
            return(forecast)
        predicted_prices_from_returns = np.repeat(last_known_
price, len(forecast)) * np.cumprod(1 + (forecast.values/
retratio_factor))
        predicted_prices_from_returns = pd.Series(predicted_
prices_from_returns, index=forecast.index)
        if predicted_prices_from_returns.isna().any() or
predicted_prices_from_returns.isnull().any():
            predicted_prices_from_returns=pd.Series([PENALTY]*len
(predicted_prices_from_returns), index=ind)
        return(predicted_prices_from_returns)
tickers = ["AAPL", "GOOGL", "MSFT", "AMZN", "TSLA", "NFLX",
"NVDA", "JPM", "BAC", "INTC"]
original_stdout = sys.stdout
subdirectory = "Images"
os.makedirs(subdirectory, exist_ok=True)
model_gru = None

```

```

model_lstm = None
random.seed(25032005)
steps = 7
metrics_summary = []
stock_cache={}

lengths=[50,100,200,300,500,1000]
for length_of_history in reversed(lengths):
    for phase in range(0,2):
        for ticker in tickers:
            original_stdout.write(f"Processing:  {ticker}
                                   {length_of_history} {phase} \n")
            if ticker in stock_cache.keys():
                data =stock_cache[ticker]
            else:
                data=None
                while data is None or data.empty():
                    data = yf.download(ticker, start='2020-01-01',
                                         end='2025-02-15')
                stock_cache[ticker]=data
            data=data.dropna()
            data[VALUE_COLUMN] = data[PRICE_COLUMN]#.pct_
                change()*100
            data_raw=data.copy()
            data_scaler=data.copy()
            data_retrratios=data.copy()
            last_known_price=data[PRICE_COLUMN].iloc[-(steps
                + 1),0]
            y_true = data[PRICE_COLUMN].iloc[-steps:,0]
            data=None
            data_train_raw = data_raw.iloc[-(length_of_history
                + steps):-steps]
            data_test_raw = data_raw.iloc[-steps:]
            data_retrratios[VALUE_COLUMN]= data_retrratios
[VALUE_COLUMN].pct_change()
            data_train_retrratios = data_retrratios.iloc
                [-(length_of_history + steps):-steps]
            data_test_retrratios = data_retrratios.iloc[-steps:]
            data_train_scaler = data_scaler.iloc[-(length_of_
                history + steps):-steps]
            data_test_scaler = data_scaler.iloc[-steps:]

            train_scaler=MinMaxScaler()
            data_train_scaler = pd.DataFrame(train_scaler.
fit_transform(data_train_scaler.iloc[:, [-1]]), columns=
[VALUE_COLUMN])
            test_scaler=MinMaxScaler()
            data_test_scaler = pd.DataFrame(test_scaler.
fit_transform(data_test_scaler.iloc[:, [-1]]), columns=
[VALUE_COLUMN])

```

```

data_test_scaler=data_test_scaler[VALUE_COLUMN]
data_train_retrratios=data_train_retrratios[VALUE_
COLUMN]
data_test_retrratios=data_test_retrratios[VALUE_
COLUMN]
data_train_raw=data_train_raw[VALUE_COLUMN]
data_test_raw=data_test_raw[VALUE_COLUMN]
if phase == 0:
    if length_of_history == 1000:
        f = forecast_gru(data_train_scaler, steps)
        f = forecast_lstm(data_train_scaler, steps)
    else:
        f = forecast_arma(data_train_retrratios, steps)
        arma_forecast = calculate_prices(RETRATIOS,
f, test_scaler, last_known_price, 1, pd.DatetimeIndex(data_
test_retrratios.index).to_period('D').to_timestamp())
        f = forecast_holtwinters(data_train_raw, steps)
        hw_forecast =calculate_prices(RAW, f, test_
scaler, last_known_price, 1, pd.DatetimeIndex(data_test_raw.
index).to_period('D').to_timestamp())
        f = forecast_garch(data_train_retrratios, steps)
        garch_forecast =calculate_prices(RETRATIOS,
f, test_scaler, last_known_price, 1, pd.DatetimeIndex(data_
test_retrratios.index).to_period('D').to_timestamp())
        f = forecast_brownian(data_train_raw, steps)
        bm_forecast=calculate_prices(RAW, f, test_
scaler, last_known_price, 1, pd.DatetimeIndex(data_test_raw.
index).to_period('D').to_timestamp())
        f=forecast_gru(data_train_scaler, steps)
        gru_forecast =calculate_prices(SCALER, f,
test_scaler, last_known_price, 1, pd.DatetimeIndex(data_
test_scaler.index).to_period('D').to_timestamp())
        f=forecast_lstm(data_train_scaler, steps)
        lstm_forecast =calculate_prices(SCALER, f,
test_scaler, last_known_price, 1, pd.DatetimeIndex(data_
test_scaler.index).to_period('D').to_timestamp())
        if phase==1:
            temp_file = open(f'Images\\results_
{sequence_length}_{epochs}_{activation}.txt', 'w',
encoding="utf-8")
            sys.stdout = temp_file

mods = {
    'ARIMA': arma_forecast,
    'HoltWinters': hw_forecast,
    'GARCH': garch_forecast,
    'Brownian': bm_forecast,

```

```

        'GRU': gru_forecast,
        'LSTM': lstm_forecast
    }
    fig, axs = plt.subplots(nrows=3, ncols=2,
                           figsize=(24, 18))
    idx=0
    for model_name, forecast in mods.items():
        y_pred = forecast.values
        rmse = compute_rmse(y_true, y_pred)
        mae = compute_mae(y_true, y_pred)
        mape = compute_mape(y_true, y_pred)
        metrics_summary.append({
            'Ticker': ticker,
            'History_length': length_of_history,
            'Model': model_name,
            'RMSE': rmse,
            'MAE': mae,
            'MAPE': mape
        })
        ax = axs[idx%3][idx//3]
        idx+=1
        ax.plot(data_raw[-
(min(lengths[2],length_of_history) + steps):].index, data_
raw[-(min(lengths[2],length_of_history) + steps):][
    PRICE_COLUMN], label='Historical
    Prices',
    color='blue')
        ax.plot(data_raw[-steps:].index, forecast.
values, label=f'{model_name} Forecast Prices',
    color='red', marker='o', linestyle
    ='--')
        ax.set_title(f'{ticker} values Historical
    {length_of_history} vs. {model_name}
    Forecast (Last {steps} Sessions)')
        ax.set_xlabel("Date")
        ax.set_ylabel("Price")
        ax.legend()
        ax.grid(True)

    plt.tight_layout()
    plt.savefig(f"Images\\iterative_{length_of_
history}.jpg", bbox_inches='tight')
    plt.close()
    # plt.show()

metrics_df = pd.DataFrame(metrics_summary)
print("\nEvaluation Metrics Summary:")
print(metrics_df)
overall_avg = metrics_df.groupby('Model')[['RMSE', 'MAE',
    'MAPE']].mean().reset_index()

```

```

print(overall_avg)
fig, ax = plt.subplots(2, 2, figsize=(12, 12))
for i, metric in enumerate(['RMSE', 'MAE', 'MAPE']):
    ax[i//2,i%2].bar(overall_avg['Model'], overall_avg[metric])
    ax[i//2,i%2].set_title(f"Average {metric}")
    ax[i//2,i%2].set_ylabel(metric)
plt.tight_layout()
plt.savefig("Images\\overall_average_comparison.png")
plt.close()

```

```

for length in lengths:
    length_n_avg = metrics_df.query(f"`History_length`
=={length}").groupby('Model')[['RMSE', 'MAE', 'MAPE']].
mean().reset_index()
    print(f"results for history length {length}:")
    print(length_n_avg)
    fig, ax = plt.subplots(2, 2, figsize=(12, 12))
    for i, metric in enumerate(['RMSE', 'MAE', 'MAPE']):
        ax[i//2,i%2].bar(length_n_avg['Model'], length_n_
            avg[metric])
        ax[i//2,i%2].set_title(f"Average {metric}")
        ax[i//2,i%2].set_ylabel(metric)
    fig.suptitle(f"Comparison forecast metrics for History
        Length {length} ")
    plt.tight_layout()
    plt.savefig(f"Images\\iterative_length_{length}_average_
        comparison.jpg")
    plt.close()

```

```

for ticker in tickers:
    ticker_avg = metrics_df.query(f"`Ticker`=='{tic
ker}")
    .groupby('Model')[['RMSE', 'MAE', 'MAPE']].mean().
    reset_index()
    print(f"results for enterprise {ticker}:")
    print(ticker_avg)
    fig, ax = plt.subplots(2, 2, figsize=(12, 12))
    for i, metric in enumerate(['RMSE', 'MAE', 'MAPE']):
        ax[i//2,i%2].bar(ticker_avg['Model'], ticker_avg
            [metric])
        ax[i//2,i%2].set_title(f"Average {metric}")
        ax[i//2,i%2].set_ylabel(metric)
    fig.suptitle(f"Comparison of forecast metrics for Enterprise
        {ticker} ")
    plt.tight_layout()
    plt.savefig(f"Images\\iterative_ticker_{ticker}_average_
        comparison.jpg")
    plt.close()
temp_file.close()
sys.stdout = original_stdout

```

The results of analysis are presented on [Figures 5.6–5.9](#).

On average, the GRU model demonstrates the lowest overall MAPE (2.94%), outperforming other approaches in capturing short-term dynamics, followed by GARCH and ARIMA. Brownian motion, a naïve benchmark, yields extremely high errors, indicating its unsuitability for stock price forecasting tasks in this context.

Evaluation Metrics Summary:

	Ticker	History_length	Model	RMSE	MAE	MAPE
0	AAPL	1000	ARIMA	6.117813	4.914656	2.070109
1	AAPL	1000	HoltWinters	6.315889	5.015965	2.110405
2	AAPL	1000	GARCH	5.909398	4.808476	2.027957
3	AAPL	1000	Brownian	139.401561	129.935843	54.934887
4	AAPL	1000	GRU	8.939226	7.555341	3.269338
..	...	...	...	...	...	...
355	INTC	50	HoltWinters	2.962005	2.238659	9.770191
356	INTC	50	GARCH	2.839885	2.151835	9.399945
357	INTC	50	Brownian	13.701603	12.473373	56.617128
358	INTC	50	GRU	2.472297	1.862164	8.137780
359	INTC	50	LSTM	2.351416	1.830175	8.057886

[360 rows x 6 columns]

	Model	RMSE	MAE	MAPE
0	ARIMA	8.610607	7.521266	3.651690
1	Brownian	170.137299	159.767792	54.895465
2	GARCH	8.801048	7.686067	3.601557
3	GRU	8.539157	7.324647	2.938004
4	HoltWinters	8.933034	7.756900	3.707922
5	LSTM	61.413933	33.333706	5.575125

results for history length 50:

	Model	RMSE	MAE	MAPE
0	ARIMA	8.183090	7.079654	3.697613
1	Brownian	170.137299	159.767792	54.895465
2	GARCH	9.400466	8.259406	3.995609
3	GRU	8.295416	7.102966	2.690850
4	HoltWinters	9.536237	8.310556	4.116952
5	LSTM	323.561309	160.892784	17.949807

results for history length 100:

	Model	RMSE	MAE	MAPE
0	ARIMA	9.040296	7.948821	3.803060
1	Brownian	170.137299	159.767792	54.895465
2	GARCH	8.854043	7.787722	3.697903
3	GRU	8.086304	6.890907	2.735583
4	HoltWinters	8.851219	7.723593	3.717802
5	LSTM	9.746176	8.817355	3.142121

Figure 5.6 Stock values forecasting performance comparison. (Continued)

Source: Own research.

results for history length 200:

	Model	RMSE	MAE	MAPE
0	ARIMA	8.949331	7.886594	3.816810
1	Brownian	170.137299	159.767792	54.895465
2	GARCH	8.969723	7.879934	3.584821
3	GRU	8.789934	7.616198	2.991694
4	HoltWinters	8.818615	7.709357	3.678982
5	LSTM	8.452223	7.221535	2.945255

results for history length 300:

	Model	RMSE	MAE	MAPE
0	ARIMA	8.654093	7.602138	3.662255
1	Brownian	170.137299	159.767792	54.895465
2	GARCH	8.263864	7.172725	3.450498
3	GRU	8.456247	7.243022	3.025150
4	HoltWinters	8.773658	7.583755	3.635619
5	LSTM	8.647521	7.433213	3.036376

results for history length 500:

	Model	RMSE	MAE	MAPE
0	ARIMA	8.515707	7.398302	3.494828
1	Brownian	170.137299	159.767792	54.895465
2	GARCH	8.530757	7.415923	3.406162
3	GRU	8.818314	7.606788	3.038781
4	HoltWinters	8.958975	7.712842	3.548859
5	LSTM	8.966886	7.751017	3.174575

results for history length 1000:

	Model	RMSE	MAE	MAPE
0	ARIMA	8.321127	7.212087	3.435577
1	Brownian	170.137299	159.767792	54.895465
2	GARCH	8.787438	7.600690	3.474350
3	GRU	8.788726	7.487999	3.145967
4	HoltWinters	8.659503	7.501296	3.549318
5	LSTM	9.109483	7.886332	3.202615

results for enterprise AAPL:

	Model	RMSE	MAE	MAPE
0	ARIMA	6.245335	4.979155	2.096377
1	Brownian	139.401561	129.935843	54.934887
2	GARCH	5.833030	4.761935	2.009383
3	GRU	7.681588	6.499231	2.784445
4	HoltWinters	6.162444	4.939486	2.080140
5	LSTM	7.932457	6.715103	2.877507

results for enterprise GOOGL:

	Model	RMSE	MAE	MAPE
0	ARIMA	6.930767	6.400380	3.458372
1	Brownian	106.031434	99.726463	53.781186
2	GARCH	6.678176	6.159873	3.328456
3	GRU	5.211189	4.827836	2.606936
4	HoltWinters	7.273620	6.749085	3.646142
5	LSTM	4.864706	4.562356	2.463112

...

Figure 5.6 (Continued)

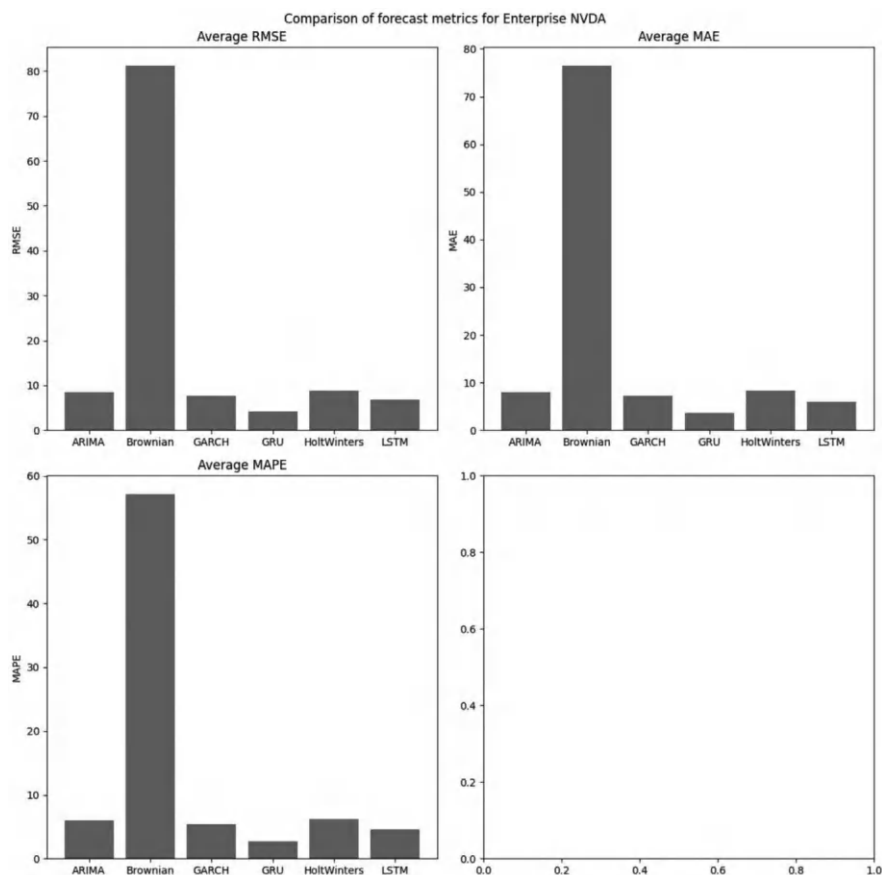


Figure 5.7 Stock values forecasting performance comparison for NVIDIA.

Source: Own research.

Performance varies with the length of historical data used for training. For a 50-step history, GRU significantly outperforms other models, achieving a MAPE of 2.69%, while the LSTM model performs poorly due to high RMSE and instability. However, as the training history increases to 100–300 steps, LSTM stabilizes and begins to match GRU's performance. Beyond 500 steps, traditional models such as ARIMA and GARCH consistently deliver competitive results, suggesting that classical time series models benefit from larger datasets and capture long-term structures effectively. Holt-Winters remains stable but never dominates, while Brownian motion remains an underperforming baseline across all horizons.

When forecasting specific assets, the results indicate substantial heterogeneity in model performance. For Apple, GARCH achieves the best accuracy with a MAPE of 2.01%, marginally outperforming ARIMA and Holt-Winters, while neural network models (LSTM and GRU) fall behind, possibly due to overfitting

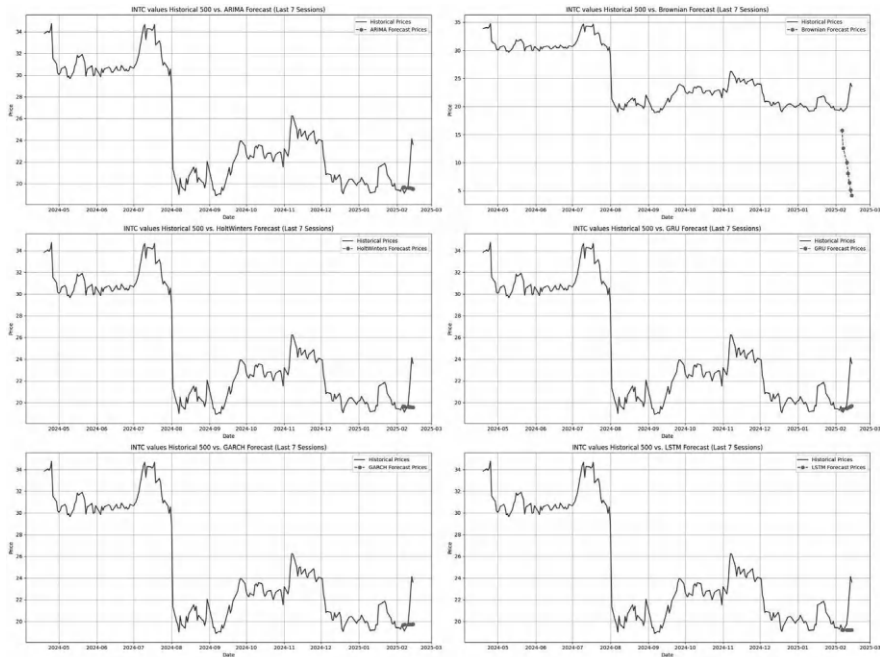


Figure 5.8 Stock values forecasting for 500-day history models.

Source: Own research.

or sensitivity to recent volatility. Conversely, for Google, deep learning methods excel, with LSTM and GRU achieving the lowest errors, indicating their strength in capturing nonlinear and short-term fluctuations that may be present in the price dynamics of more volatile or less mean-reverting assets.

In summary, GRU emerges as the most consistent performer across average conditions, combining low MAPE and stable MAE/RMSE profiles, particularly for shorter historical windows. LSTM requires longer histories to stabilize, but eventually approaches or surpasses traditional models in certain scenarios. ARIMA and GARCH remain strong classical baselines, especially as the length of historical data increases.

This kind of simulation may be repeated in various scenarios, with changed enterprises which stock values are taken into account or with different time range, but also with:

- Different number of layers of recurrent networks;
- Different items in history_length used for training;
- Different activation function for each layer;
- Resetting the models after each training (training the models each time from scratch);
- With constant history length, apply variant forecasting step length;
- With constant history length and steps length, apply variant sequence lengths in models;

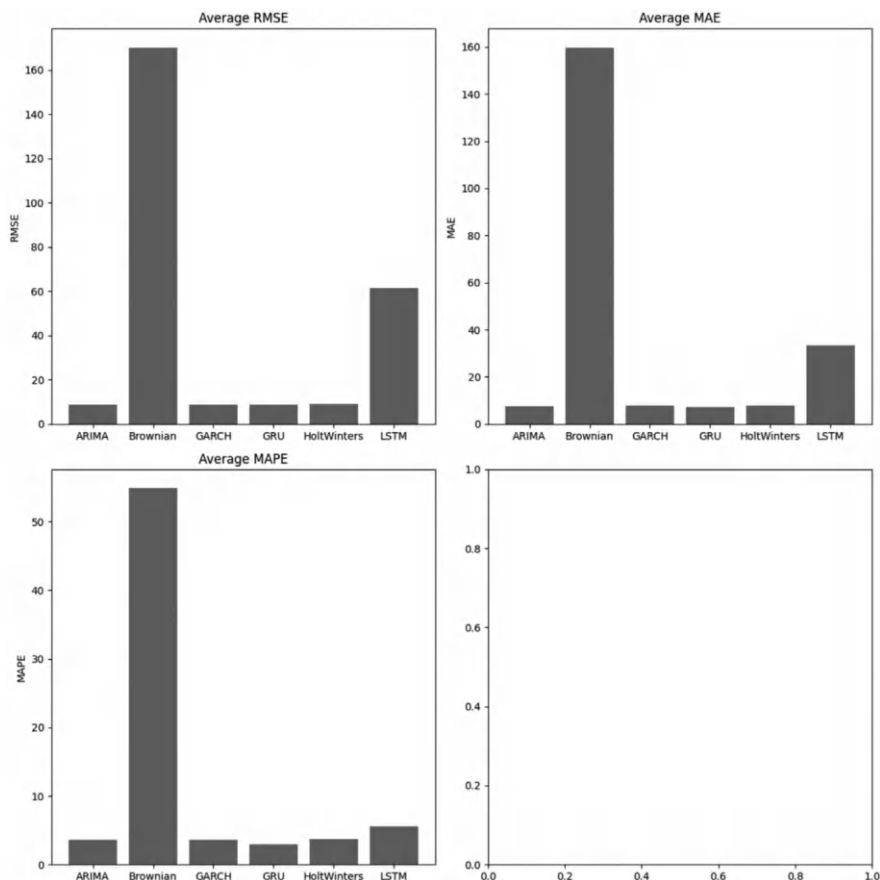


Figure 5.9 Stock values forecasting performance comparison (bar plots for root mean squared error (RMSE), mean absolute error (MAE) and mean absolute percentage error (MAPE)).

Source: Own research.

- Compare various normalization methods in data preprocessing phase.

The implementations of the above scenarios may be found in the GitHub repository for this book.

5.3 Research application example: application of CNN for stock price charts in prediction of the direction of growth

The last research study predicting stock price movements (growth or decrease) using historical stock data represented as candlestick charts over specific time periods (e.g., 60, 30, 15, and 7 days). The goal is to classify whether the stock value will increase (1) or decrease (0) based on patterns in the candlestick data/images. The following assumptions have been made in experiment scenario design phase:

Candlestick charts (cf. [Figure 5.10](#)) are used to represent historical stock price movements, with each “candle” showing the open, high, low, and close prices for a specific time interval (e.g., daily).

The input data consists of sequences of candlestick charts over fixed periods (e.g., 20, 10, and 5 days), capturing trends and patterns in price movements.

The study examines multiple time windows (e.g., 20, 10, and 5 days) to analyze how different historical horizons impact prediction accuracy.

The dependent variable is a binary label: 1 (growth): the stock price increases over a defined future period (e.g., the next day or week); or 0 (decrease): the stock price decreases or remains unchanged over the same period.

Historical stock data is converted into candlestick charts for the specified time periods.

The data normalization step to ensure consistency across different stocks and time frames is omitted because the scaling is included in candlestick chart creation.

The binary labels (growth or decrease) are generated by comparing future prices with the current price.

No images are saved, convolutional model training is processed directly in memory. (Please note, that this may be a limitation for low-capacity GPUs – for example, the author encountered memory problems on MX 150 chipset.)



Figure 5.10 Sample candlestick chart used for convolutional network stock price movement prediction model.

Source: Own research.

The time range of the study is 15 March 2023 to 15 March 2025. The last value in each time series is forecasted and then compared with the actual one.

The first 50 from <http://www.nasdaqtrader.com/dynamic/SymDir/nasdaqlisted.txt> stock tickers are taken into account. Actions for which it was not possible to complete the data were omitted.

The implementation of stock movement procedure is included in the code below:

```
import sys
import yfinance as yf
import pandas as pd
import mplfinance as mpf
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D,
Flatten, Dense
import numpy as np
from PIL import Image
import io
import pickle
import os
import requests
import random
from datetime import datetime
import pickle
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score, confusion_matrix
from tensorflow.keras.callbacks import EarlyStopping
import time

analysis_window_lengths=[20,10,5]
# analysis_window_lengths=[20]
min_series_len=10
# analysis_window_lengths=[60,30,15,7]
EPOCHS=500
FOLDER = "data"
DOWNLOAD_DATA=True

def download_data(ticker)-> pd.DataFrame:
    try:
        ticker = yf.Ticker(ticker)
        stock_data = ticker.history(period='max')
    except:
        return pd.DataFrame()
    try:
        stock_data = stock_data.iloc[stock_data.index.tz_
convert(None) > datetime.strptime('2023-03-15', '%Y-%m-%d'), :]
```

```

        stock_data = stock_data.iloc[stock_data.index.tz_
convert(None) <= datetime.strptime('2025-03-15', '%Y-%m-%d'), :]
    except:
        pass
    return stock_data

def get_stock_image(stock_data,i,ticker):
    # print(f"{ticker} {i}")
    # fig, ax = mpf.plot(stock_data, type='candle', style='yahoo',
    #                     returnfig=True, update_width_
config=dict(volume_width=0.1), volume=True)

    # Download sample data for a ticker (e.g., AAPL)

    # Create a custom minimal style with no background,
grid, or labels
'''
    minimal_style = mpf.make_mpf_style(
        base_mpf_style='default',
        rc={
            'axes.grid': False, # No gridlines
            'axes.facecolor': 'none', # Transparent axes
background
            'figure.facecolor': 'black', # Transparent figure
background
            'xtick.color': 'none', # No x-axis tick labels
(colors set to none)
            'ytick.color': 'none', # No y-axis tick labels
            'axes.edgecolor': 'none' # No border around axes
        }
    )
'''
    # Define the style
    custom_style = mpf.make_mpf_style(
        base_mpf_style='charles', # start from a clean base
        rc={
            'axes.edgecolor': 'none',
            'axes.facecolor': 'black',
            'figure.facecolor': 'black',
            'xtick.color': 'black',
            'ytick.color': 'black',
            'grid.color': 'black'
        },
        marketcolors=mpf.make_marketcolors(
            up='green', down='red',
            edge='inherit', wick='inherit',
            volume='inherit'
        )
    )

```

```

# Plot the chart with no axes, grid, or labels
mpf.plot(
    stock_data,
    type='candle',
    style=custom_style,
    axisoff=True,
    ylabel='',
    ylabel_lower='',
    tight_layout=True, volume=True, returnfig=True
)

memory_buffer = io.BytesIO()
plt.savefig(memory_buffer, format='png', bbox_inches='
tight', pad_inches=0, transparent=False)
plt.close()
return memory_buffer

def prepare_images(stock_data, analysis_window_length, last,
ticker) ->list:
    images=[get_stock_image(stock_data.iloc[i:i + anal-
ysis_window_length,:],i,ticker) for i in range(0,
len(stock_data)-last-1)]
    return images

def preprocess_images(memory_buffers):
    images = []
    for buffer in memory_buffers:
        buffer.seek(0)
        image = Image.open(buffer)

        # Convert RGBA to RGB if necessary
        if image.mode == 'RGBA':
            image = image.convert('RGB')
        image = image.resize((256,256)) # Resize to a
standard size
        image = np.array(image,dtype=np.float32) # Convert
to a numpy array
        images.append(image)
        buffer.close()

    return images

random.seed(25032005)
tf.keras.utils.set_random_seed(25032005)

def create_model():
    model = Sequential([
        Conv2D(16, (3, 3), activation='relu', input_
shape=(256, 256, 3)),

```



```

        MaxPooling2D(),
        Conv2D(32, (3, 3), activation='relu'),
        MaxPooling2D(),
        Conv2D(64, (3, 3), activation='relu'),
        MaxPooling2D(),
        Conv2D(32, (5, 5), activation='relu'),
        MaxPooling2D(),
        Conv2D(16, (10, 10), activation='relu'),
        MaxPooling2D(),
        Conv2D(8, (4, 4), activation='relu',padding="same"),
        MaxPooling2D(),
        Flatten(),
        Dense(8, activation='relu'),
        Dense(2, activation="softmax")
    ])
    model.compile(optimizer='Adam', loss="sparse_categorical_crossentropy", metrics=['accuracy'])
    return model

all_models={awl:create_model() for awl in analysis_window_lengths}

early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=10,
    restore_best_weights=True
)
# URL for Nasdaq-listed tickers
url = "http://www.nasdaqtrader.com/dynamic/SymDir/nasdaqlisted.txt"

# Create the FOLDER "data" if it does not exist
os.makedirs(FOLDER, exist_ok=True)

# Define the local file path
filepath = os.path.join(FOLDER, "nasdaqlisted.txt")

response = requests.get(url)
if response.status_code == 200:
    with open(filepath, "w", encoding="utf-8") as file:
        file.write(response.text)
    print(f"File downloaded and saved to {filepath}")
else:
    print("Error downloading file:", response.status_code)

df = pd.read_csv(filepath, sep="|", engine="python",
skipfooter=1)
print(df.shape)
print(df.head())
df=df.iloc[-25:]

```

```

# X_all_images={awl:np.empty(shape=[0,128,128,3]) for awl in
analysis_window_lengths}
y_test=[]
x_test=[]
start_time = time.time() # Start timing
for iter,ticker in enumerate(df.Symbol):
    history_data=download_data(ticker)
    if len(history_data) >
analysis_window_lengths[0]+min_series_len:
        to_test_x={analysis_window_length:None for analysis_
window_length in analysis_window_lengths}
        for analysis_window_length in analysis_window_lengths:
            print(f"***** {ticker} {iter}
{analysis_window_length}*****")
            Y = history_data['Close'].pct_change()[analysis_
window_lengths[0]+1:]
            X = prepare_images(history_data,analysis_win-
dow_length,analysis_window_lengths[0],ticker)[:~1] # we dont
know last one
            end_time = time.time() # End timing
            print( f"?? Time taken before {ticker} - window
{analysis_window_length}: {end_time - start_time:.2f} seconds")
            start_time = time.time() # Start timing
            X_images = preprocess_images(X)
            X_images = [X_image / 255.0 for X_image in X_
images] # Normalize pixel values
            # X_images = [np.where(X_image < 0, 0, X_image * 10)
for X_image in X_images ] # Normalize pixel values
            all_models[analysis_window_length].fit(np.
array(X_images[:~1]), np.array([1 if y >=0 else 0 for y in
Y[:~1]]),
                                                    epochs
=EPOCHS,validation_split=0.2, batch_size=32, callbacks=
[early_stopping])
            to_test_x[analysis_window_length]=X_images[~1]
            if analysis_window_lengths[~1]==analysis_win-
dow_length:
                y_test.append(1 if Y.iloc[~1]>=0 else 0)
                x_test.append(to_test_x)

y_pred=np.zeros(len(y_test))
r_pred=np.zeros((len(y_test),2))
all_p={}
all_r={}
for analysis_window_length in analysis_window_lengths:
    print(f" prediction: {analysis_window_length}:")
    r=all_models[analysis_window_length].predict(np.array
([x[analysis_window_length] for x in x_test]), batch_size=32)
    r_pred+=r
    print(r)

```

```

# p=np.array(r[:,0])
p=np.argmax(r,axis=1)
print(p)
all_p[analysis_window_length]=p
all_r[analysis_window_length]=r
temp_file=open('results.txt', 'w',encoding="utf-8")
orig_out=sys.stdout
sys.stdout=temp_file
print(y_pred)
print(y_test)
print(accuracy_score(y_test, y_pred))
print(precision_score(y_test, y_pred))
print(recall_score(y_test, y_pred))
print(f1_score(y_test, y_pred))
print(roc_auc_score(y_test, y_pred))
temp_file.close()
sys.stdout=orig_out

```

The results of stock movement prediction are presented in [Figure 5.11](#).

The deep CNN model developed for stock movement prediction demonstrates promising classification performance, particularly in identifying upward trends. The model achieves an accuracy score of 0.737, indicating that approximately 74% of stock movement predictions align with the actual direction. Most notably, it attains a recall score of 1.0, meaning the model correctly identifies all actual positive (upward movement) cases. This high recall comes at the cost of precision (0.667), suggesting a moderate rate of false positives – some predicted increases are not realized.

The model's F1 score of 0.8 reflects a favorable balance between precision and recall, making it particularly useful in high-recall applications where missing positive trends is costly. Furthermore, the Receiver Operating Characteristic Area Under the Curve (ROC AUC) score of 0.722 indicates good discrimination capability between upward and downward movements. These results suggest that the CNN is well suited for scenarios where capturing all potential gains is prioritized, albeit with a trade-off in prediction specificity. The refinements listed below could aim to boost precision without compromising recall, improving reliability for practical financial decision-making.

```

(y_pred) [0 1 1 0 0 1 1 1 1 1 1 1 1 0 1 1 1 1]
(y_test) [0, 1, 1, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 1, 1]
(accuracy_score) 0.7368421052631579
(precision_score) 0.6666666666666666
(recall_score) 1.0
(f1_score) 0.8
(roc_auc_score) 0.7222222222222222

```

Figure 5.11 Stock price movement convolutional network model prediction results

Source: Own research.

The results show that the model can help support decisions regarding transactions on the stock market, but it should not be treated as an “omniscient oracle”. On the other hand, predicting stock values with 100% efficiency is virtually impossible due to a combination of inherent market characteristics, limitations of models, and the unpredictable nature of economic and human behavior. The study should, therefore, be treated as a “proof of concept”, and it can be repeated for other assets, other dates, and in different scenarios:

- Increasing the complexity of the convolutional network model (with a risk of overfitting);
- Forecasting only based on historical data of the audited company and not on the model for all those included in the study (with a risk of underfitting);
- Segmentation of the market into industries and creation of separate models for each industry;
- Segmentation of the market into segments related to volume and creation of separate models for different volume ranges.

The author will leave these analyses as open issues, encouraging readers to conduct their own experiments.

References

- Bai, S., Kolter, J. Z., & Koltun, V. (2018). An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*.
- Bhansali, R. J. (1997). Direct autoregressive predictors for multistep prediction: Order selection and performance relative to the plug in predictors. *Statistica Sinica*, 7(2), 425–449.
- Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31(3), 307–327.
- Box, G. E. P., & Jenkins, G. M. (1970). *Time series analysis: Forecasting and control*. Holden-Day.
- Brockwell, P. J., & Davis, R. A. (2016). *Introduction to time series and forecasting* (3rd ed.). Springer.
- Case, K. E., & Shiller, R. J. (1989). The efficiency of the market for single-family homes. *The American Economic Review*, 79(1), 125–137.
- Cleveland, R. B., Cleveland, W. S., McRae, J. E., & Terpenning, I. (1990). STL: A seasonal-trend decomposition procedure based on Loess. *Journal of Official Statistics*, 6(1), 3–73.
- Dickey, D. A., & Fuller, W. A. (1979). Distribution of the estimators for autoregressive time series with a unit root. *Journal of the American Statistical Association*, 74(366), 427–431. <https://doi.org/10.2307/2286348>
- Einstein, A. (1905). On the movement of small particles suspended in stationary liquids required by the molecular-kinetic theory of heat. *Annalen der Physik*, 17, 549–560.
- Elman, J. L. (1990). Finding structure in time. *Cognitive Science*, 14(2), 179–211.
- Engle, R. F. (1982). Autoregressive conditional heteroskedasticity with estimates of the variance of UK inflation. *Econometrica*, 50(4), 987–1007.
- Fama, E. F. (1965). The behavior of stock-market prices. *Journal of Business*, 38(1), 34–105.
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.
- Hodrick, R. J., & Prescott, E. C. (1997). Postwar US business cycles: An empirical investigation. *Journal of Money, Credit and Banking*, 29(1), 1–16.

- Holt, C. C. (1957). Forecasting trends and seasonals by exponentially weighted moving averages. *O.N.R. Memorandum*.
- Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and practice* (2nd ed.). OTexts.
- Jiang, L., & Yu, J. (2014). *A new hedonic regression for real estate prices applied to the Singapore residential market* (Cowles Foundation Discussion Paper No. 1969). Cowles Foundation for Research in Economics, Yale University.
- Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Transactions of the ASME—Journal of Basic Engineering*, 82(Series D), 35–45.
- Kunitomo, N., & Yamamoto, T. (1985). Properties of predictors in misspecified autoregressive time series models. *Journal of the American Statistical Association*, 80(392), 941–950.
- Lim, B. (2020). *Deep learning for time series prediction and decision making over time* (Doctoral dissertation, University of Oxford).
- McKibbin, W. J., & Fernando, R. (2020). The global macroeconomic impacts of COVID-19: Seven scenarios. *Brookings Institution Working Paper*.
- Nichiforov, C., Stamatescu, I., Făgărășan, I., & Stamatescu, G. (2017, October). Energy consumption forecasting using ARIMA and neural network models. In *2017 5th International Symposium on Electrical and Electronics Engineering (ISEEE)* (pp. 1–4). IEEE.
- Nowak, B., & Bartnicki, G. (2019). Forecasting the time interval of the day with the maximum boilers gas consumption. *Zeszyty Naukowe Instytutu Gospodarki Surowcami Mineralnymi Polskiej Akademii Nauk*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008. <https://doi.org/10.48550/arXiv.1706.03762>
- Wiener, N. (1923). Differential space. *Journal of Mathematics and Physics*, 2(1), 131–174.
- Winters, P. R. (1960). Forecasting sales by exponentially weighted moving averages. *Management Science*, 6(3), 324–342.
- Yu, F., & Koltun, V. (2016). Multi-scale context aggregation by dilated convolutions. In Y. Bengio & Y. LeCun (Eds.), *International Conference on Learning Representations (ICLR)*. <https://arxiv.org/abs/1511.07122>
- Zhou, H., Zhang, S., Peng, J., et al. (2021). Informer: Beyond efficient transformer for long sequence time-series forecasting. *AAAI Conference on Artificial Intelligence*, 35(12), 11106–11115.

6 Sentiment analysis and text mining with deep learning models

6.1 Sentiment analysis fundamentals

Sentiment analysis is a field at the intersection of natural language processing (NLP), data science, biometrics, and deep learning dealing with the identification, extraction, and quantification of feelings contained in (usually short) fragments of text. These are often opinions about purchases, the level of satisfaction with the service, and movie or concert reviews. Hence, its initial name was opinion mining. Text mining is a broader term meaning of the process of transforming unstructured textual data into structured, meaningful insights. Besides sentiment analysis, it typically involves techniques such as tokenization and named entity recognition, enabling organizations to uncover patterns, trends, and relationships hidden within large volumes of text.

In its simplest form, sentiment analysis assigns a category of “positive” or “negative” to the analyzed opinion. The sentence, “Great product for a very good price” should be assigned the category “positive” and the sentence, “The device did not work properly from the moment of purchase, despite the very high price” should be assigned to “negative”.

The task seems quite simple, but there are many problems to keep in mind that make the problem of sentiment attribution not a trivial “linear” issue.

The same word can have different emotional overtones: “The product is cheap and reliable” vs “The materials feel cheap and flimsy”.

A sentence containing sentiment words may not express any sentiment: “The product is great, but I haven’t tried it yet”, “This terrible weather is perfect for staying indoors and relaxing”. Sentences may be sarcastic, making sentiment words hard to deal with: “Oh, great! Another delay in my flight. Just what I needed”, “What a fantastic idea to have a picnic in the rain”.

Sentences may be sarcastic even without sentiment words: “Yeah, because standing in this long line is exactly how I wanted to spend my day.”, “Sure, missing my deadline was totally part of the plan”. (No sentiment words, but the sarcasm implies disappointment or annoyance.)

There are a lot of sentences without sentiment words that finally express sentiment: “I wouldn’t recommend this restaurant to my friends”, “The phone barely lasted a day on a full charge”.

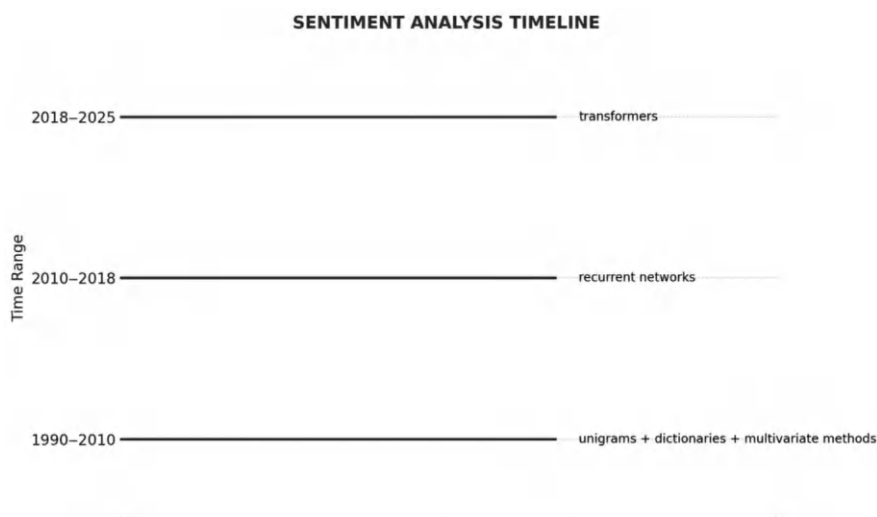


Figure 6.1 Sentiment analysis timeline.

Source: Own work.

Opinions may be double negated: “I don’t dislike this book at all”.

The sentences with the same set of words but in a different order may include opposite sentiments: “What a disaster, not a single moment of joy” vs “Not a disaster, what a single moment of joy”.

The same words may have different emotional loadings: “Maestro” vs “Maestro (of disasters)”.

The development of sentiment analysis may be divided into three “epochs”: **unigrams and dictionaries**, for which context analysis was omitted or has a low impact on the result; **recurrent networks**, analyzing rather the sentences than words; and **transformers** with attention used for extended context analysis. This timeline is presented in [Figure 6.1](#), and in [Section 6.7](#) the representatives of each family are compared in the IMDB dataset.

6.2 Bag-of-words methods

The first attempts at sentiment analysis were related to unigrams and n-grams ([Shannon, 1948](#)). Unigrams are just the set of all words used in sentence/text fragments, while n-grams contain information about the frequency.¹ [Joachims \(1998\)](#) used support vector machines (SVMs) to classify text represented by vectors with Term Frequency-Inverse Document Frequency ([Robertson, 2004](#); [Sparck Jones, 1972](#)) measures on two datasets: Reuters Newswire 1987 (assigning to one of 135 categories) and Ohsumed (23 disease categories). [Cristianini and Shawe-Taylor \(2000\)](#) and [Pang et al. \(2002\)](#) used the Naïve Bayes method and SVMs to classify

sentiment. The first attempts used unigrams (equivalent to the popular bag-of-words name) to represent words in analyzed text fragments. The unigrams analysis is often empowered with dictionaries.

Later on, “usual” algorithms of machine learning were supported by features and mechanisms specific to text analysis dictionaries and part of speech (POS) analysis.

6.2.1 Dictionaries of positive and negative words

The idea of finding words with precise emotional meanings like “great” or “disaster” in the analyzed text is quite intuitive. So, to classify or to amplify the classification process, the measures of the number/proportions of “positive” and “negative” words may be used. The challenge in the early applications of this type was to create a large dictionary of positive and negative words. The scope was limited initially, having only a few simple dictionaries with predefined sentiment. The first approach used the WordNet (Miller et al., 1990) dictionary with a list of synonyms and antonyms. Manually collected seed dictionaries iteratively added words not included in positive or negative categories based on the current belongingness of the analyzed word synonym (or antonym). Such an approach was used by [Hu and Liu \(2004\)](#) and [Valitutti et al. \(2004\)](#). [Mohammad et al. \(2009\)](#) additionally added automatic antonym patterns (likely/unlikely, armed/disarmed, etc.). [Kamps et al. \(2004\)](#) proposed WordNet distance method to estimate the semantic similarity of a given adjective. Examples of using positive and negative dictionaries (in some sources called lexicon-based methods) may be found in [Taboada et al. \(2011\)](#), [Ding et al. \(2008\)](#), [Hu and Liu \(2004\)](#), [Kim and Hovy \(2004\)](#), and [Zagibalov and Carroll \(2008\)](#).

6.2.2 POS – The part of speech

Dictionaries analysis was often empowered by Part of Speech (*i.e.*, *nouns, pronouns, verbs, adjectives, adverbs, prepositions, conjunctions, interjections, etc.* tagging (see [Brill, 1992](#); [Church, 1989](#); [Santorini, 1990](#)))

Words in analyzed text fragments were tagged with their part of speech assignment, taking into account that some POS, like adjectives, adverbs, and interjections, may contain higher emotional loading than others. [Turney \(2002\)](#) composed POS tags into syntactic patterns in a three-step unsupervised learning algorithm.

6.3 Recurrent neural networks-based sentiment analysis

Around the beginning of the second decade of the 21st century, recurrent neural networks (RNNs) began to gain importance in the analysis of textual data. [Mikolov et al. \(2010\)](#) stated that,

“Recurrent neural networks outperformed significantly state of the art back-off models in all our experiments, most notably even in case when backoff models were trained on much more data.”

And, “Recurrent Neural Network Language Model is probably the simplest language model today and very likely also the most intelligent”. [Irsoy and Cardie \(2014\)](#) used the deep and shallow RNNs in both direct subjective expressions (DSEs) and expressive subjective expressions (ESEs) as defined in [Wiebe and Riloff \(2005\)](#) and draw conclusions that deep RNNs performs best for both DSE and ESE detection, achieving a new state-of-the-art performance for the stricter proportional overlap measure.

[Tang et al. \(2015\)](#) proposed a bottom-up neural network for document-level sentiment classification that first learns sentence representations via convolutional neural networks or long short-term memory, then adaptively encodes sentence semantics and relations with a gated recurrent neural network, achieving superior performance over state-of-the-art methods on large-scale IMDB and Yelp datasets while demonstrating the effectiveness of gated recurrent networks over standard recurrent networks.

6.4 Embeddings, *Transformer-based models*

Modern techniques of sentiment analysis with deep learning models are almost entirely based (among others) on embedding ideas.

Embeddings refer to learned vector representations of entities such as words, sentences, or other discrete units in a continuous vector space. By mapping high-dimensional or symbolic data into lower-dimensional numeric vectors, embeddings capture essential semantic or syntactic properties, enabling more efficient and meaningful computations in downstream tasks. Unlike one-hot vectors, which grow linearly with vocabulary size and fail to encode relationships, embeddings allow related items to reside in closer proximity, reflecting underlying linguistic or conceptual similarities ([Bengio et al., 2003](#)).

One of the earliest influential works in this domain is Word2Vec, introduced by [Mikolov et al. \(2013\)](#). Word2Vec models learn embeddings by predicting context words (the Skip-gram model) or by predicting a center word from its surrounding context (the continuous bag-of-words (CBOW) model). The Skip-gram variant, for example, takes a target word w_t and tries to predict words $\{w_{t-k}, \dots, w_{t-1}, w_{t+1}, \dots, w_{t+k}\}$ within a specified window k . Over many training iterations, words sharing similar contexts end up with vectors that lie close in the embedding space, capturing semantic relationships such as analogies (e.g., “king” - “man” + “woman” $\approx$ “queen”). These embeddings, typically 100–300 dimensions, are then used in tasks like sentiment analysis or named entity recognition, significantly improving performance over sparse, high-dimensional representations.

A related but distinct approach is GloVe ([Pennington et al., 2014](#)). While Word2Vec focuses on local context windows, GloVe is a global log-bilinear regression model that factorizes a word co-occurrence matrix. Specifically, GloVe constructs embeddings by minimizing a weighted least-squares objective:

$$J = \sum_{i,j} f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2 \quad (6.1)$$

where:

$X_{\{ij\}}$ is the co-occurrence count of word i with word j ,

$w_i^\top, \widetilde{w}_j$ are the word and context embeddings, respectively.

The function $f(X_{\{ij\}})$ suppresses the impact of large counts.

Beyond standalone embeddings like Word2Vec and GloVe, modern deep architectures often learn embeddings end-to-end. In transformer models (Vaswani et al., 2017), there is an “embedding layer” that transforms discrete tokens into continuous vectors of dimension $d_{\{model\}}$ (described in section 2.4.6.).

In real applications, embedding creation can follow different routes:

- **Pretrained static embeddings:** Word2Vec or GloVe are trained offline on large text corpora; the resulting vectors are used “as is” for downstream tasks. This method is straightforward, but does not adapt to new data distributions during finetuning.
- **End-to-end learned embeddings:** Neural architectures incorporate an embedding layer updated jointly with the main model. This produces representations tailored to the specific training objective and domain.
- **Hybrid approaches:** One may initialize a model with pre-trained embeddings (Word2Vec or GloVe) and then continue training them within a larger neural network, thus combining the benefits of large-scale unsupervised training and task-specific fine-tuning.

Overall, embeddings serve as a foundation in modern deep learning for NLP and beyond. They reduce data sparsity, preserve semantic relationships, and integrate seamlessly with advanced architectures. Whether derived from local context (Word2Vec), global co-occurrence statistics (GloVe), or end-to-end transformer layers, embeddings remain indispensable for capturing structure and meaning in high-dimensional data.

Modern attempts at sentiment analysis are based on attention transformers. The most eminent representant of this approach are “easiest available which make it most popular” BERT model (Devlin et al., 2019) and RoBERTa (Liu et al., 2019).

Zekaoui et al. (2023) examined the performance of BERT, GPT I,II,II, ALBERT, RoBERTa, XLNet, DistilBERT, XLM-RoBERT, BART, ConvBERT, Reformer, T5, ELECTRA, Longformer, DeBERTa in sentiment analysis task achieving for each model substantial progress to non-neural networks-based models and recurrent neural network-based model.

6.5 Hugging Face repository ecosystem

Hugging Face is an AI research organization and open-source platform specializing in NLP and deep learning model sharing (Wolf et al., 2020). It is best known for its transformers library, which provides pre-trained models such as BERT, GPT,

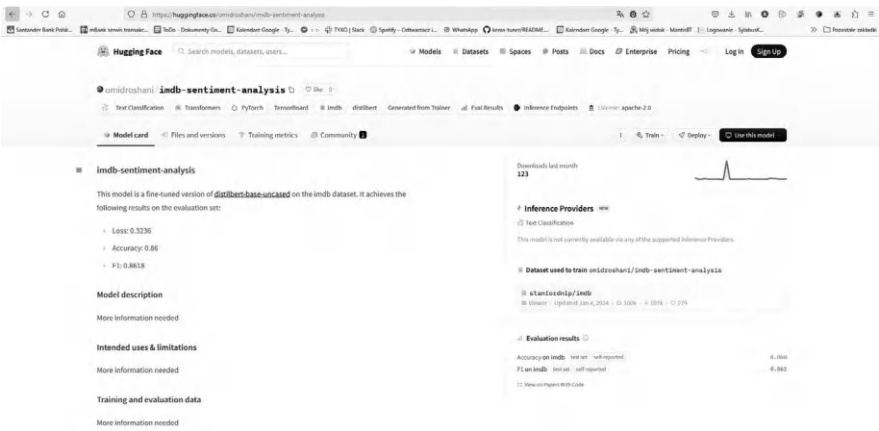


Figure 6.2 Hugging Face pipelines.

Source: <http://huggingface.co>.

and T5 for tasks like text classification, translation, and sentiment analysis. Hugging Face democratizes access to modern deep learning models by offering user-friendly application programming interfaces (APIs), model hubs, and cloud-based inference tools, enabling seamless model deployment. Beyond NLP, its ecosystem supports computer vision and reinforcement learning, making it a central hub for AI research and collaboration. Sample Hugging Face pipeline is presented in Figure 6.2.

One of the core concepts of Hugging Face and other pre-trained model collaboration sites' ecosystem is **pipeline**. It refers to a concept broader than a pre-trained model, providing the core predictions. At the same time, a pipeline wraps the model with preprocessing and postprocessing steps to offer a simple, task-specific interface. Pipeline identifies the particular task and specific model used to realize the task. The selected pipelines are presented in section 6.6

6.6 Introduction to natural language processing (NLP) and large language models (LLM)

NLP is a field of artificial intelligence that focuses on enabling computers to interpret, generate, and interact with human language. It involves algorithms and models designed to understand the structure and meaning of text or speech, thereby facilitating tasks such as translation, summarization, and sentiment analysis.

LLMs (see, e.g., Alammari & Grootendorst, 2023) are a class of deep learning models that learn statistical patterns of language from massive text corpora. They often use transformer-based architectures to capture context and long-range dependencies in text, enabling them to generate coherent and contextually relevant

responses or documents. Through training on billions of parameters, LLMs can learn grammar, facts, reasoning heuristics, and semantic relationships among words and phrases.

The following are among the main tasks for which LLMs are leveraged:

- **Text generation:** Producing coherent and contextually appropriate text on various topics.
- **Question answering:** Providing short or detailed answers to user queries based on learned knowledge.
- **Summarization:** Condensing lengthy documents into concise summaries while retaining key information.
- **Translation:** Converting text from one language to another with high fluency and accuracy.
- **Sentiment analysis:** Identifying emotional tone or polarity (e.g., positive, negative) within text.
- **Text classification:** Categorizing or tagging text according to predefined classes (e.g., spam detection, topic classification).
- **Token classification:** Assigning labels to each token in an input sequence.
- **Dialogue systems/chatbots:** Engaging in interactive, context-aware conversations with users.
- **Fill-mask:** Predicting missing tokens in a sequence.
- **Semantic search:** Searching computer applications, not only by strict string comparison, but rather by their meaning, means that if in the search field input “dog” is entered, the position with “rottweiler”, “dachshund”, and “terrier” will be displayed.
- **Dense retrieval:** Encoding of both queries and documents into continuous vector representations, then finds relevant documents by measuring the similarity between their vectors and the query’s vector.
- **Re-ranking:** Rescoring a subset of candidate results of search engines with different (more powerful) LLMs to produce a refined ranking.
- **RAG:** RAG combines search with text generation to address the hallucination problem in LLMs.

The transformers library and Hugging Face’s pipelines will be used to complete the selected tasks.

6.6.1 Text generation

Text generation tasks are handled by autoregressive language models (e.g., GPT series). These models are trained to predict the next token given a sequence of previous tokens, capturing both local and global dependencies in text. During inference, techniques like sampling, beam search, or nucleus sampling are used to generate coherent and contextually appropriate continuations based on the model’s learned probability distribution over the vocabulary.

To answer one of the commonly asked questions in economics we will use GPT2 model of text-generation pipeline:

```
from transformers import pipeline
generator = pipeline("text-generation", model="gpt2")
question = (
    "What do you think about the difference between the
    schools of interventionism"
    "of Maynard Keynes and the monetarist school of Milton
    Friedman?"
)
response = generator(question, max_length=1000, do_
sample=True, temperature=0.7, num_return_sequences=3)
print("Generated Answer:")
print(response[0]['generated_text'])
```

The above code fragment after execution gives the following answer (Figure 6.3).

Generated Answer:

What do you think about the difference between the schools of interventionism of Maynard Keynes and the monetarist school of Milton Friedman? Does he believe that the government should be allowed to spend more of its money, or should it have to spend less? Do you think that the government should have a say in the economy, or should it have to decide how much to spend? Do you think that the government should be allowed to spend more on education, or should it have to decide how much to spend? Do you think that the government should have to pay more to help poor children, or should it have to pay less to help the rich? Do you think that the government should be allowed to spend more on education, or should it have to pay less to help the poor?

And what do you think about the differences between the two schools?

The two are far more open to the idea of privatisation than the Keynesian school. But the Keynesian school does support the idea of the government funding schools, and the government may be able to spend a little more of its money, but it should have to spend less, or else it will have to spend more. And we may be talking about a very different kind of school.

If you have a question, please use the form below if you want to ask a question about the question or if you would like to have the opportunity to answer.

Figure 6.3 Text-generation task with GPT2 model results. (Continued)

Source: Own research with Python/transformers.

If you are interested in learning more about our School of Interventionism and the other schools of interventionism, please email us at education@britishpolicy.com.

For more information about the School of Interventionism, please visit our website.

Click here for the full School of Interventionism and the other schools of interventionism.

Figure 6.3 (Continued)

The same question asked of the DeepSeek model gives a slightly different answer.

```
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM,
GenerationConfig

model_name = "deepseek-ai/deepseek-llm-7b-chat"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name, torch_
dtype=torch.bfloat16, device_map="auto", offload_folder="data")
model.generation_config = GenerationConfig.from_pretrained
(model_name)
model.generation_config.pad_token_id = model.generation_
config.eos_token_id

messages = [
    {"role": "user", "content": "What do you think about the
difference between the schools of interventionism"
    "of Maynard Keynes and the monetarist school of Milton
Friedman?"}]
input_tensor = tokenizer.apply_chat_template(messages, add_
generation_prompt=True, return_tensors="pt")
outputs = model.generate(input_tensor.to(model.device),
max_new_tokens=500)

result = tokenizer.decode(outputs[0][input_tensor.
shape[1]:], skip_special_tokens=True)
print(result)
```

The DeepSeek model answer is presented in [Figure 6.4](#).

The difference between the schools of interventionism of Maynard Keynes and the monetarist school of Milton Friedman lies in their respective approaches to managing the economy and stabilizing the currency.

Maynard Keynes was a proponent of interventionist policies, which involve the government intervening in the economy to stabilize demand, inflation, and employment levels. Keynes believed that in times of economic downturn, the government should increase spending and lower taxes to stimulate economic growth. He also argued that governments should manage their balance sheets and engage in deficit spending to stabilize the economy and prevent deflation.

On the other hand, Milton Friedman was a proponent of monetarism, which focuses on controlling the money supply to stabilize prices and stabilize the economy. Friedman believed that a stable monetary base was crucial for economic stability, and that controlling the money supply through monetary policy was more effective than fiscal policy in achieving economic stability. He also believed that inflation was caused by excessive money supply growth, and that controlling the money supply could prevent inflation.

Overall, the difference between the schools of interventionism and monetarism lies in their respective approaches to managing the economy and stabilizing the currency. While interventionism emphasizes government intervention to stabilize the economy, monetarism emphasizes controlling the money supply to achieve economic stability.

Figure 6.4 Text-generation task with DeepSeek model results.

Source: Own research with Python/transformers.

6.6.2 *Question answering*

Extractive question answering models rely on the model's ability to comprehend both a question and a passage of context. Typically, the model outputs two distributions: one for the start position and one for the end position of the answer span within the context. This is achieved by finetuning the transformer with a task-specific head, allowing it to leverage its contextual embeddings to identify relevant text spans that answer the question.

6.6.3 *The summarization task*

Summarization is a sequence-to-sequence task where the model compresses lengthy text into a shorter, coherent summary. Typically, encoder–decoder architectures

(like BART or T5) are used. The encoder first processes the full text to generate context-rich representations, and the decoder generates a summary by attending to these representations. Techniques such as beam search and length normalization are often used during decoding to improve summary quality.

To summarize the fragment of (the working version of) this book the following code may be used.

```
from transformers import pipeline
summarizer = pipeline("summarization")

text = """
Semantic Search shifts the focus from literal keyword
matching to meaning-based retrieval, enabling systems to find
relevant information by understanding the intent behind a
query (rather than relying solely on overlapping keywords).
Major technology players like Google and Microsoft rapidly
adopted transformer-based models (e.g., BERT) for semantic
search, demonstrating its impact on user-facing systems. By
leveraging contextual embeddings, semantic search greatly
improves upon traditional search engines that rank documents
primarily by keyword frequency.
```

Key Principle

Instead of comparing raw tokens, semantic search encodes both queries and documents into continuous vector representations, capturing semantic relationships. This allows the system to retrieve items that share conceptual similarities, even if their wording differs from the query. For instance, the phrase "job opportunities" can match documents containing "positions available" or "vacancies."

Advantages

1. **Improved Accuracy:** Searching by concept fosters more relevant results and reduces the noise of purely keyword-based matches.
2. **User Satisfaction:** Because results align with user intent, semantic search enhances user experience in everything from web searches to enterprise knowledge bases.
3. **Wider Applications:** Beyond web search, semantic retrieval is integral to chatbots, recommendation systems, and domain-specific solutions (e.g., legal or medical).

Integration with Language Models

Modern Large Language Models (LLMs) supply the embeddings that power semantic search. They learn from massive corpora, allowing them to grasp subtle linguistic relationships. When a user enters a query, the system transforms the query into an embedding, compares it to precomputed embeddings of documents, and retrieves the closest matches. This synergy

of LLMs + semantic search forms the basis for advanced retrieval techniques, including dense retrieval, reranking, and retrieval-augmented generation (RAG).

Dense retrieval (see [Karpukhin et al. 2020](#)) uses embedding-based similarity to find the nearest neighbors of a query vector within a large document corpus. Both the query and documents are transformed into numeric vectors, and the system retrieves documents whose vectors lie closest to the query vector in embedding space. Figure references from the text depict queries mapped into the same space as documents.

Workflow

1. **Embedding:** Convert the query (e.g., "latest AI trends") and each candidate document into dense vectors via a trained neural encoder (e.g., a transformer).
2. **Similarity Search:** Use a vector database or a nearest neighbor algorithm to find documents whose embeddings are closest to the query's embedding (often via cosine similarity or Euclidean distance).
3. **Ranking:** Return the top kk nearest neighbors as search results.

Practical Benefits

- **Scalability:** Modern approximate nearest neighbor (ANN) techniques handle billions of vectors efficiently.
- **Robustness:** Embeddings capture synonyms and contextual meaning, enabling better recall for semantically equivalent but lexically different queries.
- **Flexible Adaptation:** Different domains (healthcare, finance, etc.) can train or fine-tune embedding models on domain-specific corpora.

In other words, dense retrieval relies heavily on vector-based representations, enabling fast similarity searches in large datasets and providing more contextually relevant matches than classic keyword-based methods.

Reranking is a subsequent step in many multi-stage search pipelines. After an initial retrieval (which might be keyword-based or dense), a reranker—often a more powerful language model—re-scores a subset of candidate results to produce a refined ranking. For instance, Bing significantly boosted search quality by adding BERT-like rerankers to reorder top results.

1. **Initial Retrieval:** The system collects a short list of potentially relevant documents via a fast method (e.g., dense retrieval).

2. **Reranker Input:** The query plus each candidate document are fed into a reranking model.
 3. **Relevance Scoring:** The reranker assigns a relevance score to each candidate, capturing deeper semantic nuances than the first-pass retrieval.
 4. **Reordering:** Documents are sorted by these new scores, typically boosting the most relevant or contextually correct results to the top.
- **Enhanced Precision:** By focusing on a smaller subset of documents, the reranker can afford heavier computation, leading to more accurate scoring.
 - **Flexibility:** Existing search pipelines can integrate a reranker without a complete overhaul—just add it as a final stage.
 - **Changing the order of results** can significantly improve relevance underscoring the potential impact of a reranker.

Dense retrieval excels at broad coverage (high recall) by retrieving top candidates quickly. Reranking refines these candidates for improved precision. Combining the two yields a robust search pipeline: a fast approximate retrieval stage, followed by an advanced model that invests more computation in analyzing fewer results.

Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (see Lewis 2020) combines search with text generation to address the hallucination problem in large language models. By retrieving relevant documents and providing them to the LLM as context, RAG systems produce more factual, grounded answers and can cite their sources. RAG not only boosts accuracy and reliability of generative AI but also unlocks real-time and private data usage

Steps

1. **Retrieval:** For a given query, the system uses dense retrieval or another method to find relevant documents from a knowledge base.
2. **Grounded Generation:** The LLM is prompted with the user's query plus the retrieved documents, encouraging it to formulate an answer tied to actual data.
3. **Citation or Evidence:** Many RAG systems highlight the sources used, increasing user trust and transparency.

Advanced Techniques

- **Query Rewriting:** The LLM can refine user queries to improve retrieval.

- **Multi-Hop Retrieval:** For complex queries, the system runs multiple retrieval steps (e.g., "largest car manufacturers" followed by "their EV lineups").
- **Agentic RAG:** The LLM acts more autonomously, possibly searching multiple data sources or updating them.
- **Evaluation:** Methods include citation recall, precision, faithfulness, and fluency. Libraries like Ragas implement "LLM-as-a-judge" to automate part of this evaluation.

RAG helps chatbots provide more reliable answers, enables "chat with my data" functionalities, and ensures LLM outputs are grounded in domain-specific or real-time content.

Industries such as healthcare, finance, or corporate knowledge management benefit from retrieving the latest internal documents. Retrieval-Augmented Generation (RAG) addresses LLM limitations by weaving semantic search and document retrieval into the generation process, reducing hallucinations and enhancing the factual quality of responses. It has rapidly become one of the most popular ways to harness LLMs for tasks demanding accurate, up-to-date information.

"""

```
summary_result = summarizer(text, max_length=150, min_
length=40, do_sample=False)
```

```
print("Summary:")
print(summary_result[0]['summary_text'])
```

With the results shown in [Figure 6.5](#).

6.6.4 *The translation task*

Machine translation is another sequence-to-sequence task. The source language text is encoded into continuous representations, and a decoder generates the target language text. The transformer's attention mechanisms enable the model to align corresponding parts of the source and target texts, learning complex mappings between languages. Training typically involves large parallel corpora and the use of autoregressive generation techniques during inference.

semantic search enhances user experience in everything from web searches to enterprise knowledge bases . reranking is a subsequent step in many multi-stage search pipelines . the system uses dense retrieval to retrieve relevant documents.

Figure 6.5 Summarization (internal transformers library model) task results.

Source: Own research with Python/transformers.

In the next example, a well-known English saying is translated into Polish.

```
from transformers import AutoModelForCausalLM, AutoTokenizer

model_id = "ModelSpace/GemmaX2-28-2B-v0.1"
tokenizer = AutoTokenizer.from_pretrained(model_id)

model = AutoModelForCausalLM.from_pretrained(model_id)

text = "Translate this from English to Polish:\nEnglish: How  
much wood would the woodchuck chuck ? \Polish:"
inputs = tokenizer(text, return_tensors="pt")

outputs = model.generate(**inputs, max_new_tokens=50)
print(tokenizer.decode(outputs[0], skip_special_tokens=True))
```

With the results presented in [Figure 6.6](#) (which the author assumes are not fully valid, but to judge this the elementary knowledge of Polish language is required).

6.6.5 Text classification

Text classification pipelines build on models that learn to encode semantic and contextual information from text. During pre-training (often using unsupervised objectives), models learn rich language representations. For classification, a task-specific head (typically a fully connected layer) is added on top of the final hidden state (or a designated token in BERT) and finetuned using a supervised loss (usually cross-entropy) to discriminate among classes.

6.6.6 Token classification

Token classification tasks assign a label to each token in an input sequence. Models use self-attention mechanisms to capture dependencies across the entire sequence, which is crucial for understanding context. A classification layer is applied on top of each token's representation, allowing the model to recognize and label entities (such as names, dates, or organizations) based on both the token itself and its surrounding context.

```
Translate this from English to Polish:  
English: How much wood would the woodchuck chuck ?  
Polish:Ile drewna zchwyci szczur?
```

Figure 6.6 Translation task with Gemma model results.

Source: Own research with Python/transformers.

6.6.7 *Dialogue systems*

Conversational pipelines extend text generation by incorporating context management across multiple turns of dialogue. These models may use special tokens or memory mechanisms to maintain conversational state. The underlying models are often fine-tuned on dialogue datasets, enabling them to generate responses that are coherent, not only on a single-turn basis, but also across an extended conversation.

6.6.8 *Fill-mask*

Also known as masked language modeling, this task involves predicting missing tokens in a sentence. The transformer model is pre-trained by randomly masking out some tokens and learning to predict them using the surrounding context. This self-supervised learning objective forces the model to understand bidirectional context, which is later leveraged in tasks like fill-mask during inference.

6.6.9 *Semantic search*

Semantic search shifts the focus from literal keyword matching to meaning-based retrieval, enabling systems to find relevant information by understanding the intent behind a query (rather than relying solely on overlapping keywords). Major technology players like Google and Microsoft rapidly adopted transformer-based models (e.g., BERT) for semantic search, demonstrating its impact on user-facing systems. By leveraging contextual embeddings, semantic search greatly improves upon traditional search engines that rank documents primarily by keyword frequency.

Instead of comparing raw tokens, semantic search encodes both queries and documents into continuous vector representations, capturing semantic relationships. This allows the system to retrieve items that share conceptual similarities, even if their wording differs from the query. For instance, the phrase “job opportunities” can match documents containing “positions available” or “vacancies”.

Among advantages of semantic search the following may be enlisted:

- **Improved accuracy:** Searching by concept fosters more relevant results and reduces the noise of purely keyword-based matches.
- **User satisfaction:** Because results align with user intent, semantic search enhances user experience in everything from web searches to enterprise knowledge bases.
- **Wider applications:** Beyond web search, semantic retrieval is integral to chatbots, recommendation systems, and domain-specific solutions (e.g., legal or medical).

Modern LLMs supply the embeddings that power semantic search. They learn from massive corpora, allowing them to grasp subtle linguistic relationships.

When a user enters a query, the system transforms the query into an embedding, compares it to precomputed embeddings of documents, and retrieves the closest matches. This synergy of LLMs + semantic search forms the basis for advanced retrieval techniques, including dense retrieval, reranking, and RAG.

6.6.10 Dense retrieval

Dense retrieval (see [Karpukhin et al., 2020](#)) uses embedding-based similarity to find the nearest neighbors of a query vector within a large document corpus. Both the query and documents are transformed into numeric vectors, and the system retrieves documents whose vectors lie closest to the query vector in embedding space. Figure references from the text depict queries mapped into the same space as documents.

The dense retrieval system usually work with the workflow routine:

- **Embedding:** Convert the query (e.g., “latest AI trends”) and each candidate document into dense vectors via a trained neural encoder (e.g., a transformer).
- **Similarity search:** Use a vector database or a nearest neighbor algorithm to find documents whose embeddings are closest to the query’s embedding (often via cosine similarity or Euclidean distance).
- **Ranking:** Return the top k-nearest neighbors as search results.

Among practical benefits of dense retrieval the following are worth to mention:

- **Scalability:** Modern approximate nearest neighbor (ANN) techniques handle billions of vectors efficiently.
- **Robustness:** Embeddings capture synonyms and contextual meaning, enabling better recall for semantically equivalent but lexically different queries.
- **Flexible adaptation:** Different domains (healthcare, finance, etc.) can train or finetune embedding models on domain-specific corpora.

In other words, dense retrieval relies heavily on vector-based representations, enabling fast similarity searches in large datasets and providing more contextually relevant matches than classic keyword-based methods.

6.6.11 Reranking

Reranking is a subsequent step in many multi-stage search pipelines. After an initial retrieval (which might be keyword-based or dense), a reranker – often a more powerful language model – re-scores a subset of candidate results to produce a refined ranking. For instance, the Bing search engine significantly boosted search quality by adding BERT-like rerankers to reorder top results.

Dense retrieval excels at broad coverage (high recall) by retrieving top candidates quickly. Reranking refines these candidates for improved precision. Combining the two yields a robust search pipeline: a fast approximate retrieval stage,

followed by an advanced model that invests more computation in analyzing fewer results.

6.6.12 Retrieval-augmented generation (RAG)

RAG (see [Lewis et al., 2020](#)) combines search with text generation to address the hallucination problem in LLMs. By retrieving relevant documents and providing them to the LLMs as context, RAG systems produce more factual, grounded answers and can cite their sources. RAG not only boosts accuracy and reliability of generative AI, but also unlocks real-time and private data usage.

Usually Retrieval augmented generation executes with the following steps:

- 1 Retrieval: For a given query, the system uses dense retrieval or another method to find relevant documents from a knowledge base.
- 2 Grounded generation: The LLM is prompted with the user’s query plus the retrieved documents, encouraging it to formulate an answer tied to actual data.
- 3 Citation or evidence: Many RAG systems highlight the sources used, increasing user trust and transparency.

RAG may be extended with the following advanced techniques:

- Query rewriting: The LLM can refine user queries to improve retrieval.
- Multi-hop retrieval: For complex queries, the system runs multiple retrieval steps.
- Agentic RAG: The LLM acts more autonomously, possibly searching multiple data sources or updating them.

RAG helps chatbots provide more reliable answers, enables “chat with my data” functionalities, and ensures LLMs’ outputs are grounded in domain-specific or real-time content. Industries such as healthcare, finance, or corporate knowledge management benefit from retrieving the latest internal documents. RAG addresses LLMs’ limitations by weaving semantic search and document retrieval into the generation process, reducing hallucinations and enhancing the factual quality of responses. It has rapidly become one of the most popular ways to harness LLMs for tasks demanding accurate, up-to-date information.

6.7 Research application example: comparison of sentiment analysis of IMDb reviews for BoW, GRU, and BERT

The IMDb reviews dataset was introduced by [Maas et al. \(2011\)](#) and contains 25,000 positive (labeled “positive” with IMDb rating ≥ 7 stars) and negative 2,500 (labeled “negative” with IMDb rating ≤ 4 stars) reviews. The “neutral” 5 or 4 star reviews are omitted. Due to its popularity, it’s included in the Keras library.

The dataset preparation process for model building requires a detailed description accompanied by several key considerations. The original representation of a

review in the dataset is vector of words. All words used are stored in the database available as a dictionary of values by `imdb.get_word_index()`. However, *Count-Vectorizer* preparing data for the bag-of-words (BOW) approach requires the input as a vector of words. Later on, the representation switches to unigrams (X_{train_bow} , X_{test_bow}) which are back numerical, but the particular coding of $X_{train_bow}/X_{test_bow}$ differs from X_{train}/X_{test} . The procedure of data preparation contains almost 30 lines (which is quite a high number as the data loading step):

```
import random
import time
import torch
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score, classification_report
from sklearn.model_selection import train_test_split
from transformers import BertTokenizer,
BertForSequenceClassification, TextClassificationPipeline
from tensorflow.keras.preprocessing.sequence import sequence
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.layers import SimpleRNN, LSTM, GRU,
Bidirectional, Dense, Embedding
from tensorflow.keras.datasets import imdb
from tensorflow.keras.models import Sequential
import numpy as np

random.seed(25032005)

max_features = 100000
maxlen = 350

(x_data, y_data), _ = imdb.load_data(num_words=max_features)

X_train, X_test, y_train, y_test = train_test_split(x_data,
y_data, test_size=0.2, random_state=25032005,
stratify=y_data)

X_train = pad_sequences(X_train, maxlen=maxlen)
X_test = pad_sequences(X_test, maxlen=maxlen)

word_index = imdb.get_word_index()
index_word = {index + 3: word for word, index in word_index.
items()}
index_word[0], index_word[1], index_word[2] = "<PAD>",
"<START>", "<UNK>"

def decode_review(encoded_review):
```



```
return " ".join(index_word.get(i, "<UNK>") for i in
encoded_review)
```

```
X_train_text = [decode_review(review) for review in X_train]
X_test_text = [decode_review(review) for review in X_test]
```

Having appropriate datasets prepared, the actual analysis may be conducted with the code similar to the one below:

```
print("***** BOW *****")
vectorizer = CountVectorizer()
X_train_bow = vectorizer.fit_transform(X_train_text)
X_test_bow = vectorizer.transform(X_test_text)
classifier = MultinomialNB()
classifier.fit(X_train_bow, y_train)
y_pred = classifier.predict(X_test_bow)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_
test, y_pred))
```

The results of analysis are presented in [Figure 6.7](#).

The overall accuracy at 86% seems to be satisfying. To look closer into the prediction of reviews with incorrect assignment the following code may be useful.

```
def first_index_not_equal(list1, list2, ommited_nr=1):
    _count = 0
    for i, (x, y) in enumerate(zip(list1, list2)):
        if (x != y) and (x == 1):
            _count += 1
            if (_count == ommited_nr):
                return i
    return -1

z = first_index_not_equal(y_test, y_pred, 2)
```

******* BOW *******

Accuracy: 0.86

Classification Report:

	precision	recall	f1-score	support
0	0.84	0.88	0.86	2500
1	0.87	0.83	0.85	2500
accuracy			0.86	5000
macro avg	0.86	0.86	0.86	5000
weighted avg	0.86	0.86	0.86	5000

Figure 6.7 Naïve Bayes BoW analysis on IMDb review dataset results.

Source: Own calculations with TensorFlow and transformers.

```

print(z)
if z!= -1:
    print( X_test_text[z])
    print( "actual:" + "positive" if y_test[z] else "negative" )
    print( "predicted:  " + "positive" if y_pred[z] else
           "negative")

```

The Naïve Bayes model has falsely classified the following sentence as negative:

“the only thing serious about this movie is the humor well worth the rental price i’ll bet you watch it twice it’s obvious that sutherland enjoyed his role”

And falsely assigned the following sentence as positive:

“i did not see this film in the theater i confess to an anti vinnie <UNK> bias who the hell was john travolta to be making movies i remember the oscar broadcast that year with travolta looking absolutely devastated when he didn’t win how dare he when there were real actors in the running i’m sorry john you should have won after catching this film on cable years ago i fell in love with the entire movie bud sissy uncle bob wes all wonderfully done i also` confess to never passing it by when i channel surf i have to stop and watch over the years i’ve learned to do most of the dialogue dance with my thumbs in my waistband and learned to appreciate travolta more the only disappointing thing to me was the oversight on the soundtrack of some of the music from urban cowboy looking for love defines the film but urban cowboy was chock full of classics that didn’t make it to the soundtrack it should have been a double cd”

Staying with the BOW approach, the model may be extended by multi-layer perceptron usage.

```

print("***** MLP *****")
vectorizer = CountVectorizer(max_features=max_features,
stop_words="english")
X_train_bow = vectorizer.fit_transform(X_train_text)
X_test_bow = vectorizer.transform(X_test_text)

print("Building the MLP model...")
mlp_model = MLPClassifier(
    hidden_layer_sizes=(128, 64),
    activation="relu",
    solver="adam",
    max_iter=10,
    random_state=25032005,
    verbose=True
)

print("Training the MLP model...")

```

```

mlp_model.fit(X_train_bow, y_train)

print("Evaluating the model...")
y_pred = mlp_model.predict(X_test_bow)
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report
      (y_test, y_pred))

```

The usage of multi-layer perceptron has improved the classification accuracy, to a level that seems to be the upper limit for the BOW approach. The results are presented in [Figure 6.8](#).

To improve the prediction, modern methods related to RNNs and transformer-based networks should be used.

In the next example, the embeddings and gated recurrent unit (GRU) network will be used for a similar task: assign sentiment to IMDb reviews.

```

Building the MLP model...
Training the MLP model...
Iteration 1, loss = 0.35272699
Iteration 2, loss = 0.10059020
Iteration 3, loss = 0.02942713
Iteration 4, loss = 0.00750841
Iteration 5, loss = 0.00192775
Iteration 6, loss = 0.00099336
...
Iteration 15, loss = 0.00034723
Iteration 16, loss = 0.00033691
Iteration 17, loss = 0.00032799
Iteration 18, loss = 0.00032009
Iteration 19, loss = 0.00031280
Training loss did not improve more than tol=0.000100 for
10 consecutive epochs. Stopping.
Evaluating the model...
Accuracy: 0.87
Classification Report:

```

	precision	recall	f1-score	support
0	0.89	0.86	0.87	2500
1	0.86	0.89	0.88	2500
accuracy			0.87	5000
macro avg	0.88	0.87	0.87	5000
weighted avg	0.88	0.87	0.87	5000

Figure 6.8 Multilayer perceptron BoW analysis on IMDb review dataset results.

Source: Own calculations with TensorFlow and transformers.

```

print("***** GRU *****")
vocab_size = max(max(x_data) )+ 1000
# Keeping a fixed length of all reviews to max 400 words
max_words = 400
embd_len = 64

x_train = sequence.pad_sequences(X_train, maxlen=max_words)
x_test = sequence.pad_sequences(X_test, maxlen=max_words)

x_valid, y_valid = x_train[:64], y_train[:64]
x_train_, y_train_ = x_train[64:], y_train[64:]

# Defining GRU model
gru_model = Sequential(name="GRU_Model")
gru_model.add(Embedding(vocab_size,
                        embd_len))
gru_model.add(GRU(128,
                  activation='tanh',
                  return_sequences=False))
gru_model.add(Dense(1, activation='sigmoid'))

# Printing the Summary
print(gru_model.summary())

# Compiling the model
gru_model.compile(
    loss="binary_crossentropy",
    optimizer='adam',
    metrics=['accuracy']
)
np.array(y_train_)
y_train_ = np.array(y_train_)
y_valid = np.array(y_valid)
history2 = gru_model.fit(x_train_, y_train_,
                        batch_size=64,
                        epochs=10,
                        verbose=1,
                        validation_data=(x_valid, y_valid))

y_pred=gru_model.predict(x_test)
y_pred=[1 if y>0.5 else 0 for y in y_pred]
print()
print("GRU model Score---> ", gru_model.evaluate(x_test,
np.array(y_test), verbose=1))
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")

```

```
print("Classification Report:\n", classification_report(y_test,
y_pred))
```

With the results presented in [Figure 6.9](#).

```
***** GRU *****
```

```
Model: "GRU_Model"
```

Layer (type)	Output Shape	Param #
embedding (Embedding)	?	0 (unbuilt)
gru (GRU)	?	0 (unbuilt)
dense (Dense)	?	0 (unbuilt)

```
Total params: 0 (0.00 B)
```

```
Trainable params: 0 (0.00 B)
```

```
Non-trainable params: 0 (0.00 B)
```

```
None
```

```
Epoch 1/10
```

```
312/312 ————— 232s 735ms/step - accuracy: 0.6389 - loss:
0.6149 - val_accuracy: 0.8594 - val_loss: 0.3342
```

```
Epoch 2/10
```

```
312/312 ————— 233s 745ms/step - accuracy: 0.9000 - loss:
0.2552 - val_accuracy: 0.8906 - val_loss: 0.3599
```

```
...
```

```
312/312 ————— 233s 747ms/step - accuracy: 0.9984 - loss:
0.0057 - val_accuracy: 0.8594 - val_loss: 0.5460
```

```
Epoch 10/10
```

```
312/312 ————— 232s 745ms/step - accuracy: 0.9987 - loss:
0.0046 - val_accuracy: 0.9062 - val_loss: 0.6899
```

```
157/157 ————— 13s 82ms/step
```

```
157/157 ————— 12s 73ms/step - accuracy: 0.8571 - loss:
0.8693
```

```
GRU model Score---> [0.8370697498321533, 0.8611999750137329]
```

```
Accuracy: 0.86
```

```
Classification Report:
```

	precision	recall	f1-score	support
0	0.82	0.93	0.87	2500
1	0.92	0.79	0.85	2500
accuracy			0.86	5000
macro avg	0.87	0.86	0.86	5000
weighted avg	0.87	0.86	0.86	5000

Figure 6.9 GRU analysis on IMDb review dataset results.

Source: Own calculations with TensorFlow and transformers.

In the last approach, the BERT model will be used for IMDb sentiment analysis.

```
print("***** BERT *****")
y_pred = []
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = BertForSequenceClassification.from_pretrained('gritli/
bert-sentiment-analyses-imdb')
model.eval()
pipe = TextClassificationPipeline(model=model, tokenizer=
tokenizer, top_k=None)
start_time = time.time()
for i,text in enumerate(X_test_text):
    print(f"**** {i}/{len(X_test_text)} -- {time.time() -
start_time} seconds ****")
    text=" ".join(text.split()[:360])
    predicted_class = int(pipe(text)[0][0]["label"].split("_")
[-1])
    y_pred += [predicted_class]

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.2f}")
print("Classification Report:\n", classification_report(y_test,
y_pred))
```

The results presented in [Figure 6.10](#).

This benchmark study showed that in the task of sentiment analysis on the IMDb dataset, various models demonstrate differing levels of performance.

```
***** BERT *****
Device set to use cpu
**** 0//5000 -- 0.0 seconds ****
**** 1//5000 -- 0.43171072006225586 seconds ****
...
**** 4997//5000 -- 2331.1417458057404 seconds ****
**** 4998//5000 -- 2331.6775970458984 seconds ****
**** 4999//5000 -- 2332.0517194271088 seconds ****

Accuracy: 0.93
Classification Report:
```

	precision	recall	f1-score	support
0	0.90	0.96	0.93	2500
1	0.96	0.89	0.92	2500
accuracy			0.93	5000
macro avg	0.93	0.93	0.93	5000
weighted avg	0.93	0.93	0.93	5000

Figure 6.10 BERT transformer-based family analysis on IMDb review dataset results.

Source: Own calculations with TensorFlow and transformers.

The traditional Naïve Bayes with BOW method achieves a solid accuracy of 0.86, with balanced precision and recall for both sentiment classes, indicating a consistent yet modest classification ability. The multi-layer perceptron using the same BOW representation slightly improves upon this with an accuracy of 0.87 and higher macro-averaged precision (0.88) and F1 score (0.87), suggesting that even a simple neural network architecture benefits from better capacity to model nonlinear patterns within the text data.

In contrast, deep learning models utilizing sequence-based or contextual representations exhibit more distinct characteristics. The GRU model, while matching the 0.86 accuracy of Naïve Bayes, displays greater class imbalance in recall, particularly excelling at detecting negative reviews (recall = 0.93) while underperforming for positive ones (recall = 0.79). This asymmetry may reflect GRU's sensitivity to specific sequential patterns in the text. Finally, the BERT-based model significantly outperforms all others, achieving a 0.93 accuracy and F1 scores above 0.92 for both classes. Its higher performance is attributed to its deep contextual understanding of language, capturing semantic nuances beyond the reach of BOW-based or recurrent models, making it the most effective approach in this comparison.

Note

- 1 [Shannon \(1948\)](#) introduced the probability of occurrence of words in n-grams, but it's equivalent to relative frequency.

References

- Alammar, J., & Grootendorst, M. (2024). *Hands-on large language models: Language understanding and generation*. O'Reilly Media, Inc.
- Bengio, Y., Ducharme, R., Vincent, P., & Jauvin, C. (2003). A neural probabilistic language model. *Journal of Machine Learning Research*, 3, 1137–1155.
- Brill, E. (1992). A simple rule-based part-of-speech tagger. *Proceedings of the Workshop on Speech and Natural Language*, 112–116.
- Church, K. W. (1989). A stochastic parts program and noun phrase parser for unrestricted text. In *International Conference on Acoustics, Speech, and Signal Processing*, (pp. 695–698). IEEE.
- Cristianini, N., & Shawe-Taylor, J. (2000). *An introduction to support vector machines and other kernel-based learning methods*. Cambridge University Press.
- Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bi-directional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, I*, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- Ding, X., Liu, B., & Yu, P. S. (2008). A holistic lexicon-based approach to opinion mining. In *Proceedings of the Conference on Web Search and Web Data Mining (WSDM-2008)* (pp. 231–240). ACM. <https://doi.org/10.1145/1341531.1341561>
- Hu, M., & Liu, B. (2004). Mining and summarizing customer reviews. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD-2004)* (pp. 168–177). ACM. <https://doi.org/10.1145/1014052.1014073>
- Irsoy, O., & Cardie, C. (2014). Opinion mining with deep recurrent neural networks. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, October 25-29, 2014, Doha, Qatar. (pp. 720–728).

- Joachims, T. (1998). Text categorization with support vector machines: Learning with many relevant features (*LS8 Report 23*). Universität Dortmund.
- Kamps, J., Marx, M., Mokken, R. J., & de Rijke, M. (2004). Using WordNet to measure semantic orientation of adjectives. In *Proceedings of the 4th International Conference on Language Resources and Evaluation (LREC-2004)* (Vol. 4, pp. 1115–1118).
- Karpukhin, V., Oğuz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., & Yih, W.-T. (2020, April 10). *Dense passage retrieval for open-domain question answering* (Version 3) [arXiv preprint]. arXiv. <https://doi.org/10.48550/arXiv.2004.04906>
- Kim, S.-M., & Hovy, E. (2004). Determining the sentiment of opinions. In *Proceedings of the 20th International Conference on Computational Linguistics (COLING-2004)* (pp. 1367–1373). Geneva, Switzerland.
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.
- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019). RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint arXiv:1907.11692*. <https://doi.org/10.48550/arXiv.1907.11692>
- Maas, A. L., Daly, R. E., Pham, P. T., Huang, D., Ng, A. Y., & Potts, C. (2011). Learning Word Vectors for Sentiment Analysis. *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies* Portland, Oregon, June 19–24, 2011. (pp. 142–150).
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*.
- Mikolov, T., Karafiát, M., Burget, L., Cernocký, J., & Khudanpur, S. (2010). Recurrent neural network based language model. *Proceedings of the 11th Annual Conference of the International Speech Communication Association (INTERSPEECH)*, 1045–1048.
- Miller, G. A. (1990). Nouns in WordNet: A lexical inheritance system. *International Journal of Lexicography*, 3(4), 245–264.
- Mohammad, S., Dunne, C., & Dorr, B. (2009, August). Generating high-coverage semantic orientation lexicons from overtly marked words and a thesaurus. In *Proceedings of the 2009 conference on empirical methods in natural language processing* Singapore, 6-7 August 2009 (pp. 599–608).
- Pang, B., Lee, L., & Vaithyanathan, S. (2002). Thumbs up? Sentiment classification using machine learning techniques. *Proceedings of the ACL-02 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 10, 79–86. <https://doi.org/10.3115/1118693.1118704>
- Pennington, J., Socher, R., & Manning, C. (2014). GloVe: Global vectors for word representation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (pp. 1532–1543), October 25–29, 2014, Doha.
- Robertson, S. (2004). Understanding inverse document frequency: On theoretical arguments for IDF. *Journal of Documentation*, 60(5), 503–520. <https://doi.org/10.1108/00220410410560582>
- Santorini, B. (1990). *Part-of-speech tagging guidelines for the Penn Treebank Project*. University of Pennsylvania, School of Engineering and Applied Science, Department of Computer and Information Science.
- Shannon, C. E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- Sparck Jones, K. (1972). A statistical interpretation of term specificity and its application in retrieval. *Journal of Documentation*, 28(1), 11–21. <https://doi.org/10.1108/eb026526>
- Taboada, M., Brooke, J., Tofiloski, M., Voll, K., & Stede, M. (2011). Lexicon-based methods for sentiment analysis. *Computational Linguistics*, 37(2), 267–307. https://doi.org/10.1162/COLI_a_00049
- Tang, D., Qin, B., & Liu, T. (2015, September). Document modeling with gated recurrent neural network for sentiment classification. In *Proceedings of the 2015 conference on*

- empirical methods in natural language processing* (pp. 1422–1432), Lisbon, Portugal, 17–21 September 2015.
- Turney, P. D. (2002). Thumbs up or thumbs down? Semantic orientation applied to sentiment analysis. *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics* (pp. 417–424). Association for Computational Linguistics, USA. <https://doi.org/10.3115/1073083.1073153>
- Valitutti, A., Strapparava, C., & Stock, O. (2004). Developing affective lexical resources. *PsychNology Journal*, 2(1), 61–83.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30, 5998–6008. <https://doi.org/10.48550/arXiv.1706.03762>
- Wiebe, J., & Riloff, E. (2005). Creating subjective and objective sentence classifiers from unannotated texts. In A. Gelbukh (Ed.), *Computational Linguistics and Intelligent Text Processing* (pp. 486–497). Springer. https://doi.org/10.1007/978-3-540-30586-6_53
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... Rush, A. M. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations* (pp. 38–45), November 16–20, 2020. Punta Cana.
- Zagibalov, T., & Carroll, J. (2008). Unsupervised classification of sentiment and objectivity in Chinese text. In *Proceedings of the Third International Joint Conference on Natural Language Processing* (pp. 304–311). Asian Federation of Natural Language Processing. <https://aclanthology.org/108-1040/>
- Zekaoui, N. E., Yousfi, S., Rhanoui, M., & Mikram, M. (2023). Analysis of the evolution of advanced transformer-based language models: Experiments on opinion mining. *arXiv preprint arXiv:2308.03235*.

7 Other applications of deep learning models

7.1 Cluster analysis with deep learning models

7.1.1 Procedure of cluster analysis

Cluster analysis, introduced by Tryon in 1939, is a set of algorithms used to group similar objects into categories, aiding in developing taxonomies through exploratory data analysis ([Tryon, 1939](#)).

The main application area in economics is to find segments on the market (segmentation process) but also to find similar groups of financial institutions, insurance companies, and capital fund as well as for regional analysis (findings of cities, voivodeships or countries with similar levels of economic phenomena). There are many good sources describing the subsequent steps of cluster analysis (see, e.g., [Everitt et al., 2011](#); [Gordon, 1999](#); [Hennig et al., 2015](#); [Walesiak, 2009b](#)) so not all the details will be discussed in this chapter, but instead there will be a focus on clustering methods, which might be improved in some way by deep learning algorithms.

The procedure of cluster analysis can be stated in seven points (cf. [Walesiak, 2009b](#)):

- 1 Choosing the object and variables in research;
- 2 Choosing the variables' normalization formula;
- 3 Choosing the distance measure (only for classification methods not based directly on data matrix);
- 4 Choosing the classification algorithm;
- 5 Setting the number of clusters;
- 6 Evaluating the quality of results;
- 7 Describing and interpreting the results.

The first six points will be described in [Sections 7.1.1.1–7.1.1.6](#). The seventh slightly exceeds the thematic scope

7.1.1.1 Choosing the objects and variables in research

In cluster analysis only those variables that are believed to help discriminate the data should be included (see [Milligan, 1996](#)). Three classes of approach in

the choice of variables for cluster analysis may be distinguished (Gnanadesikan et al., 1995):

- variable selection (selecting a subset of relevant variables to help to recover cluster structure in data) – Heuristic Identification of Noisy Variables (HINoV) method for metric data (see Carmone et al., 1999);
- variable weighting (introducing relative importance of the variables according to their weights) – Clustering Objects on Subsets of Attributes (COSA) (Friedman and Meulman, 2004);
- transforming original data into reduced factor space via, for example, factor analysis or principal component analysis.

7.1.1.2 *Choosing the variables' normalization formula*

The same normalization procedures as described in section 3.1 may be used for data preprocessing in cluster analysis procedure.

7.1.1.3 *Choosing the distance measure*

Typical distance measures used in clustering procedure are presented in Table 7.1.

Table 7.1 Distance measures used in clustering procedure

Name of measure	Distance d_{ik}
Minkovsky ($p \geq 1$)	$\sqrt[p]{\sum_{j=1}^m z_{ij} - z_{kj} ^p}$
City (Manhattan) ($p = 1$)	$\sum_{j=1}^m z_{ij} - z_{kj} $
Euclidean ($p = 2$)	$\sqrt{\sum_{j=1}^m (z_{ij} - z_{kj})^2}$
Tchebychev (maximum) ($p \rightarrow \infty$)	$\max_j z_{ij} - z_{kj} $
Canberra	$\sum_{j=1}^m \frac{ z_{ij} - z_{kj} }{(z_{ij} + z_{kj})}$
Bray-Curtis	$\frac{\sum_{j=1}^m z_{ij} - z_{kj} }{\sum_{j=1}^m (z_{ij} + z_{kj})}$

(Continued)

Table 7.1 (Continued)

Name of measure	Distance d_{ik}
GDM1	$\frac{1}{2} - \frac{\sum_{j=1}^m (z_{ij} - z_{kj})(z_{kj} - z_{ij}) + \sum_{j=1}^m \sum_{l=1, l \neq i, k}^n (z_{ij} - z_{lj})(z_{kj} - z_{lj})}{2 \left[\sum_{j=1}^m \sum_{l=1}^n (z_{ij} - z_{lj})^2 \cdot \sum_{j=1}^m \sum_{l=1}^n (z_{kj} - z_{lj})^2 \right]^{\frac{1}{2}}}$
GDM2	$\frac{1}{2} - \frac{\sum_{j=1}^m a_{ikj} b_{kij} + \sum_{j=1}^m \sum_{l=1, l \neq i, k}^n a_{ilj} b_{klj}}{2 \left[\left(\sum_{j=1}^m a_{ikj}^2 + \sum_{j=1}^m \sum_{l=1, l \neq i, k}^n a_{ilj}^2 \right) \left(\sum_{j=1}^m b_{kij}^2 + \sum_{j=1}^m \sum_{l=1, l \neq i, k}^n b_{klj}^2 \right) \right]^{\frac{1}{2}}}$ $a_{ipj}(b_{krj}) = \begin{cases} 1 & \text{jeżeli } x_{ij} > x_{pj} \left(x_{kj} > x_{rj} \right) \\ 0 & \text{jeżeli } x_{ij} = x_{pj} \left(x_{kj} = x_{rj} \right) \\ -1 & \text{jeżeli } x_{ij} < x_{pj} \left(x_{kj} < x_{rj} \right) \end{cases}$ $p = k, l; r = i, l$
Sokal – Michener	$\frac{m - m_r}{m}$

Where:

$i, k, l = 1, \dots, n$ – object numbers, $j = 1, \dots, m$ – variable number; m – number of variables, m_r – number of variables for which A_i, A_k there is an equality relation $x_{ij} (x_{kj}, x_{lj})$ between objects, – i -th (k -th, l -th) observation on the j -th variable; $z_{ij} (z_{kj}, z_{lj})$ – normalized value of the j -th variable for the i -th (k -th, l -th) object.

Source: Walesiak (2009a).

7.1.1.4 Choosing the classification algorithm

The clustering algorithm may be divided into five main groups:

1 Hierarchical aggregative clustering methods (Gordon, 1999, p. 79):

- Ward hierarchical clustering;
- Single link hierarchical clustering;
- Complete link hierarchical clustering;
- Average link hierarchical clustering;
- Mcquitty (1966) hierarchical clustering;
- Centroid hierarchical clustering.

2 Hierarchical divisive clustering (Gordon, 1999, p. 79):

- Divisive ANALysis (DIANA).

3 Optimization methods:

- Partitioning around medoids, also called k-medoids method (Kaufman & Rousseeuw, 1990);
- k-means algorithm (Aldenderfer & Blashfield, 1984; Everitt et al., 2011).

4 Untypical cluster shapes methods:

- Spectral clustering (Ng et al., 2002);
- Mean shift clustering (Comaniciu & Meer, 1999);

5 Model based clustering.

7.1.1.5 Setting the number of clusters

To find the optimal number of clusters, usually one of the cluster quality indices is used. Each cluster has its optimum criterion and the optimal number of clusters is calculated by this criterion. There are almost 200 cluster quality indices defined; the most popular and effective are described in Table 7.2.

Table 7.2 Cluster quality indices used in cluster analysis procedure

Name	Formula	Criterion
1 Caliński Harabasz	$G1(u) = \frac{B_u / (u-1)}{W_u / (n-u)}, G1(u) \in R_+$	$\hat{u} = \arg \max_u \{G1(u)\}$
2 Baker Hubert	$G2(u) = \frac{s(+)-s(-)}{s(+)+s(-)}, G2(u) \in (-1, 1)$	$\hat{u} = \arg \max_u \{G2(u)\}$
3 Hubert Levine	$G3(u) = \frac{D(u) - r \cdot D_{min}}{r \cdot D_{max}}, G3(u) \in (0, 1)$	$\hat{u} = \arg \min_u \{G3(u)\}$
4 Silhouette	$S(u) = \frac{1}{n} \sum_{i=1}^n \frac{b(i) - a(i)}{\max\{a(i); b(i)\}},$ $S(u) \in (-1, 1)$	$\hat{u} = \arg \max_u \{S(u)\}$
5 Krzanowski Lai	$KL(u) = \left \frac{DIFF_u}{DIFF_{u+1}} \right , KL(u) \in R_+$ $DIFF_u = (u-1)^{2/m} W_{u-1} - u^{2/m} W_u$	$\hat{u} = \arg \max_u \{KL(u)\}$
6 Davies Bouldin	$DB(u) = \frac{1}{u} \sum_{r=1}^u \max_{s \neq r} \left(\frac{S_r + S_s}{d_{rs}} \right)$	$\hat{u} = \arg \min_u \{DB(u)\}$

(Continued)

Table 7.2 (Continued)

	Name	Formula	Criterion
7	Hartigan	$H(u) = \left(\frac{W_u}{W_{u+1}} - 1 \right) (n - u - 1), H(u) \in R_+$	minimal u , for which $H(u) \leq 10$
8	Gap	$Gap(u) = \frac{1}{B} \sum_{b=1}^B \log W_{ub} - \log W_u$ $Gap(u) \in R$	minimal u , for which $diffu \geq 0$

Where:

B_u – interclass covariance matrix, W_u – intraclass covariance matrix, tr – trace of matrices, $B_u(W_u) = tr(B_u)(tr W_u)$, $r, s = 1, \dots, u$ – class number, u – number of classes, $i, k = 1, \dots, n$ – object number, n – number of objects, m – number of variables, $s(+)$ – number of pairs of compatible distances, $s(-)$ – number of pairs of incompatible distances, $D(u)$ – sum of all intraclass distances, r – number of intra-class distances, $D_{min}(D_{max})$ – the smallest (largest) intra-class distance, $a(i) = \sum_{k \in \{P_r \setminus i\}} d_{ik} / (n_r - 1)$ – the average distance of the object i from other objects belonging to the class P_r , $b(i) = \min_{s \neq r} \{d_{is}\}$, $d_{iP_s} = \sum_{k \in P_s} d_{ik} / n_s$ – the average distance of the object i from objects belonging to the class P_s , B – the number of generated sets of observations, $s_u = sd_u \sqrt{1 + 1/B}$, sd_u – the standard deviation from the value $\{\log W_{ub}\}$, $diffu = Gap(u) - Gap(u+1) + s_{u+1}$, $d_{rs} = \sqrt{\sum_{j=1}^m |z_{rj} - z_{sj}|^p}$ – the distance between the centers of gravity (medoids) of the r and s classes ($-p=1$ urban distance, $p=2$ – Euclid distance); $S_r = \sqrt[q]{\frac{1}{n_r} \sum_{i \in P_r} \sum_{j=1}^m |x_{ij}^r - z_{rj}|^q}$ – measure of dispersion of objects in class: $q=1$ – average distance of objects in r th class from the center of gravity (medoids) of class; $q=2$ – standard deviation of the distance of objects in the r -th class from the center of gravity (medoids) of the class.

Source: Walesiak (2009b) based on Caliński and Harabasz (1974), Hubert (1974), Milligan and Cooper (1985), Kaufman and Rousseeuw (1990), Hartigan (1975), Tibshirani et al. (2001), Davies and Bouldin (1979).

7.1.1.6 Evaluating the quality of results

The replication analysis is commonly used to evaluate cluster stability (see Breckenridge, 2000, pp. 262–263; Milligan, 1996, pp. 368–369).

A synthetic measure to measure the overall quality of classification (relative compactness and class separability) was proposed by Rousseeuw (1987) (see Kaufman & Rousseeuw, 1990, pp. 83–88). The silhouette index is commonly used for clustering evaluation purposes, but there are some situations in which it may lead to some not altogether correct conclusions (cf. Dudek, 2020).

7.1.2 Advantage of using SpectralNet instead of spectral clustering algorithm

While standard cluster analysis algorithms are usually sufficient for most applications for clusters based more or less on a multivariate normal distribution, there is an area of application where spectral clustering is far superior to other

algorithms. Of course, one can argue whether two spirals are true clusters (cf. Hennig, 2015). However, most readers probably would agree that the partitioning is determined by the shapes of the two spirals (cf. Figure 7.1) and not, for example, by a vertical line running through the center of the scatter plot. For this type of cluster, spectral classification reflects the structure of unusual shapes much better than other methods.

Spectral classification (Ng et al., 2002; Karatzoglou, 2006; von Luxburg, 2006) is a relatively new and rapidly developing approach in cluster analysis. It is not strictly a new algorithm, but rather a way to prepare data for known methods of cluster analysis (e.g., *k*-means). This approach gives very promising results, especially for unusual (i.e. not from normal distribution) shapes. In most studies, this technique is used in combination with the classical *k*-means method, but this assumption is not based on any theoretical basis. The spectral classification procedure

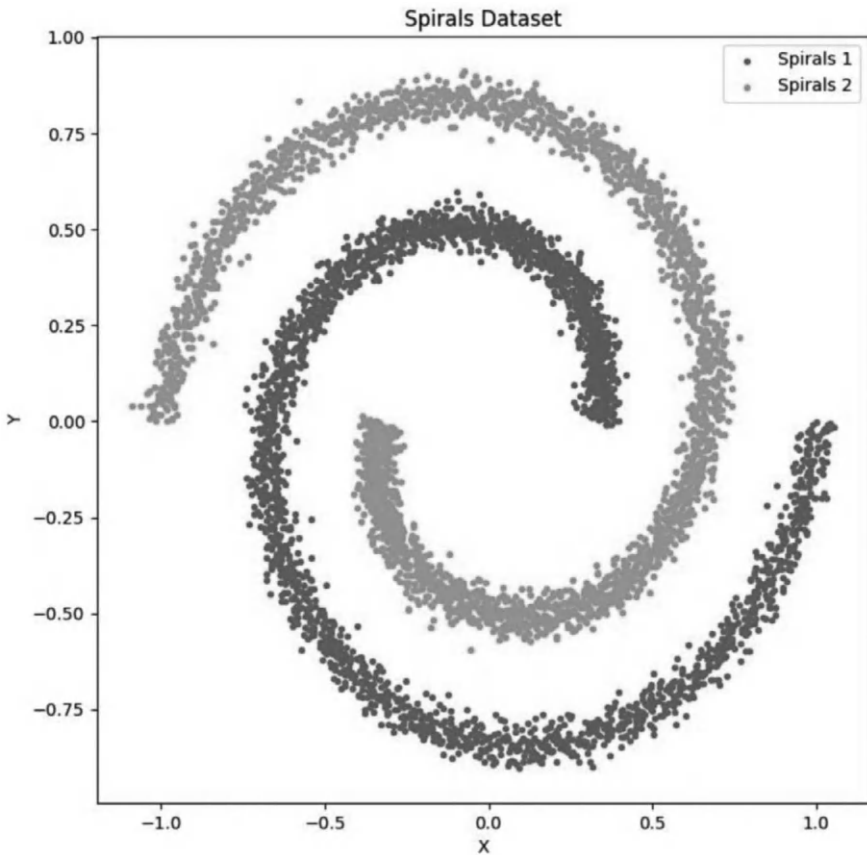


Figure 7.1 Artificially generated `mlbench_spirals`.

Source: Own work, with Python/Matplotlib based on R `mlbench` package (Leisch & Dimitriadou, 2024).

has many variants, for all of which six most important stages can be distinguished (according to [Ng et al., 2002](#)):

- 1 Construction of a data matrix $\mathbf{X} = [x_{ij}]$ with dimensions $n \times m$ ($i = 1, \dots, n$ – object number, $j = 1, \dots, m$ – variable number). For metric data, normalize the values of the variables.
- 2 Use of a kernel estimator to calculate $\mathbf{A} = [A_{ik}]$ the *affinity matrix* between objects.
- 3 Construction of a normalized Laplacian matrix $\mathbf{L} = \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$ ($\mathbf{D}$ – diagonal weight matrix on the main diagonal of which there are sums of each row from, $\mathbf{A} = [A_{ik}]$ and outside the main diagonal there are zeros). In fact, the normalized Laplacian matrix takes the form: $\mathbf{I} - \mathbf{L}$.
- 4 Calculation of eigenvalues and corresponding eigenvectors (of length equal to one) for $\mathbf{L}$. Ordering eigenvectors by decreasing eigenvalues. The first u eigenvectors (u – number of classes) form a matrix $\mathbf{E} = [e_{ij}]$ with dimensions $n \times u$.
- 5 Normalization of this matrix according to the formula $y_{ij} = e_{ij} / \sqrt{\sum_{j=1}^u e_{ij}^2}$ ($i = 1, \dots, n$ – object number, $j = 1, \dots, u$ – variable number, u – number of classes). With this normalization, the length of each row vector of the matrix $\mathbf{Y} = [y_{ij}]$ is equal to one.
- 6 The matrix $\mathbf{Y}$ is the starting point for the application of classical methods of cluster analysis.

Individual variants of algorithms in the spectral approach may differ (cf. Walesiak & Dudek, 2009), such as:

- the type of kernel estimator in step 2;
- the formula for constructing the Laplacian matrix in step 4 (see e.g. [von Luxburg \[2006\]](#));
- determining the value of the parameter σ – *the kernel width* of the number of clusters.

In the original proposal, [Ng et al. \(2002\)](#) postulate the use of the k-means algorithm as the final cluster analysis algorithm in step 6.

To confront the strength of spectral clustering vs spirals (other untypical shapes may be used as well) the following Python code may be used.

```
import matplotlib.pyplot as plt
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans, SpectralClustering,
AgglomerativeClustering
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score

def mlbench_spirals(n, cycles=1, sd=0):
    x = np.zeros((n, 2))
```



```

    c2 = np.random.choice(np.arange(1, n + 1), size=n//2,
replace=False)
    c1 = np.ones(n, dtype=int)
    c1[c2 - 1] = 2

    x[~np.isin(np.arange(1, n + 1), c2), :] =
mlbench_1spiral(n=n - len(c2), cycles=cycles, sd=sd)
    x[np.isin(np.arange(1, n + 1), c2), :] =
-mlbench_1spiral(n=len(c2), cycles=cycles, sd=sd)

    return {'x': x, 'classes': c1}

def mlbench_1spiral(n, cycles=1, sd=0):
    w = np.linspace(0, cycles, n)
    x = np.zeros((n, 2))
    x[:, 0] = (2 * w + 1) * np.cos(2 * np.pi * w) / 3
    x[:, 1] = (2 * w + 1) * np.sin(2 * np.pi * w) / 3

    if sd > 0:
        e = np.random.normal(0, sd, n)
        xs = np.cos(2 * np.pi * w) - np.pi * (2 * w + 1) *
np.sin(2 * np.pi * w)
        ys = np.sin(2 * np.pi * w) + np.pi * (2 * w + 1) *
np.cos(2 * np.pi * w)
        nrm = np.sqrt(xs**2 + ys**2)
        x[:, 0] += e * ys / nrm
        x[:, 1] -= e * xs / nrm

    return x

n_points = 5000
d = mlbench_spirals(n_points,1,0.03)
X,y_true=d['x'],d['classes']

plt.scatter(X[y_true==1,0],X[y_true==1,1], label='Spirals
1', marker='.')
plt.scatter(X[y_true==2,0],X[y_true==2,1], label='Spirals
2', marker='.')
plt.title('Spirals Dataset')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.savefig('spirals.jpg')
plt.show()

kmeans = KMeans(n_clusters=2, random_state=42).fit(X)
spectral = SpectralClustering(n_clusters=2, affinity='nearest_
neighbors', random_state=42).fit(X)

```

```

ward = AgglomerativeClustering(n_clusters=2, linkage='ward').
    fit(X)
gmm = GaussianMixture(n_components=2, random_state=42).fit(X)
gmm_labels = gmm.predict(X)

# Compute ARI scores
ari_kmeans = adjusted_rand_score(y_true, kmeans.labels_)
ari_spectral = adjusted_rand_score(y_true, spectral.labels_)
ari_ward = adjusted_rand_score(y_true, ward.labels_)
ari_gmm = adjusted_rand_score(y_true, gmm_labels)

print(f"ARI (Adjusted Rand Index) Scores:")
print(f"KMeans: {ari_kmeans:.3f}")
print(f"Spectral Clustering: {ari_spectral:.3f}")
print(f"Ward (Agglomerative): {ari_ward:.3f}")
print(f"Gaussian Mixture (Model-based): {ari_gmm:.3f}")

plt.figure(figsize=(15, 8))

# KMeans
plt.subplot(2, 2, 1)
plt.scatter(X[:, 0], X[:, 1], c=kmeans.labels_,
            cmap='viridis', s=5)
plt.title(f'KMeans Clustering (ARI: {ari_kmeans:.3f})')

# Spectral Clustering
plt.subplot(2, 2, 2)
plt.scatter(X[:, 0], X[:, 1], c=spectral.labels_, cmap='
viridis', s=5)
plt.title(f'Spectral Clustering (ARI: {ari_spectral:.3f})')

# Ward (Agglomerative)
plt.subplot(2, 2, 3)
plt.scatter(X[:, 0], X[:, 1], c=ward.labels_, cmap='viridis',
            s=5)
plt.title(f'Ward Clustering (ARI: {ari_ward:.3f})')

# Gaussian Mixture (Model-based)
plt.subplot(2, 2, 4)
plt.scatter(X[:, 0], X[:, 1], c=gmm_labels, cmap='viridis',
            s=5)
plt.title(f'GMM Clustering (ARI: {ari_gmm:.3f})')

plt.suptitle('Comparison of Clustering Methods on Two Spirals')
plt.tight_layout()
plt.savefig('spirals_clustering_results.jpg')
plt.show()

```

ARI (Adjusted Rand Index) Scores:
KMeans: -0.000
Spectral Clustering: 1.000
Ward (Agglomerative): 0.000
Gaussian Mixture (Model-based): -0.000

Figure 7.2 The results of `mlbench_spirals` dataset clustering with spectral clustering, k-means, Ward agglomerative clustering, and Gaussian mixture model-based clustering.

Source: Own calculations, with Python/scikit-learn.

The spectral approach is confronted with *k-means* method, *Ward agglomerative clustering* and *Gaussian mixture model-based clustering* with Adjusted Rand Index (ARI) (Hubert & Arabie, 1985) used for clustering evaluation. The results are presented in [Figures 7.2 and 7.3](#). Values closer to one indicate a high level of real cluster structure discovery.

The clustering results on the `mlbench_spirals` dataset show significant variation across different methods, highlighting their effectiveness in handling complex, non-convex structures. Spectral clustering achieved a perfect ARI score of 1.000, indicating that it precisely captured the true underlying cluster structure of the dataset. In contrast, k-means and Gaussian mixture model-based clustering both scored around 0.000 or slightly negative, demonstrating their inability to correctly partition the data. This suggests that these methods struggle with the dataset's spiral-shaped clusters, likely due to their reliance on Euclidean distance, which does not perform well for non-linearly separable data.

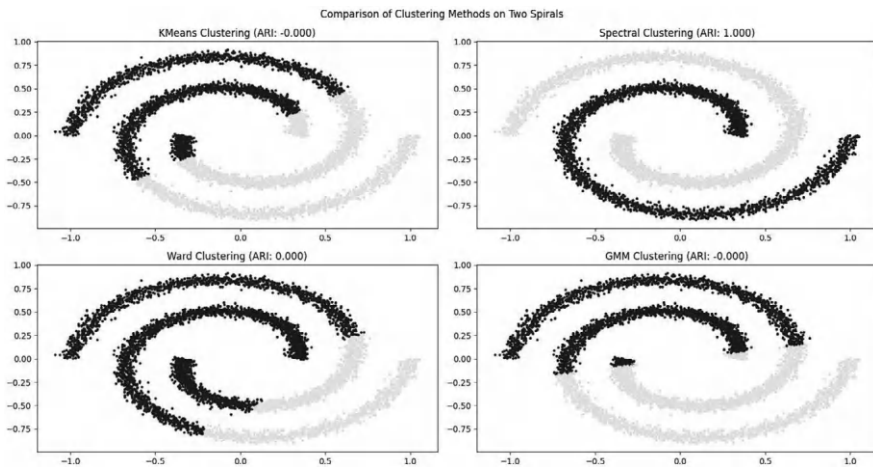


Figure 7.3 The results of `mlbench_spirals` dataset clustering with spectral clustering, k-means, Ward agglomerative clustering and Gaussian mixture model-based clustering (scatterplots with predicted cluster marked).

Source: Own calculations, with Python/scikit-learn/Matplotlib.

Similarly, Ward's agglomerative clustering also resulted in an ARI of 0.000, showing that it failed to detect the correct cluster structures. This is expected, as Ward's method is best suited for hierarchical clustering with compact, spherical clusters rather than intricate spiral patterns. The stark contrast between spectral clustering and the other methods suggests that graph-based approaches, which capture connectivity rather than simple distance metrics, are far more effective in handling such complex clustering scenarios. Thus, for datasets with non-linear structures like `mlbench_spirals`, spectral clustering emerges as the most suitable technique.

Spectral clustering applies clustering to a projection of the normalized Laplacian, which requires eigenvalues calculation in the fourth step of the above procedure. The practical impact of this step may be crucial and sometimes due to its complexity makes the procedure impossible to be executed on even the most powerful computer/tensor processing units (TPU) systems.¹

SpectralNet, proposed by [Shaham et al. \(2018\)](#), combines the strength of the spectral approach with the power of deep neural networks. SpectralNet is a neural network-based approach to spectral clustering that learns to assign unlabeled data points ($\mathcal{X} = \{x_1, \dots, x_n\} \subseteq R^d$) into k clusters. The objective is to learn a mapping ($F_\theta : R^d \rightarrow R^k$) that assigns points to clusters using a cluster assignment function ($c : R^k \rightarrow \{1, \dots, k\}$). SpectralNet generalizes spectral clustering by learning a parametric mapping that enables out-of-sample extensions for unseen data without recomputing similarities. The training process consists of three stages: (a) unsupervised learning of affinity through a Siamese network, (b) learning the spectral map (F_θ) by optimizing a spectral clustering objective with an orthogonality constraint, and (c) applying k-means clustering in the embedded space for final assignments.

7.1.2.1 Learning the spectral map (F_θ):

The central component of SpectralNet is learning the spectral map θ via an unsupervised loss that encourages similar points to be embedded close together. The spectral loss is defined as:

$$\mathcal{L}_{\text{SpectralNet}}(\theta) = E[w(x, x') | y - y']^2 \quad (7.1)$$

where $F_\theta(x)$ and $y' = F_\theta(x')$ are embeddings, and $w(x, x')$ is the affinity between points x and x' . To prevent trivial solutions where all embeddings collapse to a single point, SpectralNet enforces an orthogonality constraint:

$$E[yy^T] = I_{k \times k}. \quad (7.2)$$

In practice, these expectations are replaced by empirical estimates using minibatches. For a minibatch $X = \{x_1, \dots, x_m\}$, the empirical loss is:

$$L_{\text{SpectralNet}}(\theta) = \frac{1}{m^2} \sum_{i,j=1}^m W_{i,j} |y_i - y_j|^2, \quad (7.3)$$

where:

$W_{i,j} = w(x_i, x_j)$ is the affinity matrix

Y is the matrix of embeddings.

The orthogonality constraint for the minibatch is stated as:

$$\frac{1}{m} Y^T Y = I_{k \times k}. \quad (7.4)$$

To enforce this constraint, SpectralNet employs a linear output layer that orthogonalizes embeddings using the QR decomposition obtained via the Cholesky decomposition of $Y^T Y$.

SpectralNet approximates spectral clustering by learning embeddings that align with the eigenvectors of the graph Laplacian. The loss can be rewritten in terms of the unnormalized graph Laplacian $L = D - W$:

$$L_{\text{SpectralNet}}(\theta) = \frac{2}{m^2} \text{tr}(Y^T L Y), \quad (7.5)$$

where: (D) is the diagonal degree matrix with $D_{i,i} = \sum_j W_{i,j}$.

Minimizing this loss with an orthogonality constraint recovers the subspace spanned by the smallest k eigenvectors of the graph Laplacian, analogous to standard spectral clustering. SpectralNet can also use the normalized graph Laplacian $L = I - D^{-\frac{1}{2}} W D^{-\frac{1}{2}}$ with the modified loss:

$$L_{\text{SpectralNet}}(\theta) = \frac{1}{m^2} \sum_{i,j=1}^m W_{i,j} \left| \frac{y_i}{d_i} - \frac{y_j}{d_j} \right|^2, \quad (7.6)$$

where: $d_i = D_{i,i}$.

Unlike traditional spectral clustering, SpectralNet is scalable due to its stochastic training and ability to generalize to unseen data. It approximates the eigenvectors of the Laplacian while enabling efficient clustering through k-means on the learned embeddings.

The SpectralNet model will be used to cluster BigLink dataset (based on Chain-link dataset from the Fundamental Clustering and Projection Suite (FCPS) (Ullsch & Löttsch, 2020) with slight changes in direction and a higher number of objects). The visualization of this dataset is presented in Figure 7.4.

The `spectralnet` library used to be included in `pip` environment with the usual:

```
pip install spectralnet
```

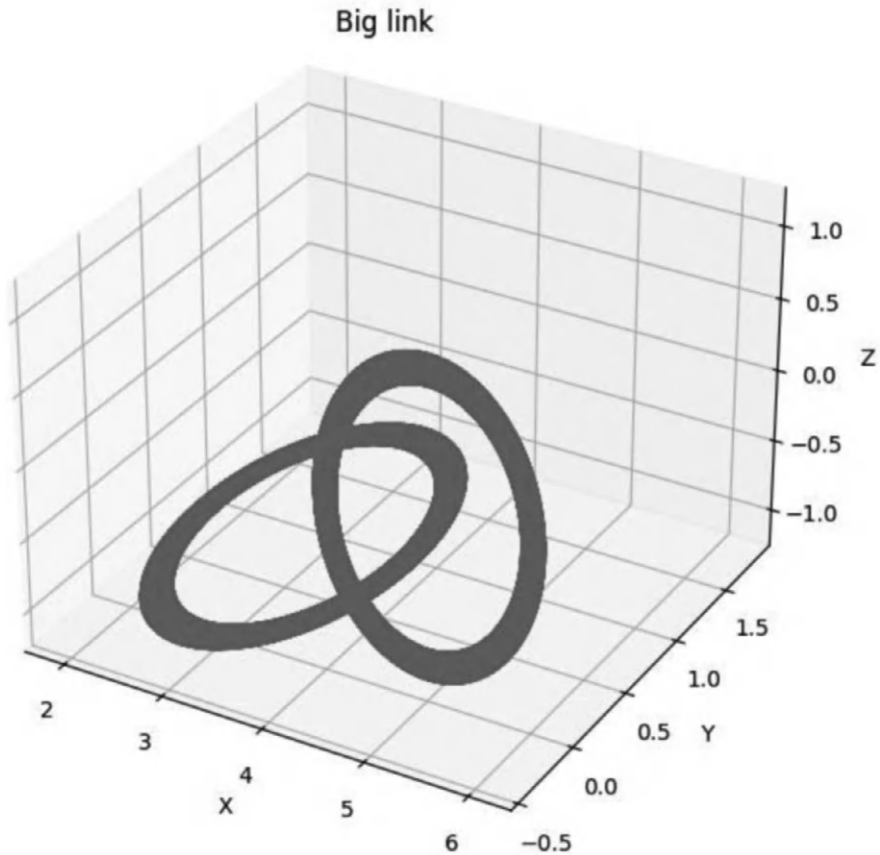


Figure 7.4 BigLink artificially generated dataset with 1,500,000 objects.

Source: Own work, with Python/Matplotlib based on R based on Chainlink dataset from FCPS (Ultsch & Lötsch, 2020).

but in case of problems (the author experienced such a situation) it may also be cloned from GitHub by:

```
git clone https://github.com/shaham-lab/SpectralNet.git
cd SpectralNet pip install -r requirements.txt
```

Next, creation of *setup.py* in the library directory:

```
from setuptools import setup, find_packages

setup(
    name='my_library',
    version='0.1',
```

```

    packages=find_packages(),
    install_requires=[],
)
pip install -e.

```

And then inclusion in the code:

```

import sys
sys.include("path to spectralnet")

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from spectralnet import SpectralNet
from sklearn.metrics import adjusted_rand_score
import torch

def generate_adjusted_perpendicular_rings(num_points,
radius, tube_radius, thickness=0.1, shift=(0, 0, 0)):
    theta1 = np.random.uniform(0, 2 * np.pi, num_points)
    r1 = np.random.uniform(tube_radius - thickness, tube_
radius + thickness, num_points)
    x1 = (radius + r1 * np.cos(theta1)) + shift[0]
    y1 = (r1 * np.sin(theta1)) + shift[1]
    z1 = np.zeros(num_points) + shift[2]
    labels1 = np.zeros(num_points)

    theta2 = np.random.uniform(0, 2 * np.pi, num_points)
    r2 = np.random.uniform(tube_radius - thickness, tube_
radius + thickness, num_points)
    x2 = (radius + r2 * np.cos(theta2))
    y2 = np.zeros(num_points)
    z2 = (r2 * np.sin(theta2))
    labels2 = np.ones(num_points)

    return (x1, y1, z1, labels1), (x2, y2, z2, labels2)

ring1, ring2 = generate_adjusted_perpendicular_rings(num_
points=1500000, radius=5, tube_radius=1, thickness=0.1,
shift=(-2, 0.7, -1))

x = np.concatenate([ring1[0], ring2[0]])
y = np.concatenate([ring1[1], ring2[1]])
z = np.concatenate([ring1[2], ring2[2]])
labels = np.concatenate([ring1[3], ring2[3]])

df_adjusted_rings = pd.DataFrame({'x': x, 'y': y, 'z': z,
'label': labels})

```

```

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')
scatter = ax.scatter(x, y, z, c=labels, cmap='coolwarm',
alpha=0.3, s=5, edgecolors='none')
ax.set_title('3D Rings ')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
plt.savefig("biglink.jpg")
plt.show()

```

```

X = df_adjusted_rings[['x', 'y', 'z']].values
y_true = df_adjusted_rings['label'].values

```

```

spectralnet = SpectralNet(
    n_clusters=2,
    should_use_ae=False,
    should_use_siamese=False,
    spectral_batch_size=500,
    spectral_epochs=10,
    spectral_is_local_scale=False,
    spectral_n_nbg=20,
    spectral_scale_k=2,
    spectral_lr=1e-3,
    spectral_lr_decay=1e-1,
    spectral_hiddens=[128]*3+[2],
)

```

```

spectralnet.fit(torch.tensor(X, dtype=torch.float32)) # X is
the dataset and it should be a torch.Tensor
y_pred = spectralnet.predict(torch.tensor(X, dtype=torch.float32))

```

```

ari = adjusted_rand_score(y_true, y_pred)
print(f"ARI (Adjusted Rand Index) for SpectralNet: {ari:.3f}")

```

```

fig = plt.figure(figsize=(10, 7))
ax = fig.add_subplot(111, projection='3d')
ax.scatter(X[:, 0], X[:, 1], X[:, 2], c=y_pred,
cmap='plasma', alpha=0.6, s=5, edgecolors='w')
ax.set_title(f'BigLink SpectralNet Clustering Results (ARI:
{ari:.3f})')
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_zlabel('Z')
plt.savefig("biglink_clustered.jpg")
plt.show()

```



```
Training SpectralNet:
Train Loss: 0.0000060, Valid Loss: 0.0000000, LR: 0.001000:
10%|█          | 1/10 [11:32<1:43:50, 692.24s/it]
ARI (Adjusted Rand Index) for SpectralNet: 1.000
```

Figure 7.5 BigLink SpectralNet clustering results.

Source: Own work, with Python/Matplotlib based on R based on Chainlink from FCPS (Ultsch & Lötsch, 2020).

Figures 7.5 and 7.6 present SpectralNet clustering results on BigLink dataset.

7.1.3 *Research application example: clustering with SpectralNet of crypto assets*

SpectralNet will be used to find partitions in the crypto currencies market for 1000 currencies with the biggest market capitalization.

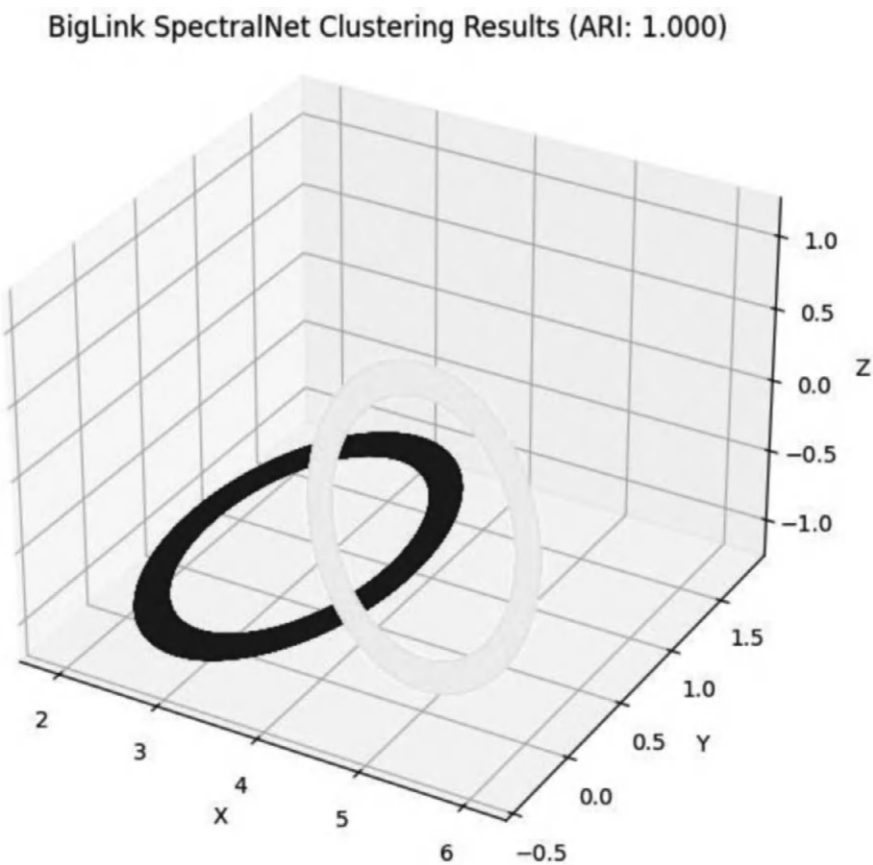


Figure 7.6 BigLink SpectralNet clustering results three-dimensional plot with clusters assigned.

Source: Own work, with Python/Matplotlib based on on Chainlink from FCPS (Ultsch & Lötsch, 2020).

For each cryptocurrency, the standard deviation normalized by mean of price, volume, and capacity will be calculated as variability measure, in this group linear regression slope coefficient as trend estimator is additionally used.

In price and capitalization group variables, bitcoin price and capitalization are certainly outliers, in fact measured on different scale. In this case normalization (MinMaxScaler) is not a rational choice (transformed bitcoin value will be equal to 1, while transformed values of different coins will be equal to 0). Instead of normalization, $\text{Log1p } \log(\hat{x}+1)$ will be used for mean price and mean capitalization.

We will be using free interface of [coingecko.com](https://www.coingecko.com) service. The maximal time range of this plan is limited to half a year, thus the time range of our analysis is set to second half of 2024.

Due to limitations of the free plan sometimes the following message:

```
Error: {'error': {'status': {'timestamp': '2025-02-19T05:40:07.985+00:00', 'error_code': 10012, 'error_message': 'Your request exceeds the allowed time range. Public API users are limited to querying historical data within the past 365 days. Upgrade to a paid plan to enjoy full historical data access: https://www.coingecko.com/en/api/pricing. '}}}
```

may appear in console, but the code just waits 100 seconds and then repeats the downloading attempt.

The data acquisition step of the analysis will be processed with the following Python code:

```
from datetime import datetime
import time
from scipy.stats import linregress
import os
import requests
import pandas as pd
import pickle
import numpy as np
from sklearn.neighbors import kneighbors_graph
from scipy.sparse.linalg import eigsh
import torch
from spectralnet import SpectralNet
from torchvision import datasets, transforms
import itertools
import matplotlib.pyplot as plt
import seaborn as sns

subdirectory = "cache"
os.makedirs(subdirectory, exist_ok=True)
random.seed(25032005)
# Define parameters for the API request
```

```

vs_currency = 'usd'
order = 'market_cap_desc'
per_page = 250
total_coins = 1000
pages = total_coins // per_page

all_coins = []

for page in range(1, pages + 1):
    url = f'https://api.coingecko.com/api/v3/coins/markets'
    params = {
        'vs_currency': vs_currency,
        'order': order,
        'per_page': per_page,
        'page': page
    }
    response = requests.get(url, params=params)
    data = response.json()
    all_coins.extend(data)

df_coins = pd.DataFrame(all_coins, columns=['id', 'symbol',
'name', 'market_cap', 'current_price'])

def get_historical_data(crypto_id, vs_currency, from_date,
to_date):
    url = f'https://api.coingecko.com/api/v3/coins/
{crypto_id}/market_chart/range'
    params = {
        'vs_currency': vs_currency,
        'from': str(int(from_date.timestamp())),
        'to': str(int(to_date.timestamp())),
    }

    response = requests.get(url, params=params)
    data = response.json()

    if 'prices' not in data:
        print("Error:", data)
        return None

    # Convert data to DataFrame
    df = pd.DataFrame(data['prices'], columns=['timestamp',
'price'])
    df['timestamp'] = pd.to_datetime(df['timestamp'], unit='ms')

    if 'market_caps' in data:
        df['market_cap'] = [item[1] for item in data['market_
caps']]
    if 'total_volumes' in data:

```

```

        df['volume'] = [item[1] for item in data['total_
            volumes']]

    return df

vs_currency = 'usd'
from_date = datetime(2024, 6, 1)
to_date = datetime(2024, 12, 31)
Std_of_Price = []
Std_of_Market_Cap = []
Std_of_Volume = []
Loglp_Price = []
Loglp_Market_Cap = []
linreg_slope_price = []
ommitted_coins=[]
for crypto_id in df_coins.id:
    print(crypto_id)
    if os.path.exists(f'cache/{crypto_id}.pkl'):
        with open(f'cache/{crypto_id}.pkl', 'rb') as f:
            df = pickle.load(f)
            from_cache = True
    else:
        df=None
        from_cache = False
        while df is None:
            df = get_historical_data(crypto_id, vs_currency,
from_date, to_date)
            if not df is None:
                with open(f'cache/{crypto_id}.pkl', 'wb') as f:
                    pickle.dump(df, f)
            else:
                time.sleep(100)
        if df.empty:
            ommitted_coins.append(crypto_id)
            continue

        if df['price'].mean()==0 or df['market_cap'].mean()==0
or df['volume'].mean()==0:
            ommitted_coins.append(crypto_id)
            continue

        std_price = df['price'].std()/df['price'].mean()
        std_market_cap = df['market_cap'].std()/df['market_
            cap'].mean()
        std_volume = df['volume'].std()/df['volume'].mean()

        loglp_price = np.loglp(df['price'].mean())
        loglp_market_cap = np.loglp(df['market_cap'].mean())
        slope, _, _, _ = linregress(range(len(df['price'])),
df['price'])

```

```

Std_of_Price+=[std_price]
Std_of_Market_Cap+=[std_market_cap]
Std_of_Volume+=[std_volume]
Loglp_Price+=[loglp_price]
Loglp_Market_Cap+=[loglp_market_cap]
linreg_slope_price+=[slope]

coin_characteristics=pd.DataFrame
    ({'Std_of_Price':Std_of_Price,
      'Std_of_Market_Cap':Std_of_Market_Cap,
      'Std_of_Volume':Std_of_Volume,
      'loglp_price':Loglp_Price,
      'loglp_market_cap':Loglp_Market_Cap,
      'linreg_slope_price':linreg_slope_price})
if not from_cache:
    time.sleep(10)
coin_characteristics.index=[not_ommitted for not_ommitted in
df_coins.id if not_ommitted not in ommitted_coins]

```

To find the adequate number of clusters the eigenvalues gap method will be used.

```

n_neighbors = 10
adjacency_matrix = kneighbors_graph(coin_characteristics, n_
neighbors, mode='connectivity').toarray()

degree_matrix = np.diag(np.sum(adjacency_matrix, axis=1))

laplacian = degree_matrix - adjacency_matrix
eigenvalues, _ = eigsh(laplacian, k=10, which='SM')

import matplotlib.pyplot as plt
plt.plot(range(1, len(eigenvalues) + 1), eigenvalues,
marker='o')
plt.xlabel("Index")
plt.ylabel("Eigenvalue")
plt.title("Eigenvalue Spectrum")
plt.savefig("eigenvalue_gap.jpg")
plt.show()

```

To estimate the number of clusters, the eigenvalues gap method will be used. The eigenvalues are presented in [Figure 7.7](#).

The biggest gap is related to the eighth eigenvalue, thus the number of clusters for SpectralNet model is set to eight.

```

nr_clusters=8

X_t = torch.cat([torch.tensor(np.array(coin_characteristics),
dtype=torch.float32)])

```

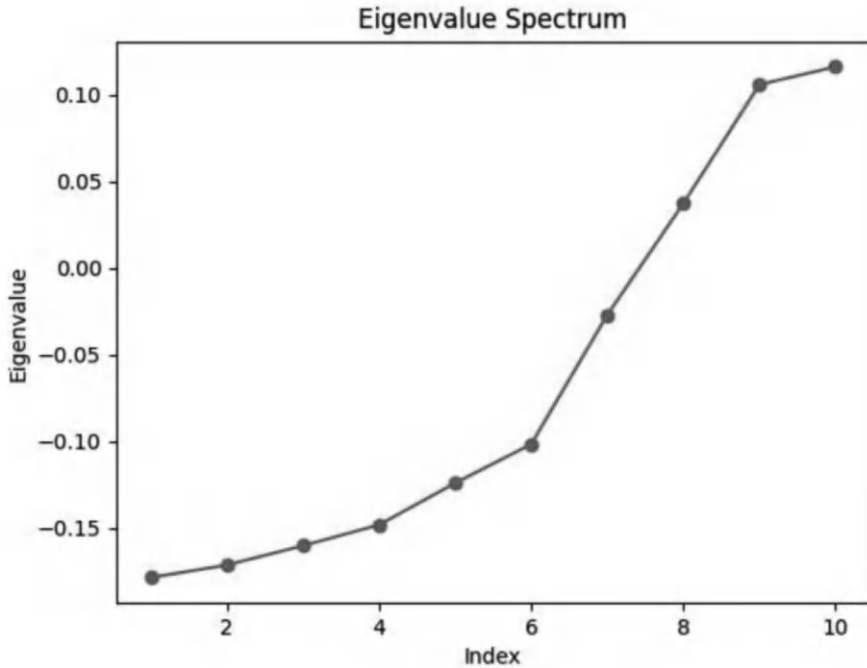


Figure 7.7 Eigenvalues gap for cryptocurrencies analysis.

Source: Own work, with Python/Matplotlib.

```
spectralnet = SpectralNet(
    n_clusters=nr_clusters,
    should_use_ae=False,
    should_use_siamese=False,
    spectral_batch_size=500,
    spectral_epochs=10,
    spectral_is_local_scale=False,
    spectral_n_nbg=20,
    spectral_scale_k=2,
    spectral_lr=1e-3,
    spectral_lr_decay=1e-1,
    spectral_hiddens=[128]*3+[8],
)

spectralnet.fit(X_t) # X is the dataset and it should be a
torch.Tensor
predicted_clusters = spectralnet.predict(X_t)

print(predicted_clusters)
# Visualize the true labels vs predicted clusters
```

```

plt.figure(figsize=(6, 6))
plt.scatter(coin_characteristics.iloc[:,2], coin_
characteristics.iloc[:,5], c=predicted_clusters,
cmap='viridis', s=50)
plt.title('Predicted Clusters')
plt.savefig("coins_clusters.jpg")
plt.show()

variable_combinations = list(itertools.combinations(coin_
characteristics.columns, 2))

num_combinations = len(variable_combinations)
num_cols = 4 # Number of columns in the grid layout
num_rows = (num_combinations + num_cols - 1) // num_cols #
Compute rows needed

fig, axes = plt.subplots(num_rows, num_cols, figsize=(15, num_
rows * 5))
axes = axes.flatten()

for i, (var_x, var_y) in enumerate(variable_combinations):
    ax = axes[i]
    sns.scatterplot(x=coin_characteristics[var_x],
y=coin_characteristics[var_y], hue=predicted_clusters,
palette="viridis", ax=ax, edgecolor='none', alpha=0.7)
    ax.set_xlabel(var_x)
    ax.set_ylabel(var_y)
    ax.set_title(f"{var_x} vs {var_y}")

for j in range(i + 1, len(axes)):
    fig.delaxes(axes[j])

plt.tight_layout()
plt.savefig('Predicted_Clusters_all.jpg')
plt.show()

for i in range(8):
    print(f"Cluster {i+1} characteristics")
    print(coin_characteristics.index[predicted_clusters == i])
    ticker=coin_characteristics.index[predicted_clusters ==
    i][0]
    print("min, max, mean, sd")
    print(coin_characteristics.iloc[predicted_clusters ==
i,:].min())
    print(coin_characteristics.iloc[predicted_clusters ==
i,:].max())
    print(coin_characteristics.iloc[predicted_clusters ==
i,:].mean())

```

```

    print(coin_characteristics.iloc[predicted_clusters ==
i,:].std())
    with open(f'cache/{ticker}.pkl', 'rb') as f:
        df = pickle.load(f)
    plt.figure(figsize=(12, 6))
    plt.plot(df["timestamp"], df["price"], label='Historical
Prices', color='blue')
    plt.title(f"Cluster {i+1} {ticker} ")
    plt.xlabel("Date")
    plt.ylabel(f"{ticker} price in USD")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(f"cluster{i+1}_{ticker}_time_series.jpg")
    plt.show()

for i in range(8):
    print(f"Cluster {i+1} characteristics")
    print(coin_characteristics.index[predicted_clusters == i])
    ticker=coin_characteristics.index[predicted_clusters ==
i][0]
    print("min, max, mean, sd")
    print(coin_characteristics.iloc[predicted_clusters ==
i,:].min())
    print(coin_characteristics.iloc[predicted_clusters ==
i,:].max())
    print(coin_characteristics.iloc[predicted_clusters ==
i,:].mean())
    print(coin_characteristics.iloc[predicted_clusters ==
i,:].std())
    with open(f'cache/{ticker}.pkl', 'rb') as f:
        df = pickle.load(f)
    plt.figure(figsize=(12, 6))
    plt.plot(df["timestamp"], df["price"], label='Historical
Prices', color='blue')
    plt.title(f"Cluster {i+1} {ticker} ")
    plt.xlabel("Date")
    plt.ylabel(f"{ticker} price in USD")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.savefig(f"cluster{i+1}_{ticker}_time_series.jpg")
    plt.show()

```

The results of the analysis are presented in [Figures 7.8](#) and [7.9](#).

The clustering results suggest distinct groupings of cryptocurrencies based on their characteristics such as price volatility, market capitalization, and volume trends. Cluster 1 includes large, well-established cryptocurrencies like Ethereum,

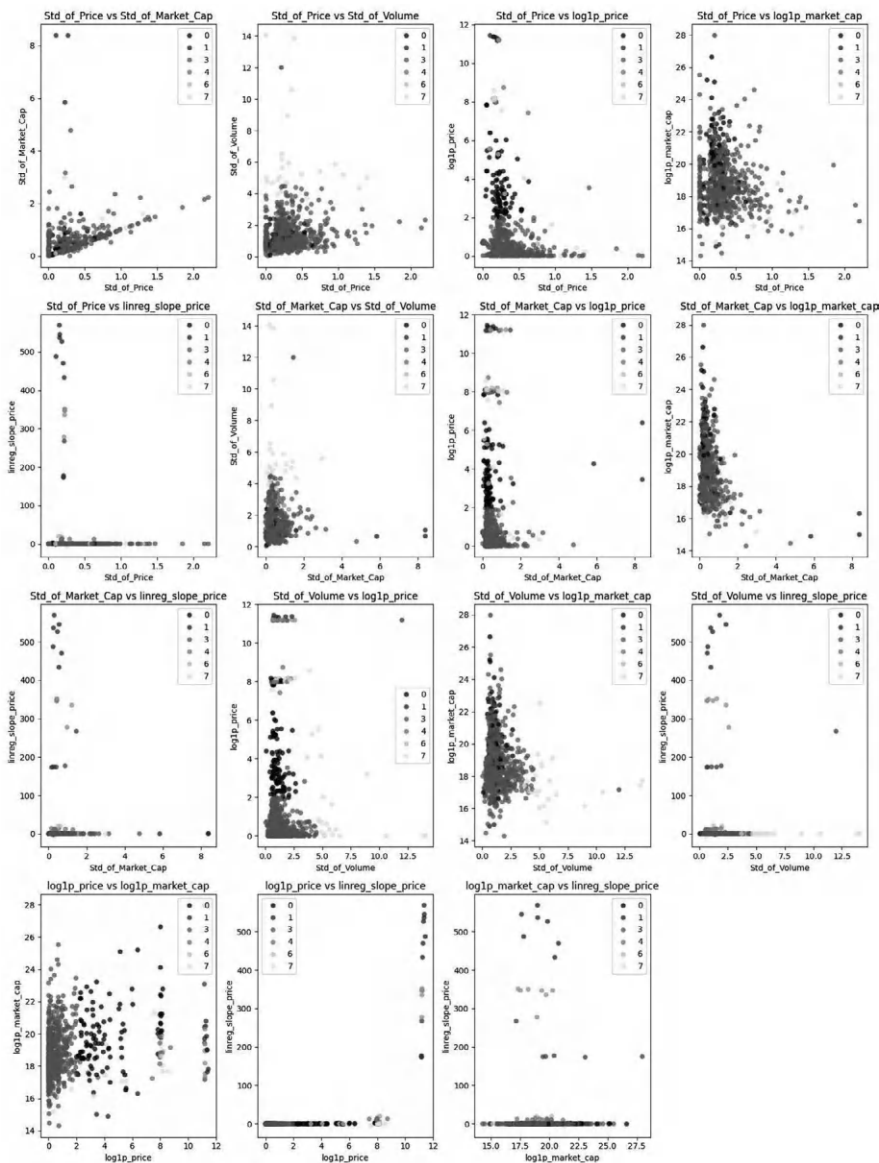


Figure 7.8 Cryptocurrencies analysis scatterplots with cluster structure.

Source: Own work, with Python/Matplotlib.

Binance Coin, and Solana, which exhibit relatively stable prices and high market capitalizations, indicating a mature asset class. Cluster 2 consists of various wrapped Bitcoin versions and other pegged assets, demonstrating a unique market

Cluster 1 characteristics

```
Index(['ethereum', 'binancecoin', 'solana', 'staked-ether',
      'chainlink',
      'wrapped-steth', 'litecoin', 'avalanche-2', 'weth',
      'hyperliquid',
      ...,
      'socean-staked-sol', 'binance-peg-avalanche',
      'gogopool', 'spectral',
      'snowbank', 'kyros-restaked-sol', 'tokenbot-2',
      'melon', 'alchemix',
      'veraone'],
      dtype='object', length=104)
```

```
min, max, mean, sd
```

```
Std_of_Price          0.042492
```

```
Std_of_Market_Cap     0.042664
```

```
Std_of_Volume         0.093018
```

```
loglp_price           1.997617
```

```
loglp_market_cap      16.352498
```

```
linreg_slope_price    -1.841865
```

```
dtype: float64
```

```
Std_of_Price          0.634821
```

```
Std_of_Market_Cap     1.599330
```

```
Std_of_Volume         2.664857
```

```
loglp_price           8.181305
```

```
loglp_market_cap      26.623325
```

```
linreg_slope_price    2.079796
```

```
dtype: float64
```

```
Std_of_Price          0.232847
```

```
Std_of_Market_Cap     0.317503
```

```
Std_of_Volume         1.109372
```

```
loglp_price           4.247807
```

```
loglp_market_cap      19.966387
```

```
linreg_slope_price    0.125568
```

```
dtype: float64
```

```
Std_of_Price          0.109549
```

```
Std_of_Market_Cap     0.239469
```

```
Std_of_Volume         0.495968
```

```
loglp_price           1.963824
```

```
loglp_market_cap      1.909218
```

```
linreg_slope_price    0.497186
```

```
dtype: float64
```

Cluster 2 characteristics

```
Index(['coinbase-wrapped-btc', 'lombard-staked-btc', 'wbnb',
      'ether-fi-staked-btc', 'abtc', 'wrapped-avax', 'ipmb',
      'lorenzo-stbtc'],
```

Figure 7.9 Cryptocurrencies analysis cluster belongingness and characteristics. (Continued)

Source: Own work, Python/Matplotlib.

```

    'general-layer-btc', 'solv-protocol-solvbtc-core'],
    dtype='object')
min, max, mean, sd
Std_of_Price          0.102553
Std_of_Market_Cap     0.237292
Std_of_Volume         0.640148
loglp_price           3.438830
loglp_market_cap      14.888823
linreg_slope_price    -0.153556
dtype: float64
Std_of_Price          0.267991
Std_of_Market_Cap     8.387271
Std_of_Volume         2.344075
loglp_price           11.444656
loglp_market_cap      20.788615
linreg_slope_price    569.258617
dtype: float64
Std_of_Price          0.175563
Std_of_Market_Cap     2.563974
Std_of_Volume         1.110240
loglp_price           9.348148
loglp_market_cap      18.072756
linreg_slope_price    356.951181
dtype: float64
Std_of_Price          0.053516
Std_of_Market_Cap     3.503810
Std_of_Volume         0.551173
loglp_price           3.290961
loglp_market_cap      2.135388
linreg_slope_price    249.293861
dtype: float64
Cluster 3 characteristics
Index(['bitcoin', 'wrapped-bitcoin', 'solv-btc',
      'bitcoin-avalanche-bridged-btc-b', 'tbtc',
      'ignition-fbtc'],
      dtype='object')
min, max, mean, sd
Std_of_Price          0.205466
Std_of_Market_Cap     0.160108
Std_of_Volume         0.675636
loglp_price           11.168296
loglp_market_cap      17.162375
linreg_slope_price    173.156743
dtype: float64
Std_of_Price          0.216934
Std_of_Market_Cap     1.454414
Std_of_Volume         11.993550

```

Figure 7.9 (Continued)

```

loglp_price          11.195209
loglp_market_cap     27.971283
linreg_slope_price   267.328802
dtype: float64
Std_of_Price         0.208058
Std_of_Market_Cap    0.564346
Std_of_Volume        2.983641
loglp_price          11.174768
loglp_market_cap     21.296024
linreg_slope_price   190.266755
dtype: float64
Std_of_Price         0.004377
Std_of_Market_Cap    0.504454
Std_of_Volume        4.439908
loglp_price          0.010117
loglp_market_cap     3.782669
linreg_slope_price   37.775636
dtype: float64
Cluster 4 characteristics
Index(['ripple', 'tether', 'usd-coin', 'dogecoin',
      'cardano', 'tron',
      'stellar', 'sui', 'shiba-inu', 'usds',
      ...,
      'pocket-network', 'retardio', 'orion-protocol',
      'coinweb',
      'advertise-coin', 'tether-eurt', 'spring-staked-sui',
      'gitcoin',
      'amaterasu-omikami', 'richquack'],
      dtype='object', length=748)
min, max, mean, sd
Std_of_Price          5.555237e-04
Std_of_Market_Cap     1.204728e-15
Std_of_Volume         3.377200e-02
loglp_price           7.856199e-10
loglp_market_cap      1.429111e+01
linreg_slope_price    -1.562692e-01
dtype: float64
Std_of_Price          2.203349
Std_of_Market_Cap     4.772265
Std_of_Volume         4.477927
loglp_price           3.533130
loglp_market_cap      25.516667
linreg_slope_price     0.445634
dtype: float64
Std_of_Price          0.316971
Std_of_Market_Cap     0.422609
Std_of_Volume         1.286984

```

Figure 7.9 (Continued)

```

loglp_price           0.402223
loglp_market_cap      18.639629
linreg_slope_price     0.001464
dtype: float64
Std_of_Price          0.244220
Std_of_Market_Cap     0.380572
Std_of_Volume         0.789786
loglp_price           0.483523
loglp_market_cap      1.399061
linreg_slope_price     0.020352
dtype: float64
Cluster 5 characteristics
Index(['binance-peg-weth', 'l2-standard-bridged-weth-base',
      'arbitrum-bridged-weth-arbitrum-one', 'super-oeth',
      'polygon-pos-bridged-weth-polygon-pos', 'yearn-finance',
      'l2-standard-bridged-weth-blast', 'l2-standard-bridged-
      weth-optimism',
      'kelp-gain', 'renzo-restaked-lst', 'avalanche-bridged-
      weth-avalanche',
      'ronin-bridged-weth-ronin', 'meta-2'],
      dtype='object')
min, max, mean, sd
Std_of_Price          0.170547
Std_of_Market_Cap     0.154297
Std_of_Volume         0.554251
loglp_price           7.429947
loglp_market_cap      17.236226
linreg_slope_price     4.902228
dtype: float64
Std_of_Price          0.627789
Std_of_Market_Cap     1.275386
Std_of_Volume         2.266431
loglp_price           8.733918
loglp_market_cap      21.171996
linreg_slope_price     14.191371
dtype: float64
Std_of_Price          0.222693
Std_of_Market_Cap     0.430188
Std_of_Volume         1.100530
loglp_price           8.003579
loglp_market_cap      19.310877
linreg_slope_price     10.862219
dtype: float64
Std_of_Price          0.125527
Std_of_Market_Cap     0.312014
Std_of_Volume         0.549612
loglp_price           0.272865

```

Figure 7.9 (Continued)

```

loglp_market_cap      1.023111
linreg_slope_price    2.224905
dtype: float64
Cluster 6 characteristics
Index(['solv-protocol-solvbtc-bbn', 'arbitrum-bridged-
      wbtc-arbitrum-one',
      'pumpbtc', 'polygon-bridged-wbtc-polygon-pos',
      'optimism-bridged-wbtc-optimism',
      'avalanche-old-bridged-wbtc-avalanche', 'cronos-
      bridged-wbtc-cronos'],
      dtype='object')
min, max, mean, sd
Std_of_Price          0.214934
Std_of_Market_Cap     0.410027
Std_of_Volume         0.668278
loglp_price           11.194948
loglp_market_cap      17.332843
linreg_slope_price    277.685163
dtype: float64
Std_of_Price          0.223831
Std_of_Market_Cap     1.217993
Std_of_Volume         2.591209
loglp_price           11.210932
loglp_market_cap      20.283177
linreg_slope_price    351.427934
dtype: float64
Std_of_Price          0.221592
Std_of_Market_Cap     0.615361
Std_of_Volume         1.410528
loglp_price           11.207707
loglp_market_cap      18.766938
linreg_slope_price    336.537220
dtype: float64
Std_of_Price          0.003045
Std_of_Market_Cap     0.335099
Std_of_Volume         0.786934
loglp_price           0.005810
loglp_market_cap      1.120665
linreg_slope_price    26.455172
dtype: float64
Cluster 7 characteristics
Index(['mantle-restaked-eth', 'treehouse-eth'], dtype='object')
min, max, mean, sd
Std_of_Price          0.139557
Std_of_Market_Cap     0.479642
Std_of_Volume         1.249157
loglp_price           8.157081

```

Figure 7.9 (Continued)

```

loglp_market_cap      18.921923
linreg_slope_price    19.425527
dtype: float64
Std_of_Price          0.170528
Std_of_Market_Cap     0.615038
Std_of_Volume         1.986769
loglp_price           8.197533
loglp_market_cap      20.194832
linreg_slope_price    19.797371
dtype: float64
Std_of_Price          0.155042
Std_of_Market_Cap     0.547340
Std_of_Volume         1.617963
loglp_price           8.177307
loglp_market_cap      19.558377
linreg_slope_price    19.611449
dtype: float64
Std_of_Price          0.021900
Std_of_Market_Cap     0.095740
Std_of_Volume         0.521571
loglp_price           0.028603
loglp_market_cap      0.900082
linreg_slope_price    0.262933
dtype: float64
Cluster 8 characteristics
Index(['leo-token', 'maker', 'snek', 'stakewise-v3-oseth',
      'liquid-staked-ethereum', 'staked-frax-ether',
      'bridged-wrapped-ether-starkgate', 'wrapped-beacon-eth',
      'wrapped-ether-mantle-bridge', 'constellation-labs',
      'reserve-protocol-eth-plus', 'compound-ether', 'origin-
      ether',
      'global-dollar', 'centrifuge', 'bridged-wrapped-steth-
      gnosis',
      'tokamak-network', 'universal-eth', 'wrapped-xrp',
      'taraxa', 'carry',
      'societe-generale-forge-eurcv', 'edge', 'laine-stake',
      'whiteheart',
      'thena', 'compound-usd-coin', 'tornado-cash', 'ankreth',
      'token-pocket',
      'defiway', 'sportpoint', 'nftx', 'hairdao', 'pinksale'],
      dtype='object')
min, max, mean, sd
Std_of_Price          0.005413
Std_of_Market_Cap     0.096257
Std_of_Volume         0.619661
loglp_price           0.001952
loglp_market_cap      15.153351
linreg_slope_price    -5.549828

```

Figure 7.9 (Continued)

```

dtype: float64
Std_of_Price          1.404842
Std_of_Market_Cap     2.974969
Std_of_Volume         14.035895
loglp_price           8.573635
loglp_market_cap      22.522161
linreg_slope_price    0.478488
dtype: float64
Std_of_Price          0.338573
Std_of_Market_Cap     0.589979
Std_of_Volume         4.586892
loglp_price           3.862565
loglp_market_cap      18.000035
linreg_slope_price    -0.185289
dtype: float64
Std_of_Price          0.345944
Std_of_Market_Cap     0.583900
Std_of_Volume         3.288582
loglp_price           3.587500
loglp_market_cap      1.470272
linreg_slope_price    0.948125
dtype: float64

```

Figure 7.9 (Continued)

behavior with relatively high price stability, but significant differences in volume and market cap fluctuations.

Cluster 3 stands out as it contains Bitcoin and its derivatives, characterized by very high prices and market capitalization with substantial price movements over time. Cluster 4, the largest group, contains a diverse mix of altcoins, stablecoins, and tokens with low price volatility but widely varying market caps. This suggests a heterogeneous cluster where different fundamental characteristics still lead to similar market behavior. In contrast, Cluster 5 includes various wrapped Ethereum tokens, DeFi assets, and staking derivatives, exhibiting moderate volatility and higher price trends.

Another notable group is Cluster 6, which contains Bitcoin-backed assets and derivatives, but with distinct market behavior compared to Cluster 3. These assets show significantly lower volatility and more predictable price movements. Cluster 7 is a small but distinct cluster consisting of Mantle-restaked ETH and Treehouse-ETH, suggesting assets focused on staking or restaking protocols. Finally, Cluster 8 includes a mix of governance tokens, DeFi assets, and wrapped assets, characterized by highly varying volume fluctuations and market capitalization changes, possibly indicating assets with lower liquidity and higher speculation.

Overall, the clustering captures clear distinctions between native assets, wrapped assets, stablecoins, staking derivatives, and governance tokens. Notable clusters include Bitcoin's dominance in Cluster 3, Ethereum's strong positioning in Cluster 1, and stablecoins and low-volatility assets grouping in Cluster 4. The presence of smaller, highly specific clusters such as Cluster 7 suggests the emergence of niche financial instruments within the crypto ecosystem.

The time series plot for representatives of Clusters 3, 4, 8 are presented in [Figures 7.10–7.12](#).

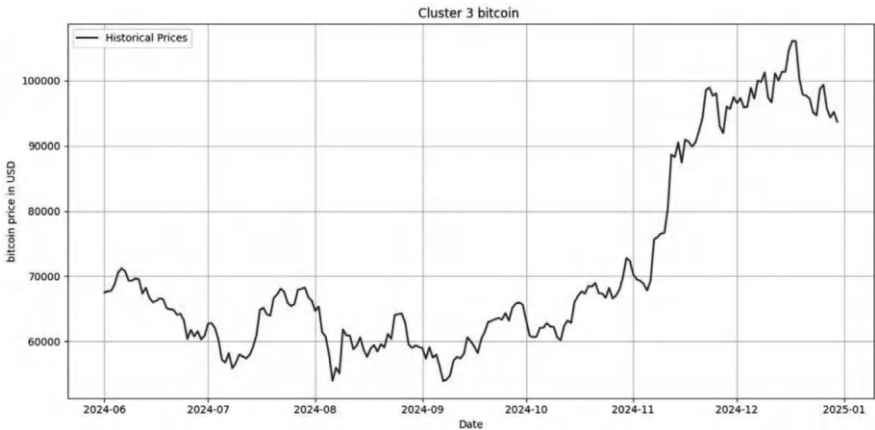


Figure 7.10 Bitcoin time series plot as representative of Cluster 3.

Source: Own work, with Python/Matplotlib.

7.2 Recommender systems based on neural networks

7.2.1 History of recommender systems: content-based filtering, matrix factorization techniques, collaborative filtering, hybrid techniques

Recommender systems are software tools and techniques providing suggestions for items to be of use to a user. The suggestions provided are aimed at supporting their users in various decision-making processes, such as what items to buy, what music to listen to, or what news to read (Ricci et al., 2021, p. viii).

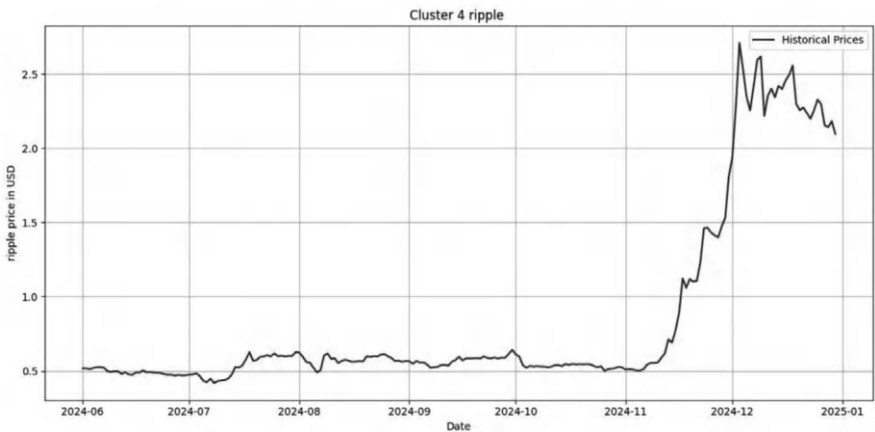


Figure 7.11 Ripple bitcoin time series plot as representative of Cluster 4.

Source: Own work, with Python/Matplotlib.

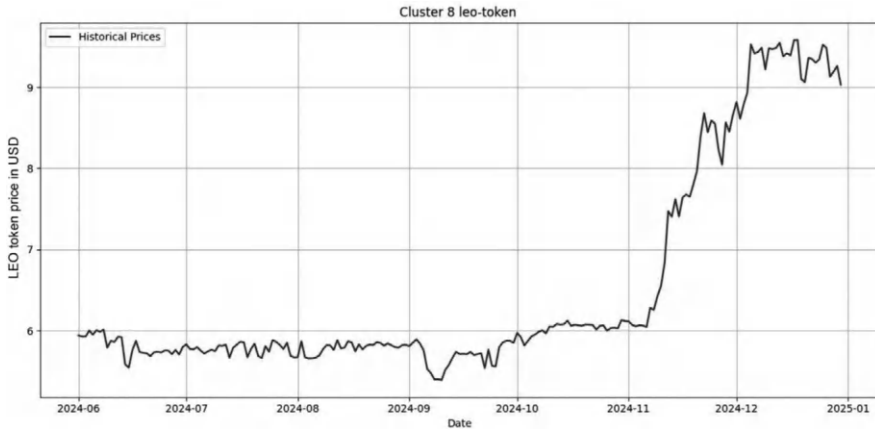


Figure 7.12 LEO Token time series plot as representative of Cluster 8.

Source: Own work, with Python/Matplotlib.

The first recommendation system was probably fully analog: librarians recommending books based on personal expertise and rental history. Since then, several types of recommender systems have been developed.

7.2.1.1 Content-based filtering

Content-based filtering (see, e.g., Pazzani, 2007) recommends items similar to those the user has liked in the past, based on item attributes. For instance, if a user likes a particular movie, the system will recommend movies with similar genres, actors, directors, etc.

7.2.1.2 Collaborative filtering

Collaborative filtering (Herlocker et al., 2004; Linden et al., 2003) recommends items based on the preferences of other users. It assumes that if users agreed on the past, they will agree again in the future. Two main types of collaborative filtering may be distinguished:

- **User-based collaborative filtering:** Finds users similar to the target user and recommends items they liked.
- **Item-based collaborative filtering:** Finds items similar to those the user has liked and recommends them.

7.2.1.3 Matrix factorization techniques

Matrix factorization techniques, such as singular value decomposition (see Stewart, 1993), decompose the user–item interaction matrix into the product of two

lower-dimensional matrices. These techniques are particularly effective in dealing with large, sparse datasets. Netflix Prize winners used matrix factorization techniques to predict user ratings for films (Koren et al., 2009).

7.2.1.4 Hybrid recommender systems

Hybrid recommender systems combine two or more recommendation techniques to improve accuracy and robustness. These systems can mixed content-based, collaborative filtering, and other techniques to leverage their complementary strengths. A hybrid system may use collaborative filtering to identify similar users and content-based filtering to recommend items based on those users' profiles (Burke, 2002).

7.2.1.5 Deep learning-based recommender systems

Salakhutdinov et al. (2007) proposed a Restricted Boltzmann Machine-based recommender system. Sedhain et al. (2015) introduced AutoRec, an autoencoder-based collaborative filtering recommendation method. This model takes either user- or item-based ratings from the rating matrix R as input. An encoding–decoding process generates output while optimizing model parameters to minimize reconstruction errors. Wu et al. (2016) used a recurrent neural network to model the effect of user historical sequence behavior on the user's current behavior, then recommended items for the user and predicted the user's behavior (cf. Mu, 2018).

He et al. (2017) proposed neural collaborative filtering (NCF), which will be discussed in section 7.2.2.

Evaluation measures of recommender systems are similar to, but slightly different from measures used in discriminant analysis. The most commonly used ones are mean average precision (MAP), normalized discounted cumulative gain (NDCG@K), Precision@K, and Recall@K.

MAP evaluates the ranking quality of recommended items by averaging the precision scores at the positions where relevant items appear. Higher MAP values indicate better ranking of relevant items. MAP is given by formula 7.7.

$$MAP = \frac{1}{|U|} \sum_{u \in U} AP(u) \quad (7.7)$$

where:

Average precision (AP) for a user u is:

$$AP(u) = \frac{1}{|R_u|} \sum_{k=1}^N P@k \cdot rel_k$$

$|U|$ – total number of users;

R_u – relevant items for user u ;

$P@k$ – precision at position k ;
 rel_k – 1 if the item at position k is relevant, otherwise 0.

NDCG@K (see He et al., 2015) measures ranking quality by assigning higher importance to correctly recommended relevant items appearing earlier in the recommendation list. $NDCG@K = 1$ means a perfect ranking. Higher values indicate better ranking of relevant items.

$$NDCG@K = \frac{DCG@K}{IDCG@K} \quad (7.8)$$

where:

$DCG@K = \sum_{i=1}^K \frac{rel_i}{\log_2(i+1)}$ - discounted cumulative gain (DCG);

$IDCG@K = \sum_{i=1}^{|REL_K|} \frac{1}{\log_2(i+1)}$ ideal DCG (IDCG) is computed by sorting the ideal

ranking of relevant items;

rel_i – relevance score (typically binary: 1 if relevant, 0 otherwise);

K = cutoff for ranking.

Precision@K measures how many of the top- K recommended items are actually relevant. Higher precision means more relevant items in the top- K recommendations.

$$Precision@K = \frac{|\text{relevant items in top } K|}{K} \quad (7.9)$$

where:

K – number of recommendations considered;

relevant items – items that are actually preferred by the user.

Recall@K measures how many of the relevant items were successfully recommended in the top K results. Higher recall means the model retrieves more of the relevant items. $Precision@K$ and $Recall@K$ are complementary: increasing K improves recall but may lower precision.

$$Recall@K = \frac{|\text{relevant items in top } K|}{|\text{total relevant items}|} \quad (7.10)$$

where:

$|\text{total relevant items}|$ – total relevant items available.

7.2.1.6 Neural collaborative filtering

NCF, introduced by He et al. (2017) is a deep learning-based recommendation system that extends traditional collaborative filtering by incorporating nonlinear

interactions between users and items. NCF employs neural networks to learn user-item interactions from implicit feedback. It uses the neural matrix factorization (NeuMF) model in which generalized matrix factorization and a multi-layer perceptron to enhance recommendation quality.

Let:

- u be a user;
- i be an item (book, movie, etc.);
- y_{ui} be the implicit feedback (e.g., 1 if a user interacted with an item, 0 otherwise);
- p_u and q_i be the user and item embeddings, respectively.

7.2.1.7 Generalized matrix factorization

NCF extends matrix factorization with a neural network. Instead of computing user-item interactions using a simple dot product:

$$\hat{y}_{ui} = p_u^T q_i \quad (7.11)$$

Generalized matrix factorization applies a learnable function f parameterized by Θ

$$\hat{y}_{ui} = f(p_u, q_i; \Theta) \quad (7.12)$$

where: f can be any differentiable function

Users and items are embedded into a latent space. A single-layer neural network applies a transformation. The output is a predicted interaction score in range 0-1.

Instead of computing user-item interactions via dot-product, NCF concatenates the embeddings and feeds them into a multi-layer perceptron:

$$\begin{aligned} z_{ui} &= \text{concat}(p_u, q_i) \\ h_1 &= \sigma(W_1 z_{ui} + b_1) \\ h_2 &= \sigma(W_2 h_1 + b_2) \\ &\vdots \\ h_L &= \sigma(W_L h_{L-1} + b_L) \\ \hat{y}_{ui} &= \sigma(W_{out} h_L + b_{out}) \end{aligned} \quad (7.13)$$

where:

h_l is the hidden layer at level l ;

W_l, b_l are the weights and biases;

$\sigma(x)$ is an activation function;

$\hat{y}_{ui}$ is the final predicted interaction score.

NCF implementation with NeuMF merges general matrix factorization and multi-layer perceptron:

$$\begin{aligned} z_{ui} &= \text{concat}(z_{ui}^{GMF}, h_L^{MLP}) \\ \hat{y}_{ui} &= \sigma(W_{final} z_{ui} + b_{final}) \end{aligned} \quad (7.14)$$

NCF is trained using binary cross-entropy loss, as implicit feedback is often binary (recommended/not recommended, liked/not liked):

$$\mathcal{L} = - \sum_{(u,i) \in \mathcal{D}} \left[y_{ui} \log \hat{y}_{ui} + (1 - y_{ui}) \log (1 - \hat{y}_{ui}) \right] \quad (7.15)$$

where:

$y_{\{ui\}}$ is the actual feedback (1 for interaction, 0 otherwise);

$\hat{y}_{ui}$ is the predicted probability.

The model is optimized using stochastic gradient descent or ADAM optimizer.

7.2.2 Research application example: NCF system for bookstore

In this example, the NCF algorithm will be used to create recommendations for a book evaluation and recommender system.

The dataset used in research, available at <https://github.com/rochitasundar/Col-laborative-Filtering-Book-Recommendation-System>, is derived from the Book-Crossing dataset, originally compiled by Ziegler et al. (2005). This comprehensive dataset encompasses 1,149,780 ratings pertaining to 271,379 books, contributed by 278,858 users. The ratings are both explicit, ranging from 1 to 10, and implicit, indicating user interactions without a specific rating. The dataset also includes demographic details of users, such as age and location, enhancing its utility for personalized recommendation systems.

The dataset is organized into three primary tables:

- 1 **Users:** Contains anonymized user IDs along with demographic information like age and location (not used in the research analysis)
- 2 **Books:** Provides details about books, including ISBN, title, author, publication year, publisher, and URLs for the book's cover images in varying sizes.
- 3 **Book-ratings:** Records the interaction between users and books, capturing both explicit ratings (1–10 scale) and implicit interactions (denoted by a rating of 0).

The analysis is narrowed to the first 12,345 items due to numerical complexity (analysis on full dataset took over three days even on four graphics processing units (GPUs) but readers with “stronger” GPU/Tensor processing unit (TPU) systems

may just remove the line `recommender_data=recommender_data.iloc[:12345,:]`).

```
import os
import tensorflow as tf
import sys
import pandas as pd
import tensorflow as tf
from sklearn.model_selection import train_test_split
from recommenders.utils.timer import Timer
from recommenders.models.ncf.ncf_singlenode import NCF
from recommenders.models.ncf.dataset import Dataset as
    NCFDataset
from recommenders.datasets import movielens
from recommenders.datasets.python_splitters import
python_chrono_split
from recommenders.evaluation.python_evaluation import (
    map, ndcg_at_k, precision_at_k, recall_at_k
)
import requests

urls = {
    "BX-Book-Ratings.csv": "https://github.com/rochitasundar/
Collaborative-Filtering-Book-Recommendation-System/raw/refs/
heads/master/BX-Book-Ratings.csv",
    "BX-Books.csv": "https://github.com/rochitasundar/
Collaborative-Filtering-Book-Recommendation-System/raw/refs/
heads/master/BX-Books.csv"
}

# Directory to save the files
save_dir = "data/recommender"
os.makedirs(save_dir, exist_ok=True)

def download_file(url, save_path):
    response = requests.get(url, stream=True)
    if response.status_code == 200:
        with open(save_path, "wb") as file:
            for chunk in response.iter_content(chunk_
                size=1024):
                file.write(chunk)
            print(f"Downloaded: {save_path}")
    else:
        print(f"Failed to download {url}")

# Download each file
for file_name, url in urls.items():
    file_path = os.path.join(save_dir, file_name)
```

```

download_file(url, file_path)

print("Download complete.")

df_ratings = pd.read_csv(f"{save_dir}/BX-Book-Ratings.csv",
                        delimiter=';',
                        encoding='ISO-8859-1')

df_books = pd.read_csv(f"{save_dir}/BX-Books.csv",
                      delimiter=';',
                      encoding='ISO-8859-1',
                      on_bad_lines='skip')

print("Ratings shape:", df_ratings.shape)
print("Books shape:", df_books.shape)

print(df_books.head())
df_books["Book-id"]=df_books.index
merged_df=pd.merge(df_ratings,df_books,"inner","ISBN")
merged_df["book"]=merged_df["Book-Title"] + " " +merged_
df["Book-Author"]
recommender_data=merged_df[["User-ID","Book-id","book","Book-
Rating"]]
recommender_data=recommender_data.iloc[:12345,:]
print("recommender data :", recommender_data.shape)

print("System version: {}".format(sys.version))
print("Pandas version: {}".format(pd.__version__))
print("Tensorflow version: {}".format(tf.__version__))
print("Num GPUs Available: ", len(tf.config.list_physical_
devices('GPU'))))

TOP_K = 10
EPOCHS = 150
BATCH_SIZE = 256
SEED = 25032005

recommender_data.columns=["userID","itemID","book","rating"]
recommender_data.loc[:,"rating"]=recommender_data["rating"].
astype("float")
train, test = train_test_split(recommender_data, test_size=
0.2,random_state=SEED)
train=train.sort_values(by="userID",ascending=True)
test=test.sort_values(by="userID",ascending=True)

test = test[test["userID"].isin(train["userID"].unique())]
test = test[test["itemID"].isin(train["itemID"].unique())]
train_file = "data/train.csv"

```



```

test_file = "data/test.csv"
recommender_data.to_csv(train_file, index=False)
recommender_data.to_csv(test_file, index=False)
data = NCFDataset(train_file=train_file, test_file=test_file, seed=
    SEED,col_user="userID",col_rating="rating",col_item="itemID")
model = NCF (
    n_users=data.n_users,
    n_items=data.n_items,
    model_type="NeuMF",
    n_factors=4,
    layer_sizes=[16,8,4],
    n_epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    learning_rate=1e-3,
    verbose=10,
    seed=SEED
)

with Timer() as train_time:
    model.fit(data)

print("Took {} seconds for training.".format(train_time))

model.save('data/model_serialized.tensorflow')
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000)
with Timer() as test_time:
    users, items, preds = [], [], []
    item = list(test.itemID.unique())
    for user in test.userID.unique():
        user = [user] * len(item)
        users.extend(user)
        items.extend(item)
        preds.extend(list(model.predict(user, item, is_
            list=True)))

    all_predictions = pd.DataFrame(data={"userID": users,
    "itemID":items, "prediction":preds})

    merged = pd.merge(train, all_predictions, on=["userID",
    "itemID"], how="outer")

    all_predictions = merged[merged.rating.isnull()].drop
    ('rating', axis=1).drop('book', axis=1)

print("Took {} seconds for prediction.".format(test_time))
all_predictions["prediction"]=all_predictions["prediction"]*10

```

```

eval_map = map(test, all_predictions, col_prediction=
    'prediction', k=TOP_K)
eval_ndcg = ndcg_at_k(test, all_predictions, col_prediction=
    'prediction', k=TOP_K)
eval_precision = precision_at_k(test, all_predictions, col_
    prediction='prediction', k=TOP_K)
eval_recall = recall_at_k(test, all_predictions, col_
    prediction='prediction', k=TOP_K)

print("MAP:\t%f" % eval_map,
      "NDCG:\t%f" % eval_ndcg,
      "Precision@K:\t%f" % eval_precision,
      "Recall@K:\t%f" % eval_recall, sep='\n')

users_to_make_recommendations = [list(train.userID.unique())
    [i] for i in [25, 3, 5]]

for user in users_to_make_recommendations:
    print(f"\nUser ID: {user}")

    user_ratings = recommender_data[recommender_data
    ["userID"] == user]
    print("Books rated by user:")
    print(user_ratings[["book", "rating"]].sort_values
    (by="rating", ascending=False).head(10))
    user_ratings.to_csv(f"data/user_ratings_{i}.csv", index =
    False)

    rated_books = set(user_ratings["itemID"])
    unrated_books = list(set(recommender_data["itemID"]) -
    rated_books)
    user_predictions = model.predict([user] * len(unrated_
    books), unrated_books, is_list=True)
    predictions_df = pd.DataFrame({"itemID": unrated_books,
    "prediction": user_predictions})
    predictions_df.prediction=predictions_df.prediction*10
    predictions_df = predictions_df.merge(df_books, left_
    on="itemID", right_on="Book-id")

    top_10_books = predictions_df.sort_values(by=
    "prediction", ascending=False).head(10)
    print("\nTop 10 recommended books for user:")
    print(top_10_books[["Book-Title", "Book-Author",
    "prediction"]])
    top_10_books.to_csv(f"data/top_10_books_{i}.csv",
    index=False)

```

The evaluation measures estimated for the NCF model are presented in [Figure 7.13](#).

MAP: 0.145478
NDCG: 0.226666
Precision@K: 0.074342
Recall@K: 0.392811

Figure 7.13 Results of NCF for bookstore recommender system.

Source: Own work, with Python/Matplotlib.

The results for recommendations for three selected users are presented in [Table 7.3](#).

The User ID: 75 has rated only one book: Life’s Little Instruction Book (5 stars), which focuses on self-help and philosophy. The Tao of Pooh, Seabiscuit, and The Prince went unrated. The unrated books include classics and history.

Table 7.3 The already rated and recommended books for three selected users

<i>User</i>	<i>Type</i>	<i>Book</i>	<i>Author</i>	<i>Rate/ Prediction</i>
75	Rated	Life’s Little Instruction Book	H. Jackson Brown Jr.	5
		The Tao of Pooh	Benjamin Hoff	0
		Seabiscuit	Laura Hillenbrand	0
		The Prince	Niccolo Machiavelli	0
	Recommended	Don’t Sweat the Small Stuff	Richard Carlson	9.59
		The Street Lawyer	John Grisham	9.53
		Chicken Soup for the Woman’s Soul	Jack Canfield	9.31
		The Secret Life of Bees	Sue Monk Kidd	9.06
		Reviving Ophelia	Mary Pipher	9.04
		The Bean Trees	Barbara Kingsolver	8.87
		Harry Potter and the Sorcerer’s Stone	J. K. Rowling	8.38
		Ender’s Game	Orson Scott Card	8.16
		Yeats Is Dead!	Joseph O’Connor	8.15
		Couplehood	Paul Reiser	8.15
		New Vegetarian: Bold and Beautiful Recipes	Jane Price	0
		Chicken Soup for the Woman’s Soul	Jack Canfield	6.26
		Memoirs of a Geisha	Arthur Golden	4.16
		31 Days of Praise	Ruth Myers	4.12
10	Rated	The Sparrow	Mary Doria Russell	3.92
		Heat and Dust	Ruth Prawer Jhabvala	3.28
	Recommended	Where the Heart Is	Billie Letts	3.1
		Table For Two	Nora Roberts	2.98
		Ender’s Game	Orson Scott Card	2.76
		Winds Of War	Herman Wouk	2.58
		Teen Love: On Relationships	Kimberly Kirberger	2.35

(Continued)

Table 7.3 (Continued)

User	Type	Book	Author	Rate/ Prediction
14	Rated	Flood: Mississippi 1927	Kathleen Duey	6
		Mary-Kate & Ashley: Switching Goals	Mary-Kate & Ashley Olsen	5
		Tell Me This Isn't Happening	Robynn Clairday	5
	Recommended	Wild Animus	Rich Shapero	0
		Shipping News	Annie Proulx	9.99
		Mystic River	Dennis Lehane	9.98
		Power of Positive Thinking	Norman V Peale	9.98
		The Shipping News	Annie Proulx	9.96
		Chipmunks ABC	Golden Press	9.95
		A Passage to India	E.M. Forster	9.95
		Prize Stories 1994	William Abrahams	9.95
		Life of Pi	Yann Martel	9.93
		Auerbach Will	Stephen Birmingham	9.9
		Dreams Die First	Harold Robbins	9.9

Source: Own calculations, with Python/TensorFlow and recommender library.

Top recommended books for User ID: 75:

- Don't Sweat the Small Stuff (Richard Carlson) – self-help, aligns with their rated book.
- The Street Lawyer (John Grisham) – legal fiction, different genre but popular.
- Chicken Soup for the Woman's Soul (Jack Canfield) – self-help, highly relevant.

User ID: 10 has interacted with a book about vegetarian cooking – New Vegetarian: Bold and Beautiful Recipes (unrated) – but the rating is missing, making it hard to determine their preference.

Top recommended books for User ID: 10:

- Chicken Soup for the Woman's Soul (Jack Canfield) – probably matched by food domain.
- Memoirs of a Geisha (Arthur Golden).
- 31 Days of Praise (Ruth Myers) – religious book, unlikely to be relevant.

User ID: 14 has rated young adult fiction and historical fiction – Flood: Mississippi 1927 (6 stars), Mary-Kate & Ashley: Switching Goals (5 stars), Tell Me This Isn't Happening (5 stars) but not rated Wild Animus (unrated).

Top recommended books for User ID: 10:

- Shipping News (Annie Proulx) – literary fiction; partially relevant.
- Mystic River (Dennis Lehane) – crime fiction; partially relevant.
- Power of Positive Thinking (Norman Peale).

7.3 Selected non-economical applications of deep learning models

7.3.1 Computer vision, neural style transfer, face detection

7.3.1.1 Neural style transfer

Neural style transfer (NST) is a technique that involves adapting the style – colors – brush texture, and stroke thickness of one graphic to another photo while maintaining its original layout and structural elements. A typical application of NST is generating artificial artwork from photographs, such as applying the aesthetic of renowned paintings to user-provided photos.

Most of the applications of NST are based on the approach proposed by [Gatys et al. \(2015\)](#).

The Visual Geometry Group (VGG) convolutional neural network (CNN) is used for image generation. The focus is set on the feature space provided by the convolutional and pooling layers. Content reconstruction involves matching feature responses from different layers of the network. Style reconstruction is based on the correlations between filter responses, represented as Gram matrices.

Images are generated by minimizing the mean-squared distance between the original and generated images' Gram matrices (7.16–7.22).

$$\mathcal{L}_{content}(\overline{p}, \overline{x}, l) = \frac{1}{2} \sum_{i,j} (F_{ij}^l - P_{ij}^l)^2 \quad (7.16)$$

where:

F_{ij}^l, P_{ij}^l is the activation of the i -th filter at position j in layer l of destination and source image

$$\frac{\partial \mathcal{L}_{content}}{\partial F_{ij}^l} = \begin{cases} F_{ij}^l - P_{ij}^l & \text{if } F_{ij}^l \geq 0 \\ 0 & \text{if } F_{ij}^l < 0 \end{cases} \quad (7.17)$$

$$G_{ij}^l = \sum_k F_{ik}^l F_{jk}^l \quad (7.18)$$

$$E_l = \frac{1}{4N_l^2 M_l^2} \sum_{i,j} (G_{ij}^l - A_{ij}^l)^2 \quad (7.19)$$

$$\mathcal{L}_{style}(\overline{a}, \overline{x}) = \sum_{l=0}^L \mathbf{w}_l E_l \quad (7.20)$$

$$\frac{\partial E_l}{\partial F_{ij}^l} = \begin{cases} \frac{1}{N_l^2 M_l^2} \left((F^l)^T (G^l - A^l) \right)_{ji} & \text{if } F_{ij}^l \geq 0 \\ 0 & \text{if } F_{ij}^l < 0 \end{cases} \quad (7.21)$$

$$\mathcal{L}_{total}(\overline{\vec{p}}, \vec{a}, \vec{x}) = \alpha \mathcal{L}_{content}(\overline{\vec{p}}, \vec{x}) + \beta \mathcal{L}_{style}(\vec{a}, \vec{x}) \quad (7.22)$$

To combine content and style, a loss function (7.22) is defined with weighting factors for both content and style reconstruction, and the ratio of weighting factors can be adjusted to control the final image's style.

Johnson et al. (2016) introduced a fast, feedforward network for style transfer trained with perceptual (feature-based) losses instead of pixel-wise losses, enabling real-time style transfer and high resolution using a single-pass approach rather than iterative optimization. Li and Wand (2016) combined Markov Random Field constraints with CNN features to better capture local style patterns and improve style transfer fidelity by enforcing patch-level style consistency. Ulyanov et al. (2016) proposed a fully convolutional feedforward architecture for fast texture and style generation. Li et al. (2017) developed a style transfer method that uses whitening and coloring transforms in feature space enabling arbitrary style transfer without retraining, aligning content image statistics to those of the style image. Luan et al. (2017) addressed photorealistic style transfer by preserving spatial structures and color coherence, and ensured the results remained photorealistic, avoiding distortions common in artistic style transfer. Huang and Belongie (2017) introduced adaptive instance normalization to match style statistics within the network.

Although few mature libraries are implementing neural style transfer, and even popular TensorFlow has custom models available on its hub for this purpose (see section 2.7), we will use custom code based on Chollet (2017) to examine precisely the process and - which is usually not possible - to have the possibility of saving intermediate steps.

```
import keras
from scipy.optimize import fmin_l_bfgs_b
from imageio import imwrite
import time
import tensorflow;
tensorflow.compat.v1.disable_eager_execution()
keras.__version__

from keras.preprocessing.image import load_img, img_to_array
target_image_path = 'ad.jpg'
style_reference_image_path = 'transfer_style_reference_2.jpg'
width, height = load_img(target_image_path).size
img_height = 400
img_width = int(width * img_height / height)
import numpy as np
from keras import backend as K
from keras.applications import vgg19
from matplotlib import pyplot as plt

def preprocess_image(image_path):
    img = load_img(image_path, target_size=(img_height, img_
        width))
    img = img_to_array(img)
```

```

img = np.expand_dims(img, axis=0)
img = vgg19.preprocess_input(img)
return img

def deprocess_image(x):
    x[:, :, 0] += 103.939
    x[:, :, 1] += 116.779
    x[:, :, 2] += 123.68
    x = x[:, :, ::-1]
    x = np.clip(x, 0, 255).astype('uint8')
    return x

target_image = K.constant(preprocess_image(target_image_path))
style_reference_image =
K.constant(preprocess_image(style_reference_image_path))
combination_image = K.placeholder((1, img_height, img_width, 3))
input_tensor = K.concatenate([target_image,
                               style_reference_image,
                               combination_image], axis=0)

model = vgg19.VGG19(input_tensor=input_tensor,
                    weights='imagenet',
                    include_top=False)

print('Model został załadowany.')
def content_loss(base, combination):
    return K.sum(K.square(combination - base))

def gram_matrix(x):
    features = K.batch_flatten(K.permute_dimensions(x, (2, 0, 1)))
    gram = K.dot(features, K.transpose(features))
    return gram

def style_loss(style, combination):
    S = gram_matrix(style)
    C = gram_matrix(combination)
    channels = 3
    size = img_height * img_width
    return K.sum(K.square(S - C)) / (4. * (channels ** 2) *
    (size ** 2))

def total_variation_loss(x):
    a = K.square(
        x[:, :img_height - 1, :img_width - 1, :] - x[:, 1:,
:img_width - 1, :])
    b = K.square(
        x[:, :img_height - 1, :img_width - 1, :] - x[:,
:img_height - 1, 1:, :])
    return K.sum(K.pow(a + b, 1.25))

outputs_dict = dict([(layer.name, layer.output) for layer in
model.layers])
content_layer = 'block5_conv2'
style_layers = ['block1_conv1',
                'block2_conv1',
                'block3_conv1',

```

```

        'block4_conv1',
        'block5_conv1']
total_variation_weight = 1e-4
style_weight = 1.
content_weight = 0.025
loss = K.variable(0.)
layer_features = outputs_dict[content_layer]
target_image_features = layer_features[0, :, :, :]
combination_features = layer_features[2, :, :, :]
loss = loss+ content_weight * content_loss(target_image_features,
                                           combination_features)

for layer_name in style_layers:
    layer_features = outputs_dict[layer_name]
    style_reference_features = layer_features[1, :, :, :]
    combination_features = layer_features[2, :, :, :]
    sl = style_loss(style_reference_features,
combination_features)
    loss += (style_weight / len(style_layers)) * sl
loss += total_variation_weight *
total_variation_loss(combination_image)
fetch_loss_and_grads = K.function([combination_image],
[loss, grads])
class Evaluator(object):
    def __init__(self):
        self.loss_value = None
        self.grads_values = None
    def loss(self, x):
        assert self.loss_value is None
        x = x.reshape((1, img_height, img_width, 3))
        outs = fetch_loss_and_grads([x])
        loss_value = outs[0]
        grad_values = outs[1].flatten().astype('float64')
        self.loss_value = loss_value
        self.grad_values = grad_values
        return self.loss_value
    def grads(self, x):
        assert self.loss_value is not None
        grad_values = np.copy(self.grad_values)
        self.loss_value = None
        self.grad_values = None
        return grad_values
evaluator = Evaluator()

result_prefix = 'style_transfer_result'
iterations = 20
x = preprocess_image(target_image_path)
x = x.flatten()
for i in range(iterations):
    print('Początek iteracji numer', i+1)
    start_time = time.time()

```



```

x, min_val, info = fmin_l_bfgs_b(evaluator.loss, x,
                                fprime=evaluator.grads,
maxfun=20)
print('Current loss value:', min_val)
img = x.copy().reshape((img_height, img_width, 3))
img = deprocess_image(img)
fname = result_prefix + '_at_iteration_%d.png' % i
imwrite(fname, img)
end_time = time.time()
print('Image saved as', fname)
print('Iteration no %d finished in %ds' % (i, end_time -
start_time))
plt.imshow(load_img(target_image_path, target_size=(img_
height, img_width)))
plt.figure()
plt.imshow(load_img(style_reference_image_path, target_
size=(img_height, img_width)))
plt.figure()
plt.imshow(img)
plt.show()

```

The script has been used four times to transform the author's image with those in [Figure 7.14](#) (also listed below) to create the images shown in [Figure 7.15](#).

- Socrealism poster <http://s.twojahistoria.pl/uploads/2017/12/Fragment-socrealistycznego-plakatu-z-has%C5%82em-Damy-ojczy%C5%BAnie-wi%C4%99cej-chleba.jpg>



Figure 7.14 Source image and style images for NST task.

Source: Own research.



Figure 7.15 The transformed images with NST task.

Source: Own research.

- Air+Man+Space by Lyubov Sergeevna Popova https://en.wikipedia.org/wiki/Lyubov_Popova#/media/File:Popova_Air_Man_Space.jpg
- Femme assise dans un fauteuil (Pablo Picasso) https://en.wikipedia.org/wiki/Pablo_Picasso#/media/File:Pablo_Picasso,_1913-14,_Woman_in_a_Chemise_in_an_Armchair_oil_on_canvas,_149.9_x_99.4_cm,_Metropolitan_Museum_of_Art.jpg
- Starry Night by Vincent Van Gogh https://commons.wikimedia.org/wiki/File:Van_Gogh_-_Starry_Night_-_Google_Art_Project.jpg

The author supposes that applying NST to some individual custom photographs would be much more interesting for readers, but while reproducibility is one of the main assumptions of this book, the photo used in this particular example is available at https://github.com/a-dudek-ue/erwdlm_book/nst_photo/.

7.3.1.2 Face detection

Face detection is not new method of deep learning, but the application of convolutional networks, regression inside convolutional layers and confidence threshold for final decision improved drastically the effects of its application. The most mature (at the moment of this book preparation) implementation is included in YOLOv8 model through OpenCV library.

YOLOv8 face model defines 143 convolutional layers, bounding box regression with focal loss and confidence threshold at level 0.3. We will use it to detect faces on probably the most famous and recognizable photograph in the world (cf. Figure 7.16).

The code below uses the pre-trained YOLOv8n-faces model. Due to some restrictions of Hugging Face, direct access to the *.pt file with the pre-trained



Figure 7.16 Lunch atop a skyscraper.

Source: https://upload.wikimedia.org/wikipedia/commons/thumb/6/62/Lunch_atop_a_Skyscraper.jpg/2560px-Lunch_atop_a_Skyscraper.jpg. CCA-SA 3.0 licence.

model requires complicated data downloading routine and its much simpler to just download the file "manually" and place it in the same folder where script is located.

```
import random
import cv2
import numpy as np
import requests
from pathlib import Path
from PIL import Image
from ultralytics import YOLO

url = "https://upload.wikimedia.org/wikipedia/commons/
thumb/6/62/Lunch_atop_a_Skyscraper.jpg/2560px-Lunch_atop_a_
Skyscraper.jpg"
headers = {
    'User-Agent': 'MyCustomUserAgent/1.0'
}
resp = requests.get(url, headers=headers, stream=True)
resp.raise_for_status()
```

```

img_array = np.asarray(bytearray(resp.content), dtype=np.uint8)
img = cv2.imdecode(img_array, cv2.IMREAD_COLOR)

# Download YOLOv8n-face model if not exists from
# https://huggingface.co/junjiang/GestureFace/blob/main/
yolov8n-face.pt
model_path = Path("yolov8n-face.pt")

model = YOLO(str(model_path))
results = model.predict(source=img, imgsiz=640, conf=0.3)

for result in results:
    for box in result.bboxes.xyxy:
        x1, y1, x2, y2 = (z-10 if i<2 else z+10 for (i,z) in
enumerate(map(int, box)))
        for i in range(x1, x2, 10):
            cv2.line(img, (i, y1), (i + 5, y1), (255, 0, 255), 6)
            cv2.line(img, (i, y2), (i + 5, y2), (255, 0, 255), 6)
        for i in range(y1, y2, 10):
            cv2.line(img, (x1, i), (x1, i + 5), (255, 0, 255), 6)
            cv2.line(img, (x2, i), (x2, i + 5), (255, 0, 255), 6)

output_path = "faces_detected.jpg"
cv2.imwrite(output_path, img)
im = Image.open(output_path)
im.save(output_path, dpi=(300, 300))

print("Saved faces_detected.jpg with magenta dotted rectangle
face markers!")

```

The results of the face detection routine are presented in [Figure 7.17](#). All the faces were recognized correctly and the borders around faces are drawn in the correct places.

7.3.2 Audio style transfer

The aim of audio style transfer, sometimes called audio conversion, is similar to NSF for images: adopt the style (in this case, timbre and intonation) of the audio fragment without changing the content. However, the techniques used for achieving this goal differ significantly. A typical approach is to convert the waveforms into time-frequency representations such as spectrograms and then train neural networks to manipulate these spectrograms according to the target style. By optimizing losses that balance fidelity to the original content with proximity to the desired style, the model iteratively adjusts parameters to produce a result that preserves the source's melodic and rhythmic structure, but exhibits the target's timbral characteristics. Advanced variants use self-supervised or adversarial training to refine these transformations, producing more realistic results.



Figure 7.17 Lunch atop a skyscraper with faces detected.

Source: https://upload.wikimedia.org/wikipedia/commons/thumb/6/62/Lunch_atop_a_Skyscraper.jpg/2560px-Lunch_atop_a_Skyscraper.jpg, CCA-SA 3.0 licence; face detection applied.

This field is rapidly expanding, with new algorithms being introduced frequently; however, several notable applications of deep learning models deserve special mention:

- **CycleGAN-VC** (Kaneko & Kameoka, 2017) – a parallel-data-free voice-conversion method that employs a cycle-consistent adversarial network (CycleGAN) with gated CNNs and identity-mapping loss to learn mappings between source and target speech without parallel data. The method effectively reduces over-smoothing, preserves linguistic information, and achieves high-quality converted speech, as demonstrated by objective and subjective evaluations showing performance comparable to traditional methods requiring parallel data.
- **Tacotron 2** (Shen et al., 2018) is a neural network-based text-to-speech system that combines a sequence-to-sequence model to predict mel-scale (a perceptual scale used to represent audio frequencies in a way that aligns with human hearing) spectrograms from text and a modified WaveNet vocoder to generate high-quality time-domain waveforms. The system achieves near-human-level speech synthesis, with a mean opinion score of 4.53, and simplifies the WaveNet architecture by using mel spectrograms as an efficient acoustic intermediate representation.

- **Wavenet** – a generative model for raw audio (Van Den Oord et al., 2016; DeepMind/Google).
- **Text input** – the model uses textual input as the content to be converted into speech.
- **Text-to-speech model** – the text input is processed by a text-to-speech model like Tacotron. Tacotron is designed to convert textual information into a mel-spectrogram, representing the speech features corresponding to the input text.
- **Voice synthesis model** (e.g., WaveNet) – the mel-spectrogram generated by the text-to-speech model is passed to a voice synthesis model. One of them is WaveNet, a deep generative model that takes these spectrograms and generates a waveform that closely resembles human speech.

$$p(x|h) = \prod_{t=1}^T p(x_t | x_1, \dots, x_{t-1}, h) \quad (7.23)$$

$$z = \tanh(W_{f,k} * x + V_{f,k}^T h) \odot \sigma(W_{g,k} * x + V_{g,k}^T h) \quad (7.24)$$

where:

h influences the output distribution across all timesteps, e.g. a speaker embedding in a Text-to-Speech model;

$V * _k$ is a learnable linear projection;

$V_{g,k}^T$ is broadcast over the time dimension;

$\odot$ - convolutional operator.

Output speech: The output of the voice synthesis model is an audio waveform that can be played back as speech, replicating the voice and speech patterns of a chosen speaker.

7.3.3 Multimodal deep learning

Multimodal deep learning is an advanced area of artificial intelligence that focuses on processing and integrating data from multiple modalities, such as text, images, audio, and video, to improve model performance and enable more comprehensive understanding. Unlike unimodal models, which operate on a single type of data, multimodal models leverage the complementary information present in different modalities to achieve richer representations and better generalization. This approach is particularly useful in applications like image captioning, where a model must generate textual descriptions from visual inputs, or in video analysis, where both visual and auditory data are combined for tasks like action recognition (Baltrušaitis et al., 2019).

One of the key techniques in multimodal deep learning is cross-modal representation learning, which aims to align data from different modalities into a shared embedding space. For example, models like Contrastive Language – Image Pre-training (CLIP) by Radford et al. (2021) use contrastive learning to align text and image embeddings, enabling tasks like zero-shot image classification. Another popular approach is multimodal fusion, where data from different modalities are combined at various stages of the model architecture. Early fusion integrates raw

data inputs, while late fusion combines high-level features extracted from each modality. Hybrid fusion methods, such as those used in ViLBERT (Lu et al., 2019), combine both approaches to leverage the strengths of each.

Another important technique is attention mechanisms, which allow models to dynamically focus on the most relevant parts of multimodal data. For instance, transformer-based models like multimodal transformers (Tsai et al., 2019) use self-attention and cross-attention to process and align information from different modalities. These models have been highly effective in tasks like video-text retrieval and multimodal sentiment analysis. Additionally, generative models like DALL·E (Ramesh et al., 2021), stable diffusion (Rombach et al., 2022), and SORA (OpenAI, 2024) have demonstrated the ability to generate high-quality images and video clips from textual descriptions, showcasing the power of multimodal learning in creative applications.

Researchers are actively exploring solutions like modality-specific pre-training and adversarial training to address these issues. As multimodal models continue to evolve, they are expected to play a crucial role in advancing AI applications, from healthcare diagnostics to autonomous systems, by enabling machines to understand and interact with the world in a more human-like manner (Baltrušaitis et al., 2019).

Beside commercial systems, multimodal deep learning tasks are available in the transformers library we already know. The area of their application is increasing every week and probably by the time of reading the book this fragment will be somehow out of date, but an simple example task of recognizing an image (presented in Figure 7.18) and transforming it into a text description can be accomplished with the following few lines of code using the *Bootstrapped Language Image Pretraining* pipeline:

```
from transformers import BlipProcessor, BlipForConditional
    Generation
from PIL import Image
import torch
import requests

url = "https://upload.wikimedia.org/wikipedia/commons/
thumb/0/0b/Eiffel_tower-Paris.jpg/960px-Eiffel_tower-Paris.jpg"
image = Image.open(requests.get(url, stream=True).raw).
    convert("RGB")
processor = BlipProcessor.from_pretrained("Salesforce/
blip-image-captioning-base")
model = BlipForConditionalGeneration.from_pretrained
    ("Salesforce/blip-image-captioning-base")
inputs = processor(images=image, return_tensors="pt")
with torch.no_grad():
    out = model.generate(**inputs)
caption = processor.decode(out[0], skip_special_tokens=True)
print("Caption:", caption)
```

Caption: a very wide shot of the eiff tower



Figure 7.18 Sample image to describe its content.

Source: https://upload.wikimedia.org/wikipedia/commons/thumb/0/0b/Eiffel_tower-Paris.jpg/2560px-Eiffel_tower-Paris.jpg. CCA-SA 3.0 licence.

7.3.4 Usage of large language models through web interface and API

Transformers library (and other large language model (LLM) libraries) is the most preferable way of leveraging the power of language models (besides the creation - of course- of its own custom Large Language model, which is far beyond the scope of this book). It has, however, one con requirement: significant computational GPU/TPU power. When we just want to use LLMs inside our analysis/simulation/custom app, the most convenient way is to use the application programming interface (API), which is available (unfortunately, at the moment of writing the book, only commercially) for all big players in the LLMs market. In this chapter, we will create a simple GUI API in Python with the backup of well-known OpenAI tools through the API.

To check the capabilities of language and multimodal models we will create an application that will solve equations and generate images by prompt.

```
import openai
import requests
import io
import tkinter as tk
```


332 *Deep Learning Models for Economic Research*

```
from tkinter import Label, Entry, Button, Canvas, PhotoImage
from PIL import Image, ImageTk
import datetime

OPENAI_API_KEY = "my-api-key"
openai.api_key = OPENAI_API_KEY

# Function to generate an image from text
def generate_image():
    prompt = text_input.get()
    if not prompt:
        status_label.config(text="Please enter a
        description.")
        return

    status_label.config(text="Generating image...")

    try:
        response = openai.Image.create(
            prompt=prompt,
            n=1,
            size="512x512"
        )
        image_url = response["data"][0]["url"]

        image_data = requests.get(image_url).content
        image = Image.open(io.BytesIO(image_data))

        tk_image = ImageTk.PhotoImage(image)

        canvas.image = tk_image
        canvas.create_image(0, 0, anchor=tk.NW, image=tk_image)
        status_label.config(text="Image generated successfully!")

        timestamp = datetime.datetime.now().strftime
        ("%Y%m%d_%H%M%S")
        filename = f"generated_image_{timestamp}.png"
        image.save(filename)
        status_label.config(text=f"Image saved as '{filename}'")

    except Exception as e:
        status_label.config(text=f"Error: {e}")

def solve_equation():
    equation = equation_input.get()
    if not equation:
        status_label.config(text="Please enter an equation.")
    return
```

```

try:
    response = openai.ChatCompletion.create(
        model="gpt-4",
        messages=[
            {"role": "system", "content": "Solve the
following equation and provide only the roots:"},
            {"role": "user", "content": equation}
        ]
    )

    solution = response["choices"][0]["message"]["content"]
    status_label.config(text=f"Roots: {solution}")

except Exception as e:
    status_label.config(text=f"Error: {e}")

root = tk.Tk()
root.title("AI Image Generator")
root.geometry("512x800")

Label(root, text="Enter text for image generation:").pack(pady=5)
text_input = Entry(root, width=80)
text_input.pack(pady=5)

generate_button = Button(root, text="Generate Image",
command=generate_image)
generate_button.pack(pady=10)

Label(root, text="Or enter equation to solve (e.g.,  $x^2 - 4 = 0$ ):").pack(pady=5)
equation_input = Entry(root, width=80)
equation_input.pack(pady=5)

solve_button = Button(root, text="Solve Equation",
command=solve_equation)
solve_button.pack(pady=10)

status_label = Label(root, text="", fg="blue")
status_label.pack(pady=5)

canvas = Canvas(root, width=512, height=512, bg="white")
canvas.pack(pady=10)

root.mainloop()

```

The sample results of program execution are presented in [Figures 7.19](#) and [7.20](#). The first task was solved correctly; the second one should be judged by the readers.

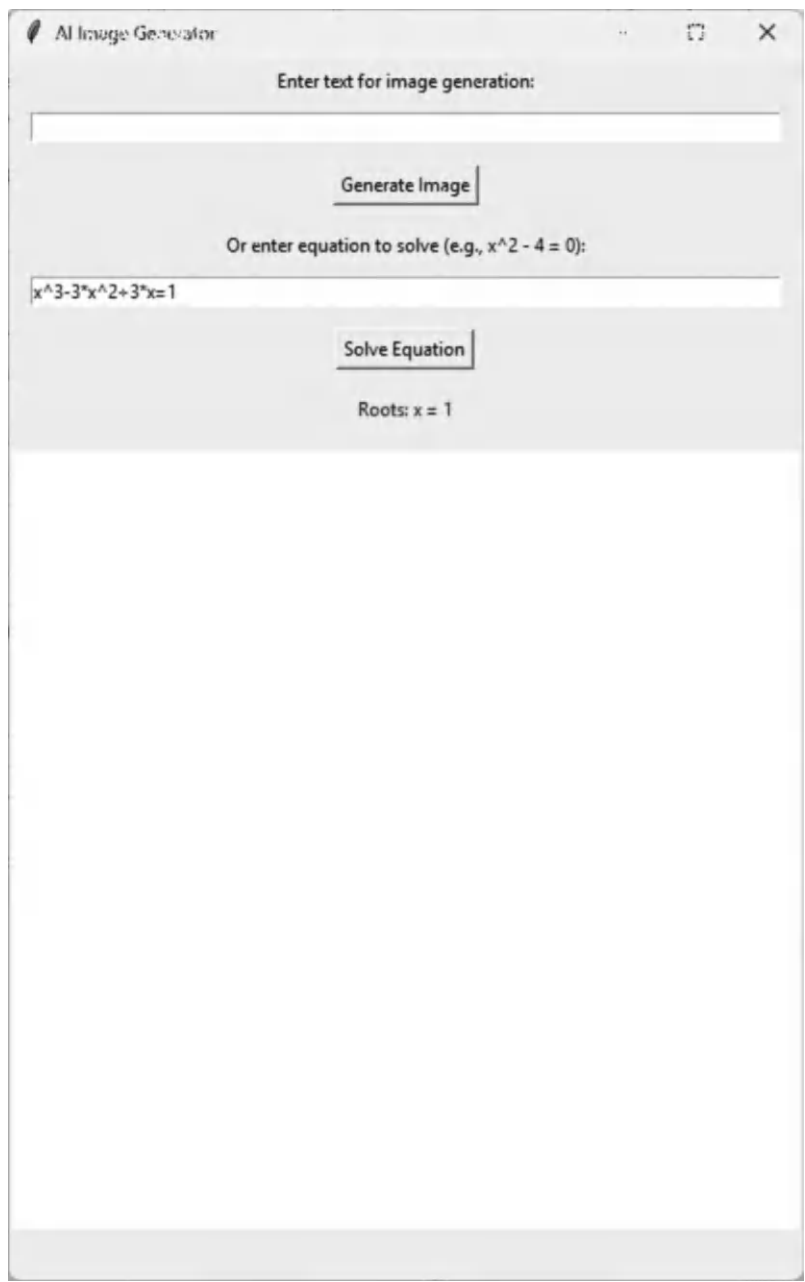


Figure 7.19 Sample equation solved with Open AI API.

Source: Own work, with Python/OpenAI API.

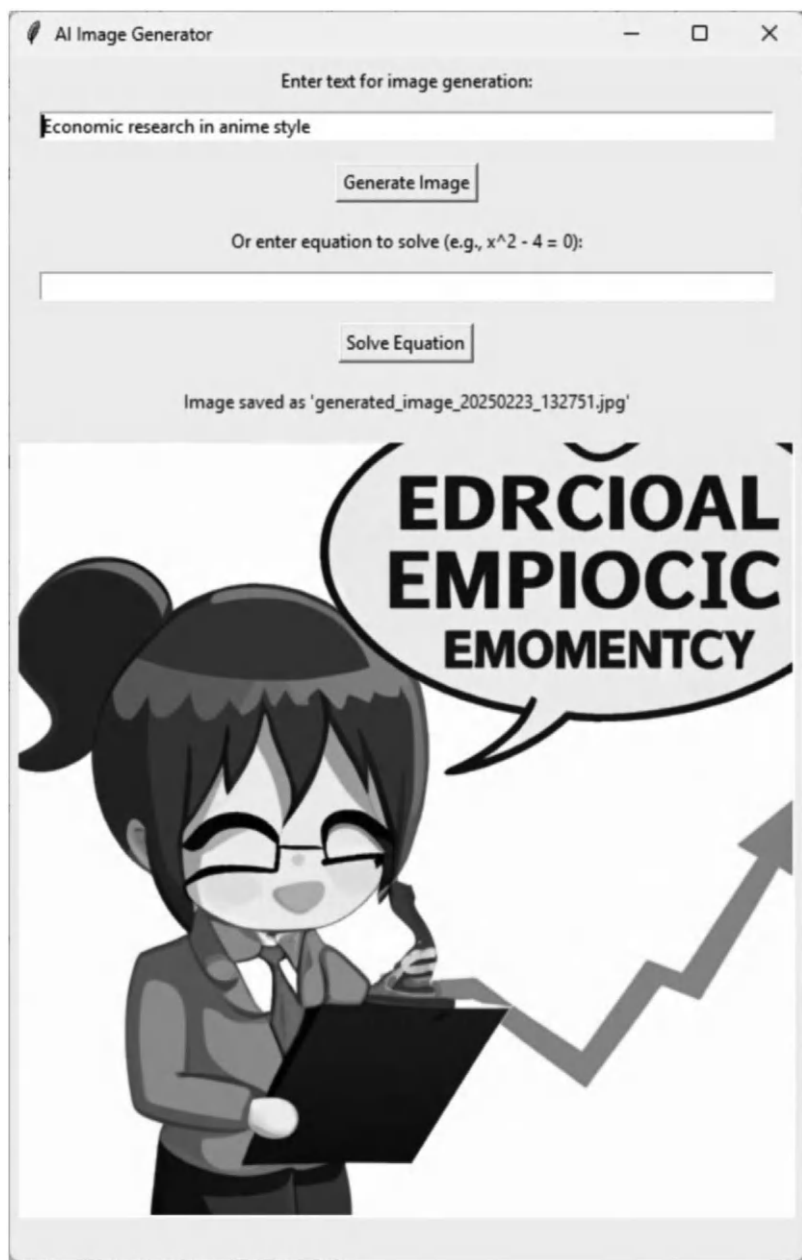


Figure 7.20 Sample images generated with Open AI API.

Source: Own work, with Python OpenAI API.

Note

- 1 Programs in R language usually reports memory error and Python sciklit-learn implementations execute very long – for more than one million observations, the author never got the results.

References

- Aldenderfer, M. S., & Blashfield, R. K. (1984). *Cluster analysis*. Sage.
- Anderberg, M. R. (1973). *Cluster analysis for applications*, Academic Press.
- Baltrušaitis, T., Ahuja, C., & Morency, L. P. (2018). Multimodal machine learning: A survey and taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 41(2), 423–443.
- Breckenridge, J. N. (2000). Validating cluster analysis: Consistent replication and symmetry. *Multivariate Behavioral Research*, 35(2), 261–285.
- Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331–370.
- Calinski, R. B., & Harabasz, J. (1974). A dendrite method for cluster analysis. *Communications in Statistics*, 3, 1–27.
- Card, D., & Krueger, A. B. (1994). Minimum wages and employment: A case study of the fast-food industry in New Jersey and Pennsylvania. *American Economic Review*, 84(4), 772–793.
- Carmone, F. J., Kara, A., & Maxwell, S. (1999). HINoV: A new method to improve market segment definition by identifying noisy variables. *Journal of Marketing Research*, 36, 501–509.
- Chollet, F. (2017). *Deep learning with Python*. Manning Publications.
- Comaniciu, D., & Meer, P. (1999). Mean shift analysis and applications. In *Proceedings of the International Conference Computer Vision (ICCV'99)*, Kerkyra, Greece (pp. 1197–2003). IEEE
- Davies, D. L., & Bouldin, D. W. (1979). A cluster separation measure. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1(2), 224–227.
- Dudek, A. (2020). Silhouette index as clustering evaluation tool. In K. Jajuga, J. Batóg, & M. Walesiak (Eds.), *Classification and data analysis: Theory and applications* (Vol. 28, pp. 19–33). Springer International Publishing.
- Everitt, B., Landau, S., Leese, M., & Stahl, D. (2011). *Cluster analysis*. John Wiley & Sons. <https://doi.org/10.1002/9780470977811>
- Friedman, J. H., & Meulman, J. J. (2004). Clustering objects on subsets of attributes (with discussion). *Journal of the Royal Statistical Society Series B: Statistical Methodology*, 66(4), 815–849.
- Gatys, L. A., Ecker, A. S., & Bethge, M. (2015). A neural algorithm of artistic style. *arXiv preprint arXiv:1508.06576*. <https://arxiv.org/abs/1508.06576>
- Gnanadesikan, R., Kettenring, J. R., & Tsao, S. L. (1995). Weighting and selection of variables for cluster analysis. *Journal of Classification*, 12, 113–136.
- Gordon A. D. (1999). *Classification*. Chapman and Hall/CRC.
- Hartigan J. A. (1975). *Clustering algorithms*. Wiley.
- He, X., Chen, T., Kan, M. Y., & Chen, X. (2015, October). Trirank: Review-aware explainable recommendation by modeling aspects. In *Proceedings of the 24th ACM international on conference on information and knowledge management* (pp. 1661–1670).
- He, X., Liao, L., Zhang, H., Nie, L., Hu, X., & Chua, T. S. (2017). Neural collaborative filtering. In *Proceedings of the 26th International Conference on World Wide Web* (pp. 173–182). <https://doi.org/10.1145/3038912.3052569>

- Hennig, C. (2015). What are the true clusters?. *Pattern Recognition Letters*, 64, 53–62.
- Hennig, C., Meila, M., Murtagh, F., & Rocci, R. (Eds.). (2015). *Handbook of cluster analysis*. Chapman and Hall/CRC. <https://doi.org/10.1201/b19706>
- Herlocker, L., Konstan, J. A., Terveen, L. G., & Riedl, J. T. (2004). Evaluating collaborative filtering recommender systems. *ACM Transactions on Information Systems*, 22(1), 5–53.
- Hubert, L. J. (1974). Approximate evaluation technique for the single-link and complete-link hierarchical clustering procedures. *Journal of the American Statistical Association*, 69(347), 698–704.
- Hubert, L. J., & Arabie, P. (1985). Comparing partitions. *Journal of Classification*, 1, 193–218.
- Huang, X., & Belongie, S. (2017). Arbitrary style transfer in real-time with adaptive instance normalization. In *Proceedings of the IEEE International Conference on Computer Vision* (pp. 1510–1519). IEEE. <https://doi.org/10.1109/ICCV.2017.167>
- Johnson, J., Alahi, A., & Fei-Fei, L. (2016). Perceptual losses for real-time style transfer and super-resolution. In B. Leibe, J. Matas, N. Sebe, & M. Welling (Eds.), *Computer Vision – ECCV 2016. Lecture Notes in Computer Science* (Vol. 9906, pp. 694–711). Springer. https://doi.org/10.1007/978-3-319-46475-6_43
- Kaneko, T., & Kameoka, H. (2017). Parallel-data-free voice conversion using cycle-consistent adversarial networks. *arXiv preprint arXiv:1711.11293*.
- Karatzoglou, A. (2006). *Kernel methods. Software, algorithms and applications*, Dissertation, Wien, Technical University.
- Kaufman, L., & Rousseeuw, P. J. (1990). *Finding groups in data: an introduction to cluster analysis*. Wiley.
- Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer*, 42(8), 30–37. <https://doi.org/10.1109/MC.2009.263>
- Leisch, F., & Dimitriadou, E. (2024). *_mlbench: Machine Learning Benchmark Problems_*. R package version 2.1-5. <https://CRAN.R-project.org/package=mlbench>
- Li, C., & Wand, M. (2016). *Combining Markov random fields and convolutional neural networks for image synthesis*. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 598–607). IEEE. <https://doi.org/10.1109/CVPR.2016.71>
- Li, J., Monroe, W., Shi, T., Jean, S., Ritter, A., & Jurafsky, D. (2017). Adversarial learning for neural dialogue generation. *arXiv preprint arXiv:1701.06547*.
- Linden, G., Smith, B., & York, J. (2003). Amazon.com recommendations: Item-to-item collaborative filtering. *IEEE Internet Computing*, 7(1), 76–80.
- Lu, J., Batra, D., Parikh, D., & Lee, S. (2019). Vilbert: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks. In H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, & R. Garnett (Eds.), *Advances in neural information processing systems* Curran Associates, Inc. (Vol. 32).
- Luan, F., Paris, S., Shechtman, E., & Bala, K. (2017). *Deep photo style transfer*. <https://arxiv.org/abs/1703.07511>
- McQuitty L.L. (1966). Similarity analysis by reciprocal pairs for discrete and continuous data. *Educational and Psychological Measurement*, (26), 825–831.
- Milligan, G. W. (1996). Clustering validation: Results and implications for applied analyses. In W. P. Arabie, L. J. Hubert, & G. de Soete (Eds.), *Clustering and classification* (pp. 341–375). World Scientific.
- Milligan, G. W., & Cooper M. C. (1985). An examination of procedures for determining the number of clusters in a data set. *Psychometrika*, 2, 159–179.
- Mu, R. (2018). A survey of recommender systems based on deep learning. *IEEE Access*, 6, 69009–69022.
- Ng, A., Jordan, M., & Weiss, Y. (2002). On spectral clustering: Analysis and an algorithm. In W. T. Dietterich, S. Becker, & Z. Ghahramani (Eds.), *Advances in neural information processing systems* (14, pp. 849–856). MIT Press.

- OpenAI. (2024, February 15). *Sora: Text-to-video model*. OpenAI. <https://openai.com/sora>
- Pazzani, M. J., & Billsus, D. (2007). Content-based recommendation systems. In *The adaptive web: methods and strategies of web personalization* (pp. 325–341). Springer.
- Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., & Sutskever, I. (2021, July). *Learning transferable visual models from natural language supervision*. In M. Meila & T. Zhang (Eds.), *Proceedings of the 38th International Conference on Machine Learning* (Vol. 139, pp. 8748–8763). PMLR
- Ramesh, A., Pavlov, M., Goh, G., Gray, S., Voss, C., Radford, A., ... Sutskever, I. (2021, July). Zero-shot text-to-image generation. In *International Conference on Machine Learning* (pp. 8821–8831). PMLR.
- Ricci, F., Rokach, L., & Shapira, B. (2021). *Recommender systems handbook*. Springer.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 10684–10695). New Orleans, LA, USA.
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65.
- Salakhutdinov, R., Mnih, A., & Hinton, G. (2007, June). Restricted Boltzmann machines for collaborative filtering. In *Proceedings of the 24th International Conference on Machine Learning* (pp. 791–798). Association for Computing Machinery. New York, NY, USA.
- Sedhain, S., Menon, A. K., Sanner, S., & Xie, L. (2015). AutoRec: Autoencoders meet collaborative filtering. In *Proceedings of the 24th International Conference on World Wide Web* (pp. 111–112). Florence, Italy.
- Shaham, U., Stanton, K., Li, H., Nadler, B., Basri, R., & Kluger, Y. (2018). Spectralnet: Spectral clustering using deep neural networks. *arXiv preprint arXiv:1801.01587*.
- Shen, J., Pang, R., Weiss, R. J., Schuster, M., Jaitly, N., Yang, Z., ... Wu, Y. (2018, April). Natural TTS synthesis by conditioning wavenet on mel spectrogram predictions. In *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (pp. 4779–4783). IEEE.
- Stewart, G. W. (1993). On the early history of the singular value decomposition. *SIAM Review*, 35(4), 551–566. <https://doi.org/10.1137/1035134>
- Tibshirani, R., Walther, G., & Hastie, T. (2001). Estimating the number of clusters in a data set via the gap statistic. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 63(2), 411–423. <https://doi.org/10.1111/1467-9868.00293>
- Tryon, R. C. (1939). *Cluster analysis: Correlation profile and orthometric (factor) analysis for the isolation of unities in mind and personality*. Edwards Brothers.
- Tsai, Y. H. H., Bai, S., Liang, P. P., Kolter, J. Z., Morency, L. P., & Salakhutdinov, R. (2019, July). Multimodal transformer for unaligned multimodal language sequences. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics* (Vol. 2019, p. 6558–6569). Florence, Italy.
- Ulyanov, D., Vedaldi, A., & Lempitsky, V. (2016). Texture networks: Feed-forward synthesis of textures and stylized images. In M. F. Balcan & K. Q. Weinberger (Eds.), *Proceedings of the 33rd International Conference on Machine Learning* (pp. 1349–1357). JMLR. org. <https://arxiv.org/abs/1603.03417>
- Ullsch, A., & Lötsch, J. (2020). The Fundamental Clustering and Projection Suite (FCPS): A dataset collection to test the performance of clustering and data projection algorithms. *Data*, 5(1), 13. <https://doi.org/10.3390/data5010013>
- Van Den Oord, A., Dieleman, S., Zen, H., Simonyan, K., Vinyals, O., Graves, A., ... Kavukcuoglu, K. (2016). Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*.

- von Luxburg, U. (2006). *A tutorial on spectral clustering*. Max Planck Institute for Biological Cybernetics, Technical Report TR-149.
- Walesiak, M. (2009a). Podstawowe zagadnienia statystycznej analizy wielowymiarowej (Basic issues of statistical multivariate analysis). In M. Walesiak, & E. Gatnar (Eds.), *Statystyczna analiza danych z wykorzystaniem program [Statistical data analysis with use of R program]*(pp. 62–80). PWN.
- Walesiak M. (2009b). Analiza skupień (Cluster Analysis). In M. Walesiak, & E. Gatnar (Eds.), *Statystyczna analiza danych z wykorzystaniem programu [Statistical data analysis with use of R program]*(pp. 407–433). PWN.
- Walesiak M., Dudek A. (2009), Odległość GDM dla danych porządkowych a klasyfikacja spektralna(GDM distance for ordinal data and spectral clustering), *Prace Naukowe Uniwersytetu Ekonomicznego we Wrocławiu nr 84 (Ekonometria 27)*, s. 9–19.
- Wu, C., Wang, J., Liu, W., & Liu, J. (2016). Recurrent neural network based recommendation for time heterogeneous feedback. *Knowledge-Based Systems*, 109, 90–103.
- Ziegler, C. N., McNee, S. M., Konstan, J. A., & Lausen, G. (2005, May). Improving recommendation lists through topic diversification. In *Proceedings of the 14th International Conference on World Wide Web* (pp. 22–32). Association for Computing Machinery.

Appendix 1

Data availability

Table A1.1 Data availability for datasets and images used in the book

Chapter	Dataset	Source	License and remarks
1	No datasets		
2	All dataset generated artificially by the author		
3	Car insurance frauds	Kaggle	CC0: Public Domain license, called from application programming interface (API), require API KEY generation or manual downloading
			https://www.kaggle.com/datasets/shivamb/vehicle-claim-fraud-detection
4	Dataset for explanation of VIP and SHapley Additive exPlanations (SHAP) generated artificially		
	California housing	scikit-learn library	BSD 3-Clause “New” or “Revised” license, built into library
	Inflation analysis	DBNomics	DBNomics database license: ODbL called directly from API, no additional requirements
	The pyramids of animals	Wikimedia	CC BY-SA 3.0 license, called by http request, no additional requirements
			https://upload.wikimedia.org/wikipedia/commons/thumb/6/6f/Kozyra_K_piramida.jpg/800px-Kozyra_K_piramida.jpg
	Tha Jaffa Gate in Jerusalem	Wikimedia	CC BY-SA 3.0 license, called by http request, no additional requirements
			https://upload.wikimedia.org/wikipedia/commons/thumb/3/36/Jaffa_Gate_Jerusalem_02.JPG/1280px-Jaffa_Gate_Jerusalem_02.JPG
5	Stock historical data	yfinance package	Called directly from API, no additional requirements
			https://legal.yahoo.com/us/en/yahoo/terms/product-atos/apitnc/index.html
6	IMDb reviews	TensorFlow Keras library	Permissive MIT license, called from API, require API KEY generation or manual downloading

(Continued)

Table A1.1 (Continued)

Chapter	Dataset	Source	License and remarks
7	Library recommender system	GitHub	Public repository. No direct downloading
	https://github.com/rochitasundar/Collaborative-Filtering-Book-Recommendation-System/		
	Neural style transfer	Socrealism poster	No direct downloading or use, Reader should acquire by themselves or use another style file
		Air+Man+Space (Lyubov' Sergeevna Popova)	No direct downloading or use, Reader should acquire by themselves or use another style file
		Femme assise dans un fauteuil (Pablo Picasso)	No direct downloading or use, Reader should acquire by themselves or use another style file
		Starry Night (Vincent Van Gogh)	No direct downloading or use, Reader should acquire by themselves or use another style file
		Author's photo	Available on https://github.com/a-dudek-ue/erwdlm_book
	Crypto market clusters	CoinGecko	No direct downloading, requires credentials to CoinGecko
	Lunch atop a skyscraper	Wikimedia	CC BY-SA 3.0 license, called by http request, no additional requirements
	https://upload.wikimedia.org/wikipedia/commons/thumb/6/62/Lunch_atop_a_Skyscraper.jpg/2560px-Lunch_atop_a_Skyscraper.jpg		
	Eiffel Tower photo	Wikimedia	CC BY-SA 3.0 license, called by http request, no additional requirements
	https://upload.wikimedia.org/wikipedia/commons/thumb/0/0b/Eiffel_tower-Paris.jpg/2560px-Eiffel_tower-Paris.jpg		

Appendix 2

Introduction to Python

Describing the Python language in a chapter spanning several dozen pages is a highly challenging task. Covering all aspects of the language, from the basics to advanced features, along with key functions and interesting examples, is a task not for a single book, but rather a series of books, each spanning several hundred or even over a thousand pages (incidentally, the ones I relied on while preparing this section are listed at the end). By its very nature, this must be a subjective selection of functions that are most useful for understanding the content of this book.

A2.1 Characteristics

Python implements several programming paradigms at the same time. Object-oriented programming, structured programming, and functional programming are possible in Python. Types are checked dynamically, and garbage collection is used to manage memory.

The creator of Python is the Dutchman Guido van Rossum and the name itself, which probably does not surprise some, comes from the popular BBC series “Monty Python’s Flying Circus”. Work on the first Python interpreter began in 1989 as the successor to the ABC language. All versions up to 1.2 were developed at the Center for Mathematics and Computer Science (CWI) in Amsterdam, where Guido worked at the time. Since version 2.1, Python has been released as an Open-Source project by the non-profit Python Software Foundation. There are many people working on Python development right now, but Guido is still involved in the process. The creator himself emigrated to the United States in 1995, where he worked for Google in the years 2005–2013.

An important moment in the history of Python was the creation of the second main branch – Python 3 in 2008. Since then, version 2 and version 3 have been developed separately, but the time of version 2 is starting to pass, as evidenced by the already announced end of support date of 12 April 2020. Language development is carried out using Python Enhancement Proposal (PEP). These documents are proposals for extensions or changes to the language in the form of an article that is discussed among Python programmers. Each document contains a description of the proposed solution, a justification, and the current status. Once a consensus is reached, the proposals are accepted or rejected.

The language is portable to many operating systems such as Windows, Mac, OS/2, with possible integration with COM, .NET_Framework, and CORBA objects.

The language is under an open-source license, which allows for multi-user development and open distribution.

All Python scripts are executed by the interpreter. Operation may look different in different systems, but its operation is independent. Python is developed as an Open Source project managed by the Python Software Foundation, which is a non-profit organization. The standard implementation of the language is CPython (written in C), but there are others, such as Jython (written in Java), CLPython written in Common Lisp, IronPython (for the .NET platform), and PyPy (written in Python).

Key features of Python include the following properties (cf. with [Lutz, 2013](#); [Matthes, 2019](#); [Ramalho, 2015](#)):

- Types are checked dynamically (unlike, e.g., Java);
- A garbage collector is used to manage memory;
- All values are passed by reference;
- It is sometimes qualified as a scripting language;
- There is no encapsulation, as is the case in C++ or Java, but there are mechanisms that allow you to achieve a similar effect:
- It is possible to create functions with a variable number of arguments;
- It is possible to create functions with arguments with default values;
- It is possible to use Lambda expressions.

A2.2 Data science

Python's biggest competitor in this field is R, which from the very beginning was created with statistics in mind, which, as we know, has a wide range of applications in the field of artificial intelligence. To follow are the advantages of Python in terms of data science:

- Python is a general-purpose language, which means that in addition to the possibility of using specialized libraries, for example, for data science, you can build a complete desktop or web application without the need to integrate with other solutions;
- Python as a scripting language, and additionally in combination with Jupyter (IPython) notebook, allows for very fast testing and prototyping by writing code “on the fly”, which means that there is no need to compile the code and run it later, which significantly speeds up the process of creating a working solution;
- A rich palette of very good quality libraries for artificial intelligence and data science examples include NumPy, Pandas, SciPy, Matplotlib or scikit-learn – described in [section 2.6](#).

A2.3 Types in Python

Every Python object is classified as immutable (immutable) or changeable (mutable). Among basic types, numbers, *strings* and *tuples*. are immutable while *lists*, *dictionaries*, and *sets* can be freely changed in place, such a distinction is crucial in the way Python works, but immutability can be used, among other things, to guarantee that an object remains constant throughout the program's lifecycle; modifiable (non-constant) values of objects can change at any time and place [Table A2.1](#) presents the main types.

A2.4 Data types

Table A2.1 Basic Python types

<i>Type category</i>	<i>Type Name</i>	<i>Description</i>
None	types.nonetype	None object (null object)
Numbers	bool	True or False (Boolean)
	int	Integer
	float	Floating point
	complex	Complex number
Collections	set	Mutable set
	frozenset	Immutable set
Sequences	str	String
	unicode	Unicode character string
	list	List
	tuple	Tuple
	range	Invariant sequence
Mapping	dict	Dictionary
Files	file	File

Source: Python documentation

A2.5 Comments

Comments start with a “#” sign and end at the end of the line. Multiline comments can be inserted as multi-line *strings* (bounded by “"""or"""”) without any action (e.g., assignments). These strings are not treated as expressions by the interpreter.

A2.6 Functional programming

Python has the availability of functional syntax. As you might expect, this greatly simplifies the processing of lists and other collections. One such construction is the so-called list comprehension, as in the following example of calculating the first five powers of two:

```
numbers = [1, 2, 3, 4, 5]
powers_of_two= [2 ** n for n in numbers]
```

A2.7 Lambda

Lambda blocks can only contain expressions, not statements. Thanks to them, we can define and return a function using a name in the local scope.

```
def create_summator(x):
    def temp(s):
        print(f"{x} + {y} = {x + y}")
    return temp
```

This function can also be implemented using nested lambda expressions.

```
create_sumator = lambda x: lambda y: print(f"{x} + {y} = {x + y}")
```

A2.8 Syntax

Python shares syntax features in C, Java, and Perl. However, you should watch out for important differences. Instead of `{`, Python uses indentation. The number of spaces in an indentation can vary, but all instructions inside the block must be indented by the same amount.

The code below is an example of a block in which the code will run correctly.

```
if True:
    print "Truth"
else:
    print "False"
```

Whereas the following code is an example of a listed error with a different number of indentations in a single block of code.

```
if True:
    print "Answer"
    print "Truth"
else:
    print "Answer"
    print "False"
```

Document number PEP8 is one (but not the only) proposal for organizing Python code. The original text of the document is available at <https://www.python.org/dev/peps/pep-0008/>.

A2.9 Indentation

In the Python code, you will not find the brackets known from PHP, Java or C# for separating code blocks, defining the framework of the body of a method or class, or the scope of operations in a loop. Here, appropriately set indents and empty lines between the above-mentioned elements are used for this purpose. For people who have never dealt with such code organization before, it may be surprising, but it becomes understandable and intuitive quite quickly like in the example below:

```
if score >= 100:
    print("Victory! ")
```

Each successive level of nesting in a code block is preceded by a space in the form of multiples of four spaces (a single indentation value). It is also acceptable to use tabs as indentation, but spaces are recommended and, in addition, in Python 3 the use of spaces and tabs as indentation at the same time is not allowed.

Indentation is also used in situations where a line of code is too long and needs to be broken into more lines. The recommended line length according to PEP8

is 79 characters. In the example above arguments of called function are splitted into two lines.

```
sent = send_message(e_mail, topic,
                    very_long_parameter_more_than_80_characters_overall_
                    message)
```

In this case, the indentation reaches the open character of the arguments list declaration of the called method or function. However, exceptions to this rule are allowed, allowing the use of a smaller indentation for subsequent lines like in the example below.

```
sent = send_message(e_mail_,
                    topic, message)
```

The documentation itself says that this is optional formatting, so it should only be used out of necessity and not as a rule.

Declaring variables such as a list, array, tuple, or dictionary often improves their readability thanks to indentation, which is the main principle that guided the rules of formatting code in Python.

```
list = [
    1, 2, 3,
    4, 5, 6,
]
```

In the case of breaking lines with arithmetic operators or brackets, readability is again the principle of transferring the operator to a new line.

```
profit = (revenue
          -costs
          -taxes)
```

A2.10 Blank lines

Top-order functions and class definitions are separated from other code blocks by two blank lines.

```
def do_something():
    return "done"

def do_something_too():
    return "also done"
```

A2.11 Strings

Strings (str type) are the base type for storing various length text. Strings are immutable, which means that for change of even one char a new string needs to be created. Common literals and string operations are presented in [Table A2.2](#).

Table A2.2 String and literals, main utility functions

Operation	Interpretation
<code>S = ""</code>	Empty string
<code>S = "platypus animal named 'Platypus mammal'"</code>	Quotation marks
<code>S = 'p\n\tax00typus'</code>	Special character sequences
<code>S = """... many poems... """</code>	Block in triple quotes
<code>S = r'\temp\platypus animal'</code>	Raw string (no escaping)
<code>S = b' platypus animal '</code>	Byte strings; in version 2.6, 2.7 and 3.x
<code>S = u'platypus animal'</code>	Unicode string; in version 2.x and 3.3+
<code>S1 + S2</code>	Concatenation
<code>S*3</code>	Concatenation, repetition
<code>S[i]</code>	Access to single character
<code>S[i:j]</code>	Access to sequence (slice) of characters
<code>len(s)</code>	Length
<code>"little %s parrot" % kind</code>	String formatting expression
<code>"a {0} parrot".format(kind)</code>	Formatting strings in 2.6, 2.7 and 3.x
<code>S.find('ie')</code>	Search
<code>S.rstrip()</code>	Remove white space
<code>S.replace('ie', 'xx')</code>	Overriding
<code>S.split(',')</code>	Split in place of the separator instance
<code>S.isdigit()</code>	Content review
<code>S.lower()</code>	Case conversion
<code>S.endswith(platypus animal)</code>	Checking the end of a string
<code>'platypus animal'.join(strlist)</code>	Joining with a separator
<code>S.encode('latin-1')</code>	Unicode encoding
<code>B.decode('utf8')</code>	Unicode decoding, etc.
<code>for x in S: print(x)</code>	Iteration
<code>' platypus animal ' in S</code>	belonging
<code>[c*2 for c in S]</code>	Comprehension list for string
<code>re.match('pl(.*)pus', line)</code>	Pattern matching

Source: Python documentation.

A2.12 Packages

Most big data analysis methods are not standard Python commands, but are implemented in extension packages. Each package should be **one time** installed.

`pip install package_name`

or

`./venv/Scripts/pip install package_name`

Each time we want to use an **installed** package, the *import* command should be used.

A2.13 Python basics

= set statement

```
variable = 4
x= 7 **3 +5
```


() – function call

```
amazon = yf.download('AMZN', start, end)
plt.plot(x, y)
```

Exercise
calculate $\sin\left(\frac{\pi}{2}\right)$:

```
import numpy as np
y = np.sin (np.pi/2)
print(y)
```

A2.14 Lists, dictionaries, tuples

Lists are the most flexible object type of an ordered collection. Unlike strings, lists can contain any kind of object – numbers, strings, or even other lists. Unlike strings, lists can also be modified in place by assigning offsets and slices, calls to list methods, or statements that remove elements. These are mutable objects.

Python lists do most of the work on collection data structures, which in lower level languages such as C have to be implemented manually. Below is an overview of the most important properties of Python lists.

Lists are ordered collections of arbitrary objects. From a functional point of view, lists are places that collect other objects, so they can be treated as groups. They also maintain the order of the elements from left to right (so they are sequences).

List items can be accessed using the offset position. As with strings, you can retrieve a list element from the list by indexing the list at the object's offset position. Because the list items are ordered by position, it is possible to slice or concatenate.

Lists are variable in length, heterogeneous, and can be nested at will. Unlike strings, lists can grow and shrink in place, that is, their length can change. They can also contain any type of object, not just single-character strings, so they are heterogeneous. Because lists can contain other complicated objects, they also support arbitrary nesting. In Python, it is possible to create lists consisting of other lists.

Lists belong to mutable sequences. As for the assignment to the appropriate category, the lists can be modified in place (they are variable) and respond to all sequence operations used with strings, such as indexing, slicing, and concatenation. Sequence operations work on lists in the same way as they do on strings. The only difference is that operations such as concatenation and stubbing, when applied to lists, return new lists rather than new strings. Because lists are mutable, they also support other operations that strings do not support (e.g., delete operations or index assignments that change the list in place).

Lists are arrays of references to objects. Technically, lists contain zero or more references to other objects. Lists can remind us of arrays of indicators (addresses). Retrieving an item from a Python list is almost as fast as indexing a C array. Python always stores a reference to the object itself, not a copy of it (unless explicitly requested).

Main operations on lists in Python are presented in [Table A2.3](#).

Along with lists, dictionaries are one of the most flexible built-in data types in Python. If you think of lists as ordered collections of objects, dictionaries are unordered collections. The primary difference between the two data types is that in dictionaries, items are stored and retrieved by keys rather than offset position. While lists can play a similar role to arrays in other programming languages, dictionaries replace records, lookup tables, and any other type of aggregation in which the names of objects are more meaningful than their positions. For example,

Table A2.3 List operations

<i>Operation</i>	<i>Interpretation</i>
<code>L = []</code>	Empty list
<code>L = [123, 'abc', 1.23, {}]</code>	Four elements – indices 0 to 3
<code>L = ['Adam', 40.0, ['dev', 'mgr']]</code>	Nested sublists
<code>L = list('Platypus')</code>	List of elements of an iterated object
<code>L = list(range(-4, 4))</code>	List of sequential integers
<code>L[i]</code>	Index
<code>L[i][j]</code>	Index
<code>L[i:j]</code>	Slice
<code>len(L)</code>	Length
<code>L1 + L2</code>	Concatenation
<code>L*3</code>	Repetition
<code>for x in L: print(x)</code>	Iteration
<code>3 in L</code>	Belonging
<code>L.append(4)</code>	Methods: adding elements
<code>L.extend([5,6,7])</code>	
<code>L.insert(i,X)</code>	
<code>L.index(X)</code>	
<code>L.count(X)</code>	Search
<code>L.sort()</code>	Sort
<code>L.reverse()</code>	Reverse
<code>L.copy()</code>	Deep copy
<code>L.clear()</code>	Cleanup
<code>L.pop(i)</code>	Methods, expressions: decreasing the list
<code>L.remove(X)</code>	
<code>del L[i]</code>	
<code>del L[i:j]</code>	
<code>L[i:j] = []</code>	Assign to index
<code>L[i] = 3</code>	
<code>L[i:j] = [4,5,6]</code>	
<code>L = [x**2 for x in range(5)]</code>	Comprehension list

Source: Based on Python documentation.

dictionaries can replace many search engines and data structures that would have to be implemented manually in lower level languages. Because dictionaries are a highly optimized built-in type, indexing them is a very fast search operation. Dictionaries sometimes also act as records and symbol tables from other programming languages. They can also represent, for example, the so-called sparse data structures, i.e. those in which most of the elements are empty. Below is a brief summary of their properties.

Dictionaries are sometimes accessed by key, not by offset value. Dictionaries are sometimes referred to as associative arrays or hash arrays, especially by users of other programming languages. They bind a set of values to keys so that a dictionary item can be retrieved using the key under which it is stored. To retrieve the dictionary components, the indexing operation is the same as for lists, but the index is a key rather than a relative offset value.

Dictionaries are unordered collections of arbitrary objects. Unlike a list, items stored in a dictionary are not stored in any particular order. In fact, the order of dictionary elements in Python is pseudo-random, so they can be searched faster. Keys provide a symbolic (rather than physical) location of items in the dictionary.

Dictionaries are variable length, heterogeneous, and can be arbitrarily nested. Like lists, dictionaries can expand and contract in place (without creating new copies), and can contain objects of any type. They also support nesting to any depth (they can include lists, other dictionaries, etc.). Each key can have only one associated value, but if desired, such a value can be a collection of multiple objects, and each value can be stored under any number of keys.

Dictionaries belong to the category of mutable mappings. Dictionaries can be changed in-place by assigning them to indexes (which are mutable). However, they do not support sequential operations that operate on strings and lists. Because dictionaries are unordered collections, all operations based on a fixed order of elements (e.g., concatenation or slices) do not make much sense in their case. Instead, dictionaries are the only built-in type that represents a category of mappings (objects that map keys to values).

Dictionaries are tables of object references (associative arrays). If lists are tables of object references that support item access, dictionaries are unordered tables of object references that support key access. Internally, dictionaries are implemented as associative arrays (data structures that support very fast retrieval) that can expand and contract on demand. What's more, Python uses optimized hashing algorithms to find keys, making this retrieval really fast. Like lists, dictionaries store references to objects.

Main operations on dictionaries in Python are presented in [Table A2.4](#).

The last object of a collection classified as Python basic types is a tuple. Tuples form simple groups of objects. They work exactly like lists, but they cannot be modified in place (they are immutable) and are usually written as a series of elements in regular parentheses rather than square brackets. Although they do not support as many methods, they share most properties with lists. Below is a brief summary of the properties of tuples.

Table A2.4 Dictionary operations

Operation	Interpretation
<code>D = {}</code>	Empty dictionary
<code>D = {'name': 'Adam', 'age': 40}</code>	Two-element dictionary
<code>E = {'cto': {'name': 'Adam', 'age': 40}}</code>	Nesting
<code>D = dict(name='Adam', age=40)</code>	Alternative techniques of creation
<code>D = dict([('name', 'Adam'), ('age', 40)])</code>	
<code>D = dict(zip(keylist, valueslist))</code>	Keywords, key-values, pinned key-values, key lists
<code>D = dict.fromkeys(['name', 'age'])</code>	
<code>D['name']</code>	
<code>D['cto']['age']</code>	Indexing by key
<code>'age' in D</code>	Methods: check key association
<code>D.keys()</code>	Methods: all keys
<code>D.values()</code>	All values
<code>D.items()</code>	All key+value tuples
<code>D.copy()</code>	Copy
<code>D.clear()</code>	Cleaning (removes all components)
<code>D.update(D2)</code>	Connecting by keys
<code>D.get(key, default?)</code>	Get by key if exists, default value otherwise (or None when not specified)
<code>D.pop(key, default?)</code>	Delete and return key if exists, default value otherwise (or error key when not specified)
<code>D.setdefault(key, default?)</code>	If key is in the dictionary, returns its value. If not, inserts key with the given default value and returns default
<code>D.popitem()</code>	Removing and returning last inserted pair (key, value),
<code>len(D)</code>	Length (number of entries stored)
<code>D[key] = 42</code>	Add or modify keys
<code>del D[key]</code>	Delete an item by key
<code>list(D.keys())</code>	List of all keys defined in dictionary
<code>D1 & D2</code>	Intersection
<code>D1 D2</code>	Union
<code>D = {x: x*2 for x in range(10)}</code>	Comprehension dictionaries

Source: Based on Python documentation.

Tuples are ordered collections of arbitrary objects. Like strings and lists, tuples are ordered collections of objects, so they maintain the order of elements from left to right. As in the case of lists, any type of object can be embedded into tuples.

Tuples are accessed by the offset value. Similar to strings and lists, tuple elements are accessed by the offset value (rather than by key). Tuples support all operations that are based on offset values, including indexing and slicing.

Tuples belong to the category of immutable sequences. Like strings and lists, tuples are sequences and they support many of the operations that refer to these two types of objects. Like strings, however, tuples are immutable, so they do not support any in-place modifying operations that apply to lists.

Table A2.5 Tuple operations

Operation	Interpretation
()	Blank tuple
T = (0,)	A single-element tuple (not an expression)
T = (0, 'Ni', 1.2, 3)	Four-element tuple
T = 0, 'Ni', 1.2, 3	Another four-element tuple (the same as above no brackets needed)
T = ('Adam', ('dev', 'mgr'))	Nested tuples
T = tuple('luncheon meat')	Tuple of elements in an iterable object
T[i]	Index
T[i][j]	Index index
T[i:j]	Slice
len(T)	Length
T1 + T2	Concatenation
T2*3	Repetition
for x in T: print(x)	Iteration

Source: Based on Python documentation.

Tuples have a fixed length, are heterogeneous and can be nested at will. Because tuples are immutable, you can't resize them without creating a copy. Tuples, on the other hand, can store any type of object, including other complex objects (e.g., lists, dictionaries, and other tuples), and can be nested virtually freely.

Tuples are arrays of references to objects. Like lists, tuples are best thought of as arrays of references to objects. Tuples store access points to other objects (references), and tuple indexing is quite fast.

Main operations on tuples in Python are presented in [Table A2.5](#).

A2.15 Control statements and comprehension

Python, as every modern structural programming language, implements control statements for condition checking and for loop execution.

if Statement is used for conditional execution.

```
if condition:
    statement
elif another_condition:
    statement
else:
    statement
```

for loop is used for iterating over sequences.

```
for item in iterable:
    statement
    <if (condition1):
```

```

        continue>
    <if (condidtion2):
        break>
else:
    statement_loop_not_broken

```

continue skips the rest of the loop for the current iteration and moves to the next iteration.

break exits the loop completely when a condition is met.

else block is executed when no break was called inside loop.

Continue and break section are optional.

Let us observe the instructions in three simple actions regarding the numbers array created with the following code:

```

import random
random.seed(25032005)
numbers=random.choices(range(20), k=50)

```

To filter the elements higher than 5 we should use the code similar to this one:

```

filtered_list = []
for x in numbers:
    if x > 5:
        filtered_list.append(x)
print(filtered_list)

```

With the result:

```

[12, 17, 8, 15, 14, 14, 8, 7, 6, 12, 7, 14, 19, 19, 12, 12,
18, 9, 6, 16, 7, 10, 8, 14, 14, 7, 6, 12, 19, 11, 13, 10,
13, 12, 7, 13, 8, 15]

```

The next example creates a new list with squares of numbers list:

```

squared_list = []
for x in numbers:
    squared_list.append(x**2)
print(squared_list)

```

With the result:

```

[144, 289, 9, 9, 25, 1, 16, 64, 9, 225, 196, 196, 64, 49,
36, 144, 49, 196, 361, 361, 9, 144, 144, 324, 81, 36, 256,
4, 49, 100, 64, 196, 196, 49, 36, 144, 361, 121, 169, 100,
169, 144, 49, 16, 169, 1, 25, 64, 1, 225]

```

In a slightly more sophisticated example, the list elements are squared for even ones and left unchanged otherwise:

```
processed_list = []
for x in numbers:
    if x % 2 == 0:
        processed_list.append(x**2)
    else:
        processed_list.append(x)
print(processed_list)
```

Which creates the list:

```
[144, 17, 3, 3, 5, 1, 16, 64, 3, 15, 196, 196, 64, 7, 36,
144, 7, 196, 19, 19, 3, 144, 144, 324, 9, 36, 256, 4, 7,
100, 64, 196, 196, 7, 36, 144, 19, 11, 13, 100, 13, 144, 7,
16, 13, 1, 5, 64, 1, 15]
```

The unique feature of Python is **comprehension** – the mechanism for **make loops and conditions more compact and readable**.

The general form of comprehension for lists may be written as:

```
new_list = [expression for item in iterable if condition]
```

Where:

expression → The operation or transformation applied to each item.

for item in iterable → Iterates over a sequence (e.g., list, range, string).

if condition (*optional*) → Filters elements based on a condition.

The **loop** and **for** examples given above may be replaced by:

1 Filtering a list

```
filtered_list = [x for x in numbers if x > 5]
```

2 Transforming a list (replacing `for` loop)

```
squared_list = [x**2 for x in numbers]
```

3 Combining `if` and `for`

```
processed_list = [x**2 if x % 2 == 0 else x for x in
numbers]
```

Using list comprehensions is undoubtedly very convenient, but getting carried away with them at first can lead to a situation where the code is understandable only to its author. It's important to always maintain a delicate balance between shortening the code with list comprehensions and keeping it readable. For example, to display all non-decreasing sequences of natural numbers with a given sum, you can use the following command:

```
target_sum = 8
sequences = []
dp = [[] for _ in range(target_sum + 1)]
dp[0] = [[]]
for current_sum in range(1, target_sum + 1):
    for start in range(1, current_sum + 1):
        for seq in dp[current_sum - start]:
            if not seq or (seq and seq[-1] <= start):
                new_seq = seq + [start]
                dp[current_sum].append(new_seq)
for seq in dp[target_sum]:
    print(seq)
```

With the result:

```
[1, 1, 1, 1, 1, 1, 1, 1]
[1, 1, 1, 1, 1, 1, 2]
[1, 1, 1, 1, 2, 2]
[1, 1, 2, 2, 2]
[2, 2, 2, 2]
[1, 1, 1, 1, 1, 3]
[1, 1, 1, 2, 3]
[1, 2, 2, 3]
[1, 1, 3, 3]
[2, 3, 3]
[1, 1, 1, 1, 4]
[1, 1, 2, 4]
[2, 2, 4]
[1, 3, 4]
[4, 4]
[1, 1, 1, 5]
[1, 2, 5]
[3, 5]
[1, 1, 6]
[2, 6]
[1, 7]
[8]
```

Or equivalent using comprehension with the same result:

```
def func_with_comprehension (suma):
    lista = [[]]
```



```
for i in range(1, suma + 1):
lista.append([c+ [i - j] for j in range(i) for c in lista[j]
if      not c or c[-1] <= i - j])
return lista[suma]
for f in func_with_comprehension(8):
print(f)
```

The author leaves it to the reader’s judgment which of these forms seems more elegant, but at the same time understandable.

A2.16 Python – expressions

Table A2.6 presents the expression components in Python.

Table A2.6 Python operators and expressions

<i>Symbol / operator</i>	<i>meaning</i>
yield	Generator function send protocol
lambda arguments: expression	Generate anonymous function
x if y else z	A ternary choice expression (x is evaluated only if y is true)
x or y	Logical OR (y is evaluated only if x is false)
x and y	Logical AND (y is evaluated only if x is true)
not x	Logical negation
x in y, x not in y	Affiliation (objects that can be iterated over, sets)
x is y, x is not y	Object identity testing
x < y, x <= y, x > y, x >= y	Comparison operators, subsets, and supersets of sets;
x == y, x != y	Equality operators
x y	Bitwise OR, sum of sets
x ^ y	Bitwise XOR, symmetric set difference
x & y	Bitwise AND, intersection of sets
x << y, x >> y	Shift x to the left or right by y bits
x + y	Addition, lists concatenation;
x - y	Subtraction, sets difference
x * y	Multiplication, lists repetition;
x % y	The remainder of the division, format
x / y, x // y	Dividing: real and without rest
-x, +x	Negation, identity
~x	Bitwise NOT
x** y	Power
x[i]	Indexing (sequences, renditions, etc.)
x[i:j:k]	Slices
x(...)	Call (functions, methods, class, other objects that can be called)
x.attribute,	Attribute reference

Source: Based on Python documentation.

A2.17 Division

Python offers various types of divisions: true division (/) returning a float, floor division (//) rounding down to the nearest integer, and modulo (%) returning the remainder with the same sign as the divisor, while also exposing floating-point precision limits and rounding toward negative infinity for negative operands in floor division.

```
7/2
3.5
7//2
3
7.3/2.4
3.0416666666666665
7.3//2.4
3.0
7%2
1
7.1%2
1.0999999999999996
7.1%2.6
1.8999999999999995
5//2, -5//2
(2, -3)
```

A2.18 Multiple comparisons

Python supports chained comparisons, evaluating expressions like $2 < 3 < 4 < 5$ as a sequence of pairwise comparisons with logical AND, and $1 == 2 < 3$ as $(1 == 2)$ and $(2 < 3)$, which evaluates to False since the first condition fails.

```
2<3<4<5
True
1 == 2 < 3 # Same as: 1 == 2 and 2 < 3
False
```

A2.19 Imaginary/complex numbers

Complex numbers are represented as two floating point numbers, a real part and an imaginary part, which are denoted by adding the letters j or J to the imaginary part. Complex numbers with a non-zero real part can also be written by combining two parts with the + sign.

```
1J*1J
(-1+0j)
2+1J*3
(2+3j)
(2+1j) * (2+1j)
(3+4j)
```

```
(2+1j)**2
(3+4j)
(1j)**(1/2)
(0.7071067811865476+0.7071067811865476j)
```

A2.20 Fractional numbers

Fractional numbers can be represented using the `Fraction` type – this provides better accuracy for pure fractional calculations.

```
from fractions import Fraction
Fraction(1,3)+Fraction(1,6)
Fraction(1, 2)
Fraction(1,3)**2/2 +Fraction(5,18)
Fraction(1, 3)
```

A2.21 Collections and object comparisons

Operator `is` in Python compares object identity, `==` compares objects (lists, sets, dictionaries) content.

```
a=list(range(1,10))
b=list(range(9,0,-1))
a is b
False
a == b
False
set(a) is set(b)
False
set(a) == set(b)
True
```

A2.22 Slices

Slices are a very convenient mechanism for accessing selected sub-sequences in lists (strings, tuples). Slices consist of two, optionally three, parts divided by a colon defining the start and end of the sequence, optionally step value. Negative values indicate place counted from the end of the vector in the case of the second parameter and, in the case of the third, a descending step (which may change the order of the result).

```
t=list(range(1,21))
print(t)
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
t[1:12:2]
[2, 4, 6, 8, 10, 12]
t[1:-4:3]
[2, 5, 8, 11, 14]
```

```

t[::-1]
[20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
t[::-2]
[20, 18, 16, 14, 12, 10, 8, 6, 4, 2]
t[19::-2]
[20, 18, 16, 14, 12, 10, 8, 6, 4, 2]
t[18::-2]
[19, 17, 15, 13, 11, 9, 7, 5, 3, 1]

```

A2.23 Object-oriented programming

A class definition begins with the class keyword, the class name, optionally the name of the base class, and the colon symbol.

```

class Human:

    name=""
    age = None
    gender = ""

    def hello(self):
        print ("Hello. My name is", self.name)

    def move(self):
        print ("I set off on the road")
    def think(self):
        if self.age < 2:
            print ("Still learning")
        else:
            print ("No problem")

```

Each class object contains a default argument-less constructor.

```

john= Human()
john.name = "John"
john.age = 23
john.gender= "man"

print ('Object -',john.name, john.age, john.gender)
john.hello()
john.move()
john.thought()

```

The other constructors require `__init__` definitions with list or arguments (*self* as the first one).

```

def __init__(self, name, age, gender):
    self.name = first name

```

```

        self.age = age
        self.gender = gender

john= Human("John", 23, "man")
print ('Object -',john.name, john.age, john.gender,'\n');jon.hello()
john.move ()
john.think ()

```

Python defines two keywords usually used in this content:

self – current object;
super – a reference to the parent class.

A2.24 Inheritance

The strength of object-oriented programming lies in the inheritance mechanism, among others, allowing the creation of descendant classes inheriting all the fields and methods from parent classes, but allowing the modification of methods (polymorphism) and definitions of new methods specific for descendant.

```

class Bird:          # base class

    def __init__ (self, species, speed):
        self.species = species
        self.speed = speed

    def fly(self):
        print ("Here", self.species, ". It starts, and I'm
about to reach speed", self.speed)

    def releaseNoise(self):
        Pass

class Eagle (Bird): # Bird is base class

    def hunt(self):
        print ("Here", self.species, ". I started the
hunt")

class Penguin (Bird): # Bird is base class

    def slide (self):
        print ("Here", self.species, ". I started to slide")

    def fly(self):
        print ("Here", self.species, ". I'm sorry, but I'm
not flying")

```

```

o = Bird("eagle",100)
p = Bird("penguin",-1)
o.fly()
p.fly()
o.hunt()
p.slide ()

class Eagle (Bird): # Bird is base class
    def __init__ (self, speed):
        super().__init__("eagle", speed)

class Penguin (Bird): # Bird is base class
    def __init__(self, speed):
        super().__init__("penguin", speed)

o = Eagle (100)
p = Penguin(-1)
o.fly()
p.fly()
o.hunt()
p.slide ()

```

A2.25 Encapsulation

The encapsulation (mechanism of restricting access to class internal methods/function and fields/variables) in Python is much simpler than in Java or C++ and comes down to the fact that fields whose name begins with `__` are not available outside the class

A2.26 NumPy

NumPy provides high-performance multidimensional arrays. It supports advanced indexing and Boolean masking and enables fast element-wise arithmetic operations. The library includes a wide range of mathematical functions that may be used for vectors/arrays instead of single values.

NumPy defines variety of function to process one, two or more-dimensional array. The most popular ones are presented in [Table A2.7](#).

In the examples below we will assume that NumPy is loaded with the usual:

```
import numpy as np
```

Some of the functions from [Table A2.7](#) may be used in **two ways**. As np. function or equivalent NumPy array object method call.

```

a = np.array([1, 2, 3], [4,5,6])
np.sum(a,axis=1)
#or
a.sum(axis=1)

```

Table A2.7 Selected NumPy function

<code>np.array()</code>	Creates a NumPy array from a list, tuple, or existing data structure, allowing efficient storage and operations.
<code>np.zeros()</code>	Generates an array filled with zeros of a specified shape and data type.
<code>np.ones()</code>	Creates an array of ones with a specified shape and data type.
<code>np.eye()</code>	Produces an identity matrix where diagonal elements are 1 and all others are 0.
<code>np.full()</code>	Generates an array of a given shape filled with a specified constant value.
<code>np.arange()</code>	Returns an array with evenly spaced values within a specified range, similar to Python's <code>range()</code> .
<code>np.linspace()</code>	Creates an array with evenly spaced values over a defined interval.
<code>np.reshape()</code>	Changes the shape of an array without modifying its data.
<code>np.ravel()</code>	Flattens a multidimensional array into a one-dimensional array as view if possible.
<code>np.flatten()</code>	Flattens a multidimensional array into a one-dimensional array.
<code>np.concatenate()</code>	Combines multiple arrays along a specified axis.
<code>np.vstack()</code>	Stacks multiple arrays vertically (row-wise).
<code>np.hstack()</code>	Stacks multiple arrays horizontally (column-wise).
<code>np.split()</code>	Splits an array into multiple sub-arrays along a specified axis.
<code>np.mean()</code>	Computes the arithmetic mean (average) of array elements along an axis. Argument <code>axis</code> defines the way of calculation 0 – for rows reduction (column-wise), 1 for columns reduction (row-wise) or globally for whole array if argument is set to <code>None</code> or omitted. The same concerns <code>np.median</code> , <code>np.std</code> , <code>np.sum</code> , <code>np.min</code> , <code>np.max</code> functions
<code>np.median()</code>	Finds the median value of an array along a given axis.
<code>np.std()</code>	Calculates the standard deviation of an array along an axis.
<code>np.sum()</code>	Returns the sum of all elements in an array or along a specified axis.
<code>np.min()</code> / <code>np.max()</code>	Finds the minimum (<code>np.min()</code>) or maximum (<code>np.max()</code>) value in an array along an axis.
<code>np.dot()</code>	Computes the dot product of two arrays (or matrix multiplication for two-dimensional arrays).
<code>np.random.rand()</code>	Generates an array of random numbers uniformly distributed between 0 and 1.
<code>np.resize()</code>	Resizes an array to a specified shape, repeating elements if necessary to fill the new size but not preserving original data beyond the new shape.
<code>np.savetxt()</code>	Writes a NumPy array to a CSV (or text) file with customizable delimiters, formatting, and headers. <code>fname</code> → The file name or file object where the data will be saved. <code>X</code> → The NumPy array to be written to the file. <code>delimiter</code> → The string used to separate values (default is “ <code>;</code> ”; use “ <code>,</code> ” for CSV). <code>fmt</code> → The format string or sequence of format specifiers for saving values (e.g., “ <code>%d</code> ” for integers, “ <code>%.2f</code> ” for two-decimal floats). <code>header</code> → A string added at the beginning of the file as a header (default is empty). <code>comments</code> → A string prepended to header lines (default is “ <code>#</code> ”).

(Continued)

Table A2.7 (Continued)

<code>np.genfromtxt()</code>	Loads a CSV file into a NumPy array, handling missing values and different data types more flexibly than <code>np.loadtxt()</code>. <code>fname</code> → The file name, file object, or URL from which data is loaded. <code>delimiter</code> → The string or character that separates values (default is whitespace). <code>dtype</code> → Specifies the data type of the resulting array (default is float). <code>skip_header</code> → The number of lines to skip at the start of the file (default is 0). <code>usecols</code> → Specifies which columns to read (e.g., <code>[0, 2]</code> loads only columns 1 and 3). <code>filling_values</code> → Defines default values for missing data (default is <code>np.nan</code> for floats). <code>names</code> If <code>names</code> is True, the field names are read from the first line after the first <code>encodingstr</code>, optional Encoding used to decode the inputfile.
------------------------------	--

Source: Own synthesis based on NumPy documentation.

Both commands give the results:

```
array([ 6, 15])
```

The situation when both ways of use functions/methods give valid result is probably most common; however, there may be various combinations of relationship between both syntaxes.

- For many universal functions (ufuncs), both forms work the same as in the example above.
- Some functions exists only as `np.function()` – it refers mainly to array creation (`np.array`, `np.zeros`, `np.eye`) and extended arithmetic calculation functions (`np.sin`, `np.tan`).

```
arr = np.array([1, 2, 3])

# Exists
print(np.exp(arr))
[ 2.71828183  7.3890561 20.08553692]

print(arr.exp()) # AttributeError: 'numpy.ndarray' object
has no attribute 'exp'
arr2=np.eye(4) # AttributeError: 'numpy.ndarray' object
has no attribute 'eye'
```

Some operations like `.T` (transposition) and `reshape()` are available only as array methods:

```
arr = np.array([[1, 2], [3, 4]])
```



```
# Works (method of ndarray)
print(arr.T) #
[[1 3] [2 4]]

# Does not work - no standalone function (np.function)
equivalent
print(np.T(arr)) # AttributeError: module 'numpy' has no
attribute 'T'
```

Some operations were benchmarked and indicate the difference in performance: The `array.function()` syntax can sometimes be faster because it avoids additional overhead:

```
arr = np.random.rand(1000000)
np.sum(arr) # Typically slightly slower
arr.sum()   # Typically slightly faster
```

The best practice seems to be usage of `np.function()` when working with various data types, and preferring `array.function()` for NumPy-specific optimizations.

Python indices differ from R indices – counting just starts from 0 which is obvious for most language programmers, but may be misleading for data scientists. Thus, for:

```
a = np.array([1, 2, 3], [4,5,6])
a[1,1]
```

gives 5 not 1 and to access the first (0-indexed) element in first(0-indexed) row the syntax `a[0,0]` should be used with 1 as result

Python data storage order is different than R (and Fortran) data storage order. Data are stored in row by row order, but this behavior is configurable.

In opposite to R, the default way of storing two- and more dimensional data in NumPy is row-major order (row by row).

Both arrays created with the code:

```
nparray1=np.array([list(range(1,6)),list(range(6,11))])
print(nparray1)
```

```
nparray2=np.array(list(range(1,11)))
nparray2.resize(2,5)
print(nparray2)
```

give:

```
[[ 1  2  3  4  5]
 [ 6  7  8  9 10]]
```

For researchers used to using Fortran and R-like conversion, the option `order='F'` is provided in reshape function.

```
nparray3=np.array(list(range(1,11)))
nparray3a=nparray3.reshape(2,5,order="F")
print(nparray3a)
```

```
[[ 1  3  5  7  9]
 [ 2  4  6  8 10]]
```

Np. functions don't implement a variant number of arguments, instead lists are used. The very common error for beginners is to miss the `[]` symbols. This kind of error is easy to spot for arrays creation.

```
import numpy as np
arr = np.array([1, 2, 3]) # Passing a list
arr = np.array(1, 2, 3) # TypeError: array() takes from 1
to 2 positional arguments but 3 were given
```

However, in functions like `np.concat`, `np.vstack`, `np.hstack` this convention may be misleading and quite unintuitive.

```
a = np.array([1, 2])
b = np.array([3, 4])
result = np.concatenate([a, b]) # List of arrays
result = np.concatenate(a, b) # TypeError: concatenate()
takes 1 positional argument but 2 were given

a = np.array([1, 2])
b = np.array([3, 4])
stacked = np.vstack(a, b) # TypeError: vstack() takes 1
positional argument but 2 were given
stacked = np.vstack([a, b]) # Passing a list of arrays
```

There may be more than two axes. NumPy natively processes the three-, four- or more dimensional arrays. Lets suppose that array `data3d` contains beginnings and ends of intervals describing variable in object. To find the centers of those intervals we may use code similar to the one below:

```
data3d=np.array(np.array(random.sample(range(9999), k=200)))
data3d.resize(10,10,2)
mask = data3d[:, :, 0] > data3d[:, :, 1]
temp = data3d[mask, 0].copy()
data3d[mask, 0] = data3d[mask, 1]
data3d[mask, 1] = temp
print(data3d.shape)
```

```
print(data3d[:5, :, :])
print(data3d.mean(axis=2))
```

```
(10, 10, 2)
[[[4867 6320]
  [2557 3223]
  [2769 4232]
  [1744 2079]
  [8127 9350]
  [7871 9601]
  [1026 6436]
  [1361 7386]
  [4026 7765]
  [1225 3947]]
 [ [ 789 1800]
  [3491 8360]
  [3896 4942]
  [2337 3544]
  [ 473 5347]
  [1443 6288]
  [7827 9895]
  [2007 6102]
  [3371 5648]
  [6520 7663]]
 [[1887 3119]
  [2318 2858]
  [2788 6498]
  [6215 8488]
  [1745 2603]
  [1974 5277]
  [1662 9339]
  [4885 6269]
  [ 248 2150]
  [3672 9788]]
 [ [ 745 8242]
  [2992 8244]
  [ 527 9426]
  [3379 8597]
  [9020 9357]
  [3105 8290]
  [5371 8886]
  [6986 9851]
  [4149 9101]
  [ 980 9574]]
 [[1156 3396]
  [ 515 9001]
  [ 298 3555]
  [4186 7246]
  [1054 3798]
```

```

[3809 7544]
[ 894 7003]
[5618 8183]
[ 338 7455]
[3597 6617]]]
Centers of intervals
[[5593.5 2890. 3500.5 1911.5 8738.5 8736. 3731. 4373.5
5895.5 2586. ]
[1294.5 5925.5 4419. 2940.5 2910. 3865.5 8861. 4054.5
4509.5 7091.5]
[2503. 2588. 4643. 7351.5 2174. 3625.5 5500.5 5577.
1199. 6730. ]
[4493.5 5618. 4976.5 5988. 9188.5 5697.5 7128.5 8418.5
6625. 5277. ]
[2276. 4758. 1926.5 5716. 2426. 5676.5 3948.5 6900.5
3896.5 5107. ]
[3579. 1774.5 3421. 7814. 5980. 1524. 6004.5 5303.
3807.5 6928. ]
[4899. 1284. 3170. 1204. 5272.5 4919. 852. 1444.
9350.5 6867. ]
[1933.5 9463.5 7535.5 8529. 8516.5 7758.5 4689. 5119.
738.5 6658.5]
[6154.5 4520. 6795. 8347. 2266. 8968. 4540. 7758.5
3546. 4144. ]
[8960.5 8021. 5367. 5450.5 3764.5 4552. 5799. 4746.5
4607. 4022. ]]

```

Please notice that while the raw data array is three-dimensional, transformation by 3rd (indexed as 2nd) axis reduces it to “normal” two-dimensional array.

Shallow and deep copy mechanisms. A shallow copy in NumPy (e.g., using `arr.view()`) creates a new array object that shares the same data memory as the original, meaning changes in one affects the other, whereas a deep copy (e.g., using `arr.copy()`) creates a completely independent copy of the data, so modifications in one do not affect the other.

```

a = np.array([1, 2, 3])
b = a.view()    # b is a view, changes affect both
c = a.copy()    # c is a deep copy, independent of a
a[0] = 100
print("View:", b) #100,2,3
print("Copy:", c) # 1,2,3

```

A2.27 Operations on matrices

In NumPy, matrix multiplication is performed using the `@` operator or `np.matmul()`, which follows standard linear algebra rules.

```

# Matrix multiplication using @ operator
matrix3 = np.array([[2, 3], [1, 3]])

```

```
matrix4 = np.array([[-1, 2], [-2, 4]])
print(matrix3 @ matrix4) # Matrix multiplication
# or
# print(matrix3.dot(matrix4))
```

To compute the determinant of a square matrix, NumPy provides `np.linalg.det(matrix)`, which returns a scalar value indicating whether the matrix is singular.

```
# Compute determinant of a square matrix
det_matrix = np.array([[2, -1], [-4, 2]])
print(np.linalg.det(det_matrix)) # Determinant
```

The inverse of a non-singular square matrix can be computed using `np.linalg.inv(matrix)`, which returns a matrix such that $A @ A_inv = I$, where I is the identity matrix.

```
# Compute the inverse of a non-singular square matrix
matrix5 = np.array([[4, 3], [2, -1]])
inverse_matrix5 = np.linalg.inv(matrix5)
print(inverse_matrix5) # Inverse matrix
```

Given a system of linear equations $A * X = B$, `np.linalg.solve(A, B)` computes the unknown matrix X efficiently.

```
# Solve the system A * X = B
A = np.array([[2, 3], [1, 3]])
B = np.array([[8, 16], [-7, 14]])
solution = np.linalg.solve(A, B)
print(solution) # Solution of linear system
```

The transpose of a matrix, which swaps rows and columns, is obtained using `matrix.T`.

```
# Transpose of a matrix
matrix_t = np.array([[1, 3], [4, 5]])
print(matrix_t.T) # Transposed matrix
```

A2.28 Operations on two- or more dimensional structures using indices

A2.28.1 Selection of arrays by individual dimensions

Let's say there is data in an array `x` that is 10 by 10. If we specify a string of values in the array indices, then respectively:

`x[:3, :]` – returns an array consisting of the first three rows of the array `x`

`x[:, [1, 3, 6, 8]]` – returns an array consisting of columns 2, 4, 7, and 9 of array `x` – (remembering that indexing starts from 0)

`x[[2, 3, 5], :5]` – returns an array consisting of rows 3, 4 and 6 of this row and columns 1–5 of the array `x`

`x[[2, 3, 5], :-2]` – returns an array consisting of rows 3, 4 and 6 of this row and columns 1–8 of the array `x`

`excluded_columns = np.delete(x, [2, 5], axis=1)` – returns an array consisting of all rows and columns except columns 3 and 6 of the array `x`

`x[1::2, 1::2]` – returns an array consisting of even rows and even (but odd-indexed) columns of the array `x`

A2.28.2 Selection with condition control

To display only those elements of the array that meet the specified condition, this condition must be specified in the index of the array. For example, for the array created with the following statements:

```
x = np.array([
    [2.51609811, 2.01637889, -2.29660690, -2.38689849],
    [-0.88439329, 0.90350289, 2.29785191, -2.40363831],
    [2.25574143, 2.47826722, 0.53269117, 2.13913917],
    [-0.09888813, -0.26250027, -0.84799771, 0.60034724],
    [0.09858682, -0.05320598, -0.34644348, 0.53650563],
    [2.07103444, -0.78758520, -0.15558528, 0.08591188],
    [-0.50501079, -2.14470565, 0.01478956, 0.58878101],
    [-2.37929928, 0.89528323, 2.65159822, -0.41176659],
    [-0.08729011, 2.07769143, 0.92844337, -0.20592205],
    [0.45091464, -0.38330014, 0.78053559, -2.08507603]
])

print("Original Matrix: ")
print(x)
```

To display only those rows for which there are negative values in the 3rd column, issue the command:

```
filtered_x = x[x[:, 2] < 0, :]

print("\nRows where the 3rd column has negative values: ")
print(filtered_x)
```

A2.28.3 Checking how many elements of a vector the condition is valid for

In order to check how many elements of an array a condition holds, use the `sum` function for an array with an appropriately entered condition. For example, to count

how many elements in the first column of the array from the previous example are greater than 0.4, run the following command:

```
count = np.sum(x[:, 0] > 0.4)
[1] 4
```

A2.29 Pandas

The central concept in the Pandas library is `DataFrame`. While we may think of NumPy arrays as a fast effective multidimensional representation of data of the same types, the Pandas `DataFrame` contains heterogeneous data with functionality closer to database tables. Each column in `pandas.DataFrame` is of class `pandas.Series` and `pandas.Series` may be processed separately.

The name of the library comes from the aggregation of panel data: “Python data analysis”.

Pandas is built on top of NumPy, using NumPy arrays as its underlying data structure for efficient storage and computation. Pandas enhances NumPy by providing high-level, labeled data structures (`Series` and `DataFrame`) and flexible indexing, enabling easier manipulation of tabular data. Pandas seamlessly integrates with NumPy, allowing NumPy functions to be applied directly to Pandas objects and supporting conversion between Pandas and NumPy arrays for computational efficiency.

[Table A2.8](#) presents the selected functions of the Pandas library.

Again, in the examples below we will assume that Pandas is loaded with the usual: `import pandas as pd`

`DataFrames` may be created from dictionaries of form: “str”:list or “str”:pd.series. Pd series may be created independently from lists.

```
data = {'Name': ['Anthony', 'Bill', 'Charlie'], 'Age': [25, 30, 35]}
df = pd.DataFrame(data)
print(df)
s = pd.Series([10, 20, 30, 40])
print(s)
datas = pd.DataFrame({'s': s})
print(datas)
```

Indexing, masking, slicing, and selecting the Pandas `DataFrame` is similar to NumPy. However, there are two important mechanisms that are not present in NumPy. In Pandas, `.loc` and `.iloc` are indexing methods for selecting data from a `DataFrame`. `.loc` is label-based indexing, allowing selection using row/column labels or Boolean conditions, and supports slicing with inclusive endpoints. In contrast, `.iloc` is position-based indexing, retrieving data by integer positions (zero-based) and supporting standard Python slicing. While `.loc` allows you to select specific rows/columns using names (e.g., `df.loc["row_label", "col_label"]`), `.iloc` requires numerical indices (e.g., `df.iloc[0, 1]`). Both methods enable the selecting of single values, slices, or subsets of a `DataFrame`, but

Table A2.8 Selected Pandas functions

<code>read_csv</code>	Imports a CSV file into a DataFrame.
<code>read_excel</code>	Imports data from Excel files into a DataFrame.
<code>head</code>	Shows the first few rows of a DataFrame.
<code>tail</code>	Shows the last few rows of a DataFrame.
<code>info</code>	Prints a summary of a DataFrame, including column types and non-null counts.
<code>describe</code>	Generates summary statistics (count, mean, std, etc.) for numeric columns.
<code>drop</code>	Removes specified rows or columns from a DataFrame.
<code>rename</code>	Renames row labels or column names.
<code>fillna</code>	Replaces missing values with a specified value or method.
<code>isna (or isnull)</code>	Detects missing values in a DataFrame.
<code>dropna</code>	Removes rows or columns with missing values.
<code>groupby</code>	Groups data and applies aggregate functions to each group.
<code>pivot</code>	Reshapes data based on column values, turning unique values into separate columns.
<code>pivot_table</code>	Creates a spreadsheet-style pivot table with aggregate functions.
<code>merge</code>	Combines DataFrames based on common columns or indices (SQL-style joins). Left_on – name of the field to join. Right_On how – type of join
<code>join</code>	Combines DataFrames on index or key columns with a simpler syntax than merge. on – name of the field to join how – type of join (see examples below)
<code>concat</code>	Concatenates DataFrames along a particular axis (rows or columns).
<code>astype</code>	Converts a DataFrame column to a specific data type.
<code>sort_values</code>	Sorts a DataFrame by the values of one or more columns.
<code>sort_index</code>	Sorts a DataFrame by its row or column labels.
<code>sample</code>	Randomly selects rows or columns from a DataFrame.
<code>unique</code>	Finds the unique values in a Series.
<code>nunique</code>	Counts the number of distinct values in a Series or DataFrame column.
<code>apply</code>	Applies a function along an axis (rows or columns) of a DataFrame.
<code>applymap</code>	Applies a function elementwise to every cell in a DataFrame.
<code>map</code>	Substitutes each value in a Series with another value, typically from a dict or function.
<code>copy</code>	Creates a deep or shallow copy of a DataFrame or Series.
<code>to_csv</code>	Exports a DataFrame to a CSV file.
<code>to_excel</code>	Exports a DataFrame to an Excel file.
<code>shape</code>	Returns the dimensionality (rows, columns) of a DataFrame.
<code>columns</code>	Returns or sets the column labels of a DataFrame.
<code>index</code>	Returns or sets the row labels of a DataFrame.
<code>dtypes</code>	Shows the data types of each column in a DataFrame.
<code>memory_usage</code>	Estimates the memory usage of a DataFrame.
<code>rename_axis</code>	Renames the axis labels or index levels of a DataFrame.
<code>melt</code>	Unpivots a DataFrame from wide format to long format.
<code>stack</code>	Stacks the prescribed axis of a DataFrame into a Series (longer format).
<code>unstack</code>	Performs the inverse of stack, pivoting a level of the index labels.
<code>resample</code>	Aggregates time-series data based on a new frequency.
<code>rolling</code>	Performs rolling window calculations on time-series or numeric data.

Source: Own synthesis based on Pandas documentation.

`.loc` preserves labels, while `.iloc` strictly follows numerical order. Those methods are preferred over two-dimensional `[]` indexing, which will probably be disabled in future versions of NumPy.

```
data = {
    "Name": ["Anthony", "Bill", "Charlie", "David", "Eve"],
    "Age": [25, 30, 35, 40, 45],
    "Salary": [50000, 60000, 70000, 80000, 90000]
}

df = pd.DataFrame(data, index=["A", "B", "C", "D", "E"]) #
Assigning custom row labels

# Examples of `.iloc[]` (Position-Based Indexing)
iloc_examples = {
    "Single Value (Row 2, Column 1)": df.iloc[2, 1],
    "Entire Row (Row 3)": df.iloc[3],
    "Multiple Rows (Rows 1-3)": df.iloc[1:4],
    "Specific Columns (Rows 1-3, Columns 0 & 2)":
df.iloc[1:4, [0, 2]],
    "Last Row": df.iloc[-1],
    "Every Second Row": df.iloc[::2]
}

# Examples of `.loc[]` (Label-Based Indexing)
loc_examples = {
    "Single Value (Row 'C', Column 'Age')": df.loc["C",
"Age"],
    "Entire Row (Row 'D')": df.loc["D"],
    "Multiple Rows (Rows 'B' to 'D')": df.loc["B":"D"],
    "Specific Columns (Rows 'B' to 'D', Columns 'Name' &
'Salary')": df.loc["B":"D", ["Name", "Salary"]],
    "Filtered Rows (Age > 30)": df.loc[df["Age"] > 30],
    "Last Row (Using Label)": df.loc["E"]
}

#[ ] one dimensional indexing gives access to pd.series in
columns
column_access_examples = {
    "Single Column ('Age')": df["Age"],
    "Multiple Columns ('Name' and 'Salary')": df[["Name",
"Salary"]],
    "Accessing a Single Value ('Age' of Row 3)": df["Age"][2],
    "Filtering (People with Salary > 60,000)":
df[df["Salary"] > 60000],
    "Adding a New Column ('Bonus' = 10% of Salary)":
df.assign(Bonus=df["Salary"] * 0.1),
    "Selecting Rows with Even Indexes from 'Age'":
df["Age"].iloc[::2]
}
```

There is no syntax for accessing columns by their index; however, the following construction may be used in this case:

```
df[df.columns[0]]
```

A2.30 Selection by column names

Let us assume that a DataFrame with five columns is created:

```
data = np.random.rand(10, 5)
df = pd.DataFrame(data, columns=["speed", "velocity",
"combustion", "altitude", "temperature"])
```

`df[["velocity", "combustion"]]` – returns a DataFrame consisting of columns named `velocity` and `combustion` (assuming that there are columns with such names in `df`).

`df["combustion"]` – returns a vector consisting of the column “combustion” (assuming that there are columns with such names in the data `df`).

`df.iloc[:, 0] = np.arange(1, 11)` – set numbers from 1 to 10 into the first column of the `df` DataFrame.

In the *Salaries* set of the *R cardata* package available in the *statsmodel* library, there are data regarding salaries of professors at an American university, the variable `discipline` represents the discipline of science associated with the lecturer (“A” or “B”), the variable `sex` means the gender of the lecturer, `yrs.service` the number of years of service, and the `salary` the salary value.

Command:

```
import statsmodels.api as sm
salaries = sm.datasets.get_rdataset("Salaries", "carData").
data
print(salaries.head())
a = salaries[salaries["discipline"] == "A"]
```

selects lecturers from discipline “A”. Then statement:

```
b = a[a["sex"] == "Male"]
```

selects men and the command: Then command:

```
c = b[b["yrs.service"] > 15]
```

selects lecturers with more than 15 years of professional experience, and the below instructions display the results

```
d = c.sort_values(by="salary", ascending=False)
print(d)
rank discipline yrs.since.phd yrs.service sex salary
1 Prof A 43 43 Small 205500
2 Prof A 42 18 Malé 194800
```

```

3 Prof A 33 18 Small 186023
4 Prof A 27 23 Small 172505
5 Prof A 35 25 Small 168635
6 Prof A 32 28 Small 168500
7 Prof A 40 19 Small 166605
8 Prof A 31 27 Small 163200
...
96 AssocProf A 30 23 Male 74000
97 Prof A 49 43 Small 72300
98 AssocProf A 45 39 Small 70700
99 Prof A 51 51 Malé 57800

```

A2.31 Combine all commands using method chaining

The data filtering routines may be condensed to one command:

```

d = (
    salaries[salaries["discipline"] == "A"]
    .query("sex == 'Male'")
    .query("yrs.service > 15")
    .sort_values(by="salary", ascending=False)
)

print(d)

```

In the next example, the command:

```

medium = salaries[salaries["salary"].between(120000, 180000)]
print(medium)

```

selects all lecturers with an annual salary between <120000,18000>.

In the *diamonds* dataset from the *seaborn* library, there is data describing the price and other attributes of nearly 54,000 diamonds. To display only those with a weight (*carat*) greater than 1, a color (*color*) of “D” or “E”, and a clarity (*clarity*) of “VVS1” or “IF”, sorted by price in ascending order, the following command may be used:

```

import seaborn as sns

# Load diamonds dataset from seaborn
diamonds = sns.load_dataset("diamonds")
filtered_diamonds = (
    diamonds
    .query("carat > 1")
    .query("color in ['D', 'E']")
    .query("clarity in ['VVS1', 'IF']")
    .sort_values(by="price", ascending=True)
)

print(filtered_diamonds.head(123))

```

Selecting only the `carat`, `color`, and `price` columns from the *diamonds* set is done by the following instructions:

```
selected_diamonds = diamonds[["carat", "color", "price"]]
print(selected_diamonds)
```

Changing the `price` column by lowering the value to 90% of the original value can be done by the command:

```
diamonds["price"] = diamonds["price"] * 0.9
print(diamonds)
```

A2.32 Joining DataFrames horizontally (inner/left/right ... join)

The Pandas library also includes functions for connecting datasets horizontally and vertically. Let the `join1.csv` and `join2.csv` files contain data on the employees of a certain company and their positions.

	Name	Surname	PESEL
1	Jan	Duda	91041957778
2	Adrian	Kowalski	96120556257
3	Stefan	Nowak	66110694195
4	Anton	Marushenko	44012723551
5	Michal	Wisniewski	94020772898
6	Adam	Zębik	12242137715

	PESEL	Department	Position
1	12242137715	Construction	Cashier
2	44012723551	Repairs	Seller
3	66110694195	Repairs	Seller
4	91041957778	Construction	Salesman
5	94020772898	Renovations	Senior Specialist
6	96120556257	Ceramics	Cashier

A2.33 Merging data from `join1.csv` and `join2.csv`

We assume:

- `join1.csv` has Name, Surname, and PESEL.
- `join2.csv` has PESEL, Department, and Position.

To join DataFrames horizontally the following commands may be used:

```
j1 = pd.read_csv("join1.csv")
j2 = pd.read_csv("join2.csv")
j = j1.merge(j2, on="PESEL", how="inner")
print(j)
```

As a result of the merger, the resulting set always includes those rows from the first set and from the second set for which the column values appear in both the first and second set. There are variations of that behaviour:

- `left_join` all rows from the first set are inserted into the resulting set, if there is no corresponding data in the second set, they are filled in by NA;
- `right_join` all rows from the second set are inserted into the result set, if there is no corresponding data in the first set, they are completed by NA;
- `full_join` combination of `left_join` and `right_join`;
- `semi_join` all rows from the first set are inserted into the result set that have the corresponding data in the second set, but are not filled in by NA (the result contains the same columns as the first set);
- `anti_join` all non-matching rows from both sets, padded by NA values, are inserted into the result set.

```
left_join_result = j1.merge(j2, on="PESEL", how="left")
right_join_result = j1.merge(j2, on="PESEL", how="right")
full_join_result = j1.merge(j2, on="PESEL", how="outer")
semi_join_result = j1[j1["PESEL"].isin(j2["PESEL"])]
anti_join_result = j1[~j1["PESEL"].isin(j2["PESEL"])]
```

Vertical merging (Unions) may be completed in Pandas with `pd.concat` function.

A2.34 Kaggle API Key acquisition (section 3.3)

To use directly `kaggle.api` inside Python code the process of submitting and downloading authorization credentials are needed.

Acquiring an API key from Kaggle and using it with the Kaggle API inside Python involves several steps. Below is a detailed guide to help you through the process:

Step 1: Create a Kaggle account.

Step 2: Download your Kaggle API key:

- 1 Log in to your Kaggle account.
- 2 Click on your profile picture in the top-right corner of the Kaggle homepage.
- 3 Select “Account” from the dropdown menu.
- 4 Scroll down to the API section.
- 5 Click on the “Create New API Token” button. This will download a file named `kaggle.json` to your computer.

- The `kaggle.json` file contains your API credentials, including your username and API key.

Step 3: Install the Kaggle Python package.

Step 4: Set Up the Kaggle API key:

- 1 Locate the kaggle.json file you downloaded earlier.
- 2 Move the kaggle.json file to the appropriate directory:
 - On Windows: Place it in C:\Users\<YourUsername>\.kaggle\.
 - On Linux/Mac: Place it in ~/.kaggle/.

If the .kaggle directory does not exist, create it.

- 3 Ensure the file permissions are secure:

- On Linux/Mac, run the following command in your terminal:

```
bash
Copy
chmod 600 ~/.kaggle/kaggle.json
```

After acquiring the API_KEY, one can directly import data from Kaggle into Python's working memory using the following commands:

```
from kaggle.api.kaggle_api_extended import KaggleApi

# Set your Kaggle API key (replace 'your_username' and
# 'your_api_key' with actual values) uncomment two
subsequent lines
# in case of problems
# os.environ['KAGGLE_USERNAME'] = "your_username"
# os.environ['KAGGLE_KEY'] = "your_api_key"

dataset_name = "shivamb/vehicle-claim-fraud-detection"
dataset_path = "vehicle-data"
dataset_file_name = "fraud_oracle.csv"

api = KaggleApi()
api.authenticate()

api.dataset_download_file(dataset_name, file_name=dataset_file_
name, path=dataset_path)
```

A2.35 PyTorch installation (section 4.1.4)

PyTorch installation is a little more sophisticated than usual installation with pip. The proper syntax creator wizard is available at: <https://pytorch.org/> (cf. Figure A2.1).

The installation command depends on the operating system and presence of CUDA hardware and drivers in the host system. The installation string for MS Windows without CUDA is, for example, equal to:

```
pip3 install torch torchvision torchaudio
```

NOTE: Latest PyTorch requires Python 3.9 or later.

PyTorch Build	Stable (2.5.1)		Preview (Nightly)	
Your OS	Linux	Mac	Windows	
Package	Conda	Pip	LibTorch	Source
Language	Python		C++ / Java	
Compute Platform	CUDA 11.8	CUDA 12.1	CUDA 12.4	ROCm 6.2
				CPU
Run this Command:	<code>pip3 install torch torchvision torchaudio</code>			

Figure A2.1 PyTorch installation syntax wizard.

Source: <https://pytorch.org/>

A2.36 Neural style transfer with TensorFlow hub pipeline

```
import tensorflow_hub as hub
import tensorflow as tf
import numpy as np
from PIL import Image
import os

def load_image(image_path, image_size=(512, 512)):
    """
    Loads an image from disk, converts it to a float32
    tensor, and resizes.
    The resizing here sets a fixed maximum dimension but
    preserves aspect ratio.
    """
    img = Image.open(image_path)
    if img.mode != "RGB":
        img = img.convert("RGB")

    img.thumbnail(image_size, Image.ANTIALIAS)
    img = np.array(img)
    img = tf.convert_to_tensor(img, dtype=tf.float32)
    img = tf.expand_dims(img, 0)
    return img

def save_image(tensor, output_path):
    """
    Saves a tensor as an image to the specified file.
    """
    tensor = tf.squeeze(tensor, axis=0)
```

```

tensor = tf.clip_by_value(tensor, 0.0, 255.0)
tensor = tf.cast(tensor, tf.uint8)
img = Image.fromarray(tensor.numpy())
img.save(output_path)
print(f"Saved stylized image to: {output_path}")

def main():
    style_image_filenames = [
        "transfer_style_reference_1.jpg",
        "transfer_style_reference_2.jpg",
        "transfer_style_reference_3.jpg",
        "transfer_style_reference_4.jpg",
    ]

    content_image_path = "input.jpg"
    content_image = load_image(content_image_path)

    print("Loading style transfer model from TF Hub... ")
    hub_model = hub.load('https://kaggle.com/models/google/
arbitrary-image-stylization-v1/frameworks/TensorFlow1/
variations/256/versions/1')

    for i, style_path in enumerate(style_image_filenames,
start=1):
        style_image = load_image(style_path)

        print(f"Applying style from {style_path} to
{content_image_path}...")
        outputs = hub_model(tf.constant(content_image),
tf.constant(style_image))
        stylized_image = outputs[0]
        output_filename = f"styled_{i}.jpg"
        save_image(stylized_image, output_filename)

if __name__ == "__main__":
    main()

```

A2.37 creation script:

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC

np.random.seed(25032205)

body_x = np.random.normal(-0.2, 1.1, 400)
body_y = np.random.normal(0, 0.5, 400)

trunk_x = np.random.normal(1.8, 0.2, 100)
trunk_y = -2*trunk_x + np.random.normal(2.5, 0.3, 100)

```


380 *Appendix 2*

```
leg1_x = np.random.normal(-0.7, 0.15, 70)
leg1_y = np.linspace(-0.5,-2,70)+ np.random.normal(0, 0.04, 70)

leg2_x = np.random.normal(-2, 0.15, 50)
leg2_y = np.linspace(-0.3,-1.5,50)+ np.random.normal(0, 0.04, 50)

# Combine the dataset
X = np.vstack([np.column_stack([body_x, body_y]),
               np.column_stack([leg1_x, leg1_y]),
               np.column_stack([leg2_x, leg2_y]),
               np.column_stack([trunk_x, trunk_y])
               ])

y = np.ones(len(X)) # Class label for "Elephant"

noise_x = np.random.uniform(-4, 4, 500)
noise_y = np.random.uniform(-3, 3, 500)
X_noise = np.column_stack([noise_x, noise_y])
y_noise = np.zeros(len(X_noise)) # Class label for
"Background (Balloon-like)"

# Merge data and labels
X_final = np.vstack([X, X_noise])
y_final = np.hstack([y, y_noise])

X_body=np.column_stack([body_x,body_y])
X_rest = np.vstack([np.column_stack([leg1_x, leg1_y]),
                    np.column_stack([leg2_x, leg2_y]),
                    np.column_stack([trunk_x, trunk_y])])
X_final = np.vstack([X_body, X_noise])

clf = SVC(kernel="rbf", C=1, gamma=0.5)
clf.fit(X_final, np.hstack([np.ones(len(X_body)),np.zeros
(len(X_noise))]))

xx, yy = np.meshgrid(np.linspace(-2.5, 2.5, 200),
np.linspace(-2, 2, 200))
Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

fig, axes = plt.subplots(2, 1, figsize=(8, 12))

axes[0].scatter(X[:, 0], X[:, 1], marker='.', label="original
dataset", alpha=0.7, color="gray")
fig.suptitle("Improperly Trained Model Confusing an Elephant
Silhouette with Amoeba")
axes[0].set_xlabel("Feature 1")
```

```

axes[0].set_ylabel("Feature 2")
axes[0].legend()

axes[1].scatter(X[:, 0], X[:, 1], marker='.', label="Elephant
Data", alpha=0.6, color="gray")
axes[1].scatter(body_x, body_y, marker='s', label="Amoeba-
like subset", alpha=0.8, color="gray")
axes[1].contour(xx, yy, clf.decision_function(np.c_[xx.
ravel(), yy.ravel()]).reshape(xx.shape), levels=[0],
linewidths=2, colors="black", linestyle="dashed")
axes[1].set_xlabel("Feature 1")
axes[1].set_ylabel("Feature 2")
axes[1].legend()

plt.tight_layout()
plt.savefig("amoeba_elephant_stacked.jpg")
plt.show()

```

References

- Lutz, M. (2013). *Learning Python* (5th ed.). O'Reilly Media.
- Matthes, E. (2019). *Python crash course: A hands-on, project-based introduction to programming* (2nd ed.). No Starch Press.
- Ramalho, L. (2015). *Fluent Python: Clear, concise, and effective programming*. O'Reilly Media.

Appendix 3

Introduction to R

A3.1 R language

R (R Core Team, 2023), is a statistical computing environment and at the same time a programming language that works in this environment. The creators of the first version of the application are Ross Ihaka and Robert Gentleman from the University of Auckland. The choice of name, coinciding with the initial of the names of both authors, was justified by the fact that it was designed as a free equivalent of the S language used in the commercial S-PLUS software. Since August 1997, its development has been carried out by an international team of programmers called the R Core Team. R is supported from the organizational and financial side by the R Foundation, chaired by Ihaka and Gentleman. The first official version (1.0.0) of the R language was released on 29 February 2000. Since then, new versions of the package have been released regularly, several times a year. Since the end of 2004, versions of the package with the main number 2 have been published; since April 2013 the versions of the package have been numbered in the form 3.<version_number>.<subversion_number>; and since April 2020 in the form of 4..<version_number>.<subversion_number>.

R can run on Linux, Windows, and MacOS. Binary installation versions are available for each of them on the project's homepage in the CRAN (The Comprehensive R Archive Network – <http://cran.r-project.org/>) section. Its source files are also available, and with some practice with Gnu's Not Unix (GNU) tools (<https://www.gnu.org/>), you can compile them and install the program that way.

Among the many statistical applications, R stands out for several advantages (cf. Walesiak & Gatnar 2009):

- it is distributed under the *GPL* license, which means that it can be used for free for any (even commercial) purposes (cf. Kopczewska et al., 2019, p. 18);
- thanks to the built-in package mechanism, it has a very wide functionality. In addition to the routines available in the “pure” R language (version 4.3.2), there are more than twenty thousand packages containing virtually all relevant algorithms for multivariate statistical analysis. Many researchers, when proposing new methods, simultaneously create software in **the R** language that implements them;

- it allows you to freely combine different methods and create your own research procedures;
- allows you to dynamically combine **R** code with libraries written in *C*, which can significantly speed up calculations at critical points of complex algorithms;
- it has very good documentation for both the program and the add-on packages. The PDF files included in the packages and, in many cases, extended by additional vignettes, can often be used as a complete source of knowledge about algorithms. In addition, all user manuals contain references to the most eminent references from the literature on the subject, which makes them excellent starting points for an in-depth literature analysis of the subject;
- it contains a large collection of real data sets and enables the generation of artificial data according to a wide variety of criteria;
- it can import and export data and charts in many popular formats;
- it has the ability to directly retrieve data from many statistical databases and even social networks such as X (Twitter) directly through web APIs (or web services).

All this makes **R** a true “lingua franca” of statistics of the 21st century.

The only “downside” of the **R** environment is the lack of a graphical interface like that offered by commercial statistical frameworks. After gaining some experience in using the program, however, the lack of “windows” ceases to be a problem, while flexibility is more and more appreciated, especially the ability to freely combine methods by treating the output data from one procedure as input to another. Such freedom is not usually offered by commercial packages, and the price to pay for it, consisting of learning a few functions for viewing, editing, and saving data, does not seem too excessive. Especially since the release of version 2.0 of *RStudio* frontend (distributed as a separate program, created in C++ and the Qt library), offering many extensions to the graphical interface of the environment.

A3.1.1 Packages

“Don’t ask if there’s anything you can do in **R**, ask yourself how?” This sentence, originally referring to the *Unix* system, well reflects the multitude of possibilities and implemented algorithms of the **R** environment. The strength of the program lies in the variety of procedures and methods it offers. The vast majority of known algorithms for multivariate statistical analysis are implemented in the **R** language, although due to their diversity and continuous development, not all of them are located in the core of the system, and many are placed in additional packages.

In version 4.3.x, the **R** system has 20,243 packages published on *CRAN* (as of 15 January 2024).

The vast majority of packages are hosted on *CRAN* servers, although some are hosted in other repositories and even GIT version control system servers such as GitHub, GitLab or Bitbucket. Changing the repositories that **R** uses during a session is done by commanding the *Packages|Select repositories...*

For most purposes, choosing the first two repositories is sufficient. However, novice users can select all items and have the access to packages found in all repositories. To use features from packages, you need to do two things:

A3.1.1.1 Install the package on your system

This is a one-time activity. Once a package is installed on your system, it stays there forever. Installation of package(s) can be done using the `install.packages()` function:

```
install.packages(c("bdl","keras"))
Installing packages in 'C:/Users/andrew/Documents/R/
win-library/4.3'
(because 'lib' is not specified)
--- Please select a CRAN mirror to use in this session ---
Installing additional dependencies 'config', 'reticulate',
'tensorflow', 'tfruns'
---
package 'bd1' was successfully unpacked and MD5 sums were checked
package 'keras' was successfully unpacked and MD5 sums were checked

The downloaded binary packages are in
C:\Users\andrew\AppData\Local\Temp\RtmpiCANK1\
downloaded_packages
```

An alternative way to install a package in Windows is to choose *Packages | Install package(s)*. In the next step, you need to select the server from which the package should be downloaded. This server will be used for the rest of the session, unless it is changed by the *Packages|Set up a CRAN mirror....* The next step is to select the package itself.

Sometimes, but less often, package authors do not make them available through CRAN or another web repository. In this case, the package must be downloaded from the website of the author(s), and in this instance, the procedure for installing the package is a little different:

- download the `<package_name_and_version>.zip` file from the author's home-page and save it to any directory on your local disk;
- choose *Packages|Install package(s) from local files...* and point to the file containing the package.

A3.1.1.2 Load the package into memory

Unlike installing a package, loading the package into memory should be done at the beginning of each session in which the data and functions from the package will be used. To do this, use the menu command *Packages|Load the package...* or

function library. The features in the package are not available before the package is loaded, and if you try to use them, you receive an error message, as in the following example:

```
x <- data.Normalization(c(c(1,3,4,2,3), "n1"))
Error: could not find function "data.Normalization"
library(clusterSim)
Loading required package: ade4
Loading required package: cluster
Loading required package: R2HTML
Loading required package: e1071
Loading required package: class
x <- date.Normalization(c(c(1,3,4,2,3), "n1"))
print(x)
[1] -2.4032928 0.3508232 2.2278812 -0.5262348 0.3508232
```

A list of packages **installed** on the system, along with a short description of them, is returned by the `library()` command.

The list of packages **loaded** into memory is returned by the `search()` function:

```
search()
[1] ". GlobalEnv" "package:clusterSim" "package:e1071"
[4] "package:class" "package:R2HTML" "package:cluster"
[7] "Package:ade4" "Package:Stats" "Package:Graphics"
[10] "package:grDevices" "package:utils" "package:datasets"
[13] "package:methods" "Autoloads" "package:base"
```

To permanently remove an installed package, use the `detach("package:nazwa_pakietu")` command. The exception is the base package, which cannot be removed from the R installation.

```
detach("package:base")
Error in detach(pos): detaching "package:base" is not allowed
```

There is no need to “unload” packets from memory, as this is done automatically every time the **R** session ends, or when the *Miscellaneous | Delete all objects* is called.

The `help(package="<package_name>")` function displays more detailed information about the package, its version, dependencies with other packages, content, and authors.

One of the parts of information displayed by the `help(package=...)` function is the package version. Changing the version of a package is in no way related to changing the version of the **R environment** itself (which can be read from the *Help|About menu* or by writing `version()`). What’s more, due to the

relatively long time between subsequent versions of the **R** program, it is very common that several versions of a package usually appear between subsequent versions of the entire environment. The *Packages|Update the packages...* command allows you to update all packages in the system.

A3.2 Foundations of the R language

A3.2.1 Literals

[Table A3.1](#) presents literals (literal constants representing data) found in the **R** environment.

A3.2.2 Comments

The **#** character denotes a comment in **R**. Any statements that occur from the first occurrence of **#** to the end of the line are not interpreted by **R**. As in other

Table A3.1 Literals in R

<i>Literal</i>	<i>Example</i>	<i>Meaning</i>
"<string>" TRUE, T	"Commentary"	Character literal True (Boolean constant)
FALSE, F		False (Boolean constant)
NULL		A literal denoting an empty object (e.g., a string that does not contain any character)
ON		A literal indicating that there is no value associated with it for a given object/variable
Nan		A literal indicating that a numerical error occurred in the calculation of the value of the object/variable (e.g., dividing by zero or taking the root of a negative number)
Inf		A literal denoting infinity, e.g., when dividing a number by 0
<String of numbers>	1	A numeric literal without a subtype
<String of digits>L	20L	Numeric literal integral subtype
<string of digits>.<string of digits>	2.5	Numeric literal floating-point subtype (real numbers)
0x<a string of digits>	0x18	Numeric literal, hexadecimal integral subtype (0x18 corresponds to 24L)
<string of digits>e<string of digits>	1e3	Numeric literal, integer subtype in scientific notation (1e3 corresponds to 1*10^3 i.e. 1000)
pi	print(pi,digits =15)	constant π (in this case 3.141592653589790)

Source: Author's own elaboration based on R documentation.

programming languages, comments are most often used to explain more complex pieces of code, especially in batch mode.

```
for(i in 1:number of media){ # Selection of variables for
the model using the
# capacity of Professor Hellwig's information carrier
coefficients
t <- i
combination=c()
  for(j in 1:number){
    if((t %% 2)==1) combination<-cbind(combination, c(j))
  }
t <- t%%2 # determination of candidate variables for the method
}
# End of calculation of the integral capacity of information
carrier coefficients
}
```

A3.2.3 Object names

An object name in **R** can consist of any string of letters, numbers, underscores, or periods. However, it cannot start with a number, an underscore, or a *dot-digit pair*. Names beginning with periods are usually used to denote internal program variables, and it is not recommended to use them for other purposes, although their use does not generate an error.

R names are case-sensitive, so the variables `data`, `DATA`, and `Data` are three completely different variables that can be used simultaneously in an **R** environment (although using variable names that differ only in case greatly increases the risk of a bug in the program).

The **R language**, unlike many other programming languages, does not require declaring objects and specifying their type before the first use, which on the one hand is a facilitation, but on the other hand requires precision and increased attention, because, for example, a sequence of instructions:

```
> variable_1 <- 5
> if (variable1>4) print("Condition met")
```

results in an error message:

```
Error: object "variable1" not found
```

A3.2.4 Set operator

Setting the value of an expression for a variable can be done in **R** in three ways:

```
variable <- expression
variable = expression
expression -> variable
```


These operators can be freely combined. E.g. issuing a command:

```
variable1=variable2 <- 2+2 -> variable3
```

will cause `variable1`, `variable2`, and `variable3` to be 4.

The `<-` operator can apply not only to numbers, but also to entire objects, for example, vectors or matrices:

```
normalization.column.<- (x - mean(x))/(sd(x))
# sd(standard deviation) - standard deviation, mean - mean
Matrix.sum.<- Matrix1+Matrix.2
```

The fourth type of substitution operator is the `<<-` operator, which is used to create global variables inside a function.

A3.2.5 Attributes

Each R compound object has a defined list of attributes associated with it. Attributes can be column names, variable category names, or array dimensions. To display a list of attributes associated with an object, use the `attributes()` command. An example of how to invoke this command:

```
library(clusterSim)
date(data_binary)
attributes(data_binary)
$names
[1] "v_1", "v_2", "v_3", "v_4", "v_5", "v_6", "v_7", "v_8",
   "v_9", "v_10"
$class
[1] "data.frame"
$row.names
[1] "1" "2" "3" "4" "5" "6" "7" "8"
```

informs that the `data_binary` object is a data frame, contains 10 columns named `v_1`, `v_2`, ..., `v_10`, and eight rows named after their sequential numbers.

A list of the most important attributes that can describe objects is presented in [Table A3.2](#).

Attributes of **R** objects can be displayed and (almost) freely changed using the `attr(object_name, attribute_name)` function. For the `data_binary` object from the previous example, the

```
print(attr(data_binary, "row.names"))
```

Table A3.2 Summary of the most important attributes related to objects in **R**

<i>Attribute</i>	<i>Meaning</i>
class	Object class
col.names	Column names
dim	Array/matrix dimensions
dimnames	The names of the individual dimensions (most often corresponds to the names/col.names and row.names)
levels	Categories (levels) of the nominal variable
names	Names of list items/columns in the data array
row.names	Row names
Tsp	For Time Series: Start, End, and Frequency

Source: Author's own elaboration based on **R** documentation.

causes the following to appear in the **R** environment window:

```
[1] "1" "2" "3" "4" "5" "6" "7" "8"
```

On the other hand, the construction of

```
attr(data_binary, "names") <- c("column_one",
  "column_two", "column_three", "column_four",
  "column_five", "column_six", "column_seven",
  "column_eight", "column_nine", "column_last")
```

renames columns in the data frame.

Until version 2.4 of the **R** language, it was possible to use the list item selector (\$) to display and change attributes, but in newer versions of the program, this technique is not recommended, and the consequence is that appropriate warnings will appear when trying to use it.

A3.2.6 Object types

In **R**, everything is an object. Since statistical calculations are usually performed not on single numbers, but on complex structures, such as matrices or data arrays, **R** objects store not only single numerical values, but also vectors, lists, tables (frames) of data, or multidimensional arrays (cf. [Biecek, 2017](#), pp. 43–87; [Komsta, 2004](#), pp. 7–20). Many packages define their own types of objects, but there are a few basic types of objects (i.e., using object-oriented programming terminology – classes) that are sufficient to program most common tasks solved using the **R** language.

A3.2.6.1 Vectors

A vector is a string of numbers, text strings, or true/false Boolean values. It is one of the most important and commonly used types of objects.

The most common constructor (a function that creates an object of a data type) of a vector is the `c(values_separated_by_commas)` function.

```
vector1 <- c(3,4,2,4,5,7)
```

creates a vector with six elements.

Subsequent elements of the vector are indexed, and access to the components is provided by the `[]` selector. For example, for the variable created above, `vector1[3]` is 2 and `vector1[5]` is 5.

Other vector constructors are:

- `<start>:<end>`

A vector containing all integer values in the range, for example:

```
vector2 <- 2:8
```

is equivalent to the

```
vector2 <- c(2,3,4,5,6,7,8)
```

Statement

```
vector3 <- 5:-1
```

creates a vector consisting of seven elements: 5, 4, 3, 2, 1, 0, -1.

A3.2.6.2 Seq

The `seq` function also returns a sequence of numbers in an interval (or, more precisely, an arithmetic sequence). In its simplest form, it is equivalent to the `:`. In addition, it can be used to determine the difference and/or length of the string and to define the parameters in such a way that the created string consists of numbers with a fractional part. Command

```
vector4 <- seq(3, 6)
```

creates a vector consisting of elements 3, 4, 5, 6.

A statement with a defined string difference (= 5)

```
vector5 <- seq(2, 23, 5)
```

It forms a vector consisting of elements 2, 7, 12, 17, 22.

The next command returns a string of length 4, but the arguments passed to the function are defined in such a way that the difference in the string

calculated from them is not an integer, so the resulting vector contains non-integer elements:

```
vector6 <- seq(12, 25, length.out=4)
print(vector6)
[1] 12.00000 15.66667 20.33333 25.00000
```

The argument to `along.with` `seq` can also be another vector. In this case, the function returns a string of the same length as the length of the string defined by this argument.

Statement

```
vector7 <- seq(10,20, along.with=c(14,23,17,2,6,10))
```

creates a vector consisting of the numbers 10, 12, 14, 16, 18, 20.

A3.2.6.3 *Rep*

The `rep` function returns the vector formed by replicating the elements of the first argument. Example calls to this function produce vectors with the following elements:

```
vector8 <- rep(10, 20)
print(vector8)
[1] 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10
vector9 <- rep(c(10,13,17),5)
print( vector9)
[1] 10 13 17 10 13 17 10 13 17 10 13 17 10 13 17
vector9a <- rep(c(10,13,17),each=5)
print( vector9a)
[1] 10 10 10 10 10 13 13 13 13 13 17 17 17 17 17
vector10 <- rep(c(1,4,-3), c(2,3,5))
# or alternatively
vector10 <- c(rep(1,2),rep(4,3),rep(-3,5))
print(vector10)
[1] 1 1 4 4 4 -3 -3 -3 -3 -3
```

A3.2.6.4 *Sample*

A vector of random numbers in a given interval and length can be created by a command using the `sample` function:

```
vector11 <- sample(1:10, 4)
print(vector11)
[1] 7 6 8 5
vector12 <- sample(1:20, 25, rep=TRUE) # last argument means
that elements can be repeated
```

```
print(vector12)
[1] 14 12 5 7 8 4 9 18 3 17 3 18 13 9 7 8 19 20 10 5 10 15 8 1 18
```

A3.2.6.5 *Vectors generated according to a given distribution*

The group of functions `r-<distribution>` (`rnorm`, `runif`, `rt`, `rweibull` – the full list is available at <https://stat.ethz.ch/R-manual/R-devel/library/stats/html/Distributions.html>) generates a vector of `n` numbers according to the selected distribution. A vector of 10 numbers according to a normal distribution with a mean of 0 and a standard deviation of 1 (default arguments) forms the command:

```
vector13<-runif(10)
print(vector13)
[1] 0.2875775 0.7883051 0.4089769 0.8830174 0.9404673 0.0455565
0.5281055 0.8924190 0.5514350
[10] 0.4566147
```

A vector of 15 numbers according to the student's t-distribution with 6 degrees of freedom forms the command:

```
vector14<-rt(15,df=6)
print(vector14)
[1] 1.6397575 -1.7703227 1.5970612 2.3598845 0.3904608
1.4423866 -1.1187452 -0.3534601 -0.0699085
[10] -0.2522411 -0.8927852 0.5152456 -0.1107163 -0.1501835
-4.6767790
```

A3.2.6.6 *Vector length*

The `length()` function returns the number of elements in a vector. If an object `vector11` is created with a

```
vector15 <- seq(1, 30, 7)
```

then function

```
length(vector15)
```

\returns 5.

A3.2.6.7 *Vector vs simple types*

An **R**-specific feature is that there is no distinction between vectors and variables consisting of single values. A single variable is treated simply as a vector consisting of a single element, even if it is formed by substitution of a `<-`-value variable. So, after issuing the command:

```
variable <- 15
```

`is.vector(variable)` returns TRUE (this function is described in more detail in the section on type conversions)

```
is.vector(variable)
[1] TRUE
```

The length of the variable vector is 1

```
print(length(variable))
[1] 1
```

and the first element of the vector is 15

```
print(variable[1])
[1] 15
```

A3.2.6.7.1 NOMINAL VARIABLE WITH DEFINED CATEGORIES (ENUMERATION TYPE, FACTOR)

A nominal variable with defined categories can be thought of as a structure that allows meaningful names to be given to categories that correspond to individual values assumed by elements. For example, to create a nominal variable with defined categories, you can use the command:

```
factor <- factor(c(2,3,4), levels=1:5)
```

In this case, it has five categories of variables defined. With a nominal variable created in this way, the

```
print(factor)
```

displays

```
[1] 2 3 4
Levels: 1 2 3 4 5
```

i.e., a nominal variable with values 2, 3, 4 with defined categories 1, 2, 3, 4, 5.

To rename categories (levels), use the `levels` attribute. If you change the value of this attribute for a previously created vector, the interpretation of the names of individual categories is also changed. When you issue a command:

```
attr(factor, "levels") <-c("bad", "could_be", "average",
"good", "perfect")
```

or as noted in the previous paragraph

```
levels(factor) <- c("bad", "could be", "average", "good",
"perfect")
```

Redisplaying the factor variable with the print command will result in:

```
[1] could_be average good
Levels: bad could_be average good perfect
```

An attempt to substitute an element not defined in the levels attribute under a nominal variable, with defined categories, ends with an error message, and the value of this element will be unspecified (=NA).

```
factor[3] <- "neutral"
Warning message:
invalid factor level, NAs generated in: ' [<-.factor' ('*tmp*',
  3, value="neutral")
```

If you need to know only the numerical values of the elements of a nominal variable, without interpreting the category names, you can use the `as.numeric()` function.

```
as.numeric(factor)
[1] 2 3 4
```

If you set `ordered` to `TRUE` when you create a nominal variable, you get an ordinal ordered variable (corresponding to the ordinal scale), but because of its rather unclear interpretation in the **R** language, **it is not recommended to use this type of variable.**

A3.2.6.7.2 LIST

The meaning of lists differs from the ones known from Python. Implementing various algorithms for multivariate statistical analysis in **R**, it is quite common for a procedure to result not in a single value, but in a complex object containing a lot of information. For example, if you want to contain the mean, maximum, and minimum values of vector `x` in one object, you would create a list with the `list` function:

```
x <- c(2,3,2,5,4,3,6,7,8)
descriptionStructures <- list(average=mean(x),
minimum=min(x),maximum=max(x))
```

When you display the `StructureDescription` object, all the components are displayed.

```
print(descriptionStructures)
$average
[1] 4.444444
$minimum
[1] 2
$maksimum
[1] 8
```

To refer to the individual components of a compound object, use the `$`selector, for example:

```
print(description$maximum)
```

or the `[[ number ]]` selector to select the item with the specified number. The previous command can also be written as:

```
print(structuredescription)[[3]]
```

However, the latter method is rarely used and applies to lists for which the names of the member attributes have not been defined for various reasons.

If a complex object was created by executing a function and we do not know its structure, the names of the constituent elements are included in the `names` attribute. In the following example, we see the names of the components of the list returned by the `nnet` function, which performs the procedure of recognizing unsolicited e-mails (spam) using a neural network:

```
Library(kernlab)
library(nnet)
data(spam)
testSet<-c(1:100,1001:1100,2201:2400,3101:3200)
spam.nn2 <-nnet(type ~., data = spam, subset = -testSet,
  size = 2, rank = 0.1,
  decay = 5e-4,Hess=FALSE, maxit = 200)
print(names(spam.nn2))
[1] "n" "nunits" "nconn" "conn" "nsunits"
[6] "Decay", "Entropy", "Softmax", "Censored", "Value"
[11] "WTS", "Convergence", "Fitted.values", "Residuals", "Lev"
[16] "Call", "Terms", "Coefnames", "XLeays"
```

More detailed information about the structure of a complex object is displayed by the function `str`, which returns not only the names of its members, but also their types, array lengths, and current values. For the previous example, using `str` instead of `names` will display a lot more data.

```
str(spam.nn2)
List of 19
 $n : num [1:3] 57 2 1
 $nunits : int 61
 $ nconn : num [1:62] 0 0 0 0 0 0 0 0 0 0 0 ...
 $conn : num [1:119] 0 1 2 3 4 5 6 7 8 9 ...
 $nsunits : int 61
 $ decay : num 5e-04
 $entropy : logs TRUE
 $softmax : logs FALSE
 $ censored : logs FALSE
```



```

$ value : num 513
$wts : num [1:119] 1.9009 -0.0891 -0.14 0.1957 -2.6814 ...
$convergence : int 1
$fitted.values: num [1:4101, 1] 0.987 0.985 0.863 0.985
0.863 ...
.. - attr(*, "dimnames")=List of 2
.. .. $ : chr[1:4101] "101" "102" "103" "104" ...
.. .. $ : NULL
$residuals : num [1:4101, 1] 0.0131 0.0148 0.1373 0.0148
0.1373 ...
.. - attr(*, "dimnames")=List of 2
.. .. $ : chr[1:4101] "101" "102" "103" "104" ...
.. .. $ : NULL
$lev : chr [1:2] "nonspam" "spam"
$ call : language nnet.formula(formula = type ~ ., data =
spam, size = 2, rank = 0.1, decay = 5e-04, hess = FALSE,
maxit = 200| __truncated__
$ terms :Classes 'terms', 'formula' language type ~ make +
address + all + num3d + our + over + remove + internet +
order + mail + receive + will + peopl| __truncated__ ...
.. .. - attr(*, "variables")= language list(type, make,
address, all, num3d, our, over, remove, internet, order,
mail, receive, will, people, repor| __truncated__ ...
.. .. - attr(*, "factors")= int [1:58, 1:57] 0 1 0 0 0 0 0
0 0 0 0 ...
.. .. .. - attr(*, "dimnames")=List of 2
.. .. .. .. $ : chr [1:58] "type" "make" "address" "all" ...
.. .. .. .. $ : chr [1:57] "make" "address" "all" "num3d" ...
.. .. - attr(*, "term.labels")= chr [1:57] "make" "address"
"all" "num3d" ...
.. .. - attr(*, "order")= int [1:57] 1 1 1 1 1 1 1 1 1 1 ...
.. .. - attr(*, "intercept")= int 1
.. .. - attr(*, "response")= int 1
.. .. - attr(*, ". Environment")=<environment: R_GlobalEnv>
.. .. - attr(*, "predvars")= language list(type, make,
address, all, num3d, our, over, remove, internet, order,
mail, receive, will, people, repor| __truncated__ ...
.. .. - attr(*, "dataClasses")= Named chr [1:58] "factor"
"numeric" "numeric" "numeric" ...
.. .. .. - attr(*, "names")= chr [1:58] "type" "make"
"address" "all" ...
$coefnames : chr[1:57] "make" "address" "all" "num3d" ...
$ xlevels : Named list()
- attr(*, "class")= chr [1:2] "nnet.formula" "nnet"

```

Based on this information, it can be deduced that the `fitted.values` component is an array with 4101 rows and one column containing the calculated probability values for assignment to classes (spam | non-spam).

A3.2.6.7.3 ARRAY

An array is an object that contains a group of elements of the same type. What distinguishes an array from vectors is the fact that the elements of an array can be indexed in one or more dimensions. To create an array, use the `array(initial_data, dimensions)` function. For example, the command:

```
array1 <- array(0, c(2,3))
```

Creates a 2x3 array filled with zeros. Command

```
array2 <- array(, c(4,2))
```

creates a 4x2 array with no specified values (NA).

Referencing individual elements of the array is also done by the `[]` selector, as in the case of vectors, but inside the selector you must specify indices for all dimensions separated by commas. For example, if `array3` (created with `array3<-rbind(c(1,8,10),c(-7,3,-5),c(1,-8,-5),c(-3,-2,-3))`) contains values

```
      [,1] [,2] [,3]
[1,]   1   8  10
[2,]  -7   3  -5
[3,]   1  -8  -5
[4,]  -3  -2  -3
```

this reference to `array3[2,3]` will return -5, and to `array3[3,1]` will return 1.

Omitting one of the dimensions causes the result object to contain all the elements indexed in that dimension (for two-dimensional arrays, if the index is omitted in the first dimension, the value of the expression is the entire column, and if in the second, the value of the expression is the entire row), i.e. the expression

```
array3[2,]
```

is a vector

```
[1] -7 3 -5
```

and the expression

```
array3[,1]
```

is a vector

```
[1] 1 -7 1 -3
```

R is very flexible when it comes to indexing, selecting, or sorting data in arrays. For a more detailed description and examples of these operations, see [section A3.6](#).

Two-dimensional arrays (and matrices) can also be created by combining vectors and/or other two-dimensional arrays, or even single numerical values (the latter are appropriately duplicated to form a column/row). The `cbind` and `rbind` functions are used for this purpose, with the former combining columns and the latter combining table rows. For example, the

```
array4 <- cbind(1:5, rep(2,5), 5:1,-1)
```

creates an array

```
      [,1] [,2] [,3] [,4]
[1,]  1  2  5 -1
[2,]  2  2  4 -1
[3,]  3  2  3 -1
[4,]  4  2  2 -1
[5,]  5  2  1 -1
```

A statement

```
array5 <- NULL
array5 <- rbind(array5, C(0,30,45,60,90))
array5 <- rbind(array5, sin(array5*PI/180))
```

creates an array

```
      [,1]  0 30.0 45.0000000 60.0000000  90
[2,]  0 0.5 0.7071068 0.8660254  1
```

On the other hand, the

```
array6 <- cbind(rbind(array(1, c(2,2)), array(2, c(3,2))),
               array(3, c(5,2)))
```

creates an array

```
      [,1] [,2] [,3] [,4]
[1,]  1  1  3  3
[2,]  1  1  3  3
[3,]  2  2  3  3
[4,]  2  2  3  3
[5,]  2  2  3  3
```

The `dim` attribute is used to specify the dimensions of the array. For the array created in the previous example, the `dim(array6)` function will return `[1] 5 4`.

In addition, for two-dimensional arrays, the `nrow()` function returns the number of rows in the array, and the `ncol()` function returns the number of its columns.

```
nrow(array6)
[1] 5
ncol(array6)
[1] 4
```

A3.2.6.7.4 MATRIX

A matrix is an array that has two dimensions. Since **R** is a language for statistical calculations, matrices are naturally often used. For this reason, it is a separate data type, but from the user/programmer's point of view, there is not much difference between matrices and two-dimensional arrays, because matrix functions also work if their arguments are two-dimensional arrays. The most convenient way to create a matrix is to use the `matrix` function, which by default treats the input data as successive columns of the matrix. Command

```
matrix1 <- matrix(c(1,3,4,-1,2,4,3,2,1), ncol=3, nrow=3)
```

creates a matrix with three columns (`ncol` argument) and three rows (`nrow` argument)

```
      [,1] [,2] [,3]
[1,]   1  -1   3
[2,]   3   2   2
[3,]   4   4   1
```

By using the `byrow` argument, it is also possible to change the way data is entered to “row by row” (which is standard for Python). Statement

```
matrix2 <- matrix(c(2,3,1,-1,2,3,3,-2,2), ncol=3, nrow=3,
byrow=TRUE)
```

creates a matrix

```
      [,1] [,2] [,3]
[1,]   2   3   1
[2,]  -1   2   3
[3,]   3  -2   2
```

R contains many valuable functions related to matrix calculus.

%% – **matrix multiplication operator**. For example, matrix multiplication $\begin{bmatrix} 2 & 3 \\ 1 & 3 \end{bmatrix} \times \begin{bmatrix} -1 & 2 \\ -2 & 4 \end{bmatrix}$ is performed by a sequence of instructions:

```
matrix3 <- matrix(c(2,3,1,3), ncol=2, nrow=2, byrow=TRUE)
matrix4 <- matrix(c(-1,2,-2,4), ncol=2, nrow=2, byrow=TRUE)
print(matrix3%%matrix4)
      [,1] [,2]
[1,]   -8  16
[2,]   -7  14
```

det() – **compute the determinant of a square matrix**. To compute the determinant of a matrix $\begin{bmatrix} 2 & -1 \\ -4 & 2 \end{bmatrix}$, you can use the

```
det(matrix(c(2,-1,-4,2), ncol=2, nrow=2))
[1] 0
```

solve(A) – **the inverse matrix of the non-singular square matrix A**. In the next example, the inverse matrix of the matrix will be calculated $\begin{bmatrix} 4 & 2 \\ 3 & -1 \end{bmatrix}$. This time the matrix will be created using the array function, because **R** treats two-dimensional arrays the same as matrices.

```
matrix5 <- array(c(4,3,2,-1), c(2,2))
solve(matrix5)
      [,1] [,2]
[1,]  0.1  0.2
[2,]  0.3 -0.4
```

solve(A,B) – **solution of the system $A \cdot X = B$** . Reversing the example of matrix multiplication, we have:

```
solve(matrix(c(2,3,1,3), ncol=2, nrow=2, byrow=TRUE),
      matrix(c(-8.16, -7.14), ncol=2, nrow=2, byrow=TRUE))
      [,1] [,2]
[1,]   -1  2
[2,]   -2  4
```

t(A) – **transposed matrix**. Command

```
t(matrix(c(1,3,4,5), ncol=2, nrow=2, byrow=TRUE))
```

will return a matrix

```
      [,1] [,2]
[1,]    1    4
[2,]    3    5
```

A3.2.6.7.5 DATA FRAME

An important limitation of arrays/matrices concerns the homogeneity of their elements. They can contain numeric data, data in the form of text strings, or TRUE/FALSE Boolean variables, but in a single array this data cannot be mixed, and **R** implicitly converts the data according to these rules:

- if there are text variables in the array, then TRUE/FALSE numbers and values are converted to strings,
- if there are no text variables in the array and they are numeric, then the value TRUE is converted to 1 and the value FALSE to 0.

As long as numerical data is used for calculations, this conversion is not much of a problem. However, if the algorithm can apply to non-metric data, then a data frame is a structure that is more suitable for storing the data, in which individual columns may differ from each other in types. For example, the following instructions:

```
dataframe1 <- data.frame (LETTERS[1:10], 1:10, rep(c(F,T), 5))
names(dataframe1) <- c("Initial","Order","Is even")
```

create a table of data like this:

```
Initial order Is even
1 A 1 FALSE
2 B 2 TRUE
3 C 3 FALSE
4 D 4 TRUE
5 E 5 FALSE
6 F 6 TRUE
7 G 7 FALSE
8 H 8 TRUE
9 I 9 FALSE
10 J 10 TRUE
```

containing a column of strings, a column of numbers, and a column of Boolean values TRUE/FALSE. The example uses the R-standard LETTERS array containing 26 uppercase letters of the Latin alphabet.

The `data.frame` function is rarely used to create data frames, more often processed by the `read.csv` or `read_excel` functions.

A3.2.6.7.6 TIBBLE

A more modern and optimized version of data frames for big data sets are *tibble/tbl_df* objects (this name does not have a Polish translation at the time of writing). To use it, you need to install the `tibble` package. The most important

statistical databases (EUROSTAT, BDL, DBNomics) usually return data in this format.

```
Library(Eurostat)
woman_life_expectancy<-get_eurostat("sdg_03_10",
  filters=list(geo=eu_countries$code,time="2021",sex="F"))
class(woman_life_expectancy)
[1] "tbl_df", "tbl", "data.frame"
tibble::is_tibble(woman_life_expectancy)
[1] TRUE
```

A3.2.7 *Type conversion*

In **R** (if possible) you can change the type using the conversion function. All of these functions are in `as.type_name` format. The conversion can apply to simple types, for example:

```
t <- TRUE
print(as.integer(t))
```

results in

```
[1] 1
```

provided that the action of the

```
"5" + "4"
```

gives an error

```
Error in "5" + "4": non-numeric argument to binary operator
```

because "5" and "4" are text strings in this case, it's an operation

```
as.integer("5") + as.integer("4")
```

resulting in

```
[1] 9
```

Classes of complex objects can also be converted to other classes, as in the following example, which uses functions to convert first to a nominal variable with defined categories, and then back to a vector to display its elements in ascending order for a given vector without repetition:

```
data <-c(2,4,5,3,4,2,3,4,1,3,4,6,7,3,4,5,7)
without_repetition <- as.vector(levels(as.factor(data)))
print(without_repetition)
[1] "1" "2" "3" "4" "5" "6" "7"
```

Table A3.3 Key functions of simple type conversion

<i>as.<...></i>	<i>Converting to...</i>
numeric	A numeric type with no specified subtype
integer	Numeric type, integral
real	Numeric type, real subtype
singles	Numeric type, real subtype
string	Text type
logical	Boolean type (TRUE and FALSE)

Source: Author's own elaboration based on R documentation.

It is also valid to cast arrays to simple types, for example, a sequence of instructions

```
array7 <- array(c(0,1,1,0,1,0), c(2,3))
as.logical(array7)
```

returns vector

```
[1] FALSE TRUE TRUE FALSE TRUE FALSE
```

Tables A3.3 and A3.4 present selected conversion functions. Table A3.3 presents functions for simple types, and Table A3.4 presents conversion functions for classes of complex objects.

The type names from Tables A3.3 and A3.4 can also be used in functions of the `is.type_name` form, which, instead of converting, check whether an argument is of a given type by returning an appropriate Boolean value.

```
> is.logical("String")
[1] FALSE
> is.vector(1:100)
[1] TRUE
```

Table A3.4 Key functions of complex object class conversions

<i>as.<...></i>	<i>Converting to...</i>
vector	Vector
factor	Nominal variable with defined categories
matrix	Matrix (two-dimensional array)
array	Array
List	List
data.frame	Data frame
tibble::as_tibble	Tibble object

Source: Author's own elaboration based on R documentation.

Table A3.5 Arithmetic operators in **R**

<i>Operator</i>	<i>Meaning</i>
+	Addition
-	Subtraction
*	Multiplication
/	Floating-point division (real numbers)
^	Exponentiation
**	Exponentiation (second way)
%/%	Integer division
%%	Remainder of integer division

Source: Author’s own elaboration based on R documentation.

A3.2.8 Operators

A3.2.8.1 Simple arithmetic operators

Table A3.5 presents the meaning of arithmetic operators in the **R** language.

All operators from **Table A3.5** can apply to single numbers (although, as mentioned earlier, they are actually one-element vectors), vectors, arrays, and matrices. There are a few rules that apply:

- Operations on two numbers result in a number.
- Operations on two vectors of the same length result in a vector of the same length.
- Operations in the form of *<vector of numbers> operator <number>* result in a numerical vector in which each element was created by performing an operation on successive elements of the initial sequence and a given number.
- If the arguments of the action are two numerical vectors of different lengths, then the shorter of these vectors is replicated as many times as necessary to be longer or equal to the longer one, and if after replication the lengths of both arguments do not match, the action is executed, but the program generates *a warning message*.
- If both arguments are multidimensional objects, all dimensions of the objects must be compatible in order for the action to be effective.

These principles are worth following with examples:

```
7+8 results in 15
9^.5 results in 3
c(2, 4, 7) *4 results in 8 16 28
1:10^2 returns the squares of consecutive natural numbers from 1 to 10
c(2, 3, -1, 5) *1:2 results in 2 6 -1 10
1:5*3:4 evaluates to 3, 8, 9, 16, 15 and warns that the size of the longer string is
not a multiple of the shorter string
```

Warning message:
longer object length is not a multiple of shorter object
length in: 1:5 * 3:4

`as.logical((1:10)%%2)` will result in a vector

TRUE FALSE TRUE FALSE TRUE FALSE TRUE FALSE TRUE FALSE

(note that this vector checks numbers from 1 to 10 whether it is an odd number (TRUE) or an even number (FALSE)).

While multiplication

```
matrix(c(c(1,3,4,5,2,3), ncol=2, nrow=3, byrow=TRUE) *
matrix(c(-3,2,
-1,2,3,-2), ncol=2, nrow=3, byrow=TRUE)
```

returns the result of a matrix:

```
  [,1] [,2]
[1,]  -3   6
[2,]  -4  10
[3,]   6  -6
```

It is worth noting that the last example is not a matrix multiplication, but only a multiplication of the corresponding elements of an array. On the other hand, the following

```
matrix(c(c(1,3,4,5,2,3), ncol=2, nrow=3, byrow=TRUE) *
matrix(c(3,4,2,-2,3,-4), ncol=3, nrow=2, byrow=TRUE)
```

will end with a dimension mismatch error message:

```
Error in matrix(c(1, 3, 4, 5, 2, 3), ncol=2, nrow=3,
byrow=T) * : non-conformable arrays
```

A3.2.8.2 Substitution for vectors/complex structures

Just as it is possible to perform arithmetic operations on vectors and two- or more dimensional structures, their results can be substituted for other vectors/multidimensional structures. The rules for this substitution are similar to those in the previous section:

- Substitution of another vector (column or row of an array) of the same length for a vector (or column or row of an array) results in the substitution of the corresponding element of the substituted vector at each position of this vector.
- Substitution of a single number for a vector (or a column or row of an array) results in the substitution of the same substituted number at each position of that vector.
- Substitution of another vector (column or row of an array) of different lengths under a vector (or a column or row of an array) is preceded by an appropriate

replication of the substituted vector (if it is shorter), as many times as necessary for it to be longer or equal to this vector, and if the lengths are not different after replication, the substitution is “truncated” and the program generates a warning.

- If the object to be substituted and the object substituting are multidimensional objects with the same number of dimensions, the substitution takes place for all elements if the total number of elements in the object to which the substitution takes place and the object being substituted is the same. At the same time, it should be remembered that the first dimension for a matrix is the one corresponding to the columns, so the results do not always have to be consistent with our intuition.
- If the object to be substituted and the substituting object are multidimensional objects with different dimensions, the substitution is either limited to the appropriate number of dimensions. (If the object substituting has fewer dimensions than the object being substituted it is duplicated to extend to sufficient number of dimensions.)
- If the object to be substituted and the object substituting are multidimensional objects with an equal number of dimensions and additionally the indices of the object to which the substitution takes place are filtered, the substitution is possible only if the corresponding dimensions match.

Again, it is worth following these principles with examples.

Let the variables *x*, *m1*, *m2*, *m3* contain a vector and three matrices, respectively:

```
x<-c(9,1,5,4,3,8,2,10,7,6)
m1<-matrix(c(66,31,50,88,98,93,94,29, 9, 57,
72,14,56,19,21,74,35,51,79, 12,
63,89,10, 8,91,55,15,30,68, 83,
99,17, 2,73,38,32,33,46,60, 64,
78,53,97,45,69,52,85,71,82, 13,
25,92,81,49, 5, 6,96,62,37, 43,
40, 3,58,22,77,59,20,61,95, 1,
11,18,48, 7,34,70,87,41,67, 76,
23,75,47,16,28,39,44,26, 4, 27,
100,65,42,84,80,36,90,24,54, 86),10,10,byrow=TRUE)
m2<-matrix(1:6,2,3)
m3<-matrix(7:12,3,2,byrow=TRUE)
print(x)
[1] 9 1 5 4 3 8 2 10 7 6
print(m1)
      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
[1,] 66  31  50  88  98  93  94  29   9  57
[2,] 72  14  56  19  21  74  35  51  79  12
[3,] 63  89  10   8  91  55  15  30  68  83
[4,] 99  17   2  73  38  32  33  46  60  64
```

```

[5,] 78 53 97 45 69 52 85 71 82 13
[6,] 25 92 81 49 5 6 96 62 37 43
[7,] 40 3 58 22 77 59 20 61 95 1
[8,] 11 18 48 7 34 70 87 41 67 76
[9,] 23 75 47 16 28 39 44 26 4 27
[10,] 100 65 42 84 80 36 90 24 54 86
print(m2)
      [,1] [,2] [,3]
[1,] 1 3 5
[2,] 2 4 6
print(m3)
      [,1] [,2]
[1,] 7 8
[2,] 9 10
[3,] 11 12

```

Assignment of vector for vector:

```

x[]<-1:10
print(x)
[1] 1 2 3 4 5 6 7 8 9 10

```

Assignment of a number to the vector:

```

x[]<-7
print(x)
[1] 7 7 7 7 7 7 7 7 7 7

```

Assignment of a vector for a vector of a different length:

```

x[]<-1:2
print(x)
[1] 1 2 1 2 1 2 1 2 1 2

```

Assignment of a vector that is a column of a matrix under a vector:

```

x[]<-m1[,1]
print(x)
[1] 66 72 63 99 78 25 40 11 23 100

```

Assignment of a vector that is a row of a matrix under a vector:

```

x[]<-m1[9,]
print(x)
[1] 23 75 47 16 28 39 44 26 4 27

```

Assignment of a number for a vector that is a column of an array:

```
m1[,7]<-7
print(m1)
```

	[,1]	[,2]	[,3]	[,4]	[,5]	[,6]	[,7]	[,8]	[,9]	[,10]
[1,]	66	31	50	88	98	93	7	29	9	57
[2,]	72	14	56	19	21	74	7	51	79	12
[3,]	63	89	10	8	91	55	7	30	68	83
[4,]	99	17	2	73	38	32	7	46	60	64
[5,]	78	53	97	45	69	52	7	71	82	13
[6,]	25	92	81	49	5	6	7	62	37	43
[7,]	40	3	58	22	77	59	7	61	95	1
[8,]	11	18	48	7	34	70	7	41	67	76
[9,]	23	75	47	16	28	39	7	26	4	27
[10,]	100	65	42	84	80	36	7	24	54	86

Assignment of a number for all elements of an array:

[illegible]

Vector assignment for all elements of the array (duplication in two dimensions):

[illegible]

Assignment of a matrix under a vector (constraint to the first dimension):

```
x[]<-m2
Warning message:
In x[] <- m2 :
  number of items to replace is not a multiple of replacement
  length
print(x)
[1] 1 2 3 4 5 6 1 2 3 4
```

Assignment of the matrix for the matrix (order determined by the first dimension):

```
m2[,]<-m3[,]
print(m2)
      [,1] [,2] [,3]
[1,]   7  11  10
[2,]   9   8  12
```

Assignment of a matrix for a matrix bounded by filters with mismatched dimensions – an impossible operation:

```
m1[1:2,]<-m3
Error in m1[1:2, ] <- m3 :
  number of items to replace is not a multiple of replacement
  length
```

Assignment of a matrix for a matrix bounded by filters with corresponding dimensions:

```
m1[1:2,5:7]<-m3
print(m1)
      [,1] [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10]
[1,]   1   1   1   1   7  11  10   1   1   1
[2,]   2   2   2   2   9   8  12   2   2   2
[3,]   3   3   3   3   3   3   3   3   3   3
[4,]   4   4   4   4   4   4   4   4   4   4
[5,]   5   5   5   5   5   5   5   5   5   5
[6,]   6   6   6   6   6   6   6   6   6   6
[7,]   7   7   7   7   7   7   7   7   7   7
[8,]   8   8   8   8   8   8   8   8   8   8
[9,]   9   9   9   9   9   9   9   9   9   9
[10,]  10  10  10  10  10  10  10  10  10  10
```

A3.2.8.3 Boolean operators

`==`, `!=`, `>`, `>=`, `<`, `<=` are comparison operators: equal, unequal, greater, greater or equal to, lesser, less than or equal to.

```
2*2==4
[1] TRUE
```

```
7>3
```

```
[1] TRUE
```

```
-7>-3
```

```
[1] FALSE
```

```
-7>=-3
```

```
[1] FALSE
```

Comparison operators, according to the same rules as for arithmetic operators, can apply to vectors. For example, a comparison of

```
c(2, 3, 5, 2, 4) > 5:1
```

will return the values of

```
[1] FALSE, FALSE, TRUE, FALSE, TRUE
```

A comparison

```
c(2, 3, 5, 2, 4) == 2
```

gives

```
[1] TRUE FALSE FALSE TRUE TRUE
```

&& – logical “and”

The condition x is greater than 7 and x is less than 15 can be written as

```
(x>7 && x<15)
```

The condition x does not equal four and x does not equal 6 as

```
(x!=4 && x!=6)
```

|| – logical “or”

The condition x equals 3 or x equals 5 is written as:

```
(x==3 || x==5)
```

! – logical negation

The condition “Object is not a matrix” is written as:

```
(!is.matrix(object))
```

~ (tilde)

The tilde is meaningful for functions related to regression analysis. It separates the name of the explanatory variable from the explanatory names. For example, calling a function

```
lm(demand~price+spend.on.advertisement+competitive.product.
price)
```

means that in the model you are building, the variable that will be explained will be demand, and the variables that will explain it will be its price, advertising spending, and the price of a competing product.

The tilde is often used in conjunction with a dot, and `~.` is an abbreviated notation that means that in the model all variables are explanatory variables, for example:

```
rpart(class ~., data=x)
optFederov(~., full)
```

It is worth remembering that in **R** there are also single `&` and `|` - also performing logical “and” and “or” operations, but they apply to vectors, not single logical conditions.

A3.3 Functions in packages

The strength of the **R** language and environment is the variety of functions that implement methods of multivariate statistical analysis, calculate statistical quantities, or perform many other complex tasks placed in many different packages.

An **R** function is called by specifying its name and call arguments. The name of a function can be preceded by the name of the package in which it appears and the symbols `::`, although this convention is rarely used, usually for functions from different packages with repeated names. In **R** function calls, function arguments can be supplied with or without a name, and may or may not have default values. The exact call syntax for each function is given in its help file.

For example, for the `cluster` function. Gen from the `clusterSim` package defined as:

```
cluster. Gen<-
function(numObjects=50,means=NULL,cov=NULL,fixedCov=TRUE,
model=1,
  dataType="m",numCategories=NULL,numNoisyVar=0,
numOutliers=0,
  rangeOutliers=c(1,10),inputType="csv2",inputHeader=TRUE,inp
utRowNames=TRUE,
  outputCsv="",outputCsv2="",outputColNames=TRUE,outputRowNames=
TRUE)
```


The following five calls accomplish the same thing:

```
data1<-cluster.Gen(100,NULL,NULL,TRUE,5,"m",,12)
data2<-cluster.Gen(100,,,5,"m",12)
data3<-cluster.Gen(numObjects = 100, model = 5, dataType =
"m",numNoisyVar = 12)
data4<-cluster.Gen(100, model = 5, dataType = "m",numNoisyVar
= 12)
data5<-clusterSim::cluster.Gen(numNoisyVar = 12,numObjects =
100, dataType = "m", model = 5)
```

In the first and second invocations, the order of the arguments is fixed; in the third, fourth, and fifth invocations, the names of the arguments are given, in which case their order can be arbitrary. In addition, in the fifth call, the function name is preceded by the package name and the `::` operator.

In function calls, you can also pass indirect arguments for functions that are referenced by the main function. In the declaration, the symbol `...` and, in each case, its meaning and the list of dependent functions, can be found in the help file.

For example, for the

```
data.Normalization (x,type="n0",normalization="column",...)
```

`...` means the argument passed to the functions `sum`, `mean`, `min`, `sd`, `mad` and in particular the argument `na.rm` (which is an argument to these functions and is not an argument to `data.Normalization`) as in the example:

```
Z2 <- data.Normalization(data_ratio,type="n10",normalization=
"row",na.rm=FALSE)
```

A3.4 Mathematical functions

The **R** language offers a set of functions that allows calculations of the values of basic trigonometric, exponential, or logarithmic functions. For example, to calculate the sine of 30 degrees, one would run the command

```
sin(30*pi/180)
[1] 0.5
```

The calculation of the base 2 logarithm of 8192 is done by the

```
log2(8192)
[1] 13
```

[Table A3.6](#) summarizes the most important mathematical functions defined in the **R** language.

Table A3.6 Selected **R** mathematical functions

Function / arguments	Meaning
cos(x)	Cosine of an angle
sin(x)	Sine of an angle
tan(x)	Tangent of an angle
acos(x)	Inverse cosine of an angle
asin(x)	Inverse sine of an angle
atan(x)	Inverse tangent of an angle in the range $<-\pi/2, \pi/2>$
atan2(y, x)	Inverse tangent of an angle in the interval $(-\pi/\pi>$
log	Natural logarithm
log10	Decimal logarithm
log2	Base 2 logarithm
logb(x, base)	Arbitrary base logarithm
log1p(x)	Log(1+x)
exp	Base e exponential function
expm1(x)	$\exp(x) - 1$
factorial(x)	Factorial of a number
sqrt(x)	Root of a number
abs(x)	Absolute value of a number
cosh(x)	Hyperbolic cosine of an angle
sinh(x)	hyperbolic sine of an angle
tanh(x)	Hyperbolic tangent of an angle
acosh(x)	Inverse hyperbolic cosine an angle
asinh(x)	Inverse hyperbolic sine an angle
atanh(x)	Inverse hyperbolic tangent of an angle
round(x,digits)	Rounding a number to a specified number of decimal places
ceiling(x)	Rounding a number to the nearest integer greater than it
floor(x)	Rounding a number to the nearest integer less than it

Source: Author's own elaboration based on R documentation based on Crawley's work (2007).

A3.5 Aggregation functions

Many **R** functions take a vector as an argument, returning a single number calculated from it. These functions perform simple statistical operations, and the most important of them are presented in Table A3.7, in which it is assumed that a vector variable contains natural numbers from 1 to 100 created by the statement:

```
vector <- 1:100
```

The functions in Table A3.7 always calculate a single numeric value. This is true even for matrices. For example, for a matrix created with the statement:

```
matrix6 <- matrix(c(2,4,3,2,5,4,3,2,4,5,1,2,4,5,3), nrow=5,
ncol=3, byrow=TRUE)
print(matrix6)
```

```

  [,1] [,2] [,3]
[1,]  2   4   3
[2,]  2   5   4
[3,]  3   2   4
[4,]  5   1   2
[5,]  4   5   3
```

Table A3.7 Selected aggregate functions

<i>Function</i>	<i>Meaning</i>	<i>Function call</i>	<i>Result</i>
mad	Median absolute D=deviation	mad(vector)	37.065
max	Maximum value	max(vector)	100
mean	Average	mean(vector)	50.5
median	Median	median(vector)	50.5
min	Minimum value	min(vector)	1
Prod	product of elements	prod(vector)	9.332622E+157
Quantile	Quantile 50%	quantile(vector,0.5)	50.5
	30%	quantile(vector,0.3)	30.7
sd	Standard deviation	sd(vector)	29.01149
sum	Sum of elements	sum(vector)	5050

Source: Author’s own elaboration based on R documentation.

function

```
mean(matrix6)
```

displays

[1] 3.266667

Aggregation functions also work if the input contains elements of a Boolean type. In this case, FALSE values are treated as zeros and TRUE values are treated as ones, for example:

```
mean(c(T,T,T,F,T))
```

returns 0.8, and the

```
sum(c(4,5,4,3,5,7,3,6,9))%3==0)
```

will display 4, which is the number of vector elements divisible by 3.

Sometimes, however, instead of calculating the average or other aggregate functions from the entire matrix, it is necessary to calculate the average for each column/row independently. This is made possible by the `apply(array, aggregation_type, aggregation_function)` function. The argument `aggregation_type` can take the following values: 1 – calculations are made for rows, 2 – calculations are made for columns, and `c(1,2)` – calculations are made

for both rows and columns, for three-dimensional arrays. Calls to this function can be anything like the following:

```
apply(matrix6, 1, "mean")
[1] 3.000000 3.666667 3.000000 2.666667 4.000000
apply(matrix6, 2, "sum")
[1] 16 17 16
```

A3.6 Operations on two- or more dimensional structures using indices

One important part of the **R** language, corresponding to lambda expressions in other programming languages, is a set of operators and functions designed for operations on data arrays and tables. A way that seems to be more difficult, but more flexible, is to index data structures appropriately by means of expressions within the `[ , ]` operator. Using the indexing option is quite difficult for users who are not programmers at the same time, and with them in mind, the `dplyr` package has been prepared that performs similar functions with simpler commands. The latter way will be discussed in the [section A3.6.1](#).

A3.6.1 Selection of arrays by individual dimensions

Let's say there is data in an array `x` that is 10 by 10. If we specify a string of values in the array indices, then respectively:

```
x[1:3, ] – returns an array consisting of the first three rows of the array x.
x[, c(2, 4, 7, 9)] – returns an array consisting of 2, 4, 7 and 9th columns of
array x
x[c(3, 4, 5), 1:5] – returns an array consisting of rows 3, 4 and 6 of this row
and columns 1-5 of the array x.
x[, -c(3, 6)] – returns an array consisting of all rows and columns except the
third and sixth columns of the array x.
x[(1:5)*2, (1:5)*2] – returns an array consisting of even rows and even
columns of the array x.
x[, c("velocity", "combustion")] – returns an array consisting of col-
umns named velocity and combustion array x (Assuming that there are columns
with such names in array x).
x$combustion – (for data frames) returns a vector consisting of the column
“combustion”, assuming that there are columns with such names in the data
frame x).
cbind(x$speed, x$ combustion) – (for data frames) returns an array con-
sisting of columns named speed and burning of the data frame x (assuming there
are columns with such names in x).
x[, 1] <- 1:10 – enter the numbers from 1 to 10 into the first column of the x
table.
```

A3.6.2 Selection with condition control

To display only those elements of the array that meet the specified condition, this condition must be specified in the index of the array. For example, to make the array

```
x <- matrix(c(
  2.51609811, -0.88439329, 2.25574143, -0.09888813, 0.09858682,
  2.07103444, -0.50501079, -2.37929928, -0.08729011, 0.45091464,
  2.01637889, 0.90350289, 2.47826722, -0.26250027, -0.05320598,
  -0.78758520, -2.14470565, 0.89528323, 2.07769143, -0.38330014,
  -2.29660690, 2.29785191, 0.53269117, -0.84799771,
  -0.34644348, -0.15558528, 0.01478956, 2.65159822, 0.92844337,
  0.78053559,
  -2.38689849, -2.40363831, 2.13913917, 0.60034724, 0.53650563,
  0.08591188, 0.58878101, -0.41176659, -0.20592205, -2.08507603
), nrow = 10, byrow = TRUE)
print(x)
```

	[,1]	[,2]	[,3]	[,4]
[1,]	2.51609811	2.01637889	-2.29660690	-2.38689849
[2,]	-0.88439329	0.90350289	2.29785191	-2.40363831
[3,]	2.25574143	2.47826722	0.53269117	2.13913917
[4,]	-0.09888813	-0.26250027	-0.84799771	0.60034724
[5,]	0.09858682	-0.05320598	-0.34644348	0.53650563
[6,]	2.07103444	-0.78758520	-0.15558528	0.08591188
[7,]	-0.50501079	-2.14470565	0.01478956	0.58878101
[8,]	-2.37929928	0.89528323	2.65159822	-0.41176659
[9,]	-0.08729011	2.07769143	0.92844337	-0.20592205
[10,]	0.45091464	-0.38330014	0.78053559	-2.08507603

To display only those rows for which there are negative values in the third column, issue the command

```
x[x[,3]<0,]
```

	[,1]	[,2]	[,3]	[,4]
[1,]	2.51609811	2.01637889	-2.2966069	-2.38689849
[2,]	-0.09888813	-0.26250027	-0.8479977	0.60034724
[3,]	0.09858682	-0.05320598	-0.3464435	0.53650563
[4,]	2.07103444	-0.78758520	-0.1555853	0.08591188

A3.6.3 Checking how many elements of a vector the condition is valid for

In order to check how many elements of an array a condition holds, use the `sum` function for an array with an appropriately entered condition. For example, to count how many elements in the first column of the array from the previous example are greater than 0.4, run the following command:

```
sum(x[,1]>0.4)
[1] 4
```

A3.6.3.1 Collation

To sort a vector, use the sort statement

```
sort(c(c(5,8,4,3,11,4,6,3,2)))
```

In this case, it returns:

```
2 3 3 4 4 5 6 8 11
```

Sorting arrays by the content of a selected column is a little different. For example, to sort an array, use the following:

```
# Create the array
array <- matrix(c(
  69, 27, 78,
  18, 7, 28,
  39, 72, 93,
  42, 96, 50,
  92, 76, 49,
  11, 85, 90,
  14, 63, 17
), nrow = 7, byrow = TRUE)
colnames(array) <- c("variable1", "variable2", "variable3")
print(array)
```

	variable1	variable2	variable3
1	69	27	78
2	18	7	28
3	39	72	93
4	42	96	50
5	92	76	49
6	11	85	90
7	14	63	17

in descending order by `variable2`, issue the command

```
sorted <- array[order(array[, "variable2"],
  decreasing=TRUE), ]
print(sorted)
```

	variable1	variable2	variable3
4	42	96	50
6	11	85	90
5	92	76	49
3	39	72	93
7	14	63	17
1	69	27	78
2	18	7	28

A3.6.3.2 Changing the value in a column/row

A field from arrays selected by filtering mechanisms can be changed by a substitution statement (with the limitations described in [section A3.2.8.2](#)).

For a tab array with 7 rows and 5 columns:

```
tab <- array(c(
  0.56911257, 0.09053295, 0.84315577, 0.11848042, 0.84234902,
  0.74942732, 0.28903022,
  0.83966961, 0.31713652, 0.40104607, 0.03188484, 0.90281947,
  0.17683654, 0.16214515,
  1.0000000, 0.1428571, 0.2000000, 0.5000000, 0.1666667,
  0.2500000, 0.3333333,
  0.50636501, 0.09740855, 0.81447360, 0.62361654, 0.50534079,
  0.62848473, 0.49019316,
  0.82012962, 0.88635066, 0.38771900, 0.77555019, 0.49061076,
  0.09769058, 0.84718891
), dim = c(7, 5))
print(tab)
```

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	0.56911257	0.83966961	1.0000000	0.50636501	0.82012962
[2,]	0.09053295	0.31713652	0.1428571	0.09740855	0.88635066
[3,]	0.84315577	0.40104607	0.2000000	0.81447360	0.38771900
[4,]	0.11848042	0.03188484	0.5000000	0.62361654	0.77555019
[5,]	0.84234902	0.90281947	0.1666667	0.50534079	0.49061076
[6,]	0.74942732	0.17683654	0.2500000	0.62848473	0.09769058
[7,]	0.28903022	0.16214515	0.3333333	0.49019316	0.84718891

Command execution:

```
tab[,3]<-1/tab[,3]
```

will cause the values of the elements in the third column to take their inverses:

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	0.56911257	0.83966961	1.0000000	0.50636501	0.82012962
[2,]	0.09053295	0.31713652	7.0000002	0.09740855	0.88635066
[3,]	0.84315577	0.40104607	5.0000000	0.81447360	0.38771900
[4,]	0.11848042	0.03188484	2.0000000	0.62361654	0.77555019
[5,]	0.84234902	0.90281947	5.9999999	0.50534079	0.49061076
[6,]	0.74942732	0.17683654	4.0000000	0.62848473	0.09769058
[7,]	0.28903022	0.16214515	3.0000000	0.49019316	0.84718891

and the execution of commands

```
tab<-rbind(tab,NA)
tab[8,]<-apply(tab,2,sum,na.rm=TRUE)
```

will add an additional row to the array with a total of 1...7 rows:

	[,1]	[,2]	[,3]	[,4]	[,5]
[1,]	0.56911257	0.83966961	1.000000	0.50636501	0.82012962
[2,]	0.09053295	0.31713652	7.000002	0.09740855	0.88635066
[3,]	0.84315577	0.40104607	5.000000	0.81447360	0.38771900
[4,]	0.11848042	0.03188484	2.000000	0.62361654	0.77555019
[5,]	0.84234902	0.90281947	5.999999	0.50534079	0.49061076
[6,]	0.74942732	0.17683654	4.000000	0.62848473	0.09769058
[7,]	0.28903022	0.16214515	3.000000	0.49019316	0.84718891
[8,]	3.50208827	2.83153820	28.000001	3.66588238	4.30523972

A3.7 Operations on data structures using the dplyr package

The dplyr package includes many features that make it easier to manipulate complex data structures (data frames and tibble objects). The most important features and operators of this package include:

- `inner_join()` – join by columns with the order;
- `union()` – join by lines;
- `filter()` – filter content by field;
- `arrange()` – sort by field(s);
- `mutate()` – change the values of fields;
- `distinct()` – select only non-repeating objects;
- `select()` – select a specific column(s) from the data frame;
- `%>%` – combine filtering and sorting operations into a pipeline (requires magrittr package);
- `%in%` – when filtering – the operator of containing an element in another set;
- `between()` – when filtering – the operator for containing an element in the interval.

In the `Salaries` dataset of the `carData` package, there are data on the salaries of professors at an American university, the variable `discipline` means the discipline of science associated with the lecturer (“A” or “B”), the variable `sex` means the gender of the lecturer, `yrs.service` the number of years of service, and the `salary` their salary value.

Command:

```
library(carData)
library(dplyr)
a<-filter(Salaries, discipline=="A")
```

selects lecturers from discipline “A”. On the other hand, the

```
b<-filter(a, sex=="Male")
```


selects men and the command

```
c<-filter(b, yrs.service>15)
```

selects lecturers with more than 15 years of professional experience, and the instruction

```
d<-arrange(c,-salary)
```

sorts the data from the highest to the smallest salary

```
print(d)
```

	rank	discipline	yrs.since.phd	yrs.service	sex	salary
1	Prof	A	43	43	Male	205500
2	Prof	A	42	18	Male	194800
3	Prof	A	33	18	Male	186023
4	Prof	A	27	23	Male	172505
5	Prof	A	35	25	Male	168635
6	Prof	A	32	28	Male	168500
7	Prof	A	40	19	Male	166605
8	Prof	A	31	27	Male	163200
...						
96	AssocProf	A	30	23	Male	74000
97	Prof	A	49	43	Male	72300
98	AssocProf	A	45	39	Male	70700
99	Prof	A	51	51	Male	57800

All above commands can be combined into one using the pipelining operator

```
d<-Salaries %>% filter(discipline=="A") %>% filter(sex=="Male")
%>%
filter(yrs.service>15) %>% arrange(-salary)
```

The command

```
medium<-filter(Salaries, between(salary,120000,180000))
```

selects all lecturers with an annual salary between <120000,18000>.

In the set of diamonds of the `ggplot2` package there is data describing the price and other attributes of nearly fifty-five thousand diamonds. To display only those with a weight (variable `carat`) of more than 1 carat, a color (variable `color`) “D” or “E”, and a scale of purity (variable `clarity`) “VV1” or “IF”, sorted from the lowest to the highest price, issue the command:

```
diamonds%>%filter(carat>1)%>% filter(color=="E" | color=="D") %>%
filter(clarity %in% c("VVS1","IF"))%>%
arrange(price) %>% print(n=123)
```

```
# A tibble: 123 x 10
  carat cut      color clarity depth table price      x      y      z
  <dbl> <ord>    <ord> <ord>    <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  1.01 Fair      E      VVS1    66.1  55    8529  6.19  6.1   4.06
2  1.02 Very Good E      VVS1    62.6  56    9498  6.35  6.5   4.02
3  1.02 Very Good E      IF     61.7  60   10029  6.38  6.52  2.98
4  1.04 Premium  E      VVS1    62.5  59   10350  6.41  6.46  4.02
5  1.01 Very Good E      VVS1    62.2  54   10498  6.41  6.31  4.02
6  1.04 Premium  E      VVS1    62.5  59   10539  6.46  6.41  4.02
7  1.01 Good     E      VVS1    62.1  59   10567  6.31  6.34  2.99
...
121 1.07 Premium  D      IF     60.9  58   18279  6.67  6.57  4.03
122 1.04 Very Good D      IF     61.3  56   18542  6.53  6.55  4.01
123 1.28 Ideal    E      IF     60.7  57   18700  7.09  6.99  4.27
```

Selecting only the `carat`, `color`, and `price` columns from the diamonds set is done by commanding the following:

```
diamonds%>%select(c(carat,color,price))
# A tibble: 53,940 x 3
  carat color price
  <dbl> <ord> <int>
1  0.23  E      326
2  0.21  E      326
3  0.23  E      327
4  0.290 I      334
5  0.31  J      335
6  0.24  J      336
7  0.24  I      336
8  0.26  H      337
9  0.22  E      337
10 0.23  H      338
# ... with 53,930 more rows
```

Changing the `price` column by lowering the value to 90% of the original value can be done by the command:

```
Diamonds_with_decreased_price<-diamonds%>%mutate(price=0.9*price)
```

The `dplyr` package also includes functions for connecting horizontal and vertical datasets. Let the `join1.csv` and `join2.csv` files contain data on the employees of a certain company and their positions.

```
Name Surname PESEL
1 Jan Duda 91041957778
2 Adrian Kowalski 96120556257
3 Stefan Nowak 66110694195
4 Anton Marushenko 44012723551
5 Michal Wisniewski 94020772898
6 Adam Zębik 12242137715
```

```

PESEL Department Position
1 12242137715 Construction Cashier
2 44012723551 Repairs Seller
3 66110694195 Repairs Seller
4 91041957778 Construction Salesman
5 94020772898 Renovations Senior Specialist
6 96120556257 Ceramics Cashier

```

In order to obtain a complete table containing both names and surnames as well as positions in the order of the first table, the following construction should be used:

```

library(dplyr)
j1<-data.frame(
  Name = c("John", "Adrian", "Stefan", "Anthony", "Michał",
    "Adam"),
  surname = c("Duda", "Kowalski", "Nowak", "Maruszenko",
    "Wiśniewski", "Zębik"),
  PESEL = c("91041957778", "96120556257", "66110694195",
    "44012723551", "94020772898", "12242137715")
)
j2<-data.frame(
  PESEL = c("12242137715", "44012723551", "66110694195",
    "91041957778", "94020772898", "96120556257"),
  Department = c("Construction", "Repairs", "Repairs",
    "Construction", "Repairs", "Ceramics"),
  Job Title = c("Cashier", "Salesperson", "Salesperson",
    "Salesperson", "Senior Specialist", "Cashier")
)
j<- j1 %>% inner_join(j2,by="Pesel")j<- j1 %>%
inner_join(j2,by="Pesel")
print(j)

```

```

Name, Surname, PESEL, Department, Position
1 Jan Duda 91041957778 Construction Salesman
2 Adrian Kowalski 96120556257 Ceramics Cashier
3 Stefan Nowak 66110694195 Repairs Seller
4 Antoni Maruszenko 44012723551 Repairs Seller
5 Michał Wiśniewski 94020772898 Repairs Senior Specialist
6 Adam Zębik 12242137715 Construction Cashier

```

For specifying `inner_join` argument to indicate the name of the column `by` argument may be used. In case the concatenated column in the second table has a different name, it can be defined as `data1%>%inner_join(data2, by=c("nazwa_1"="name _2"))`.

As a result of the merger, the resulting set always includes those rows from the first set and from the second set for which the column values appear in both

the first and second set. There are variations of the inner command that behave differently:

- `left_join()` – all rows from the first set are inserted into the resulting set, if there is no corresponding data in the second set, they are filled in by NA;
- `right_join()` – all rows from the second set are inserted into the result set, if there is no corresponding data in the first set, they are completed by NA;
- `full_join()` – combination of `left_join()` and `right_join()`;
- `semi_join()` – all rows from the first set are inserted into the result set that have the corresponding data in the second set, but are not filled in by NA (the result contains the same columns as the first set);
- `anti_join()` – all non-matching rows from both sets, padded by NA values, are inserted into the result set.

A3.8 Complex programming constructs

A3.8.1 Conditional statement

As in almost all modern programming languages R defines conditional statement with syntax described below:

```
if (condition) statement
if (condition){
  Instruction1
  Instruction2
  ...
  InstructionN
}
if (condition){
  Instruction1
  Instruction2
  ...
  Instruction2
}
else{
  Alternative_Instruction1
  Alternative_Instruction2
  ...
  Alternative_InstructionN
}
```

A statement (first form) or statement string (second form) is executed only if a condition is met. Otherwise, if an else block (the third form) is specified in the statement, the alternate statement string is executed. The following instructions check if a random number between 1 and 100 is even.

```
number <- sample(1:100, 1)
if (number%%2==0){
```

```

print("Even number drawn")
}
else{
  print("Odd number drawn")
}

```

A3.8.2 For loop

Loops are used when, during the analysis, there is a need to perform the same actions for different values of a given parameter (most often these are consecutive values from a specific range, although it is possible to pass the loop through the values of any vector).

For example, displaying the i -th powers of two for and in the interval 1.10 can be done by the script:

```

for(i in 1:10){
  print(paste("2 to the power of ",i,"equals",2^i))
}

```

In contrast, the calculation of the sine for 0,30,45,60 and 90 degrees is performed by the loop

```

for(i in c(0,30,45,60,90)){
  print(paste("sine ",i,"is",sin(pi*i/180)))
}

```

Virtually any set of instructions can be executed inside the loop. In the next example, a loop is made to perform a k-means cluster analysis for the number of classes varying from 2 to 10 and visualize the results in a single graph.

```

library(mlbench)
set.seed(123456)
data<-mlbench.shapes(500)$x
windows()
par(mfrow=c(3,3))
for(i in 2:10){
  classes<-kmeans(data,i)$cluster
  Colors<-Rainbow(i) [Classes]
  plot(data,col=colors,main=paste("Split
into",and,"class(s)"))
}

```

The calculation of the Calinski-Harabasz indices for these classifications is carried out by the following routine:

```

library(mlbench)
library(clusterSim)

```

```

set.seed(123456)
data<-mlbench.shapes(500)$x
for(i in 2:10){
  classes<-kmeans(data,i)$cluster
  ch<-index. G1(data,classes)
  print(paste("The index of Calinski Harabasz when divided
into ",i, "class(s) is ",ch))
}

```

A3.8.3 Loop with searching for data that meets a certain condition

Loops are often combined with conditional statements to determine which elements of a set satisfy a given condition, to find the minimum or maximum value in a given set, to find the first value that satisfies a given condition, etc.

```

library(mlbench)
library(clusterSim)
set.seed(123456)
data<-mlbench.shapes(500)$x
maks_index_CH<-0
opt_no_classes<-2
for(i in 2:10){
  classes<-kmeans(data,i)$cluster
  ch<-index. G1(data,classes)
  if(ch> maks_index_CH){
    opt_no_classes <-i
    maks_index_CH <-ch
  }
}
print(paste("The optimal number of classes found from the CH
index is ", opt_no_classes))

```

In **R**, there are many out-of-the-box functions that make it easier to search for data that satisfies certain conditions, such as `which.min` and `which.max`, which make it easier to write loops of this type. The above fragment can be simplified:

```

library(mlbench)
library(clusterSim)
set.seed(123456)
data<-mlbench.shapes(500)$x
indices<-rep(0.10)
for(i in 2:10){
  classes<-kmeans(data,i)$cluster
  ch<-index. G1(data,classes)
  indexes[i]<-ch
}
print(paste("The optimal number of classes found from the CH
index is ",which.max(indexes)))

```

A3.8.4 Loop with data aggregation

A common use of loops is data aggregation (full or partial). Calculating the sum of elements from a set can be done by a loop:

```
vector<-c(1,4,6,2,3,4,5,6,7,8,2,3,4,5,6,7,8,9,1,2,3)
Total<-0
for(w in vector){
  sum=sum+w
}
print(paste("The sum of vector elements is",sum))
```

Which in **R** can be simplified to:

```
vector<-c(1,4,6,2,3,4,5,6,7,8,2,3,4,5,6,7,8,9,1,2,3)
print(paste("The sum of vector elements is",sum(vector)))
```

Calculating how many times each digit occurs in the vector is carried out by the fragment

```
vector<-c(1,4,6,2,3,4,5,6,7,8,2,3,4,5,6,7,8,9,1,2,3)
instances<-array(0,c(max(vector),2))
instances[,1]<-1:max(vector)
for(w in vector){
  occurrences[w,2]<-occurrences[w,2]+1
}
print("The number of occurrences of each vector element is:")
print(instances)
```

Which, in turn, in **R** can be simplified as:

```
vector<-c(1,4,6,2,3,4,5,6,7,8,2,3,4,5,6,7,8,9,1,2,3)
print("The number of occurrences of each vector element is ")
instances<-cbind(1:max(vector),as.matrix(table(vector)))
print(instances)
```

A3.8.5 While loop

Another “standard” feature of R as modern programming language is loop statement.

The formal definition of while loop statement is presented below:

```
while(condition) statement
while(condition){
  Instruction1
  Instruction2
  ...
  InstructionN
}
```

The statements are executed until the condition is no longer met. For example, the execution of the classification sequence from the [section A3.8.3](#) using the `while` loop may look like:

```
library("clusterSim")
date(data_ratio)
x <- as.matrix(data_ratio)
no_class <- 3
while(liczb_klas<7){
  cl <-kmeans(x, liczba_klas, 50)
  print(cl$cluster)
  no_class <- no_class+1
}
```

A3.9 Other useful functions

R offers many valuable auxiliary function usefull in various implementations of particular problem. Several of them are described in [Table A3.8](#).

Table A3.8 Other important **R** functions

<i>Function</i>	<i>Characteristics</i>
formula	A function used when you don't want to treat an argument passed to another function as a string, but as a formula (usually containing ~). For example, if you want to perform a factor analysis of all variables in the <code>z</code> array, you don't need to pass all variable names to the <code>factanal</code> function in the formula, but you just need to use the function according to the syntax: <code>> factanal(formula(paste("~",paste(names(z),collapse="+")),3).</code>
is.na	Returns a string of values, where TRUE means that at a given position in a vector or array, the value of an element is unspecified Example: <pre>> array8 <- array(c(2,2)) > array8[1,1] <- 3 > array8[2,2] <- 4 > print(array8) [,1] [,2] [1,] 3 ON [2,] ON 4 > is.na(array8) Returns [,1] [,2] [1,] FALSE TRUE [2,] TRUE FALSE The function can also be used to change all unspecified values, e.g. to 5 > array8[is.na(array8)] <- 5 > print(array8) [,1] [,2] [1,] 3 5 [2,] 5 4</pre>

(Continued)

Table A3.8 (Continued)

Function	Characteristics
is.nan	Returns a string of values where TRUE means that an error occurred at a given position in the vector or array when it was calculated. For example, in order to divide completely the corresponding elements of two vectors, assuming that 0 divided by 0 is not to generate an error, and the result of such an operation is also to be 0, you can use e.g. the statement: vector12 <- c(3,0,5,3,5,6,2,5,7,9,13,12,0,5,0) vector13 <- c(2,0,2,1,3,2,1,5,3,2,6,4,0,6,0) result <- vector12%%vector13 print(result) [1] 1 NaN 2 3 1 3 2 1 2 4 2 3 NaN 0 NaN result[is.nan(result)] <- 0 print(result) [1] 1 0 2 3 1 3 2 1 2 4 2 3 0 0 0
is.null	Function to check if a given object is NULL. You cannot use a regular comparison operator for this purpose, as this would generate an error argument of length zero. So, to check if the object is not empty, use the if(!is.null(object)) { ... }
list	Displays all R objects currently stored in memory. In the windowed version, it corresponds to the <i>Misc List objects</i> .
options	Displays (if the function is called without arguments) or sets (in the form of options (parameter=value)) parameters of the R environment, such as the default editor (editor parameter), the default graphics device (device parameter), the number of columns in the printout (width parameter), the way warnings are displayed (warn parameter), etc. Particularly important for Polish users, R 's environment is to use this function to specify that the symbol used to display the decimal separator should be a comma, instead of the default dot via the command: options(OutDec=",").
paste	Text concatenation of variables. This function is often used in conjunction with print when the printout is to be composed of constant text and the value of a variable. Example: > zm1 <- d2<-2 > result <- zm1*zm2 > print(paste(zm1,"*", zm2, "equals", result)) will display the inscription "2 * 2 equals 4" Command: > string<-paste(LETTERS,collapse="_") Creates a variable containing a string: [1] "A_B_C_D_E_F_G_H_I_J_K_L_M_N_O_P_Q_R_S_T_U_V_W_X_Y_Z" In contrast, the

(Continued)

Table A3.8 (Continued)

Function	Characteristics
	<pre>for(i in 1:5){ print(paste(i, "squared equals ", i^2)) }</pre> displays <pre>[1] "1 squared equals 1" [1] "2 squared equals 4" [1] "3 squared equals 9" [1] "4 squared equals 16" [1] "5 squared equals 25"</pre>
rm	Deletes the selected object or all objects from memory. <pre>> rm(list=ls(all=TRUE))</pre> In this form, the function corresponds to the <i>Misc Remove all objects</i> in the windowed version of R.
sink	Redirects the current output stream to a file. When you use the function, all print statements in the executing code and statements displaying data in interactive mode send the text not to the screen, but to the file specified in the function (e.g., <code>sink("session.txt")</code>). The redirection will end when the <code>sink()</code> command is issued.
summary	A function that displays the most important aggregate information about objects. What exactly it displays depends on the class of the object being displayed. E.g., for a nominal variable with defined categories, summary displays information about how many elements there are in a given category <pre>> factor2 <- factor(c(c(1,2,3,4,2,3,4,3,4,1,2,3))) > summary(factor2) 1 2 3 4 2 4 4 3</pre>
switch	Multi-variant formula <pre>of <- switch(normalization_type, no_normalization=cbind(result, x[, i]), standardization=cbind(result, (x[, i] - mean(x[, i]))/(sd(x[, i]))), unitization=(x[, i] - min(x[, i]))/(max(x[, i]) - min(x[, i])))</pre>
write	It works similarly to print, except that the data is written to a file rather than displayed on the screen.

Source: Author's own elaboration based on R documentation based on [Dalgaard \(2002\)](#), [Everitt \(2005\)](#), and [Everitt and Hothorn \(2006\)](#).

A3.10 Creating your own functions

As in other programming languages, functions are pieces of code with a specific name and argument list that return values as a single variable or complex object. An example definition of a function that checks whether a given number is a prime number in R might look like this:

```
is.first <- function(x){
  !sum(x%%2:(x-1)==0) || x==2
}
# If any number in the range <2,x-1> is a divisor x then
FALSE is returned
```

No variables used in functions, except for the return value, are visible to the outside. The only exception to this situation is the use of global variables (the `<<-` operator, assigning assign to the global environment – cf. [Gągolewski, 2014](#), pp. 439–440).

If the function is to return more than one value, it is convenient to use a list in the return statement (the list statement combined with the return).

For example, the procedure of finding the greatest common multiple of two numbers can be implemented in the function

```
GCD <-function(x,y) {
  GCD <-1
  n<-2
  iteration_no<-0
  components<-NULL
  while (n<min(x,y)) {
    if(is.first(n) && (x%%n)==0 && (y%%n)==0) {
      GCD= GCD *n
      x=x%%n
      y=y%%n
      components<-c(components,n)
    }
    else{
      n=n+1
    }
    iteration_no <- iteration_no +1
    Global #zmienna
    g_iteration_no <<- iteration_no
    # equivalently
    #assign(g_iteration_no, iteration_no,. GlobalEnv)
  }
  return (list(number=GCD,components=components))
}
n<-1716
m<-2112
GCD<-GCD(m,n)
print(paste("GCD ",n,"i",m,"is","is",GCD$number))
print(paste("obtained as the product of numbers"))
print(nwd$members)
print(paste("result was achieved after ",g_iteration_
no,"loop steps"))
```

References

- Biecek, P. (2017). *Przewodnik po pakiecie (Guide to the R package)*. GiS Publishing House.
- Crawley, M. J. (2007). *The R book*. Wiley.
- Dalgaard, P. (2002). *Introductory statistics with R*. Springer-Verlag.
- Everitt, B. S. (2005). *An R and S-PLUS® companion to multivariate analysis*. Springer-Verlag.

- Everitt, B. S., & Hothorn, T. (2006). *A handbook of statistical analyses using R*. Chapman & Hall/CRC.
- Gągolewski, M. (2014). *Programowanie w Języku R. Analiza danych, obliczenia, symulacje*, (*Programming in the R language: Data analysis, calculations, simulations*). PWN.
- Komsta, Ł. (2004). *Wprowadzenie do języka R*, (*Introduction to the R language*) [PDF]. CRAN. <http://cran.r-project.org/doc/contrib/Komsta-Wprowadzenie.pdf>
- Kopczewska, K., Kopczewski, T., & Wójcik, P. (2014). *Metody ilościowe w R. Aplikacje ekonomiczne i finansowe* (*Quantitative methods in R: Economic and financial applications* (2nd ed.)). CEDEWU.PL.
- R Core Team. (2023). *R: A language and environment for statistical computing*. R Foundation for Statistical Computing. <http://www.R-project.org>
- Walesiak, M., & Gatnar, E. (Eds.). (2009). *Statystyczna analiza danych z wykorzystaniem programu R* (*Statistical analysis of data using the R program*). PWN.

Appendix 4

R scripts

A4.1 Single-layer perceptron

```
library(R6)

SingleLayerPerceptron <- R6Class("SingleLayerPerceptron",
  public = list(
    weights = NULL,
    all_weights = NULL,

    initialize = function(input_size) {
      0:(input_size-1), divided by input_size.
      self$weights <- c(input_size, 0:(input_size - 1)) /
        input_size
      self$all_weights <- matrix(self$weights, nrow = 1)
    },

    input_extended = function(input) {
      c(1, input)
    },

    predict = function(input) {
      weighted_sum <- sum(self$input_extended(input) *
        self$weights)
      if (weighted_sum > 0) 1 else 0
    },

    train = function(inputs, labels, learning_rate = 0.1,
      epochs = 100) {
      n <- nrow(inputs)
      for (epoch in 1:epochs) {
        convergent <- TRUE
        error_val <- 0
        for (i in 1:n) {
          prediction <- self$predict(inputs[i, ])

```

```

        error <- prediction - labels[i]
        if (error != 0) {
            convergent <- FALSE
        }
        self$weights <- self$weights - learning_rate *
            error * self$input_extended(inputs[i, ])
        error_val <- error
    }
    cat(">>>>> Epoch", epoch, "Error:", error_val, "\n")
    self$all_weights <- rbind(self$all_weights, self$weights)
    if (convergent) break
}
},

predict_array = function(inputs) {
    apply(inputs, 1, self$predict)
}
)
)

training_inputs <- matrix(c(0, 0,
                            0, 1,
                            1, 0,
                            1, 1),
                          nrow = 4, byrow = TRUE)
training_labels <- c(0, 0, 0, 1)

slp <- SingleLayerPerceptron$new(input_size = 2)

slp$train(training_inputs, training_labels, learning_rate =
0.01, epochs = 200)

predicted_labels <- slp$predict_array(training_inputs)

accuracy <- sum(predicted_labels == training_labels) /
length(training_labels)
cat(sprintf("Accuracy: %.2f\n", accuracy))

print(table(Predicted = predicted_labels, Actual = training_
labels))

num_columns <- ncol(slp$all_weights)
par(mfrow = c(2, 2)) for (i in 1:num_columns) {
    plot(slp$all_weights[, i], type = "l",
        main = paste("W", i - 1),
        xlab = "Epoch", ylab = "Weight")
}
plot.new()

```

```

training_inputs <- matrix(c(
  0, 0, 0,
  0, 0, 1,
  0, 1, 0,
  0, 1, 1,
  1, 0, 0,
  1, 0, 1,
  1, 1, 0,
  1, 1, 1
), nrow = 8, byrow = TRUE)

training_labels <- c(0, 1, 1, 1, 1, 1, 1, 0)

slp <- SingleLayerPerceptron$new(input_size = 3)

slp$train(training_inputs, training_labels, learning_rate =
0.01, epochs = 300)

predicted_labels <- slp$predict_array(training_inputs)

accuracy <- sum(predicted_labels == training_labels) /
length(training_labels)
cat(sprintf("Accuracy: %.2f\n", accuracy))

print(table(Predicted = predicted_labels, Actual = training_
labels))
num_columns <- ncol(slp$all_weights)
par(mfrow = c(2, 2), oma = c(0, 0, 2, 0))
for (i in 1:num_columns) {
  plot(slp$all_weights[, i], type = "l",
       main = paste("W", i - 1),
       xlab = "Epoch", ylab = "Weight")
  legend("topright", legend = paste("W", i - 1))
}
mtext("Weights estimation in subsequent epochs (Non-separable
classes)", outer = TRUE, cex = 1.2)

```

A4.2 Multi-layer perceptron

```

library(R6)
MultiLayerPerceptron <- R6Class("MultiLayerPerceptron",
  public = list(
    input_size = NULL,
    hidden_layers = NULL,
    network = NULL,
    all_weights = NULL,

```

```

initialize = function(input_size, hidden_layers) {
  self$input_size <- input_size
  self$hidden_layers <- hidden_layers
  self$network <- list()
  self$all_weights <- list()
},

sigmoid = function(x) {
  1 / (1 + exp(-x))
},

sigmoid_derivative = function(x) {
  x * (1 - x)
},

mse_loss = function(y_true, y_pred) {
  mean((y_true - y_pred)^2)
},

mse_loss_derivative = function(y_true, y_pred) {
  2 * (y_pred - y_true) / length(y_true)
},

initialize_weights = function(size) {
  weights <- (1:size) / size
  dim(weights) <- c(size, 1)
  self$network[[ paste0("W", self$hidden_layers) ]]
    <- weights

  if (self$hidden_layers > 1) {
    for(i in 1:(self$hidden_layers - 1)) {
      weights <- rep(1:size, times = size) / size
      dim(weights) <- c(size, size)
      self$network[[ paste0("W", i) ]] <- weights
    }
  }

  weights <- rep(1:size, times = self$input_size)
    / size
  dim(weights) <- c(self$input_size, size)
  self$network[["W0"]] <- weights
},

forward_propagation = function(X) {
  cache <- list()
  L <- X
  cache[["L0"]] <- L
  for(i in 1:(self$hidden_layers + 1)) {
    key <- paste0("W", i - 1)

```



```

    L <- self$sigmoid(L %*% self$network[[ key ]])
    cache[[ paste0("L", i) ]] <- L
  }
  return(cache)
},

backward_propagation = function(cache, X, Y, learning_
rate) {
  final_key <- paste0("L", self$hidden_layers + 1)
  L <- cache[[ final_key ]]
  L_error <- self$mse_loss_derivative(Y, L)
  L_delta <- L_error * self$sigmoid_derivative(L)

  out_key <- paste0("W", self$hidden_layers)
  self$network[[ out_key ]] <- self$network[[ out_key ]] -
    learning_rate * (t(cache[[ paste0("L", self$hidden_
layers) ]]) %*% L_delta)

  for(i in rev(1:self$hidden_layers)) {
    L_error <- L_delta %*% t(self$network[[ paste0("W",
i) ]])
    L_delta <- L_error * self$sigmoid_derivative(cache[[
paste0("L", i) ]])
    key <- paste0("W", i - 1)
    self$network[[ key ]] <- self$network[[ key ]] -
      learning_rate * (t(cache[[ paste0("L", i - 1) ]])
%*% L_delta)
  }
},

predict = function(input) {
  cache <- self$forward_propagation(input)
  cache[[ paste0("L", self$hidden_layers + 1) ]]
},

train = function(X, Y, learning_rate, epochs) {
  self$initialize_weights(nrow(X))
  for(epoch in 0:(epochs - 1)) {
    cache <- self$forward_propagation(X)
    self$backward_propagation(cache, X, Y, learning_rate)
    self$all_weights[[ length(self$all_weights)
+ 1 ]] <- lapply(self$network, function(w) w)
    if (epoch %% 10 == 0) {
      loss <- self$mse_loss(Y, cache[[ paste0("L",
self$hidden_layers + 1) ]])
      cat(sprintf(">>>Epoch %d, Loss: %f\n", epoch, loss))
    }
  }
},

```

```

    predict_array = function(inputs) {
      apply(inputs, 1, function(row) self$predict
            (matrix(row, nrow = 1)))
    }
  )
)

training_inputs <- matrix(c(
  0, 0, 0,
  0, 0, 1,
  0, 1, 0,
  0, 1, 1,
  1, 0, 0,
  1, 0, 1,
  1, 1, 0,
  1, 1, 1
), nrow = 8, byrow = TRUE)

training_labels <- matrix(c(0, 1, 1, 1, 1, 1, 1, 0), nrow =
8, ncol = 1)

mlp <- MultiLayerPerceptron$new(input_size = 3, hidden_
  layers = 2)

mlp$train(training_inputs, training_labels, learning_rate =
0.1, epochs = 80000)

predicted_values <- mlp$predict_array(training_inputs)
predicted_labels <- ifelse(predicted_values > 0.5, 1, 0)

accuracy <- sum(predicted_labels == training_labels) /
length(training_labels)
cat(sprintf("Accuracy: %.2f\n", accuracy))
print(table(Predicted = predicted_labels, Actual = training_
  labels))

dims <- dim(mlp$all_weights[[1]][["W0"]])
l <- dims[1]
m <- dims[2]
par(mfrow = c(1, m), oma = c(0, 0, 2, 0))
for(i in 1:l) {
  for(j in 1:m) {
    line <- sapply(mlp$all_weights, function(snapshot)
snapshot[["W0"]][i, j])
    plot(line, type = "l", main = paste("W", i - 1, "_", j - 1,
sep = ""),
      xlab = "Epoch", ylab = "Weight")
  }
}

```

```

      legend("topright", legend = paste("W", i - 1, "_", j - 1,
sep = ""))
    }
  }
mtext("Input weights estimation in subsequent epochs\n (Non-
separable classes - MLP)", outer = TRUE, cex = 1.1)
dev.copy(png, filename = "mlp_no_separable_i.png")
dev.off()

dims <- dim(mlp$all_weights[[1]][["W1"]])
l <- dims[1]
m <- dims[2]
par(mfrow = c(1, m), oma = c(0, 0, 2, 0))
for(i in 1:l) {
  for(j in 1:m) {
    line <- sapply(mlp$all_weights, function(snapshot)
snapshot[["W1"]][i, j])
    plot(line, type = "l", main = paste("W", i - 1, "_", j - 1,
sep = ""),
      xlab = "Epoch", ylab = "Weight")
    legend("topright", legend = paste("W", i - 1, "_", j - 1,
sep = ""))
  }
}
mtext("Hidden layers weights estimation in subsequent
epochs\n (Non-separable classes - MLP)", outer = TRUE, cex =
1.1)

```

A4.3 Simple recurrent neural network

```

library(R6)

SimpleRNN <- R6Class("SimpleRNN",
  public = list(
    input_size = NULL,
    hidden_size = NULL,
    output_size = NULL,
    learning_rate = NULL,
    U = NULL,
    W = NULL,
    V = NULL,
    b_h = NULL,
    b_y = NULL,

    initialize = function(input_size, hidden_size, output_
size, learning_rate = 0.01) {
      self$input_size <- input_size
      self$hidden_size <- hidden_size

```

```

self$output_size <- output_size
self$learning_rate <- learning_rate

self$U <- matrix(rnorm(hidden_size * input_size),
nrow = hidden_size, ncol = input_size) * 0.01
self$W <- matrix(rnorm(hidden_size * hidden_size), nrow =
hidden_size, ncol = hidden_size) * 0.01
self$V <- matrix(rnorm(output_size * hidden_size), nrow =
output_size, ncol
LSTM

library(R6)

progress_bar <- function(current, total, loss, bar_length =
40) {
  fraction <- current / total
  arrow_length <- floor(fraction * bar_length)
  arrow <- paste(rep("=", arrow_length), collapse = "")
  blue_arrow <- paste0("\033[94m", arrow, "\033[0m")
  spaces <- paste(rep(" ", bar_length - arrow_length),
collapse = "")

  progress <- sprintf("\rProgress: [%s%s] %6.2f%% loss:%3.2f",
                      blue_arrow, spaces, fraction * 100, loss)
  cat(progress)
  flush.console()
}

LSTM <- R6Class("LSTM",
  public = list(
    input_size = NULL,
    hidden_size = NULL,
    output_size = NULL,
    learning_rate = NULL,

    Wf = NULL, Wi = NULL, Wc = NULL, Wo = NULL,
    bf = NULL, bi = NULL, bc = NULL, bo = NULL,
    V = NULL, b_y = NULL,

    initialize = function(input_size, hidden_size, output_
size, learning_rate = 0.01) {
      self$input_size <- input_size
      self$hidden_size <- hidden_size
      self$output_size <- output_size
      self$learning_rate <- learning_rate

      self$Wf <- matrix(rnorm(hidden_size * (input_size +
hidden_size)),
                        nrow = hidden_size, ncol = input_size +
hidden_size) * 0.01

```

```

        self$Wi <- matrix(rnorm(hidden_size * (input_size +
hidden_size)),
                                nrow = hidden_size, ncol = input_size +
hidden_size) * 0.01
        self$Wc <- matrix(rnorm(hidden_size * (input_size +
hidden_size)),
                                nrow = hidden_size, ncol = input_size +
hidden_size) * 0.01
        self$Wo <- matrix(rnorm(hidden_size * (input_size +
hidden_size)),
                                nrow = hidden_size, ncol = input_size +
hidden_size) * 0.01

        self$bf <- matrix(0, nrow = hidden_size, ncol = 1)
        self$bi <- matrix(0, nrow = hidden_size, ncol = 1)
        self$bc <- matrix(0, nrow = hidden_size, ncol = 1)
        self$bo <- matrix(0, nrow = hidden_size, ncol = 1)

        self$V <- matrix(rnorm(output_size * hidden_
size), nrow = output_size, ncol = hidden_size) * 0.01
        self$b_y <- matrix(0, nrow = output_size, ncol = 1)
    },

    sigmoid = function(x) {
        1 / (1 + exp(-x))
    },

    tanh = function(x) {
        tanh(x)
    },

    forward = function(x) {
        T <- length(x)
        h <- vector("list", T)
        c <- vector("list", T)
        y <- vector("list", T)

        h_prev <- matrix(0, nrow = self$hidden_size, ncol = 1)
        c_prev <- matrix(0, nrow = self$hidden_size, ncol = 1)

        for (t in 1:T) {
            combined <- rbind(h_prev, x[[t]])

            ft <- self$sigmoid(self$Wf %*% combined + self$bf)
            it <- self$sigmoid(self$Wi %*% combined + self$bi)
            c_hat <- self$stanh(self$Wc %*% combined + self$bc)
            ot <- self$sigmoid(self$Wo %*% combined + self$bo)

```

```

c[[t]] <- ft * c_prev + it * c_hat
h[[t]] <- ot * self$tanh(c[[t]])
y[[t]] <- self$V %*% h[[t]] + self$b_y

h_prev <- h[[t]]
c_prev <- c[[t]]
}
return(list(y = y, h = h, c = c))
},

backward = function(x, y_true, y_pred, h, c) {
T <- length(x)

dWf <- matrix(0, nrow = nrow(self$Wf), ncol = ncol(self$Wf))
dWi <- matrix(0, nrow = nrow(self$Wi), ncol = ncol(self$Wi))
dWc <- matrix(0, nrow = nrow(self$Wc), ncol = ncol(self$Wc))
dWo <- matrix(0, nrow = nrow(self$Wo), ncol = ncol(self$Wo))
dbf <- matrix(0, nrow = nrow(self$bf), ncol = ncol(self$bf))
dbi <- matrix(0, nrow = nrow(self$bi), ncol = ncol(self$bi))
dbc <- matrix(0, nrow = nrow(self$bc), ncol = ncol(self$bc))
dbo <- matrix(0, nrow = nrow(self$bo), ncol = ncol(self$bo))
dV <- matrix(0, nrow = nrow(self$V), ncol = ncol(self$V))
db_y <- matrix(0, nrow = nrow(self$b_y), ncol = ncol(self$b_y))

dh_next <- matrix(0, nrow = self$hidden_size, ncol = 1)
dc_next <- matrix(0, nrow = self$hidden_size, ncol = 1)

for (t in T:1) {
  if (t == 1) {
    h_prev <- matrix(0, nrow = self$hidden_size,
                     ncol = 1)
    c_prev <- matrix(0, nrow = self$hidden_size,
                     ncol = 1)
  } else {
    h_prev <- h[[t - 1]]
    c_prev <- c[[t - 1]]
  }

  dy <- y_pred[[t]] - y_true[[t]]
  dV <- dV + dy %*% t(h[[t]])
  db_y <- db_y + dy

  dh <- t(self$V) %*% dy + dh_next

  combined <- rbind(h_prev, x[[t]])

  Wo_activation <- self$Wo %*% combined + self$bo
  ot <- self$sigmoid(Wo_activation)
  do <- dh * self$tanh(c[[t]])
  do <- do * (ot * (1 - ot))

  dc <- dh * ot * (1 - self$tanh(c[[t]])^2) + dc_next

```

```

Wf_activation <- self$Wf %**% combined + self$bf
ft <- self$sigmoid(Wf_activation)
dft <- dc * c_prev
dft <- dft * (ft * (1 - ft))

Wi_activation <- self$Wi %**% combined + self$bi
it <- self$sigmoid(Wi_activation)
Wc_activation <- self$Wc %**% combined + self$bc
c_hat <- self$stanh(Wc_activation)
dit <- dc * c_hat
dit <- dit * (it * (1 - it))
dc_hat <- dc * it
dc_hat <- dc_hat * (1 - c_hat^2)

dWf <- dWf + dft %**% t(combined)
dWi <- dWi + dit %**% t(combined)
dWc <- dWc + dc_hat %**% t(combined)
dWo <- dWo + do %**% t(combined)
dbf <- dbf + dft
dbi <- dbi + dit
dbc <- dbc + dc_hat
dbo <- dbo + do

dcombined <- t(self$Wf) %**% dft +
             t(self$Wi) %**% dit +
             t(self$Wc) %**% dc_hat +
             t(self$Wo) %**% do
dh_next <- dcombined[1:self$hidden_size,, drop =
FALSE]
dc_next <- dc * ft
}
return(list(dWf = dWf, dWi = dWi, dWc = dWc, dWo = dWo,
           dbf = dbf, dbi = dbi, dbc = dbc, dbo = dbo,
           dV = dV, db_y = db_y))
},

update_parameters = function(dWf, dWi, dWc, dWo, dbf,
dbi, dbc, dbo, dV, db_y) {
  self$Wf <- self$Wf - self$learning_rate * dWf
  self$Wi <- self$Wi - self$learning_rate * dWi
  self$Wc <- self$Wc - self$learning_rate * dWc
  self$Wo <- self$Wo - self$learning_rate * dWo
  self$bf <- self$bf - self$learning_rate * dbf
  self$bi <- self$bi - self$learning_rate * dbi
  self$bc <- self$bc - self$learning_rate * dbc
  self$bo <- self$bo - self$learning_rate * dbo
  self$V <- self$V - self$learning_rate * dV
  self$b_y <- self$b_y - self$learning_rate * db_y
},

```

```

    train = function(x_train, y_train, epochs = 100) {
    for (epoch in 1:epochs) {
      total_loss <- 0
      for (i in seq_along(x_train)) {
        forward_out <- self$forward(x_train[[i]])
        y_pred <- forward_out$y
        h <- forward_out$h
        c_vals <- forward_out$c

        loss <- 0
        T <- length(y_pred)
        for (t in 1:T) {
          loss <- loss + sum((y_pred[[t]] - y_train[[i]]
            [[t]])^2)
        }
        total_loss <- total_loss + loss

        grads <- self$backward(x_train[[i]], y_train[[i]],
y_pred, h, c_vals)
        self$update_parameters(grads$dWf, grads$dWi,
grads$dWc, grads$dWo,
                                grads$dbf, grads$dbi,
grads$dbc, grads$dbo,
                                grads$dV, grads$db_y)

        progress_bar(epoch, epochs, total_loss)
      }
      cat("\n")
    }
  },

  predict = function(x) {
    forward_out <- self$forward(x)
    return(forward_out$y)
  }
)
)

set.seed(25032005)

x_train_vals <- seq(0, 8 * pi, length.out = 400)
y_train_vals <- cos(x_train_vals)
y_train <- matrix(y_train_vals, ncol = 1)

sequence_length <- 23
n_seq <- length(x_train_vals) - sequence_length

```


444 *Appendix 4*

```

x_seq <- vector("list", n_seq)
y_seq <- vector("list", n_seq)
for (i in 1:n_seq) {
  x_seq[[i]] <- lapply(y_train[i:(i + sequence_length - 1),
1], function(v) matrix(v, nrow = 1, ncol = 1))
  y_seq[[i]] <- lapply(y_train[(i + 1):(i + sequence_
length), 1], function(v) matrix(v, nrow = 1, ncol = 1))
}

rnn <- LSTM$new(input_size = 1, hidden_size = 10, output_
size = 1, learning_rate = 0.02)
rnn$train(x_seq, y_seq, epochs = 4000)

x_future <- seq(8 * pi, 10 * pi, length.out = 50)

last_seq <- lapply(as.vector(y_train[(nrow(y_train) -
sequence_length + 1):nrow(y_train), 1]),
  function(v) matrix(v, nrow = 1, ncol = 1))
predictions <- numeric(length(x_future))

for (i in 1:length(x_future)) {
  forward_out <- rnn$forward(last_seq)
  y_pred <- forward_out$y
  last_seq <- c(last_seq[-1], list(y_pred[[length(y_pred)]]))
  predictions[i] <- as.numeric(y_pred[[length(y_pred)]])[1]
}

jpeg("cos2_lstm.jpg", width = 800, height = 600)
plot(x_train_vals, y_train_vals, type = "l", col = "blue",
  lwd = 2,
  xlab = "x", ylab = "cos^2(x)", main = "RNN trained to
predict squared cosine wave")
lines(x_future, predictions, col = "red", lwd = 2, lty = 2)
legend("topright", legend = c("Training Data (cos^2)",
"Predictions (8pi to 10pi)"),
  col = c("blue", "red"), lty = c(1, 2), lwd = 2)
dev.off()

plot(x_train_vals, y_train_vals, type = "l", col = "blue",
  lwd = 2,
  xlab = "x", ylab = "cos^2(x)", main = "RNN trained to
predict squared cosine wave")
lines(x_future, predictions, col = "red", lwd = 2, lty = 2)
legend("topright", legend = c("Training Data (cos^2)",
"Predictions (8pi to 10pi)"),
  col = c("blue", "red"), lty = c(1, 2), lwd = 2)

```

A4.4 Vehicle claim fraud detection

```

library(dplyr)
library(caret)
library(keras)
library(tensorflow)
library(ggplot2)
library(caTools)

Sys.setenv(KAGGLE_USERNAME = "your_username")
Sys.setenv(KAGGLE_KEY = "your_api_key")

dataset_name <- "shivamb/vehicle-claim-fraud-detection"
dataset_path <- "vehicle-data"
dataset_file_name <- "fraud_oracle.csv"

download_cmd <- sprintf("kaggle datasets download -d %s -f
%s -p %s",
                        dataset_name, dataset_file_name,
dataset_path)
system(download_cmd)

zip_file <- file.path(dataset_path, paste0(dataset_file_name,
".zip"))
unzip(zip_file, exdir = dataset_path)

df <- read.csv(file.path(dataset_path, dataset_file_name),
stringsAsFactors = FALSE)

columns_to_remove <- c("PolicyNumber", "Days_Policy_
Accident", "Days_Policy_Claim",
                      "Deductible", "NumberOfCars",
"AgeOfPolicyHolder", "Month",
                      "WeekOfMonth", "DayOfWeek",
"DayOfWeekClaimed",
                      "MonthClaimed", "WeekOfMonthClaimed",
"Year")
df_cleaned <- df[, !(names(df) %in% columns_to_remove)]

one_hot_cols <- c("Sex", "AccidentArea", "MaritalStatus",
  "Fault",
                  "PolicyType", "VehicleCategory",
"AgentType", "BasePolicy", "AddressChange_Claim")
dummies <- dummyVars(~., data = df_cleaned[, one_hot_cols],
fullRank = TRUE)
encoded_one_hot_df <- as.data.frame(predict(dummies, newdata =
df_cleaned[, one_hot_cols]))

```

446 *Appendix 4*

```
df_cleaned <- df_cleaned %>% select(-all_of(one_hot_cols)) %>%
  mutate(row_id = row_number()) %>%
  left_join(encoded_one_hot_df %>% mutate(row_id = row_
number()), by = "row_id") %>%
  select(-row_id)

binary_cols <- c("PoliceReportFiled", "WitnessPresent")
df_cleaned <- df_cleaned %>%
  mutate(across(all_of(binary_cols), ~ ifelse(. == "Yes",
1.0, 0.0)))

df_cleaned$VehiclePrice <- recode(df_cleaned$VehiclePrice,
                                '20000 to 29000' = 25000.0,
                                '30000 to 39000' = 35000.0,
                                '40000 to 59000' = 45000.0,
                                'less than 20000' = 10000.0,
                                'more than 69000' = 85000.0,
                                '60000 to 69000' = 65000.0)

df_cleaned$NumberOfSuppliments
<- recode(df_cleaned$NumberOfSuppliments,
          'more than 5' = 8.0,
          '1 to 2' = 1.5,
          '3 to 5' = 4.0,
          'none' = 0.0)

df_cleaned$PastNumberOfClaims
<- recode(df_cleaned$PastNumberOfClaims,
          'more than 4' = 5.0,
          '2 to 4' = 3.0,
          'none' = 0.0,
          '1' = 1.0)

extract_years <- function(value) {
  m <- regmatches(as.character(value), regexpr("^\\d+
years", as.character(value)))
  if (length(m) > 0 && nzchar(m)) {
    as.numeric(sub(" years", "", m))
  } else {
    value
  }
}

df_cleaned$AgeOfVehicle <- df_cleaned$AgeOfVehicle %>%
  replace(. == "new", 0.0) %>%
  replace(. == "more than 7", 10.0) %>%
  apply(extract_years)

embedables <- c("Make", "RepNumber")
for (embedable in embedables) {
```

```

df_cleaned[[paste0(embedable, "_encoded")]] <- as.integer
(as.factor(df[[embedable]]))

v_vocab_size <- length(unique(df_cleaned[[paste0
(embedable, "_encoded")]]))
embedding_dim_v <- floor(sqrt(v_vocab_size))

v_input <- layer_input(shape = c(1), dtype = "int32",
name = paste0(embedable, "_input"))
v_embedding <- v_input %>%
  layer_embedding(input_dim = v_vocab_size + 1, output_dim =
embedding_dim_v, name = paste0(embedable, "_embedding")) %>%
  layer_flatten()
v_model <- keras_model(inputs=v_input, outputs=v_embedding)
v_model %>% compile(optimizer = "adam", loss = "categorical_
crossentropy")

dummy_target <- matrix(0, nrow = nrow(df_cleaned), ncol =
embedding_dim_v)
v_model %>% fit(as.matrix(df_cleaned[[paste0(embedable, "_
encoded")]]), dummy_target,
               epochs = 5, batch_size = 32, verbose = 1)

v_embeddings <- v_model %>% predict(as.matrix(df_cleaned
[[paste0(embedable, "_encoded")]]))
v_embedding_df <- as.data.frame(v_embeddings)
colnames(v_embedding_df) <- paste0(embedable, "_emb_",
0:(embedding_dim_v - 1))

df_cleaned <- df_cleaned %>% select(-all_of(c(embedable,
paste0(embedable, "_encoded"))))
) %>% bind_cols(v_embedding_df)
}

print(head(df_cleaned, 3))

y <- df_cleaned$FraudFound_P
X <- df_cleaned %>% select(-FraudFound_P)

do_boxplot <- function(filename) {
  png(filename, width = 1400, height = 800)
  boxplot(X, horizontal = TRUE, main = "Boxplot of All
Variables in df_cleaned",
          xlab = "Values", ylab = "Variables")
  dev.off()
  boxplot(X, horizontal = TRUE, main = "Boxplot of All
Variables in df_cleaned",
          xlab = "Values", ylab = "Variables")
}

```

```

do_boxplot("discriminant_fraud_boxplot.png")

X$VehiclePrice <- scale(X$VehiclePrice)
do_boxplot("discriminant_fraud_boxplot_price_standarized.jpg")

cols_to_scale <- c("Age", "NumberOfSuppliments", "AgeOfVehicle")
X[, cols_to_scale] <- scale(X[, cols_to_scale])
do_boxplot("discriminant_fraud_boxplot_all_standarized.jpg")

set.seed(123)
split <- sample.split(y, SplitRatio = 0.7)
X_train <- X[split, ]
X_test  <- X[!split, ]
y_train <- y[split]
y_test  <- y[!split]

keras_model <- keras_model_sequential() %>%
  layer_dense(units = 128, activation = 'relu', input_shape =
ncol(X_train)) %>%
  layer_dropout(rate = 0.2) %>%
  layer_dense(units = 64, activation = 'relu') %>%
  layer_dropout(rate = 0.2) %>%
  layer_dense(units = 32, activation = 'relu') %>%
  layer_dropout(rate = 0.2) %>%
  layer_dense(units = 16, activation = 'relu') %>%
  layer_dropout(rate = 0.2) %>%
  layer_dense(units = 1, activation = 'sigmoid')

keras_model %>% compile(
  optimizer = 'adam',
  loss = 'binary_crossentropy',
  metrics = c('accuracy', metric_recall(),
metric_precision())
)

learning <- keras_model %>% fit(
  as.matrix(X_train),
  y_train,
  epochs = 50,
  batch_size = 32,
  validation_data = list(as.matrix(X_test), y_test),
  class_weight = list("0" = 1, "1" = 6),
  verbose = 1
)

```

A4.5 Consumer Price Index (CPI) factors analysis

```

library(dbnomics)
library(dplyr)

```

```

library(tidyr)
library(ggplot2)
library(keras)
library(tensorflow)
library(caret)
library(lime)
library(DALEX)
library(vip)
library(pdp)

Sys.setenv(KAGGLE_USERNAME = "your_username")
Sys.setenv(KAGGLE_KEY = "your_api_key")

imf_codes <- c("AE", "AF", "AG", "AI", "AL", "AM", "AO", "AR", "AT",
               "AU", "AW", "AZ", "BA", "BB", "BD", "BE",
               "BF", "BG", "BH", "BI", "BJ", "BN", "BO", "BR", "BS",
               "BT", "BW", "BY", "BZ", "CA", "CD", "CF",
               "CG", "CH", "CI", "CL", "CM", "CN", "CO", "CR", "CV",
               "CW", "CY", "CZ", "DE", "DJ", "DK", "DM",
               "DO", "DZ", "EC", "EE", "EG", "ES", "ET", "FI", "FJ",
               "FM", "FR", "GA", "GB", "GD", "GE", "GH",
               "GL", "GM", "GN", "GP", "GQ", "GR", "GT", "GW", "GY",
               "HK", "HN", "HR", "HT", "HU", "ID", "IE",
               "IL", "IN", "IQ", "IR", "IS", "IT", "JM", "JO", "JP",
               "KE", "KG", "KH", "KI", "KM", "KN", "KR",
               "KW", "KY", "KZ", "LA", "LB", "LC", "LK", "LR", "LS",
               "LT", "LU", "LV", "LY", "MA", "MD", "ME",
               "MG", "MK", "ML", "MM", "MN", "MO", "MQ", "MR", "MS",
               "MT", "MU", "MV", "MW", "MX", "MY", "MZ",
               "NA", "NC", "NE", "NG", "NI", "NL", "NO", "NP", "NR",
               "NZ", "OM", "PA", "PE", "PG", "PH", "PK",
               "PL", "PS", "PT", "PW", "PY", "QA", "RO", "RS", "RU",
               "RW", "SA", "SB", "SC", "SD", "SE", "SG",
               "SI", "SK", "SL", "SM", "SN", "SO", "SR", "SS", "ST",
               "SV", "SX", "SY", "SZ", "TD", "TG", "TH",
               "TJ", "TL", "TN", "TO", "TR", "TT", "TV", "TZ", "UA",
               "UG", "US", "UY", "UZ", "VC", "VE", "VG",
               "VN", "VU", "WS", "XK", "YE", "ZA", "ZM", "ZW")

wb_codes <- c("ABW", "AFG", "AGO", "AIA", "ALB", "AND", "ANT", "ARE",
              "ARG", "ARM", "ASM", "ATG", "AUS",
              "AUT", "AZE", "BDI", "BEL", "BEN", "BFA", "BGD", "BGR",
              "BHR", "BHS", "BIH", "BLR", "BLZ", "BMU",
              "BOL", "BRA", "BRB", "BRN", "BTN", "BWA", "CAF", "CAN",
              "CHE", "CHL", "CHN", "CIV", "CMR", "COD",
              "COG", "COK", "COL", "COM", "CPV", "CRI", "CUB", "CYM",
              "CYP", "CZE", "DEU", "DJI", "DMA", "DNK",
              "DOM", "DZA", "ECU", "EGY", "ERI", "ESP", "EST", "ETH",
              "FIN", "FJI", "FRA", "FSM", "GAB", "GBR",

```

```

      "GEO", "GHA", "GIN", "GMB", "GNB", "GNQ", "GRC", "GRD",
      "GRL", "GTM", "GUF", "GUM", "GUY", "HKG",
      "HND", "HRV", "HTI", "HUN", "IDN", "IND", "IRL", "IRN",
      "IRQ", "ISL", "ISR", "ITA", "JAM", "JEY",
      "JOR", "JPN", "KAZ", "KEN", "KGZ", "KHM", "KIR", "KNA",
      "KOR", "KWT", "LAO", "LBN", "LBR", "LBY",
      "LCA", "LIE", "LKA", "LSO", "LTU", "LUX", "LVA", "MAC",
      "MAR", "MCO", "MDA", "MDG", "MDV", "MEX",
      "MHL", "MKD", "MLI", "MLT", "MMR", "MNE", "MNG", "MOZ",
      "MRT", "MTQ", "MUS", "MWI", "MYS", "NAM",
      "NER", "NGA", "NIC", "NIU", "NLD", "NOR", "NPL", "NRU",
      "NZL", "OMN", "PAK", "PAN", "PER", "PHL",
      "PLW", "PNG", "POL", "PRI", "PRK", "PRT", "PRY", "PSE",
      "QAT", "REU", "ROU", "RUS", "RWA", "SAU",
      "SDN", "SEN", "SGP", "SLB", "SLE", "SLV", "SMR", "SOM",
      "SRB", "SSD", "STP", "SUR", "SVK", "SVN",
      "SWE", "SWZ", "SYC", "SYR", "TCD", "TGO", "THA", "TJK",
      "TKM", "TLS", "TON", "TTO", "TUN", "TUR",
      "TUV", "TWN", "TZA", "UGA", "UKR", "URY", "USA", "UZB",
      "VCT", "VEN", "VIR", "VNM", "VUT", "WSM",
      "XKX", "YEM", "ZAF", "ZMB", "ZWE")

```

```

indicators_1 <- c("PCPI_PC_CP_A_PT", "LP_PE_NUM", "BCAXF_BP6_
USD", "RAXGFX_USD",
                  "BFDA_BP6_USD", "BK_DB_BP6_USD", "RAFA_G_
USD", "LUR_PT")

```

```

ifs_data_1 <- dbnomics::dbnomics_get_series(provider_code =
"IMF", dataset_code = "IFS",
      dimensions = list(REF_AREA = imf_codes,
FREQ = "A", INDICATOR = indicators_1),
      limit = 3000)
cpi_analysis_df_1 <- ifs_data_1 %>%
  pivot_wider(names_from = INDICATOR, values_from = value) %>%
  drop_na()

```

```

for(cc in indicators_1[3:(length(indicators_1)-1)]) {
  new_col <- gsub("USD", "USD_pc", cc)
  cpi_analysis_df_1[[new_col]] <- cpi_analysis_df_1[[cc]] /
cpi_analysis_df_1$LP_PE_NUM
}
indicators_1 <- gsub("USD", "USD_pc", indicators_1)

```

```

indicators_2 <- c('AG.LND.ARBL.ZS', 'BN.CAB.XOKA.GD.ZS', 'BX.
GSR.CMCP.ZS',
                  'NE.CON.GOVT.KD.ZG', 'NE.IMP.GNFS.ZS')

```

```

series1 <- c()
for(wb in wb_codes) {
  for(ind in indicators_2) {
    series1 <- c(series1, paste0("WB/WDI/A-", ind, "-", wb))
  }
}

```

```

    }
  }
  wdi_data_1 <- data.frame()
  batch_size <- 50
  for(i in seq(1, length(series1), by = batch_size)) {
    cat("Batch", i, "\n")
    batch_series <- series1[i:min(i + batch_size - 1,
length(series1))]
    x <- dbnomics::dbnomics_get_series(batch_series)
    wdi_data_1 <- bind_rows(wdi_data_1, x)
  }
  wdi_data_1 <- wdi_data_1 %>%
    rename(`Reference Area` = `country (label)`) %>%
    drop_na()
  cpi_analysis_df_2 <- wdi_data_1 %>%
    pivot_wider(names_from = indicator, values_from = value) %>%
    drop_na()

  indicators_3 <- c('CC.PER.RNK', 'GE.PER.RNK', 'PV.PER.
RNK', 'RQ.PER.RNK')
  series2 <- c()
  for(wb in wb_codes) {
    for(ind in indicators_3) {
      series2 <- c(series2, paste0("WB/WGI/A-", ind, "-", wb))
    }
  }
  wdi_data_2 <- data.frame()
  for(i in seq(1, length(series2), by = batch_size)) {
    cat("Batch", i, "\n")
    batch_series <- series2[i:min(i + batch_size - 1, length
(series2))]
    x <- dbnomics::dbnomics_get_series(batch_series)
    wdi_data_2 <- bind_rows(wdi_data_2, x)
  }
  wdi_data_2 <- wdi_data_2 %>%
    rename(`Reference Area` = `country (label)`) %>%
    drop_na()
  cpi_analysis_df_3 <- wdi_data_2 %>%
    pivot_wider(names_from = indicator, values_from = value) %>%
    drop_na()

  cpi_analysis_df_1 <- cpi_analysis_df_1 %>% select(-period)
  cpi_analysis_df_2 <- cpi_analysis_df_2 %>% select(-period)
  cpi_analysis_df_3 <- cpi_analysis_df_3 %>% select(-period)

  cpi_analysis_full_df <- merge(cpi_analysis_df_1, cpi_
analysis_df_2,
                                by = c("Reference Area",
"original_period"), all = FALSE)

```



```

cpi_analysis_full_df <- merge(cpi_analysis_full_df, cpi_
analysis_df_3,
                             by = c("Reference Area",
"original_period"), all = FALSE) %>% drop_na()
cpi_analysis_full_df$ind <- paste(cpi_analysis_full_
df$`Reference Area`,
                                cpi_analysis_full_df$original_
                                period, sep = "_")
rownames(cpi_analysis_full_df) <- cpi_analysis_full_df$ind

X <- cpi_analysis_full_df %>% select( c( indicators_1[3:length
th(indicators_1)], indicators_2, indicators_3 ) )
y <- cpi_analysis_full_df[[ indicators_1[1] ]]

set.seed(25032005)
X_train <- X
X_test  <- X
y_train <- y
y_test  <- y

scaler <- preProcess(X_train, method = c("center", "scale"))
X_train_scaled <- predict(scaler, X_train)
X_test_scaled  <- predict(scaler, X_test)

build_model <- function(){
  model <- keras_model_sequential() %>%
    layer_input(shape = ncol(X_train_scaled)) %>%
    layer_dense(units = 64, activation = 'relu') %>%
    layer_dense(units = 32, activation = 'relu') %>%
    layer_dense(units = 1)
  model %>% compile(optimizer = 'adam', loss = 'mean_squared_
error')
  return(model)
}

cpi_model <- build_model()
history <- cpi_model %>% fit(as.matrix(X_train_scaled), y_train,
                           epochs = 300, batch_size = 15,
verbose = 1)

loss_df <- data.frame(epoch = 1:length(history$metrics$loss),
                      loss = history$metrics$loss)
ggplot(loss_df, aes(x = epoch, y = loss)) +
  geom_line() +
  ggtitle("CPI Keras model Learning Curve") +
  xlab("Epoch") + ylab("Loss (Mean Squared Error)") +
  theme_minimal()
ggsave("cpi_learning_curve.jpg")

```

```

predict_function <- function(object, newdata) {
  as.numeric(object %>% predict(as.matrix(newdata)))
}
perm_importance <- vip::vi_permute(cpi_model,
                                   X = as.matrix(X),
                                   Y = y,
                                   metric = "rmse",
                                   pred_wrapper = predict_
                                     function,
                                   nsim = 10,
                                   target = 0)

print(perm_importance$Importance)
importance_df <- perm_importance %>% arrange(desc(Importance))
ggplot(importance_df, aes(x = reorder(Variable, Importance),
y = Importance)) +
  geom_bar(stat = "identity", fill = "skyblue") +
  coord_flip() +
  xlab("Feature") +
  ylab("Permutation Importance (RMSE)") +
  ggtitle("Permutation Feature Importance")
ggsave("cpi_vip_results.jpg")

explainer <- lime::lime(as.data.frame(X_train_scaled), model =
cpi_model, bin_continuous = TRUE)
poland_2020_index <- which(rownames(X) == "Poland_2020")
if(length(poland_2020_index) == 0) {
  stop("Row 'Poland_2020' not found in X")
}
explanation <- lime::explain(as.data.frame(X_test_scaled)
[poland_2020_index,, drop = FALSE],
                        explainer, n_features = 15)
print(explanation)
p_lime <- plot_features(explanation)
ggsave("cpi_lime_poland_2020.jpeg", plot = p_lime)

explainer_dx <- DALEX::explain(cpi_model,
                              data = as.data.frame(X),
                              y = y,
                              label = "CPI SHAP Pipeline",
                              predict_function = predict_
                                function)
shap_poland <- predict_parts(explainer_dx,
                            new_observation = as.data.
frame(X_test_scaled)[poland_2020_index,, drop = FALSE],
                            type = "shap")
print(shap_poland$result)
plot(shap_poland)
ggsave("cpi_shap_poland_2020.jpeg")

```

```

for(i in 0:2) {
  feats <- colnames(X)[(5*i + 1):min(5*(i + 1), ncol(X))]
  pd <- partial(cpi_model, pred.var = feats,
               pred.fun = predict_function,
               train = as.data.frame(X),
               grid.resolution = 20)
  p_pd <- autoplot(pd) + ggtitle(paste("Partial Dependence
Plot (features", paste(feats, collapse = ", "),")"))
  ggsave(paste0("cpi_pdp_results_", i, ".jpg"), plot = p_pd)
}

```

Index

Note: – Italicized page references refer to figures, bold references refer to tables, and page references with “n” refer to endnotes.

- 31 Days of Praise* (Myers) 319
- activation function 26–32; Gaussian Error Linear Unit (GELU) activation function 31, 31; hyperbolic tangent (tanh) activation function 28, 29; leaky Rectified Linear Unit (leaky ReLU) activation function 29, 30; linear activation function 26, 27; ReLU activation function 29, 30; sigmoid activation function 28, 28; softmax activation function 31–32; step activation function 26, 27
- activation maximization/feature inversion 196
- AdaGrad 44–45, 46
- adaline (adaptive linear neuron) 23
- ADAM algorithm 46
- Adjusted Rand Index (ARI) 286–287
- adversarial training 103, 327, 330
- Age 137
- AgeOfPolicyHolder 137
- aggregation functions 413–415, 414; *see also* functions
- Agrawal, A. 11
- AlexNet 103
- algorithms: AdaGrad 44–45, 46; ADAM 46; backpropagation 54; Boruta 125; gradient descent method 42; momentum method 45–46; optimization 36–46; Q-learning 84; Recurrent Reinforcement Learning 93; RMSProp 44, 45; stochastic 42; stochastic gradient descent (SGD) 42–44
- Allaire, J. 7, 32
- Altman, E. I. 131
- analysis: business cycle 224; cluster 277–307; cryptocurrencies 297, 300, 301–307; discriminant 34–35, 117; exchange rate and monetary policy 224; financial market 224; multivariate statistical 8, 120–121, 180, 382–383, 394, 411; principal component 76, 126, 278; regression 116; replication 281; *see also* explanatory model analysis (EMA); regression and discriminant analysis; sentiment analysis; time series analysis
- analytical filing system 5
- AND gate 48, 50, 52
- AND operator 48
- anomaly detection 25
- application programming interfaces (APIs) 1, 122, 161, 331–333, 376
- applications 3–18, 277–336; areas of 12–17; areas of application of machine learning 6–9; black-box problem 17–18; cluster analysis 277–307; machine learning in economics 6–12; non-economical 320–333; quantitative vs qualitative research 3–6; recommender systems 308–319
- area under the receiver operating characteristic curve (AUC-ROC) 24
- arithmetic operators 346, 404, 404–405, 410
- array 397–399; elements of 369–370, 416–419; multidimensional 361, 368–370, 389, 415–419; N-dimensional array (ndarray) 105; selection 368–369, 415–416

- artificial intelligence (AI) 7, 12, 17, 329, 330, 343; in economic theory 1; timeline of 13–14
- Athanasopoulos, G. 218
- attention mechanism 16, 18, 78, 78–80, 223, 225, 262, 263, 330
- audio style transfer 327–329
- Augmented Dickey-Fuller (ADF) 217
- autoencoders 17, 76
- AutoRec 310
- autoregressive integrated moving average (ARIMA) 7, 213–214
- Average precision (AP) 310–311
- Ba, J. 46
- backpropagation algorithm 54
- backpropagation mechanism 23
- bag-of-word (BOW) methods 250–251, 267–270
- Bahdanau, D. 79
- Barry, R. 161
- Barto, A. G. 82, 84
- batch normalization 101–103, 128
- Bellman equation 83–84, 92
- Belongie, S. 321
- Bengio, Yoshua 57
- Benoit, D. F. 157
- bidirectional encoder representations from transformers (BERT) 23, 105
- Biecek, P. 170
- big data 9, 11, 347, 401
- BigLink dataset 288, 289, 292
- binary classification type tasks 32–34
- binary cross-entropy (log loss) 33
- Bishop, C. M. 116
- Bitcoin 293, 300, 307, 308
- black-box model 12, 17–18, 180, 182–183
- Black & White 12
- blank lines 346
- Boltzmann Machine 94
- Book- Crossing dataset 313
- book-ratings 313, 318
- bookstore 313–319
- Boolean algebra 48
- Boolean logic operations 23
- Bootstrapped Language Image Pretraining pipeline 330
- Boruta algorithm 125
- Brownian motion methods 214, 225–239
- Bunge, Mario 6
- business cycle analysis 224
- California Housing Dataset 161; dependent variable map visualization 163; mlp_regressor 179; partial dependency plot for 181; variables in 162
- Cardie, C. 252
- Carroll, J. 251
- categorical cross-entropy 35
- Cauchy, Augustin 41
- central processing units (CPUs) 171
- ceteris paribus* 177–179
- Chicken Soup for the Woman's Soul* (Canfield) 319
- Chollet, F. 7, 32, 321
- CIFAR dataset 108
- class activation mapping (CAM) 196
- classical machine learning methods 149
- classification 25; binary classification type tasks 32–34; cluster analysis 279–280; maximum-margin classification type task 36; with missing inputs 25; multinomial classification (discriminant analysis) type tasks 34–35; spectral 282–283; text 263; token 263; unsupervised 25
- classification and regression tree (CART) method 155
- cluster analysis 277–307; classification 279–280; Optimal Variable Weighting (OVW) in 125; procedure of 277–281, 278–279; quality of 76; SpectralNet 292–307; *see also* analysis
- Clustering Objects on Subsets of Attributes (COSA) 125, 278
- cluster quality indices 280, 280–281
- Colin McRae Rally 2.0 12
- collaborative filtering 308–311, 313
- collateralized debt obligations (CDOs) 11–12
- collateralized debt spreads (CDS) 11–12
- column names 373–374, 388
- comment 344, 386–387
- complex numbers 357–358
- comprehension 344, 352–356
- Comprehensive R Archive Network (CRAN) 382–384
- computer vision 320–321
- Compute Unified Device Architecture (CUDA) 107–108
- concept detection 196
- conditional statement 423–425
- confusion matrix 146, 146, 149
- Consumer Price Index (CPI) 200, 200–201, 207, 208, 209, 210
- content-based filtering 308, 309
- continuous bag-of-words (CBOW) model 252
- continuous encoding 124

- Contrastive Language – Image Pretraining (CLIP) 329
- conversion 327, 402, 403
- convolutional networks 22
- convolutional neural networks (CNNs) 16, 23, 46, 57–63, 193, 225, 320, 321, 328; application for stock price charts 239–247; architecture 57, 59; convolutional layer 59; MNIST library 60; pooling layer 59–63, 60; PyTorch library 60
- Cristianini, N. 250
- cryptocurrencies analysis 292; characteristics 301–307; eigenvalues gap for 297; scatterplots with cluster structure 300; *see also* analysis
- custom representation 124
- cutout and mixup regularization 103
- cycle-consistent adversarial network (Cycle-GAN) 328
- data: aggregation 426; augmentation 97, 102; availability 340–341; merging 375–376; science 343; types 344, 349, 364
- DataFrame 370, 373
- datasets 313; BigLink 288, 289, 292; Book- Crossing 313; data availability for 340–341; mlbench_spirals 282, 286, 286, 287
- Days_Policy_Accident 137
- Days_Policy_Claim 137
- DBnomics repository 198–211; overview of 199–211
- Debreu, Gérard 5
- Deductable 137
- deep belief networks 15, 16, 23, 95
- Deepfellow 97
- deep learning 7; areas of application of 15; batch normalization in 101–103; characteristics of application of 16; dropout regularization in 99–101; learning task 25–26; models 1; timeline of 13–14; as unsupervised learning 25–26
- deep learning computer ecosystem 105–109; CUDA (Compute Unified Device Architecture) 107–108; Iris dataset 108–109; Kaggle 108; Keras 106; Matplotlib 107; MNIST dataset 108; NumPy 105; OpenCV/ OpenCV2 (Open Source Computer Vision Library) 107; pandas 106; Pillow 107; pipeline repositories and pre-trained models 109; PyTorch 106–107; scikit-learn 105–106; seaborn 107; TensorFlow 106
- deep neural networks 1, 7
- Deep Q-Learning 17, 92–93
- Deep Q-Network 92
- deep reinforcement learning 17, 93, 94
- denoising 26
- DenseNet (Densely Connected Convolutional Network) 104
- dense retrieval 255, 260, 265
- density estimation 26
- determinant gradient descent (DSD) 42
- dialogue systems 264
- dictionaries 348–350, 351, 370
- Ding, X. 251
- direct prediction vs stepwise recurrent prediction 222, 222
- direct subjective expressions (DSEs) 252
- discounted cumulative gain (DCG) 310, 311
- discriminant analysis 34–35, 117
- distance measures 277, 278–279
- divisions, types of 357
- DIVISive ANALYSIS (DIANA) 279
- Don't Sweat the Small Stuff* (Carlson) 319
- dplyr package 415, 419–423
- dropout regularization 99–101
- dropout regularization in deep learning 99–101
- dynamic models (state-space models) 214
- early stopping 102
- economic impact of pandemics and shocks 224
- economic theory 1, 5
- EfficientNet 104
- eigenvalues gap method 296, 297
- Einstein, A. 214
- Elman, J. L. 23
- encapsulation 343, 361
- energy and commodity price forecasting 224
- energy consumption forecasting 224
- Engle, Robert 5
- Essays in Positive Economics* (Friedman) 5
- Eurostat 199
- exchange rate and monetary policy analysis 224
- explainable artificial intelligence (XAI) 154
- explanatory model analysis (EMA) 154–211; black boxes 154–196; importance of inflation factors 198–211; LIME 180–196; models internally interpretable 155–156; origins of 155; PDP 177–180; permutation methods 157–166; SHAP 166–177; techniques of 157, 158; time limitations of 196–198

- expressions: lambda 343, 344, 415; Python 356; variable 387–388
- expressive subjective expressions (ESEs) 252
- face detection 320, 325–327, 328
- Federal Reserve Economic Data (FRED) 199
- feedforward neural networks (FNNs) 47–57; multi-layer perceptron 53–57, 54, 58; single-layer perceptron 47, 47–53, 51
- fill-mask 264
- financial market analysis 224
- floating-point architecture 1
- Flood: Mississippi 1927* (Duey) 319
- focal categorical cross-entropy 35
- focal cross-entropy loss 34
- forecasting 213; energy and commodity price 224; energy consumption 224; macroeconomic 224; public finance and tax revenue 224
- foundations 386–411; attributes 388–389, 389; comment 386–387; conversion 402, 403; expression for variable 387–388; literals 346, 347, 386, 386; object 387, 389–402; operators 404–411; *see also* R language
- fractional numbers 358
- Friedman, Milton 3, 5
- functional programming 342, 344
- functions: aggregation 413–415, 414; mathematical 26, 361, 412, 413; packages 411–412; rep 391; sample 391–392
- Fundamental Clustering and Projection Suite (FCPS) 288
- gated recurrent units (GRU) 68, 223; network architecture 68, 69
- Gatnar, E. 116
- Gatys, L. A. 320
- Gauss, C. F. 35
- Gaussian mixture model-based clustering 286, 286
- general equilibrium models 7
- generalization problem equilibrium 97
- generalized autoregressive conditional heteroskedasticity (GARCH) 7
- generative adversarial networks (GANs) 22, 25, 74–76, 77; architecture of 75; discriminator 75–76; generator 75–76; types of 77
- generative pre-trained transformer (GPT) 23
- Gentleman, Robert 382
- Gnu's Not Unix (GNU) 382
- Goodfellow, I. 25, 74–75
- GoogLeNet/Inception 104
- Gosiewska, A. 170
- Gradient Boosting model 146, 149
- gradient descent method 42
- Gram matrices 320
- Granger, Clive 5
- graphical user interface (GUI) framework 85
- graphics processing units (GPUs) 23, 191, 313, 331
- gross domestic product (GDP) 199, 224
- Growing Neural Gas 95
- Haffner, Patrick 57
- Hair, J. F. 8, 116
- Hancock, J. T. 125
- Hastie, T. 116, 128
- Hayek, Friedrich 3–4
- He, K. 104
- He, X. 310, 311
- Heilbroner, Robert L. 5
- Heuristic Identification of Noisy Variables (HINoV) 125, 278
- hierarchical clustering 279, 287
- high level of missing values 125
- hinge loss 36
- Hinton, Geoffrey 12, 23, 45
- Hochreiter, S. 23
- Holt, C. C. 214
- Hopfield, John 12
- housing and real estate markets 224
- Hovy, E. 251
- Howard, A. G. 104
- Hu, M. 251
- Huang, G. 104
- Huang, X. 321
- Hugging Face repository ecosystem 253–254
- hybrid fusion methods 330
- hybrid recommender systems 310
- Hyndman, R. J. 218
- Ihaka, Ross 382
- ImageNet 103, 128
- image recognition 106, 108, 196, 331
- IMDb reviews dataset 266–274
- imputation of missing values 26
- indentation 345–346

- inheritance mechanism 360–361
- integer encoding 124
- International Monetary Fund (IMF) 199
- Ioffe, S. 101, 128
- Iris dataset 108–109
- Irsay, O. 252
- item-based collaborative filtering 309
- Jacobian regularization 103
- Joachims, T. 250
- Johnson, J. 321
- Jordan, M. I. 23
- Kaggle 108, 132, 135
- Kaggle API key 376–377
- Kamps, J. 251
- Keras 106, 149
- Keynes, John Maynard 3
- Khoshgoftaar, T. M. 125
- Kim, S.-M. 251
- Kingma, D. P. 46
- Kleinberg, J. 11
- k-means* method 282, 286
- k-medoids method 280
- Kohonen's map 95
- Kononenko, I. 172
- Kozyra, Katarzyna 184
- Krizhevsky, A. 103
- Kullback-Leibler (KL) divergence 34
- Kuznets, Simon 5
- L1 regularization (LASSO) 99–100, 128
- L2 regularization (weight decay) 98, 128
- label encoding 124
- labor market and employment studies 224
- lack of adequacy 125
- lambda expressions 343, 344, 415;
 - see also* expressions
- Laplacian matrix 283, 287–288
- large language models (LLMs) 1, 254–266, 331–333
- learned feature visualization 192
- learned representation 124
- learning the spectral map 287–292
- Least Absolute Shrinkage and Selection Operator (LASSO) 8
- LeCun, Yann 23, 57
- Lee, S.-I. 172
- Leontief, Wassily 5
- Li, C. 321
- Li, David X. 11–12
- library directory 289–290
- Life's Little Instruction Book* 318
- liquid state machine 95
- lists 348–349, 349, 394–396
- literals 346, 347, 386, 386
- Liu, B. 251
- Local Interpretable Model-agnostic Explanations (LIME) 18, 154, 180–196
- Logistic Regression 146–147, 149
- long short-term memory (LSTM) 16, 23, 68, 74, 223, 225; network architecture 68, 69; training and function 74
- long-term capital management (LTCM) 11
- loops 352–353, 424–427; with data aggregation 426; search for data 425
- loss functions 32–36; binary classification type tasks 32–34; maximum-margin classification type task 36; multinomial classification (discriminant analysis) type tasks 34–35; regression type tasks 35–36, 37–39
- Luan, F. 321
- Lundberg, S. 172
- Luong, M. T. 79
- Maas, A. L. 266
- machine learning: application of 10; areas of application of 15; characteristics of application of 16; critique of 9–12; domains of 9; in economics 6–12; quantitative methods 6–7
- machine translation 25
- MacKay, D. J. C. 116
- macroeconomic forecasting 224
- Markowitz, Harry 5
- Marshall, Alfred 5
- mathematical functions 26, 361, 412, 413;
 - see also* functions
- Matplotlib 107
- matrix factorization 308–310, 312–313
- maximum-margin classification type task 36
- McCulloch, W. S. 23, 47
- McFadden, Daniel 5
- McQuitty L.L. 279
- mean absolute error (MAE) 35
- mean absolute percentage error (MAPE) 36
- mean average precision (MAP) 310
- mean squared error (MSE) 35
- Memoirs of a Geisha* (Golden) 319
- Menger, Carl 3
- merging data 375–376; *see also* data
- Meriwether, John W. 11
- Merton, Robert C. 11
- method chaining 374–375

- “Methodenstreit” 3
- Mikolov, T. 251–252
- Minsky, M. L. 53, 82
- mirror box 180
- mlbench_spirals dataset 282, 286, 286, 287
- Mnih, V. 92
- MNIST (Modified National Institute of Standards and Technology) dataset 108
- MobileNet 104
- model building phases 23–26; decision trees 24; test set 23; training (learning) set 23–24; validation set 23–24
- model design 120–126; object selection 126; variable aggregation 126; variable normalization 126; variable scales 120–121; variables encoding 122–125; variables selection 125
- Mohammad, S. 251
- momentum method 45–46
- Monty Python’s Flying Circus* 342
- Moody, J. 93
- Mullainathan, S. 9
- multidimensional array 361, 368–370, 389, 415–419
- multi-head self-attention 80
- multimodal deep learning 329–330
- multinomial classification (discriminant analysis) type tasks 34–35
- Multivariate Imputation by Chained Equations (MICE) 125
- multivariate statistical analysis 8, 120–121, 180, 382–383, 394, 411
- Mystic River* (Lehane) 319
- Naïve Bayes 146, 147, 149, 250, 268, 269, 274
- Nash, John 5
- natural language processing (NLP) 23, 249, 254–266
- N-dimensional array (ndarray) 105
- neoclassical economics 3, 5
- neural collaborative filtering (NCF) 310–319, 318
- neural matrix factorization (NeuMF) 312, 313
- neural networks 22, 22–105, 308–319, 328; activation function 26–32; AlexNet 103; BERT (Bidirectional Encoder Representations from Transformers) 105; components of 26–46; DenseNet (Densely Connected Convolutional Network) 104; EfficientNet 104; foundations of 22–23; GoogLeNet/Inception 104; high bias (underfitting) 95; high variance (overfitting) 95; ImageNet 103; loss functions 32–36; MobileNet 104; optimization algorithms 36–46; paradigms (architectures) of 46–95; performance metrics 36; ResNet (Residual Network) 104; selected architectures of 103–105; under/overfitting avoiding techniques for 95–103; U-Net 105; VGGNet (Visual Geometry Group Network) 103; Vision Transformer (ViT) 104; YOLO (You Only Look Once) 104–105
- Neural style transfer (NST) 320–325, 324, 325, 378–381
- “New Deal” (Roosevelt) 3
- New Vegetarian: Bold and Beautiful Recipes* 319
- Ng, A. 283
- Ng, A. Y. 128
- noise injection (gaussian/label smoothing) 102
- nominal variable 393–394, 402
- non-economical applications 320–333; API 331–333; audio style transfer 327–329; computer vision 320–321; face detection 325–327; LLMs 331–333; multimodal deep learning 329–330; NST 320–325
- normalization: batch 101–103, 128; formulae for metric data 127; spectral 102; variable 126
- normalized discounted cumulative gain (NDCG) 310, 311
- NumberOfCars 137
- NumPy 105, 361–367, 362–363, 370, 372
- object: attributes 388–389, 389; comparisons 358; name 387; -oriented programming 359–360; selection 126; types 389–402
- OpenCV/ OpenCV2 (Open Source Computer Vision Library) 107
- operations on matrices 367–368
- operators: arithmetic 404–405; substitution of vector 405–411
- Optimal Variable Weighting (OVW) 125
- optimization algorithms 36–46
- Organisation for Economic Co-operation and Development (OECD) 199
- overfitting 95–103, 126–128
- Pace, R. K. 161
- packages 383–386; dplyr 415, 419–423; functions in 411–412; installation of

- 347, 384, 385; loaded into memory 384–386
- pandas 106, 370–373, 371, 376
- Pang, B. 250
- Papert S. 53
- paradigms (architectures) of neural networks 46–95; autoencoders 76; Boltzmann Machine 94; convolutional neural networks (CNNs) 57–63; deep belief networks 95; feedforward neural networks (FNNs) 47–57; generative adversarial networks (GANs) 74–76, 77; Growing Neural Gas 95; liquid state machine 95; recurrent neural networks (RNNs) 64–74; reinforcement learning 82–94; Restricted Boltzmann Machines 95; self-organizing maps/Kohonen’s map 95; spiking neural networks 95; transformers 78–82
- partial dependence plots (PDP) 177–180
- part of speech (POS) 251
- performance metrics 36
- performance-monitoring tools 24
- permutation methods 157–166
- physical econometrics 9
- physical economics 9
- physical fixation 9
- Pillow 107
- pipeline repositories and pre-trained models 109
- Pitts, W. 23, 47
- PolicyNumber 137
- Polyak, B. T. 45
- Popper, Karl 4
- position-wise FNN 80
- Power of Positive Thinking* (Peale) 319
- prediction 213
- pre-trained language models 1
- The Prince* 318
- principal component analysis 76, 126, 278
- Principles of Economics* (Marshall) 5
- program execution 333, 334, 335
- proper model evaluation 129–132
- public finance and tax revenue forecasting 224
- “The pyramid of animals” (*Piramida Zwierząt*) (Kozyra) 184, 184, 186, 187, 188, 189
- Python 1, 48, 54, 64, 342–381; array selection 368–369; basics 347–348; blank lines 346; characteristics 342–343; column names 373–374; command 374–375; comments 344; comparisons 357; control condition 369; DataFrames 375; data science 343; defined 343; dictionaries 350, 351; divisions 357; encapsulation 361; expressions 356; fractional numbers 358; functional programming 344; imaginary/complex numbers 357–358; indentation 345–346; inheritance 360–361; Kaggle API key 376–377; Lambda 344; lists 348–349, 349; merging data 375–376; multidimensional arrays 368–370; NST with TPU 378–381; NumPy 361–367, 362–363; object comparisons 358; object-oriented programming 359–360; operations on matrices 367–368; packages 347; Pandas 370–373, 371; PyTorch installation 377, 378; slices 348, 358–359, 370; statements and comprehension 352–356; strings 346, 347; syntax 345; tuples 351–352, 352; types in 343, 344
- Python Enhancement Proposal (PEP) 342
- Python Software Foundation 342, 343
- PyTorch 106–107, 184
- PyTorch installation 377, 378
- Q-learning algorithm 84
- Q-network 92–93
- Q-table 84–85, 91, 92, 92
- quantitative methods 6–7, 8; areas of application of 15; characteristics of application of 16
- quantitative vs qualitative research 3–6
- question answering models 258
- Q-values 92–93
- Radford, A. 329
- Random Forest model 146, 149
- random walk theory 214
- R code 1, 383
- Real Time Economics* 9
- Receiver Operating Characteristic Area Under the Curve (ROC AUC) 129, 132, 147, 149, 246
- recommender systems: history of 308–313; NCF algorithm 313–319; on neural networks 308–319; types of 309–313
- recurrent networks 250
- recurrent neural networks (RNNs) 16, 64–74, 223, 225; architecture 64; GRU network architecture 68
- Recurrent Reinforcement Learning algorithm 93

- Redmon, J. 104
- regression 25
- regression analysis, defined 116
- regression and discriminant analysis
 - 116–149; data preparation 120–126;
 - ensemble techniques of 119–120;
 - feedforward networks as 116–120;
 - methods of 118–119; model design 120–126;
 - multivariate data analysis taxonomy 117; proper model evaluation 129–132;
 - properties of measurement scales 122; regularization and overfitting 126–128;
 - research application 132–149; vehicle claim fraud detection 132–149
- regression type tasks 35–36
- regularization 126–128; cutout and mixup 103; defined 97; dropout 99–101; Jacobian 103; L1 regularization (LASSO) 99–100; L2 regularization (weight decay) 98; sparse 102
- reinforcement learning 82–94, 94, 106–107, 254; cycle 85; iterative loop 84; Q-table 91, 92
- reinforcement learning cycle 85
- rep function 391
- replication analysis 281; *see also* analysis
- reranking 265–266
- ResNet (Residual Network) 104
- Restricted Boltzmann Machine-based recommender system 310
- Restricted Boltzmann Machines 15–16, 25, 95
- retrieval-augmented generation (RAG) 266
- Ribeiro, M. T. 180
- Riloff, E. 252
- Ripley, B. D. 116
- R language 336n1, 382–430; aggregation functions 413–415, 414; conditional statement 423–424; dplyr package 419–423; foundations of 386–411; function 427–429, 429–430; loops 424–427; mathematical functions 412, 413; multidimensional arrays 415–419; packages 383–386, 411–412
- Roosevelt, Franklin D. 3
- root mean squared error (MSE) 35
- Root Mean Square propagation (RMSProp) 44, 45
- Rosenblatt, F. 12, 23, 26, 47, 53
- Rosenblatt’s perceptron 12, 53
- Rousseeuw, P. J. 281
- R scripts 432–454; Consumer Price Index (CPI) factors analysis 448–454; multi-layer perceptron 434–438; simple recurrent neural network 438–444; single-layer perceptron 432–434; vehicle claim fraud detection 445–448
- rule of thumb 128
- Rumelhart, D. E. 53
- Saffell, M. 93
- Salakhutdinov, R. 310
- saliency maps 196
- sample functions 391–392; *see also* functions
- Samuel’s Checkers Program 12
- Samuelson, Paul Anthony 5
- Sanda, O. 16
- Schmidhuber, J. 23
- Scholes, Myron S. 11
- scikit-learn 105–106
- Seabiscuit* 318
- seaborn 107
- Sedhain, S. 310
- self-attention 79–81, 104–105, 223, 225, 263, 330
- self-organizing maps 95; *see also* Kohonen’s map
- semantic search 264–265
- sentiment analysis 249–274; bag-of-words methods 250–251; dictionaries of positive and negative words 251; fundamentals 249–250; Hugging Face repository ecosystem 253–254; of IMDb reviews for BoW/GRU/BERT 266–274; large language models (LLM) 254–266; natural language processing (NLP) 254–266; recurrent neural networks-based 251–252; timeline 250; transformer-based models 252–253; unigrams and dictionaries 250
- sequence-to-sequence approach 79
- Shaham, U. 287
- shallow copy 367
- Shapley, L. S. 172
- SHapley Additive exPlanations (SHAP) 18, 154, 166–177, 171
- Shawe-Taylor, J. 250
- Shipping News* (Proulx) 319
- silhouette index 281
- Simonyan, K. 103
- slices 348, 358–359, 370
- smoothing methods (exponential smoothing) 214
- social networks 383
- Spall, J. C. 42

- sparse regularization 102
- specialized language 5
- spectral classification 282–283
- spectral clustering vs. spirals 283
- SpectralNet: advantage of 281–292; of
 - crypto assets 292–307; learning the spectral map 287–292
- spectral normalization 102
- spectrograms 327–329
- Spiess, J. 9
- spiking neural networks 95
- S-PLUS software 382
- Srivastava, N. 99, 127
- statements 344, 348, 352–356, 369, 386, 425, 427
- stationarity 214–215, 216
- statistical databases 383, 402
- stochastic algorithms 42
- stochastic depth 102
- stochastic descent gradient
 - method 42
- stochastic gradient descent (SGD) 42–44
- stochastic processes (Brownian motion) 214; *see also* Brownian motion methods
- The Street Lawyer* (Grisham) 319
- strings (str type) 346, 347
- structural equation models 7
- structured output 25
- Štrumbelj, E. 172
- summarization task 258–262
- superpixels* 183–185, 186, 192, 193
- support vector machines (SVMs) 7, 116, 250
- Sutton, R. S. 82, 84
- Switching Goals* 319
- syntax 344, 345, 364, 373, 377, 378, 411, 423, 426
- synthesis and sampling 25
- Szegedy, C. 101, 104, 128

- Taboada, M. 251
- Tacotron 2 328, 329
- Tang, D. 252
- The Tao of Pooh* 318
- target encoding 124
- techniques 22–109; attention mechanisms 330; deep learning computer ecosystem 105–109; matrix factorization 308–310, 312–313; model building phases 23–26; neural networks 22–105; NST 320–325
- Tell Me This Isn't Happening* 319
- TensorFlow 106
- tensor processing units (TPU) 287, 331, 378–381

- testing with concept activation vectors (TCAV) 196
- text classification 263
- text generation 255–258
- text input 329
- text-to-speech model 329
- tibble 401–402
- time series analysis 213–247; application of CNN for stock price charts 239–247; Brownian motion methods 225–239; data transformation step 223–224; direct prediction vs stepwise recurrent prediction 222, 222; in economics 224; GARCH family methods 225–239; impact on model architecture 213–225; linearity 218–222; methods for deep learning-based 225; methods of 215; non-stationarity with unit roots 217–218; RNNs in prediction of volatility of stock prices 225–239; seasonality 223; stationarity 214–215, 216; stationary vs. non-stationary 216
- time series decomposition/seasonal decomposition of time series by loess (STL) 214
- time series plot 307, 308, 309
- token classification 263
- too high variability 125
- too low variability 125
- trainable parameters 128, 128, 154
- transcription 25
- transformed images 324, 325
- transformer-based models 252–253
- transformer-based networks 17, 23, 94
- transformers 78–82, 250; approximate solution methods 82; attention-based 80; decoder stack 81; embeddings and positional encoding 82; encoder stack 80–81; multi-head attention 81; scaled dot-product attention 81; tabular solutions methods 82
- translation task 262–263
- Tryon, R. C. 277
- Tsang, W. K. 157
- tuples 351–352, 352
- Turing Machines 64
- Turing's Imitation Game 12
- Turney, P. D. 251

- Ulyanov, D. 321
- U-Net 105
- University of Auckland 382
- unsupervised classification 25

- unsupervised/supervised learning 25–26
- user-based collaborative filtering 309
- user IDs 313, 318, 319

- Valitutti, A. 251
- van Rossum, Guido 342
- variables 277–278; aggregation 126;
 - in California Housing Dataset 162;
 - categories of 122–123, 393; encoding 122–125; expression 387–388;
 - importance measures 161, 161, 162, 165; normalization 126; scales 120–121; selection 125
- Varian, H. R. 11
- variational autoencoders (VAEs) 76
- vector 389–391; length 392; vs. simple types 392–402; substitution of 405–411
- vector auto regressive (VAR) 7
- vehicle claim fraud detection 132–149, 133–134, 135–137, 139–141, 142, 143, 144–146, 148
- VGGNet (Visual Geometry Group Network) 103
- Vision Transformer (ViT) 104
- Visual Geometry Group (VGG) 46, 320
- voice synthesis model 329
- volatility models (ARCH/GARCH) 214
- Volcker, Paul 9

- Wand, M. 321
- Ward agglomerative clustering 286, 286–287
- WaveNet 328, 329
- Webservices 122
- weight pruning 102
- Wiebe, J. 252
- Wiener, N. 214
- Wild Animus* 319
- Wilmott, Peter 11
- Wilmott.com 11
- Winters, P. R. 214
- Word2Vec 252–253
- WordNet 251
- World Bank 199
- The Worldly Philosophers: The Lives, Times, and Ideas of the Great Economic Thinkers* (Heilbroner) 5
- Wu, C. 310

- YOLOv8 face model 325
- You Only Look Once (YOLO) 46, 104–105

- Zagibalov, T. 251
- Zhang, C. 127
- Zheng, Y. 15
- Ziegler, C. N. 313
- Zisserman, A. 103